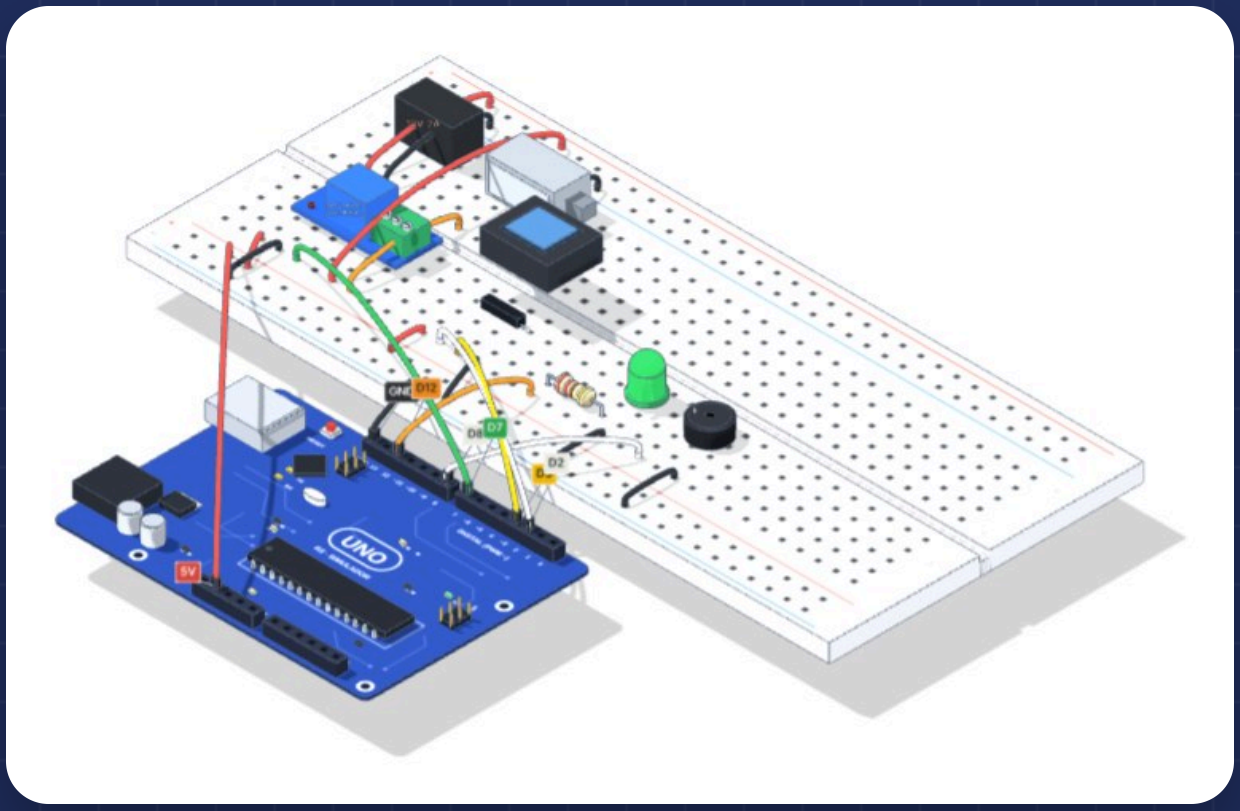
LIBRO 2026 – 2027

Banco Arduino

205 proyectos de electrónica y programación explicados paso a paso, con ilustraciones, conexiones y código



Lector de huellas · Chapas eléctricas · RFID · Sensores · Robótica · Domótica

ING. JAIR MURILLO

DOCENTE · AUTOR

LIBRO 2026 – 2027

Banco Arduino

205 proyectos de electrónica y programación explicados paso a paso para estudiantes

ING. JAIR MURILLO

Banco Arduino · Libro 2026 – 2027

Autor: Ing. Jair Murillo

Ilustraciones de circuitos generadas con el simulador Banco Arduino.

Simulador: https://andresyairrenteriamurillo-maker.github.io/banco-arduino/

Material educativo de uso en el aula. Se permite imprimirlo y fotocopiarlo con fines educativos citando al autor.

Arduino® es una marca registrada de Arduino SA. Este libro no está afiliado a Arduino SA.

PRESENTACIÓN

Bienvenidos al mundo Arduino

Este libro nace en el aula, para el aula.

Hoy la tecnología está en todas partes: en la puerta que se abre con una huella, en el semáforo de la esquina, en la alarma de la casa y en el celular que llevamos en el bolsillo. Detrás de todos esos aparatos hay una misma idea: **sensores que miden, un pequeño computador que decide y actuadores que hacen algo**. Con una placa Arduino, unos cuantos componentes y curiosidad, cualquier estudiante puede construir esas mismas soluciones.

Aquí encontrarás **205 proyectos** sencillos y prácticos, inspirados en problemas reales de la casa, el colegio y la comunidad: desde encender el primer LED hasta construir una chapa eléctrica con lector de huellas, un control de asistencia con tarjetas RFID, un sistema de riego, una alarma contra gas o un robot que esquiva obstáculos.

Cada proyecto está explicado **paso a paso**: qué piezas necesitas, cómo se conecta cada cable, el programa completo con comentarios y una explicación sencilla de cómo funciona. Todos se pueden practicar primero en el simulador **Banco Arduino**, para equivocarse sin miedo, y luego armarse con piezas reales.

La invitación es a no quedarse en copiar: modifica, prueba, rompe (en el simulador) y vuelve a armar. Los **retos** al final de cada proyecto están para eso. Equivocarse es parte de aprender, y cada error resuelto es un paso más hacia convertirte en creador de tecnología.

Ing. Jair Murillo
Docente · Libro 2026 – 2027

Este libro es de:

_____

Grado: _____ Año: _____

ÍNDICE

Contenido

Primera parte · Fundamentos

Cómo usar este libro 7
Seguridad en el laboratorio 8
Electricidad básica 9
La placa Arduino UNO y la protoboard 12
Programar el Arduino 14
Del computador al Arduino 16
El simulador Banco Arduino 18
Catálogo de componentes 19

Segunda parte · Proyectos

1. Luces y LED 23

1. Mi primer LED parpadeante 24
2. Señal de auxilio SOS 26
3. Faro de barco 28
4. Luz de vela 30
5. LED que respira 32
6. Luces de policía 34
7. Semáforo básico 37
8. Semáforo de dos vías 40
9. Auto fantástico 43
10. Contador binario de 4 bits 46
11. Luces de Navidad 49
12. Luz de emergencia estroboscópica 52
13. Latido del corazón 54
14. Ruleta de luces 56
15. Metrónomo de luz 59
16. Escalera de luces 62
17. Arcoíris con LED RGB 65
18. Semáforo con cuenta en el monitor 67
19. Dos LED alternados con velocidad creciente 70
20. Vela de cumpleaños RGB 72

2. Botones e interruptores 74

21. LED con botón 75
22. Interruptor de luz con un toque 77
23. Contador de pulsaciones 79
24. Timbre sencillo 81
25. Dimmer con dos botones 83
26. ¿Quién presiona primero? 86
27. Dado electrónico de LED 89
28. Botón de pánico 92
29. Selector de modos de luz 95
30. Juego de reflejos con dos LED 98
31. Cerradura de secuencia de botones 101
32. Lámpara con temporizador 104
33. Piano de cuatro teclas 107
34. Cuenta regresiva con botón 110
35. Pulsación larga y corta 113
36. Alarma de puerta con interruptor 116
37. Marcador de dos equipos 119
38. Semáforo peatonal con interruptor nocturno 122

3. Sonido y música 125

39. Sirena de bomberos 126
40. Escala musical 128
41. Feliz cumpleaños 130
42. Alarma de despertador 132
43. Tono que sube con la perilla 134
44. Theremin de luz 136
45. Código Morse sonoro 138
46. Metrónomo musical ajustable 141
47. Caja musical con botón 144
48. Detector de silencio 147
49. Aplauso que enciende la luz 150
50. Alarma de golpe con sirena 152
51. Piano táctil de 3 notas 155

4. Perillas y señales analógicas 157

52. Brillo con perilla 158
53. Barra de nivel con LED 160
54. Velocidad de parpadeo con perilla 163
55. Voltímetro en el monitor serie 165
56. Mezclador de colores RGB 167
57. Servo con perilla 170
58. Ventilador de velocidad variable 172
59. Adivina el número con perilla 175
60. Joystick que mueve una luz 178
61. Brazo de dos servos con joystick 181
62. Luxómetro casero 184
63. Termómetro de perilla para practicar 186

5. Luz y oscuridad 189

64. Interruptor crepuscular 190

65. Alarma de cajón abierto 192
66. Lámpara de lectura automática 194
67. Medidor de luz con barra de LED 196
68. Detector de día y noche en pantalla 199
69. Barrera láser cuenta-objetos 201
70. Girasol de un servo 204
71. Luz nocturna con colores 207

6. Temperatura y clima 209

72. Termómetro en el monitor serie 210
73. Termómetro con pantalla LCD 212
74. Alarma de calor 214
75. Estación del clima con DHT11 217
76. Alerta de humedad alta 219
77. Semáforo de confort 222
78. Termostato con perilla 224
79. Registro de temperatura máxima y mínima 227
80. Alarma de nevera abierta 229
81. Indicador de sensación de bochorno 232
82. Ventilador por temperatura con DHT11 235
83. Termómetro de barra con LED 238
84. Incubadora de huevos sencilla 241
85. Detector de fiebre para practicar 244
86. Monitor del clima con LCD y ventilador 247

7. Distancia y movimiento 250

87. Medidor de distancia en el monitor 251
88. Medidor de distancia con pantalla 253
89. Alarma de proximidad 255
90. Sensor de distancia con semáforo 258
91. Theremin ultrasónico 261
92. Contador de visitantes con ultrasonido 263
93. Detector de intrusos con PIR 266
94. Luz automática de pasillo 268
95. Saludo automático en la entrada 270
96. Radar con servo y ultrasonido 273
97. Barrera automática de parqueadero 276
98. Medidor de nivel con ultrasonido 279
99. Regla digital con botón de guardar 282
100. Detector de movimiento con sirena y luces 285
101. Detector de objetos con sensor IR 288

8. Servos y motores 290

102. Barrido de servo 291
103. Puerta automática con botón 293
104. Alimentador de mascotas con temporizador 296
105. Limpiaparabrisas con sensor de lluvia 299
106. Motor con botón de encendido 301
107. Bandera que saluda 304
108. Reloj de arena con servo 306
109. Ventilador oscilante 309
110. Compuerta de riego con servo 312
111. Seguidor de movimiento con servo 315
112. Mano robótica de un dedo 318
113. Semáforo con barrera de tren 320
114. Mano que saluda al acercarte 323

9. Pantallas 325

115. Hola mundo en la LCD 326
116. Cronómetro con LCD y botones 328
117. Mensaje que se desplaza 331
118. Contador de personas en el aula 333
119. Dado con pantalla 336
120. Nombre en la pantalla OLED 339
121. Reloj de juego de mesa 341
122. Termómetro e higrómetro en OLED 344
123. Votación electrónica 347
124. Calculadora de suma con teclado 350
125. Mi tarjeta de presentación en LCD 353

10. Seguridad y control de acceso 355

126. Lector de huellas con luz verde y roja 356
127. Cerradura con huella y servo 359
128. Chapa eléctrica con huella 362
129. Saludo personalizado con huella 365
130. Registro de asistencia con huella 368
131. Caja fuerte con huella y alarma 371
132. Cerradura con clave y teclado 374
133. Chapa eléctrica con clave 377
134. Cerradura con clave, LCD y servo 380
135. Control de acceso con tarjeta RFID 383
136. Puerta con tarjeta RFID y servo 386
137. Chapa eléctrica con tarjeta RFID 389
138. Bienvenida con tarjeta RFID y LCD 392
139. Registro de entradas con RFID 395
140. Alarma con PIR que se desactiva con clave 398
141. Alarma de ventana con imán 401
142. Timbre que solo suena con tarjeta 404
143. Doble seguridad: huella y botón de salida 407
144. Alarma de vibración para caja fuerte 410
145. Botón de emergencia con sirena y luces 413
146. Contador de intentos de acceso en LCD 416
147. Cofre del tesoro con clave 419
148. Lector de huellas con contador de usos 422
149. Doble factor: tarjeta y huella 425
150. Alarma antirrobo para bicicleta 429

11. Hogar y ambiente 431

151. Detector de fuga de gas en la cocina 432
152. Detector de incendio con llama 435
153. Bomba contra incendios automática 437
154. Luz de pasillo con sensor de movimiento 440
155. Luz nocturna inteligente 442
156. Aplausos que encienden la luz 444
157. Lámpara táctil 446
158. Alarma de inundación 448

159. Llenado automático de tanque con bomba 450
160. Tendedero que avisa la lluvia 453
161. Ventilador de dos velocidades 456
162. Calentador con relé y termostato 458
163. Monitor de humedad para el baño 461
164. Recordatorio para regar la planta 463
165. Cortina que abre con el sol 465
166. Timbre con luz para personas sordas 467
167. Pastillero con recordatorio 469
168. Medidor de calidad del aire con semáforo 472
169. Medidor de consumo de un aparato 474
170. Casa inteligente con menú en LCD 477
171. Indicador de gas en LCD 480
172. Nivel de tanque en LCD 482

12. Juegos 484

173. Simón dice de 3 colores 485
174. Adivina el número 488
175. Ruleta de la suerte 491
176. Piedra, papel o tijera 494
177. Contador de puntos para partido 497
178. Juego de puntería con luz 500
179. Carrera de botones para dos 503
180. Dado doble con LCD 506
181. Esquiva obstáculos en la LCD 509
182. Tirar la moneda 512
183. Laberinto con joystick 515
184. Juego de equilibrio con inclinación 518
185. Pregunta y respuesta con botones 520
186. Reloj de ajedrez 523

13. Medición y ciencia 526

187. Voltímetro de 0 a 5 V 527
188. Voltímetro con pantalla LCD 529
189. Amperímetro con ACS712 y LCD 531
190. Termómetro con máximo y mínimo 533
191. Cronómetro para experimentos 535
192. Medidor de velocidad con dos sensores 537
193. Medidor de estatura 540
194. Contador de vueltas con imán 543
195. Experimento: luz y distancia 546
196. Detector de terremotos (sismógrafo) 549
197. Contador de productos en banda 551
198. Alcancía que cuenta monedas 554

14. Robótica y vehículos 556

199. Carro que esquiva obstáculos 557
200. Robot seguidor de luz 560
201. Robot seguidor de línea 563
202. Grúa con joystick 566
203. Semáforo para pista de carritos 569
204. Radar de barrido con aviso de cercanía 572
205. Robot mascota que reacciona 575

Anexos

Solución de problemas 578
Glosario 580
Materiales para el kit 583
Mis notas 586

ANTES DE EMPEZAR

Cómo usar este libro

Este libro está pensado para que cualquier persona, aunque nunca haya conectado un cable, pueda armar sus propios proyectos con Arduino.

La **primera parte** explica lo básico: qué es la electricidad, cómo es la placa Arduino, cómo funciona la protoboard, cómo se escribe un programa y cómo se usa el simulador **Banco Arduino**. Léela con calma, en orden; solo toma una o dos clases.

La **segunda parte** trae **205 proyectos** organizados en 14 capítulos, de los más fáciles a los más completos. Todos los proyectos fueron armados y probados en el simulador. Cada uno trae:

🧰 **Materiales**
La lista exacta de piezas, con las resistencias y cables que necesitas.

🖼️ **Dos ilustraciones**
La vista ilustrada del circuito armado y el plano de conexiones en la protoboard.

🔌 **Conexiones paso a paso**
Cada cable explicado: de qué pata sale y a qué pin o riel llega.

💻 **El programa**
El código completo con comentarios, listo para copiar en el Arduino IDE.

💡 **¿Cómo funciona?**
La explicación sencilla de la idea del programa.

🏆 **Reto**
Una mejora para que pienses y lo hagas tuyo.

Niveles de dificultad

Símbolo	Nivel	Para quién
★☆☆	Básico	Primeros proyectos. Pocas piezas y programas cortos.
★★☆	Intermedio	Usa sensores, variables y decisiones con `if`.
★★★	Avanzado	Varias piezas, librerías, pantallas o cargas de 12 V.

Consejo para el estudiante

Primero arma el proyecto en el simulador. Cuando funcione ahí, ármalo con piezas reales: así sabrás que el circuito y el programa están bien y, si algo falla, será un cable suelto.

PRIMERO LO PRIMERO

Seguridad en el laboratorio

La electrónica con Arduino es segura cuando se respetan unas reglas sencillas.

1. **Desconecta el USB antes de mover cables.** Arma o cambia el circuito siempre sin energía.
2. **Revisa dos veces antes de conectar.** Un cable de 5V tocando GND es un corto circuito: el Arduino se calienta o se apaga.
3. **Los LED siempre llevan resistencia** (220 Ω). Sin ella se queman en segundos.
4. **Nunca conectes motores, bombas ni chapas directo a un pin.** Un pin da como máximo 20 mA; un motor pide cientos. Usa un transistor, un L298N o un relé.
5. **12 V solo por el relé.** La fuente de 12 V nunca se une con el 5V del Arduino. Lo único que comparten, si hace falta, es el GND.
6. **110 V no es para practicar.** Los proyectos de este libro usan máximo 12 V. Para controlar aparatos de la casa pide siempre la ayuda de un adulto o electricista.
7. **El lector RFID va a 3.3 V**, no a 5 V.
8. **Si algo huele a quemado o se calienta, desconecta el USB de inmediato** y avisa al docente.
9. Trabaja sobre una mesa seca, lejos de líquidos, y guarda las piezas en su caja al terminar.

> ⚠️ **Regla de oro**
> Si no estás seguro de una conexión, pruébala primero en el simulador Banco Arduino: él te avisa si hay un corto, una pieza al revés o un componente sin resistencia.

FUNDAMENTOS

Electricidad básica

Para entender un circuito basta con tres ideas: voltaje, corriente y resistencia.

Voltaje (V)

Es la «fuerza» que empuja la electricidad, como la presión del agua en una manguera. Se mide en **voltios**. El Arduino trabaja con **5 V**.

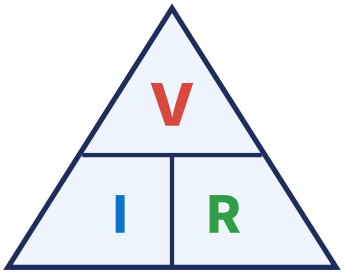
Corriente (I)

Es cuánta electricidad pasa, como el caudal de agua. Se mide en **amperios (A)** o **miliamperios (mA)**. Un LED usa unos 15 mA.

Resistencia (R)

Es lo que frena la corriente, como estrechar la manguera. Se mide en **ohmios (Ω)**.

La ley de Ohm



Las tres se relacionan con una fórmula muy famosa:

$$V = I \times R$$

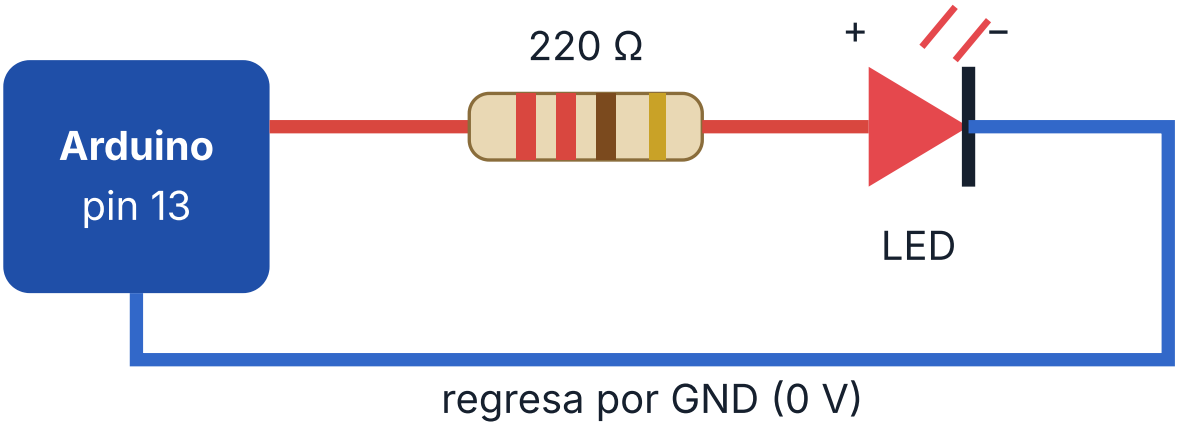
Tapa con el dedo lo que quieres calcular en el triángulo: **I = V ÷ R** y **R = V ÷ I**.

Ejemplo: ¿por qué 220 Ω para un LED?

El pin da 5 V y el LED rojo «se queda» con unos 2 V. Quedan 3 V para la resistencia. Si queremos unos 15 mA (0,015 A): R = 3 ÷ 0,015 = 200 Ω. El valor comercial más cercano es **220 Ω**.

Un circuito completo

La electricidad necesita un camino de ida y vuelta: sale del pin, pasa por la resistencia y el LED, y regresa por **GND**. Si el camino se abre en cualquier punto, nada funciona.



Circuito básico: pin → resistencia → LED → GND.

Potencia

La potencia dice cuánta energía usa un aparato: **P = V × I**, en vatios (W). Una bomba de 12 V que consume 1 A usa 12 W.

Código de colores de las resistencias

Las resistencias no traen el número escrito: lo indican con bandas de colores. Las dos primeras bandas son cifras, la tercera es el multiplicador (cuántos ceros) y la cuarta, dorada o plateada, la tolerancia.

Color	Cifra	Multiplicador
Negro	0	×1
Café	1	×10
Rojo	2	×100
Naranja	3	×1000
Amarillo	4	×10000
Verde	5	×100000
Azul	6	×1000000
Violeta	7	—
Gris	8	—
Blanco	9	—

220 Ω

rojo (2), rojo (2), café (×10) = 220 Ω. Para LED.

10 kΩ

café (1), negro (0), naranja (×1000) = 10 000 Ω. Para LDR y botones.

Señales digitales y analógicas

Digital

Solo dos valores: **HIGH** (5 V, encendido, 1) o **LOW** (0 V, apagado, 0). Un botón, un LED o un relé son digitales.

Analógica

Muchos valores intermedios. El Arduino mide de 0 a 5 V y lo convierte en un número de **0 a 1023**. Una perilla o un sensor de luz son analógicos.

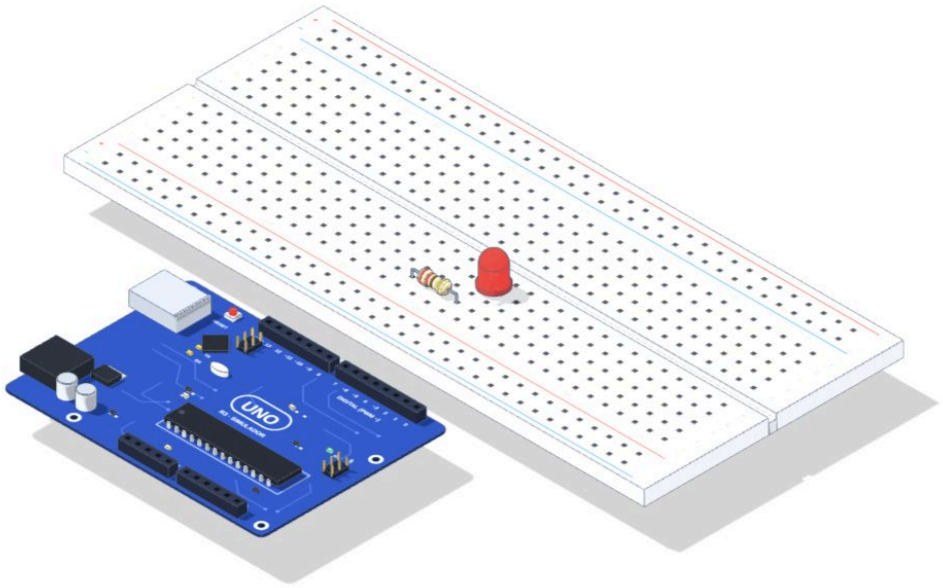
PWM: la «falsa» salida analógica

Los pines con el símbolo **~** (3, 5, 6, 9, 10 y 11) pueden encenderse y apagarse muy rápido. Con `analogWrite(pin, 0…255)` se controla qué parte del tiempo están encendidos: así se cambia el brillo de un LED o la velocidad de un motor.

FUNDAMENTOS

La placa Arduino UNO y la protoboard

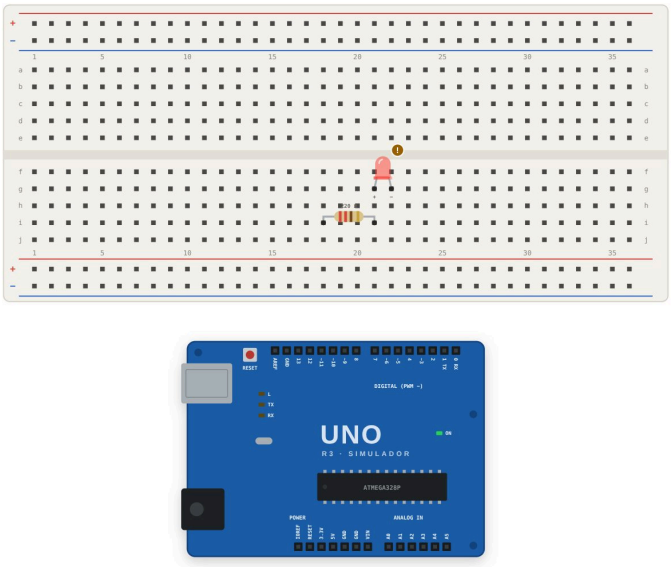
El Arduino es un pequeño computador que lee sensores y controla luces, sonidos y motores según el programa que le escribimos.



Arduino UNO y protoboard, tal como se ven en el simulador Banco Arduino.

Parte	Para qué sirve
Pines digitales D0–D13	Entradas o salidas de HIGH/LOW. D0 y D1 los usa el USB: evítalos. Los que tienen ~ hacen PWM.
Pines analógicos A0–A5	Leen voltajes de 0 a 5 V como números de 0 a 1023. A4 (SDA) y A5 (SCL) también sirven para pantallas I2C.
5V y 3.3V	Salidas de energía para los sensores.
GND	Tierra o negativo (0 V). Hay tres y están unidos.
Conector USB	Lleva el programa desde el computador y da energía de 5 V.
Jack de alimentación	Para una fuente o batería de 7 a 12 V cuando no hay USB.
Botón RESET	Reinicia el programa desde el principio.
LED «L»	Conectado al pin 13: sirve para hacer la primera prueba sin piezas.

La protoboard



Plano de la protoboard y el Arduino.

La protoboard tiene agujeros unidos por dentro con láminas metálicas:

- **Columnas numeradas (1, 2, 3…)**: los 5 agujeros de una columna (a–e o f–j) están unidos.
- **La canal del medio** separa las dos mitades: ahí se montan los chips y módulos.
- **Rieles** (línea roja + y azul –): recorren todo el tablero y reparten 5V y GND.

Por eso cada pata de un componente debe ir en una **columna distinta**; si dos patas quedan en la misma columna, quedan unidas (en corto).

FUNDAMENTOS

Programar el Arduino

Un programa de Arduino (sketch) siempre tiene dos partes.

```ino
// Mi primer programa: parpadear el LED del pin 13
void setup() {            // se ejecuta UNA vez al encender
  pinMode(13, OUTPUT);    // el pin 13 será una salida
}

void loop() {             // se repite PARA SIEMPRE
  digitalWrite(13, HIGH); // enciende
  delay(1000);            // espera 1 segundo
  digitalWrite(13, LOW);  // apaga
  delay(1000);
}
```

Las instrucciones que más vas a usar

Instrucción	Qué hace	Ejemplo
`pinMode(pin, modo)`	Define si un pin es salida o entrada.	`pinMode(2, INPUT_PULLUP);`
`digitalWrite(pin, v)`	Pone un pin en HIGH o LOW.	`digitalWrite(13, HIGH);`
`digitalRead(pin)`	Lee HIGH o LOW.	`if (digitalRead(2) == LOW)`
`analogRead(pin)`	Lee de 0 a 1023.	`int v = analogRead(A0);`
`analogWrite(pin, v)`	PWM de 0 a 255 (pines ~).	`analogWrite(9, 128);`
`delay(ms)`	Espera en milisegundos.	`delay(500);`
`millis()`	Tiempo desde que encendió.	`if (millis() - t > 1000)`
`tone(pin, Hz, ms)`	Toca una nota en el zumbador.	`tone(8, 440, 200);`
`map(v, a, b, c, d)`	Cambia de escala.	`map(v, 0, 1023, 0, 180)`
`Serial.println(x)`	Envía un mensaje al monitor serie.	`Serial.println("Hola");`
`random(a, b)`	Número al azar de a hasta b−1.	`random(1, 7)`

Variables, decisiones y repeticiones

```ino
int contador = 0;             // variable entera
float temperatura = 24.5;     // número con decimales
bool encendido = false;       // verdadero o falso
String nombre = "Ana";        // texto

if (contador > 10) {          // decisión
  Serial.println("Mayor que 10");
} else {
  Serial.println("Todavía no");
```

```
}

for (int i = 0; i < 5; i++) {  // repetir 5 veces
  tone(8, 440 + i * 100, 100);
  delay(150);
}
```

Reglas de escritura

Cada instrucción termina en punto y coma `;`. Las llaves `{ }` agrupan instrucciones. Mayúsculas y minúsculas importan: `HIGH` no es lo mismo que `high`. Lo que va después de `//` es un comentario: el Arduino lo ignora.

FUNDAMENTOS

Del computador al Arduino

Subir un programa con el Arduino IDE

1 **Instala el Arduino IDE**
Descárgalo gratis de **arduino.cc/en/software** e instálalo.

2 **Conecta el Arduino**
Usa el cable USB. Se enciende el LED «ON».

3 **Elige la placa**
Menú **Herramientas → Placa → Arduino AVR Boards → Arduino Uno**.

4 **Elige el puerto**
Herramientas → Puerto y selecciona el que dice COM (Windows) o /dev/tty (Mac/Linux).

5 **Escribe o pega el programa**
Copia el código del proyecto en la ventana del IDE.

6 **Verifica**
Pulsa el botón ✓. Si hay errores, aparecen abajo en rojo con el número de línea.

7 **Sube**
Pulsa el botón → (Subir). Las luces TX y RX parpadean y aparece «Subido».

8 **Abre el monitor serie**
Si el programa usa `Serial`, abre **Herramientas → Monitor serie** y elige **9600 baudios**.

Instalar librerías

Algunos proyectos usan librerías (por ejemplo, para la pantalla, el teclado o el lector de huellas). Se instalan una sola vez:

1. Menú **Herramientas → Administrar bibliotecas** (o el ícono de libros a la izquierda).
2. Escribe el nombre de la librería en el buscador.
3. Pulsa **Instalar**. Si pregunta por dependencias, elige **Instalar todo**.

Librería a buscar	Para
LiquidCrystal I2C	Pantalla LCD 16×2 con módulo I2C
Keypad	Teclado matricial 4×4
MFRC522	Lector de tarjetas RFID RC522
Adafruit Fingerprint Sensor Library	Lector de huellas R307 / AS608
DHT sensor library	Sensor de temperatura y humedad DHT11
Adafruit SSD1306 y Adafruit GFX	Pantalla OLED
Servo	Servomotores (ya viene instalada)

Registrar huellas en el lector

Antes de usar el lector de huellas, abre **Archivo → Ejemplos → Adafruit Fingerprint Sensor Library → enroll**, súbelo y sigue las instrucciones del monitor serie: escribe el número (1, 2, 3...) y pon el dedo dos veces. Ese número es el que devuelve `leerHuella()` en los proyectos.

FUNDAMENTOS

El simulador Banco Arduino

Todos los proyectos de este libro están dentro del simulador. Se puede usar en el computador, la tableta o el celular, incluso sin internet después de abrirlo la primera vez.

Dirección del simulador

https://andresyairrenteriamurillo-maker.github.io/banco-arduino/

1 **Busca el proyecto**
Entra a 📚 **Proyectos**, escribe su nombre (por ejemplo «chapa con huella») o filtra por categoría.

2 **Elige cómo armarlo**
🧩 **Armarlo yo, con guía** pone las piezas sin cables y te marca con ✓ cada conexión correcta. 👀 **Ver armado completo** lo deja listo.

3 **Revisa el circuito**
El panel **Revisión del circuito** avisa si falta un cable, si una pieza está al revés o si hay un corto.

4 **Ejecuta el programa**
Pulsa ▶ **Ejecutar**. Las luces, la pantalla, el zumbador y los servos se mueven como en la vida real, y el monitor serie muestra los mensajes.

5 **Interactúa con los sensores**
En el panel **Componentes** cada sensor tiene su control: botones, deslizadores, huellas, tarjetas o teclas.

6 **Mira la vista ilustrada**
La pestaña **Vista ilustrada** muestra el circuito en 3D con etiquetas, y Chispa, el robot guía, te lo explica paso a paso.

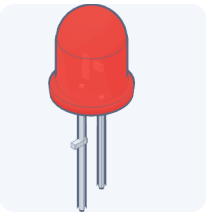
Para el docente

En el modo 👩‍🏫 **Docente** (con PIN) se pueden crear proyectos nuevos, editarlos y exportarlos para compartirlos con otros computadores.

FUNDAMENTOS

Catálogo de componentes

Estas son las piezas que aparecen en los proyectos. Cada ficha dice para qué sirve y cómo se conecta.

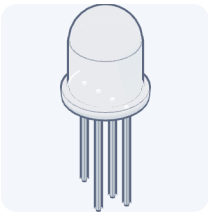


LED

Diodo que emite luz. Es la salida más sencilla para ver si el programa funciona.

Conexión: Ánodo (+, pata larga) y cátodo (–, pata corta, lado plano).

Ojo: Siempre con una resistencia de 220 Ω en serie, o se quema.

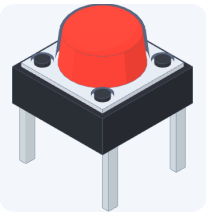


LED RGB

Tres LED (rojo, verde y azul) en uno. Mezclándolos se logra cualquier color.

Conexión: R, G, B y la pata común (la más larga) a GND.

Ojo: Usa pines con ~ (PWM) para mezclar colores con analogWrite.

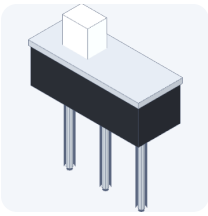


Pulsador

Botón que solo conecta mientras lo presionas.

Conexión: Dos patas: una al pin y otra a GND.

Ojo: Con INPUT_PULLUP no necesitas resistencia: presionado se lee LOW.

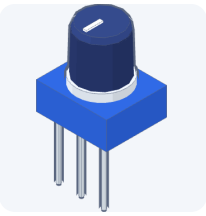


Interruptor

Se queda en la posición en que lo dejes: encendido o apagado.

Conexión: Común al pin y un extremo a GND.

Ojo: También se lee con INPUT_PULLUP.

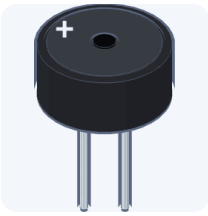


Potenciómetro

Perilla que da un voltaje variable. El Arduino lo lee de 0 a 1023.

Conexión: Extremos a 5V y GND; el centro (cursor) a A0–A5.

Ojo: Si gira al revés, intercambia los extremos.

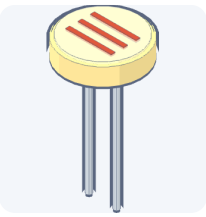


Zumbador (buzzer)

Pequeño parlante para pitidos, alarmas y melodías con tone().

Conexión: + al pin y – a GND.

Ojo: Los activos solo pitan; los pasivos tocan notas. El kit usa pasivos.



Fotorresistencia (LDR)

Cambia su resistencia con la luz. Sirve para saber si es de día o de noche.

Conexión: Una pata a 5V; la otra a A0 y a GND con una resistencia de 10 kΩ.

Ojo: Más luz = número más alto (en este montaje).

Servomotor SG90

Motor que gira a un ángulo exacto entre 0 y 180 grados.

Conexión: Café/negro GND, rojo 5V, naranja señal a un pin digital.

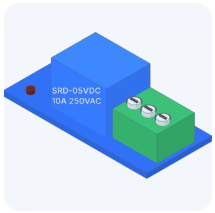
Ojo: Con 2 o más servos usa una fuente externa de 5 V.

Motor DC

Gira sin parar. Su velocidad se controla con PWM.

Conexión: Siempre a través de un transistor o un módulo L298N.

Ojo: Nunca conectes un motor directo al pin: pide demasiada corriente.

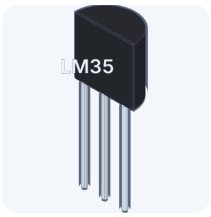


Módulo relé 5 V

Interruptor que el Arduino activa para encender cargas grandes (bombas, chapas, lámparas).

Conexión: VCC, GND, IN al pin. Lado de carga: COM y NO.

Ojo: Separa el lado de 5 V del lado de 12 V o 110 V.

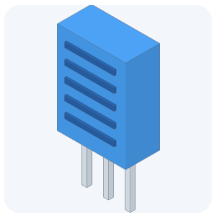


Sensor de temperatura LM35

Entrega 10 mV por cada grado Celsius.

Conexión: VCC 5V, OUT a A0–A5, GND.

Ojo: Temperatura = lectura × 500 ÷ 1023.

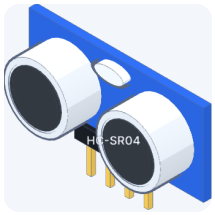


Sensor DHT11

Mide temperatura y humedad del aire en un solo módulo.

Conexión: VCC, DATA a un pin digital, GND.

Ojo: Se lee cada 2 segundos como mínimo.

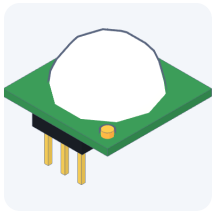


Sensor ultrasónico HC-SR04

Mide distancias con un eco de sonido (de 2 a 400 cm).

Conexión: VCC, TRIG y ECHO a pines digitales, GND.

Ojo: Distancia en cm = tiempo del eco ÷ 58.

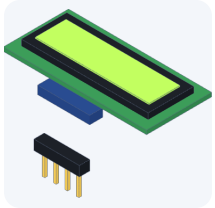


Sensor de movimiento PIR

Detecta el calor de personas o animales que se mueven.

Conexión: VCC, OUT a un pin digital, GND.

Ojo: Al encenderlo necesita unos 30 a 60 segundos para estabilizarse.

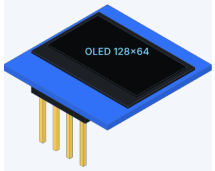


Pantalla LCD 16×2 con I2C

Muestra 2 renglones de 16 letras. Con el módulo I2C solo usa 4 cables.

Conexión: GND, VCC, SDA a A4 y SCL a A5.

Ojo: Si no se ve nada, gira el potenciómetro azul del módulo (contraste).

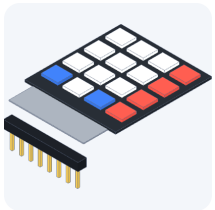


Pantalla OLED 0,96"

Pantalla pequeña de alto contraste para textos y dibujos.

Conexión: GND, VCC, SDA a A4 y SCL a A5.

Ojo: Su dirección suele ser 0x3C.

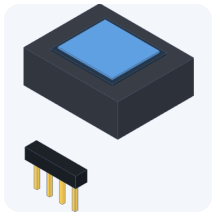


Teclado matricial 4×4

16 teclas organizadas en 4 filas y 4 columnas. Ideal para claves.

Conexión: 8 cables: filas a D9–D6 y columnas a D5–D2 (en este libro).

Ojo: La librería Keypad se encarga de descubrir qué tecla se presionó.

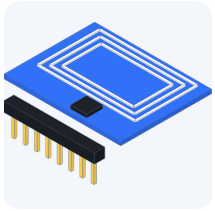


Lector de huellas R307 / AS608

Toma la imagen del dedo y la compara con las huellas guardadas en su memoria.

Conexión: Rojo 5V, negro GND, TX del lector a D2 y RX a D3.

Ojo: Primero registra las huellas con el ejemplo «enroll» de la librería.

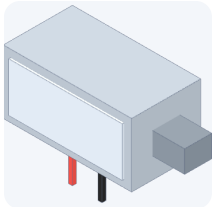


Lector RFID RC522

Lee tarjetas y llaveros sin contacto, cada uno con un número único (UID).

Conexión: SDA D10, SCK D13, MOSI D11, MISO D12, RST D9, GND y 3.3V.

Ojo: ¡Se alimenta con 3.3 V, no con 5 V!

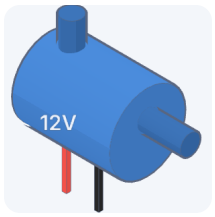


Chapa eléctrica 12 V

Cerradura con un electroimán que recoge el pestillo cuando recibe 12 V.

Conexión: + al NO del relé, – al – de la fuente de 12 V.

Ojo: Pon un diodo 1N4007 en paralelo para proteger el relé.

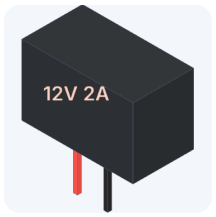


Bomba de agua 12 V

Mueve agua para riego, lavamanos o rociadores.

Conexión: + al NO del relé, – al – de la fuente de 12 V.

Ojo: Nunca la hagas funcionar en seco.

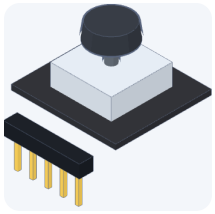


Fuente de 12 V

Da la energía de las cargas grandes (bomba, chapa).

Conexión: +12 V al COM del relé; – a la carga.

Ojo: No la conectes a los pines del Arduino.

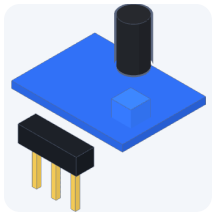


Joystick

Dos potenciómetros (X y Y) y un botón en una sola palanca.

Conexión: GND, 5V, VRx a A0, VRy a A1, SW a un pin digital.

Ojo: En reposo cada eje lee cerca de 512.

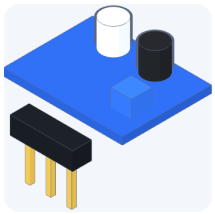


Sensor de llama

Detecta la luz infrarroja de una llama.

Conexión: VCC, GND, DO a un pin digital.

Ojo: DO = LOW cuando ve fuego.

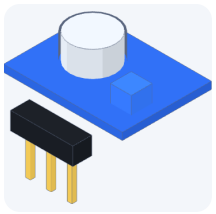


Sensor de obstáculos IR

Emite luz infrarroja y detecta si rebota en algo cercano.

Conexión: VCC, GND, OUT a un pin digital.

Ojo: OUT = LOW cuando hay un objeto. El tornillo ajusta la distancia.

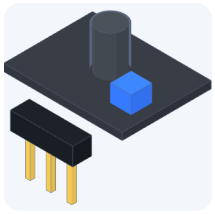


Sensor de sonido

Micrófono que avisa cuando hay ruido o un aplauso.

Conexión: VCC, GND, DO a un pin digital.

Ojo: Ajusta la sensibilidad con el tornillo del módulo.

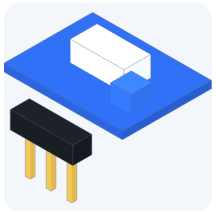


Sensor de inclinación

Una bolita metálica que cierra el circuito según la posición.

Conexión: VCC, GND, OUT a un pin digital.

Ojo: Sirve para detectar caídas o movimientos bruscos.

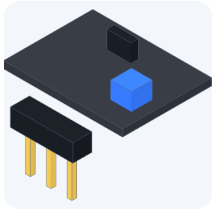


Sensor de vibración SW-420

Detecta golpes y vibraciones.

Conexión: VCC, GND, DO a un pin digital.

Ojo: Para un sismógrafo, cuenta las vibraciones por segundo.

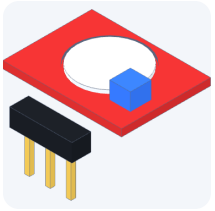


Sensor magnético Hall

Detecta la presencia de un imán.

Conexión: VCC, GND, DO a un pin digital.

Ojo: DO = LOW cuando el imán está cerca.

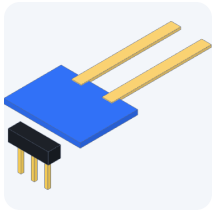


Sensor táctil TTP223

Botón sin partes móviles: detecta el toque del dedo.

Conexión: VCC, GND, SIG a un pin digital.

Ojo: Funciona incluso a través de un plástico delgado.

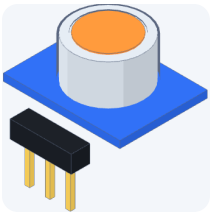


Sensor de humedad de suelo

Dos patas que se entierran para medir qué tan húmeda está la tierra.

Conexión: VCC, GND, A0 a una entrada analógica.

Ojo: Seco = número alto; mojado = número bajo.

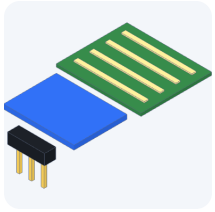


Sensor de gas MQ-2

Detecta gas propano, butano y humo.

Conexión: VCC, GND, A0 a una entrada analógica.

Ojo: Necesita calentarse 1 o 2 minutos antes de medir bien.

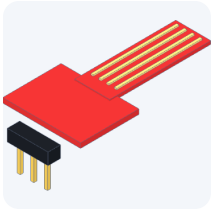


Sensor de lluvia

Una placa con pistas que detecta gotas de agua.

Conexión: VCC, GND, A0 a una entrada analógica.

Ojo: Seco = número alto.

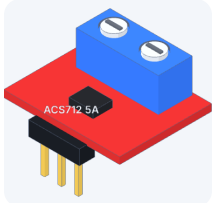


Sensor de nivel de agua

Mide qué tanto de la placa está sumergida.

Conexión: VCC, GND, S a una entrada analógica.

Ojo: Seco = 0; sumergido ≈ 650.



Sensor de corriente ACS712

Mide la corriente (amperios) que consume un aparato.

Conexión: VCC, GND, OUT a una entrada analógica. La carga pasa por los tornillos.

Ojo: Sin corriente entrega 2,5 V; cambia 0,185 V por amperio.

CAPÍTULO 1

Luces y LED

Encender, apagar, parpadear y mezclar colores. Aquí aprendes las salidas digitales y el PWM, la base de todo lo demás.

EN ESTE CAPÍTULO · 20 PROYECTOS

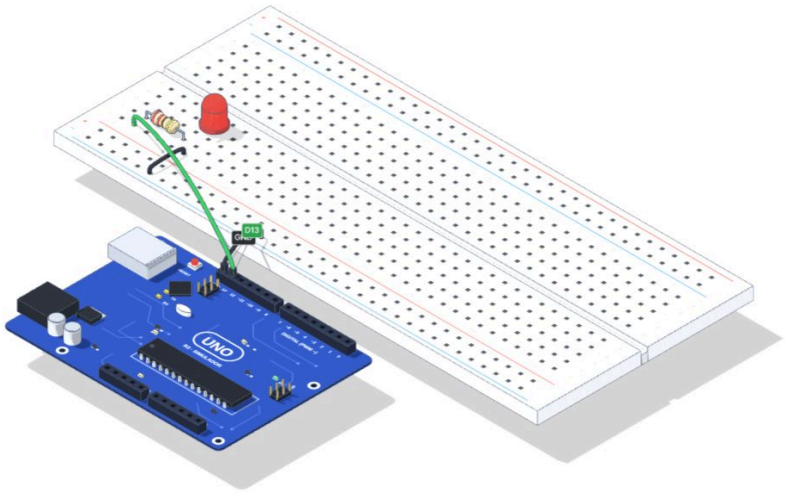
1	Mi primer LED parpadeante	★☆☆
2	Señal de auxilio SOS	★☆☆
3	Faro de barco	★☆☆
4	Luz de vela	★☆☆
5	LED que respira	★☆☆
6	Luces de policía	★☆☆
7	Semáforo básico	★☆☆
8	Semáforo de dos vías	★★☆
9	Auto fantástico	★★☆
10	Contador binario de 4 bits	★★☆
11	Luces de Navidad	★☆☆
12	Luz de emergencia estroboscópica	★☆☆
13	Latido del corazón	★☆☆
14	Ruleta de luces	★★☆
15	Metrónomo de luz	★☆☆
16	Escalera de luces	★☆☆
17	Arcoíris con LED RGB	★★☆
18	Semáforo con cuenta en el monitor	★★☆
19	Dos LED alternados con velocidad creciente	★☆☆
20	Vela de cumpleaños RGB	★★☆

PROYECTO 1 · LUCES Y LED

Mi primer LED parpadeante

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

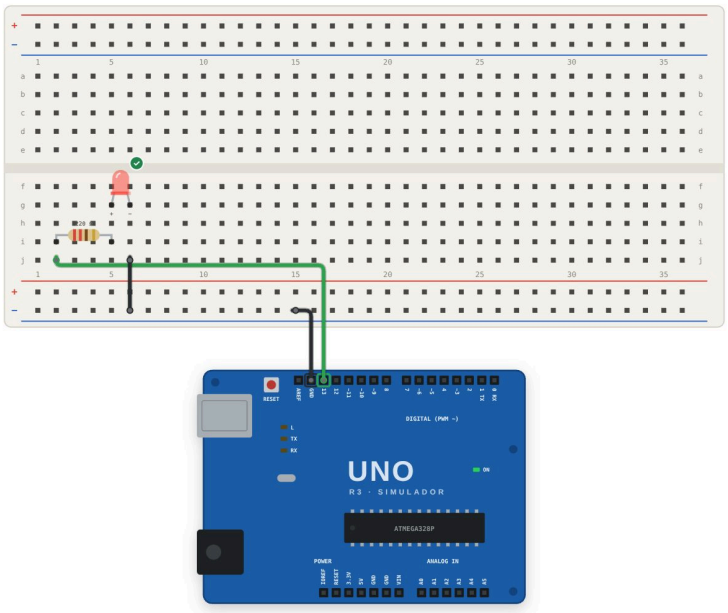
Un LED se enciende y se apaga cada medio segundo: el "Hola mundo" de Arduino.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × LED rojo (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **LED rojo (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
1  // Mi primer LED parpadeante
2  const int LED = 13;          // el LED está en el pin 13
3
4  void setup() {
5    pinMode(LED, OUTPUT);      // el pin 13 será una salida
6  }
7
8  void loop() {
9    digitalWrite(LED, HIGH);   // enciende el LED (5 V)
10   delay(500);                // espera medio segundo
11   digitalWrite(LED, LOW);    // apaga el LED (0 V)
12   delay(500);
13 }
```

💡 **¿Cómo funciona?**

En setup() el pin 13 se configura como salida. En loop(), que se repite para siempre, el pin se pone en HIGH (5 V) y luego en LOW (0 V), con una pausa entre cada cambio.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

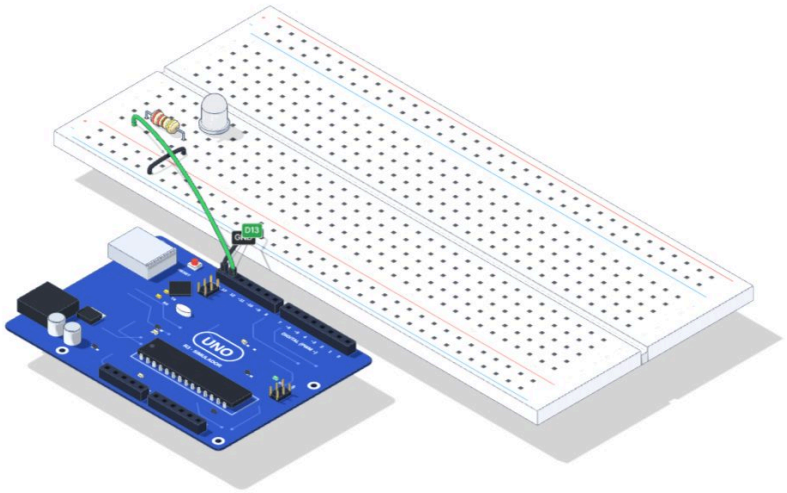
🏆 **Reto:** Cambia los dos delay() para que el LED parpadee el doble de rápido.

PROYECTO 2 · LUCES Y LED

Señal de auxilio SOS

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

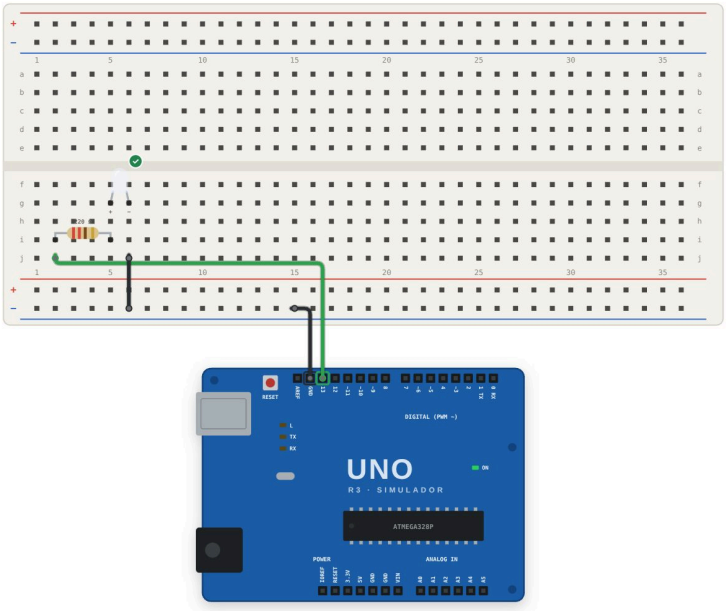
El LED envía el mensaje SOS en código Morse: tres destellos cortos, tres largos y tres cortos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Luz de auxilio (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Luz de auxilio (LED blanco)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
// SOS en código Morse
const int LED = 13;

void destello(int duracion) {
  digitalWrite(LED, HIGH);
  delay(duracion);
  digitalWrite(LED, LOW);
  delay(200);                // pausa entre destellos
}

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  for (int i = 0; i < 3; i++) destello(200);   // S: tres cortos
  for (int i = 0; i < 3; i++) destello(600);   // O: tres largos
  for (int i = 0; i < 3; i++) destello(200);   // S: tres cortos
  delay(1500);               // pausa antes de repetir
}
```

💡 ¿Cómo funciona?

La función destello() enciende el LED el tiempo que le indiquemos. Un ciclo for la repite tres veces con destellos cortos (puntos) y tres con destellos largos (rayas).

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

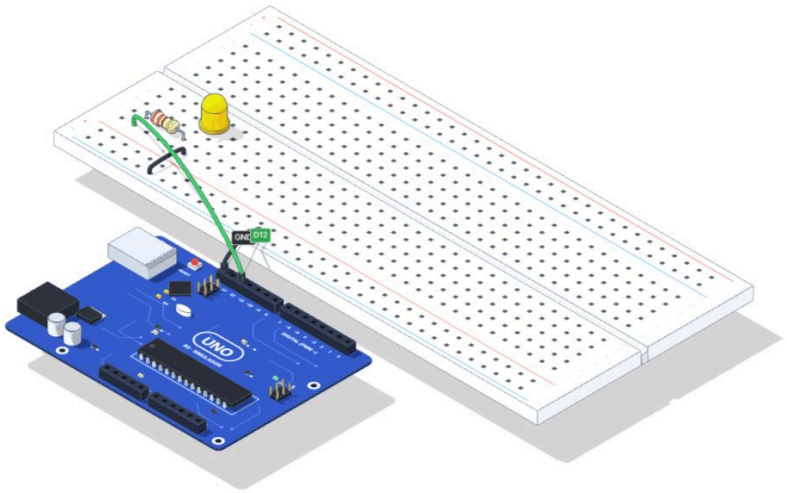
🏆 **Reto:** Escribe tu nombre en código Morse con destellos cortos y largos.

PROYECTO 3 · LUCES Y LED

Faro de barco

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

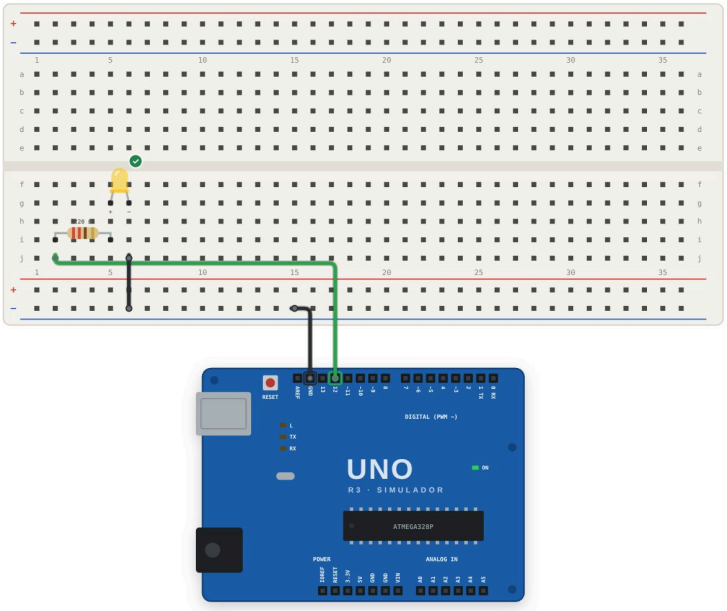
Un LED amarillo imita un faro: un destello muy rápido cada dos segundos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Luz del faro (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Luz del faro (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
1  // Faro de barco
2  const int FARO = 12;
3
4  void setup() {
5    pinMode(FARO, OUTPUT);
6  }
7
8  void loop() {
9    digitalWrite(FARO, HIGH);
10   delay(80);                 // destello muy corto
11   digitalWrite(FARO, LOW);
12   delay(150);
13   digitalWrite(FARO, HIGH);
14   delay(80);                 // segundo destello
15   digitalWrite(FARO, LOW);
16   delay(2000);               // larga pausa: así lo reconocen los barcos
17 }
```

💡 ¿Cómo funciona?

Cada faro real tiene su propio ritmo de destellos para que los marineros lo reconozcan. Aquí el ritmo es: dos destellos cortos y una pausa de dos segundos.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

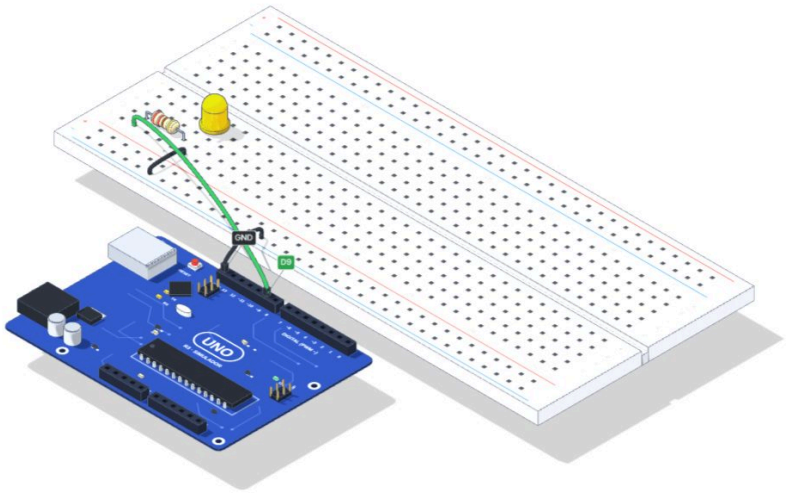
🏆 **Reto:** Inventa el ritmo de tu propio faro con tres destellos.

PROYECTO 4 · LUCES Y LED

Luz de vela

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

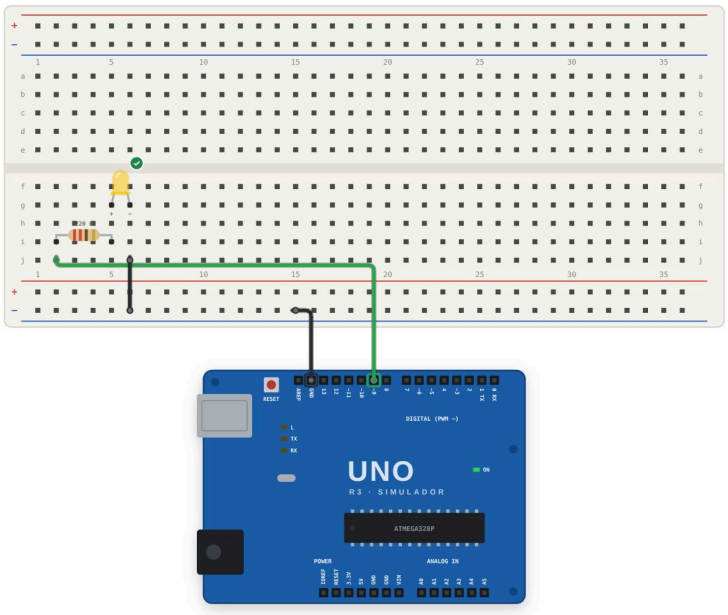
Un LED amarillo cambia su brillo al azar para parecer la llama de una vela.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Llama (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Llama (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
1  // Luz de vela: brillo al azar
2  const int LLAMA = 9;         // pin con PWM (~) para regular el brillo
3
4  void setup() {
5    pinMode(LLAMA, OUTPUT);
6  }
7
8  void loop() {
9    int brillo = random(80, 256);     // número al azar entre 80 y 255
10   analogWrite(LLAMA, brillo);       // brillo de 0 (apagado) a 255 (máximo)
11   delay(random(40, 120));           // espera también al azar
12 }
```

💡 **¿Cómo funciona?**

analogWrite() regula el brillo con PWM: el pin se enciende y apaga muy rápido, y el ojo ve un brillo intermedio. random() da números al azar, que imitan el temblor de una llama.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

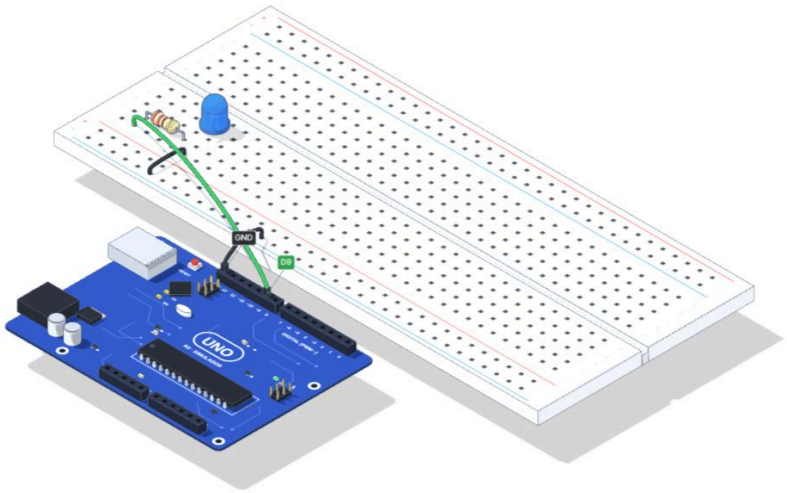
🏆 **Reto:** Agrega un segundo LED rojo para que la vela tenga dos colores.

PROYECTO 5 · LUCES Y LED

LED que respira

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

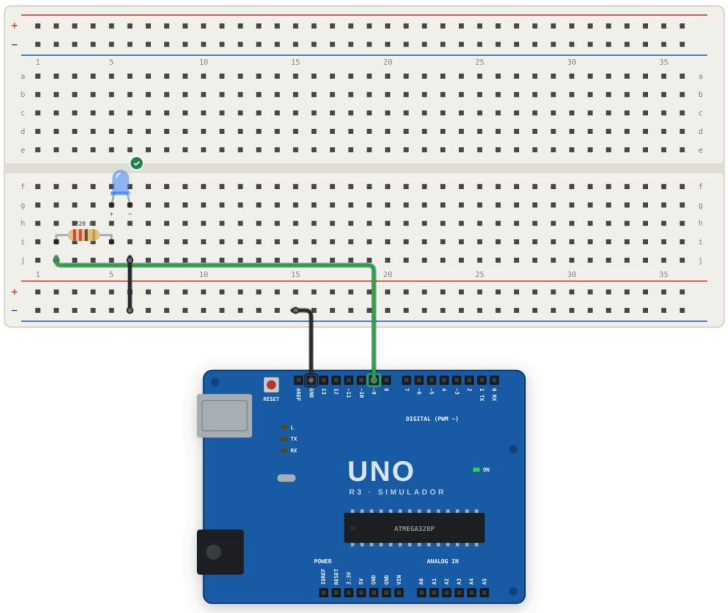
El brillo del LED sube y baja suavemente, como si respirara.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × LED azul (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **LED azul (LED azul)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
1  // LED que respira
2  const int LED = 9;           // pin con PWM (~)
3
4  void setup() {
5    pinMode(LED, OUTPUT);
6  }
7
8  void loop() {
9    for (int b = 0; b <= 255; b += 5) {   // sube el brillo
10     analogWrite(LED, b);
11     delay(20);
12   }
13   for (int b = 255; b >= 0; b -= 5) {   // baja el brillo
14     analogWrite(LED, b);
15     delay(20);
16   }
17 }
```

💡 ¿Cómo funciona?

Dos ciclos for cambian el brillo de 5 en 5: el primero de 0 a 255 y el segundo de 255 a 0. Como cada paso dura 20 ms, el cambio se ve suave.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

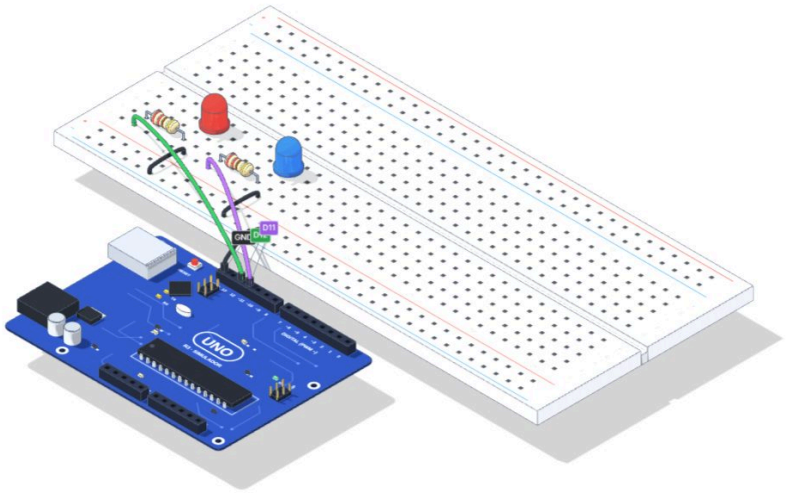
🏆 **Reto:** Haz que respire más lento cambiando el delay a 40 ms.

PROYECTO 6 · LUCES Y LED

Luces de policía

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

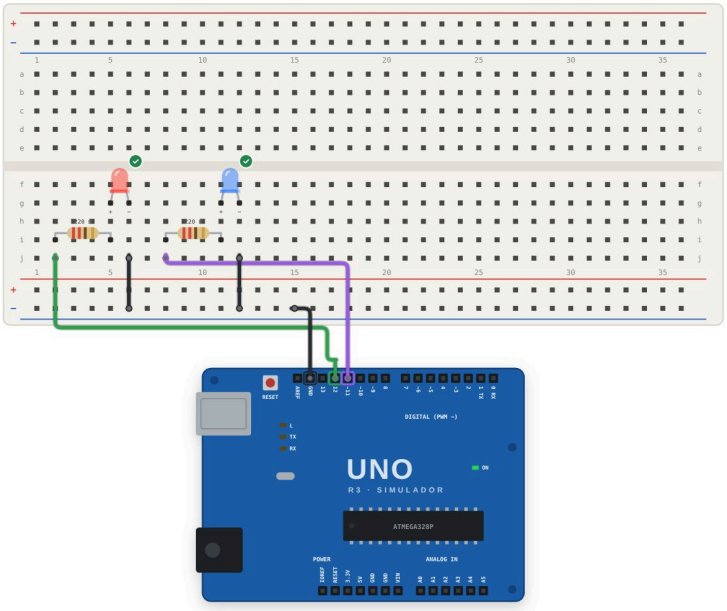
Un LED rojo y uno azul se alternan como las luces de una patrulla.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Luz roja (LED)
- 1 × Luz azul (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Luz roja (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Luz azul (LED azul)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Luces de policía
const int ROJO = 12;
const int AZUL = 11;

void destellar(int pin) {
  for (int i = 0; i < 3; i++) {     // tres destellos rápidos
    digitalWrite(pin, HIGH);
    delay(60);
    digitalWrite(pin, LOW);
    delay(60);
  }
}

void setup() {
  pinMode(ROJO, OUTPUT);
  pinMode(AZUL, OUTPUT);
}

void loop() {
  destellar(ROJO);
  destellar(AZUL);
}
```

💡 ¿Cómo funciona?

La función destellar() recibe el número de pin y hace tres destellos rápidos. Llamándola primero con el rojo y después con el azul se logra el efecto de patrulla.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

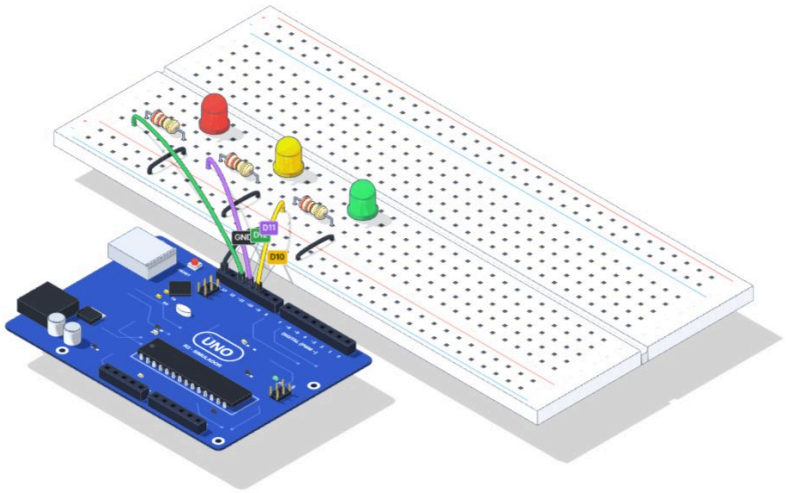
🏆 **Reto:** Agrega un zumbador que suene con cada destello.

PROYECTO 7 · LUCES Y LED

Semáforo básico

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

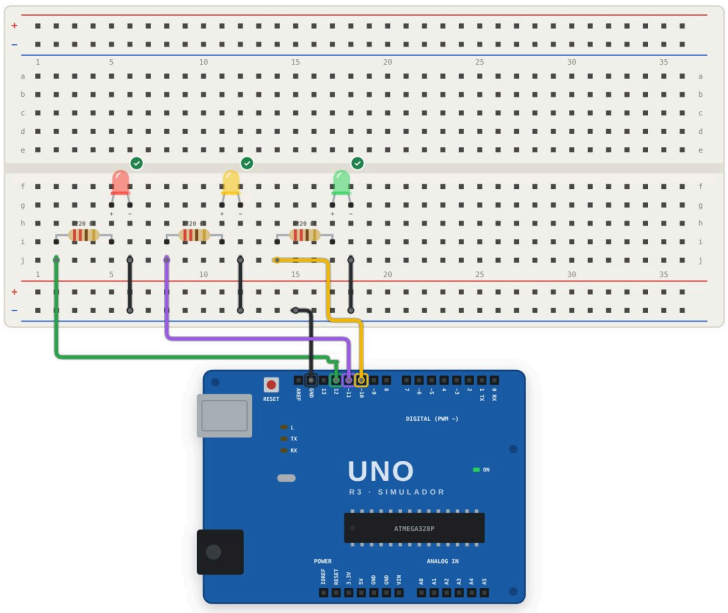
Tres LED funcionan como un semáforo: verde, amarillo y rojo con sus tiempos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Rojo (LED)
- 1 × Amarillo (LED)
- 1 × Verde (LED)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Rojo (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Amarillo (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

4 **Verde (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Semáforo básico
const int ROJO = 12;
const int AMARILLO = 11;
const int VERDE = 10;

void luz(int r, int a, int v, int tiempo) {
  digitalWrite(ROJO, r);
  digitalWrite(AMARILLO, a);
  digitalWrite(VERDE, v);
  delay(tiempo);
}

void setup() {
  pinMode(ROJO, OUTPUT);
  pinMode(AMARILLO, OUTPUT);
  pinMode(VERDE, OUTPUT);
}

void loop() {
  luz(LOW, LOW, HIGH, 4000);   // verde 4 segundos
  luz(LOW, HIGH, LOW, 1500);   // amarillo 1,5 segundos
  luz(HIGH, LOW, LOW, 4000);   // rojo 4 segundos
}
```

💡 ¿Cómo funciona?

La función luz() decide qué LED se enciende y cuánto tiempo. El loop() repite la secuencia verde, amarillo y rojo, como un semáforo real.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

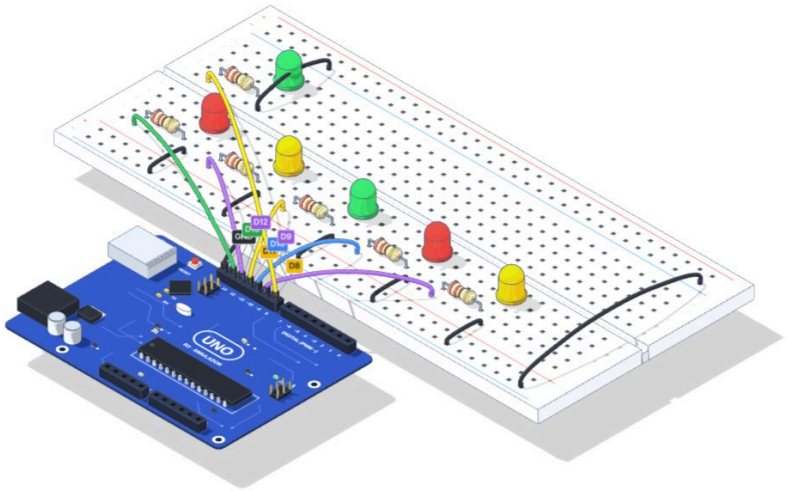
🏆 **Reto:** Agrega una fase en la que el verde parpadea antes del amarillo.

PROYECTO 8 · LUCES Y LED

Semáforo de dos vías

★★☆ Intermedio · Tiempo: 30 a 45 minutos

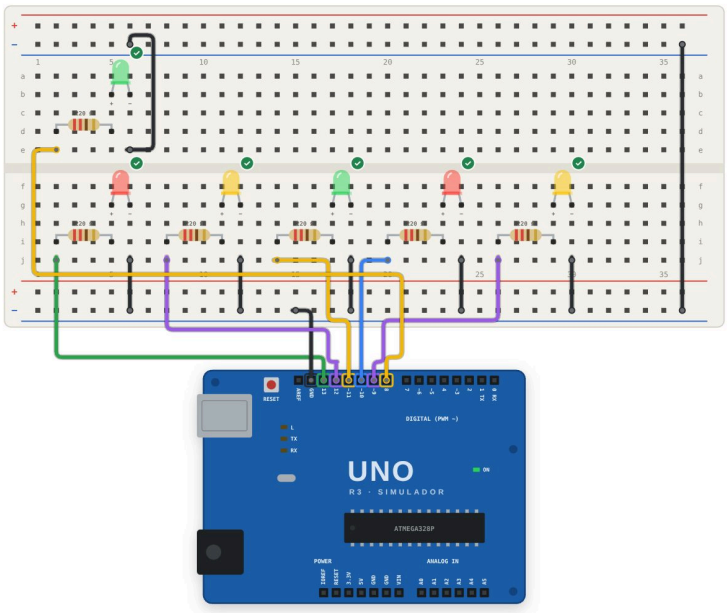
Dos semáforos coordinados para un cruce: cuando uno está en verde, el otro está en rojo.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Calle A rojo (LED)
- 1 × Calle A amarillo (LED)
- 1 × Calle A verde (LED)
- 1 × Calle B rojo (LED)
- 1 × Calle B amarillo (LED)
- 1 × Calle B verde (LED)
- 6 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Calle A rojo (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Calle A amarillo (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

4 **Calle A verde (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **Calle B rojo (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 **Calle B amarillo (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 **Calle B verde (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D8**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

8 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 140 mA.
- Consumo estimado: **~140 mA** funcionando, con picos de **~140 mA**.

2 El programa

```
// Semáforo de dos vías (cruce)
const int AR = 13, AA = 12, AV = 11;   // calle A
const int BR = 10, BA = 9, BV = 8;     // calle B

void poner(int ar, int aa, int av, int br, int ba, int bv, int tiempo) {
  digitalWrite(AR, ar); digitalWrite(AA, aa); digitalWrite(AV, av);
  digitalWrite(BR, br); digitalWrite(BA, ba); digitalWrite(BV, bv);
  delay(tiempo);
}

void setup() {
  for (int p = 8; p <= 13; p++) pinMode(p, OUTPUT);
}

void loop() {
  poner(0, 0, 1,  1, 0, 0, 4000);   // A verde, B rojo
  poner(0, 1, 0,  1, 0, 0, 1500);   // A amarillo
  poner(1, 0, 0,  0, 0, 1, 4000);   // A rojo, B verde
  poner(1, 0, 0,  0, 1, 0, 1500);   // B amarillo
```

```
20 }
```

💡 **¿Cómo funciona?**

Los seis LED se controlan juntos con la función poner(). Nunca hay dos verdes a la vez: así funcionan los cruces de verdad para evitar choques.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

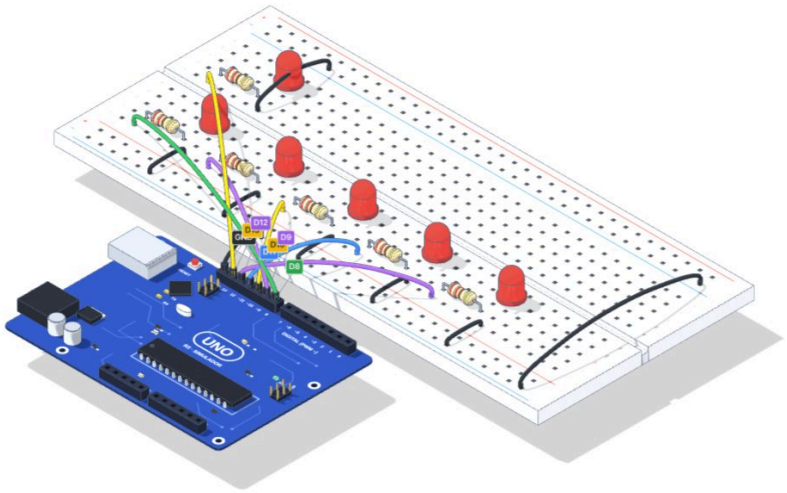
🏆 **Reto:** Agrega 1 segundo con las dos calles en rojo entre cada cambio, como en los cruces reales.

PROYECTO 9 · LUCES Y LED

Auto fantástico

★★☆ Intermedio · Tiempo: 30 a 45 minutos

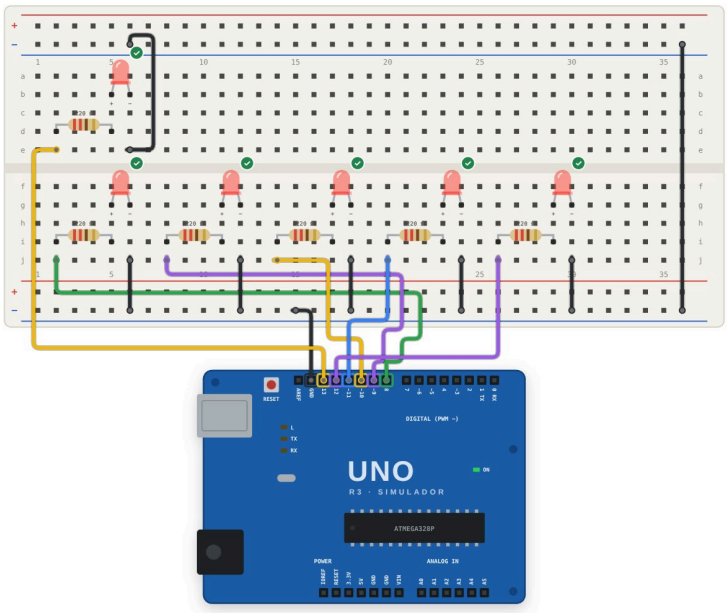
Seis LED se encienden uno tras otro de ida y vuelta, como el famoso auto de la televisión.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × LED 1 (LED)
- 1 × LED 2 (LED)
- 1 × LED 3 (LED)
- 1 × LED 4 (LED)
- 1 × LED 5 (LED)
- 1 × LED 6 (LED)
- 6 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **LED 1 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D8**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **LED 2 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

4 **LED 3 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **LED 4 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 **LED 5 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 **LED 6 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

8 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 140 mA.
- Consumo estimado: **~140 mA** funcionando, con picos de **~140 mA**.

2 El programa

```
// Auto fantástico: luz que va y viene
int leds[] = {8, 9, 10, 11, 12, 13};   // los seis pines en orden

void setup() {
  for (int i = 0; i < 6; i++) pinMode(leds[i], OUTPUT);
}

void loop() {
  for (int i = 0; i < 6; i++) {        // de izquierda a derecha
    digitalWrite(leds[i], HIGH);
    delay(80);
    digitalWrite(leds[i], LOW);
  }
  for (int i = 4; i > 0; i--) {        // de regreso
    digitalWrite(leds[i], HIGH);
    delay(80);
    digitalWrite(leds[i], LOW);
  }
```

```
19 }
```

💡 **¿Cómo funciona?**

Los pines se guardan en un arreglo (una lista). Así un ciclo for puede recorrerlos en orden, primero hacia adelante y luego hacia atrás.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

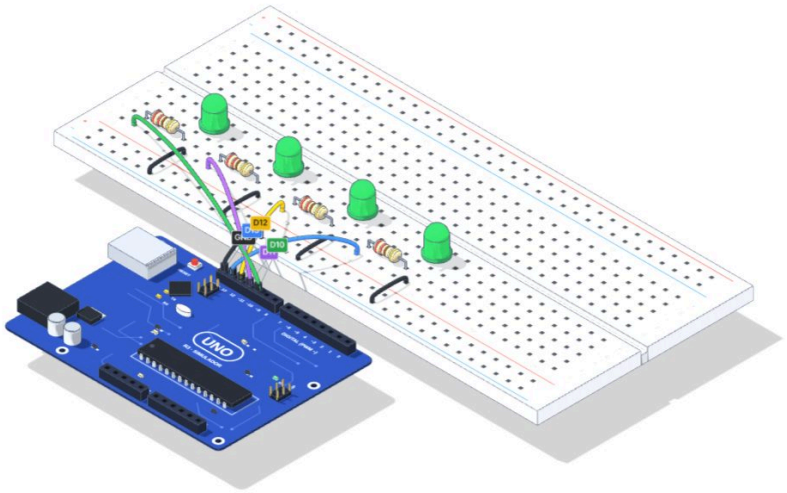
🏆 **Reto:** Haz que se enciendan dos LED vecinos al tiempo para que la luz se vea más grande.

PROYECTO 10 · LUCES Y LED

Contador binario de 4 bits

★★☆ Intermedio · Tiempo: 30 a 45 minutos

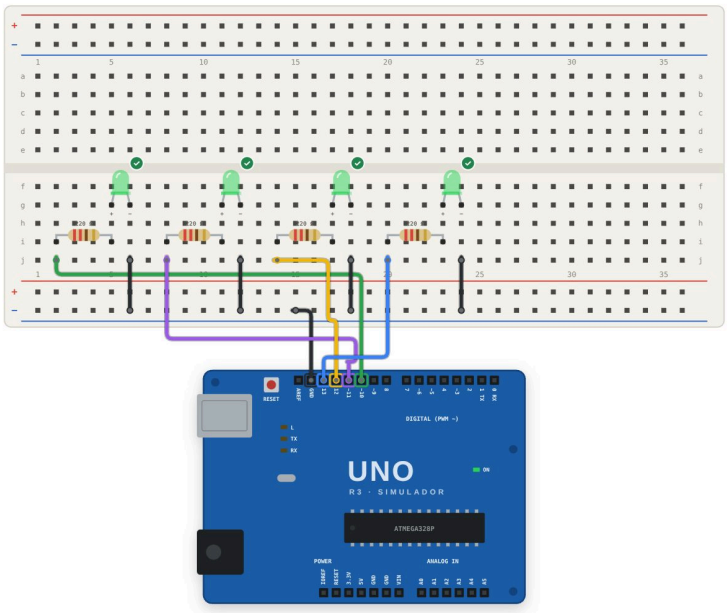
Cuatro LED muestran los números del 0 al 15 en sistema binario.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Bit 1 (LED)
- 1 × Bit 2 (LED)
- 1 × Bit 4 (LED)
- 1 × Bit 8 (LED)
- 4 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Bit 1 (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Bit 2 (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

4 **Bit 4 (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

5 **Bit 8 (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

6 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
// Contador binario de 0 a 15
int bits[] = {10, 11, 12, 13};   // valor 1, 2, 4 y 8

void setup() {
  Serial.begin(9600);
  for (int i = 0; i < 4; i++) pinMode(bits[i], OUTPUT);
}

void loop() {
  for (int n = 0; n < 16; n++) {
    Serial.print("Número: ");
    Serial.println(n);
    for (int i = 0; i < 4; i++) {
      int bit = (n >> i) & 1;       // ¿el bit i de n vale 1?
      digitalWrite(bits[i], bit);
    }
    delay(1000);
  }
}
```

💡 **¿Cómo funciona?**

Cada LED representa una potencia de 2 (1, 2, 4 y 8). La operación (n >> i) & 1 averigua si el bit número i del número está encendido. El monitor serie muestra el número en decimal para comparar.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa y abre el monitor serie a 9600.

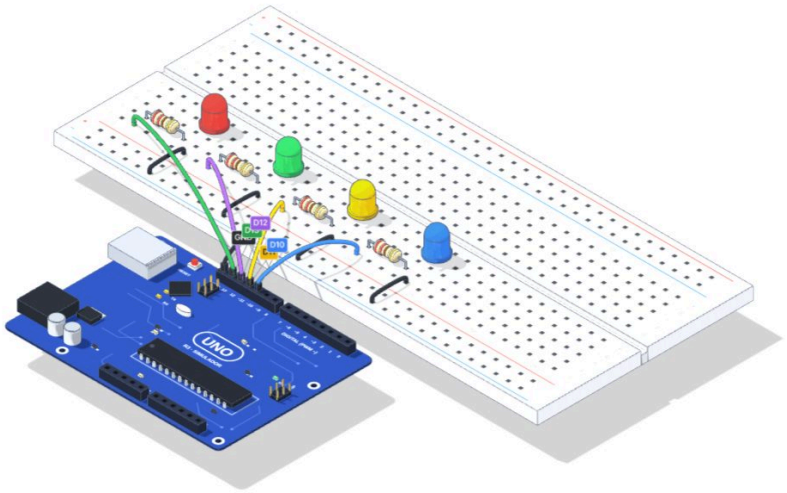
🏆 **Reto:** Cuenta hacia atrás, de 15 a 0.

PROYECTO 11 · LUCES Y LED

Luces de Navidad

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

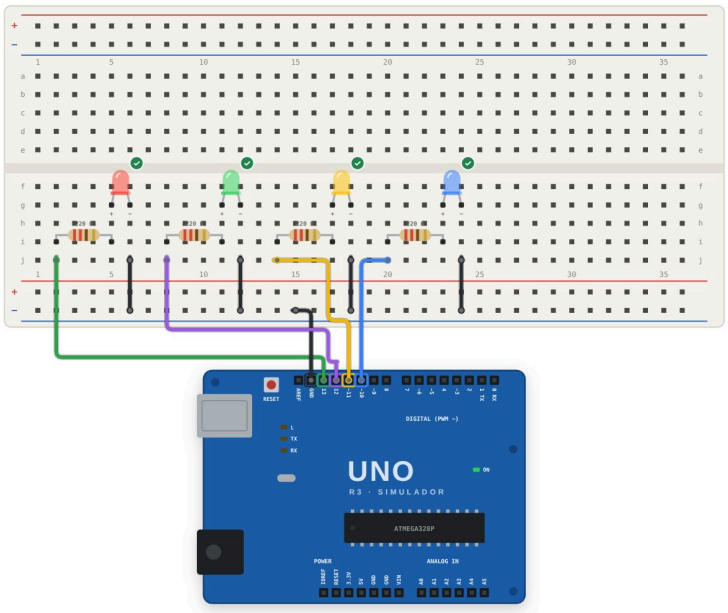
Cuatro LED de colores se encienden en patrones que cambian solos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Rojo (LED)
- 1 × Verde (LED)
- 1 × Amarillo (LED)
- 1 × Azul (LED)
- 4 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Rojo (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Verde (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

4 **Amarillo (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **Azul (LED azul)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
// Luces de Navidad con tres efectos
int luces[] = {13, 12, 11, 10};

void todas(int estado) {
  for (int i = 0; i < 4; i++) digitalWrite(luces[i], estado);
}

void setup() {
  for (int i = 0; i < 4; i++) pinMode(luces[i], OUTPUT);
}

void loop() {
  // Efecto 1: todas parpadean
  for (int k = 0; k < 4; k++) { todas(HIGH); delay(300); todas(LOW); delay(300); }
  // Efecto 2: una por una
  for (int k = 0; k < 3; k++) {
    for (int i = 0; i < 4; i++) { digitalWrite(luces[i], HIGH); delay(150); digitalWrite(luces[i], LOW); }
  }
  // Efecto 3: al azar
  for (int k = 0; k < 20; k++) {
    digitalWrite(luces[random(4)], random(2));
    delay(100);
  }
  todas(LOW);
}
```

💡 **¿Cómo funciona?**

El loop() tiene tres efectos seguidos. La función todas() enciende o apaga todas las luces juntas, y random() escoge un LED y un estado al azar.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

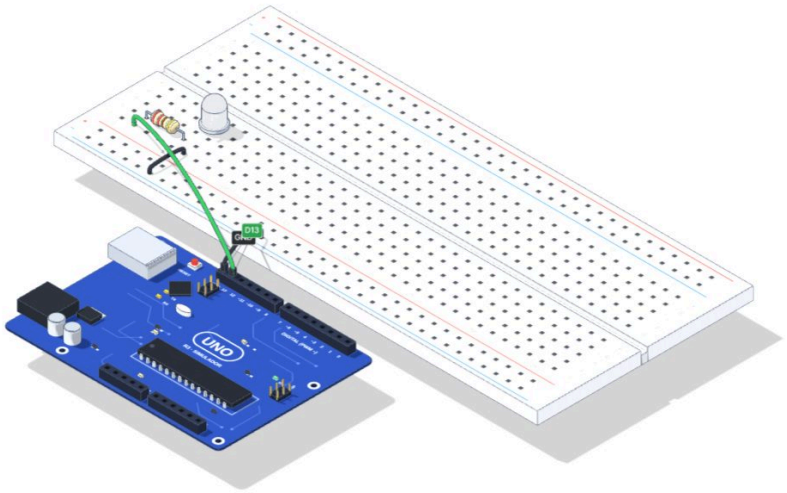
🏆 **Reto:** Crea un cuarto efecto en el que se encienden de dos en dos.

PROYECTO 12 · LUCES Y LED

Luz de emergencia estroboscópica

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

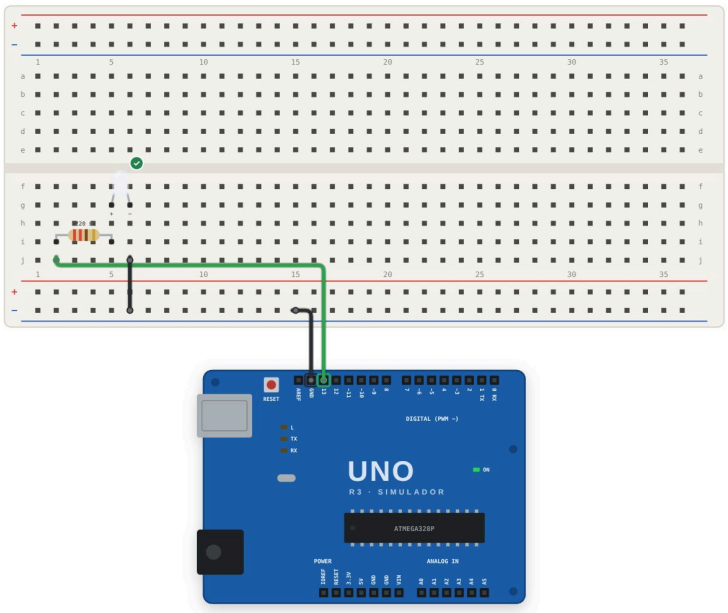
Un LED blanco da destellos muy rápidos, como las luces de emergencia.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Estrobo (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Estrobo (LED blanco)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
1  // Luz estroboscópica de emergencia
2  const int LUZ = 13;
3
4  void setup() {
5    pinMode(LUZ, OUTPUT);
6  }
7
8  void loop() {
9    for (int i = 0; i < 5; i++) {
10     digitalWrite(LUZ, HIGH);
11     delay(30);               // destello muy corto
12     digitalWrite(LUZ, LOW);
13     delay(70);
14   }
15   delay(700);                // pausa entre ráfagas
16 }
```

💡 **¿Cómo funciona?**

Los destellos de 30 ms son tan cortos que el ojo los ve como golpes de luz. Cinco destellos seguidos y una pausa forman una ráfaga.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

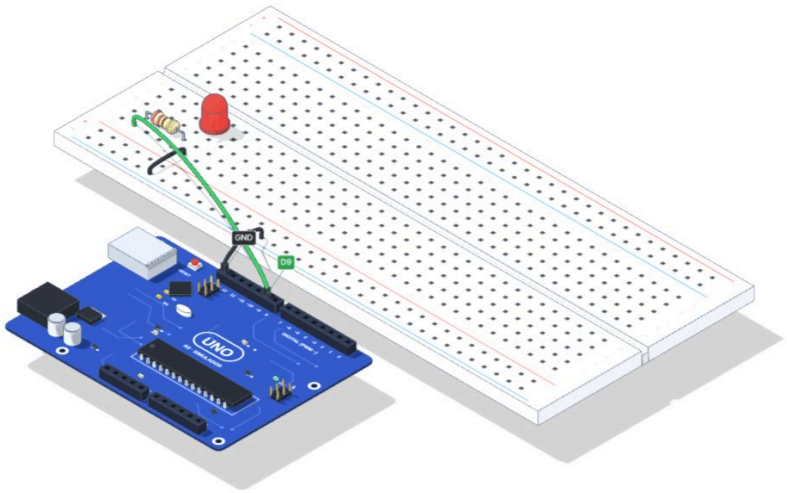
🏆 **Reto:** Cambia el número de destellos por ráfaga y observa cuál llama más la atención.

PROYECTO 13 · LUCES Y LED

Latido del corazón

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

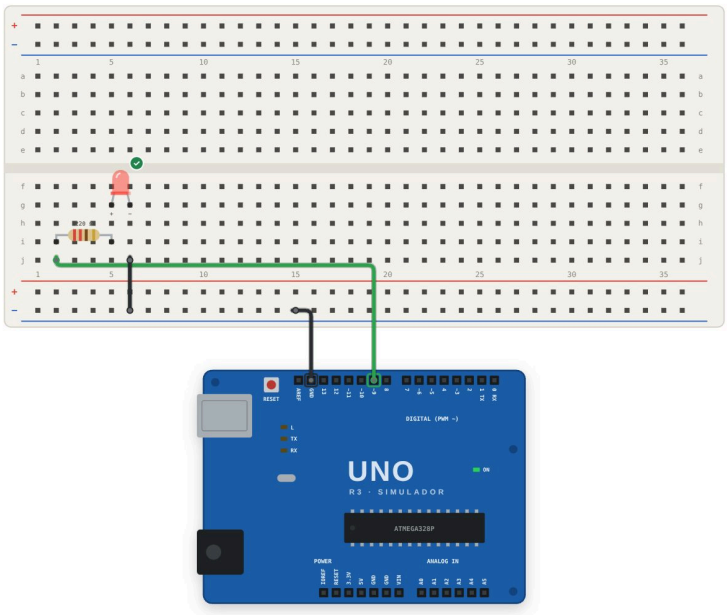
Un LED rojo late como un corazón: dos golpes rápidos y una pausa.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Corazón (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Corazón (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
// Latido del corazón con brillo suave
const int CORAZON = 9;        // pin PWM (~)

void golpe(int fuerza) {
  for (int b = 0; b <= fuerza; b += 15) { analogWrite(CORAZON, b); delay(5); }
  for (int b = fuerza; b >= 0; b -= 15) { analogWrite(CORAZON, b); delay(8); }
}

void setup() {
  pinMode(CORAZON, OUTPUT);
}

void loop() {
  golpe(255);   // primer golpe fuerte
  delay(80);
  golpe(150);   // segundo golpe más suave
  delay(700);   // pausa hasta el siguiente latido
}
```

💡 **¿Cómo funciona?**

La función golpe() sube y baja el brillo muy rápido. Un golpe fuerte seguido de uno suave imita el "pum-pum" del corazón.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

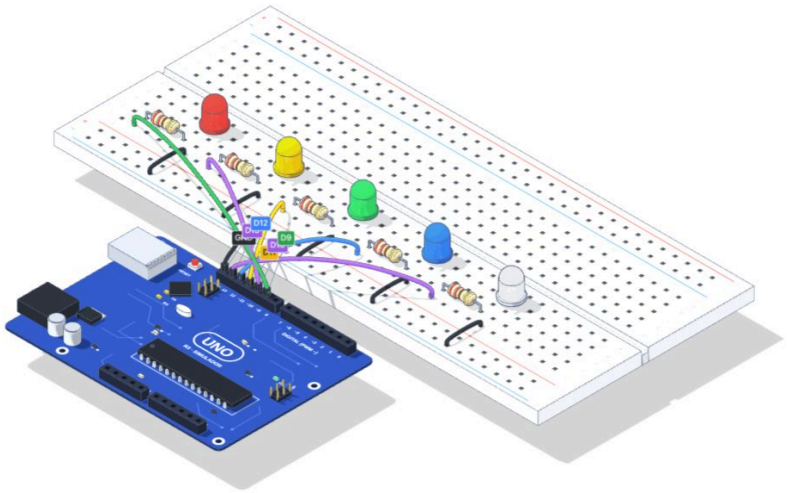
🏆 **Reto:** Acelera el latido como si el corazón estuviera haciendo ejercicio.

PROYECTO 14 · LUCES Y LED

Ruleta de luces

★★☆ Intermedio · Tiempo: 30 a 45 minutos

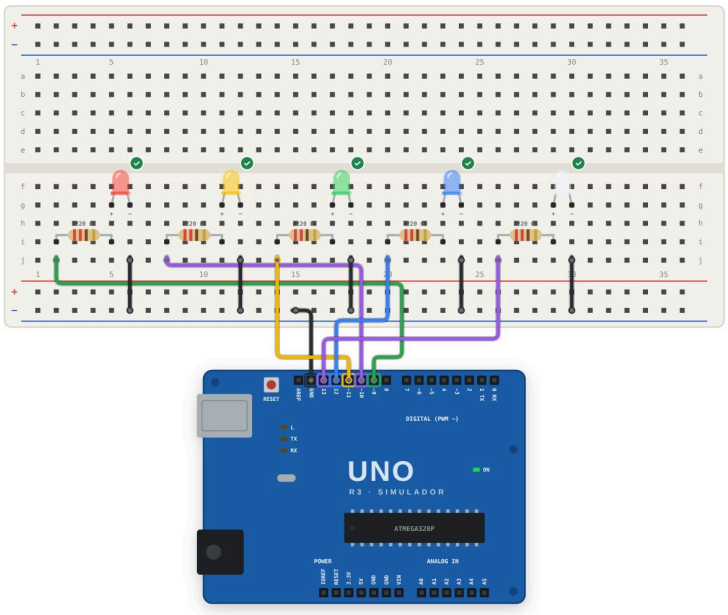
Cinco LED giran cada vez más lento hasta detenerse en uno al azar, como una ruleta.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Casilla 1 (LED)
- 1 × Casilla 2 (LED)
- 1 × Casilla 3 (LED)
- 1 × Casilla 4 (LED)
- 1 × Casilla 5 (LED)
- 5 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Casilla 1 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Casilla 2 (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

4 **Casilla 3 (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

5 **Casilla 4 (LED azul)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

6 **Casilla 5 (LED blanco)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

7 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 125 mA.
- Consumo estimado: **~125 mA** funcionando, con picos de **~125 mA**.

2 El programa

```
// Ruleta de luces
int casillas[] = {9, 10, 11, 12, 13};

void setup() {
  Serial.begin(9600);
  for (int i = 0; i < 5; i++) pinMode(casillas[i], OUTPUT);
}

void loop() {
  int vueltas = random(15, 30);      // cuántos saltos dará
  int espera = 40;
  int pos = 0;
  for (int k = 0; k < vueltas; k++) {
    digitalWrite(casillas[pos], LOW);
    pos = (pos + 1) % 5;             // pasa a la siguiente casilla
    digitalWrite(casillas[pos], HIGH);
    delay(espera);
    espera = espera + 12;            // cada vez más lento
  }
  Serial.print("Ganó la casilla ");
  Serial.println(pos + 1);
  delay(3000);
  digitalWrite(casillas[pos], LOW);
}
```

💡 **¿Cómo funciona?**

La variable pos guarda en qué casilla va la luz y el operador % hace que después de la 5 vuelva a la 1. Como la espera aumenta en cada salto, la ruleta se va frenando.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa y abre el monitor serie a 9600.

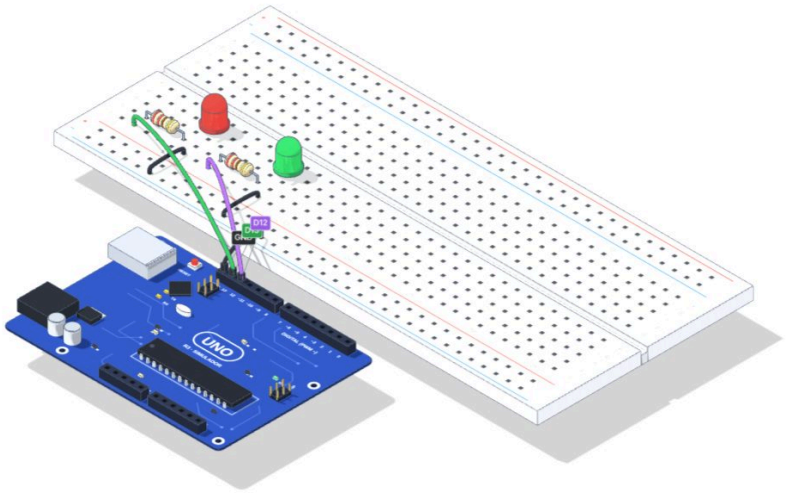
🏆 **Reto:** Agrega un botón para que la ruleta solo gire cuando lo presiones.

PROYECTO 15 · LUCES Y LED

Metrónomo de luz

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

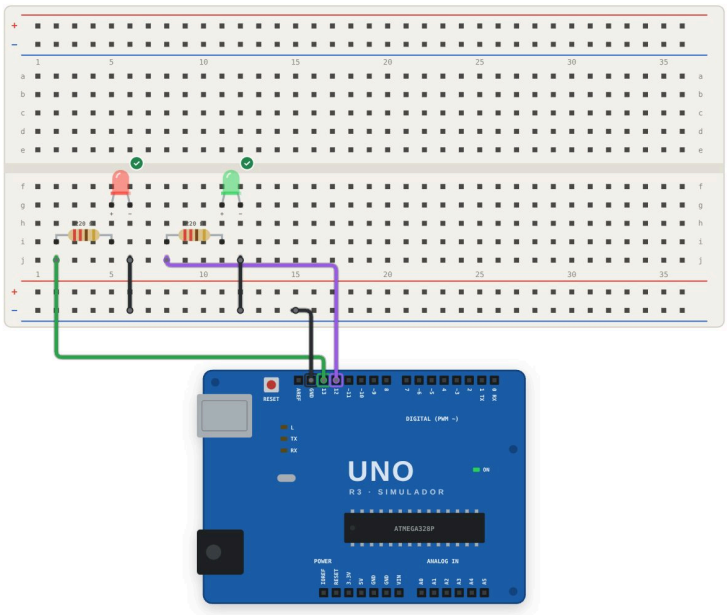
Un LED marca el ritmo de una canción: 4 tiempos, con el primero más largo.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Primer tiempo (LED)
- 1 × Otros tiempos (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Primer tiempo (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Otros tiempos (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Metrónomo de luz a 90 pulsos por minuto
const int FUERTE = 13;
const int SUAVE = 12;
const int BPM = 90;                 // pulsos por minuto

void setup() {
  pinMode(FUERTE, OUTPUT);
  pinMode(SUAVE, OUTPUT);
}

void loop() {
  int intervalo = 60000 / BPM;      // milisegundos entre pulsos
  for (int t = 1; t <= 4; t++) {
    int pin = SUAVE;
    if (t == 1) pin = FUERTE;       // el primer tiempo se marca en rojo
    digitalWrite(pin, HIGH);
    delay(100);
    digitalWrite(pin, LOW);
    delay(intervalo - 100);
  }
}
```

💡 **¿Cómo funciona?**

Un minuto tiene 60 000 milisegundos; dividiéndolos entre los pulsos por minuto se obtiene el tiempo entre pulsos. El primer tiempo de cada compás se marca con el LED rojo.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

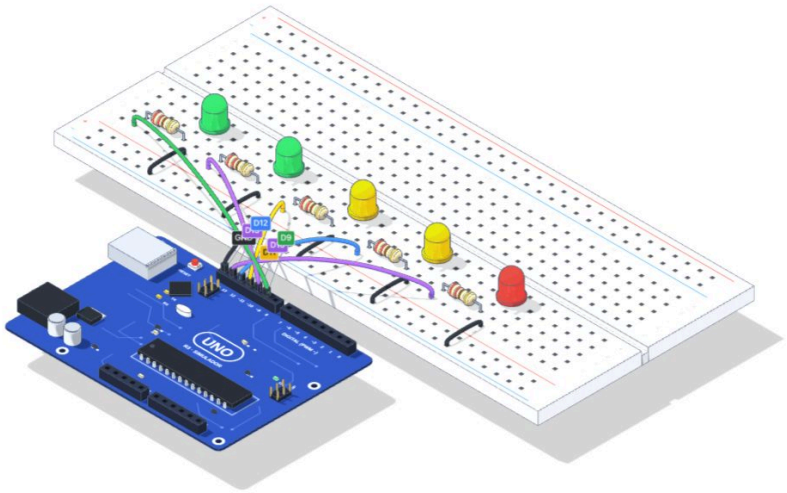
🏆 **Reto:** Agrega un zumbador para escuchar el ritmo además de verlo.

PROYECTO 16 · LUCES Y LED

Escalera de luces

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

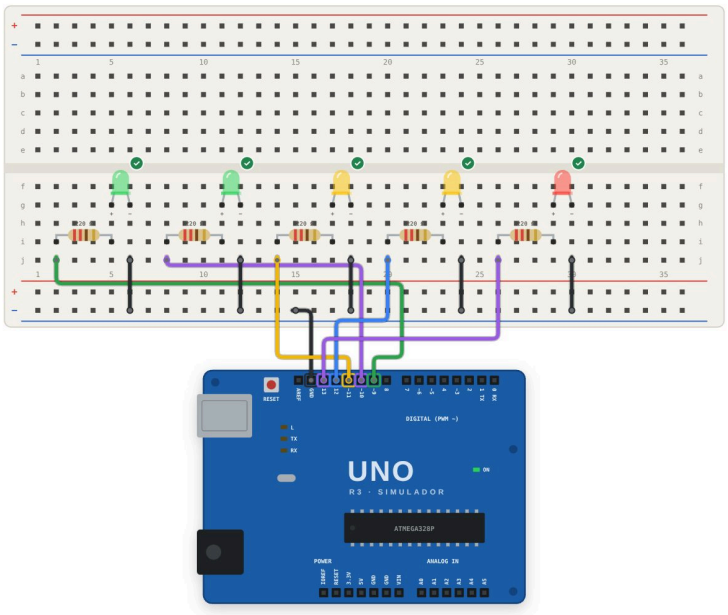
Cinco LED se encienden uno sobre otro hasta llenarse y luego se vacían.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Nivel 1 (LED)
- 1 × Nivel 2 (LED)
- 1 × Nivel 3 (LED)
- 1 × Nivel 4 (LED)
- 1 × Nivel 5 (LED)
- 5 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Nivel 1 (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Nivel 2 (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

4 **Nivel 3 (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

5 **Nivel 4 (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

6 **Nivel 5 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

7 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 125 mA.
- Consumo estimado: **~125 mA** funcionando, con picos de **~125 mA**.

2 El programa

```
// Escalera de luces
int niveles[] = {9, 10, 11, 12, 13};

void setup() {
  for (int i = 0; i < 5; i++) pinMode(niveles[i], OUTPUT);
}

void loop() {
  for (int i = 0; i < 5; i++) {      // se llena
    digitalWrite(niveles[i], HIGH);
    delay(250);
  }
  for (int i = 4; i >= 0; i--) {     // se vacía
    digitalWrite(niveles[i], LOW);
    delay(250);
  }
}
```

💡 **¿Cómo funciona?**

A diferencia del auto fantástico, aquí los LED no se apagan enseguida: se quedan encendidos hasta llenar la escalera, y luego se apagan desde arriba.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

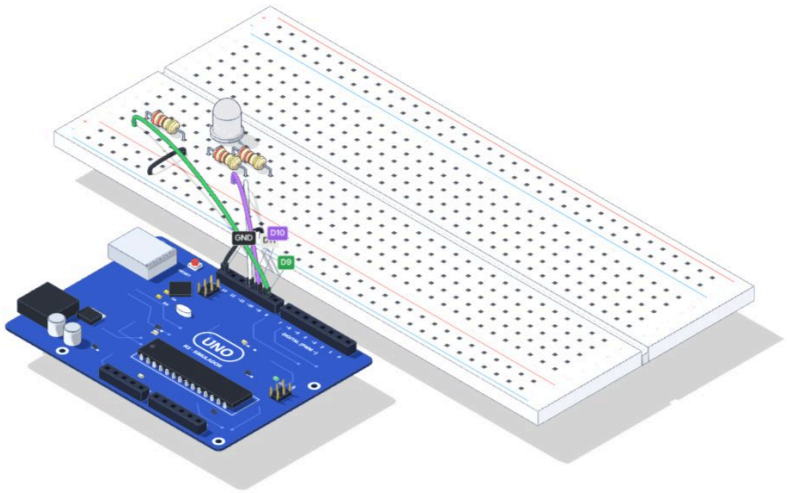
🏆 **Reto:** Haz que al llenarse los cinco parpadeen tres veces antes de vaciarse.

PROYECTO 17 · LUCES Y LED

Arcoíris con LED RGB

★★☆ Intermedio · Tiempo: 30 a 45 minutos

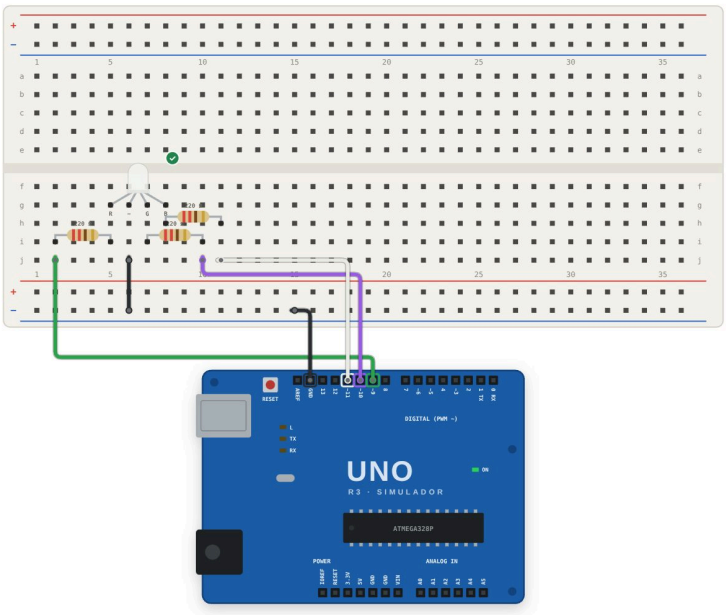
Un LED RGB recorre los colores del arcoíris mezclando rojo, verde y azul.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × LED RGB
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 LED RGB (LED RGB)

- Pata R (rojo) → resistencia de **220 Ω** → pin **~D9**
- Pata – (común, la más larga) → riel **–** (GND)
- Pata G (verde) → resistencia de **220 Ω** → pin **~D10**
- Pata B (azul) → resistencia de **220 Ω** → pin **~D11**

3 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Arcoíris con LED RGB (cátodo común)
const int R = 9, G = 10, B = 11;    // pines PWM (~)

void color(int r, int g, int b) {
  analogWrite(R, r);
  analogWrite(G, g);
  analogWrite(B, b);
  delay(700);
}

void setup() {
  pinMode(R, OUTPUT); pinMode(G, OUTPUT); pinMode(B, OUTPUT);
}

void loop() {
  color(255, 0, 0);     // rojo
  color(255, 80, 0);    // naranja
  color(255, 200, 0);   // amarillo
  color(0, 255, 0);     // verde
  color(0, 0, 255);     // azul
  color(75, 0, 130);    // índigo
  color(160, 0, 255);   // violeta
}
```

💡 ¿Cómo funciona?

Un LED RGB tiene tres LED dentro: rojo, verde y azul. Variando el brillo de cada uno con analogWrite() se mezclan para formar cualquier color.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

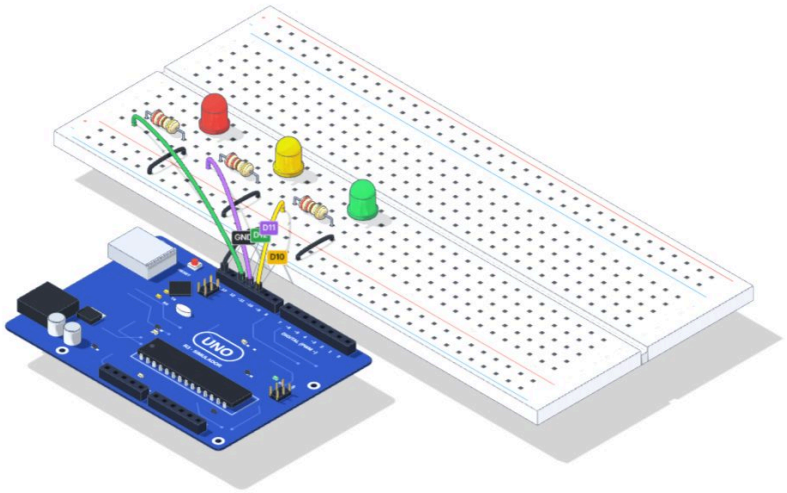
🏆 **Reto:** Agrega el color blanco (los tres al máximo) y el rosado.

PROYECTO 18 · LUCES Y LED

Semáforo con cuenta en el monitor

★★☆ Intermedio · Tiempo: 30 a 45 minutos

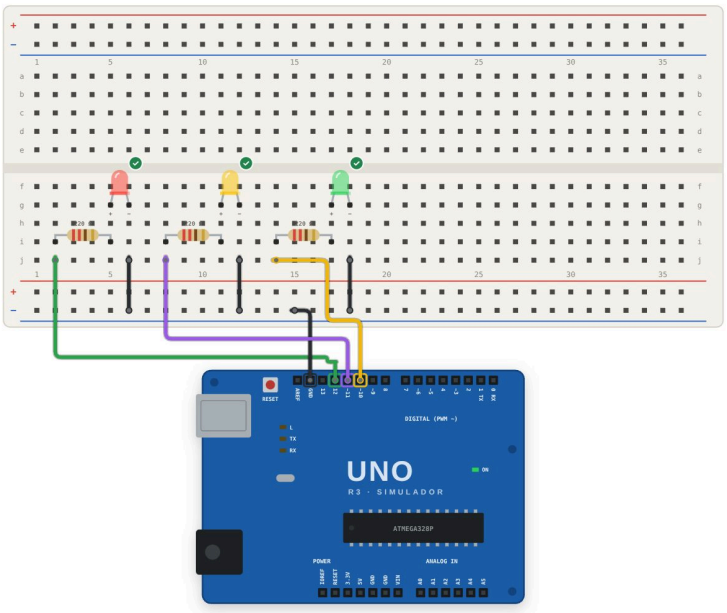
Un semáforo que además muestra en el monitor serie cuántos segundos faltan para cambiar.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Rojo (LED)
- 1 × Amarillo (LED)
- 1 × Verde (LED)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Rojo (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Amarillo (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

4 **Verde (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Semáforo con cuenta regresiva en el monitor serie
const int ROJO = 12, AMARILLO = 11, VERDE = 10;

void fase(int pin, String nombre, int segundos) {
  digitalWrite(pin, HIGH);
  for (int s = segundos; s > 0; s--) {
    Serial.print(nombre);
    Serial.print(": faltan ");
    Serial.print(s);
    Serial.println(" s");
    delay(1000);
  }
  digitalWrite(pin, LOW);
}

void setup() {
  Serial.begin(9600);
  pinMode(ROJO, OUTPUT); pinMode(AMARILLO, OUTPUT); pinMode(VERDE, OUTPUT);
}

void loop() {
  fase(VERDE, "Verde", 5);
  fase(AMARILLO, "Amarillo", 2);
  fase(ROJO, "Rojo", 5);
}
```

💡 **¿Cómo funciona?**

La función fase() enciende una luz y cuenta los segundos hacia atrás, escribiendo cada número en el monitor serie. Así se ve cómo un programa puede mostrar información al usuario.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa y abre el monitor serie a 9600.

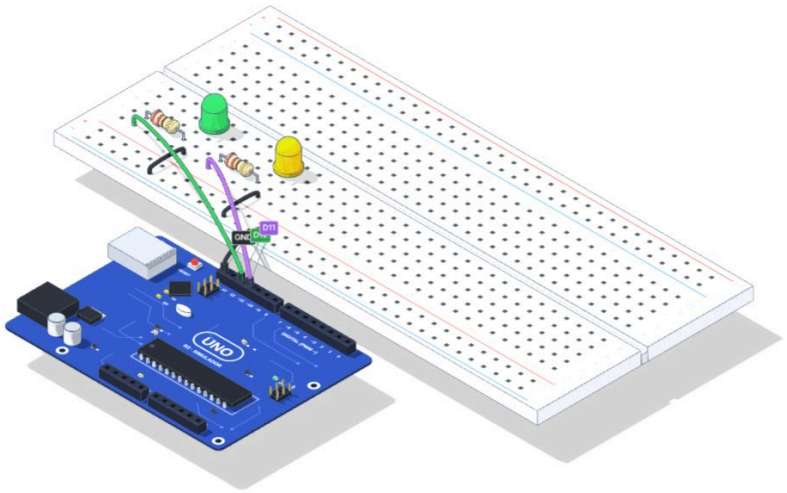
🏆 **Reto:** Muestra también cuántos ciclos completos lleva el semáforo.

PROYECTO 19 · LUCES Y LED

Dos LED alternados con velocidad creciente

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

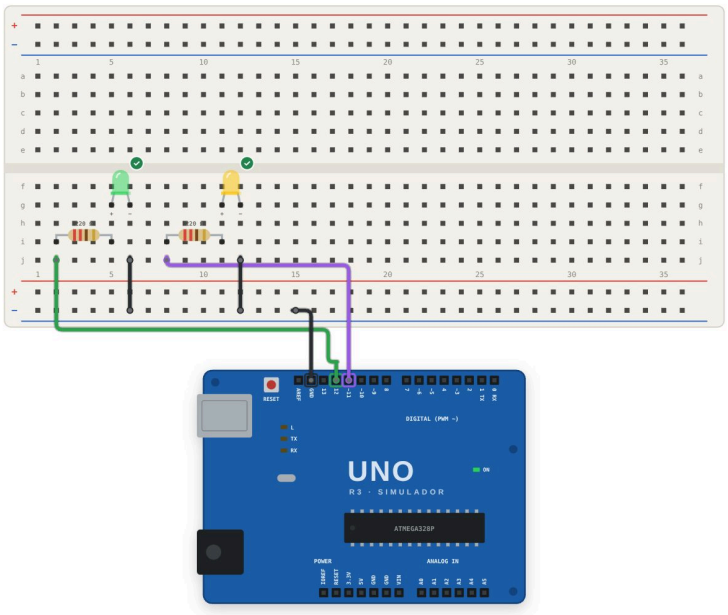
Dos LED se alternan cada vez más rápido y vuelven a empezar.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Izquierda (LED)
- 1 × Derecha (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Izquierda (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 **Derecha (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Dos LED que se alternan cada vez más rápido
const int IZQ = 12;
const int DER = 11;

void setup() {
  pinMode(IZQ, OUTPUT);
  pinMode(DER, OUTPUT);
}

void loop() {
  for (int espera = 500; espera >= 50; espera -= 50) {
    digitalWrite(IZQ, HIGH); digitalWrite(DER, LOW);
    delay(espera);
    digitalWrite(IZQ, LOW);  digitalWrite(DER, HIGH);
    delay(espera);
  }
}
```

💡 ¿Cómo funciona?

La variable espera empieza en 500 ms y baja de 50 en 50, así cada cambio es más rápido que el anterior. Cuando llega a 50 el ciclo for termina y el loop() empieza de nuevo.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

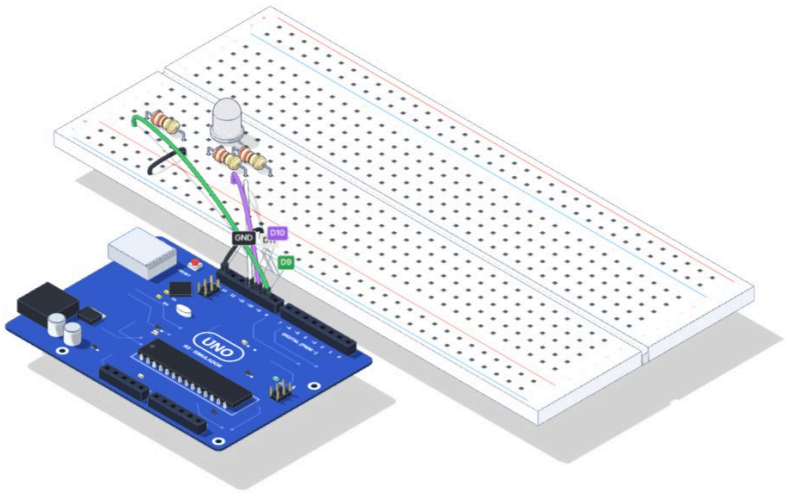
🏆 **Reto:** Haz que después de acelerar también frene poco a poco.

PROYECTO 20 · LUCES Y LED

Vela de cumpleaños RGB

★★☆ Intermedio · Tiempo: 30 a 45 minutos

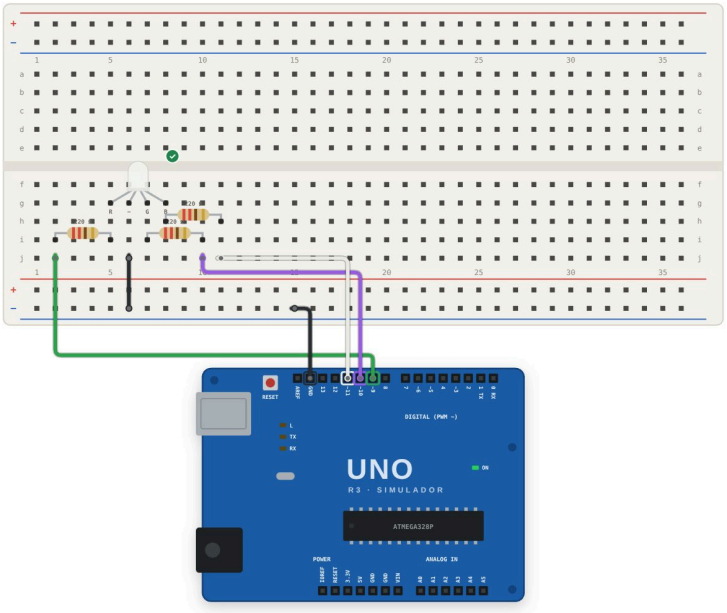
Un LED RGB cambia de color al azar como una vela de fiesta.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Vela de colores (LED RGB)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Vela de colores (LED RGB)

- Pata R (rojo) → resistencia de **220 Ω** → pin **~D9**
- Pata – (común, la más larga) → riel **–** (GND)
- Pata G (verde) → resistencia de **220 Ω** → pin **~D10**
- Pata B (azul) → resistencia de **220 Ω** → pin **~D11**

3 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Vela de fiesta con colores al azar
const int R = 9, G = 10, B = 11;

void setup() {
  pinMode(R, OUTPUT); pinMode(G, OUTPUT); pinMode(B, OUTPUT);
}

void loop() {
  analogWrite(R, random(256));
  analogWrite(G, random(256));
  analogWrite(B, random(256));
  delay(random(100, 400));
}
```

💡 ¿Cómo funciona?

Cada vez que se repite el loop() se elige un brillo al azar para el rojo, el verde y el azul. La mezcla da un color nuevo cada vez.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

🏆 **Reto:** Haz que solo aparezcan colores cálidos (rojo y naranja).

CAPÍTULO 2

Botones e interruptores

El Arduino empieza a escuchar: botones, interruptores y la resistencia pull-up interna para leer entradas digitales.

EN ESTE CAPÍTULO · 18 PROYECTOS

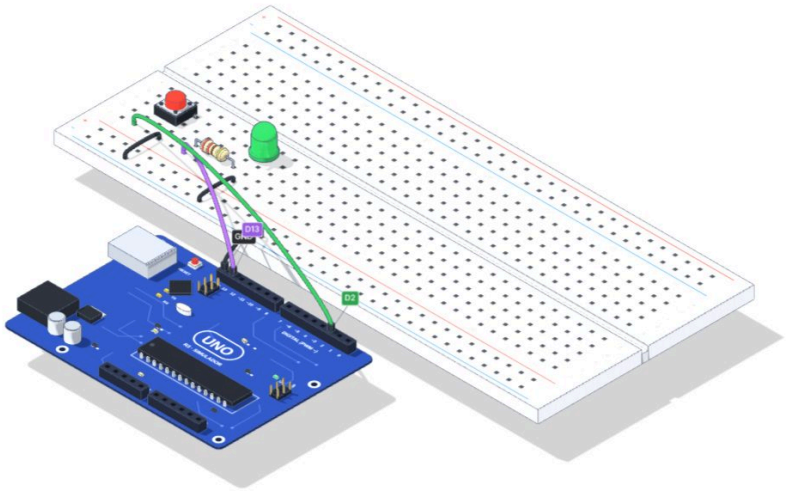
21 LED con botón ★☆☆
22 Interruptor de luz con un toque ★☆☆
23 Contador de pulsaciones ★☆☆
24 Timbre sencillo ★☆☆
25 Dimmer con dos botones ★★☆
26 ¿Quién presiona primero? ★★☆
27 Dado electrónico de LED ★★☆
28 Botón de pánico ★☆☆
29 Selector de modos de luz ★★☆
30 Juego de reflejos con dos LED ★★☆
31 Cerradura de secuencia de botones ★★☆
32 Lámpara con temporizador ★☆☆
33 Piano de cuatro teclas ★★☆
34 Cuenta regresiva con botón ★☆☆
35 Pulsación larga y corta ★★☆
36 Alarma de puerta con interruptor ★☆☆
37 Marcador de dos equipos ★★☆
38 Semáforo peatonal con interruptor nocturno ★★☆

PROYECTO 21 · BOTONES E INTERRUPTORES

LED con botón

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

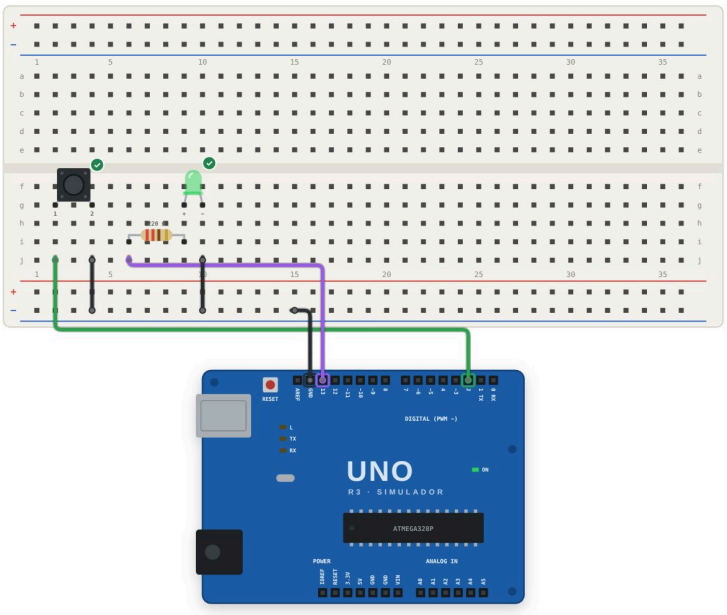
El LED se enciende solo mientras mantienes presionado el botón.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Botón
- 1 × LED verde (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Botón (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **LED verde (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
1  // LED con botón
2  const int BOTON = 2;
3  const int LED = 13;
4
5  void setup() {
6    pinMode(BOTON, INPUT_PULLUP);   // resistencia interna: suelto = HIGH
7    pinMode(LED, OUTPUT);
8  }
9
10 void loop() {
11   if (digitalRead(BOTON) == LOW) {  // presionado = LOW
12     digitalWrite(LED, HIGH);
13   } else {
14     digitalWrite(LED, LOW);
15   }
16 }
```

💡 ¿Cómo funciona?

Con INPUT_PULLUP el pin lee HIGH cuando el botón está suelto; al presionarlo, el botón une el pin con GND y lee LOW. El if decide si el LED se enciende.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

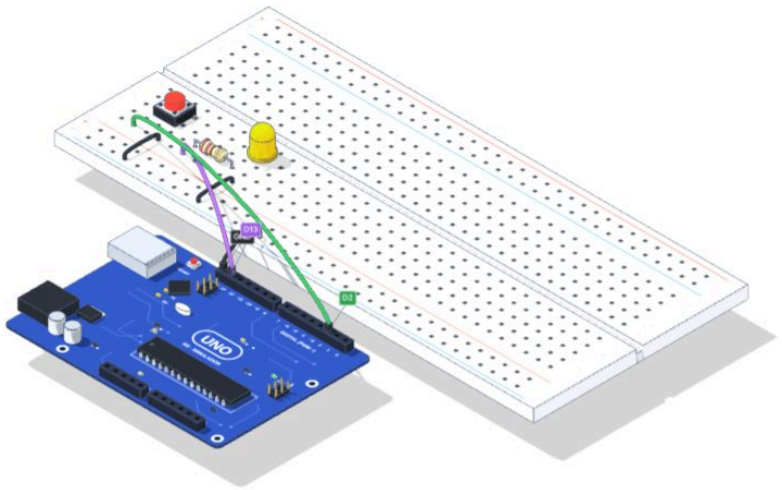
🏆 **Reto:** Invierte el funcionamiento: que el LED esté encendido y se apague al presionar.

PROYECTO 22 · BOTONES E INTERRUPTORES

Interruptor de luz con un toque

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

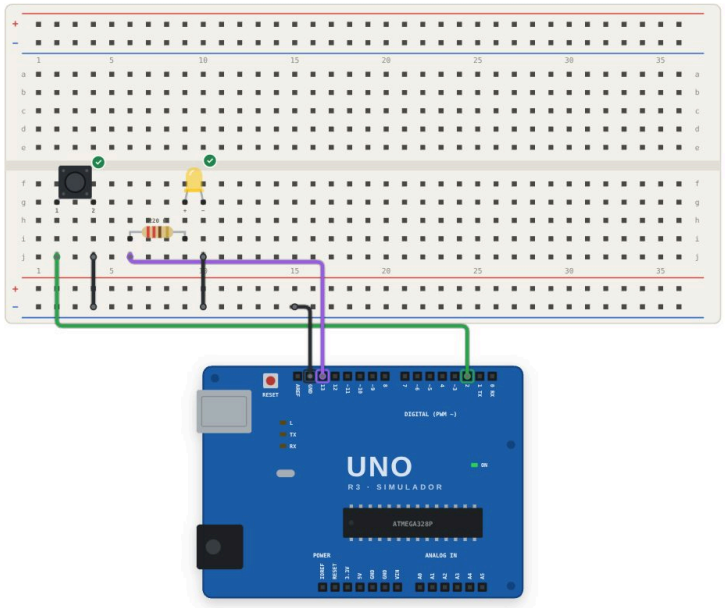
Cada vez que presionas el botón la lámpara cambia: si estaba apagada se enciende, y al revés.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Botón de la lámpara (Botón)
- 1 × Lámpara (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Botón de la lámpara (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Lámpara (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
// Interruptor de un toque (encender / apagar)
const int BOTON = 2;
const int LAMPARA = 13;
bool encendida = false;
int anterior = HIGH;

void setup() {
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(LAMPARA, OUTPUT);
}

void loop() {
  int ahora = digitalRead(BOTON);
  if (anterior == HIGH && ahora == LOW) {   // el botón se acaba de presionar
    encendida = !encendida;                 // cambia el estado
    digitalWrite(LAMPARA, encendida);
    delay(50);                              // evita rebotes
  }
  anterior = ahora;
}
```

💡 ¿Cómo funciona?

El programa recuerda cómo estaba el botón la vez anterior. Solo cuando pasa de suelto (HIGH) a presionado (LOW) cambia el estado de la lámpara; así un toque largo no la hace parpadear.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

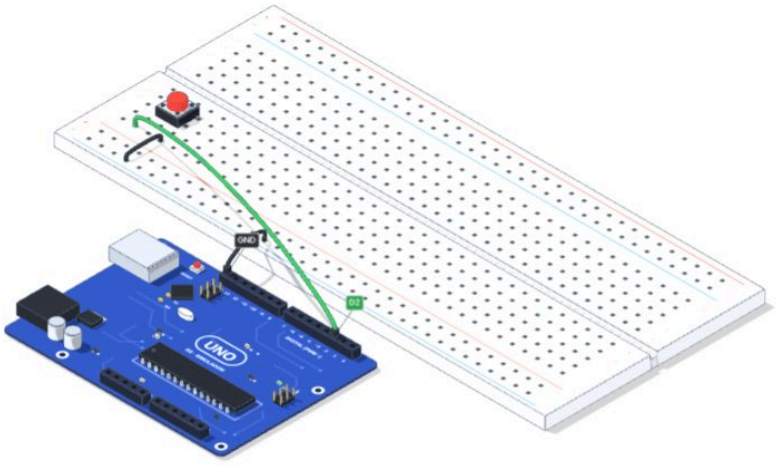
🏆 **Reto:** Agrega un segundo LED que se encienda cuando la lámpara esté apagada.

PROYECTO 23 · BOTONES E INTERRUPTORES

Contador de pulsaciones

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

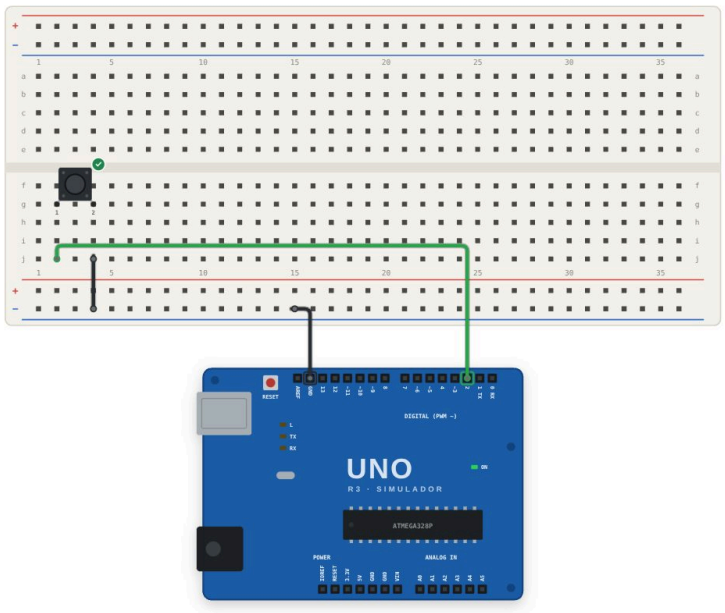
Cuenta cuántas veces presionas el botón y lo muestra en el monitor serie.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Botón
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Botón (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 50 mA.
- Consumo estimado: **~50 mA** funcionando, con picos de **~50 mA**.

2 El programa

```
// Contador de pulsaciones
const int BOTON = 2;
int cuenta = 0;
int anterior = HIGH;

void setup() {
  Serial.begin(9600);
  pinMode(BOTON, INPUT_PULLUP);
  Serial.println("Presiona el botón");
}

void loop() {
  int ahora = digitalRead(BOTON);
  if (anterior == HIGH && ahora == LOW) {
    cuenta++;
    Serial.print("Pulsaciones: ");
    Serial.println(cuenta);
    delay(50);
  }
  anterior = ahora;
}
```

💡 ¿Cómo funciona?

La variable cuenta aumenta en 1 cada vez que el botón pasa de suelto a presionado. Serial.println() envía el número al computador por el cable USB.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

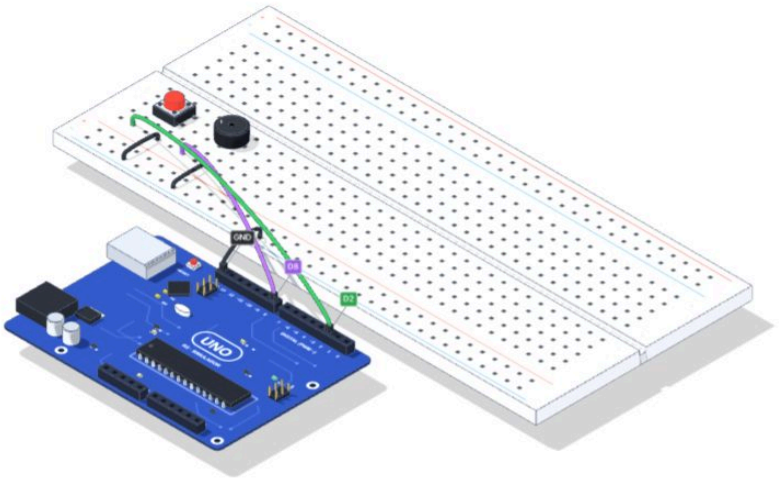
🏆 **Reto:** Haz que al llegar a 10 pulsaciones se encienda un LED de premio.

PROYECTO 24 · BOTONES E INTERRUPTORES

Timbre sencillo

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

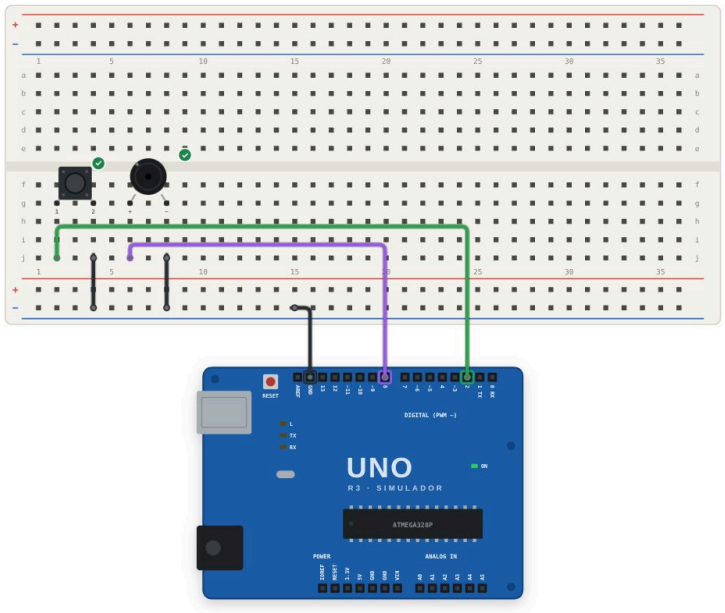
Al presionar el botón suena un "ding-dong" en el zumbador.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Botón del timbre (Botón)
- 1 × Zumbador
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Botón del timbre (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Zumbador (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Timbre ding-dong
const int BOTON = 2;
const int ZUMBADOR = 8;

void setup() {
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(ZUMBADOR, OUTPUT);
}

void loop() {
  if (digitalRead(BOTON) == LOW) {
    tone(ZUMBADOR, 880, 300);   // "ding": nota alta
    delay(350);
    tone(ZUMBADOR, 660, 500);   // "dong": nota más baja
    delay(800);
  }
}
```

💡 ¿Cómo funciona?

tone(pin, frecuencia, duración) hace sonar el zumbador. 880 Hz es una nota alta y 660 Hz una más baja; seguidas forman el sonido clásico del timbre.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

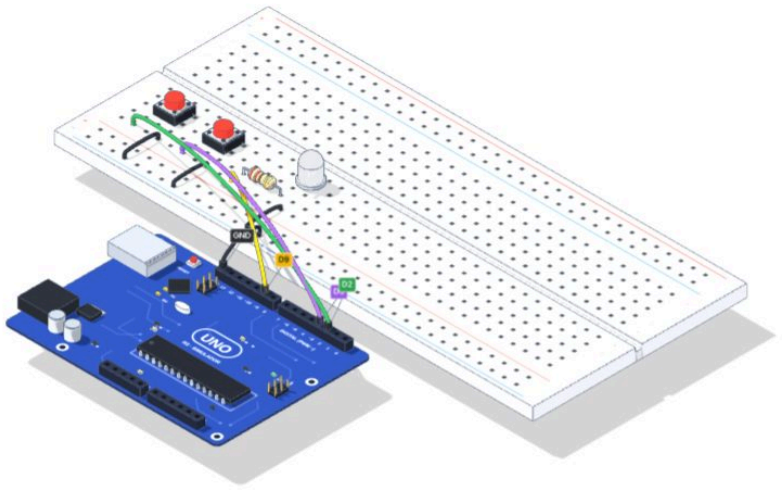
🏆 **Reto:** Cambia las notas para inventar el sonido de tu propio timbre.

PROYECTO 25 · BOTONES E INTERRUPTORES

Dimmer con dos botones

★★☆ Intermedio · Tiempo: 30 a 45 minutos

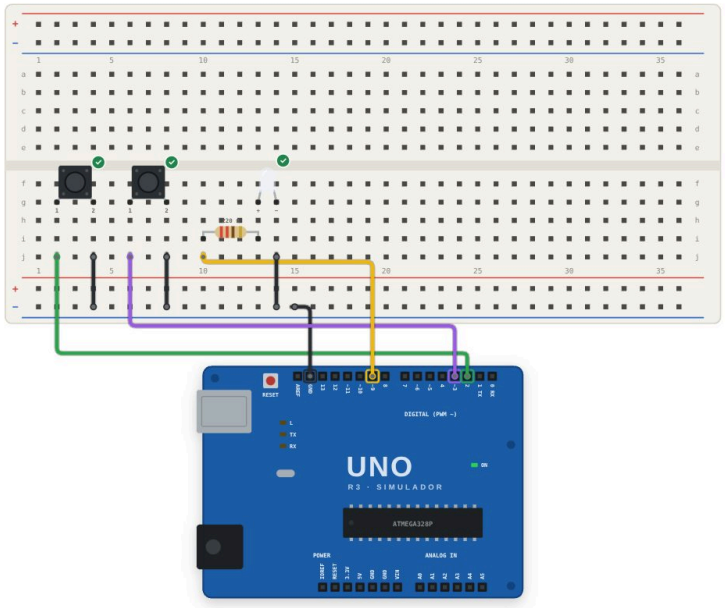
Un botón sube el brillo del LED y el otro lo baja.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Subir (Botón)
- 1 × Bajar (Botón)
- 1 × Lámpara (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Subir (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Bajar (Botón)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Lámpara (LED blanco)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
// Dimmer con dos botones
const int SUBIR = 2;
const int BAJAR = 3;
const int LAMPARA = 9;      // pin PWM (~)
int brillo = 100;

void setup() {
  Serial.begin(9600);
  pinMode(SUBIR, INPUT_PULLUP);
  pinMode(BAJAR, INPUT_PULLUP);
  pinMode(LAMPARA, OUTPUT);
}

void loop() {
  if (digitalRead(SUBIR) == LOW) brillo = brillo + 5;
  if (digitalRead(BAJAR) == LOW) brillo = brillo - 5;
  brillo = constrain(brillo, 0, 255);   // no salirse de 0 a 255
  analogWrite(LAMPARA, brillo);
  Serial.print("Brillo: ");
  Serial.println(brillo);
  delay(60);
}
```

💡 ¿Cómo funciona?

Mientras un botón está presionado el brillo cambia de 5 en 5. constrain() evita que el valor se salga del rango que acepta analogWrite().

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

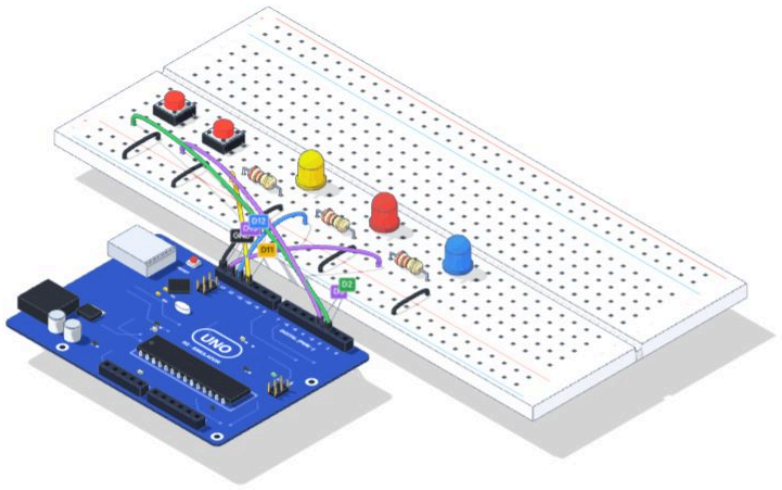
🏆 **Reto:** Muestra el brillo en porcentaje (0 % a 100 %) en el monitor serie.

PROYECTO 26 · BOTONES E INTERRUPTORES

¿Quién presiona primero?

★★☆ Intermedio · Tiempo: 30 a 45 minutos

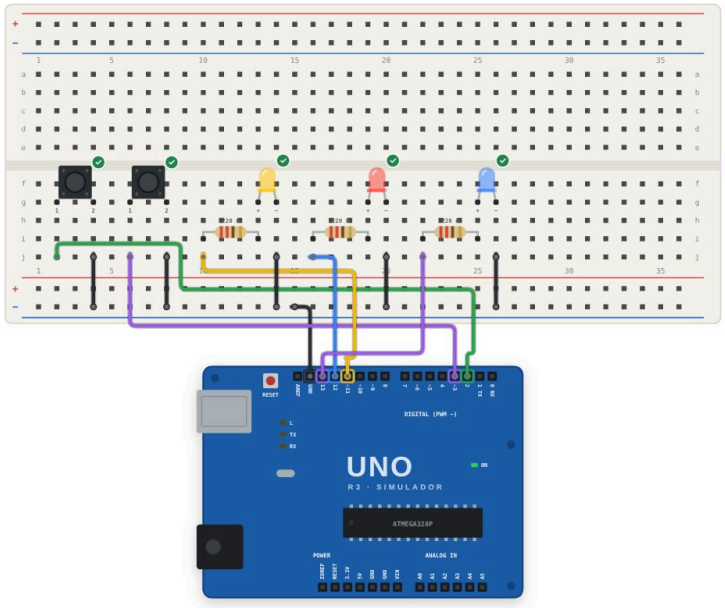
Dos jugadores esperan la luz amarilla: el primero en presionar su botón gana.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Jugador 1 (Botón)
- 1 × Jugador 2 (Botón)
- 1 × Señal (LED)
- 1 × Gana 1 (LED)
- 1 × Gana 2 (LED)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Jugador 1 (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Jugador 2 (Botón)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Señal (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 **Gana 1 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 **Gana 2 (LED azul)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// ¿Quién presiona primero?
const int J1 = 2, J2 = 3;
const int SENAL = 11, GANA1 = 12, GANA2 = 13;

void setup() {
  Serial.begin(9600);
  pinMode(J1, INPUT_PULLUP); pinMode(J2, INPUT_PULLUP);
  pinMode(SENAL, OUTPUT); pinMode(GANA1, OUTPUT); pinMode(GANA2, OUTPUT);
}

void loop() {
  digitalWrite(GANA1, LOW); digitalWrite(GANA2, LOW);
  Serial.println("Prepárense...");
  delay(random(2000, 5000));          // espera al azar
  digitalWrite(SENAL, HIGH);           // ¡ya!
  while (true) {
    if (digitalRead(J1) == LOW) { digitalWrite(GANA1, HIGH); Serial.println("¡Gana el jugador 1!"); break; }
    if (digitalRead(J2) == LOW) { digitalWrite(GANA2, HIGH); Serial.println("¡Gana el jugador 2!"); break; }
  }
  digitalWrite(SENAL, LOW);
  delay(3000);
}
```

💡 **¿Cómo funciona?**

Después de una espera al azar se enciende la señal. El ciclo while(true) revisa los dos botones una y otra vez; el primero que detecte presionado gana y break termina la espera.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

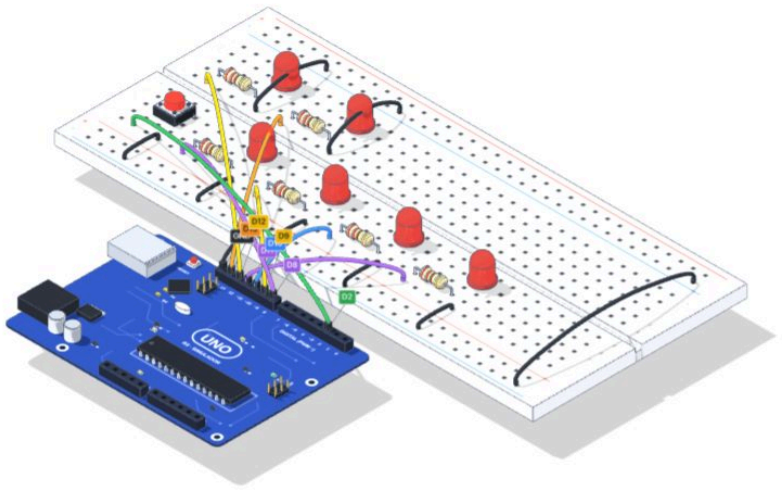
🏆 **Reto:** Castiga al que presione antes de que se encienda la señal.

PROYECTO 27 · BOTONES E INTERRUPTORES

Dado electrónico de LED

★★☆ Intermedio · Tiempo: 30 a 45 minutos

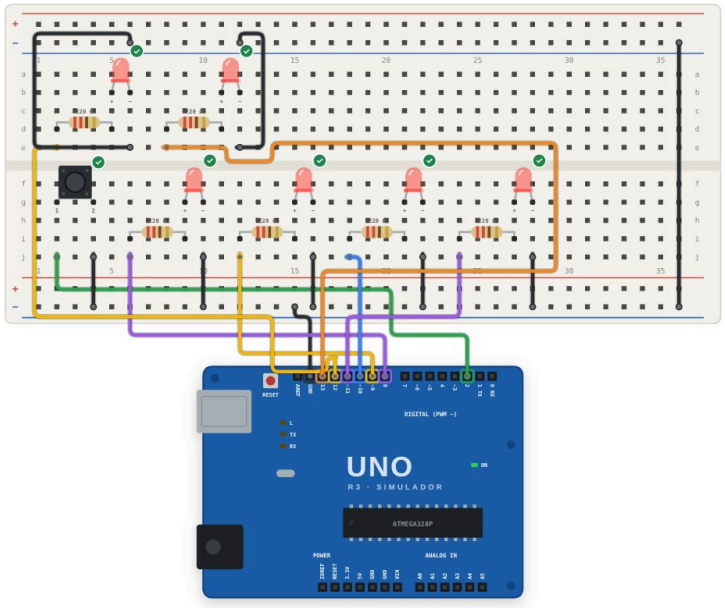
Al presionar el botón, seis LED muestran un número al azar del 1 al 6.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lanzar (Botón)
- 1 × Punto 1 (LED)
- 1 × Punto 2 (LED)
- 1 × Punto 3 (LED)
- 1 × Punto 4 (LED)
- 1 × Punto 5 (LED)
- 1 × Punto 6 (LED)
- 6 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lanzar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Punto 1 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D8**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Punto 2 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **Punto 3 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 **Punto 4 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 **Punto 5 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

8 **Punto 6 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

9 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 140 mA.
- Consumo estimado: **~140 mA** funcionando, con picos de **~140 mA**.

2 El programa

```
// Dado electrónico: enciende tantos LED como el número
const int BOTON = 2;
int puntos[] = {8, 9, 10, 11, 12, 13};

void mostrar(int n) {
  for (int i = 0; i < 6; i++) {
    if (i < n) digitalWrite(puntos[i], HIGH);
    else digitalWrite(puntos[i], LOW);
  }
}

void setup() {
  Serial.begin(9600);
  pinMode(BOTON, INPUT_PULLUP);
  for (int i = 0; i < 6; i++) pinMode(puntos[i], OUTPUT);
}

void loop() {
  if (digitalRead(BOTON) == LOW) {
```

```
    for (int k = 0; k < 10; k++) { mostrar(random(1, 7)); delay(60); }   // animación
    int dado = random(1, 7);
    mostrar(dado);
    Serial.print("Salió: ");
    Serial.println(dado);
    delay(500);
  }
}
```

💡 ¿Cómo funciona?

random(1, 7) da un número entre 1 y 6. Antes de mostrar el resultado se hace una animación rápida de números al azar, como si el dado rodara.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

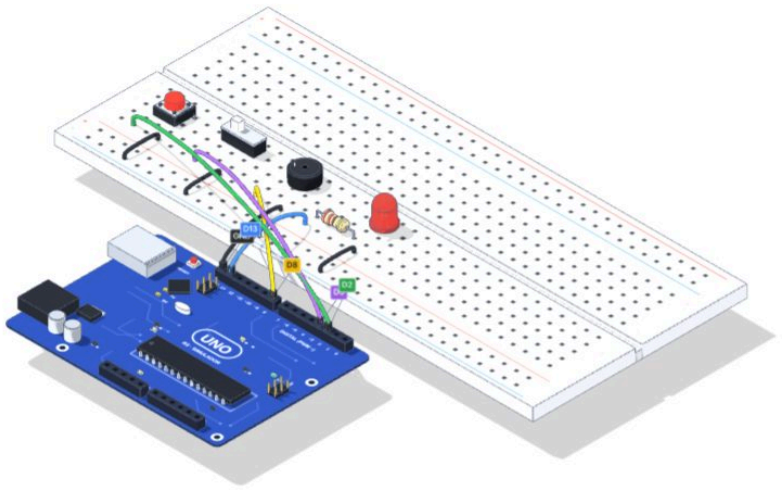
🏆 **Reto:** Usa dos botones para lanzar dos dados y sumar los resultados.

PROYECTO 28 · BOTONES E INTERRUPTORES

Botón de pánico

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

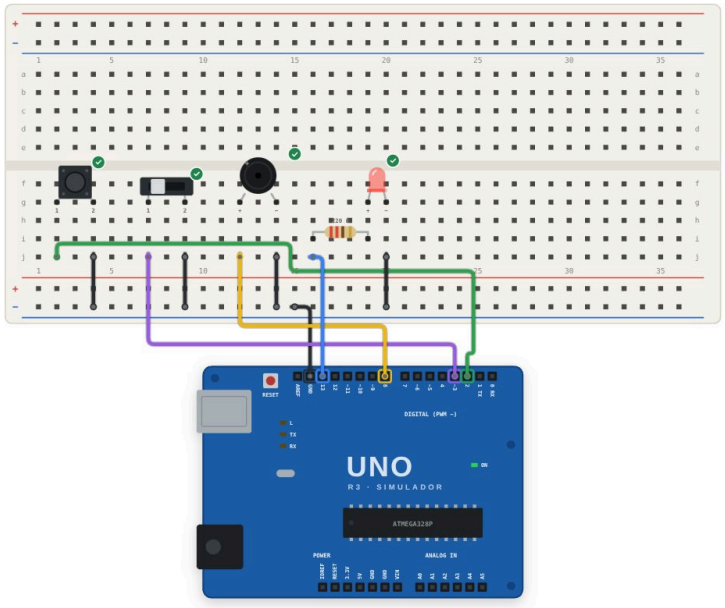
Al presionar el botón se activa una alarma de luz y sonido que solo se apaga con el interruptor.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Botón de pánico (Botón)
- 1 × Apagar alarma (Interruptor)
- 1 × Sirena (Zumbador)
- 1 × Luz de alerta (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Botón de pánico (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Apagar alarma (Interruptor)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**.

4 **Sirena (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 **Luz de alerta (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Botón de pánico
const int PANICO = 2;
const int APAGAR = 3;
const int SIRENA = 8;
const int LUZ = 13;
bool alarma = false;

void setup() {
  pinMode(PANICO, INPUT_PULLUP);
  pinMode(APAGAR, INPUT_PULLUP);
  pinMode(SIRENA, OUTPUT);
  pinMode(LUZ, OUTPUT);
}

void loop() {
  if (digitalRead(PANICO) == LOW) alarma = true;    // se activa
  if (digitalRead(APAGAR) == LOW) alarma = false;   // interruptor activado: se apaga
  if (alarma) {
    digitalWrite(LUZ, HIGH); tone(SIRENA, 1200); delay(200);
    digitalWrite(LUZ, LOW);  tone(SIRENA, 800);  delay(200);
  } else {
    digitalWrite(LUZ, LOW);
    noTone(SIRENA);
  }
}
```

💡 **¿Cómo funciona?**

La variable alarma recuerda que el botón se presionó aunque ya lo hayas soltado. Solo el interruptor la vuelve a false. Mientras está activa, la sirena alterna dos tonos.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).
- Cambia la posición del interruptor en su tarjeta del panel «Componentes».

Con piezas reales: sube el programa.

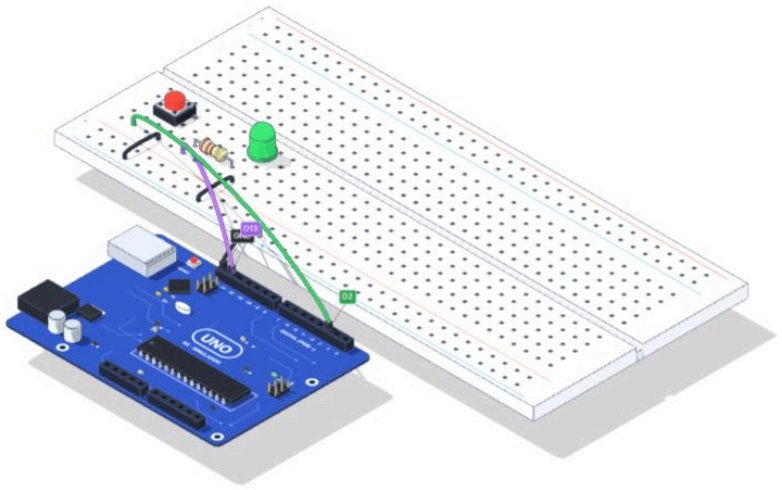
🏆 **Reto:** Haz que la alarma también se apague sola después de 30 segundos.

PROYECTO 29 · BOTONES E INTERRUPTORES

Selector de modos de luz

★★☆ Intermedio · Tiempo: 30 a 45 minutos

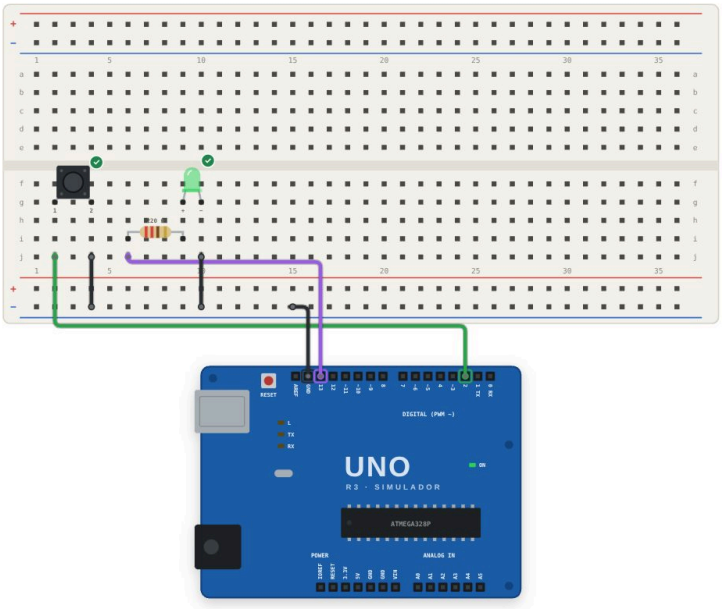
Cada toque del botón cambia el modo del LED: apagado, encendido, parpadeo lento y parpadeo rápido.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Modo (Botón)
- 1 × Luz (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Modo (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 Luz (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
// Selector de modos con un botón
const int BOTON = 2;
const int LUZ = 13;
int modo = 0;          // 0 apagado, 1 encendido, 2 lento, 3 rápido
int anterior = HIGH;
unsigned long ultimo = 0;
bool estado = false;

void setup() {
  Serial.begin(9600);
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(LUZ, OUTPUT);
}

void loop() {
  int ahora = digitalRead(BOTON);
  if (anterior == HIGH && ahora == LOW) {
    modo = (modo + 1) % 4;
    Serial.print("Modo ");
    Serial.println(modo);
    delay(50);
  }
  anterior = ahora;
  int periodo = 0;
  if (modo == 2) periodo = 600;
  if (modo == 3) periodo = 150;
  if (modo == 0) digitalWrite(LUZ, LOW);
  if (modo == 1) digitalWrite(LUZ, HIGH);
  if (periodo > 0 && millis() - ultimo > periodo) {   // parpadeo sin delay
    estado = !estado;
    digitalWrite(LUZ, estado);
    ultimo = millis();
  }
}
```

💡 **¿Cómo funciona?**

millis() cuenta los milisegundos desde que arrancó el Arduino. Comparándolo con la última vez que cambió el LED se puede parpadear sin delay(), y así el botón responde siempre al instante.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

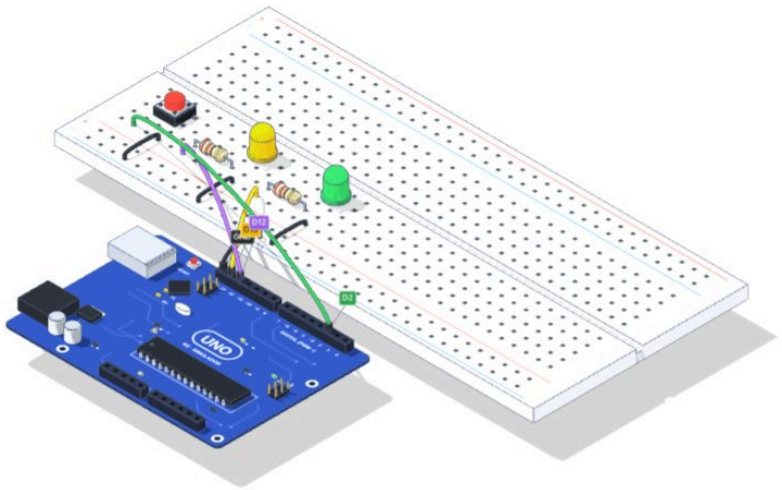
🏆 **Reto:** Agrega un quinto modo: latido del corazón.

PROYECTO 30 · BOTONES E INTERRUPTORES

Juego de reflejos con dos LED

★★☆ Intermedio · Tiempo: 30 a 45 minutos

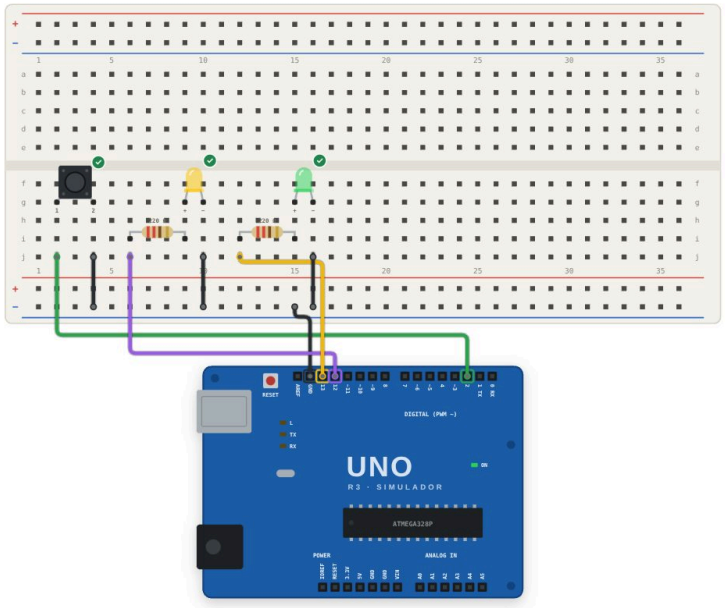
Mide en milisegundos qué tan rápido presionas el botón cuando se enciende el LED.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Botón
- 1 × Prepárate (LED)
- 1 × ¡Ya! (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Botón (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Prepárate (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **¡Ya! (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Tiempo de reacción
const int BOTON = 2;
const int LISTO = 12;
const int YA = 13;

void setup() {
  Serial.begin(9600);
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(LISTO, OUTPUT);
  pinMode(YA, OUTPUT);
}

void loop() {
  digitalWrite(LISTO, HIGH);
  delay(random(1500, 4000));
  digitalWrite(LISTO, LOW);
  digitalWrite(YA, HIGH);
  unsigned long inicio = millis();
  while (digitalRead(BOTON) == HIGH) { }        // espera a que lo presionen
  unsigned long tiempo = millis() - inicio;
  digitalWrite(YA, LOW);
  Serial.print("Tu tiempo de reacción: ");
  Serial.print(tiempo);
  Serial.println(" ms");
  delay(2500);
}
```

💡 **¿Cómo funciona?**

Al encender el LED verde se guarda la hora con millis(). Cuando se presiona el botón se resta esa hora y se obtiene cuántos milisegundos tardaste.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

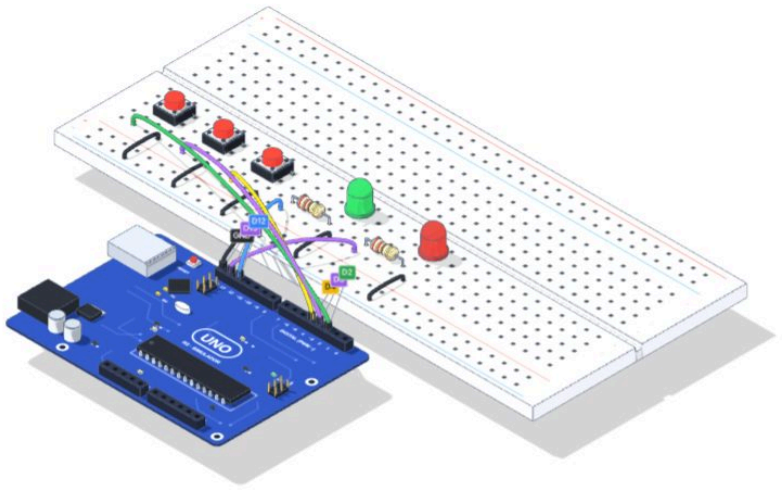
🏆 **Reto:** Guarda tu mejor tiempo y muéstralo cada vez que lo superes.

PROYECTO 31 · BOTONES E INTERRUPTORES

Cerradura de secuencia de botones

★★☆ Intermedio · Tiempo: 30 a 45 minutos

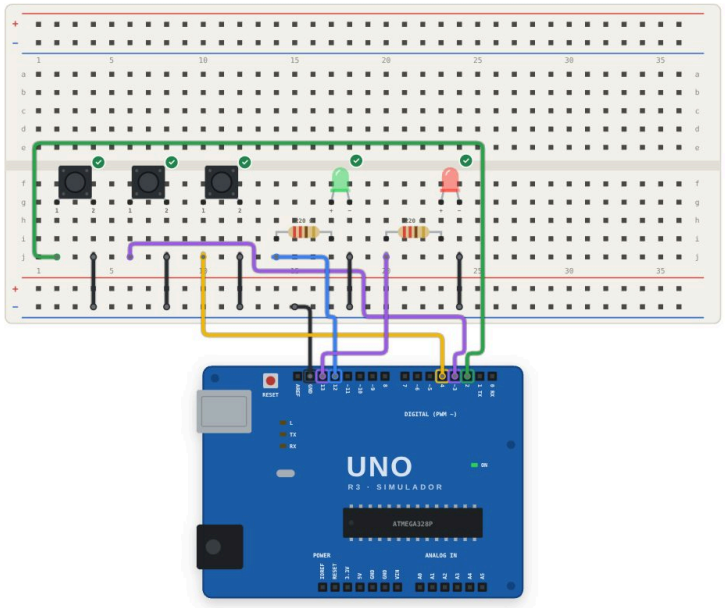
Tres botones forman una combinación secreta: solo la secuencia correcta enciende la luz verde.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Botón A (Botón)
- 1 × Botón B (Botón)
- 1 × Botón C (Botón)
- 1 × Abierto (LED)
- 1 × Error (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Botón A (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Botón B (Botón)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Botón C (Botón)**

- Pata 1 → pin **D4**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

5 **Abierto (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 **Error (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Cerradura de secuencia: la clave es A, C, B
const int A = 2, B = 3, CC = 4;
const int VERDE = 12, ROJO = 13;
int clave[] = {2, 4, 3};       // A, C, B
int paso = 0;

int leerBoton() {              // devuelve el pin presionado o 0
  if (digitalRead(A) == LOW) return A;
  if (digitalRead(B) == LOW) return B;
  if (digitalRead(CC) == LOW) return CC;
  return 0;
}

void setup() {
  Serial.begin(9600);
  pinMode(A, INPUT_PULLUP); pinMode(B, INPUT_PULLUP); pinMode(CC, INPUT_PULLUP);
  pinMode(VERDE, OUTPUT); pinMode(ROJO, OUTPUT);
}

void loop() {
  int b = leerBoton();
  if (b == 0) return;
  while (leerBoton() != 0) { }     // espera a que lo suelten
  if (b == clave[paso]) {
    paso++;
    Serial.print("Paso correcto ");
    Serial.println(paso);
    if (paso == 3) {
      Serial.println("¡Abierto!");
      digitalWrite(VERDE, HIGH); delay(3000); digitalWrite(VERDE, LOW);
      paso = 0;
    }
```

```
  } else {
    Serial.println("Clave incorrecta");
    digitalWrite(ROJO, HIGH); delay(1000); digitalWrite(ROJO, LOW);
    paso = 0;
  }
}
```

💡 ¿Cómo funciona?

El arreglo clave guarda el orden correcto. La variable paso dice cuántos botones correctos llevas. Un error devuelve paso a 0 y hay que empezar de nuevo.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

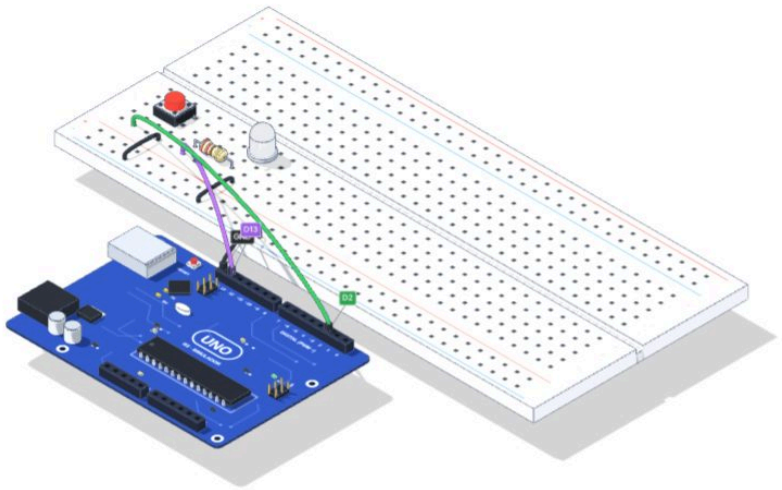
🏆 **Reto:** Cambia la clave por una de cuatro botones.

PROYECTO 32 · BOTONES E INTERRUPTORES

Lámpara con temporizador

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

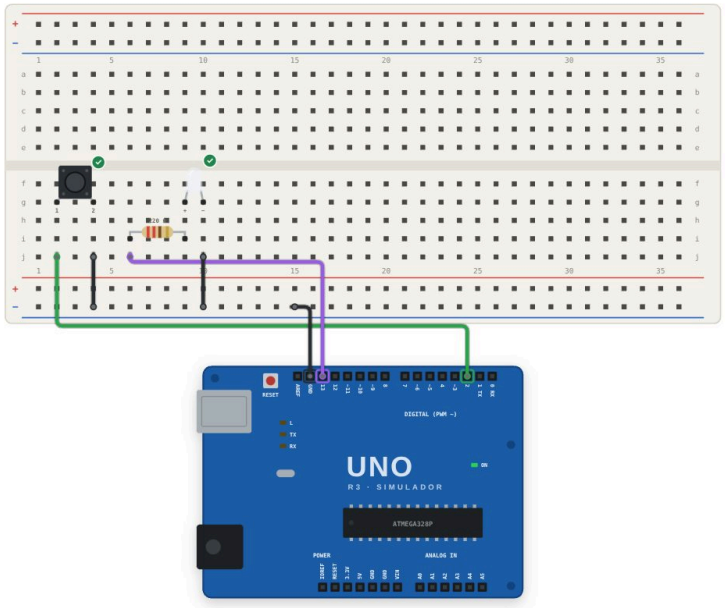
Como la luz de una escalera: al presionar el botón se enciende y se apaga sola a los 10 segundos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Botón
- 1 × Luz de la escalera (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Botón (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Luz de la escalera (LED blanco)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
// Luz de escalera con temporizador
const int BOTON = 2;
const int LUZ = 13;
const unsigned long DURACION = 10000;   // 10 segundos
unsigned long encendidaDesde = 0;
bool encendida = false;

void setup() {
  Serial.begin(9600);
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(LUZ, OUTPUT);
}

void loop() {
  if (digitalRead(BOTON) == LOW) {
    encendida = true;
    encendidaDesde = millis();          // vuelve a contar desde ahora
    digitalWrite(LUZ, HIGH);
  }
  if (encendida && millis() - encendidaDesde > DURACION) {
    encendida = false;
    digitalWrite(LUZ, LOW);
    Serial.println("Luz apagada por tiempo");
  }
}
```

💡 ¿Cómo funciona?

Cada vez que se presiona el botón se guarda el momento con millis(). Cuando pasan 10 segundos desde ese momento, el LED se apaga solo.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

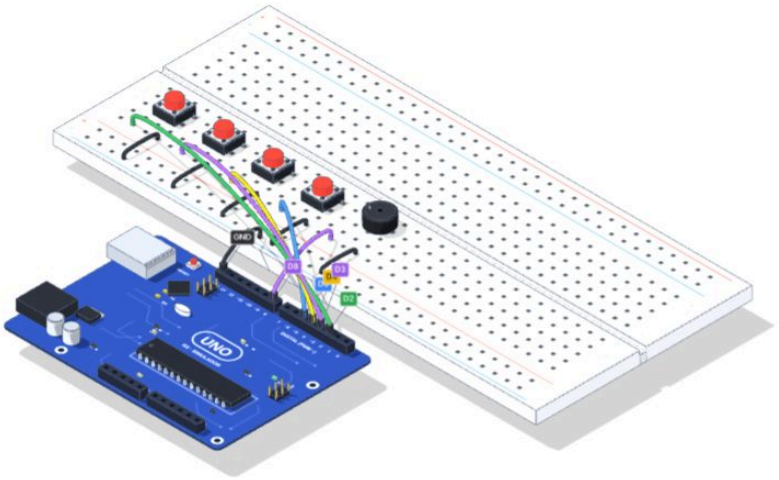
🏆 **Reto:** Haz que el LED parpadee los últimos 3 segundos para avisar que se va a apagar.

PROYECTO 33 · BOTONES E INTERRUPTORES

Piano de cuatro teclas

★★☆ Intermedio · Tiempo: 30 a 45 minutos

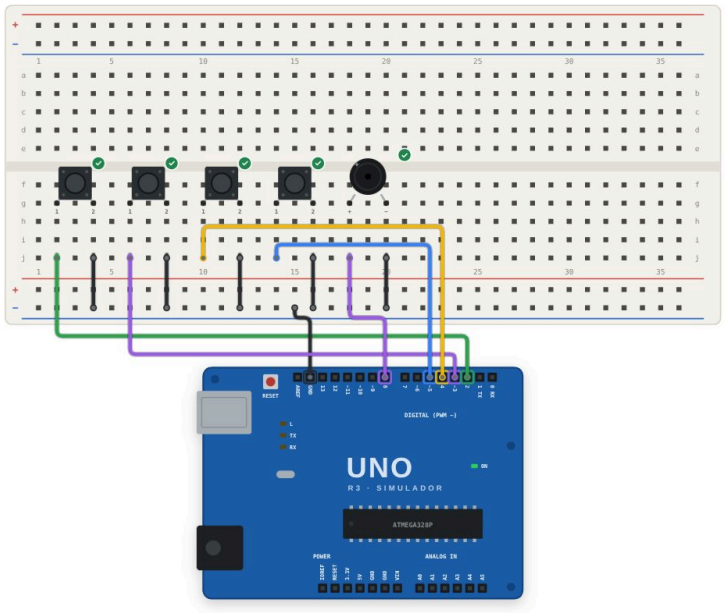
Cuatro botones tocan las notas do, re, mi y fa en el zumbador.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Do (Botón)
- 1 × Re (Botón)
- 1 × Mi (Botón)
- 1 × Fa (Botón)
- 1 × Parlante (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Do (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Re (Botón)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Mi (Botón)**

- Pata 1 → pin **D4**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

5 **Fa (Botón)**

- Pata 1 → pin **~D5**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

6 **Parlante (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

7 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Piano de cuatro teclas
int teclas[] = {2, 3, 4, 5};
int notas[] = {262, 294, 330, 349};   // do, re, mi, fa (en hercios)
const int PARLANTE = 8;

void setup() {
  for (int i = 0; i < 4; i++) pinMode(teclas[i], INPUT_PULLUP);
}

void loop() {
  int sonando = 0;
  for (int i = 0; i < 4; i++) {
    if (digitalRead(teclas[i]) == LOW) sonando = notas[i];
  }
  if (sonando > 0) tone(PARLANTE, sonando);
  else noTone(PARLANTE);
}
```

💡 ¿Cómo funciona?

Cada botón tiene asignada una frecuencia en el arreglo notas. Mientras un botón esté presionado suena su nota; si no hay ninguno, noTone() silencia el parlante.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

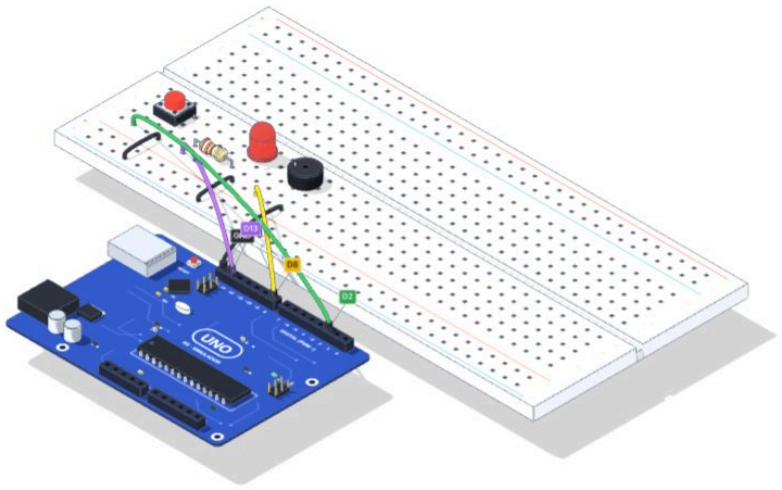
🏆 **Reto:** Agrega una quinta tecla con la nota sol (392 Hz).

PROYECTO 34 · BOTONES E INTERRUPTORES

Cuenta regresiva con botón

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

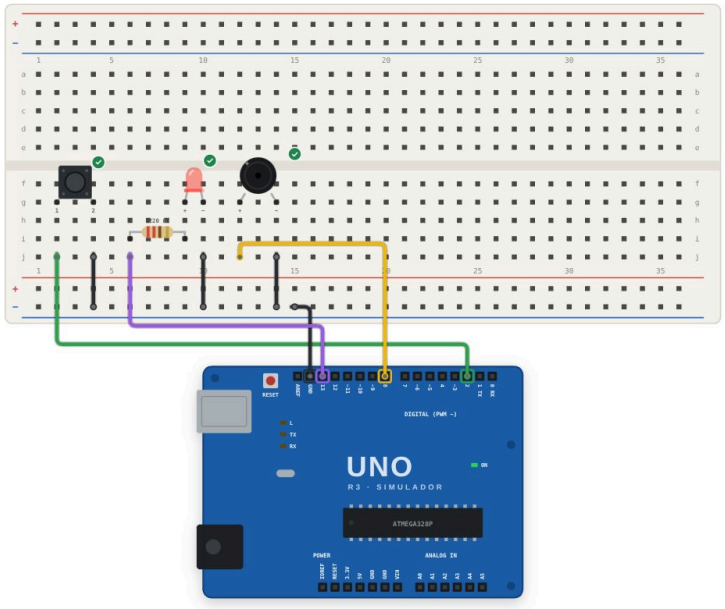
Al presionar el botón empieza una cuenta de 10 a 0 en el monitor y al final se enciende el LED: ¡despegue!



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Iniciar (Botón)
- 1 × Despegue (LED)
- 1 × Motor (Zumbador)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Iniciar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 Despegue (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Motor (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Cuenta regresiva para un cohete
const int BOTON = 2;
const int LUZ = 13;
const int ZUMBADOR = 8;

void setup() {
  Serial.begin(9600);
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(LUZ, OUTPUT);
  Serial.println("Presiona el botón para lanzar el cohete");
}

void loop() {
  if (digitalRead(BOTON) == LOW) {
    for (int n = 10; n > 0; n--) {
      Serial.println(n);
      tone(ZUMBADOR, 1000, 80);
      delay(1000);
    }
    Serial.println("¡DESPEGUE!");
    digitalWrite(LUZ, HIGH);
    tone(ZUMBADOR, 300, 2000);
    delay(3000);
    digitalWrite(LUZ, LOW);
  }
}
```

💡 **¿Cómo funciona?**

Un ciclo for cuenta desde 10 hasta 1 restando 1 cada segundo, con un pitido en cada número. Al terminar se enciende el LED y suena un tono grave largo.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

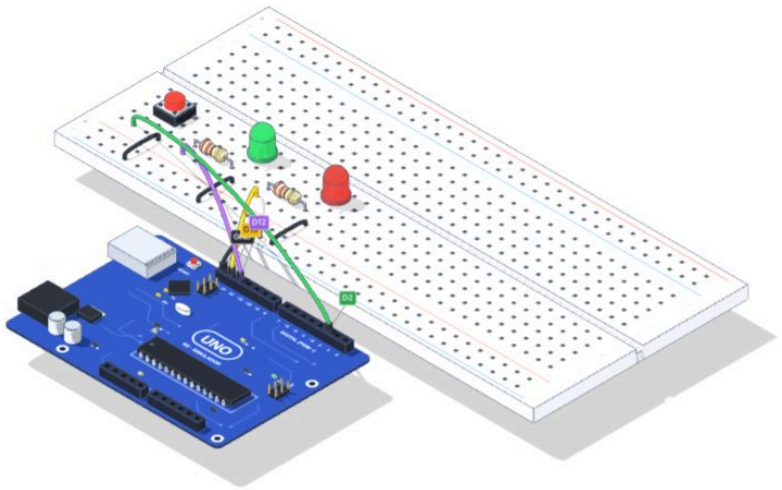
🏆 **Reto:** Agrega un segundo botón para abortar el lanzamiento.

PROYECTO 35 · BOTONES E INTERRUPTORES

Pulsación larga y corta

★★☆ Intermedio · Tiempo: 30 a 45 minutos

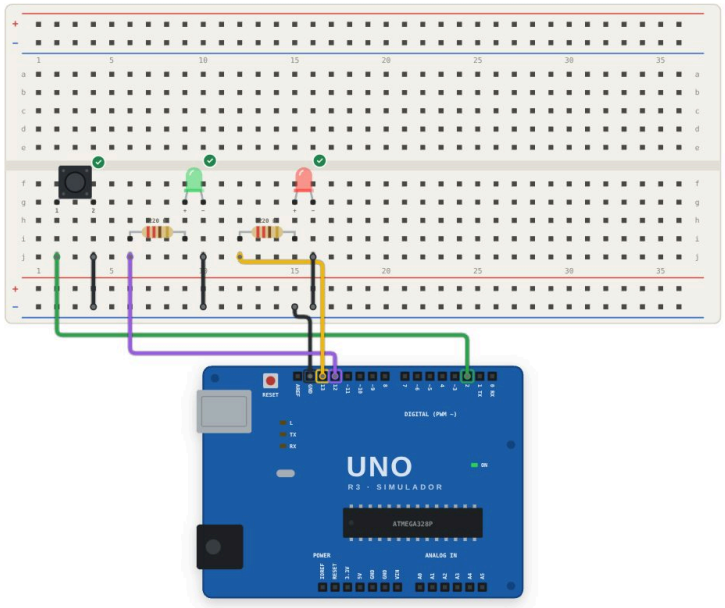
Un toque corto enciende el LED verde y un toque largo (más de 1 segundo) enciende el rojo.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Botón
- 1 × Corto (LED)
- 1 × Largo (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Botón (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Corto (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Largo (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Detectar pulsación corta o larga
const int BOTON = 2;
const int CORTO = 12, LARGO = 13;

void setup() {
  Serial.begin(9600);
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(CORTO, OUTPUT); pinMode(LARGO, OUTPUT);
}

void loop() {
  if (digitalRead(BOTON) == LOW) {
    unsigned long inicio = millis();
    while (digitalRead(BOTON) == LOW) { }   // espera a que lo suelten
    unsigned long duracion = millis() - inicio;
    Serial.print("Duró ");
    Serial.print(duracion);
    Serial.println(" ms");
    int pin = CORTO;
    if (duracion > 1000) pin = LARGO;
    digitalWrite(pin, HIGH); delay(800); digitalWrite(pin, LOW);
  }
}
```

💡 **¿Cómo funciona?**

Se mide cuánto tiempo pasa entre que el botón se presiona y se suelta. Si fue más de 1000 ms es un toque largo. Muchos aparatos usan esto para tener dos funciones en un solo botón.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

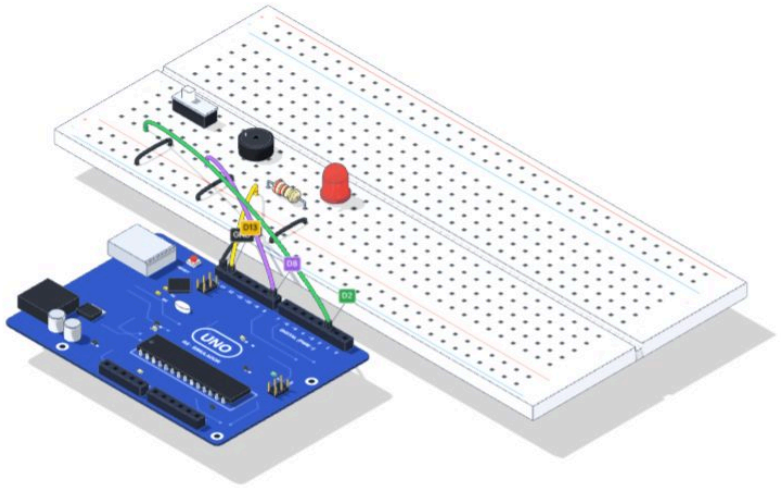
🏆 **Reto:** Agrega una tercera opción: doble toque rápido.

PROYECTO 36 · BOTONES E INTERRUPTORES

Alarma de puerta con interruptor

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

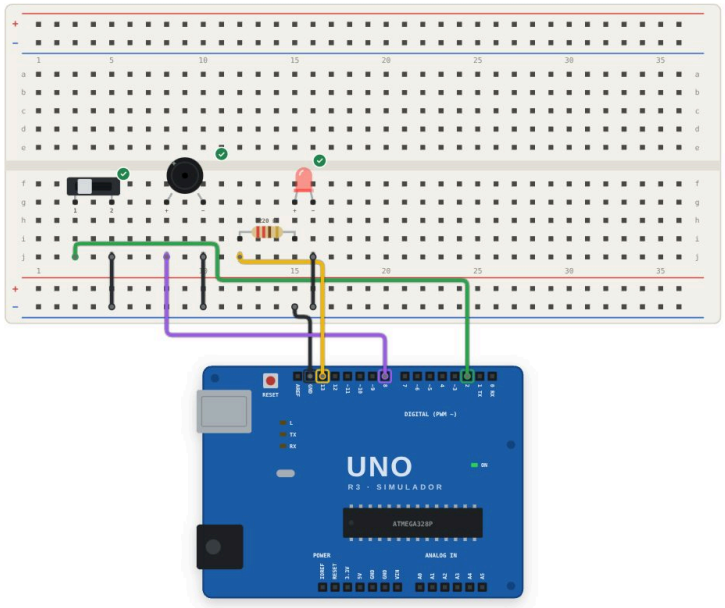
Un interruptor imita el sensor de una puerta: si la puerta se abre suena la alarma.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Puerta (activado = cerrada) (Interruptor)
- 1 × Alarma (Zumbador)
- 1 × Puerta abierta (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Puerta (activado = cerrada) (Interruptor)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**.

3 **Alarma (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 **Puerta abierta (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Alarma de puerta abierta
const int PUERTA = 2;        // activado (LOW) = puerta cerrada
const int ALARMA = 8;
const int LUZ = 13;

void setup() {
  Serial.begin(9600);
  pinMode(PUERTA, INPUT_PULLUP);
  pinMode(ALARMA, OUTPUT);
  pinMode(LUZ, OUTPUT);
}

void loop() {
  if (digitalRead(PUERTA) == HIGH) {     // interruptor abierto: puerta abierta
    digitalWrite(LUZ, HIGH);
    tone(ALARMA, 1500, 150);
    Serial.println("¡La puerta está abierta!");
    delay(400);
  } else {
    digitalWrite(LUZ, LOW);
  }
}
```

💡 ¿Cómo funciona?

Los sensores de puerta reales son interruptores que se cierran cuando la puerta está cerrada. Por eso la alarma suena cuando el pin lee HIGH (interruptor abierto).

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la posición del interruptor en su tarjeta del panel «Componentes».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

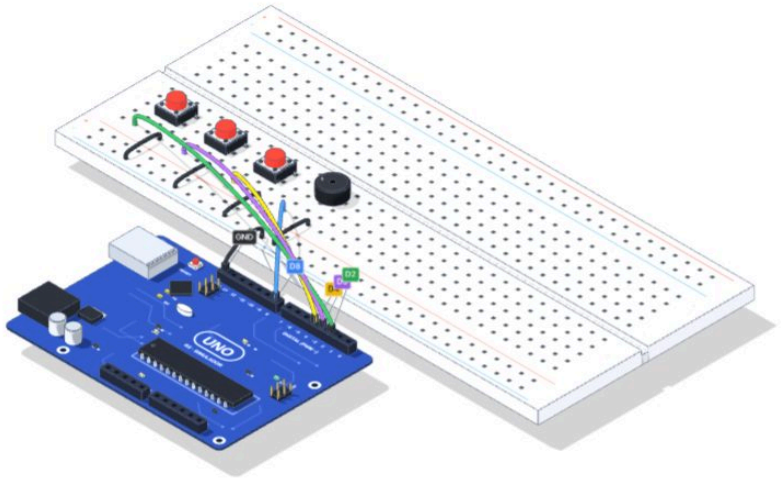
🏆 **Reto:** Haz que la alarma espere 5 segundos antes de sonar, para dar tiempo a cerrar la puerta.

PROYECTO 37 · BOTONES E INTERRUPTORES

Marcador de dos equipos

★★☆ Intermedio · Tiempo: 30 a 45 minutos

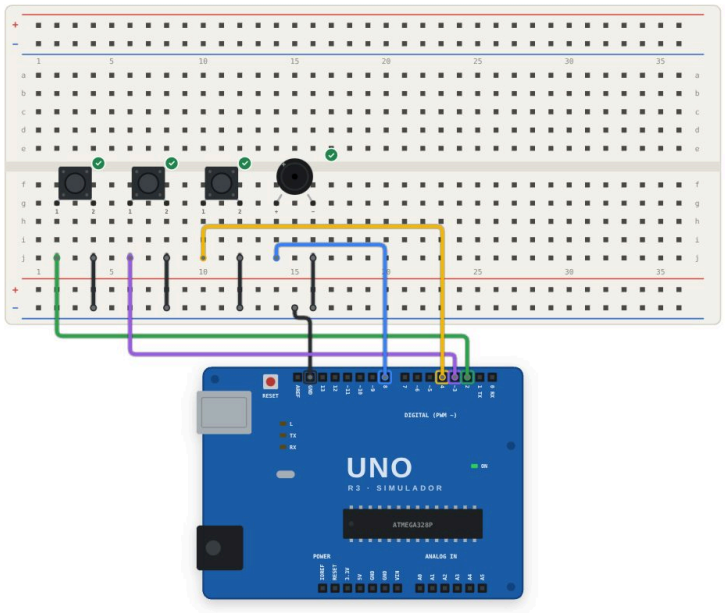
Dos botones suman goles a cada equipo y el monitor serie muestra el marcador.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Gol equipo A (Botón)
- 1 × Gol equipo B (Botón)
- 1 × Reiniciar (Botón)
- 1 × Pito (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Gol equipo A (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Gol equipo B (Botón)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Reiniciar (Botón)**

- Pata 1 → pin **D4**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

5 **Pito (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

6 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Marcador de dos equipos
const int GOL_A = 2, GOL_B = 3, REINICIAR = 4;
const int PITO = 8;
int a = 0, b = 0;

void mostrar() {
  Serial.print("Equipo A ");
  Serial.print(a);
  Serial.print(" - ");
  Serial.print(b);
  Serial.println(" Equipo B");
}

void esperarSoltar(int pin) {
  delay(30);
  while (digitalRead(pin) == LOW) { }
}

void setup() {
  Serial.begin(9600);
  pinMode(GOL_A, INPUT_PULLUP); pinMode(GOL_B, INPUT_PULLUP); pinMode(REINICIAR, INPUT_PULLUP);
  mostrar();
}

void loop() {
  if (digitalRead(GOL_A) == LOW) { a++; tone(PITO, 1500, 200); mostrar(); esperarSoltar(GOL_A); }
  if (digitalRead(GOL_B) == LOW) { b++; tone(PITO, 1500, 200); mostrar(); esperarSoltar(GOL_B); }
  if (digitalRead(REINICIAR) == LOW) { a = 0; b = 0; tone(PITO, 600, 600); mostrar();
esperarSoltar(REINICIAR); }
}
```

💡 **¿Cómo funciona?**

Cada botón suma a su propia variable. La función esperarSoltar() evita que un solo toque cuente muchos goles mientras el dedo sigue sobre el botón.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

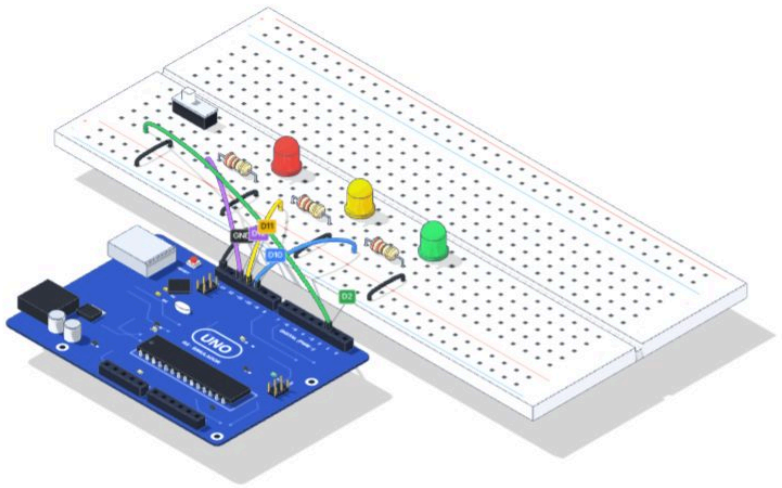
🏆 **Reto:** Muestra el marcador en una pantalla LCD en lugar del monitor serie.

PROYECTO 38 · BOTONES E INTERRUPTORES

Semáforo peatonal con interruptor nocturno

★★☆ Intermedio · Tiempo: 30 a 45 minutos

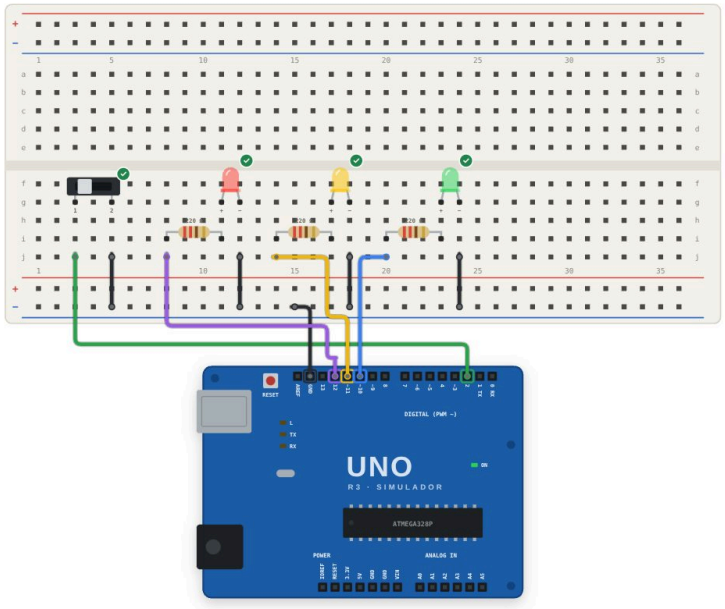
Un semáforo normal que, con el interruptor activado, pasa a modo nocturno con amarillo intermitente.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Modo noche (Interruptor)
- 1 × Rojo (LED)
- 1 × Amarillo (LED)
- 1 × Verde (LED)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Modo noche (Interruptor)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**.

3 Rojo (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Amarillo (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Verde (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Semáforo con modo nocturno
const int NOCHE = 2;
const int ROJO = 12, AMARILLO = 11, VERDE = 10;

void apagarTodo() {
  digitalWrite(ROJO, LOW); digitalWrite(AMARILLO, LOW); digitalWrite(VERDE, LOW);
}

void setup() {
  pinMode(NOCHE, INPUT_PULLUP);
  pinMode(ROJO, OUTPUT); pinMode(AMARILLO, OUTPUT); pinMode(VERDE, OUTPUT);
}

void loop() {
  if (digitalRead(NOCHE) == LOW) {          // modo nocturno
    apagarTodo();
    digitalWrite(AMARILLO, HIGH); delay(500);
    digitalWrite(AMARILLO, LOW);  delay(500);
  } else {                                  // modo normal
    apagarTodo(); digitalWrite(VERDE, HIGH);    delay(3000);
    apagarTodo(); digitalWrite(AMARILLO, HIGH); delay(1000);
    apagarTodo(); digitalWrite(ROJO, HIGH);     delay(3000);
  }
}
```

💡 **¿Cómo funciona?**

En cada vuelta del loop() se revisa el interruptor. Si está activado, el semáforo solo hace parpadear el amarillo, que en la noche significa "cruce con precaución".

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Cambia la posición del interruptor en su tarjeta del panel «Componentes».

Con piezas reales: sube el programa.

🏆 **Reto:** Haz que el cambio a modo nocturno sea automático con un sensor de luz.

CAPÍTULO 3

Sonido y música

Con un zumbador y la función tone() se hacen timbres, alarmas, sirenas y canciones.

EN ESTE CAPÍTULO · 13 PROYECTOS

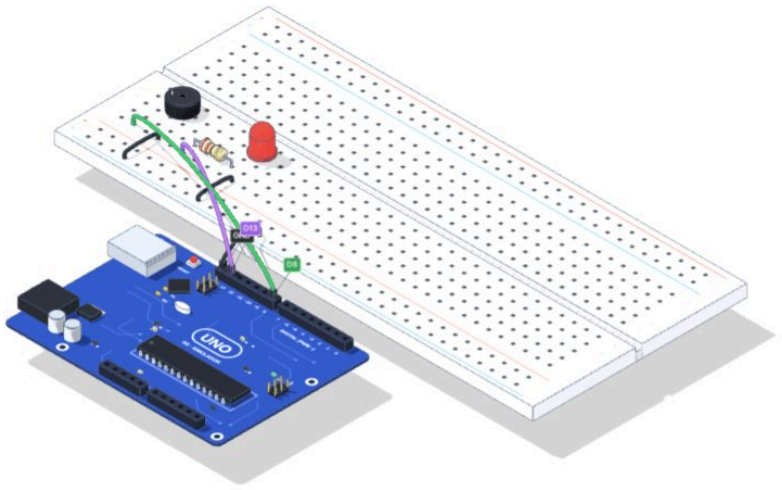
39 Sirena de bomberos ★☆☆
40 Escala musical ★☆☆
41 Feliz cumpleaños ★★☆
42 Alarma de despertador ★☆☆
43 Tono que sube con la perilla ★☆☆
44 Theremin de luz ★★☆
45 Código Morse sonoro ★★☆
46 Metrónomo musical ajustable ★★☆
47 Caja musical con botón ★★☆
48 Detector de silencio ★★☆
49 Aplauso que enciende la luz ★★☆
50 Alarma de golpe con sirena ★★☆
51 Piano táctil de 3 notas ★☆☆

PROYECTO 39 · SONIDO Y MÚSICA

Sirena de bomberos

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

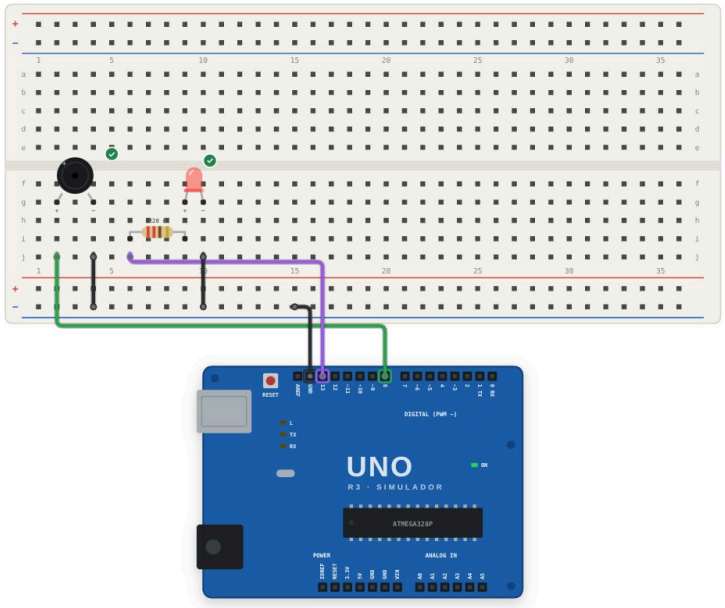
El zumbador sube y baja de tono como la sirena de un camión de bomberos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sirena (Zumbador)
- 1 × Baliza (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sirena (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

3 **Baliza (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
1  // Sirena de bomberos
2  const int SIRENA = 8;
3  const int BALIZA = 13;
4
5  void setup() {
6    pinMode(BALIZA, OUTPUT);
7  }
8
9  void loop() {
10   digitalWrite(BALIZA, HIGH);
11   for (int f = 600; f <= 1500; f += 20) { tone(SIRENA, f); delay(8); }   // sube
12   digitalWrite(BALIZA, LOW);
13   for (int f = 1500; f >= 600; f -= 20) { tone(SIRENA, f); delay(8); }   // baja
14 }
```

💡 **¿Cómo funciona?**

Un ciclo for aumenta la frecuencia del sonido poco a poco y otro la disminuye. Así el tono sube y baja sin parar, como una sirena.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

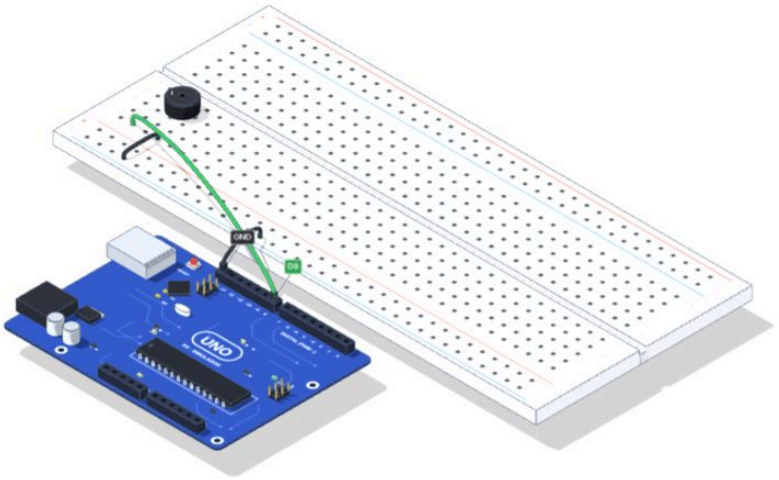
🏆 **Reto:** Haz una sirena de ambulancia que alterne solo dos tonos.

PROYECTO 40 · SONIDO Y MÚSICA

Escala musical

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

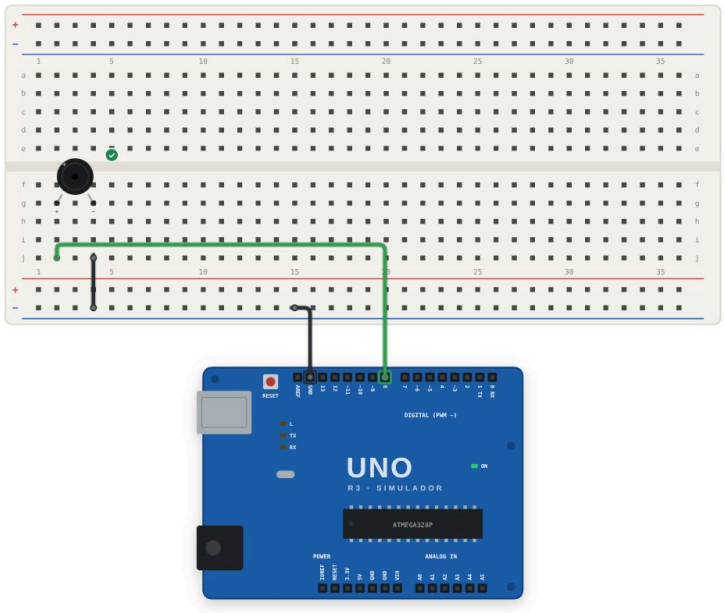
El zumbador toca la escala do, re, mi, fa, sol, la, si, do.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Parlante (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Parlante (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Escala musical
const int PARLANTE = 8;
int notas[] = {262, 294, 330, 349, 392, 440, 494, 523};
String nombres[] = {"Do", "Re", "Mi", "Fa", "Sol", "La", "Si", "Do"};

void setup() {
  Serial.begin(9600);
}

void loop() {
  for (int i = 0; i < 8; i++) {
    Serial.println(nombres[i]);
    tone(PARLANTE, notas[i], 400);
    delay(500);
  }
  delay(1500);
}
```

💡 **¿Cómo funciona?**

Cada nota musical es una frecuencia (vibraciones por segundo). El arreglo notas guarda las frecuencias de la escala y el arreglo nombres sus nombres, en el mismo orden.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa y abre el monitor serie a 9600.

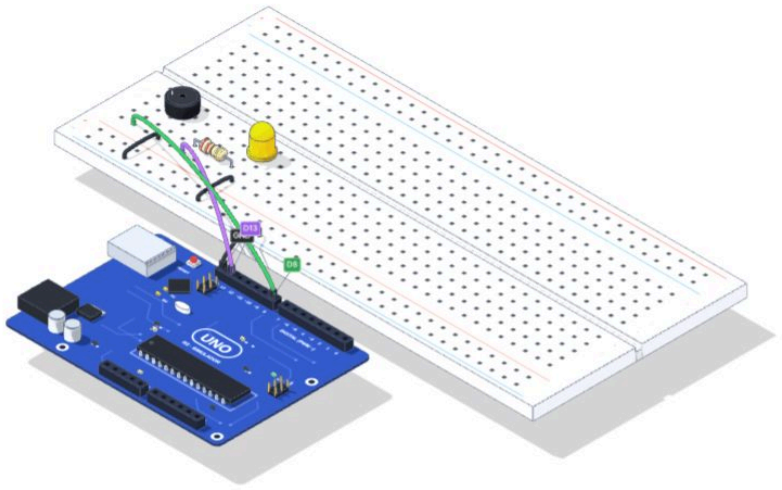
🏆 **Reto:** Toca la escala al revés, de do agudo a do grave.

PROYECTO 41 · SONIDO Y MÚSICA

Feliz cumpleaños

★★☆ Intermedio · Tiempo: 30 a 45 minutos

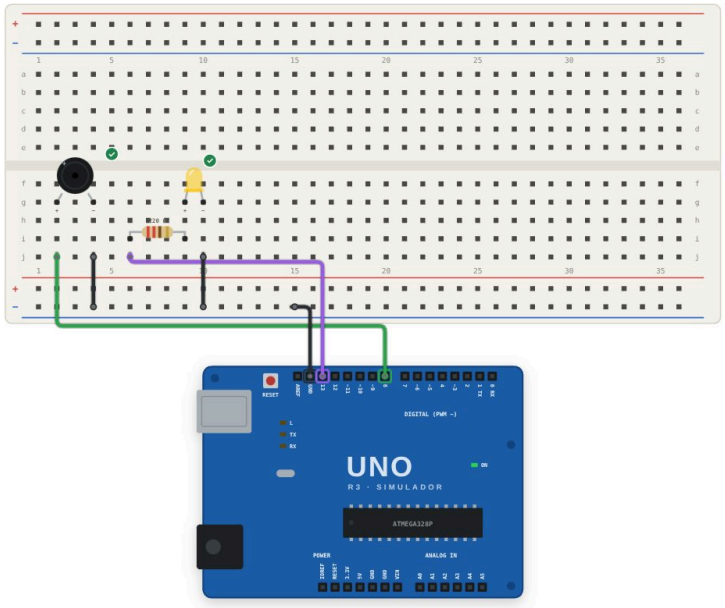
El Arduino toca la melodía de "Feliz cumpleaños".



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Parlante (Zumbador)
- 1 × Vela (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Parlante (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

3 **Vela (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Feliz cumpleaños
const int PARLANTE = 8;
const int VELA = 13;
int melodia[] = {262, 262, 294, 262, 349, 330, 262, 262, 294, 262, 392, 349,
                 262, 262, 523, 440, 349, 330, 294, 466, 466, 440, 349, 392, 349};
int tiempos[] = {3, 1, 4, 4, 4, 8, 3, 1, 4, 4, 4, 8, 3, 1, 4, 4, 4, 4, 8, 3, 1, 4, 4, 4, 8};

void setup() {
  pinMode(VELA, OUTPUT);
}

void loop() {
  for (int i = 0; i < 25; i++) {
    int duracion = tiempos[i] * 110;
    digitalWrite(VELA, HIGH);
    tone(PARLANTE, melodia[i], duracion);
    delay(duracion * 0.6);
    digitalWrite(VELA, LOW);
    delay(duracion * 0.6);
  }
  delay(3000);
}
```

💡 **¿Cómo funciona?**

La canción se guarda en dos arreglos: la frecuencia de cada nota y cuánto dura. El LED se enciende con cada nota, como una vela que baila con la música.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

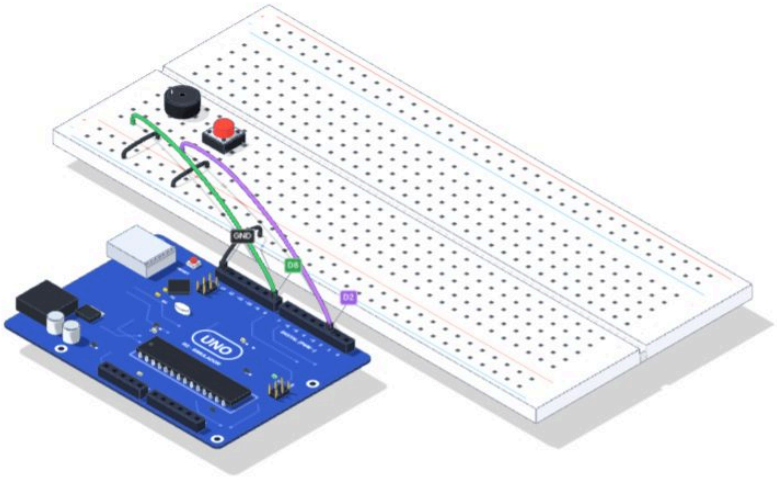
🏆 **Reto:** Escribe otra canción que conozcas cambiando los dos arreglos.

PROYECTO 42 · SONIDO Y MÚSICA

Alarma de despertador

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

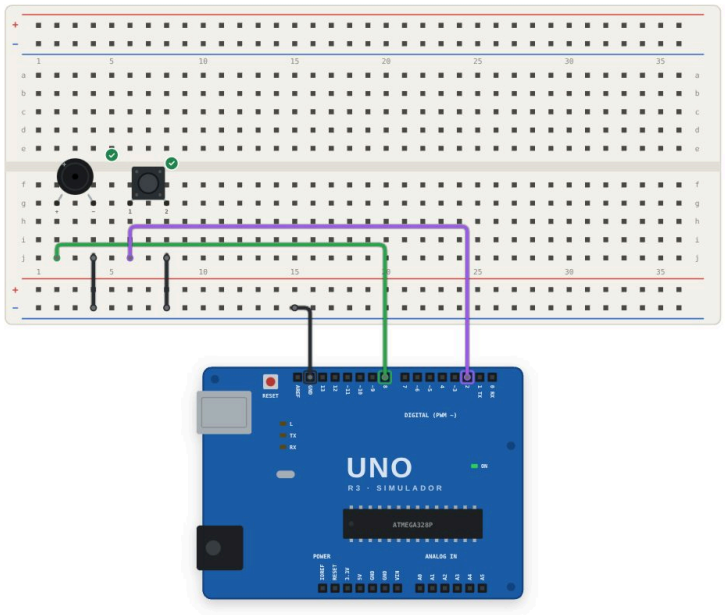
Después de un tiempo suena un "bip-bip" de despertador hasta que presionas el botón.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Despertador (Zumbador)
- 1 × Apagar (Botón)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Despertador (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

3 **Apagar (Botón)**

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
// Despertador que suena a los 10 segundos
const int ZUMBADOR = 8;
const int BOTON = 2;
const unsigned long ESPERA = 10000;
bool sonando = false;
bool apagado = false;

void setup() {
  Serial.begin(9600);
  pinMode(BOTON, INPUT_PULLUP);
  Serial.println("El despertador sonará en 10 segundos...");
}

void loop() {
  if (!apagado && millis() > ESPERA) sonando = true;
  if (sonando) {
    for (int i = 0; i < 4; i++) { tone(ZUMBADOR, 2000, 80); delay(150); }
    delay(400);
    if (digitalRead(BOTON) == LOW) {
      sonando = false; apagado = true;
      Serial.println("¡Buenos días!");
    }
  }
}
```

💡 ¿Cómo funciona?

millis() dice cuánto tiempo lleva encendido el Arduino. Cuando pasa de 10 segundos, el despertador empieza a sonar en ráfagas de cuatro pitidos, hasta que se presiona el botón.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

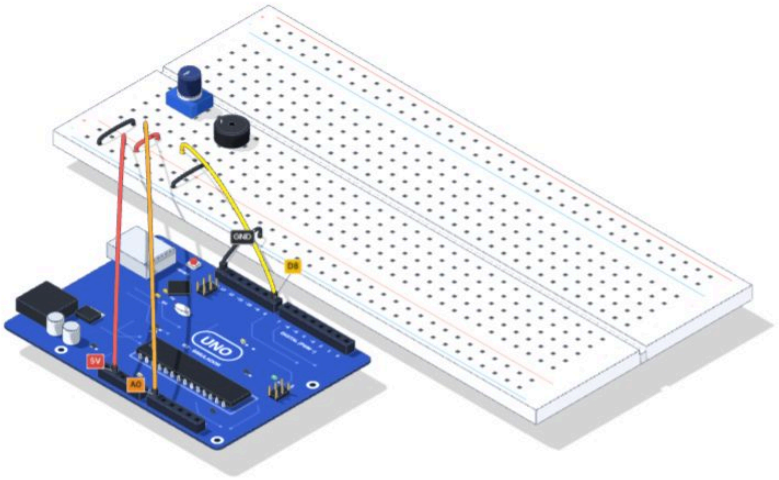
🏆 **Reto:** Agrega un botón de "5 minutos más" que lo vuelva a programar.

PROYECTO 43 · SONIDO Y MÚSICA

Tono que sube con la perilla

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

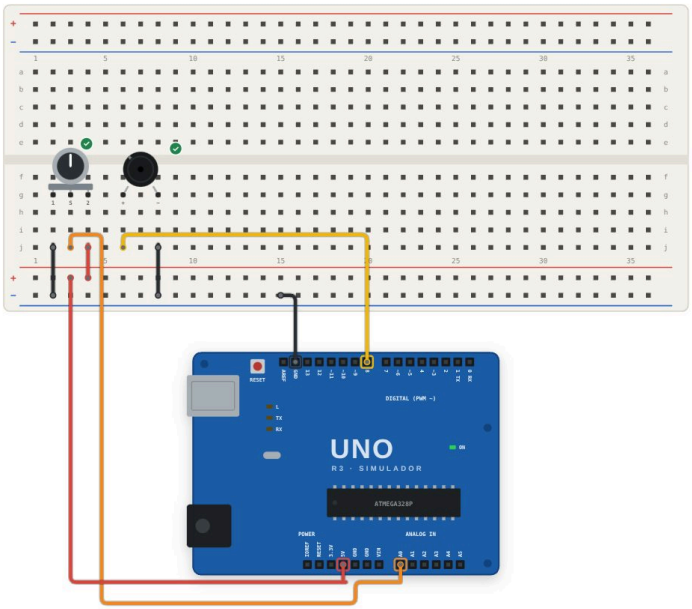
Al girar el potenciómetro cambia la nota del zumbador, de grave a agudo.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Perilla (Potenciómetro)
- 1 × Parlante (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Perilla (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Parlante (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 81 mA.
- Consumo estimado: **~81 mA** funcionando, con picos de **~81 mA**.

2 El programa

```
// Tono controlado con un potenciómetro
const int PERILLA = A0;
const int PARLANTE = 8;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int valor = analogRead(PERILLA);              // 0 a 1023
  int frecuencia = map(valor, 0, 1023, 200, 2000);
  tone(PARLANTE, frecuencia);
  Serial.print("Frecuencia: ");
  Serial.println(frecuencia);
  delay(50);
}
```

💡 ¿Cómo funciona?

analogRead() lee la posición de la perilla como un número de 0 a 1023. map() convierte ese número a una frecuencia entre 200 y 2000 Hz.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

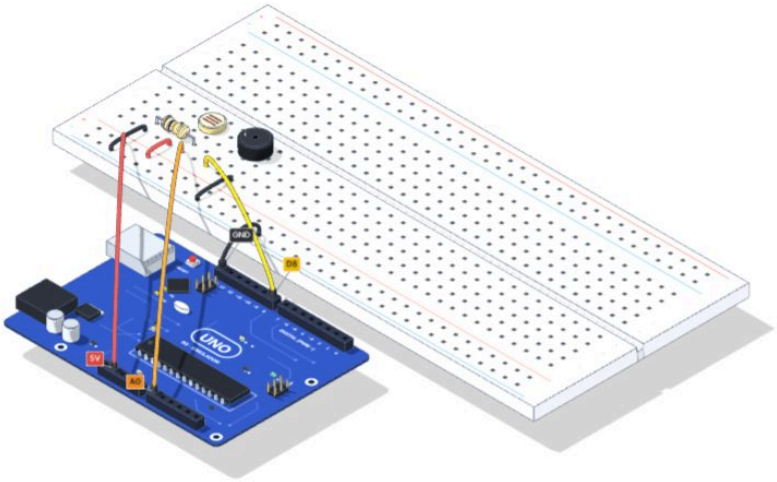
🏆 **Reto:** Haz que solo suenen las 8 notas de la escala según la posición de la perilla.

PROYECTO 44 · SONIDO Y MÚSICA

Theremin de luz

★★☆ Intermedio · Tiempo: 30 a 45 minutos

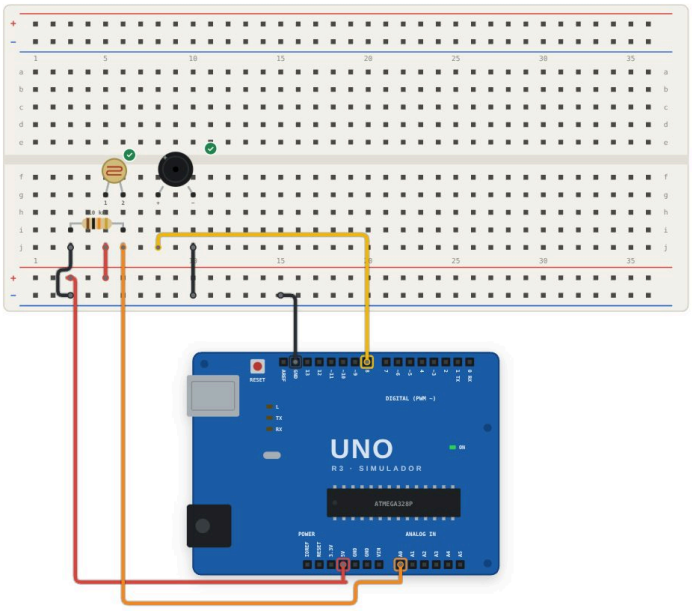
Un instrumento que se toca acercando la mano a un sensor de luz: más sombra, más grave.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de luz (Fotorresistencia)
- 1 × Parlante (Zumbador)
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de luz (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Parlante (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 81 mA.
- Consumo estimado: **~81 mA** funcionando, con picos de **~81 mA**.

2 El programa

```
// Theremin de luz
const int LUZ = A0;
const int PARLANTE = 8;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int luz = analogRead(LUZ);                    // más luz = número más alto
  int nota = map(luz, 0, 1023, 150, 1800);
  tone(PARLANTE, nota);
  Serial.println(nota);
  delay(30);
}
```

💡 ¿Cómo funciona?

La fotorresistencia cambia su resistencia con la luz, y el Arduino lo lee en A0. Al tapar el sensor con la mano baja la luz y por lo tanto la nota.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

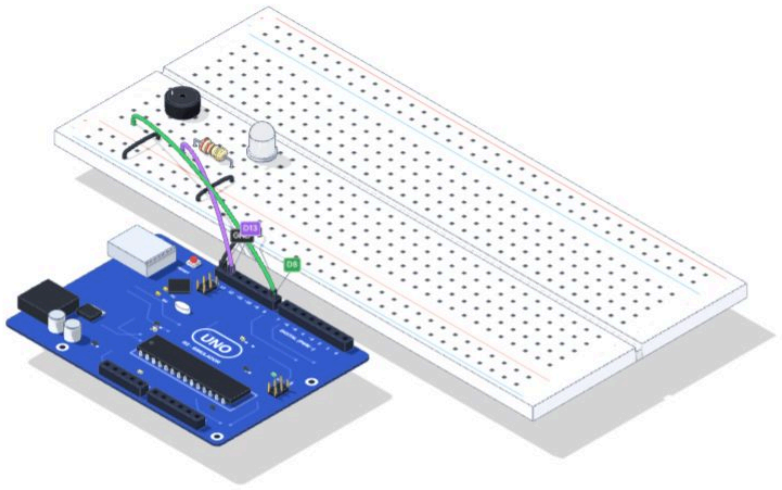
🏆 **Reto:** Haz que si está muy oscuro el instrumento se quede en silencio.

PROYECTO 45 · SONIDO Y MÚSICA

Código Morse sonoro

★★☆ Intermedio · Tiempo: 30 a 45 minutos

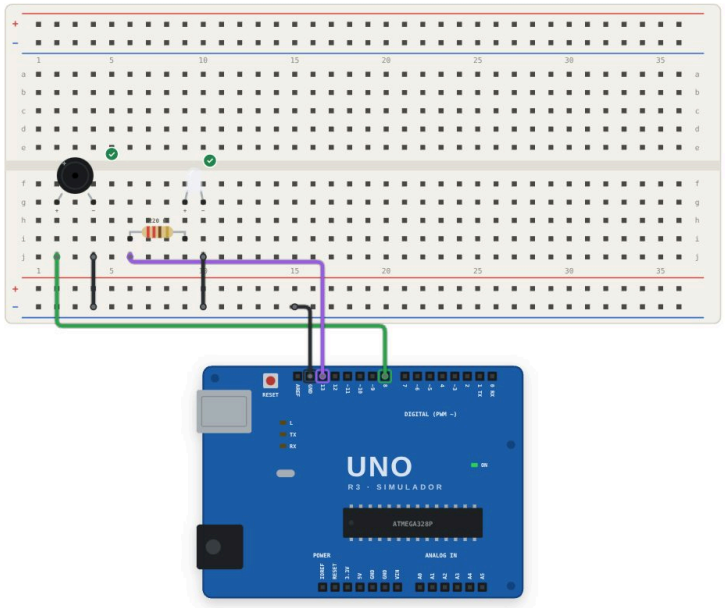
El zumbador y un LED envían la palabra HOLA en código Morse.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Telégrafo (Zumbador)
- 1 × Luz (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Telégrafo (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

3 **Luz (LED blanco)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// HOLA en código Morse
const int ZUMBADOR = 8;
const int LUZ = 13;
const int PUNTO = 150;

void senal(int duracion) {
  digitalWrite(LUZ, HIGH); tone(ZUMBADOR, 750);
  delay(duracion);
  digitalWrite(LUZ, LOW); noTone(ZUMBADOR);
  delay(PUNTO);
}

void letra(String codigo) {            // ejemplo: "...." para la H
  for (int i = 0; i < codigo.length(); i++) {
    if (codigo.charAt(i) == '.') senal(PUNTO);
    else senal(PUNTO * 3);
  }
  delay(PUNTO * 2);                    // espacio entre letras
}

void setup() {
  pinMode(LUZ, OUTPUT);
}

void loop() {
  letra("....");   // H
  letra("---");    // O
  letra(".-..");   // L
  letra(".-");     // A
  delay(2000);
}
```

💡 **¿Cómo funciona?**

Cada letra se escribe como un texto de puntos y rayas. La función letra() lo recorre carácter por carácter con charAt() y hace una señal corta o larga.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

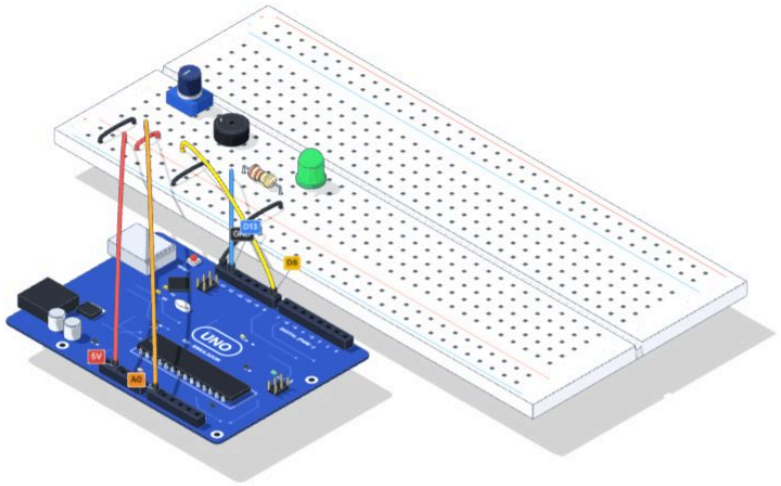
🏆 **Reto:** Agrega las letras de tu nombre.

PROYECTO 46 · SONIDO Y MÚSICA

Metrónomo musical ajustable

★★☆ Intermedio · Tiempo: 30 a 45 minutos

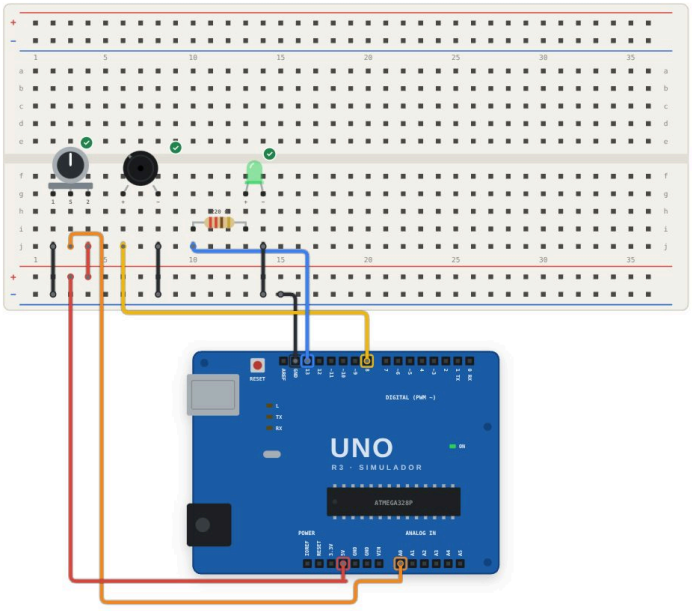
Un metrónomo que marca el ritmo con sonido; la perilla ajusta la velocidad de 40 a 200 pulsos por minuto.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Velocidad (Potenciómetro)
- 1 × Clic (Zumbador)
- 1 × Pulso (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Velocidad (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Clic (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Pulso (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 96 mA.
- Consumo estimado: **~96 mA** funcionando, con picos de **~96 mA**.

2 El programa

```
// Metrónomo ajustable
const int PERILLA = A0;
const int ZUMBADOR = 8;
const int LUZ = 13;
int tiempo = 1;

void setup() {
  Serial.begin(9600);
  pinMode(LUZ, OUTPUT);
}

void loop() {
  int bpm = map(analogRead(PERILLA), 0, 1023, 40, 200);
  int intervalo = 60000 / bpm;
  int nota = 1000;
  if (tiempo == 1) nota = 1600;          // primer tiempo más agudo
  tone(ZUMBADOR, nota, 50);
  digitalWrite(LUZ, HIGH); delay(60); digitalWrite(LUZ, LOW);
  Serial.print(bpm); Serial.println(" pulsos por minuto");
  delay(intervalo - 60);
  tiempo++;
  if (tiempo > 4) tiempo = 1;
}
```

💡 **¿Cómo funciona?**

La perilla elige los pulsos por minuto y el programa calcula los milisegundos entre pulsos. El primer tiempo de cada compás suena más agudo para no perder la cuenta.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

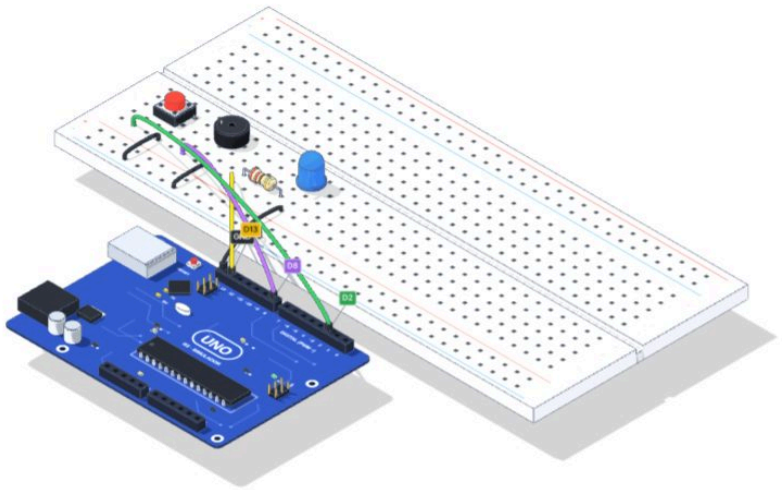
🏆 **Reto:** Agrega un interruptor para elegir compás de 3 o de 4 tiempos.

PROYECTO 47 · SONIDO Y MÚSICA

Caja musical con botón

★★☆ Intermedio · Tiempo: 30 a 45 minutos

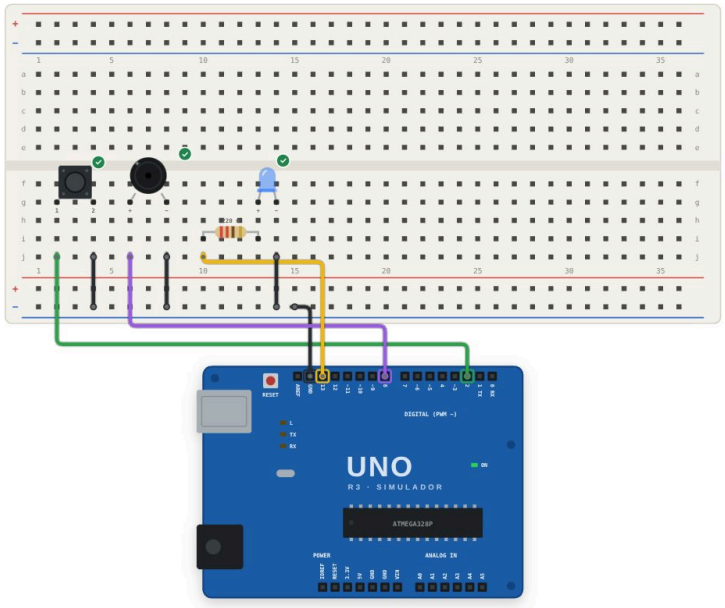
Al presionar el botón suena una canción de cuna y un LED azul acompaña cada nota.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Dar cuerda (Botón)
- 1 × Caja musical (Zumbador)
- 1 × Luz (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Dar cuerda (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Caja musical (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 **Luz (LED azul)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Caja musical: Estrellita, ¿dónde estás?
const int BOTON = 2;
const int PARLANTE = 8;
const int LUZ = 13;
int notas[] = {262, 262, 392, 392, 440, 440, 392, 349, 349, 330, 330, 294, 294, 262};
int largo[] = {1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 2};

void setup() {
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(LUZ, OUTPUT);
}

void loop() {
  if (digitalRead(BOTON) == LOW) {
    for (int i = 0; i < 14; i++) {
      int d = largo[i] * 400;
      digitalWrite(LUZ, HIGH);
      tone(PARLANTE, notas[i], d * 0.9);
      delay(d);
      digitalWrite(LUZ, LOW);
    }
  }
}
```

💡 ¿Cómo funciona?

La melodía está guardada en el arreglo notas y la duración de cada una en largo (1 = corta, 2 = larga). Solo se toca cuando el botón está presionado al inicio.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

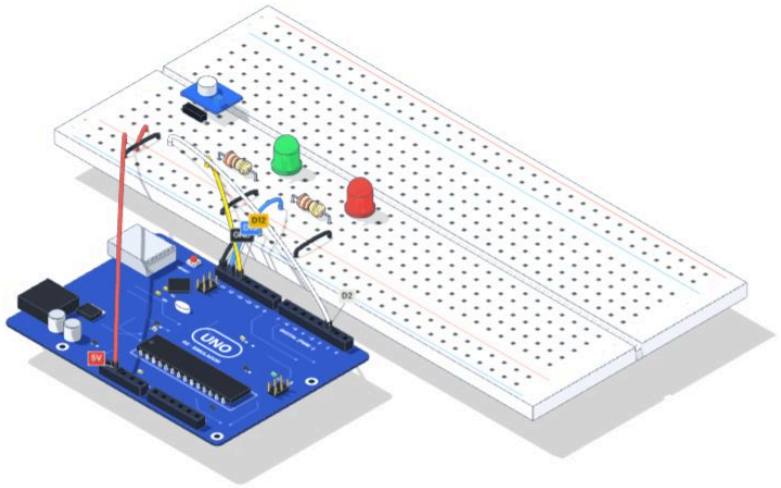
🏆 **Reto:** Completa la canción con la segunda parte.

PROYECTO 48 · SONIDO Y MÚSICA

Detector de silencio

★★☆ Intermedio · Tiempo: 30 a 45 minutos

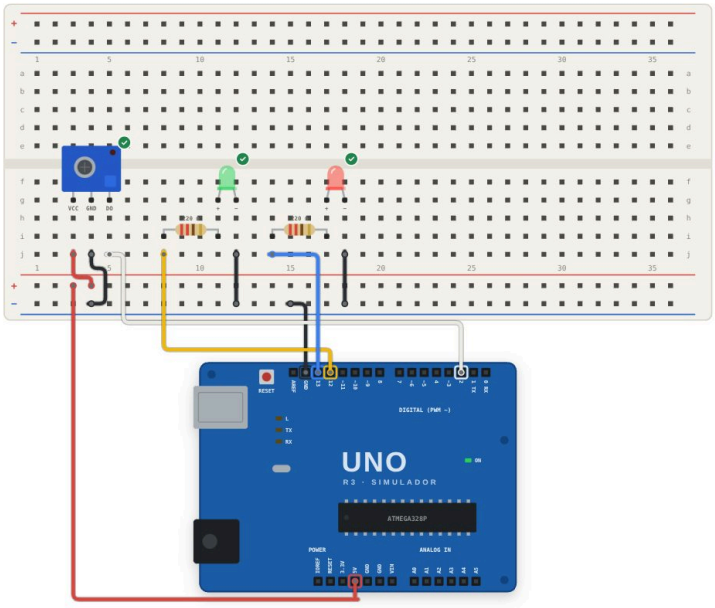
Un sensor de sonido vigila el salón: si hay mucho ruido se enciende una luz roja que pide silencio.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de sonido
- 1 × Silencio (LED)
- 1 × ¡Mucho ruido! (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de sonido (Sensor de sonido)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Silencio (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 ¡Mucho ruido! (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 85 mA.
- Consumo estimado: **~85 mA** funcionando, con picos de **~85 mA**.

2 El programa

```
// Detector de ruido para el salón
const int SONIDO = 2;        // DO del sensor: HIGH con ruido fuerte
const int VERDE = 12;
const int ROJO = 13;
int ruidos = 0;

void setup() {
  Serial.begin(9600);
  pinMode(SONIDO, INPUT);
  pinMode(VERDE, OUTPUT);
  pinMode(ROJO, OUTPUT);
}

void loop() {
  if (digitalRead(SONIDO) == HIGH) {
    ruidos++;
    Serial.print("Ruido detectado. Van ");
    Serial.println(ruidos);
    digitalWrite(VERDE, LOW);
    digitalWrite(ROJO, HIGH);
    delay(2000);                // la luz roja dura 2 segundos
  } else {
    digitalWrite(ROJO, LOW);
    digitalWrite(VERDE, HIGH);
  }
}
```

💡 **¿Cómo funciona?**

El módulo de sonido tiene un micrófono y un comparador: cuando el ruido supera el nivel fijado con su tornillo azul, la salida DO pasa a HIGH.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay ruido o un aplauso».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

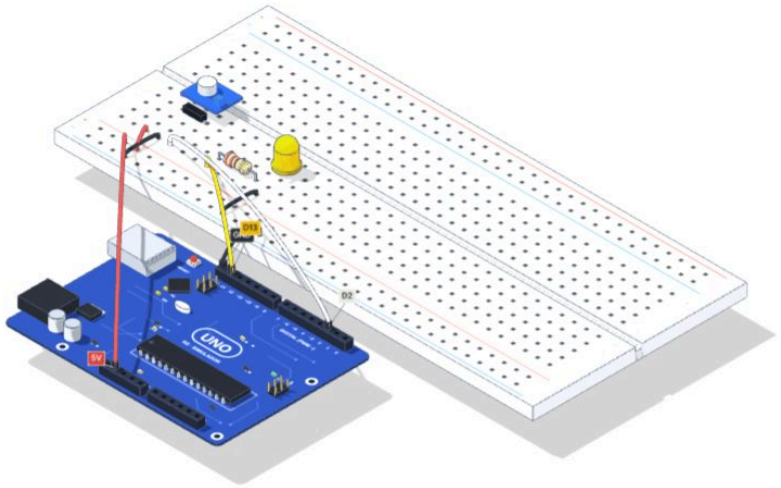
🏆 **Reto:** Cuenta los ruidos de los últimos 10 segundos y muestra una luz amarilla si son pocos.

PROYECTO 49 · SONIDO Y MÚSICA

Aplauso que enciende la luz

★★☆ Intermedio · Tiempo: 30 a 45 minutos

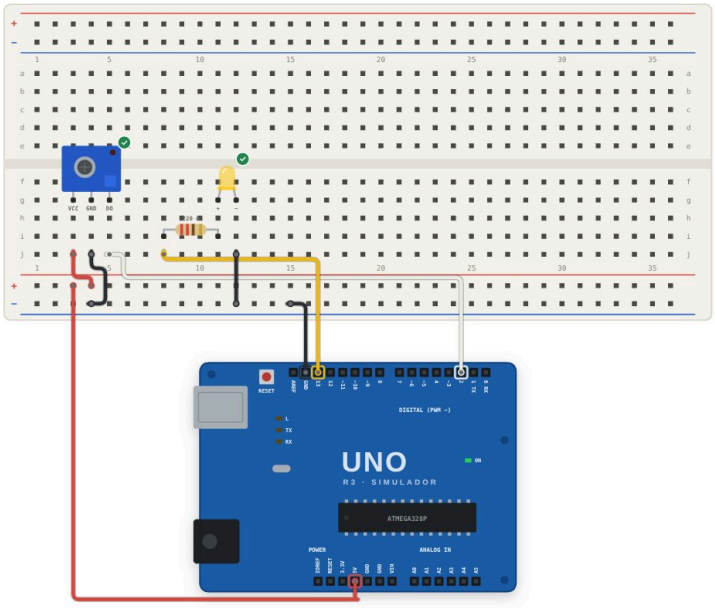
Una palmada enciende la lámpara y otra palmada la apaga.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Micrófono (Sensor de sonido)
- 1 × Lámpara (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Micrófono (Sensor de sonido)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Lámpara (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 70 mA.
- Consumo estimado: **~70 mA** funcionando, con picos de **~70 mA**.

2 El programa

```
// Luz que se enciende con un aplauso
const int MICROFONO = 2;
const int LAMPARA = 13;
bool encendida = false;

void setup() {
  Serial.begin(9600);
  pinMode(MICROFONO, INPUT);
  pinMode(LAMPARA, OUTPUT);
}

void loop() {
  if (digitalRead(MICROFONO) == HIGH) {
    encendida = !encendida;
    digitalWrite(LAMPARA, encendida);
    if (encendida) Serial.println("Luz encendida");
    else Serial.println("Luz apagada");
    delay(600);                  // espera a que termine el eco del aplauso
  }
}
```

💡 ¿Cómo funciona?

Cada vez que el sensor detecta un sonido fuerte, el estado de la lámpara se invierte con el operador !. La pausa de 600 ms evita que un solo aplauso cuente dos veces.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay ruido o un aplauso».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

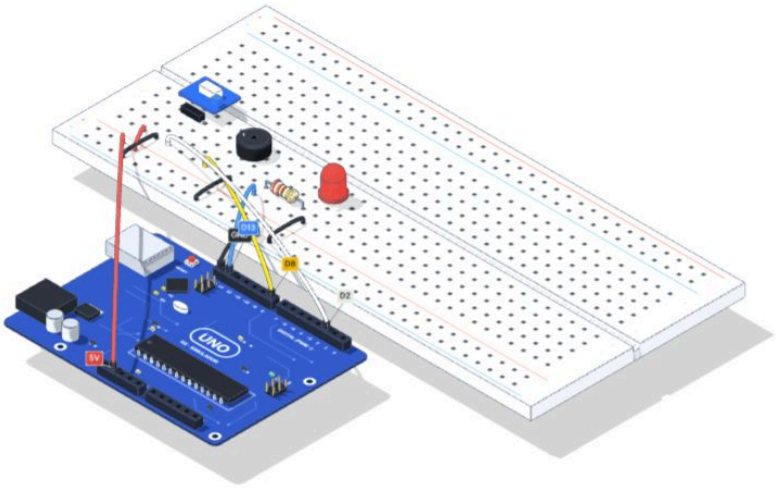
🏆 **Reto:** Haz que se necesiten dos aplausos seguidos para encender la luz.

PROYECTO 50 · SONIDO Y MÚSICA

Alarma de golpe con sirena

★★☆ Intermedio · Tiempo: 30 a 45 minutos

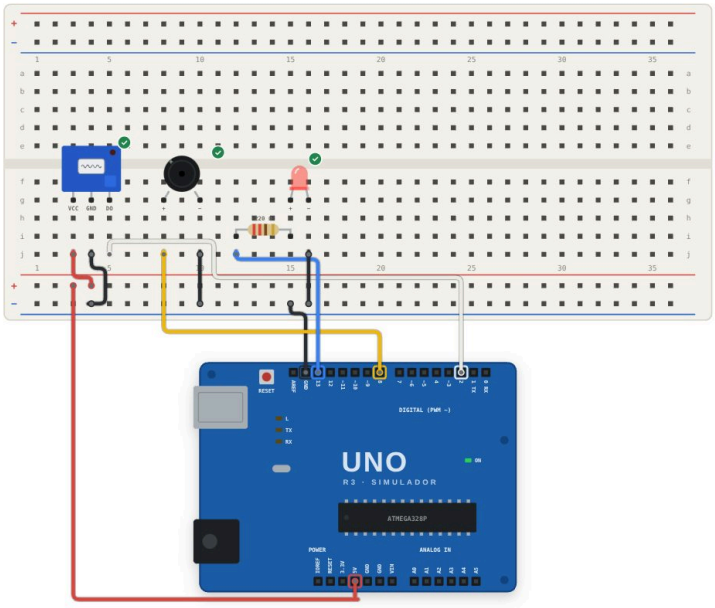
Protege una caja o vitrina: si alguien la golpea o mueve, suena la sirena.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de vibración
- 1 × Sirena (Zumbador)
- 1 × Alerta (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de vibración (Sensor de vibración)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Sirena (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Alerta (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 100 mA.
- Consumo estimado: **~100 mA** funcionando, con picos de **~100 mA**.

2 El programa

```ino
// Alarma de golpe o movimiento
const int VIBRACION = 2;     // DO: HIGH cuando hay vibración
const int SIRENA = 8;
const int ALERTA = 13;

void setup() {
  Serial.begin(9600);
  pinMode(VIBRACION, INPUT);
  pinMode(ALERTA, OUTPUT);
}

void loop() {
  if (digitalRead(VIBRACION) == HIGH) {
    Serial.println("¡Movieron la caja!");
    for (int i = 0; i < 6; i++) {
      digitalWrite(ALERTA, HIGH); tone(SIRENA, 1800); delay(150);
      digitalWrite(ALERTA, LOW);  tone(SIRENA, 1200); delay(150);
    }
    noTone(SIRENA);
  }
}
```

💡 ¿Cómo funciona?

El sensor de vibración tiene un resorte que hace contacto cuando lo sacuden, y su módulo entrega HIGH. Entonces el Arduino hace sonar seis ciclos de sirena.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay vibración o un golpe».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

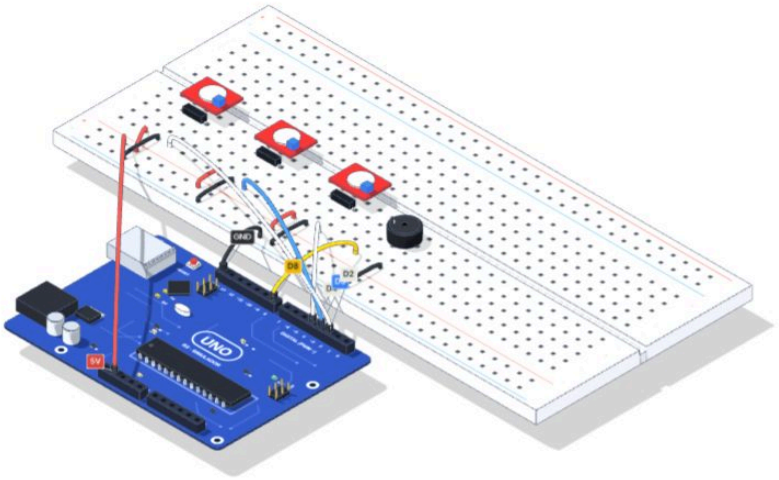
🏆 **Reto:** Agrega un interruptor para armar y desarmar la alarma.

PROYECTO 51 · SONIDO Y MÚSICA

Piano táctil de 3 notas

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

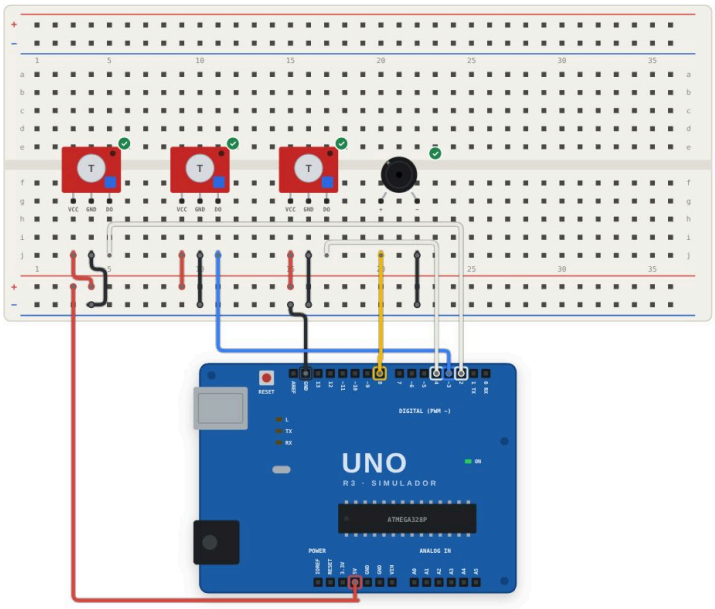
Tres sensores táctiles son las teclas de un pequeño piano.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Do (Sensor táctil TTP223)
- 1 × Mi (Sensor táctil TTP223)
- 1 × Sol (Sensor táctil TTP223)
- 1 × Parlante (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Do (Sensor táctil TTP223)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Mi (Sensor táctil TTP223)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **~D3**

4 Sol (Sensor táctil TTP223)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D4**

5 Parlante (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
int notas[] = {262, 330, 392};

void setup() {
  for (int i = 2; i <= 4; i++) pinMode(i, INPUT);
}

void loop() {
  bool suena = false;
  for (int i = 0; i < 3; i++) {
    if (digitalRead(2 + i) == HIGH) { tone(8, notas[i]); suena = true; }
  }
  if (!suena) noTone(8);
}
```

💡 ¿Cómo funciona?

El sensor táctil detecta el dedo sin necesidad de presionar. Cada sensor está asociado a una frecuencia del arreglo notas.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Lo están tocando».

Con piezas reales: sube el programa.

🏆 **Reto:** Agrega una cuarta tecla y toca una canción.

CAPÍTULO 4

Perillas y señales analógicas

El potenciómetro entrega un valor de 0 a 1023. Con map() lo convertimos en brillo, ángulos, tonos o velocidades.

EN ESTE CAPÍTULO · 12 PROYECTOS

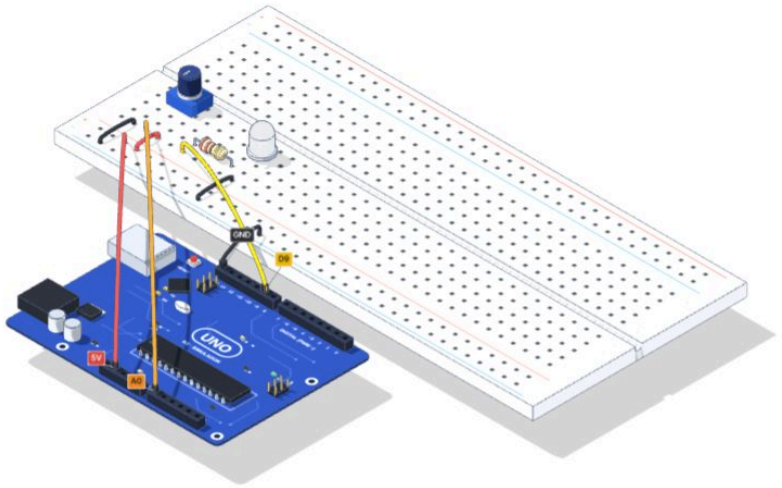
52 Brillo con perilla ★☆☆
53 Barra de nivel con LED ★★☆
54 Velocidad de parpadeo con perilla ★☆☆
55 Voltímetro en el monitor serie ★★☆
56 Mezclador de colores RGB ★★☆
57 Servo con perilla ★☆☆
58 Ventilador de velocidad variable ★★☆
59 Adivina el número con perilla ★★☆
60 Joystick que mueve una luz ★★☆
61 Brazo de dos servos con joystick ★★★
62 Luxómetro casero ★★☆
63 Termómetro de perilla para practicar ★☆☆

PROYECTO 52 · PERILLAS Y SEÑALES ANALÓGICAS

Brillo con perilla

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

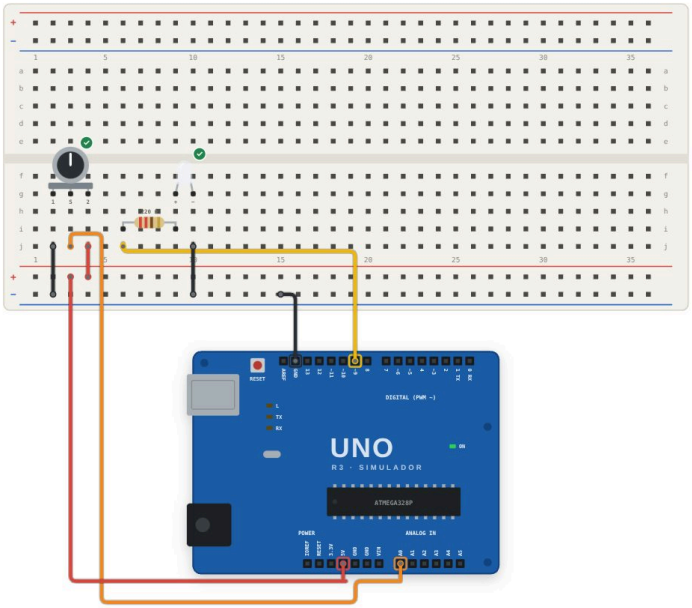
Al girar el potenciómetro el LED brilla más o menos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Perilla (Potenciómetro)
- 1 × Lámpara (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Perilla (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Lámpara (LED blanco)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 66 mA.
- Consumo estimado: **~66 mA** funcionando, con picos de **~66 mA**.

2 El programa

```
// Brillo con perilla
const int PERILLA = A0;
const int LAMPARA = 9;       // pin PWM (~)

void setup() {
  Serial.begin(9600);
  pinMode(LAMPARA, OUTPUT);
}

void loop() {
  int lectura = analogRead(PERILLA);          // 0 a 1023
  int brillo = map(lectura, 0, 1023, 0, 255); // 0 a 255
  analogWrite(LAMPARA, brillo);
  Serial.print("Lectura: "); Serial.print(lectura);
  Serial.print("  Brillo: "); Serial.println(brillo);
  delay(50);
}
```

💡 ¿Cómo funciona?

La entrada analógica entrega números de 0 a 1023, pero analogWrite() acepta de 0 a 255. La función map() hace esa conversión.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

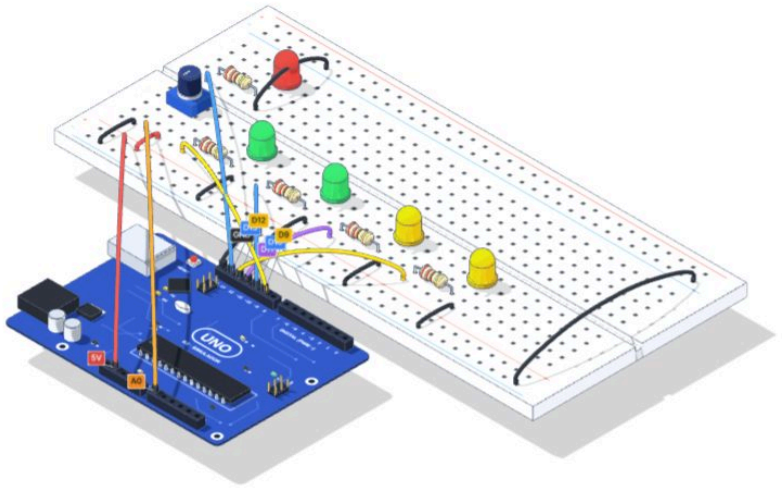
🏆 **Reto:** Haz que el LED se apague por completo cuando la perilla esté casi en cero.

PROYECTO 53 · PERILLAS Y SEÑALES ANALÓGICAS

Barra de nivel con LED

★★☆ Intermedio · Tiempo: 30 a 45 minutos

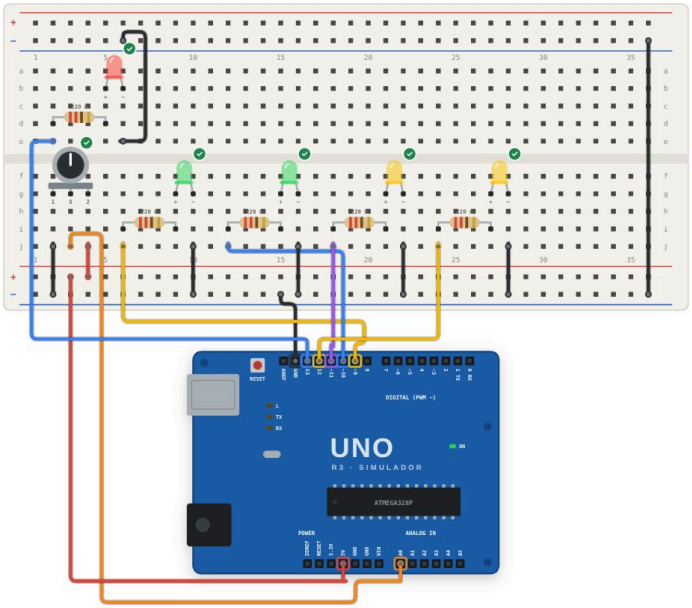
Cinco LED forman una barra que crece al girar la perilla, como un medidor de volumen.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Perilla (Potenciómetro)
- 1 × Nivel 1 (LED)
- 1 × Nivel 2 (LED)
- 1 × Nivel 3 (LED)
- 1 × Nivel 4 (LED)
- 1 × Nivel 5 (LED)
- 5 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Perilla (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Nivel 1 (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Nivel 2 (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Nivel 3 (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Nivel 4 (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 Nivel 5 (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

8 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 126 mA.
- Consumo estimado: **~126 mA** funcionando, con picos de **~126 mA**.

2 El programa

```
// Barra de nivel con 5 LED
const int PERILLA = A0;
int barra[] = {9, 10, 11, 12, 13};

void setup() {
  for (int i = 0; i < 5; i++) pinMode(barra[i], OUTPUT);
}

void loop() {
  int nivel = map(analogRead(PERILLA), 0, 1023, 0, 5);   // de 0 a 5 LED
  for (int i = 0; i < 5; i++) {
    if (i < nivel) digitalWrite(barra[i], HIGH);
    else digitalWrite(barra[i], LOW);
  }
  delay(30);
}
```

💡 **¿Cómo funciona?**

map() convierte la lectura en un número de LED a encender (0 a 5). El ciclo for enciende los que están por debajo de ese número y apaga los demás.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa.

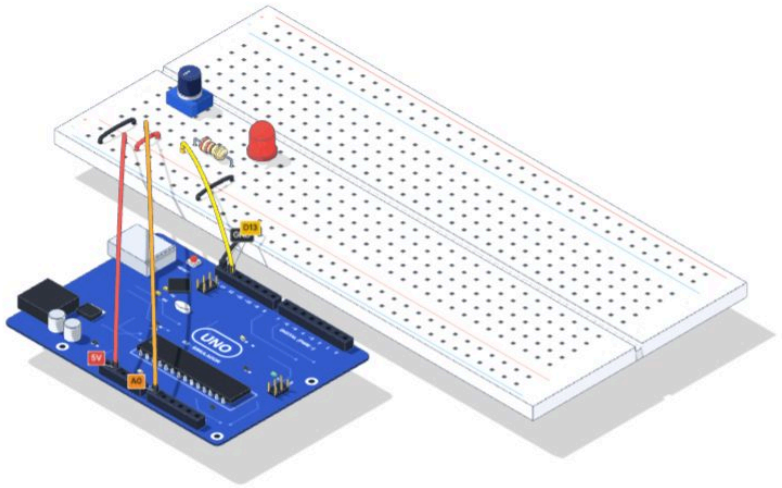
🏆 **Reto:** Haz que el LED rojo parpadee cuando la barra esté llena.

PROYECTO 54 · PERILLAS Y SEÑALES ANALÓGICAS

Velocidad de parpadeo con perilla

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

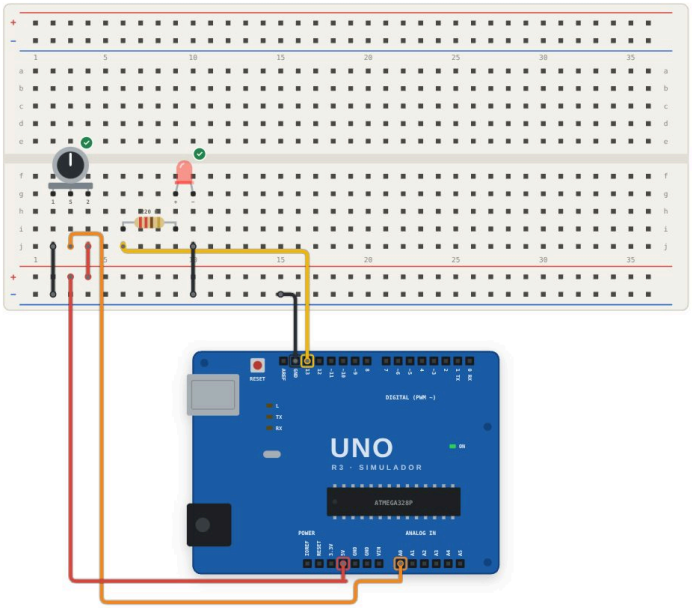
La perilla controla qué tan rápido parpadea el LED.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Velocidad (Potenciómetro)
- 1 × LED
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Velocidad (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 LED (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 66 mA.
- Consumo estimado: **~66 mA** funcionando, con picos de **~66 mA**.

2 El programa

```
// Velocidad de parpadeo ajustable
const int PERILLA = A0;
const int LED = 13;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  int espera = map(analogRead(PERILLA), 0, 1023, 50, 1000);
  digitalWrite(LED, HIGH);
  delay(espera);
  digitalWrite(LED, LOW);
  delay(espera);
}
```

💡 ¿Cómo funciona?

La lectura del potenciómetro se convierte en un tiempo de espera entre 50 y 1000 ms. Con la perilla al mínimo el LED parpadea muy rápido.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa.

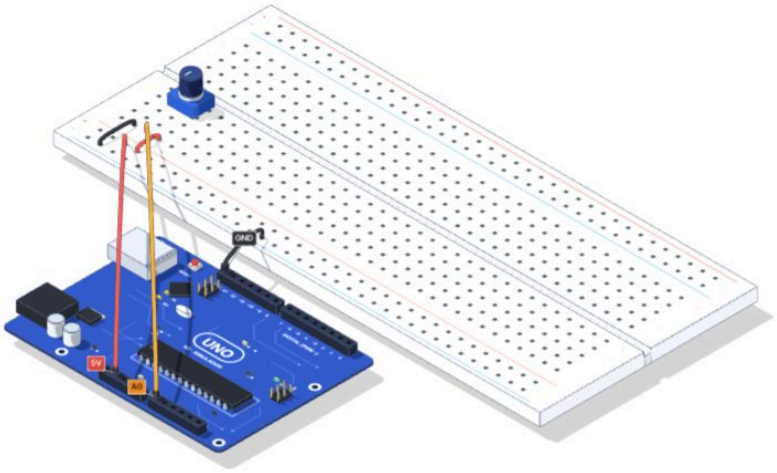
🏆 **Reto:** Invierte el sentido: que girar a la derecha haga el parpadeo más rápido.

PROYECTO 55 · PERILLAS Y SEÑALES ANALÓGICAS

Voltímetro en el monitor serie

★★☆ Intermedio · Tiempo: 30 a 45 minutos

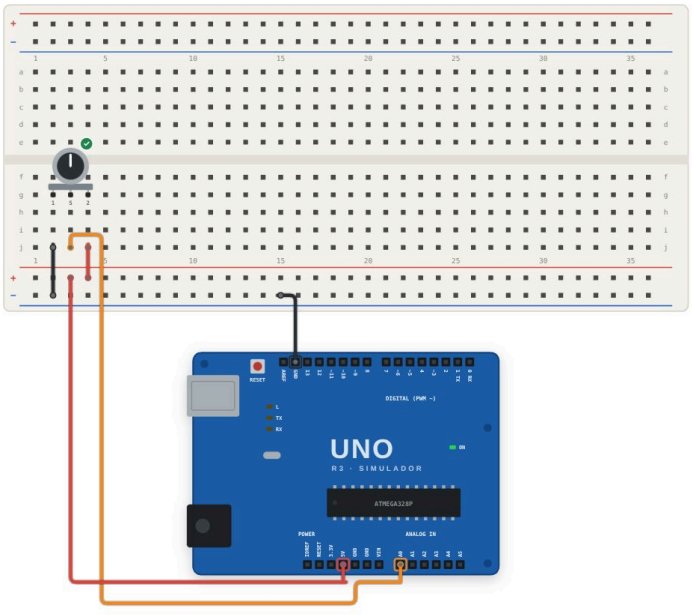
Mide el voltaje en la pata central del potenciómetro (de 0 a 5 V) y lo muestra en el monitor.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Divisor de voltaje (Potenciómetro)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Divisor de voltaje (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 51 mA.
- Consumo estimado: **~51 mA** funcionando, con picos de **~51 mA**.

2 El programa

```
// Voltímetro de 0 a 5 V
const int ENTRADA = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int lectura = analogRead(ENTRADA);
  float voltios = lectura * 5.0 / 1023.0;    // regla de tres
  Serial.print("Voltaje: ");
  Serial.print(voltios, 2);
  Serial.println(" V");
  delay(300);
}
```

💡 ¿Cómo funciona?

El Arduino divide los 5 V en 1024 pasos. Multiplicando la lectura por 5 y dividiendo por 1023 se obtiene el voltaje real con dos decimales.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

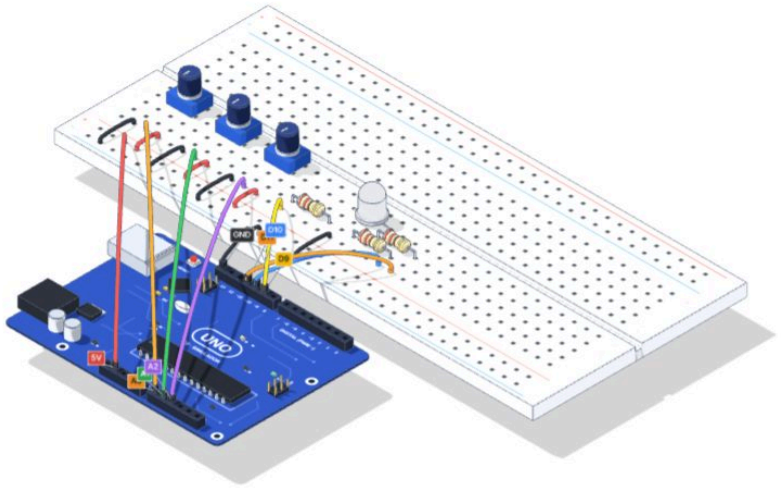
🏆 **Reto:** Agrega LED que avisen si el voltaje es menor de 1 V o mayor de 4 V.

PROYECTO 56 · PERILLAS Y SEÑALES ANALÓGICAS

Mezclador de colores RGB

★★☆ Intermedio · Tiempo: 30 a 45 minutos

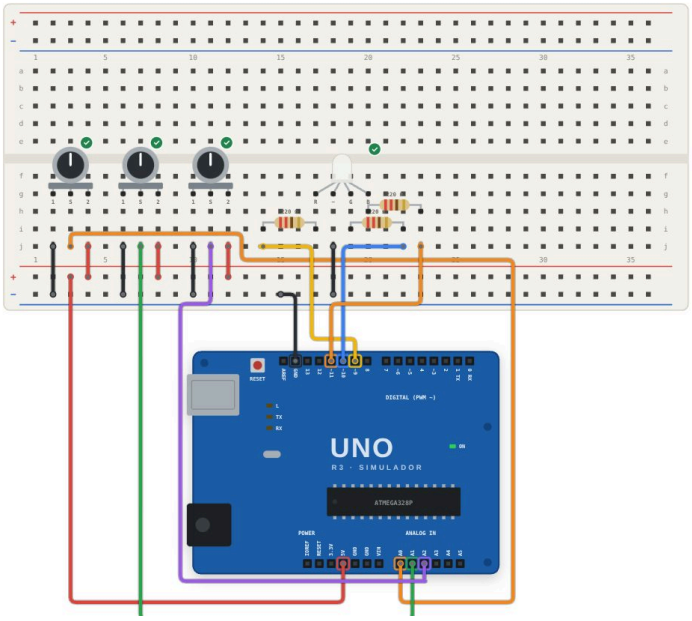
Tres perillas controlan el rojo, el verde y el azul de un LED RGB para crear cualquier color.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Rojo (Potenciómetro)
- 1 × Verde (Potenciómetro)
- 1 × Azul (Potenciómetro)
- 1 × LED RGB
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Rojo (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Verde (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A1**
- Extremo 2 (derecha) → riel **+** (5V)

4 Azul (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A2**
- Extremo 2 (derecha) → riel **+** (5V)

5 LED RGB (LED RGB)

- Pata R (rojo) → resistencia de **220 Ω** → pin **~D9**
- Pata – (común, la más larga) → riel **–** (GND)
- Pata G (verde) → resistencia de **220 Ω** → pin **~D10**
- Pata B (azul) → resistencia de **220 Ω** → pin **~D11**

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 98 mA.
- Consumo estimado: **~98 mA** funcionando, con picos de **~98 mA**.

2 El programa

```ino
// Mezclador de colores con tres perillas
const int R = 9, G = 10, B = 11;

void setup() {
  Serial.begin(9600);
  pinMode(R, OUTPUT); pinMode(G, OUTPUT); pinMode(B, OUTPUT);
}

void loop() {
  int r = analogRead(A0) / 4;     // de 0-1023 a 0-255
  int g = analogRead(A1) / 4;
  int b = analogRead(A2) / 4;
  analogWrite(R, r); analogWrite(G, g); analogWrite(B, b);
  Serial.print("R="); Serial.print(r);
  Serial.print(" G="); Serial.print(g);
  Serial.print(" B="); Serial.println(b);
  delay(100);
}
```

💡 ¿Cómo funciona?

Dividir la lectura entre 4 es otra forma de pasar de 0–1023 a 0–255. Cada perilla controla un color y la mezcla aparece en el LED RGB.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

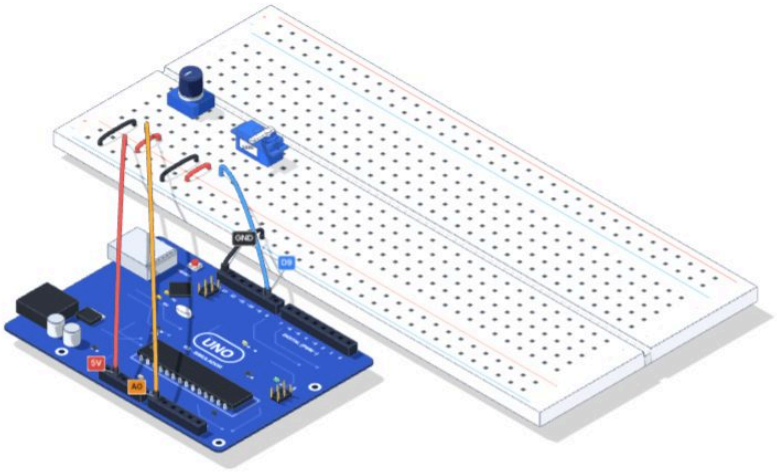
🏆 **Reto:** Busca la combinación que da el color naranja y anótala.

PROYECTO 57 · PERILLAS Y SEÑALES ANALÓGICAS

Servo con perilla

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

El brazo del servo sigue el giro de la perilla, de 0 a 180 grados.

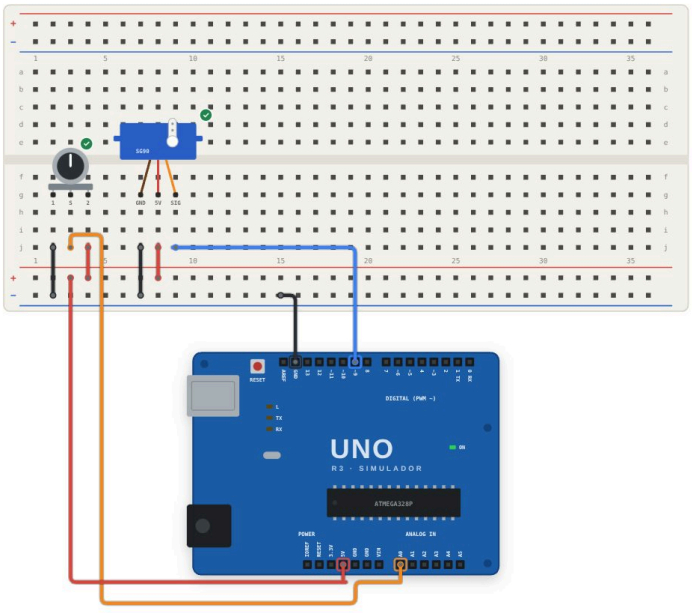


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Perilla (Potenciómetro)
- 1 × Servo (Servo SG90)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Perilla (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Servo (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 201 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~201 mA** funcionando, con picos de **~701 mA**.

2 El programa

```
#include <Servo.h>
Servo brazo;

void setup() {
  Serial.begin(9600);
  brazo.attach(9);           // señal del servo en D9
}

void loop() {
  int angulo = map(analogRead(A0), 0, 1023, 0, 180);
  brazo.write(angulo);
  Serial.print("Ángulo: ");
  Serial.println(angulo);
  delay(20);
}
```

💡 ¿Cómo funciona?

La librería Servo genera los pulsos que entiende el servo. write() recibe un ángulo de 0 a 180 grados, que obtenemos con map() desde la perilla.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

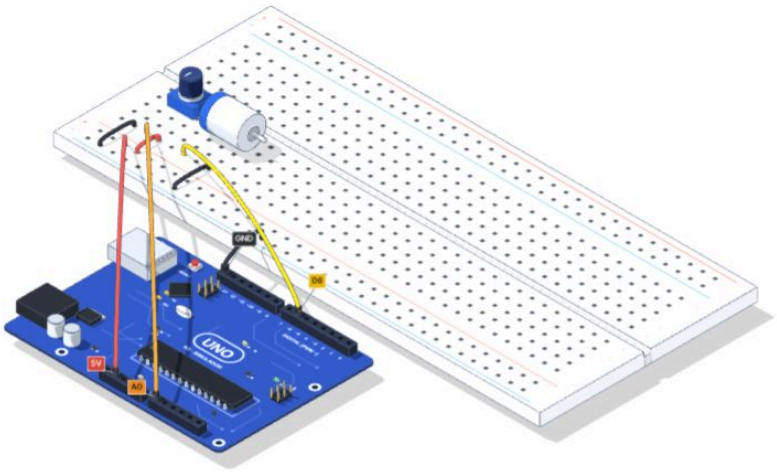
🏆 **Reto:** Limita el movimiento entre 45 y 135 grados.

PROYECTO 58 · PERILLAS Y SEÑALES ANALÓGICAS

Ventilador de velocidad variable

★★☆ Intermedio · Tiempo: 30 a 45 minutos

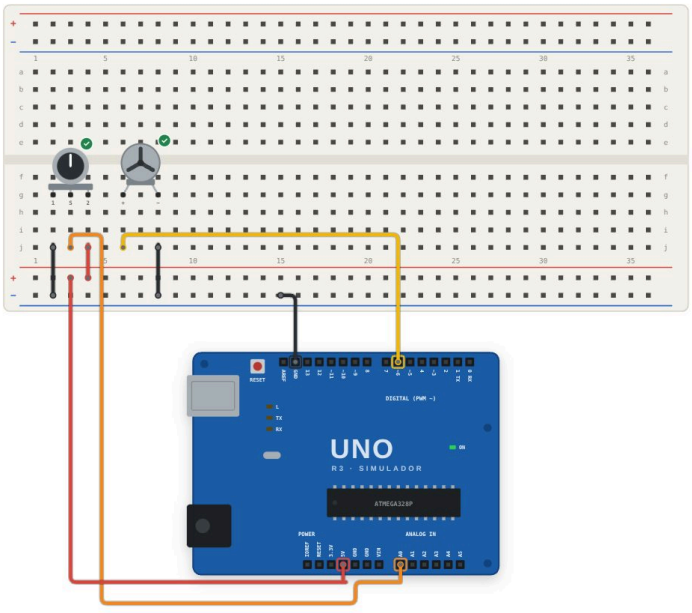
La perilla regula la velocidad de un motor de ventilador.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Velocidad (Potenciómetro)
- 1 × Ventilador (Motor DC)
- Transistor TIP120 (o 2N2222)
- Diodo 1N4007
- Resistencia 1 kΩ
- Fuente 6–12 V para los motores
- Transistor TIP120 y diodo 1N4007
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Velocidad (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Ventilador (Motor DC)

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

4 Para el proyecto real

- En la vida real el motor se conecta con un transistor (por ejemplo TIP120) y un diodo, porque un pin solo da 40 mA.

5 Energía y controladores

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motor → necesita transistor:** un pin da solo 20–40 mA y el motor pide 150–250 mA (hasta 1 A al arrancar). Conexión: pin → resistencia de 1 kΩ → base del **TIP120**; motor entre el + de la fuente y el colector; emisor a GND. Pon un **diodo 1N4007** en paralelo con el motor (la raya hacia el +) para frenar los picos de voltaje que dañan el transistor.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al + del motor (el – del motor va al colector del transistor). Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~251 mA** funcionando, con picos de **~1051 mA**.

2 El programa

```
// Ventilador de velocidad variable
const int PERILLA = A0;
const int MOTOR = 6;          // pin PWM (~)

void setup() {
  Serial.begin(9600);
  pinMode(MOTOR, OUTPUT);
}

void loop() {
  int velocidad = map(analogRead(PERILLA), 0, 1023, 0, 255);
  analogWrite(MOTOR, velocidad);
  int porcentaje = velocidad * 100 / 255;
  Serial.print("Velocidad: ");
  Serial.print(porcentaje);
  Serial.println(" %");
  delay(100);
}
```

💡 ¿Cómo funciona?

Con PWM el motor recibe pulsos de energía: con más tiempo encendido gira más rápido. Es el mismo principio del brillo del LED.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- En la vida real el motor se conecta con un transistor (por ejemplo TIP120) y un diodo, porque un pin solo da 40 mA.

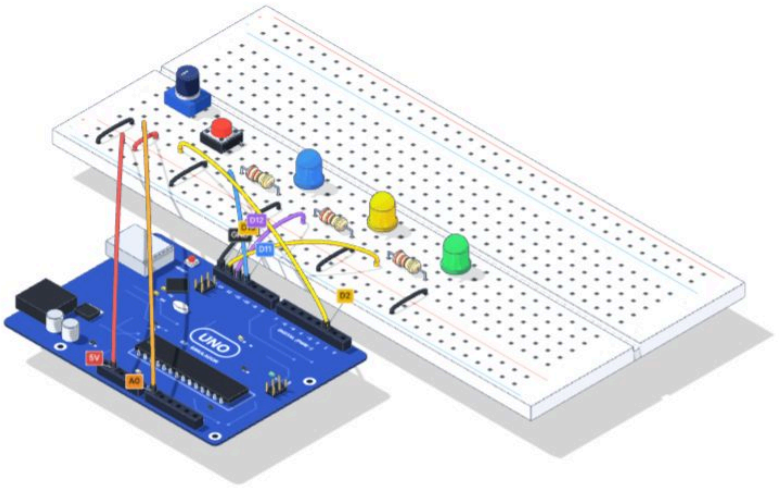
🏆 **Reto:** Agrega un botón para apagar el ventilador de inmediato.

PROYECTO 59 · PERILLAS Y SEÑALES ANALÓGICAS

Adivina el número con perilla

★★☆ Intermedio · Tiempo: 30 a 45 minutos

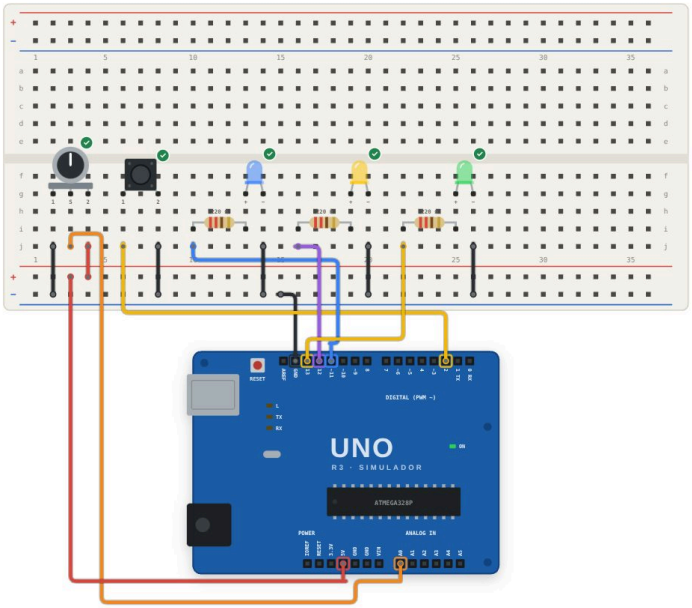
El Arduino piensa un número del 0 al 10; gira la perilla y los LED te dicen si estás frío o caliente.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Tu número (Potenciómetro)
- 1 × Probar (Botón)
- 1 × Más alto (LED)
- 1 × Más bajo (LED)
- 1 × ¡Correcto! (LED)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Tu número (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Probar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Más alto (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Más bajo (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 ¡Correcto! (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 96 mA.
- Consumo estimado: **~96 mA** funcionando, con picos de **~96 mA**.

2 El programa

```
// Adivina el número secreto
const int BOTON = 2;
const int SUBE = 11, BAJA = 12, BIEN = 13;
int secreto;

void setup() {
  Serial.begin(9600);
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(SUBE, OUTPUT); pinMode(BAJA, OUTPUT); pinMode(BIEN, OUTPUT);
  secreto = random(0, 11);
  Serial.println("Pensé un número del 0 al 10");
}

void loop() {
  int intento = map(analogRead(A0), 0, 1023, 0, 10);
  if (digitalRead(BOTON) == LOW) {
    Serial.print("Probaste el ");
    Serial.println(intento);
    digitalWrite(SUBE, LOW); digitalWrite(BAJA, LOW);
    if (intento < secreto) digitalWrite(SUBE, HIGH);
    else if (intento > secreto) digitalWrite(BAJA, HIGH);
    else {
      Serial.println("¡Adivinaste!");
```

```
      for (int i = 0; i < 5; i++) { digitalWrite(BIEN, HIGH); delay(150); digitalWrite(BIEN, LOW);
delay(150); }
      secreto = random(0, 11);
    }
    delay(400);
  }
}
```

💡 ¿Cómo funciona?

El número secreto se elige con random() al iniciar. Al presionar el botón se compara tu intento: el LED azul pide subir y el amarillo pide bajar.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.
- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

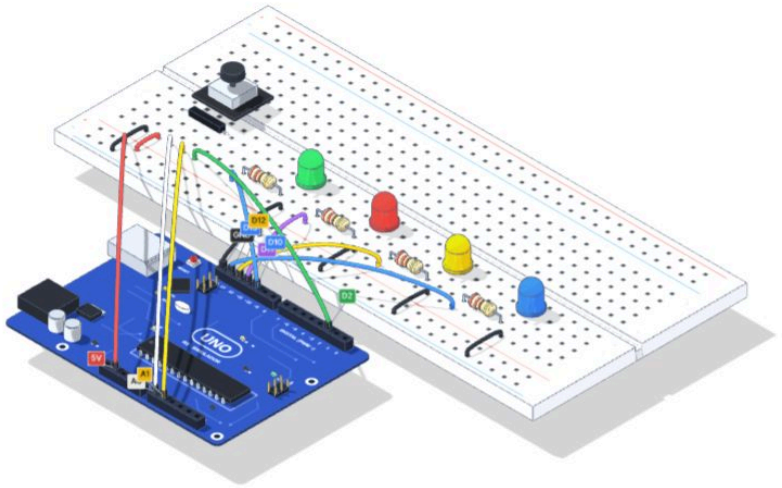
🏆 **Reto:** Cuenta cuántos intentos necesitaste.

PROYECTO 60 · PERILLAS Y SEÑALES ANALÓGICAS

Joystick que mueve una luz

★★☆ Intermedio · Tiempo: 30 a 45 minutos

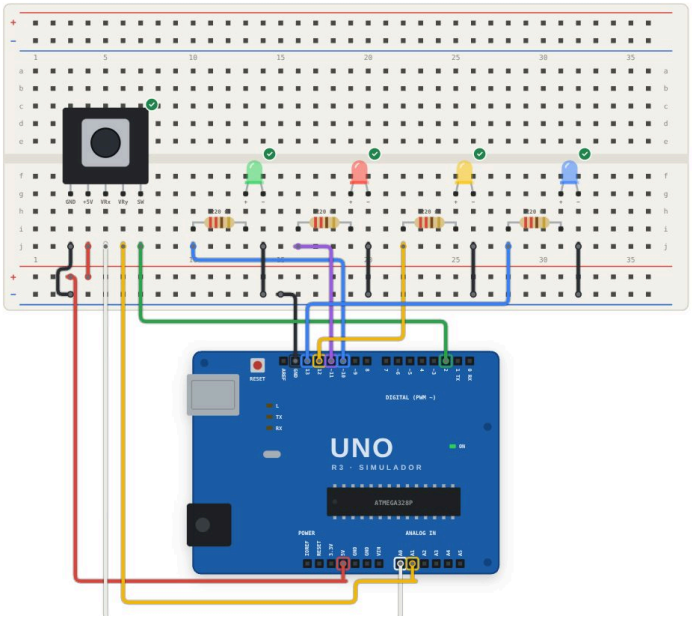
Cuatro LED muestran hacia dónde mueves el joystick: arriba, abajo, izquierda o derecha.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Joystick
- 1 × Arriba (LED)
- 1 × Abajo (LED)
- 1 × Izquierda (LED)
- 1 × Derecha (LED)
- 4 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Joystick (Joystick)

- GND → riel **–** (GND)
- +5V → riel **+** (5V)
- VRx (eje X) → pin **A0**
- VRy (eje Y) → pin **A1**
- SW (botón) → pin **D2**

3 Arriba (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Abajo (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Izquierda (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Derecha (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 111 mA.
- Consumo estimado: **~111 mA** funcionando, con picos de **~111 mA**.

2 El programa

```
// Joystick que enciende la luz de su dirección
const int ARRIBA = 10, ABAJO = 11, IZQ = 12, DER = 13;

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP);           // botón del joystick
  pinMode(ARRIBA, OUTPUT); pinMode(ABAJO, OUTPUT); pinMode(IZQ, OUTPUT); pinMode(DER, OUTPUT);
}

void loop() {
  int x = analogRead(A0);             // 0 izquierda, 512 centro, 1023 derecha
  int y = analogRead(A1);             // 0 abajo, 512 centro, 1023 arriba
  digitalWrite(DER, x > 700);
  digitalWrite(IZQ, x < 300);
  digitalWrite(ARRIBA, y > 700);
  digitalWrite(ABAJO, y < 300);
  if (digitalRead(2) == LOW) Serial.println("¡Botón del joystick!");
  delay(50);
}
```

💡 **¿Cómo funciona?**

El joystick tiene dos potenciómetros: uno por eje. En el centro cada uno lee cerca de 512. Si un eje pasa de 700 o baja de 300, se enciende el LED de esa dirección.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Mueve los deslizadores X y Y del joystick y presiona su botón.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

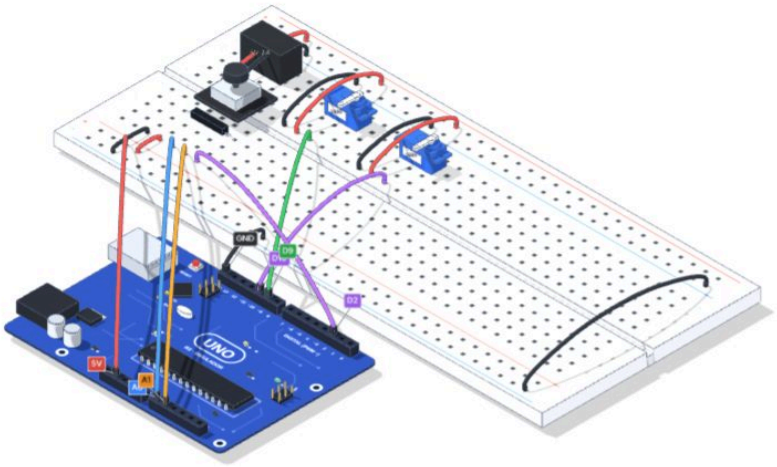
🏆 **Reto:** Haz que el botón del joystick encienda los cuatro LED a la vez.

PROYECTO 61 · PERILLAS Y SEÑALES ANALÓGICAS

Brazo de dos servos con joystick

★★★ Avanzado · Tiempo: 45 a 90 minutos

Un joystick mueve dos servos: uno gira a los lados y el otro sube y baja, como una cámara de vigilancia.

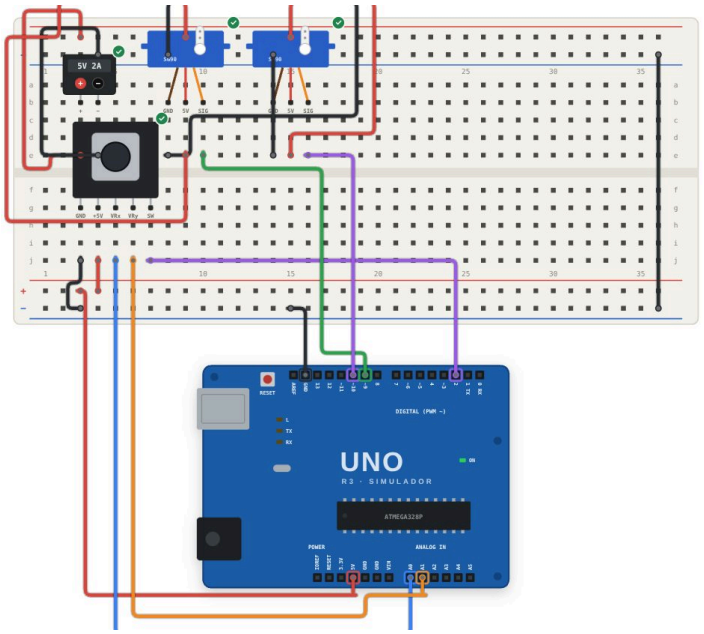


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Joystick
- 1 × Servo base (Servo SG90)
- 1 × Servo inclinación (Servo SG90)
- 1 × Fuente 5 V para los servos (Fuente 5 V externa)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Joystick (Joystick)

- GND → riel **–** (GND)
- +5V → riel **+** (5V)
- VRx (eje X) → pin **A0**
- VRy (eje Y) → pin **A1**
- SW (botón) → pin **D2**

3 Servo base (Servo SG90)

- Cable café (GND) → riel **– de arriba** (unido al GND del Arduino)
- Cable rojo (5V) → riel **+ de arriba**, que alimenta la fuente externa
- Cable naranja (señal) → pin **~D9**

4 Servo inclinación (Servo SG90)

- Cable café (GND) → riel **– de arriba** (unido al GND del Arduino)
- Cable rojo (5V) → riel **+ de arriba**, que alimenta la fuente externa
- Cable naranja (señal) → pin **~D10**

5 Fuente 5 V para los servos (Fuente 5 V externa)

- Borne + (rojo) → riel **+ de arriba**, solo para los servos (no lo unas al 5V del Arduino)
- Borne – (negro) → riel **– de arriba**, y ese riel unido con un cable al riel – de abajo, que va al GND del Arduino (GND común)

6 Para el proyecto real

- Dos servos pueden pedir más corriente de la que da el USB: aliméntalos con una fuente externa de 5 V y une los GND.

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 51 mA.
- **Servos → fuente externa:** dos servos pueden pedir más de 1 A y el USB solo da 500 mA: el Arduino se reiniciaría. Alimenta el cable rojo de los servos con una **fuente de 5 V y 2 A** (o 4 pilas AA); la señal (naranja) sigue en el Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~351 mA** funcionando, con picos de **~1351 mA**.

2 El programa

```
#include <Servo.h>
Servo base, inclinacion;
int anguloX = 90, anguloY = 90;

void setup() {
  base.attach(9);
  inclinacion.attach(10);
  pinMode(2, INPUT_PULLUP);
}

void loop() {
  int x = analogRead(A0), y = analogRead(A1);
  if (x > 700) anguloX += 2;          // mover poco a poco
  if (x < 300) anguloX -= 2;
  if (y > 700) anguloY += 2;
  if (y < 300) anguloY -= 2;
  if (digitalRead(2) == LOW) { anguloX = 90; anguloY = 90; }   // botón: volver al centro
  anguloX = constrain(anguloX, 0, 180);
  anguloY = constrain(anguloY, 20, 160);
  base.write(anguloX);
  inclinacion.write(anguloY);
```

```
  delay(20);
}
```

💡 ¿Cómo funciona?

En lugar de copiar la posición del joystick, cada servo avanza 2 grados mientras la palanca esté inclinada. Así el movimiento es suave y la posición se mantiene al soltar.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve los deslizadores X y Y del joystick y presiona su botón.

Con piezas reales: sube el programa.

⚠️ Ten en cuenta

- Dos servos pueden pedir más corriente de la que da el USB: aliméntalos con una fuente externa de 5 V y une los GND.

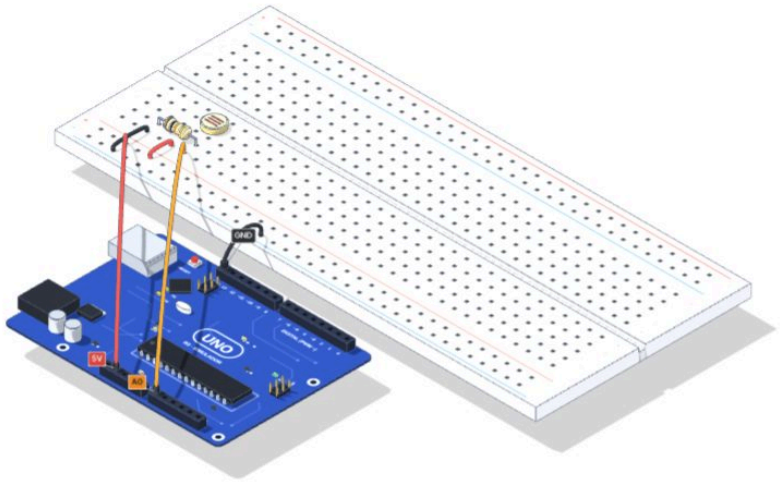
🏆 **Reto:** Agrega un LED que se encienda cuando algún servo llegue a su límite.

PROYECTO 62 · PERILLAS Y SEÑALES ANALÓGICAS

Luxómetro casero

★★☆ Intermedio · Tiempo: 30 a 45 minutos

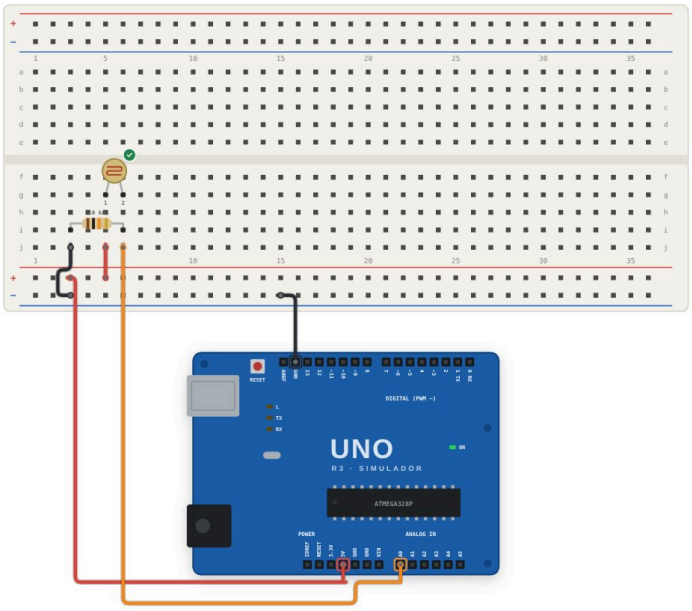
Mide la luz del lugar y la clasifica: oscuro, penumbra, normal o muy brillante.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de luz (Fotorresistencia)
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de luz (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 51 mA.
- Consumo estimado: **~51 mA** funcionando, con picos de **~51 mA**.

2 El programa

```
// Luxómetro casero
const int LUZ = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int nivel = analogRead(LUZ);
  int porcentaje = map(nivel, 0, 1023, 0, 100);
  Serial.print("Luz: ");
  Serial.print(porcentaje);
  Serial.print(" % -> ");
  if (porcentaje < 20) Serial.println("Oscuro");
  else if (porcentaje < 45) Serial.println("Penumbra");
  else if (porcentaje < 80) Serial.println("Normal");
  else Serial.println("Muy brillante");
  delay(500);
}
```

💡 ¿Cómo funciona?

Una cadena de if y else if compara la luz con varios límites, de menor a mayor, y escribe la categoría que corresponde.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

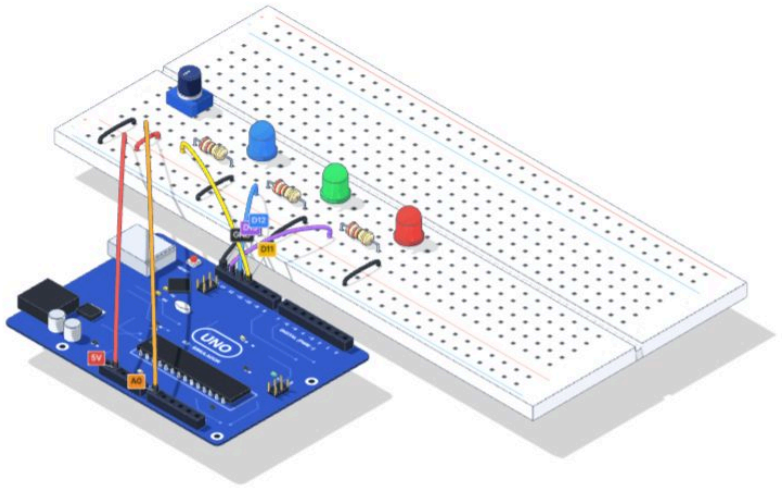
🏆 **Reto:** Mide la luz de tu salón a distintas horas del día y anótala.

PROYECTO 63 · PERILLAS Y SEÑALES ANALÓGICAS

Termómetro de perilla para practicar

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

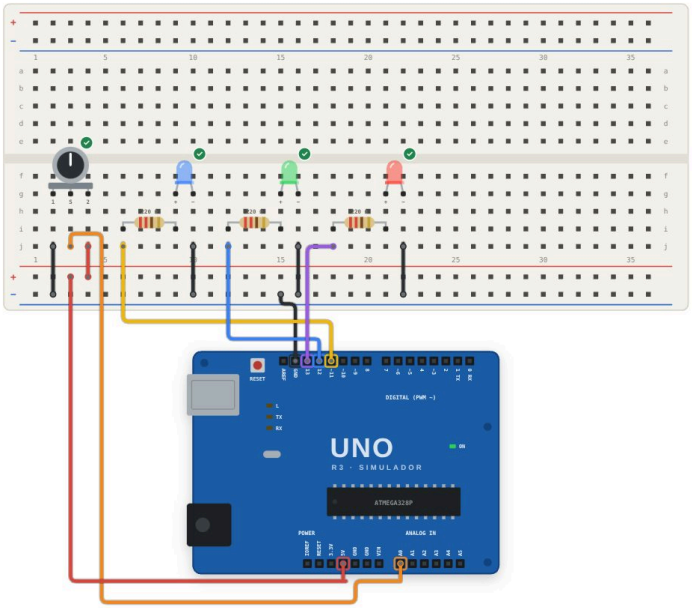
Usa la perilla como si fuera un sensor de temperatura para practicar decisiones con if.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Temperatura simulada (Potenciómetro)
- 1 × Frío (LED)
- 1 × Normal (LED)
- 1 × Calor (LED)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Temperatura simulada (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Frío (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Normal (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Calor (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 96 mA.
- Consumo estimado: **~96 mA** funcionando, con picos de **~96 mA**.

2 El programa

```
// Practicar decisiones: perilla como "temperatura"
const int FRIO = 11, NORMAL = 12, CALOR = 13;

void setup() {
  Serial.begin(9600);
  pinMode(FRIO, OUTPUT); pinMode(NORMAL, OUTPUT); pinMode(CALOR, OUTPUT);
}

void loop() {
  int temperatura = map(analogRead(A0), 0, 1023, 0, 40);   // de 0 a 40 °C
  digitalWrite(FRIO, temperatura < 15);
  digitalWrite(NORMAL, temperatura >= 15 && temperatura <= 28);
  digitalWrite(CALOR, temperatura > 28);
  Serial.print(temperatura);
  Serial.println(" °C");
  delay(200);
}
```

💡 ¿Cómo funciona?

Una comparación como temperatura < 15 vale 1 (verdadero) o 0 (falso), así que se puede usar directo en digitalWrite(). El operador && significa "y".

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

🏆 **Reto:** Cambia los límites para un lugar de clima frío.

CAPÍTULO 5

Luz y oscuridad

La fotorresistencia (LDR) le permite al Arduino saber si es de día o de noche.

EN ESTE CAPÍTULO · 8 PROYECTOS

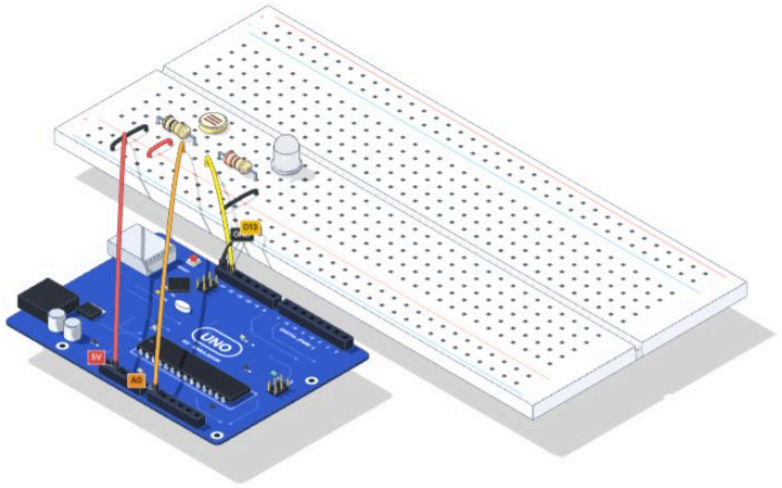
64 Interruptor crepuscular ★☆☆
65 Alarma de cajón abierto ★☆☆
66 Lámpara de lectura automática ★★☆
67 Medidor de luz con barra de LED ★★☆
68 Detector de día y noche en pantalla ★★☆
69 Barrera láser cuenta-objetos ★★☆
70 Girasol de un servo ★★★
71 Luz nocturna con colores ★★☆

PROYECTO 64 · LUZ Y OSCURIDAD

Interruptor crepuscular

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

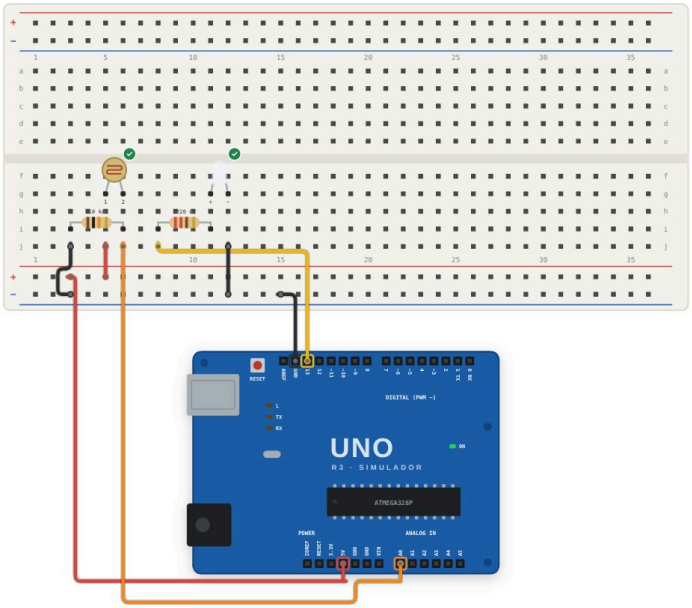
Una lámpara se enciende sola cuando oscurece, como el alumbrado de un parque.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Fotocelda (Fotorresistencia)
- 1 × Lámpara (LED)
- 1 × Resistencia 220 Ω
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Fotocelda (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Lámpara (LED blanco)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 66 mA.
- Consumo estimado: **~66 mA** funcionando, con picos de **~66 mA**.

2 El programa

```
// Interruptor crepuscular
const int FOTOCELDA = A0;
const int LAMPARA = 13;
const int UMBRAL = 350;       // por debajo de esto es "de noche"

void setup() {
  Serial.begin(9600);
  pinMode(LAMPARA, OUTPUT);
}

void loop() {
  int luz = analogRead(FOTOCELDA);
  Serial.println(luz);
  if (luz < UMBRAL) digitalWrite(LAMPARA, HIGH);
  else digitalWrite(LAMPARA, LOW);
  delay(200);
}
```

💡 ¿Cómo funciona?

La fotocelda forma un divisor de voltaje con la resistencia de 10 kΩ: con poca luz el valor leído baja. Si baja del umbral, se enciende la lámpara.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

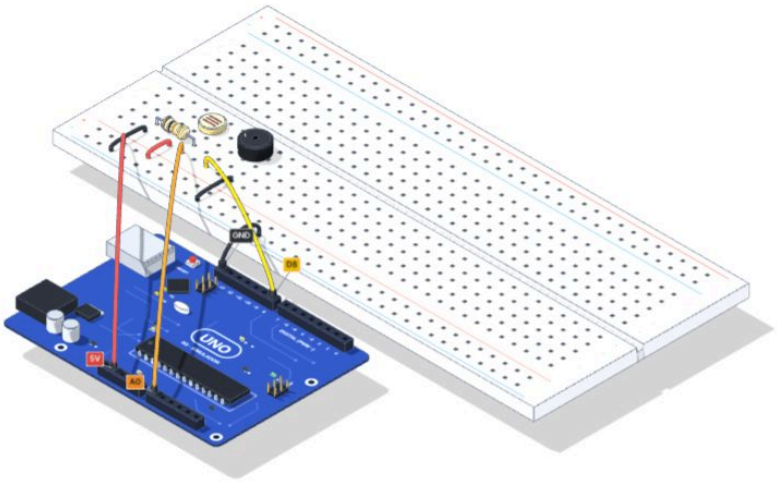
🏆 **Reto:** Agrega un margen para que la lámpara no titile cuando la luz está justo en el límite.

PROYECTO 65 · LUZ Y OSCURIDAD

Alarma de cajón abierto

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

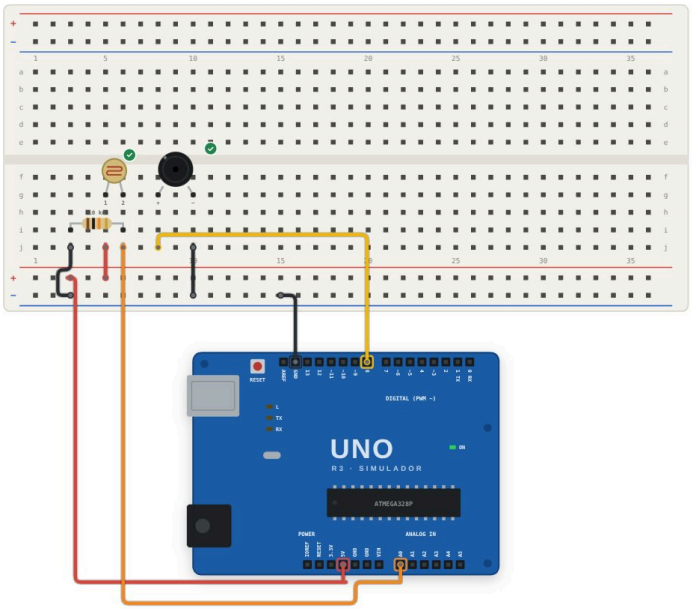
Dentro de un cajón cerrado está oscuro: si alguien lo abre y entra luz, suena la alarma.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de luz (Fotorresistencia)
- 1 × Alarma (Zumbador)
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de luz (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 81 mA.
- Consumo estimado: **~81 mA** funcionando, con picos de **~81 mA**.

2 El programa

```
// Alarma de cajón abierto
const int LUZ = A0;
const int ALARMA = 8;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int luz = analogRead(LUZ);
  if (luz > 500) {                    // entró luz: alguien abrió
    Serial.println("¡Abrieron el cajón!");
    tone(ALARMA, 2000, 200);
    delay(300);
  }
  delay(100);
}
```

💡 ¿Cómo funciona?

El programa aprovecha que dentro de un cajón cerrado no hay luz. Cuando la lectura sube por encima de 500, significa que lo abrieron.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

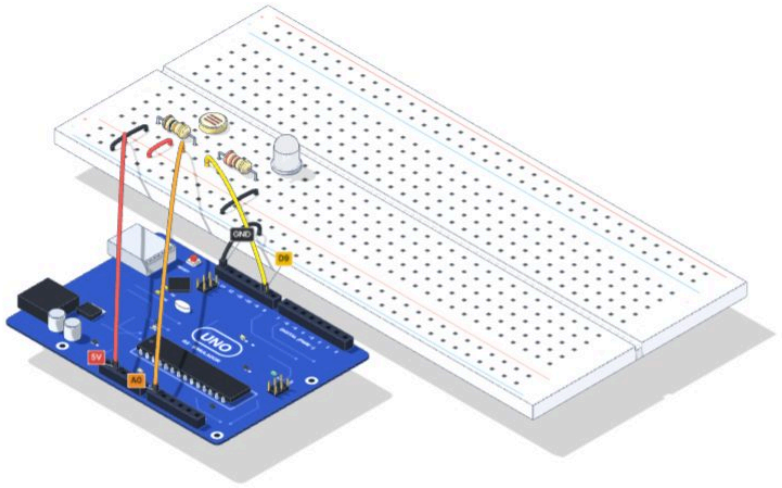
🏆 **Reto:** Cuenta cuántas veces abrieron el cajón durante el día.

PROYECTO 66 · LUZ Y OSCURIDAD

Lámpara de lectura automática

★★☆ Intermedio · Tiempo: 30 a 45 minutos

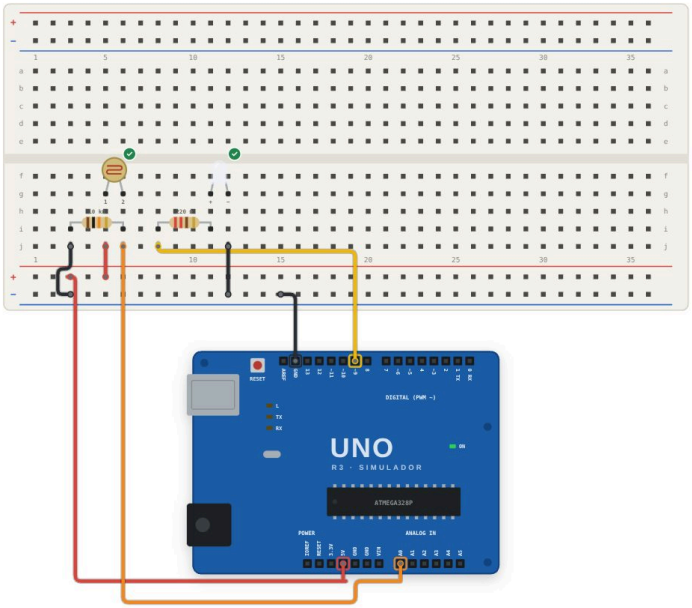
El LED da más luz cuando el lugar está más oscuro, para leer siempre con la misma iluminación.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de luz (Fotorresistencia)
- 1 × Lámpara (LED)
- 1 × Resistencia 220 Ω
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de luz (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Lámpara (LED blanco)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 66 mA.
- Consumo estimado: **~66 mA** funcionando, con picos de **~66 mA**.

2 El programa

```
// Lámpara que compensa la oscuridad
const int LUZ = A0;
const int LAMPARA = 9;        // pin PWM (~)

void setup() {
  pinMode(LAMPARA, OUTPUT);
}

void loop() {
  int ambiente = analogRead(LUZ);
  int brillo = map(ambiente, 0, 1023, 255, 0);   // al revés: menos luz, más brillo
  analogWrite(LAMPARA, brillo);
  delay(50);
}
```

💡 ¿Cómo funciona?

Fíjate que en map() el rango de salida va de 255 a 0: así, cuando el ambiente tiene poca luz (número bajo), el LED recibe un brillo alto.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa.

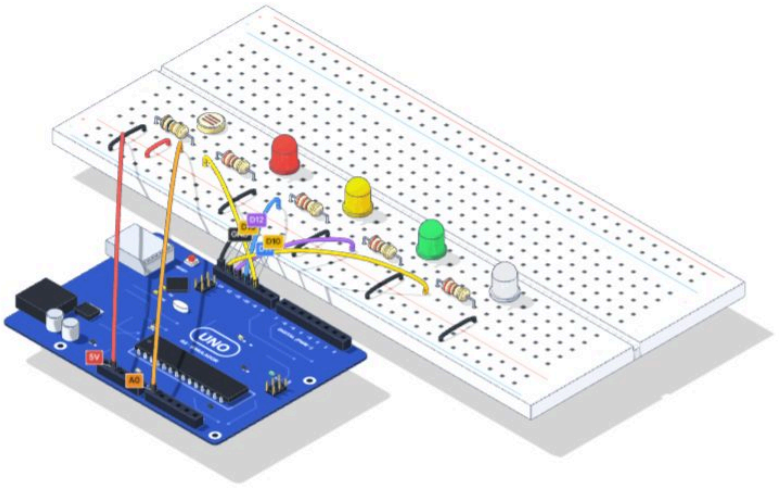
🏆 **Reto:** Pon un brillo mínimo para que la lámpara nunca se apague del todo.

PROYECTO 67 · LUZ Y OSCURIDAD

Medidor de luz con barra de LED

★★☆ Intermedio · Tiempo: 30 a 45 minutos

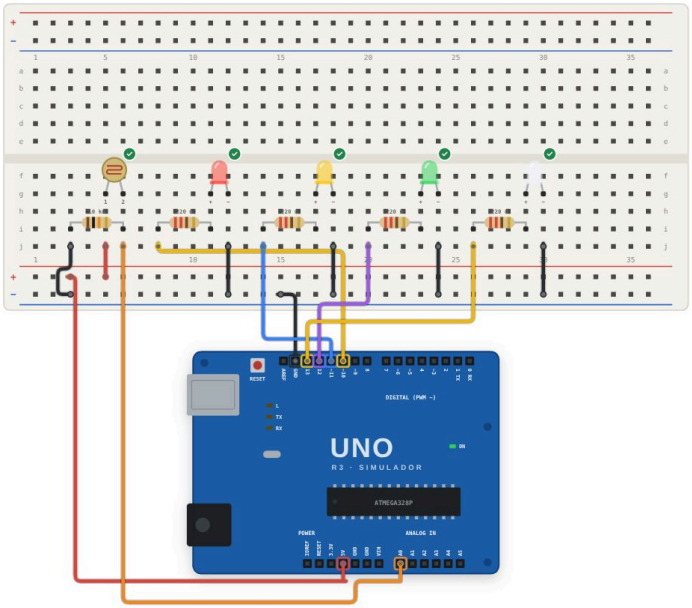
Cuatro LED muestran cuánta luz hay: más luz, más LED encendidos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de luz (Fotorresistencia)
- 1 × Poca (LED)
- 1 × Media (LED)
- 1 × Buena (LED)
- 1 × Mucha (LED)
- 4 × Resistencia 220 Ω
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de luz (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Poca (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Media (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Buena (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Mucha (LED blanco)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 111 mA.
- Consumo estimado: **~111 mA** funcionando, con picos de **~111 mA**.

2 El programa

```
// Medidor de luz con 4 LED
int barra[] = {10, 11, 12, 13};

void setup() {
  for (int i = 0; i < 4; i++) pinMode(barra[i], OUTPUT);
}

void loop() {
  int n = map(analogRead(A0), 0, 1023, 0, 4);
  for (int i = 0; i < 4; i++) digitalWrite(barra[i], i < n);
  delay(100);
}
```

💡 ¿Cómo funciona?

Igual que la barra de nivel con perilla, pero ahora la entrada es la luz. El mismo programa sirve para distintos sensores: eso es lo bueno de las entradas analógicas.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa.

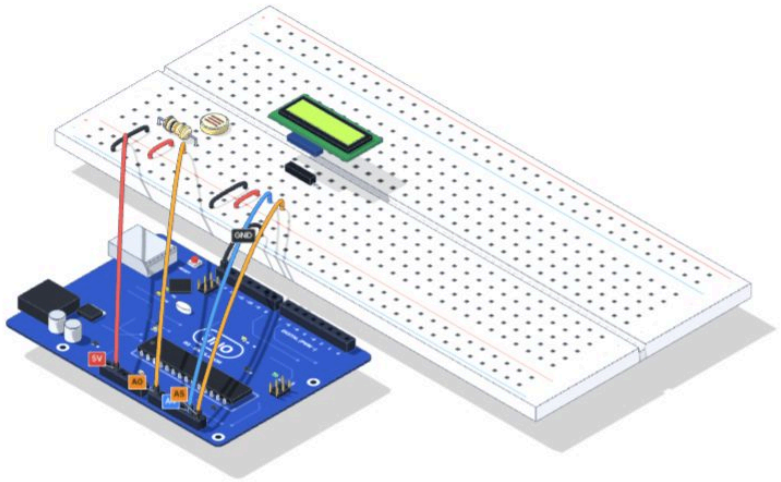
🏆 **Reto:** Agrega un zumbador que suene si hay demasiada luz.

PROYECTO 68 · LUZ Y OSCURIDAD

Detector de día y noche en pantalla

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Una pantalla LCD dice si es de día o de noche y muestra el nivel de luz.

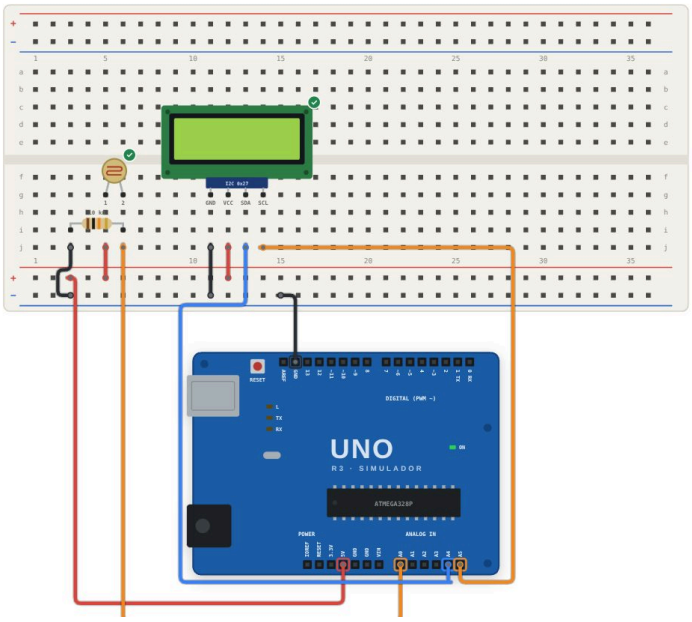


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de luz (Fotorresistencia)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de luz (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 81 mA.
- Consumo estimado: **~81 mA** funcionando, con picos de **~81 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  lcd.init();
  lcd.backlight();
}

void loop() {
  int luz = analogRead(A0);
  lcd.clear();
  lcd.setCursor(0, 0);
  if (luz > 400) lcd.print("Es de dia :)");
  else lcd.print("Es de noche");
  lcd.setCursor(0, 1);
  lcd.print("Luz: ");
  lcd.print(map(luz, 0, 1023, 0, 100));
  lcd.print(" %");
  delay(500);
}
```

💡 ¿Cómo funciona?

La pantalla LCD con módulo I2C solo necesita 4 cables. setCursor(columna, fila) elige dónde escribir: la fila 0 es la de arriba y la fila 1 la de abajo.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa.

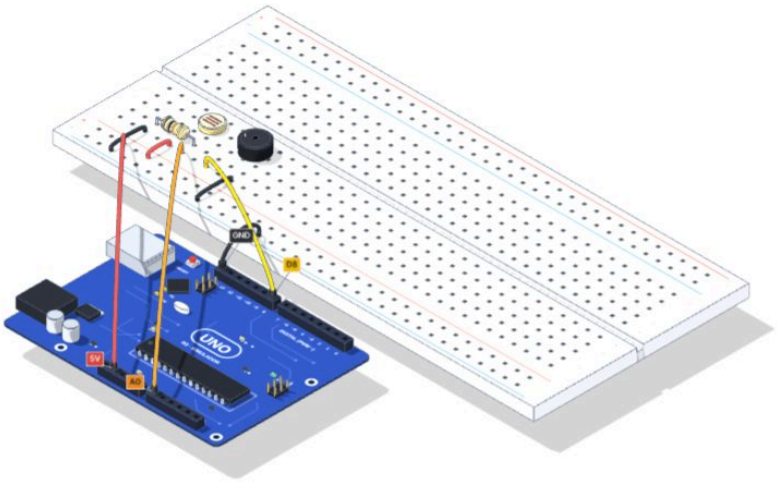
🏆 **Reto:** Agrega un LED que se encienda de noche.

PROYECTO 69 · LUZ Y OSCURIDAD

Barrera láser cuenta-objetos

★★☆ Intermedio · Tiempo: 30 a 45 minutos

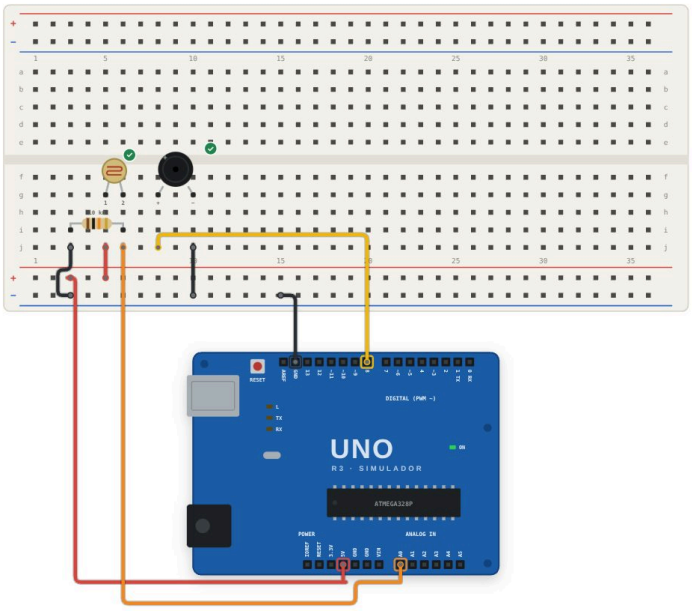
Cada vez que un objeto tapa la luz que llega al sensor, el contador suma uno.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de la barrera (Fotorresistencia)
- 1 × Bip (Zumbador)
- 1 × Resistencia 10 kΩ
- Puntero láser o linterna
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de la barrera (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Bip (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Para el proyecto real

- En la vida real se apunta un puntero láser o una linterna hacia el sensor, al otro lado del camino.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 81 mA.
- Consumo estimado: **~81 mA** funcionando, con picos de **~81 mA**.

2 El programa

```
// Contador de objetos con barrera de luz
const int SENSOR = A0;
const int BIP = 8;
int cuenta = 0;
bool tapado = false;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int luz = analogRead(SENSOR);
  if (!tapado && luz < 300) {           // un objeto empezó a tapar la luz
    tapado = true;
    cuenta++;
    tone(BIP, 1500, 60);
    Serial.print("Objetos: ");
    Serial.println(cuenta);
  }
  if (tapado && luz > 450) tapado = false;   // el objeto ya pasó
  delay(10);
}
```

💡 ¿Cómo funciona?

La variable tapado evita contar el mismo objeto muchas veces mientras pasa. Se usan dos límites distintos (300 y 450) para que una sombra temblorosa no cuente doble.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ **Ten en cuenta**

- En la vida real se apunta un puntero láser o una linterna hacia el sensor, al otro lado del camino.

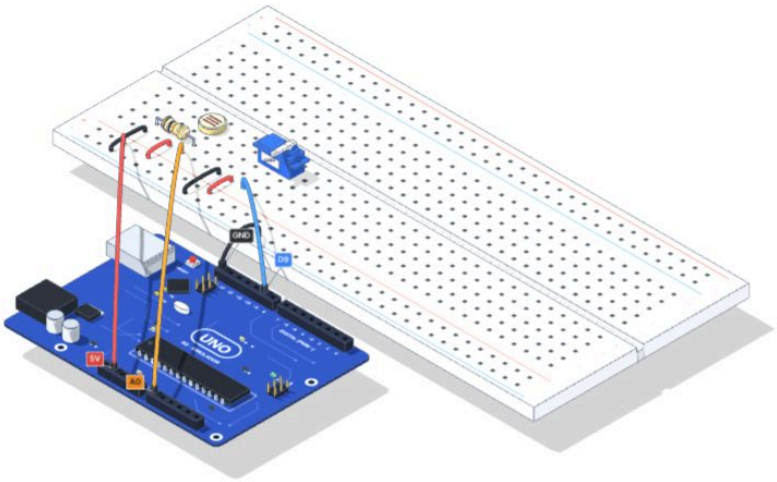
🏆 **Reto:** Muestra la cuenta en una pantalla LCD.

PROYECTO 70 · LUZY OSCURIDAD

Girasol de un servo

★★★ Avanzado · Tiempo: 45 a 90 minutos

Un servo con un sensor de luz busca hacia dónde hay más luz y se queda mirando hacia allá.

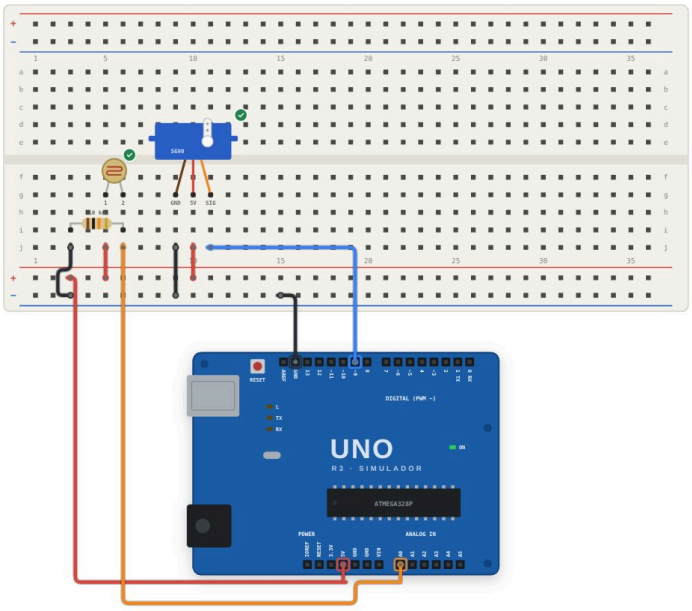


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Ojo de luz (Fotorresistencia)
- 1 × Cuello (Servo SG90)
- 1 × Resistencia 10 kΩ
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Ojo de luz (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Cuello (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 201 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~201 mA** funcionando, con picos de **~701 mA**.

2 El programa

```
#include <Servo.h>
Servo cuello;

void setup() {
  Serial.begin(9600);
  cuello.attach(9);
}

void loop() {
  int mejorAngulo = 0, mejorLuz = 0;
  for (int a = 0; a <= 180; a += 10) {    // barre de lado a lado
    cuello.write(a);
    delay(80);
    int luz = analogRead(A0);
    if (luz > mejorLuz) { mejorLuz = luz; mejorAngulo = a; }
  }
  cuello.write(mejorAngulo);              // vuelve al punto con más luz
  Serial.print("Más luz en ");
  Serial.print(mejorAngulo);
  Serial.println(" grados");
  delay(5000);
}
```

💡 ¿Cómo funciona?

El servo recorre de 0 a 180 grados de 10 en 10 y en cada posición mide la luz. Guarda el ángulo con la medida más alta y al final vuelve a él.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

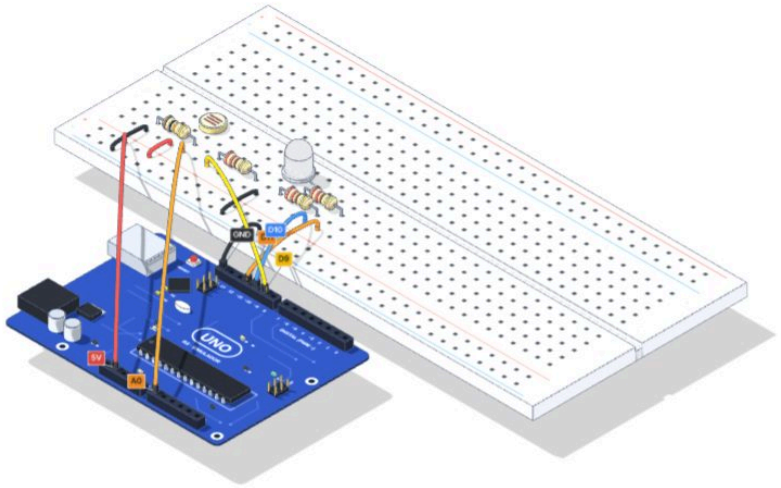
🏆 **Reto:** Haz que el servo se mueva más lento para medir con más cuidado.

PROYECTO 71 · LUZ Y OSCURIDAD

Luz nocturna con colores

★★☆ Intermedio · Tiempo: 30 a 45 minutos

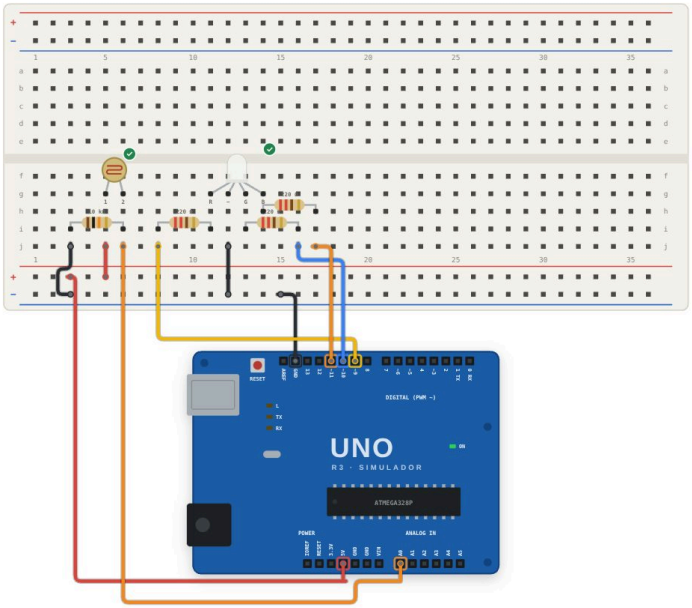
De noche un LED RGB se enciende con un color suave que cambia lentamente para ayudar a dormir.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de luz (Fotorresistencia)
- 1 × Lámpara RGB (LED RGB)
- 3 × Resistencia 220 Ω
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de luz (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Lámpara RGB (LED RGB)

- Pata R (rojo) → resistencia de **220 Ω** → pin **~D9**
- Pata – (común, la más larga) → riel **–** (GND)
- Pata G (verde) → resistencia de **220 Ω** → pin **~D10**
- Pata B (azul) → resistencia de **220 Ω** → pin **~D11**

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 96 mA.
- Consumo estimado: **~96 mA** funcionando, con picos de **~96 mA**.

2 El programa

```
// Luz nocturna de colores suaves
const int R = 9, G = 10, B = 11;
int paso = 0;

void setup() {
  pinMode(R, OUTPUT); pinMode(G, OUTPUT); pinMode(B, OUTPUT);
}

void loop() {
  if (analogRead(A0) < 300) {               // está oscuro
    paso = (paso + 1) % 360;
    float t = paso * 3.1416 / 180.0;
    analogWrite(R, 40 + 40 * sin(t));
    analogWrite(G, 20);
    analogWrite(B, 60 + 50 * sin(t + 2));
  } else {
    analogWrite(R, 0); analogWrite(G, 0); analogWrite(B, 0);
  }
  delay(40);
}
```

💡 ¿Cómo funciona?

La función seno (sin) sube y baja suavemente entre −1 y 1. Usándola para el rojo y el azul con un pequeño desfase, los colores cambian despacio sin saltos.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa.

🏆 **Reto:** Haz que la luz se apague sola después de 30 minutos.

CAPÍTULO 6

Temperatura y clima

Sensores LM35 y DHT11 para medir temperatura y humedad y tomar decisiones con ellas.

EN ESTE CAPÍTULO · 15 PROYECTOS

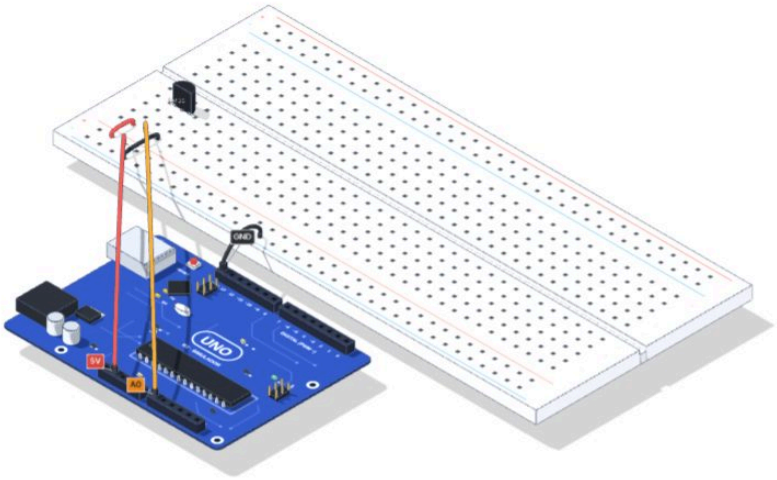
72 Termómetro en el monitor serie ★☆☆
73 Termómetro con pantalla LCD ★★☆
74 Alarma de calor ★☆☆
75 Estación del clima con DHT11 ★★☆
76 Alerta de humedad alta ★★☆
77 Semáforo de confort ★★☆
78 Termostato con perilla ★★★
79 Registro de temperatura máxima y mínima ★★☆
80 Alarma de nevera abierta ★★☆
81 Indicador de sensación de bochorno ★★☆
82 Ventilador por temperatura con DHT11 ★★☆
83 Termómetro de barra con LED ★☆☆
84 Incubadora de huevos sencilla ★★★
85 Detector de fiebre para practicar ★★☆
86 Monitor del clima con LCD y ventilador ★★★

PROYECTO 72 · TEMPERATURA Y CLIMA

Termómetro en el monitor serie

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

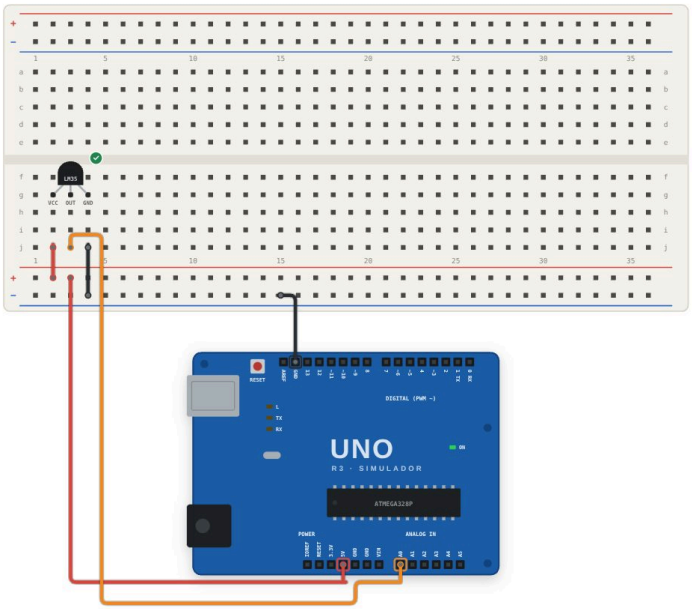
El sensor LM35 mide la temperatura y el Arduino la muestra en grados Celsius.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor LM35
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 51 mA.
- Consumo estimado: **~51 mA** funcionando, con picos de **~51 mA**.

2 El programa

```
// Termómetro con LM35
const int SENSOR = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int lectura = analogRead(SENSOR);
  float temperatura = lectura * 500.0 / 1023.0;   // 10 mV por cada grado
  Serial.print("Temperatura: ");
  Serial.print(temperatura, 1);
  Serial.println(" °C");
  delay(1000);
}
```

💡 ¿Cómo funciona?

El LM35 entrega 10 milivoltios por cada grado. Como 5 V corresponden a 1023 en la lectura, multiplicar por 500 y dividir por 1023 da los grados directamente.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

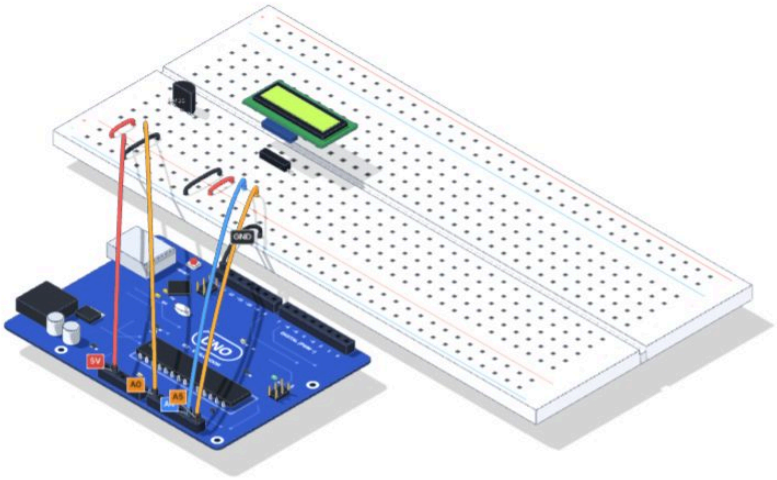
🏆 **Reto:** Muestra también la temperatura en grados Fahrenheit (F = C × 1,8 + 32).

PROYECTO 73 · TEMPERATURA Y CLIMA

Termómetro con pantalla LCD

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Un termómetro digital que muestra la temperatura en una pantalla LCD.

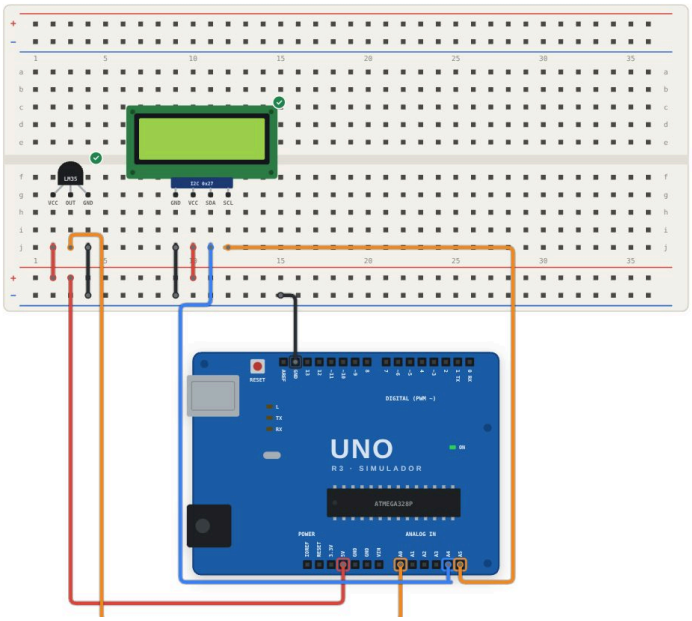


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor LM35
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 81 mA.
- Consumo estimado: **~81 mA** funcionando, con picos de **~81 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  lcd.init();
  lcd.backlight();
  lcd.print("Termometro");
}

void loop() {
  float t = analogRead(A0) * 500.0 / 1023.0;
  lcd.setCursor(0, 1);
  lcd.print("Temp: ");
  lcd.print(t, 1);
  lcd.print(" C   ");          // espacios para borrar números viejos
  delay(1000);
}
```

💡 ¿Cómo funciona?

La primera línea de la pantalla se escribe una sola vez en setup(). En loop() solo se actualiza la segunda línea; los espacios al final borran los dígitos que sobran.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.

Con piezas reales: sube el programa.

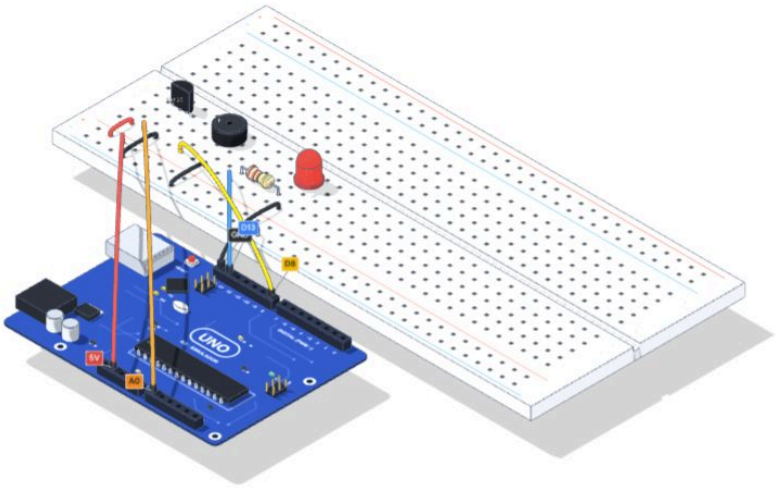
🏆 **Reto:** Agrega la temperatura máxima registrada en la primera línea.

PROYECTO 74 · TEMPERATURA Y CLIMA

Alarma de calor

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

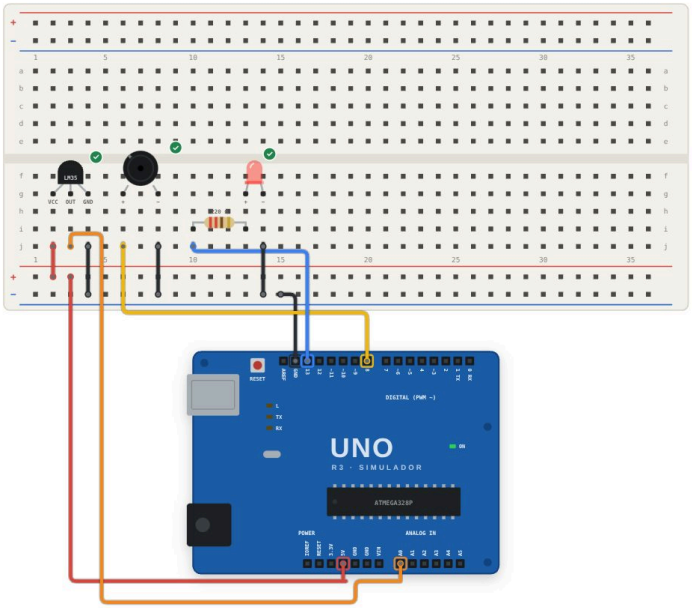
Si la temperatura pasa de 35 °C suena una alarma y se enciende un LED rojo.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor LM35
- 1 × Alarma (Zumbador)
- 1 × Peligro (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Peligro (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 96 mA.
- Consumo estimado: **~96 mA** funcionando, con picos de **~96 mA**.

2 El programa

```
// Alarma de calor
const float LIMITE = 35.0;
const int ALARMA = 8, LUZ = 13;

void setup() {
  Serial.begin(9600);
  pinMode(LUZ, OUTPUT);
}

void loop() {
  float t = analogRead(A0) * 500.0 / 1023.0;
  Serial.println(t, 1);
  if (t > LIMITE) {
    digitalWrite(LUZ, HIGH);
    tone(ALARMA, 2500, 300);
  } else {
    digitalWrite(LUZ, LOW);
  }
  delay(500);
}
```

💡 ¿Cómo funciona?

El valor límite está guardado en una constante al comienzo del programa, para que sea fácil cambiarlo sin buscarlo por todo el código.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

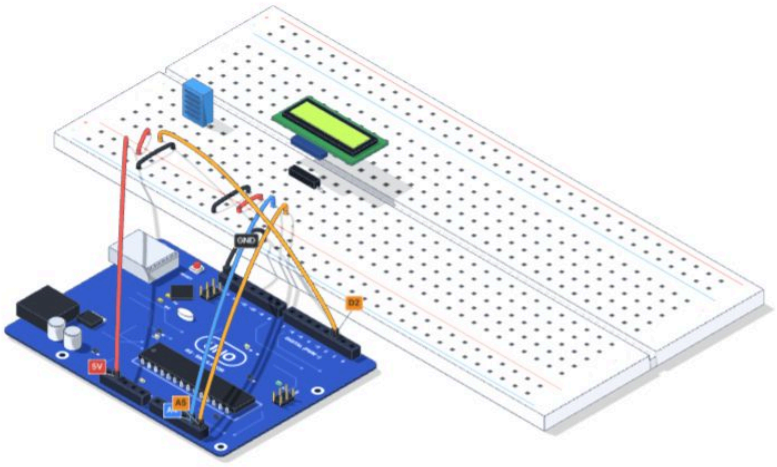
🏆 **Reto:** Agrega un segundo nivel de alerta (LED amarillo) a partir de 30 °C.

PROYECTO 75 · TEMPERATURA Y CLIMA

Estación del clima con DHT11

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Muestra en la LCD la temperatura y la humedad del aire medidas por el DHT11.

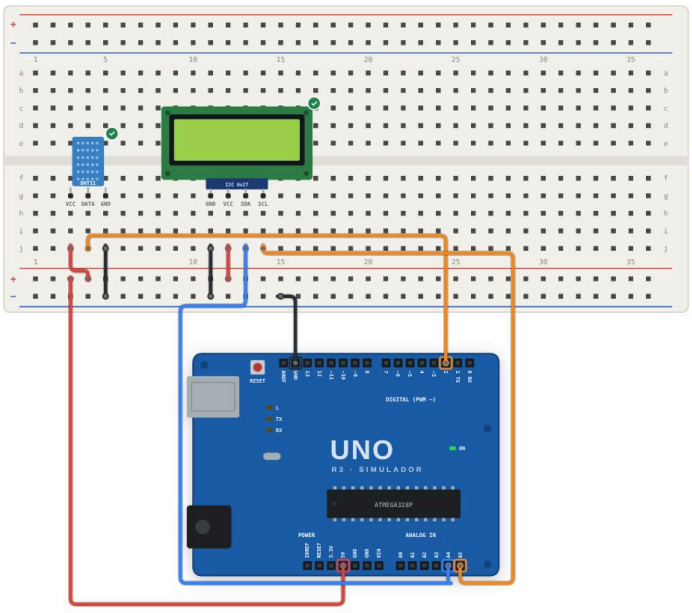


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor DHT11
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: DHT sensor library (de Adafruit) y Adafruit Unified Sensor; LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor DHT11 (Sensor DHT11)

- VCC → riel **+** (5V)
- DATA (señal) → pin **D2**
- GND → riel **–** (GND)

3 **Pantalla LCD (Pantalla LCD 16×2 I2C)**

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 82 mA.
- Consumo estimado: **~82 mA** funcionando, con picos de **~82 mA**.

2 El programa

```
#include <DHT.h>
DHT dht(2, DHT11);   // DATA del sensor en D2
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  dht.begin();
  lcd.init();
  lcd.backlight();
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  lcd.clear();
  if (isnan(t) || isnan(h)) {          // el sensor no respondió
    lcd.print("Error del sensor");
  } else {
    lcd.print("Temp: "); lcd.print(t, 0); lcd.print(" C");
    lcd.setCursor(0, 1);
    lcd.print("Humedad: "); lcd.print(h, 0); lcd.print(" %");
  }
  delay(2000);                          // el DHT11 mide cada 2 segundos
}
```

💡 ¿Cómo funciona?

El DHT11 mide temperatura y humedad y las envía por un solo cable con un código especial; la librería DHT lo traduce. isnan() detecta si la lectura falló.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura y la humedad con los deslizadores del DHT11.

Con piezas reales: sube el programa.

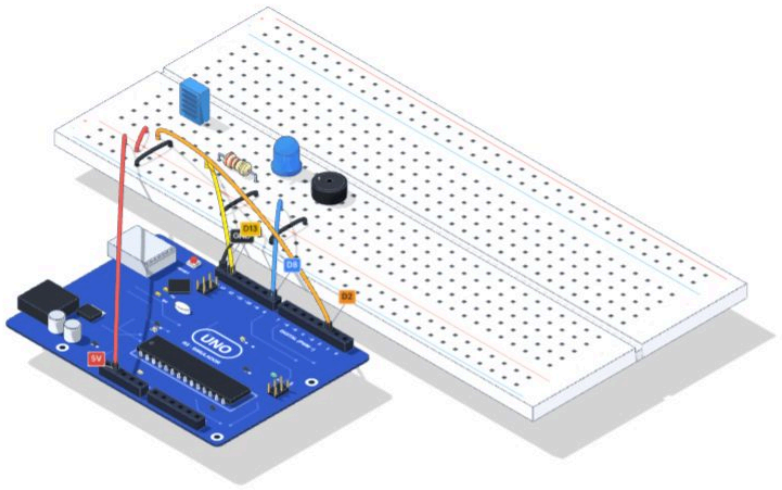
🏆 **Reto:** Agrega un LED que avise cuando la humedad pase del 80 %.

PROYECTO 76 · TEMPERATURA Y CLIMA

Alerta de humedad alta

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Avisa con un LED y un pitido cuando el aire está demasiado húmedo, para evitar el moho.

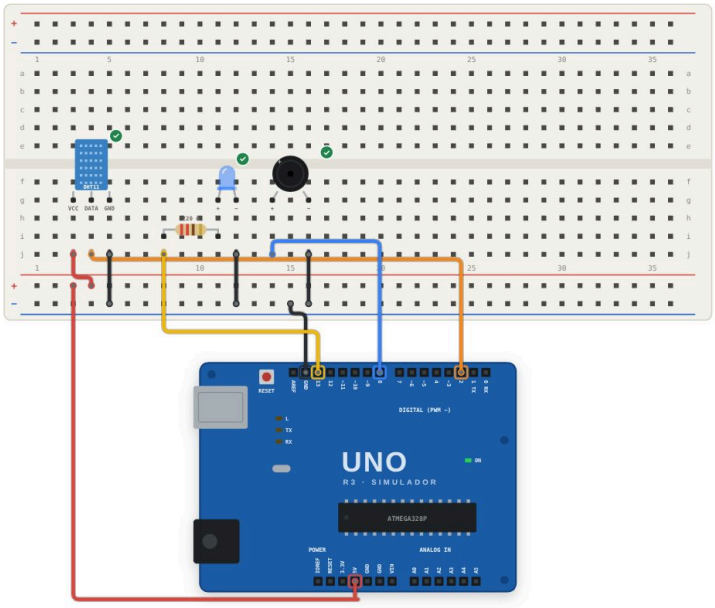


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor DHT11
- 1 × Humedad alta (LED)
- 1 × Aviso (Zumbador)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: DHT sensor library (de Adafruit) y Adafruit Unified Sensor



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor DHT11 (Sensor DHT11)

- VCC → riel **+** (5V)
- DATA (señal) → pin **D2**
- GND → riel **–** (GND)

3 Humedad alta (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Aviso (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 97 mA.
- Consumo estimado: **~97 mA** funcionando, con picos de **~97 mA**.

2 El programa

```
#include <DHT.h>
DHT dht(2, DHT11);   // DATA del sensor en D2
const int LUZ = 13, AVISO = 8;

void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(LUZ, OUTPUT);
}

void loop() {
  float h = dht.readHumidity();
  Serial.print("Humedad: "); Serial.print(h, 0); Serial.println(" %");
  if (h > 75) {
    digitalWrite(LUZ, HIGH);
    tone(AVISO, 900, 150);
  } else {
    digitalWrite(LUZ, LOW);
  }
  delay(2000);
}
```

💡 ¿Cómo funciona?

La humedad relativa dice cuánta agua hay en el aire, en porcentaje. Por encima del 75 % durante mucho tiempo aparecen hongos en paredes y ropa.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura y la humedad con los deslizadores del DHT11.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

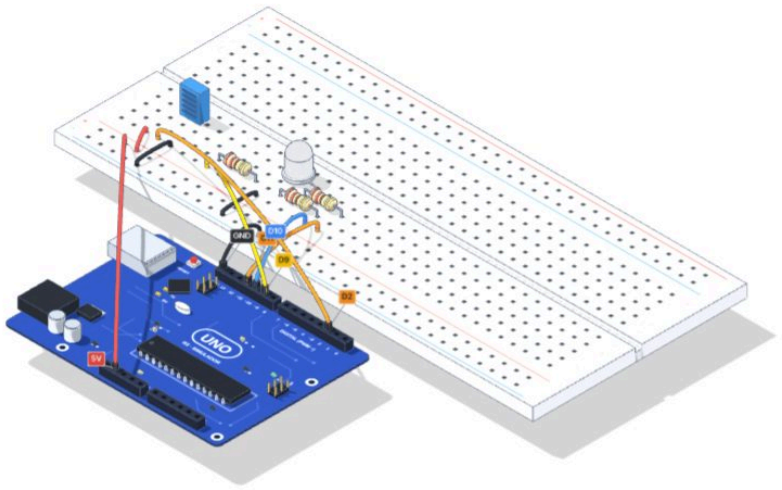
🏆 **Reto:** Muestra en el monitor si el ambiente está seco, agradable o húmedo.

PROYECTO 77 · TEMPERATURA Y CLIMA

Semáforo de confort

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Un LED RGB se pone verde si el salón está cómodo, azul si hace frío y rojo si hace calor.

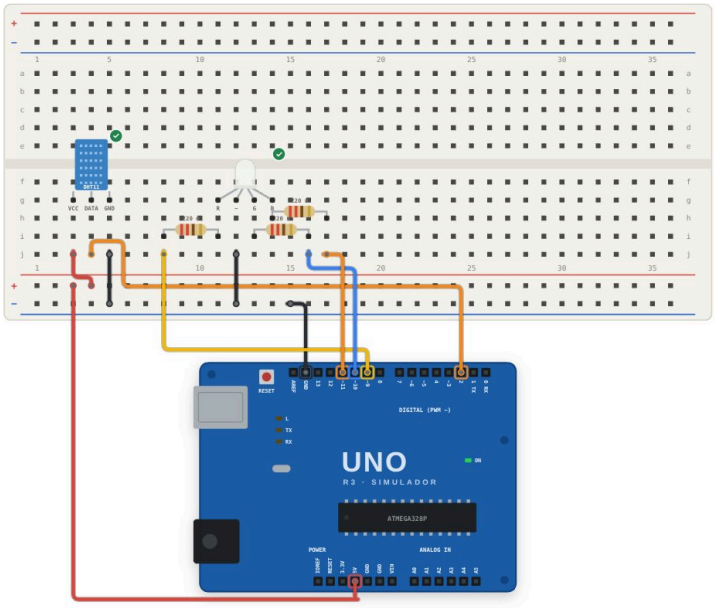


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor DHT11
- 1 × Indicador (LED RGB)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: DHT sensor library (de Adafruit) y Adafruit Unified Sensor



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor DHT11 (Sensor DHT11)

- VCC → riel **+** (5V)
- DATA (señal) → pin **D2**
- GND → riel **–** (GND)

3 Indicador (LED RGB)

- Pata R (rojo) → resistencia de **220 Ω** → pin **~D9**
- Pata – (común, la más larga) → riel **–** (GND)
- Pata G (verde) → resistencia de **220 Ω** → pin **~D10**
- Pata B (azul) → resistencia de **220 Ω** → pin **~D11**

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 97 mA.
- Consumo estimado: **~97 mA** funcionando, con picos de **~97 mA**.

2 El programa

```
#include <DHT.h>
DHT dht(2, DHT11);   // DATA del sensor en D2
const int R = 9, G = 10, B = 11;

void color(int r, int g, int b) { analogWrite(R, r); analogWrite(G, g); analogWrite(B, b); }

void setup() {
  dht.begin();
  pinMode(R, OUTPUT); pinMode(G, OUTPUT); pinMode(B, OUTPUT);
}

void loop() {
  float t = dht.readTemperature();
  if (t < 18) color(0, 0, 255);          // frío
  else if (t > 27) color(255, 0, 0);     // calor
  else color(0, 255, 0);                 // cómodo
  delay(2000);
}
```

💡 ¿Cómo funciona?

Se usa una cadena if / else if / else para escoger un solo color. La función color() hace el código más corto y fácil de leer.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura y la humedad con los deslizadores del DHT11.

Con piezas reales: sube el programa.

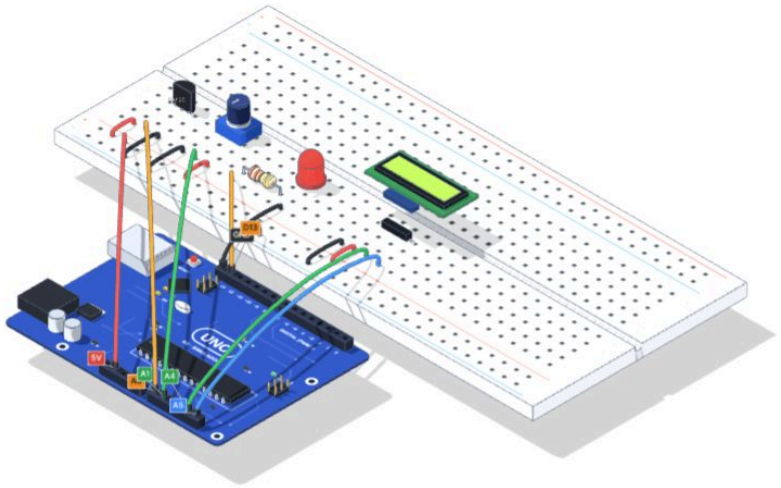
🏆 **Reto:** Usa también la humedad: si pasa de 70 %, que el color sea morado.

PROYECTO 78 · TEMPERATURA Y CLIMA

Termostato con perilla

★★★ Avanzado · Tiempo: 45 a 90 minutos

Eliges la temperatura deseada con la perilla; si hace más frío, se enciende la calefacción (LED rojo).

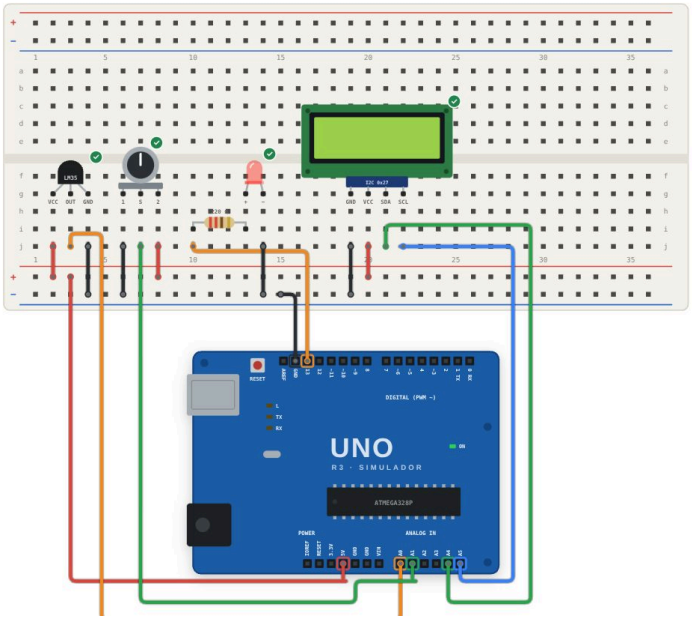


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor LM35
- 1 × Temperatura deseada (Potenciómetro)
- 1 × Calefacción (LED)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Temperatura deseada (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A1**
- Extremo 2 (derecha) → riel **+** (5V)

4 Calefacción (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 97 mA.
- Consumo estimado: **~97 mA** funcionando, con picos de **~97 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
const int CALEFACCION = 13;

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(CALEFACCION, OUTPUT);
}

void loop() {
  float actual = analogRead(A0) * 500.0 / 1023.0;
  int deseada = map(analogRead(A1), 0, 1023, 15, 30);
  bool encender = actual < deseada - 0.5;       // margen de medio grado
  digitalWrite(CALEFACCION, encender);
  lcd.setCursor(0, 0);
  lcd.print("Actual: "); lcd.print(actual, 1); lcd.print(" C  ");
  lcd.setCursor(0, 1);
  lcd.print("Quiero: "); lcd.print(deseada); lcd.print(" C  ");
  delay(500);
}
```

💡 ¿Cómo funciona?

Un termostato compara la temperatura medida con la deseada. El margen de medio grado evita que la calefacción se prenda y apague sin parar cuando están muy cerca.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.
- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa.

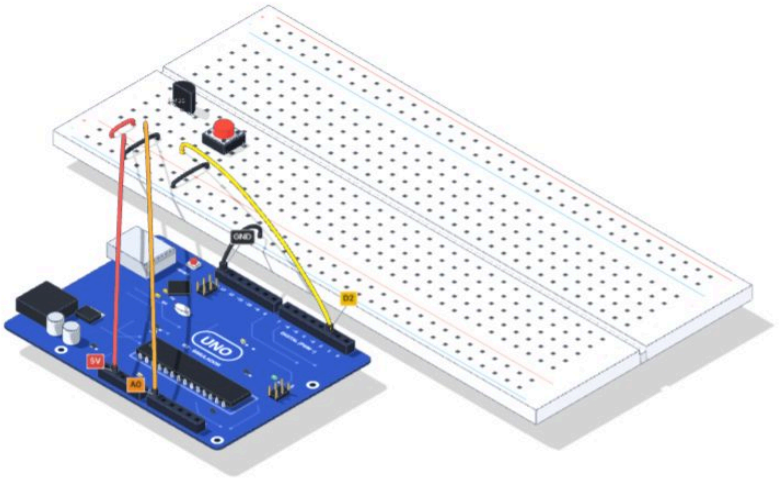
🏆 **Reto:** Agrega un ventilador que se encienda si hace 2 grados más de lo deseado.

PROYECTO 79 · TEMPERATURA Y CLIMA

Registro de temperatura máxima y mínima

★★☆ Intermedio · Tiempo: 30 a 45 minutos

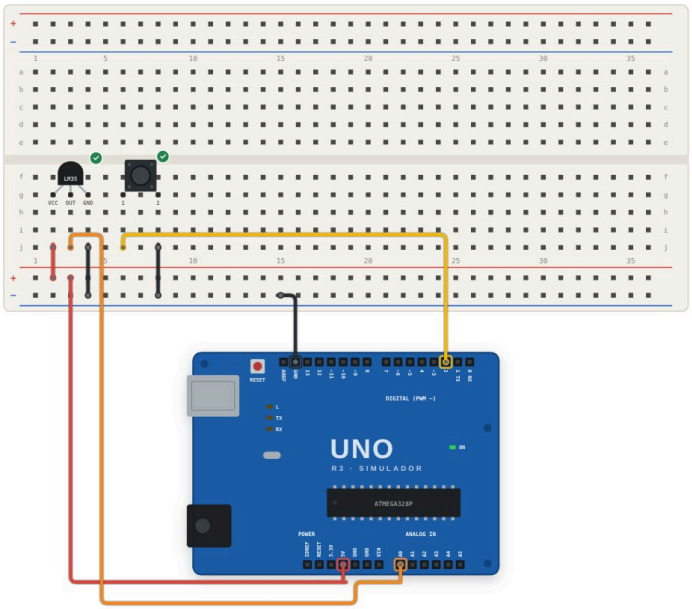
Guarda la temperatura más alta y la más baja medidas y las muestra en el monitor.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor LM35
- 1 × Borrar registro (Botón)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Borrar registro (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 51 mA.
- Consumo estimado: **~51 mA** funcionando, con picos de **~51 mA**.

2 El programa

```
// Temperatura máxima y mínima
float maxima = -100, minima = 200;

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP);
}

void loop() {
  float t = analogRead(A0) * 500.0 / 1023.0;
  if (t > maxima) maxima = t;
  if (t < minima) minima = t;
  if (digitalRead(2) == LOW) { maxima = t; minima = t; Serial.println("Registro borrado"); }
  Serial.print("Ahora: "); Serial.print(t, 1);
  Serial.print("  Máx: "); Serial.print(maxima, 1);
  Serial.print("  Mín: "); Serial.println(minima, 1);
  delay(1000);
}
```

💡 ¿Cómo funciona?

La máxima empieza en un valor muy bajo y la mínima en uno muy alto, para que la primera medición reemplace a las dos. Después solo cambian si aparece un récord.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.
- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

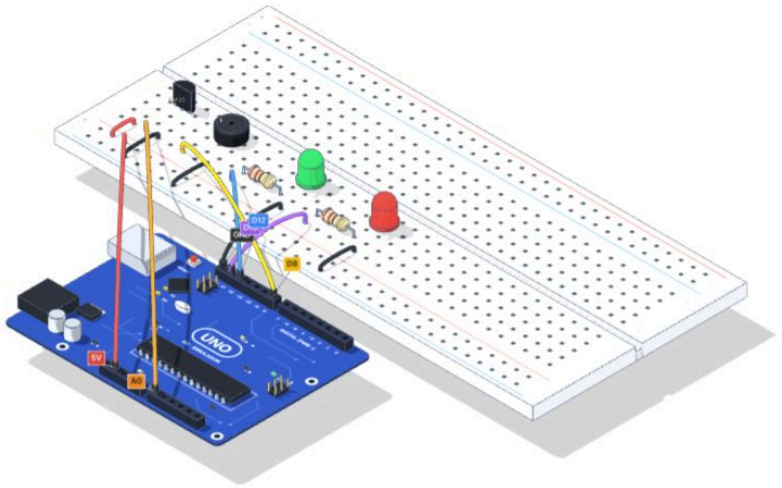
🏆 **Reto:** Calcula también el promedio de todas las mediciones.

PROYECTO 80 · TEMPERATURA Y CLIMA

Alarma de nevera abierta

★★☆ Intermedio · Tiempo: 30 a 45 minutos

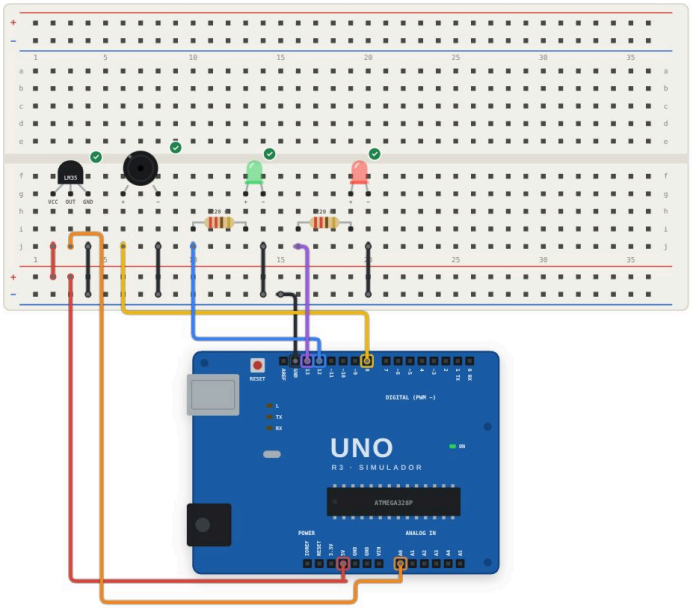
Vigila la temperatura de una nevera: si pasa de 8 °C por mucho tiempo, avisa con sonido.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor en la nevera (Sensor LM35)
- 1 × Alarma (Zumbador)
- 1 × Todo bien (LED)
- 1 × Muy caliente (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor en la nevera (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Todo bien (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Muy caliente (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 111 mA.
- Consumo estimado: **~111 mA** funcionando, con picos de **~111 mA**.

2 El programa

```
// Alarma de nevera
const float LIMITE = 8.0;
int minutosCaliente = 0;

void setup() {
  Serial.begin(9600);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
}

void loop() {
  float t = analogRead(A0) * 500.0 / 1023.0;
  if (t > LIMITE) minutosCaliente++;
  else minutosCaliente = 0;
  digitalWrite(12, minutosCaliente == 0);
  digitalWrite(13, minutosCaliente > 0);
  if (minutosCaliente >= 3) tone(8, 2000, 400);   // 3 lecturas seguidas
  Serial.print(t, 1); Serial.print(" °C  lecturas calientes: "); Serial.println(minutosCaliente);
  delay(1000);    // en una nevera real se mediría cada minuto
}
```

💡 **¿Cómo funciona?**

No basta una lectura caliente (puede ser que abrieron la puerta un momento): la alarma suena solo si se repite varias veces seguidas. Así funcionan las neveras de vacunas.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

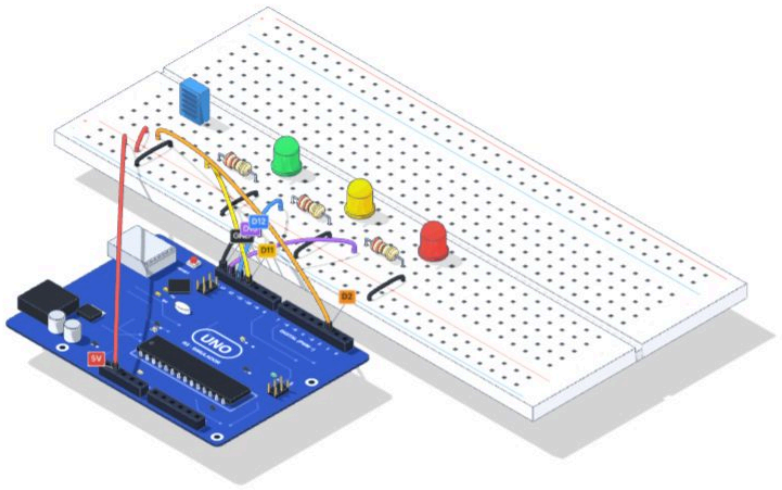
🏆 **Reto:** Cambia el tiempo entre lecturas a 60 segundos para usarlo de verdad.

PROYECTO 81 · TEMPERATURA Y CLIMA

Indicador de sensación de bochorno

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Combina temperatura y humedad para saber si el ambiente se siente sofocante.

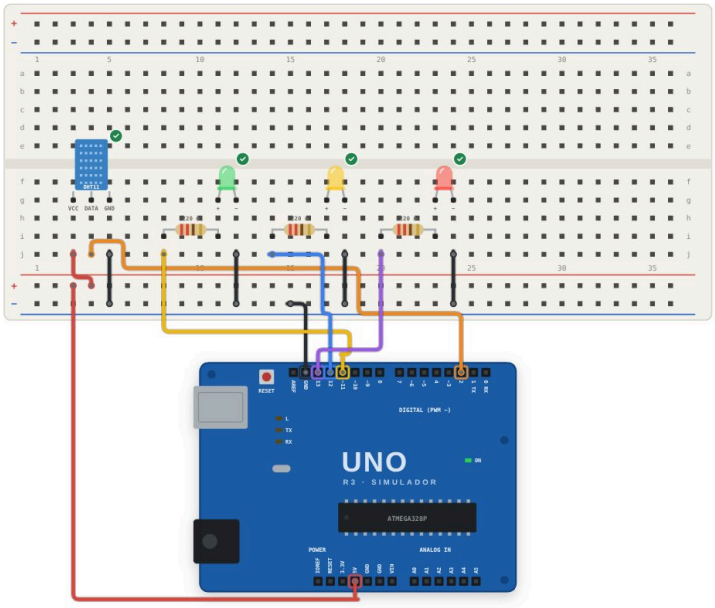


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor DHT11
- 1 × Agradable (LED)
- 1 × Pesado (LED)
- 1 × Sofocante (LED)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: DHT sensor library (de Adafruit) y Adafruit Unified Sensor



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor DHT11 (Sensor DHT11)

- VCC → riel **+** (5V)
- DATA (señal) → pin **D2**
- GND → riel **–** (GND)

3 Agradable (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Pesado (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Sofocante (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 97 mA.
- Consumo estimado: **~97 mA** funcionando, con picos de **~97 mA**.

2 El programa

```
#include <DHT.h>
DHT dht(2, DHT11);   // DATA del sensor en D2

void setup() {
  Serial.begin(9600);
  dht.begin();
  for (int p = 11; p <= 13; p++) pinMode(p, OUTPUT);
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  float sensacion = t + (h - 40) * 0.1;      // fórmula sencilla para practicar
  Serial.print("Se siente como "); Serial.print(sensacion, 1); Serial.println(" °C");
  digitalWrite(11, sensacion < 27);
  digitalWrite(12, sensacion >= 27 && sensacion < 32);
  digitalWrite(13, sensacion >= 32);
  delay(2000);
}
```

💡 ¿Cómo funciona?

Con mucha humedad el sudor se evapora menos y sentimos más calor. La fórmula suma un poco a la temperatura según cuánto pase la humedad del 40 %.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura y la humedad con los deslizadores del DHT11.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

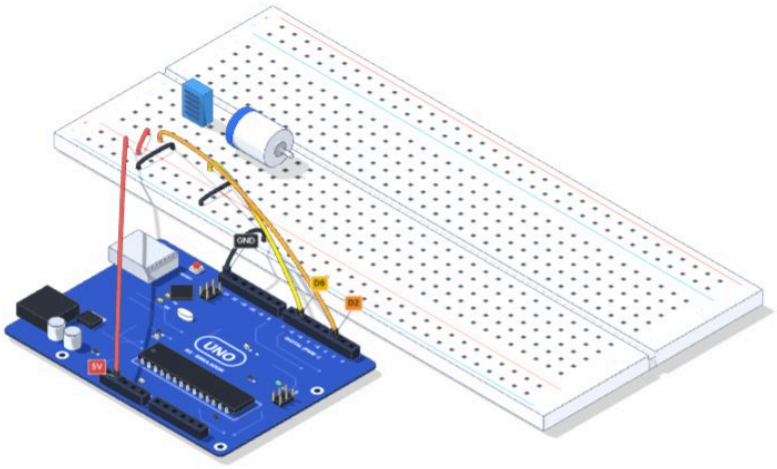
🏆 **Reto:** Investiga la fórmula oficial del índice de calor y compárala.

PROYECTO 82 · TEMPERATURA Y CLIMA

Ventilador por temperatura con DHT11

★★☆ Intermedio · Tiempo: 30 a 45 minutos

El ventilador gira más rápido cuanto más calor haga, según el DHT11.

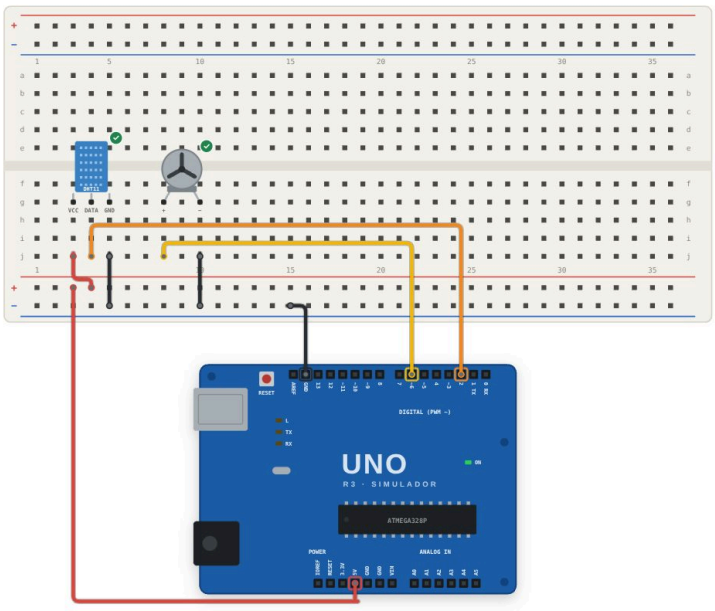


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor DHT11
- 1 × Ventilador (Motor DC)
- Transistor TIP120 (o 2N2222)
- Diodo 1N4007
- Resistencia 1 kΩ
- Fuente 6–12 V para los motores
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: DHT sensor library (de Adafruit) y Adafruit Unified Sensor



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor DHT11 (Sensor DHT11)

- VCC → riel **+** (5V)
- DATA (señal) → pin **D2**
- GND → riel **–** (GND)

3 **Ventilador (Motor DC)**

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

4 **Para el proyecto real**

- En la vida real usa un transistor o un relé para el motor del ventilador.

5 **Energía y controladores**

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motor → necesita transistor:** un pin da solo 20–40 mA y el motor pide 150–250 mA (hasta 1 A al arrancar). Conexión: pin → resistencia de 1 kΩ → base del **TIP120**; motor entre el + de la fuente y el colector; emisor a GND. Pon un **diodo 1N4007** en paralelo con el motor (la raya hacia el +) para frenar los picos de voltaje que dañan el transistor.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al + del motor (el – del motor va al colector del transistor). Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~252 mA** funcionando, con picos de **~1052 mA**.

2 El programa

```
#include <DHT.h>
DHT dht(2, DHT11);   // DATA del sensor en D2
const int VENTILADOR = 6;

void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(VENTILADOR, OUTPUT);
}

void loop() {
  float t = dht.readTemperature();
  int velocidad = 0;
  if (t > 24) velocidad = map(t, 24, 34, 100, 255);
  velocidad = constrain(velocidad, 0, 255);
  analogWrite(VENTILADOR, velocidad);
  Serial.print(t, 0); Serial.print(" °C -> velocidad "); Serial.println(velocidad);
  delay(2000);
}
```

💡 ¿Cómo funciona?

Por debajo de 24 °C el ventilador está apagado. Por encima, map() reparte la velocidad entre 100 y 255 según qué tan caliente esté.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura y la humedad con los deslizadores del DHT11.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ **Ten en cuenta**

- En la vida real usa un transistor o un relé para el motor del ventilador.

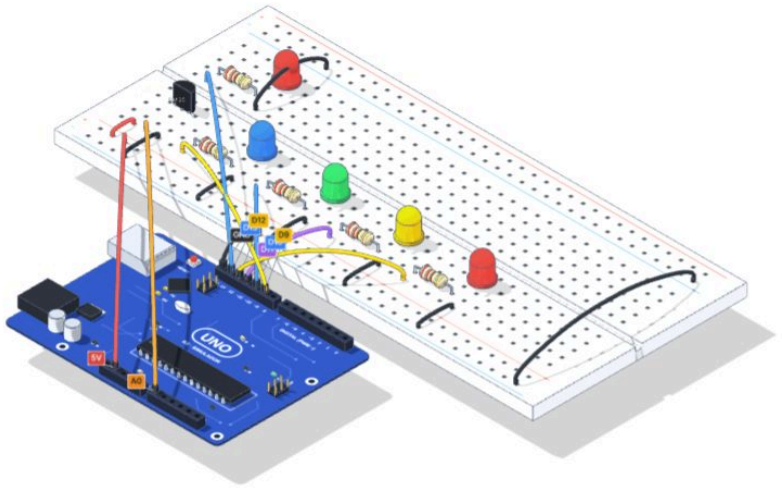
🏆 **Reto:** Agrega una pantalla LCD que muestre la temperatura y la velocidad.

PROYECTO 83 · TEMPERATURA Y CLIMA

Termómetro de barra con LED

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

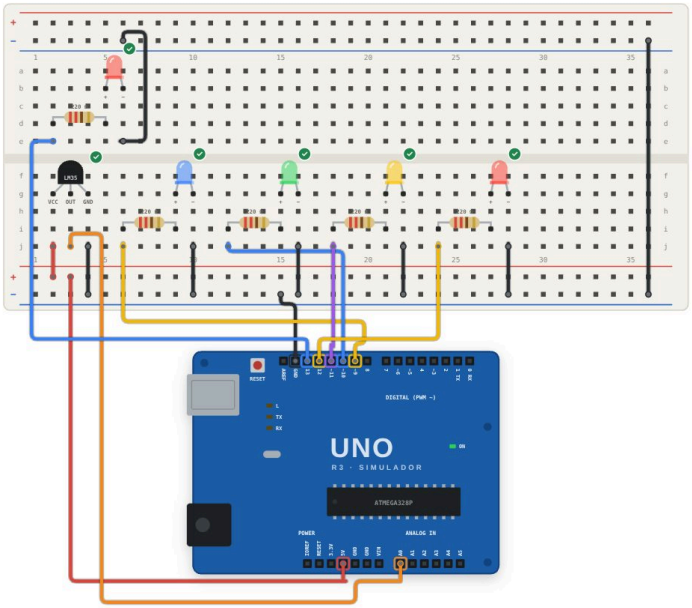
Cinco LED suben como la columna de un termómetro de mercurio.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor LM35
- 1 × 20 °C (LED)
- 1 × 24 °C (LED)
- 1 × 28 °C (LED)
- 1 × 32 °C (LED)
- 1 × 36 °C (LED)
- 5 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 20 °C (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 24 °C (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 28 °C (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 32 °C (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 36 °C (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

8 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 126 mA.
- Consumo estimado: **~126 mA** funcionando, con picos de **~126 mA**.

2 El programa

```
// Termómetro de barra
int barra[] = {9, 10, 11, 12, 13};

void setup() {
  for (int i = 0; i < 5; i++) pinMode(barra[i], OUTPUT);
}

void loop() {
  float t = analogRead(A0) * 500.0 / 1023.0;
  for (int i = 0; i < 5; i++) {
    float marca = 20 + i * 4;          // 20, 24, 28, 32 y 36 grados
    digitalWrite(barra[i], t >= marca);
  }
  delay(500);
}
```

💡 **¿Cómo funciona?**

Cada LED tiene una marca de temperatura. Si la temperatura medida alcanza la marca, el LED se enciende; así la barra crece con el calor.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.

Con piezas reales: sube el programa.

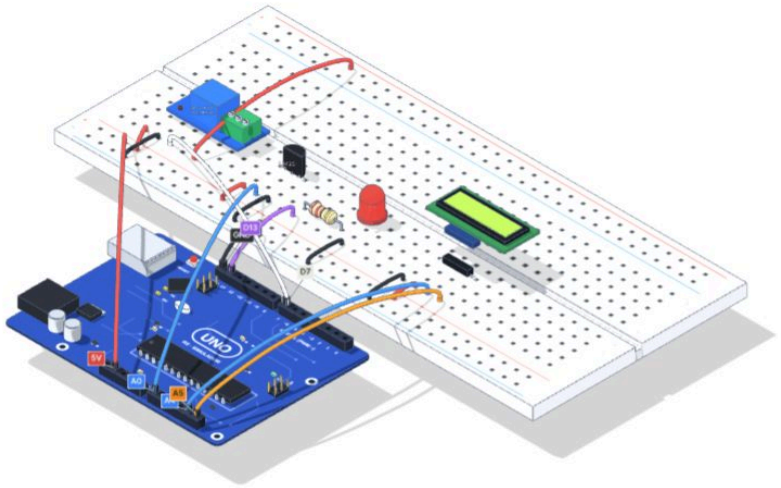
🏆 **Reto:** Cambia las marcas para medir la temperatura del cuerpo (35 a 39 °C).

PROYECTO 84 · TEMPERATURA Y CLIMA

Incubadora de huevos sencilla

★★★ Avanzado · Tiempo: 45 a 90 minutos

Mantiene la temperatura cerca de 37,5 °C encendiendo un bombillo con un relé.

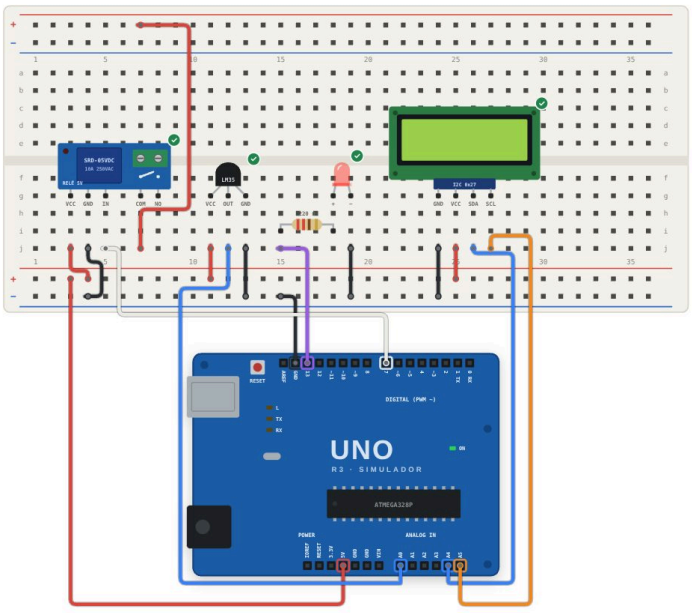


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor LM35
- 1 × Relé del bombillo (Módulo relé 5 V)
- 1 × Calentando (LED)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Resistencia 220 Ω
- Bombillo incandescente de 40 W con su roseta
- Caja de icopor
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Relé del bombillo (Módulo relé 5 V)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- IN (señal) → pin **D7**
- COM (común) → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- NO (normalmente abierto) → **+ de la carga** (bomba, chapa o motor de 12 V)

4 Calentando (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

6 Para el proyecto real

- El bombillo de 110 V solo lo conecta un adulto, con el relé bien aislado y en una caja cerrada.

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 166 mA.
- **Relé:** el lado de control (VCC, GND, IN) va al Arduino con 5 V; la carga va por COM y NO con su propia fuente. Algunos módulos se activan con LOW: si el tuyo funciona al revés, invierte HIGH y LOW en el código.
- Consumo estimado: **~166 mA** funcionando, con picos de **~166 mA**.

2 El programa

```ino
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
const int RELE = 7, LUZ = 13;
const float META = 37.5;

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(RELE, OUTPUT); pinMode(LUZ, OUTPUT);
}

void loop() {
  float t = analogRead(A0) * 500.0 / 1023.0;
  bool calentar = t < META - 0.3;
  if (t > META + 0.3) calentar = false;
  digitalWrite(RELE, calentar);
  digitalWrite(LUZ, calentar);
  lcd.setCursor(0, 0); lcd.print("Temp: "); lcd.print(t, 1); lcd.print(" C  ");
  lcd.setCursor(0, 1);
  if (calentar) lcd.print("Calentando...  ");
```

```
  else lcd.print("Temperatura OK ");
  delay(1000);
}
```

💡 **¿Cómo funciona?**

El relé es un interruptor que el Arduino activa con un pin. Así una salida de 5 V puede encender un bombillo de 110 V sin que el Arduino toque esa corriente.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.

Con piezas reales: sube el programa.

⚠️ **Ten en cuenta**

- El bombillo de 110 V solo lo conecta un adulto, con el relé bien aislado y en una caja cerrada.

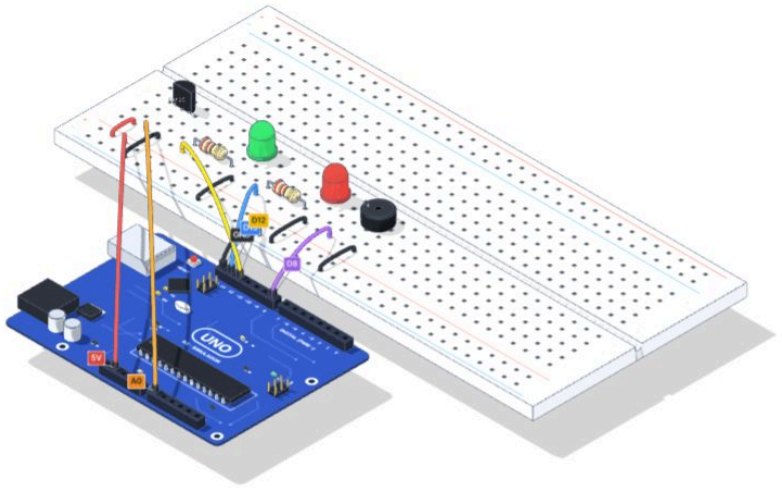
🏆 **Reto:** Agrega un zumbador que avise si la temperatura pasa de 39 °C.

PROYECTO 85 · TEMPERATURA Y CLIMA

Detector de fiebre para practicar

★★☆ Intermedio · Tiempo: 30 a 45 minutos

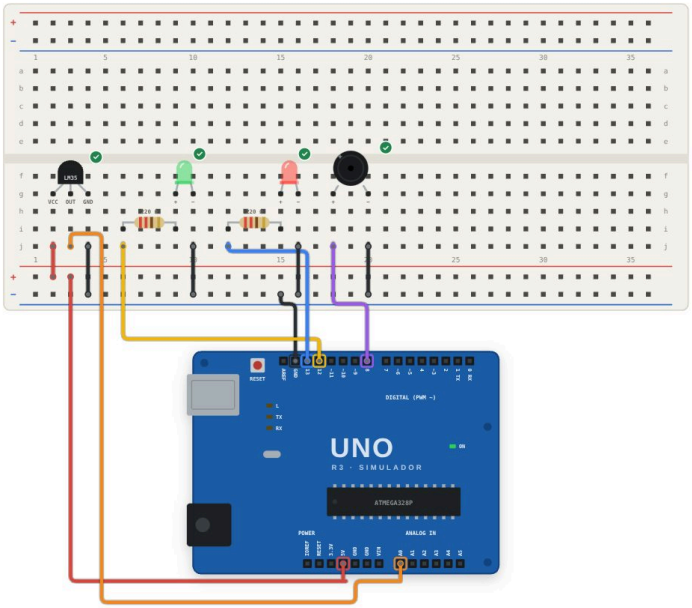
Simula un termómetro médico: indica si la temperatura del cuerpo es normal o hay fiebre.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor LM35
- 1 × Normal (LED)
- 1 × Fiebre (LED)
- 1 × Aviso (Zumbador)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Normal (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Fiebre (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Aviso (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

6 Para el proyecto real

- Es un proyecto para aprender: no reemplaza un termómetro médico.

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 111 mA.
- Consumo estimado: **~111 mA** funcionando, con picos de **~111 mA**.

2 El programa

```ino
// Detector de fiebre (para practicar)
void setup() {
  Serial.begin(9600);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
}

void loop() {
  float t = analogRead(A0) * 500.0 / 1023.0;
  Serial.print(t, 1); Serial.print(" °C: ");
  if (t >= 37.8) {
    Serial.println("fiebre");
    digitalWrite(13, HIGH); digitalWrite(12, LOW);
    tone(8, 1500, 100);
  } else {
    Serial.println("normal");
    digitalWrite(12, HIGH); digitalWrite(13, LOW);
  }
  delay(1000);
}
```

💡 ¿Cómo funciona?

La temperatura normal del cuerpo está alrededor de 36,5 °C; desde 37,8 °C se considera fiebre. El programa toma esa decisión y avisa con luz y sonido.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Es un proyecto para aprender: no reemplaza un termómetro médico.

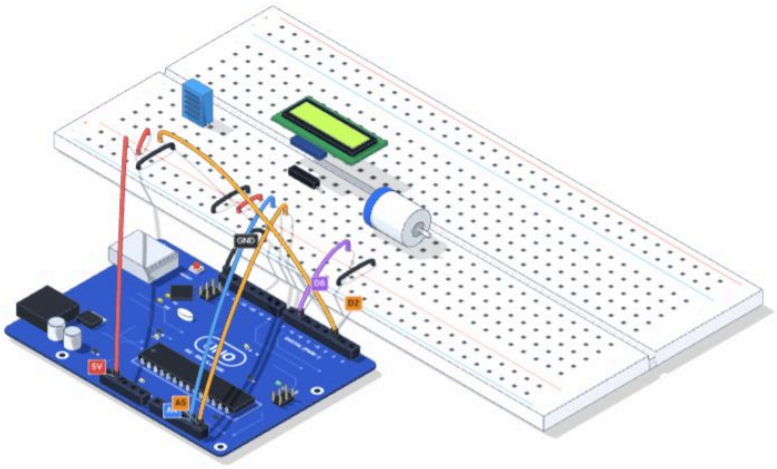
🏆 **Reto:** Toma tres medidas y usa el promedio antes de decidir.

PROYECTO 86 · TEMPERATURA Y CLIMA

Monitor del clima con LCD y ventilador

★★★ Avanzado · Tiempo: 45 a 90 minutos

La LCD muestra temperatura y humedad; el ventilador se enciende solo si hace calor.

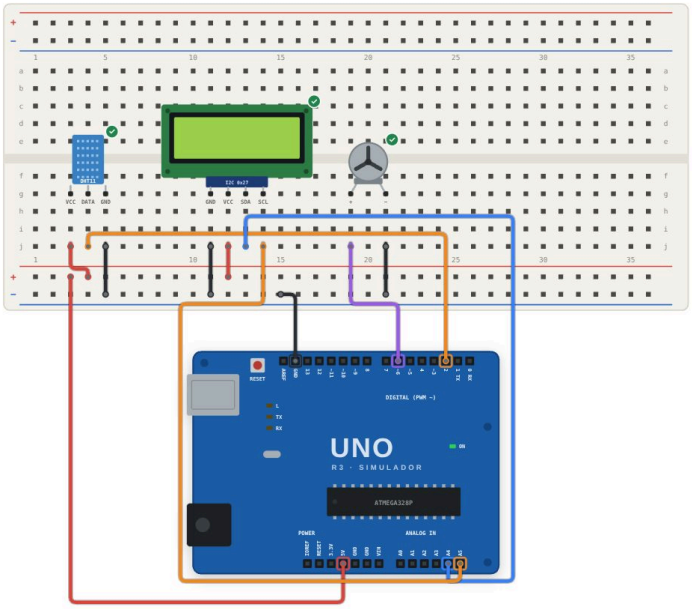


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor DHT11
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Ventilador (Motor DC)
- Transistor TIP120 (o 2N2222)
- Diodo 1N4007
- Resistencia 1 kΩ
- Fuente 6–12 V para los motores
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: DHT sensor library (de Adafruit) y Adafruit Unified Sensor; LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor DHT11 (Sensor DHT11)

- VCC → riel **+** (5V)
- DATA (señal) → pin **D2**
- GND → riel **–** (GND)

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Ventilador (Motor DC)

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

5 Energía y controladores

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motor → necesita transistor:** un pin da solo 20–40 mA y el motor pide 150–250 mA (hasta 1 A al arrancar). Conexión: pin → resistencia de 1 kΩ → base del **TIP120**; motor entre el + de la fuente y el colector; emisor a GND. Pon un **diodo 1N4007** en paralelo con el motor (la raya hacia el +) para frenar los picos de voltaje que dañan el transistor.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al + del motor (el – del motor va al colector del transistor). Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~282 mA** funcionando, con picos de **~1082 mA**.

2 El programa

```
#include <DHT.h>
DHT dht(2, DHT11);   // DATA del sensor en D2
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  dht.begin();
  lcd.init(); lcd.backlight();
  pinMode(6, OUTPUT);
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (isnan(t) || isnan(h)) { lcd.clear(); lcd.print("Error sensor"); delay(2000); return; }
  bool calor = t > 28;
  digitalWrite(6, calor);
  lcd.clear();
  lcd.print("T: "); lcd.print(t, 1); lcd.print(" C");
  lcd.setCursor(0, 1); lcd.print("H: "); lcd.print(h, 0); lcd.print("%"); lcd.print(calor ? " VENT ON" : "");
  delay(2000);
}
```

💡 **¿Cómo funciona?**

isnan() revisa si la lectura falló (por ejemplo, un cable suelto) para mostrar un error en vez de números raros.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura y la humedad con los deslizadores del DHT11.

Con piezas reales: sube el programa.

🏆 **Reto:** Agrega un LED azul que avise si la humedad pasa del 80 %.

CAPÍTULO 7

Distancia y movimiento

El sensor ultrasónico mide distancias con eco y el PIR detecta personas que se mueven.

EN ESTE CAPÍTULO · 15 PROYECTOS

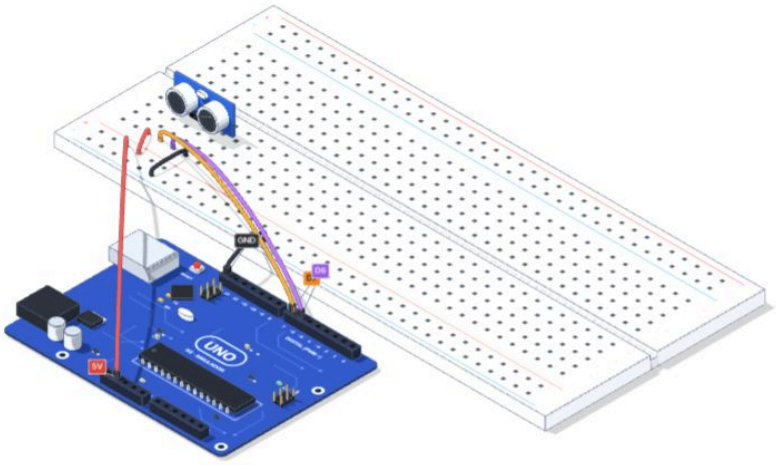
87 Medidor de distancia en el monitor ★☆☆
88 Medidor de distancia con pantalla ★★☆
89 Alarma de proximidad ★☆☆
90 Sensor de distancia con semáforo ★★☆
91 Theremin ultrasónico ★★☆
92 Contador de visitantes con ultrasonido ★★☆
93 Detector de intrusos con PIR ★☆☆
94 Luz automática de pasillo ★☆☆
95 Saludo automático en la entrada ★★☆
96 Radar con servo y ultrasonido ★★★
97 Barrera automática de parqueadero ★★☆
98 Medidor de nivel con ultrasonido ★★☆
99 Regla digital con botón de guardar ★★☆
100 Detector de movimiento con sirena y luces ★★☆
101 Detector de objetos con sensor IR ★☆☆

PROYECTO 87 · DISTANCIA Y MOVIMIENTO

Medidor de distancia en el monitor

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

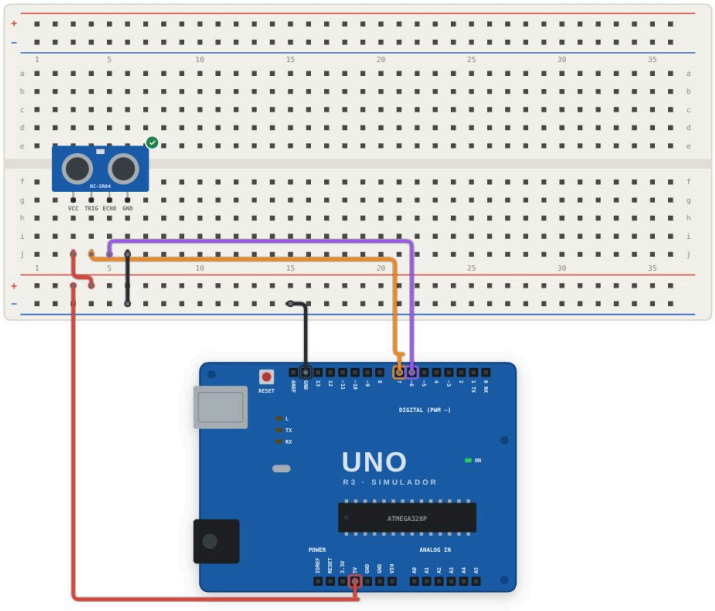
El sensor ultrasónico mide a qué distancia está un objeto y la muestra en centímetros.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor ultrasónico (Ultrasónico HC-SR04)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor ultrasónico (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
// Medidor de distancia con HC-SR04
const int TRIG = 7, ECHO = 6;

void setup() {
  Serial.begin(9600);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
}

void loop() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);   // pulso de 10 microsegundos
  digitalWrite(TRIG, LOW);
  long tiempo = pulseIn(ECHO, HIGH);     // cuánto tardó el eco
  long cm = tiempo / 58;                 // convertir a centímetros
  Serial.print("Distancia: ");
  Serial.print(cm);
  Serial.println(" cm");
  delay(300);
}
```

💡 ¿Cómo funciona?

El sensor lanza un sonido que no podemos oír y espera su eco, como un murciélago. pulseIn() mide el tiempo del eco y dividirlo entre 58 da la distancia en centímetros.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

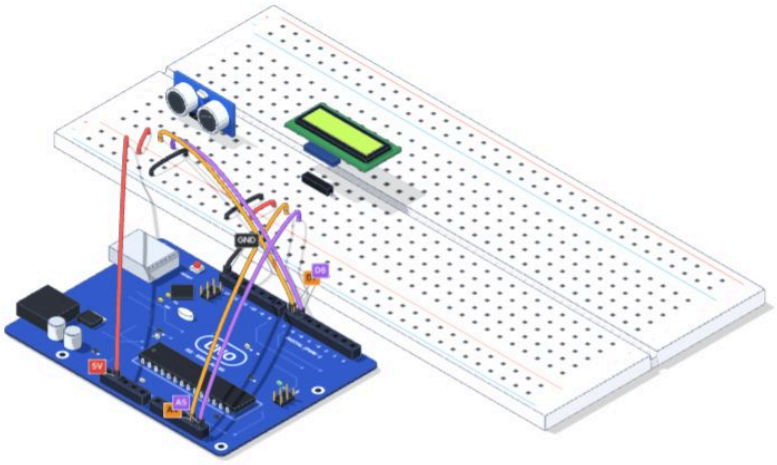
🏆 **Reto:** Muestra la distancia también en metros.

PROYECTO 88 · DISTANCIA Y MOVIMIENTO

Medidor de distancia con pantalla

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Una cinta métrica digital: la LCD muestra la distancia al objeto.

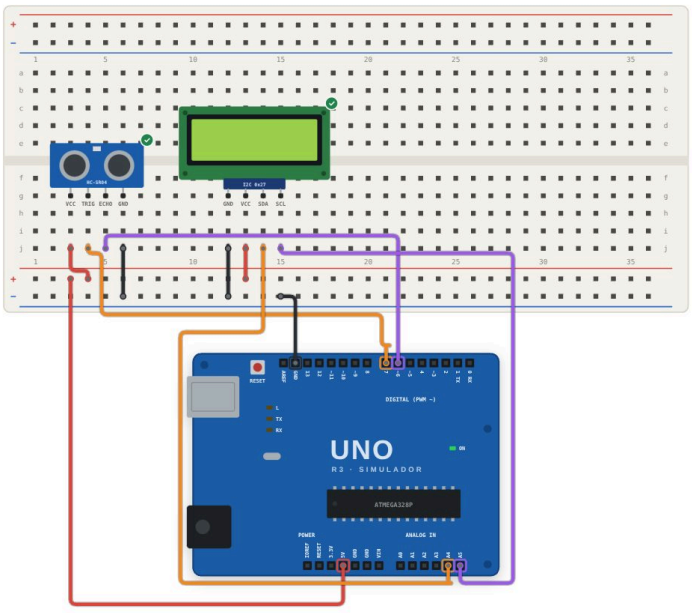


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor ultrasónico (Ultrasónico HC-SR04)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor ultrasónico (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 **Pantalla LCD (Pantalla LCD 16×2 I2C)**

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
const int TRIG = 7, ECHO = 6;

long medir() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long cm = pulseIn(ECHO, HIGH) / 58;   // en Arduino la división entera no deja decimales
  return cm;
}

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  lcd.print("Cinta metrica");
}

void loop() {
  long cm = medir();
  lcd.setCursor(0, 1);
  lcd.print(cm); lcd.print(" cm          ");
  delay(300);
}
```

💡 ¿Cómo funciona?

La función medir() encierra los pasos del sensor y devuelve los centímetros. Así el loop() queda corto y claro.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa.

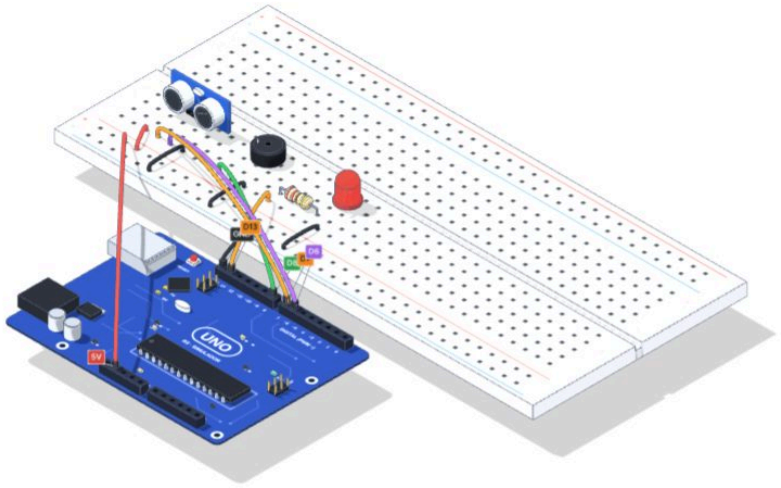
🏆 **Reto:** Agrega un botón que congele la medida en la pantalla.

PROYECTO 89 · DISTANCIA Y MOVIMIENTO

Alarma de proximidad

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

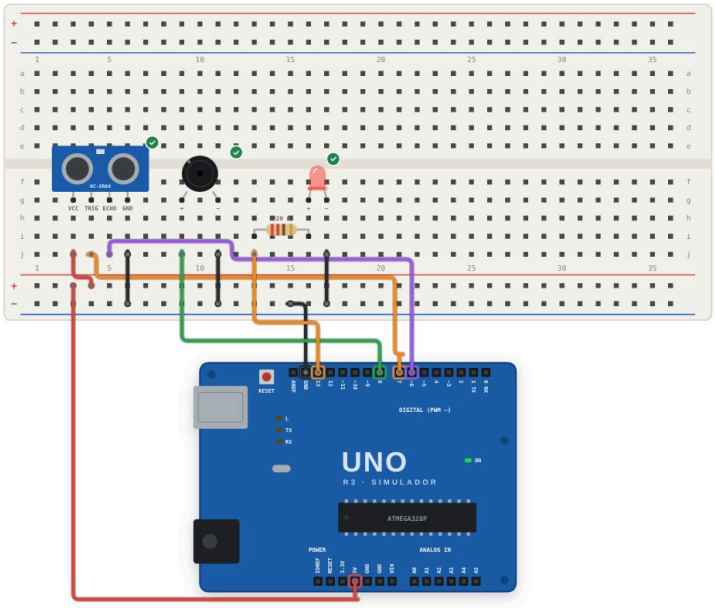
Si alguien se acerca a menos de 30 cm suena una alarma.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor ultrasónico (Ultrasónico HC-SR04)
- 1 × Alarma (Zumbador)
- 1 × Alerta (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor ultrasónico (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Alerta (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
// Alarma de proximidad
const int TRIG = 7, ECHO = 6, ALARMA = 8, LUZ = 13;

void setup() {
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT); pinMode(LUZ, OUTPUT);
}

void loop() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long cm = pulseIn(ECHO, HIGH) / 58;
  if (cm > 0 && cm < 30) {
    digitalWrite(LUZ, HIGH);
    tone(ALARMA, 1800, 150);
  } else {
    digitalWrite(LUZ, LOW);
  }
  delay(200);
}
```

💡 ¿Cómo funciona?

Se comprueba que la distancia sea mayor que 0 porque el sensor devuelve 0 cuando no recibe eco. Si está entre 1 y 29 cm, se activa la alarma.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa.

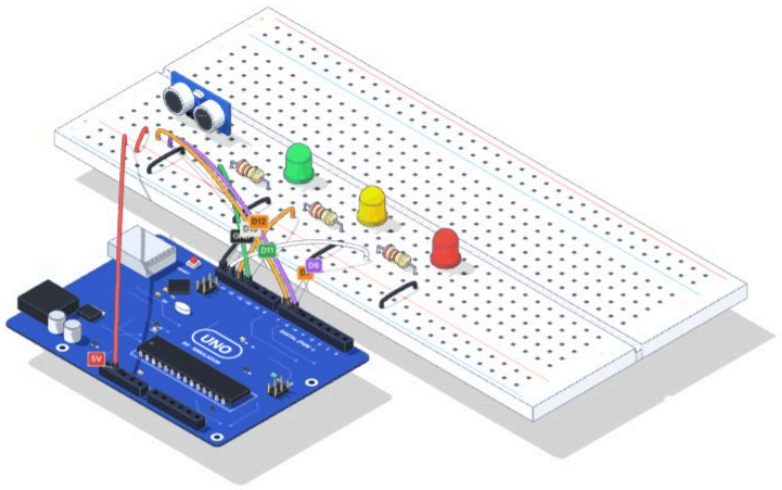
🏆 **Reto:** Haz que la alarma suene más rápido cuanto más cerca esté la persona.

PROYECTO 90 · DISTANCIA Y MOVIMIENTO

Sensor de distancia con semáforo

★★☆ Intermedio · Tiempo: 30 a 45 minutos

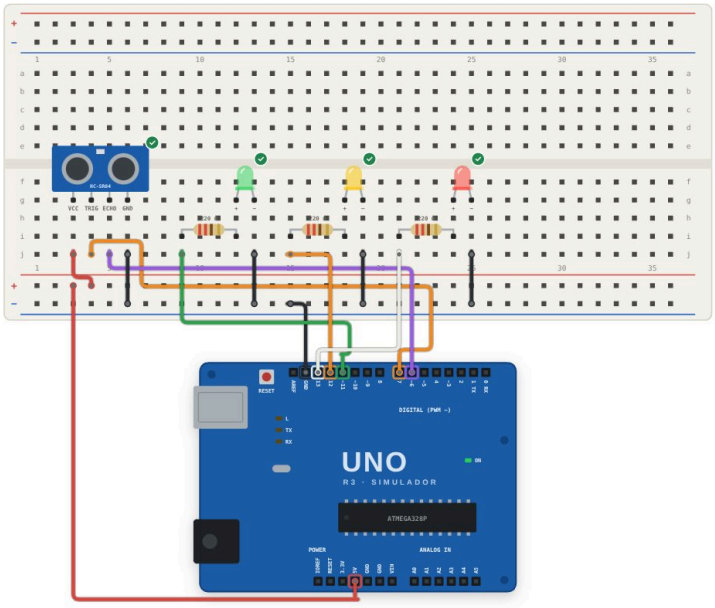
Tres LED indican la distancia: verde lejos, amarillo cuidado y rojo ¡pare!



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor ultrasónico (Ultrasónico HC-SR04)
- 1 × Lejos (LED)
- 1 × Cuidado (LED)
- 1 × Pare (LED)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor ultrasónico (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 Lejos (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Cuidado (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Pare (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```ino
// Semáforo de distancia para estacionar
const int TRIG = 7, ECHO = 6;

void setup() {
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  for (int p = 11; p <= 13; p++) pinMode(p, OUTPUT);
}

void loop() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long cm = pulseIn(ECHO, HIGH) / 58;
  digitalWrite(11, cm >= 60);
  digitalWrite(12, cm >= 25 && cm < 60);
  digitalWrite(13, cm < 25);
  delay(150);
}
```

💡 ¿Cómo funciona?

Cada LED se enciende solo en su rango de distancia. Los garajes de los centros comerciales usan sistemas parecidos para ayudar a estacionar.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa.

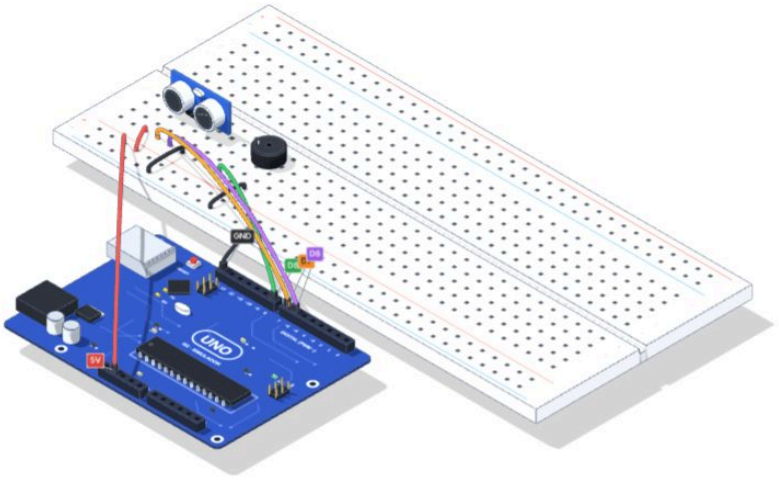
🏆 **Reto:** Haz que el rojo parpadee cuando el carro esté a menos de 10 cm.

PROYECTO 91 · DISTANCIA Y MOVIMIENTO

Theremin ultrasónico

★★☆ Intermedio · Tiempo: 30 a 45 minutos

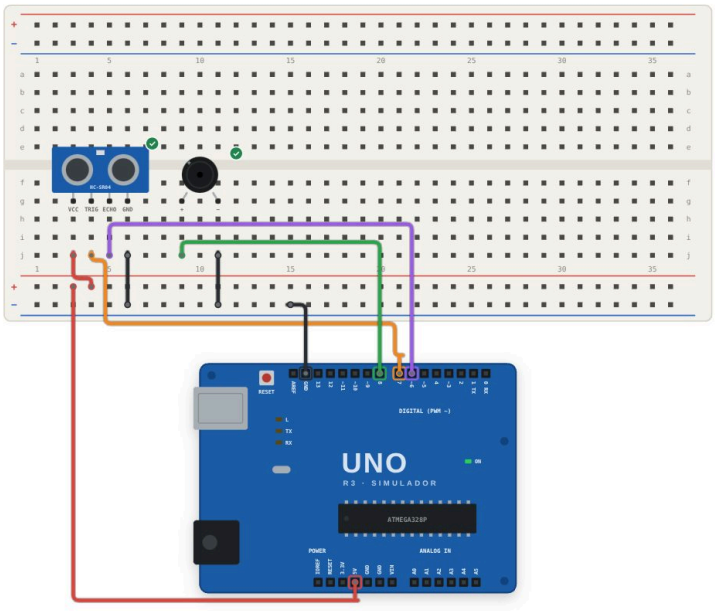
Mueve la mano sobre el sensor: cuanto más cerca, más aguda la nota.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor ultrasónico (Ultrasónico HC-SR04)
- 1 × Parlante (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor ultrasónico (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 Parlante (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
// Theremin ultrasónico
const int TRIG = 7, ECHO = 6, PARLANTE = 8;

void setup() {
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
}

void loop() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long cm = pulseIn(ECHO, HIGH) / 58;
  if (cm > 3 && cm < 50) tone(PARLANTE, map(cm, 3, 50, 1500, 200));
  else noTone(PARLANTE);
  delay(40);
}
```

💡 ¿Cómo funciona?

map() convierte la distancia (de 3 a 50 cm) en una frecuencia (de 1500 a 200 Hz). Fuera de ese rango el instrumento queda en silencio.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa.

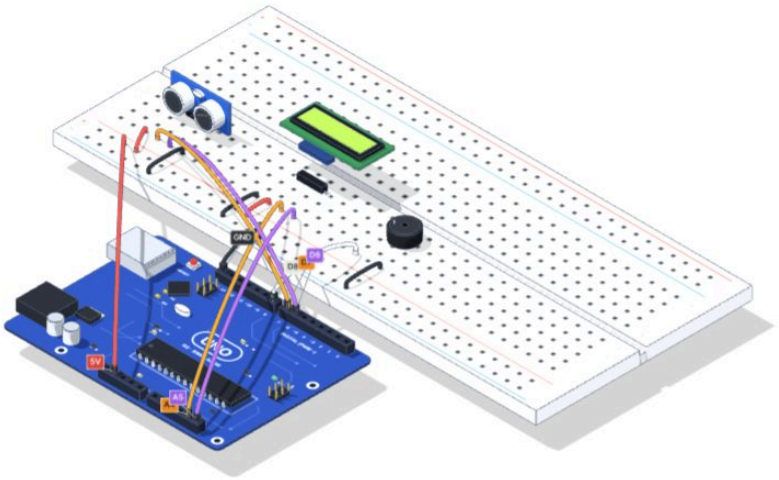
🏆 **Reto:** Haz que solo suenen notas de la escala musical.

PROYECTO 92 · DISTANCIA Y MOVIMIENTO

Contador de visitantes con ultrasonido

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Cuenta cuántas personas pasan frente al sensor de la puerta y lo muestra en la LCD.

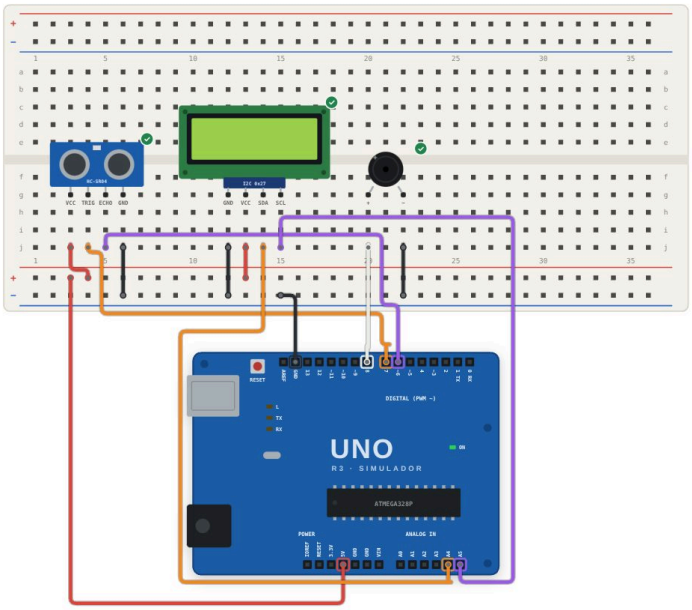


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de la puerta (Ultrasónico HC-SR04)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Bip (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de la puerta (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Bip (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 125 mA.
- Consumo estimado: **~125 mA** funcionando, con picos de **~125 mA**.

2 El programa

```ino
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
const int TRIG = 7, ECHO = 6;
int visitantes = 0;
bool alguien = false;

long medir() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long cm = pulseIn(ECHO, HIGH) / 58;   // en Arduino la división entera no deja decimales
  return cm;
}

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  lcd.print("Visitantes: 0");
}

void loop() {
  long cm = medir();
  if (!alguien && cm < 60) {        // llegó una persona
    alguien = true;
    visitantes++;
    tone(8, 1200, 80);
    lcd.setCursor(12, 0); lcd.print(visitantes);
  }
  if (alguien && cm > 80) alguien = false;   // la persona ya pasó
  delay(100);
}
```

💡 **¿Cómo funciona?**

La variable alguien recuerda que hay una persona frente al sensor, para no contarla muchas veces mientras pasa. Solo cuando se aleja se puede contar a la siguiente.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa.

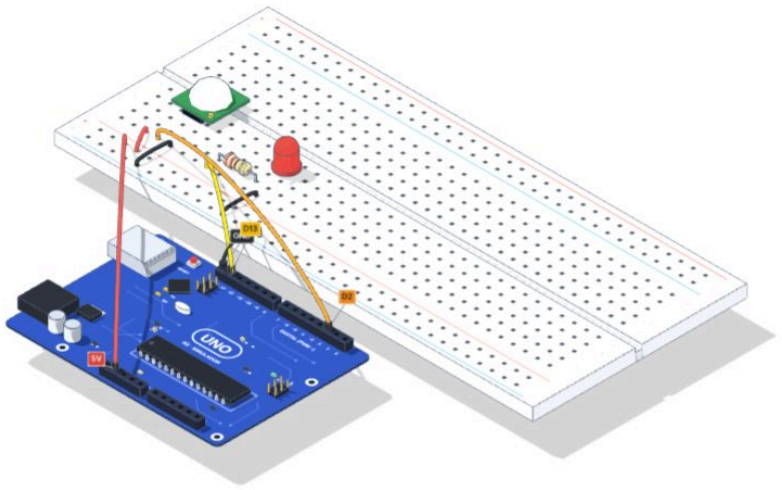
🏆 **Reto:** Agrega un botón para reiniciar la cuenta al empezar el día.

PROYECTO 93 · DISTANCIA Y MOVIMIENTO

Detector de intrusos con PIR

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

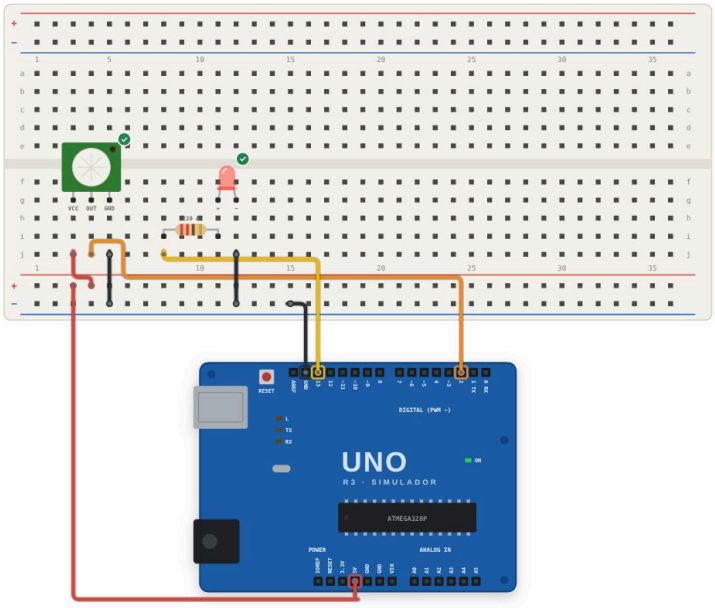
Un sensor PIR detecta el movimiento de una persona y enciende una luz roja.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor PIR
- 1 × Intruso (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor PIR (Sensor PIR)

- VCC → riel **+** (5V)
- OUT (señal) → pin **D2**
- GND → riel **–** (GND)

3 Intruso (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 66 mA.
- Consumo estimado: **~66 mA** funcionando, con picos de **~66 mA**.

2 El programa

```
1  // Detector de intrusos con PIR
2  const int PIR = 2, LUZ = 13;
3
4  void setup() {
5    Serial.begin(9600);
6    pinMode(PIR, INPUT);
7    pinMode(LUZ, OUTPUT);
8  }
9
10 void loop() {
11   if (digitalRead(PIR) == HIGH) {
12     digitalWrite(LUZ, HIGH);
13     Serial.println("¡Movimiento detectado!");
14     delay(1000);
15   } else {
16     digitalWrite(LUZ, LOW);
17   }
18 }
```

💡 ¿Cómo funciona?

El PIR detecta los cambios de calor que produce el cuerpo de una persona al moverse. Cuando eso pasa, su salida se pone en HIGH durante unos segundos.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Activa la casilla «hay movimiento» del sensor PIR.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

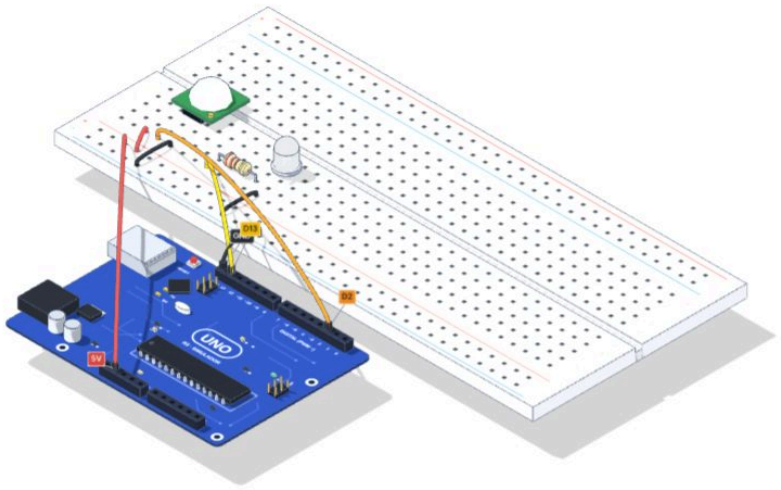
🏆 **Reto:** Cuenta cuántas veces se detectó movimiento.

PROYECTO 94 · DISTANCIA Y MOVIMIENTO

Luz automática de pasillo

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

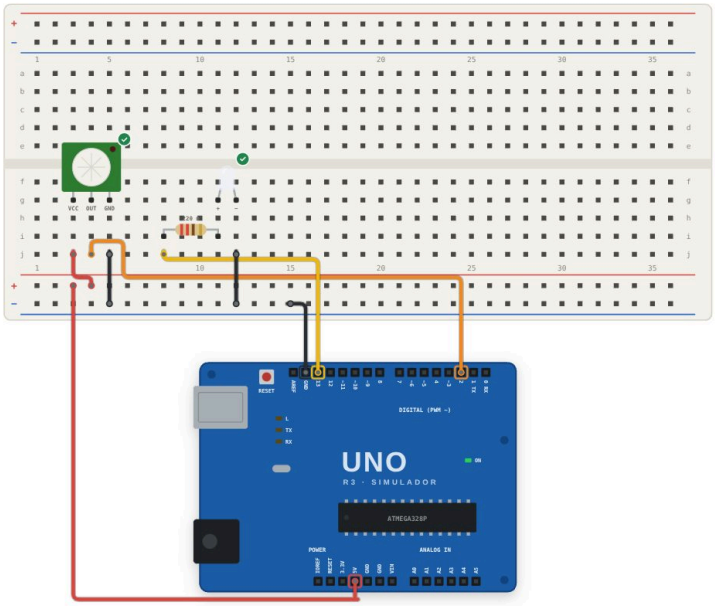
La luz del pasillo se enciende al detectar movimiento y se apaga 15 segundos después de la última persona.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor PIR
- 1 × Luz del pasillo (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor PIR (Sensor PIR)

- VCC → riel **+** (5V)
- OUT (señal) → pin **D2**
- GND → riel **–** (GND)

3 Luz del pasillo (LED blanco)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 66 mA.
- Consumo estimado: **~66 mA** funcionando, con picos de **~66 mA**.

2 El programa

```
1  // Luz de pasillo con PIR
2  const int PIR = 2, LUZ = 13;
3  const unsigned long TIEMPO = 15000;
4  unsigned long ultimoMovimiento = 0;
5
6  void setup() {
7    pinMode(PIR, INPUT);
8    pinMode(LUZ, OUTPUT);
9  }
10
11 void loop() {
12   if (digitalRead(PIR) == HIGH) ultimoMovimiento = millis();
13   if (millis() - ultimoMovimiento < TIEMPO && ultimoMovimiento > 0) digitalWrite(LUZ, HIGH);
14   else digitalWrite(LUZ, LOW);
15 }
```

💡 ¿Cómo funciona?

Cada movimiento actualiza la hora del último movimiento. La luz queda encendida mientras no hayan pasado 15 segundos desde entonces; así se ahorra energía.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Activa la casilla «hay movimiento» del sensor PIR.

Con piezas reales: sube el programa.

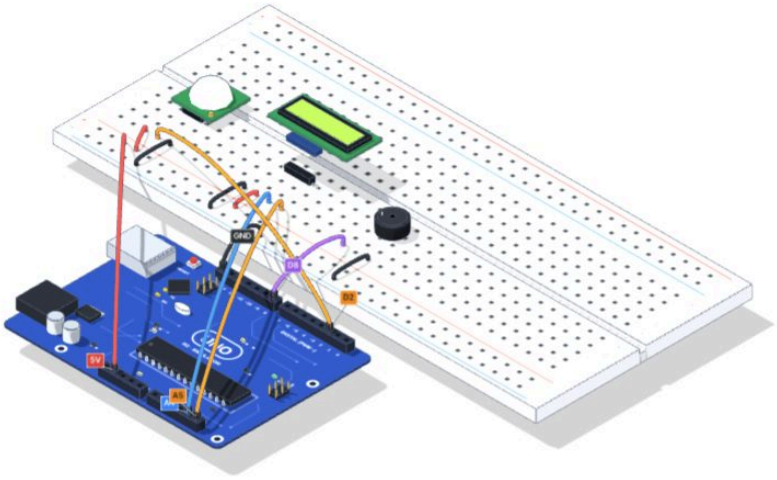
🏆 **Reto:** Combínalo con un sensor de luz para que solo funcione de noche.

PROYECTO 95 · DISTANCIA Y MOVIMIENTO

Saludo automático en la entrada

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Cuando alguien llega a la entrada, la LCD muestra "¡Bienvenido!" y suena una melodía corta.

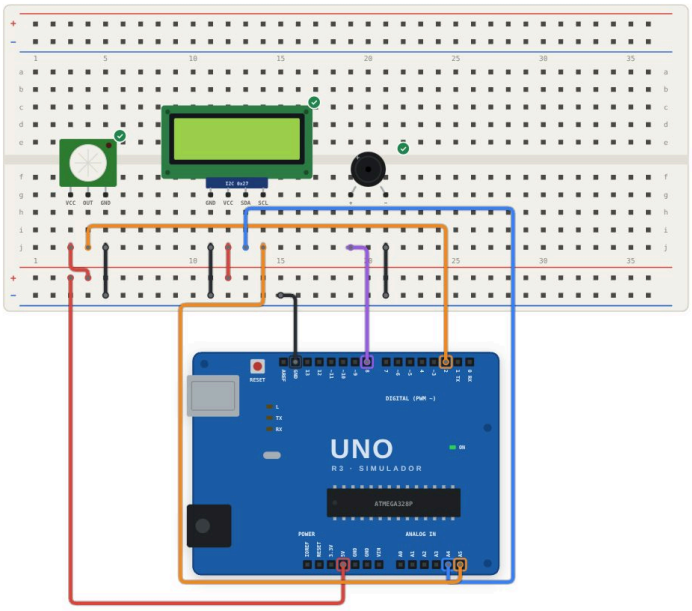


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor PIR
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Melodía (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor PIR (Sensor PIR)

- VCC → riel **+** (5V)
- OUT (señal) → pin **D2**
- GND → riel **–** (GND)

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Melodía (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 111 mA.
- Consumo estimado: **~111 mA** funcionando, con picos de **~111 mA**.

2 El programa

```ino
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
const int PIR = 2, PARLANTE = 8;

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(PIR, INPUT);
  lcd.print("Te esperamos...");
}

void loop() {
  if (digitalRead(PIR) == HIGH) {
    lcd.clear();
    lcd.print("  Bienvenido!");
    lcd.setCursor(0, 1);
    lcd.print(" Pasa adelante");
    tone(PARLANTE, 523, 150); delay(180);
    tone(PARLANTE, 659, 150); delay(180);
    tone(PARLANTE, 784, 300); delay(3000);
    lcd.clear();
    lcd.print("Te esperamos...");
  }
}
```

💡 ¿Cómo funciona?

Al detectar a alguien, el programa borra la pantalla, escribe el saludo y toca tres notas que suben (do, mi, sol). Luego vuelve al mensaje de espera.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Activa la casilla «hay movimiento» del sensor PIR.

Con piezas reales: sube el programa.

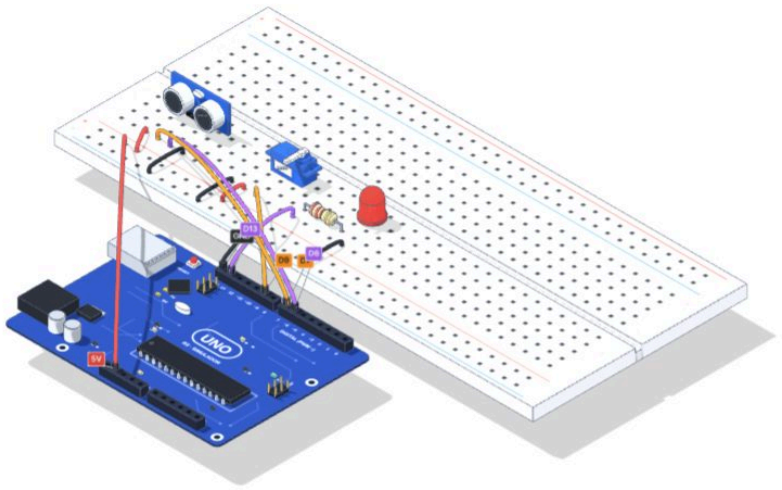
🏆 **Reto:** Cuenta cuántas personas han llegado y muéstralo en la segunda línea.

PROYECTO 96 · DISTANCIA Y MOVIMIENTO

Radar con servo y ultrasonido

★★★ Avanzado · Tiempo: 45 a 90 minutos

Un servo gira el sensor ultrasónico de lado a lado y el monitor muestra la distancia en cada ángulo, como un radar.

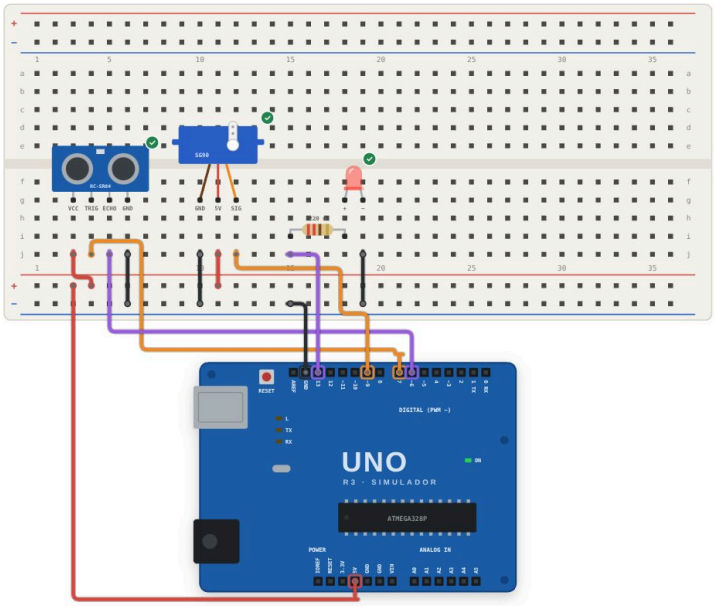


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor ultrasónico (Ultrasónico HC-SR04)
- 1 × Giro del radar (Servo SG90)
- 1 × Objeto cerca (LED)
- 1 × Resistencia 220 Ω
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor ultrasónico (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 Giro del radar (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Objeto cerca (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 230 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~230 mA** funcionando, con picos de **~730 mA**.

2 El programa

```
#include <Servo.h>
Servo giro;
const int TRIG = 7, ECHO = 6, LUZ = 13;

long medir() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long cm = pulseIn(ECHO, HIGH) / 58;   // en Arduino la división entera no deja decimales
  return cm;
}

void barrer(int desde, int hasta, int paso) {
  for (int a = desde; a != hasta + paso; a += paso) {
    giro.write(a);
    delay(60);
    long cm = medir();
    digitalWrite(LUZ, cm < 40);
    Serial.print(a); Serial.print("°: "); Serial.print(cm); Serial.println(" cm");
  }
}

void setup() {
  Serial.begin(9600);
  giro.attach(9);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT); pinMode(LUZ, OUTPUT);
}

void loop() {
  barrer(0, 180, 15);
  barrer(180, 0, -15);
}
```

💡 ¿Cómo funciona?

La función barrer() mueve el servo paso a paso y mide en cada posición. Llamándola de ida y de vuelta, el radar vigila todo el semicírculo.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

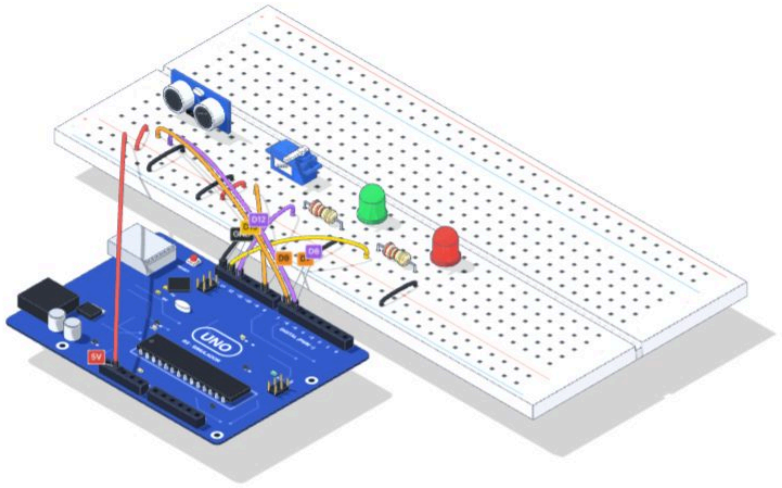
🏆 **Reto:** Guarda el ángulo donde está el objeto más cercano y apunta allí al terminar.

PROYECTO 97 · DISTANCIA Y MOVIMIENTO

Barrera automática de parqueadero

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Cuando un carro se acerca, la barrera (servo) se levanta y baja de nuevo cuando ya pasó.

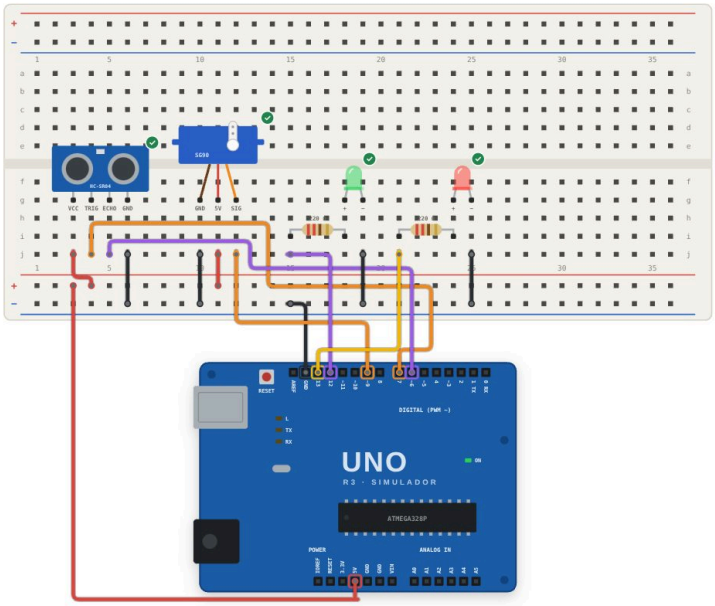


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor del carro (Ultrasónico HC-SR04)
- 1 × Barrera (Servo SG90)
- 1 × Pase (LED)
- 1 × Pare (LED)
- 2 × Resistencia 220 Ω
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor del carro (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 Barrera (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Pase (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Pare (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 245 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~245 mA** funcionando, con picos de **~745 mA**.

2 El programa

```
#include <Servo.h>
Servo barrera;
const int TRIG = 7, ECHO = 6;

long medir() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long cm = pulseIn(ECHO, HIGH) / 58;   // en Arduino la división entera no deja decimales
  return cm;
}

void setup() {
  barrera.attach(9);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
  barrera.write(0);
}

void loop() {
  if (medir() < 20) {
    digitalWrite(13, LOW); digitalWrite(12, HIGH);
    barrera.write(90);                  // subir
    while (medir() < 20) delay(100);    // esperar a que pase
    delay(2000);
    barrera.write(0);                   // bajar
```

```
    digitalWrite(12, LOW);
  }
  digitalWrite(13, HIGH);
  delay(100);
}
```

💡 ¿Cómo funciona?

Mientras el carro sigue frente al sensor, el ciclo while espera. Cuando se va, se esperan 2 segundos más por seguridad y la barrera baja.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa.

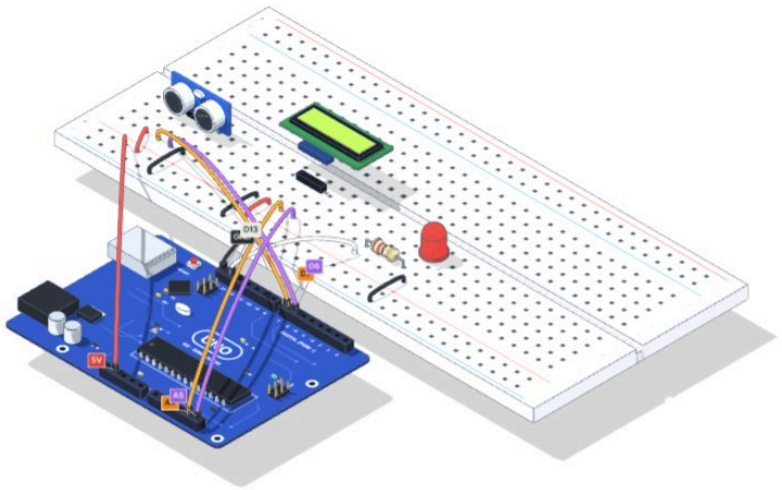
🏆 **Reto:** Agrega un contador de cupos libres en una LCD.

PROYECTO 98 · DISTANCIA Y MOVIMIENTO

Medidor de nivel con ultrasonido

★★☆ Intermedio · Tiempo: 30 a 45 minutos

El sensor, puesto en la tapa de un tanque, calcula qué porcentaje de agua tiene.

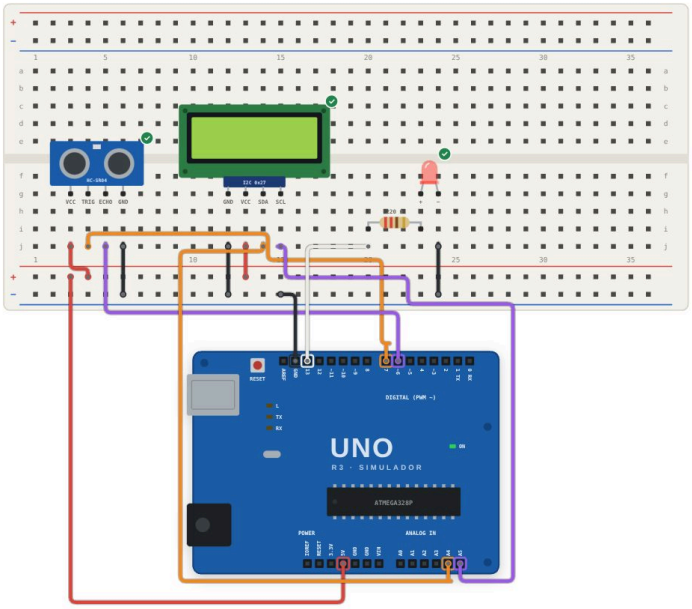


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor en la tapa (Ultrasónico HC-SR04)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Tanque bajo (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor en la tapa (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Tanque bajo (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
const int TRIG = 7, ECHO = 6;
const int ALTURA = 100;        // cm desde el sensor hasta el fondo

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT); pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long aire = pulseIn(ECHO, HIGH) / 58;          // distancia hasta el agua
  int nivel = constrain(map(aire, ALTURA, 5, 0, 100), 0, 100);
  lcd.setCursor(0, 0); lcd.print("Nivel: "); lcd.print(nivel); lcd.print(" %   ");
  lcd.setCursor(0, 1);
  if (nivel < 20) lcd.print("Llenar tanque!  ");
  else lcd.print("                ");
  digitalWrite(13, nivel < 20);
  delay(500);
}
```

💡 **¿Cómo funciona?**

El sensor mide la distancia hasta la superficie del agua. Si el agua está lejos (100 cm), el tanque está vacío; si está cerca (5 cm), lleno. map() lo convierte a porcentaje.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa.

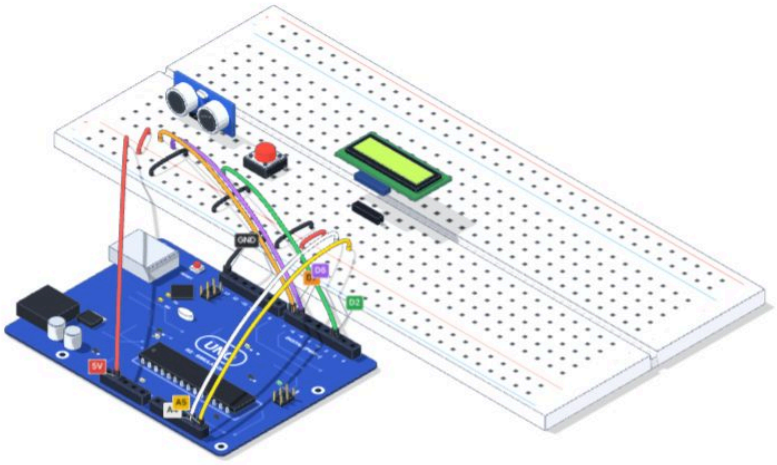
🏆 **Reto:** Calcula los litros si sabes el ancho y el largo del tanque.

PROYECTO 99 · DISTANCIA Y MOVIMIENTO

Regla digital con botón de guardar

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Mide distancias y al presionar el botón guarda la medida en la memoria para compararla.

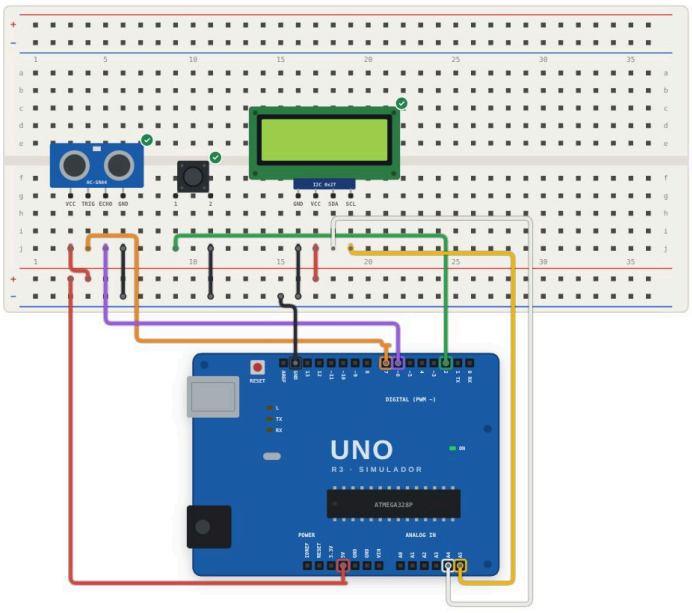


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor ultrasónico (Ultrasónico HC-SR04)
- 1 × Guardar (Botón)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor ultrasónico (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 **Guardar (Botón)**

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Pantalla LCD (Pantalla LCD 16×2 I2C)**

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
const int TRIG = 7, ECHO = 6, BOTON = 2;
long guardada = 0;

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT); pinMode(BOTON, INPUT_PULLUP);
}

void loop() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long cm = pulseIn(ECHO, HIGH) / 58;
  if (digitalRead(BOTON) == LOW) guardada = cm;
  lcd.setCursor(0, 0); lcd.print("Ahora:   "); lcd.print(cm); lcd.print(" cm   ");
  lcd.setCursor(0, 1); lcd.print("Guardada:"); lcd.print(guardada); lcd.print(" cm   ");
  delay(250);
}
```

💡 ¿Cómo funciona?

La variable guardada solo cambia cuando se presiona el botón; mientras tanto la primera línea sigue midiendo. Así se pueden comparar dos medidas.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.
- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

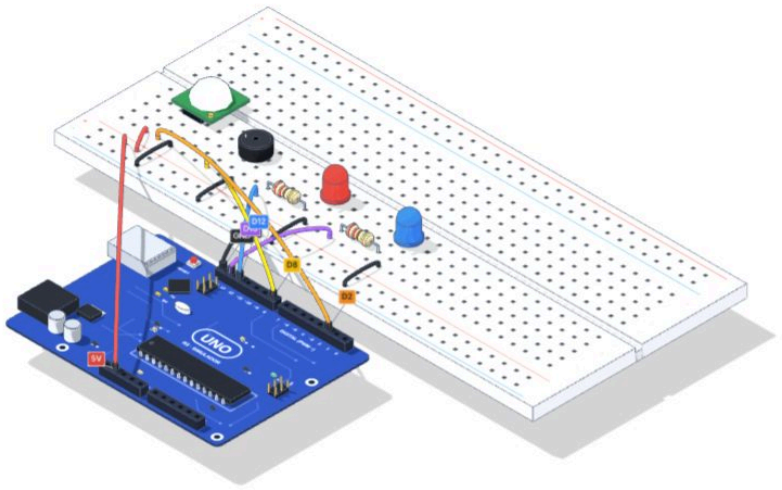
🏆 **Reto:** Muestra la diferencia entre la medida actual y la guardada.

PROYECTO 100 · DISTANCIA Y MOVIMIENTO

Detector de movimiento con sirena y luces

★★☆ Intermedio · Tiempo: 30 a 45 minutos

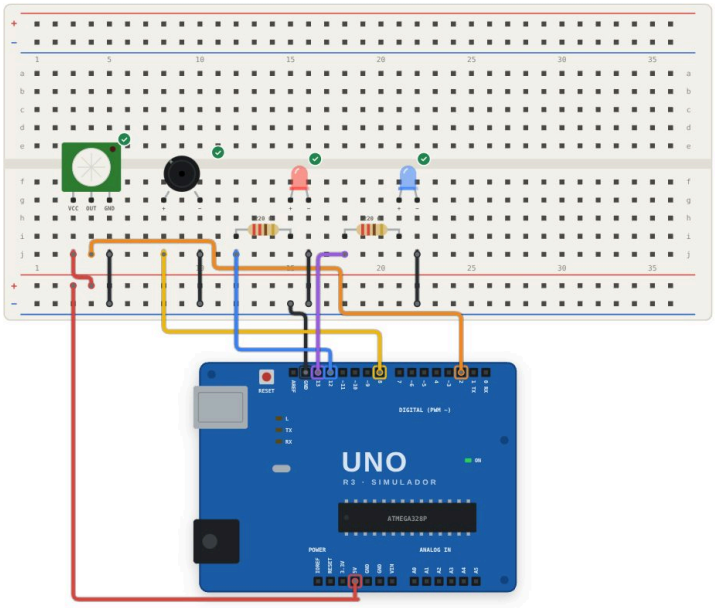
Una alarma de casa: si el PIR detecta movimiento, suenan la sirena y las luces durante 10 segundos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor PIR
- 1 × Sirena (Zumbador)
- 1 × Luz roja (LED)
- 1 × Luz azul (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor PIR (Sensor PIR)

- VCC → riel **+** (5V)
- OUT (señal) → pin **D2**
- GND → riel **–** (GND)

3 Sirena (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Luz roja (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Luz azul (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 111 mA.
- Consumo estimado: **~111 mA** funcionando, con picos de **~111 mA**.

2 El programa

```
// Alarma de casa con PIR
const int PIR = 2, SIRENA = 8, ROJA = 12, AZUL = 13;

void setup() {
  Serial.begin(9600);
  pinMode(PIR, INPUT); pinMode(ROJA, OUTPUT); pinMode(AZUL, OUTPUT);
}

void loop() {
  if (digitalRead(PIR) == HIGH) {
    Serial.println("¡ALARMA!");
    unsigned long inicio = millis();
    while (millis() - inicio < 10000) {         // 10 segundos de alarma
      digitalWrite(ROJA, HIGH); digitalWrite(AZUL, LOW); tone(SIRENA, 1500); delay(200);
      digitalWrite(ROJA, LOW); digitalWrite(AZUL, HIGH); tone(SIRENA, 1000); delay(200);
    }
    noTone(SIRENA); digitalWrite(AZUL, LOW);
  }
}
```

💡 ¿Cómo funciona?

El ciclo while repite el efecto de luces y sirena hasta que pasan 10 segundos, contados con millis().

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Activa la casilla «hay movimiento» del sensor PIR.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

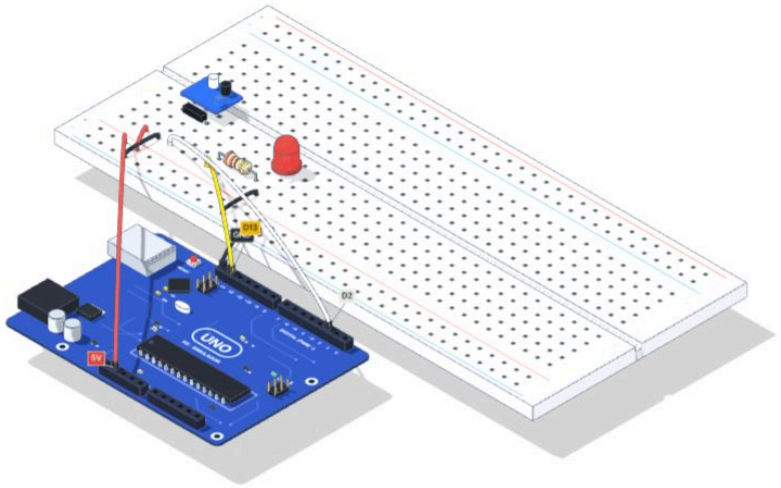
🏆 **Reto:** Agrega un interruptor para armar y desarmar la alarma.

PROYECTO 101 · DISTANCIA Y MOVIMIENTO

Detector de objetos con sensor IR

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

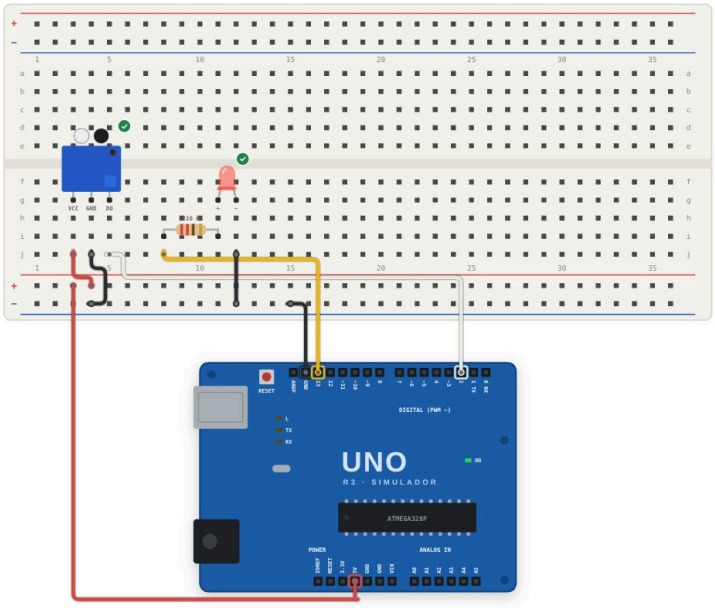
El sensor infrarrojo detecta si hay algo delante y enciende un LED.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor IR (Sensor de obstáculos IR)
- 1 × Objeto (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor IR (Sensor de obstáculos IR)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Objeto (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 70 mA.
- Consumo estimado: **~70 mA** funcionando, con picos de **~70 mA**.

2 El programa

```
1  void setup() {
2    Serial.begin(9600);
3    pinMode(2, INPUT); pinMode(13, OUTPUT);
4  }
5
6  void loop() {
7    bool hayAlgo = digitalRead(2) == LOW;
8    digitalWrite(13, hayAlgo);
9    if (hayAlgo) Serial.println("Objeto detectado");
10   delay(200);
11 }
```

💡 ¿Cómo funciona?

El sensor emite luz infrarroja (invisible) y mira si rebota. Si hay algo delante, la salida baja a LOW. El tornillo azul del sensor ajusta la distancia.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay un objeto delante».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

🏆 **Reto:** Agrega un pitido que suene al detectar el objeto.

CAPÍTULO 8

Servos y motores

Mover cosas: servos que giran a un ángulo exacto y motores que giran sin parar.

EN ESTE CAPÍTULO · 13 PROYECTOS

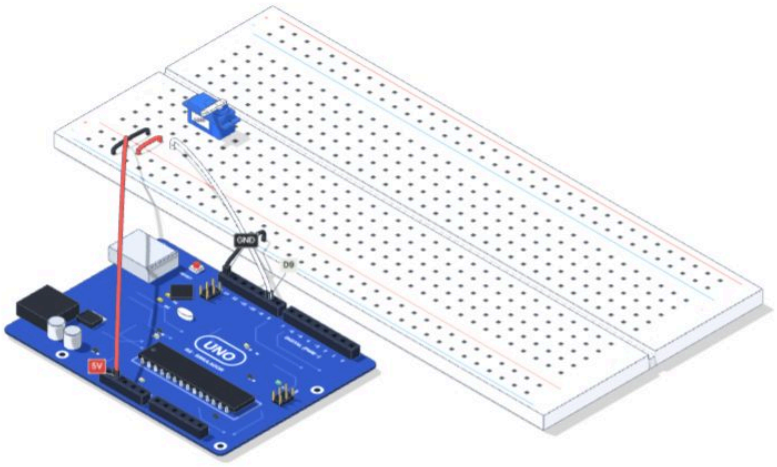
102 Barrido de servo ★☆☆
103 Puerta automática con botón ★☆☆
104 Alimentador de mascotas con temporizador ★★☆
105 Limpiaparabrisas con sensor de lluvia ★★☆
106 Motor con botón de encendido ★☆☆
107 Bandera que saluda ★☆☆
108 Reloj de arena con servo ★★☆
109 Ventilador oscilante ★★☆
110 Compuerta de riego con servo ★★☆
111 Seguidor de movimiento con servo ★★★
112 Mano robótica de un dedo ★★☆
113 Semáforo con barrera de tren ★★★
114 Mano que saluda al acercarte ★☆☆

PROYECTO 102 · SERVOS Y MOTORES

Barrido de servo

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

El servo gira de 0 a 180 grados y regresa, una y otra vez.

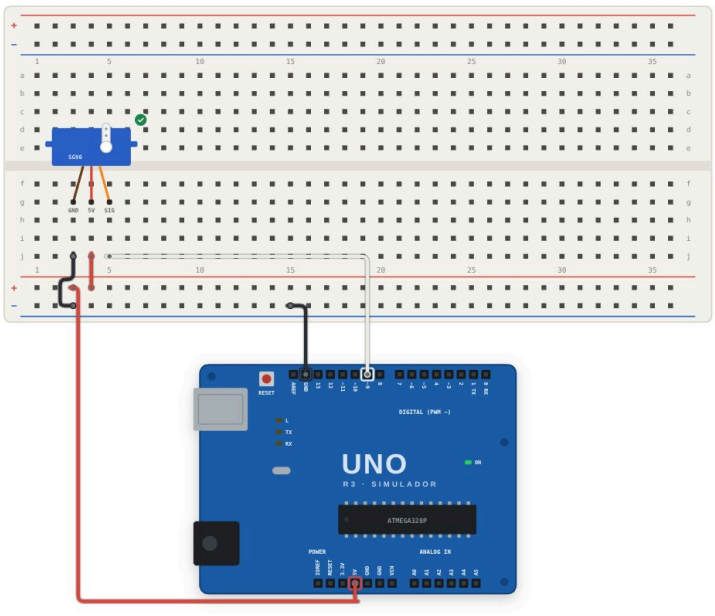


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Servo (Servo SG90)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Servo (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

3 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 200 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata − a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~200 mA** funcionando, con picos de **~700 mA**.

2 El programa

```
#include <Servo.h>
Servo motor;

void setup() {
  motor.attach(9);
}

void loop() {
  for (int a = 0; a <= 180; a++) { motor.write(a); delay(10); }
  for (int a = 180; a >= 0; a--) { motor.write(a); delay(10); }
}
```

💡 ¿Cómo funciona?

Un servo no gira libremente como un motor normal: va al ángulo exacto que le indiques. Avanzando de grado en grado se ve un movimiento suave.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

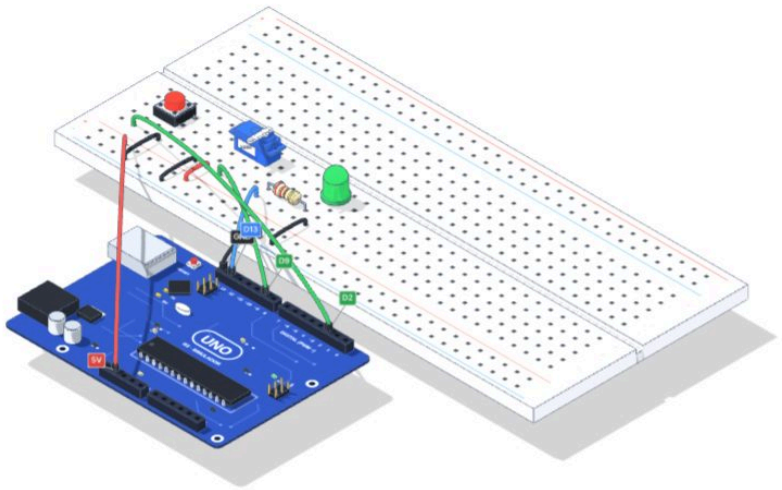
🏆 **Reto:** Haz que el barrido sea solo entre 30 y 150 grados.

PROYECTO 103 · SERVOS Y MOTORES

Puerta automática con botón

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

Al presionar el botón la puerta (servo) se abre, espera 3 segundos y se cierra.

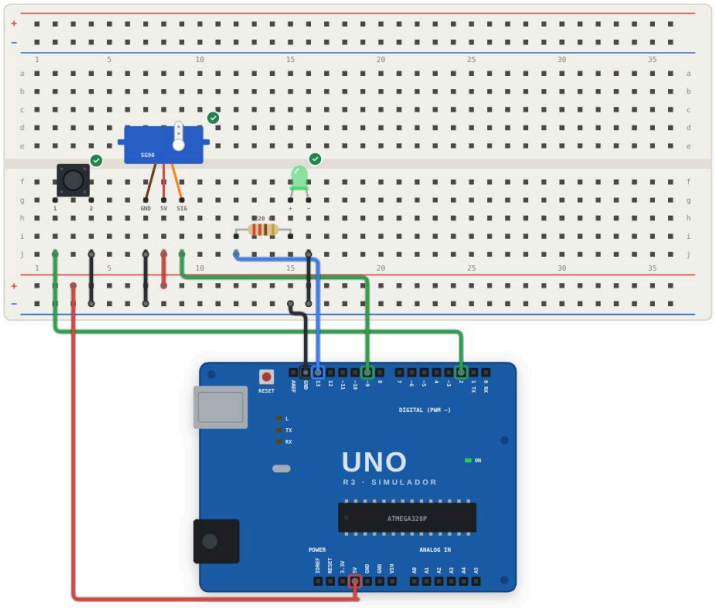


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Abrir (Botón)
- 1 × Puerta (Servo SG90)
- 1 × Puerta abierta (LED)
- 1 × Resistencia 220 Ω
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Abrir (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 Puerta (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Puerta abierta (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 215 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~215 mA** funcionando, con picos de **~715 mA**.

2 El programa

```
#include <Servo.h>
Servo puerta;
const int BOTON = 2, LUZ = 13;

void setup() {
  puerta.attach(9);
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(LUZ, OUTPUT);
  puerta.write(0);          // cerrada
}

void loop() {
  if (digitalRead(BOTON) == LOW) {
    digitalWrite(LUZ, HIGH);
    puerta.write(90);       // abierta
    delay(3000);
    puerta.write(0);
    digitalWrite(LUZ, LOW);
  }
}
```

💡 **¿Cómo funciona?**

El servo representa el brazo que mueve la puerta: 0 grados es cerrada y 90 grados abierta. El LED verde avisa mientras está abierta.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

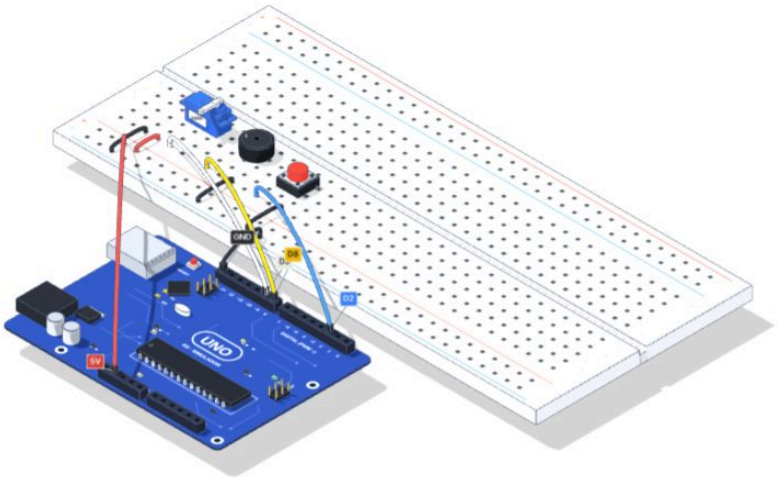
🏆 **Reto:** Haz que la puerta se cierre despacio, de grado en grado.

PROYECTO 104 · SERVOS Y MOTORES

Alimentador de mascotas con temporizador

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Cada cierto tiempo el servo abre la compuerta para soltar comida, y un pitido llama a la mascota.

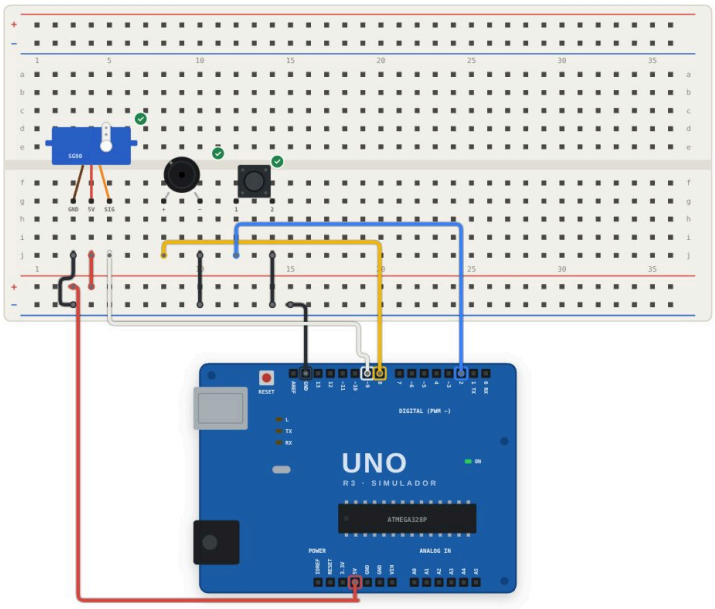


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Compuerta (Servo SG90)
- 1 × Llamado (Zumbador)
- 1 × Dar comida ya (Botón)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Compuerta (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

3 Llamado (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Dar comida ya (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 230 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~230 mA** funcionando, con picos de **~730 mA**.

2 El programa

```
#include <Servo.h>
Servo compuerta;
const unsigned long CADA = 20000;   // 20 s para probar (en la vida real: 8 horas)
unsigned long ultima = 0;

void servir() {
  tone(8, 1000, 200); delay(300); tone(8, 1300, 200);
  compuerta.write(80);   // abrir
  delay(1000);
  compuerta.write(0);    // cerrar
  ultima = millis();
  Serial.println("Comida servida");
}

void setup() {
  Serial.begin(9600);
  compuerta.attach(9);
  compuerta.write(0);
  pinMode(2, INPUT_PULLUP);
}

void loop() {
  if (millis() - ultima > CADA || digitalRead(2) == LOW) servir();
}
```

💡 **¿Cómo funciona?**

La función servir() hace todo el proceso: pitido, abrir, esperar y cerrar. Se llama cuando pasa el tiempo programado o cuando se presiona el botón.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

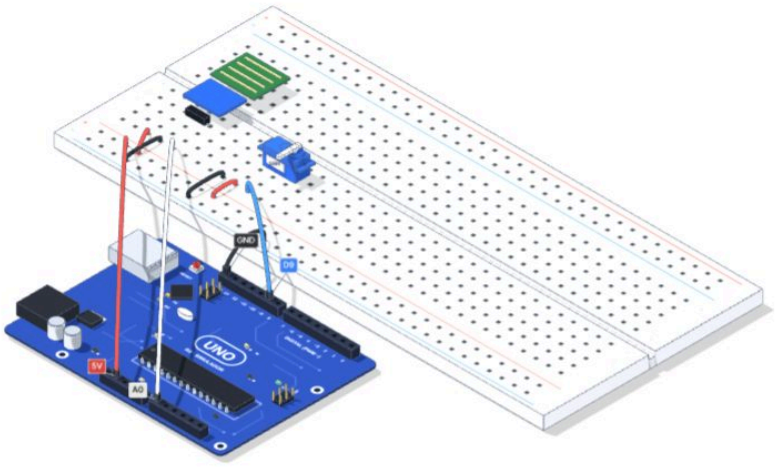
🏆 **Reto:** Cuenta cuántas porciones se han servido en el día.

PROYECTO 105 · SERVOS Y MOTORES

Limpiaparabrisas con sensor de lluvia

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Cuando el sensor detecta lluvia, el servo mueve el limpiaparabrisas; más lluvia, más rápido.

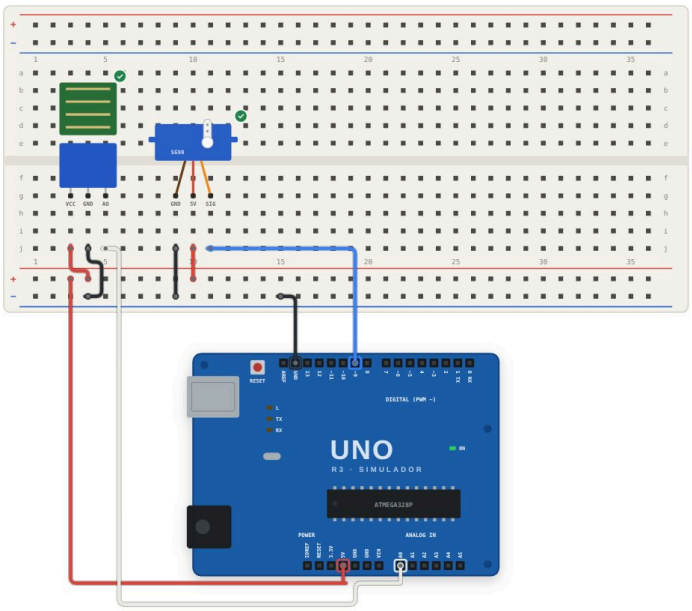


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de lluvia
- 1 × Limpiaparabrisas (Servo SG90)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de lluvia (Sensor de lluvia)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- AO (señal) → pin **A0**

3 **Limpiaparabrisas (Servo SG90)**

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 205 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~205 mA** funcionando, con picos de **~705 mA**.

2 El programa

```
#include <Servo.h>
Servo plumilla;

void setup() {
  Serial.begin(9600);
  plumilla.attach(9);
}

void loop() {
  int lectura = analogRead(A0);          // seca ≈ 1020, mojada < 500
  if (lectura < 900) {
    int espera = map(lectura, 300, 900, 3, 15);   // más agua: menos espera
    espera = constrain(espera, 3, 15);
    for (int a = 0; a <= 150; a += 3) { plumilla.write(a); delay(espera); }
    for (int a = 150; a >= 0; a -= 3) { plumilla.write(a); delay(espera); }
  } else {
    plumilla.write(0);
  }
}
```

💡 ¿Cómo funciona?

La placa del sensor tiene pistas separadas: el agua las une y baja la lectura analógica. Con menos lectura, el programa usa menos espera entre pasos y el servo va más rápido.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de agua sobre la placa.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

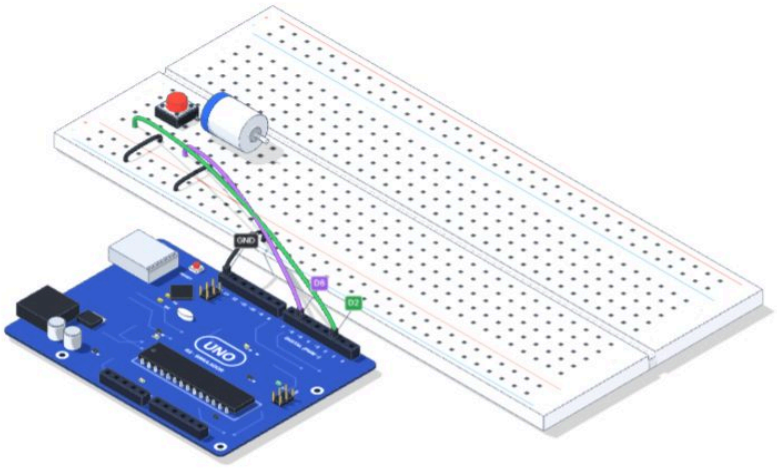
🏆 **Reto:** Agrega un LED que indique si está lloviendo.

PROYECTO 106 · SERVOS Y MOTORES

Motor con botón de encendido

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

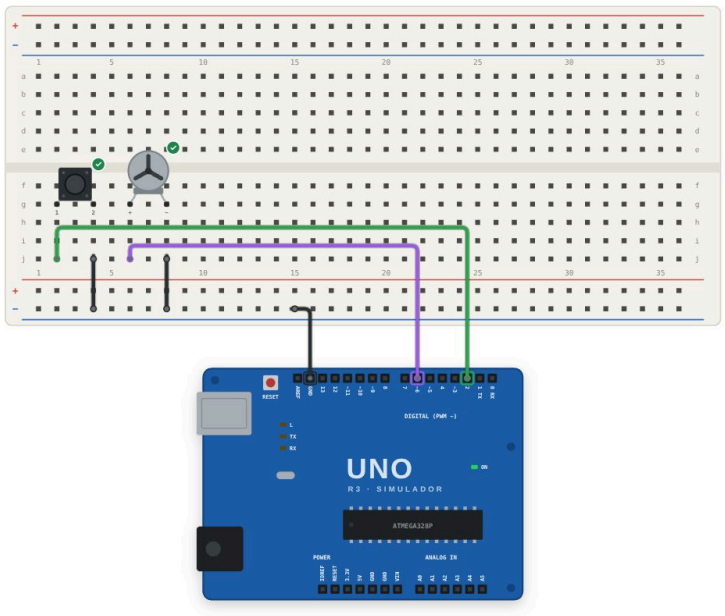
Un botón enciende y apaga un pequeño motor de ventilador.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Encender (Botón)
- 1 × Motor (Motor DC)
- Transistor TIP120 (o 2N2222)
- Diodo 1N4007
- Resistencia 1 kΩ
- Fuente 6–12 V para los motores
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Encender (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Motor (Motor DC)**

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

4 **Para el proyecto real**

- Usa un transistor TIP120 y un diodo para conectar el motor en la vida real.

5 **Energía y controladores**

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motor → necesita transistor:** un pin da solo 20–40 mA y el motor pide 150–250 mA (hasta 1 A al arrancar). Conexión: pin → resistencia de 1 kΩ → base del **TIP120**; motor entre el + de la fuente y el colector; emisor a GND. Pon un **diodo 1N4007** en paralelo con el motor (la raya hacia el +) para frenar los picos de voltaje que dañan el transistor.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al + del motor (el – del motor va al colector del transistor). Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~250 mA** funcionando, con picos de **~1050 mA**.

2 El programa

```
// Motor con botón de encendido
const int BOTON = 2, MOTOR = 6;
bool girando = false;
int anterior = HIGH;

void setup() {
  pinMode(BOTON, INPUT_PULLUP);
  pinMode(MOTOR, OUTPUT);
}

void loop() {
  int ahora = digitalRead(BOTON);
  if (anterior == HIGH && ahora == LOW) {
    girando = !girando;
    if (girando) analogWrite(MOTOR, 200);
    else analogWrite(MOTOR, 0);
    delay(50);
  }
  anterior = ahora;
}
```

💡 ¿Cómo funciona?

Es el mismo interruptor de un toque de la lámpara, pero ahora controla un motor con analogWrite() para poder elegir su velocidad.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

⚠️ **Ten en cuenta**

- Usa un transistor TIP120 y un diodo para conectar el motor en la vida real.

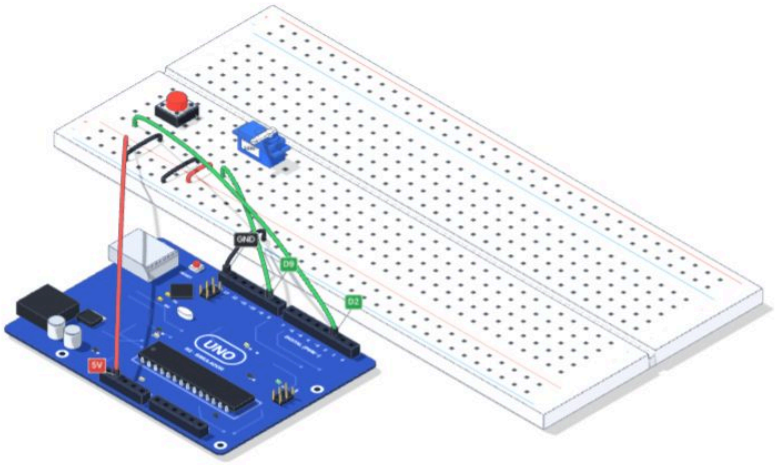
🏆 **Reto:** Agrega dos velocidades: un toque lenta, dos toques rápida.

PROYECTO 107 · SERVOS Y MOTORES

Bandera que saluda

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

Un servo con una banderita la agita tres veces cada vez que presionas el botón.

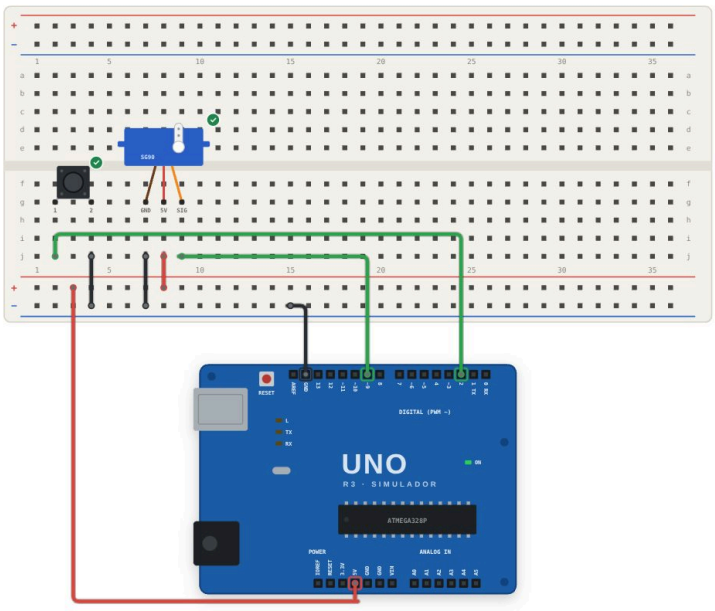


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Saludar (Botón)
- 1 × Brazo de la bandera (Servo SG90)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Saludar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Brazo de la bandera (Servo SG90)**

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 200 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~200 mA** funcionando, con picos de **~700 mA**.

2 El programa

```
#include <Servo.h>
Servo brazo;

void setup() {
  brazo.attach(9);
  pinMode(2, INPUT_PULLUP);
  brazo.write(90);
}

void loop() {
  if (digitalRead(2) == LOW) {
    for (int i = 0; i < 3; i++) {
      brazo.write(50); delay(250);
      brazo.write(130); delay(250);
    }
    brazo.write(90);
  }
}
```

💡 ¿Cómo funciona?

Alternando entre 50 y 130 grados el brazo se mueve de un lado al otro. Un ciclo for repite el movimiento tres veces.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

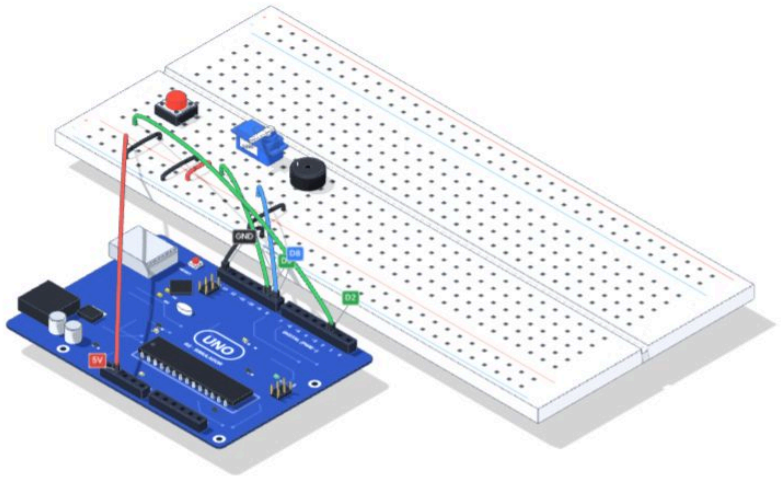
🏆 **Reto:** Haz que salude automáticamente cuando el PIR detecte a alguien.

PROYECTO 108 · SERVOS Y MOTORES

Reloj de arena con servo

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Un servo funciona como aguja de un temporizador de 30 segundos: avanza poco a poco y al final suena una alarma.

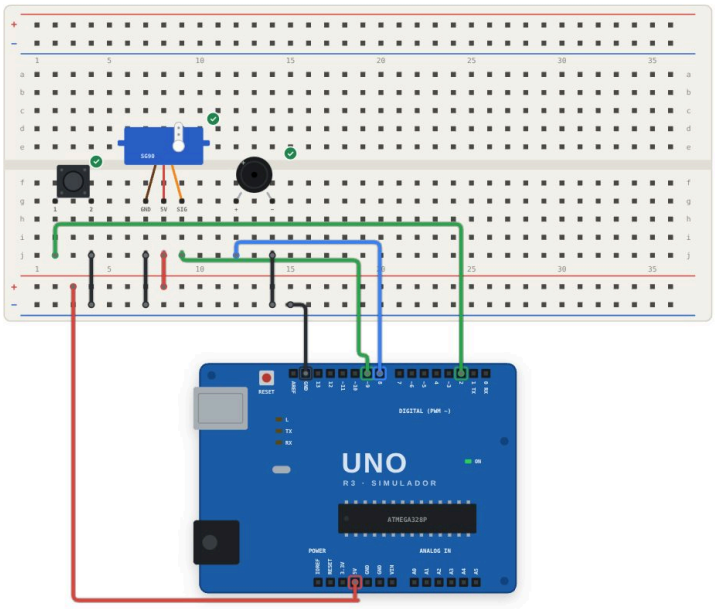


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Iniciar (Botón)
- 1 × Aguja (Servo SG90)
- 1 × Alarma (Zumbador)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Iniciar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 Aguja (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 230 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~230 mA** funcionando, con picos de **~730 mA**.

2 El programa

```ino
#include <Servo.h>
Servo aguja;
const int SEGUNDOS = 30;

void setup() {
  aguja.attach(9);
  pinMode(2, INPUT_PULLUP);
  aguja.write(0);
}

void loop() {
  if (digitalRead(2) == LOW) {
    for (int s = 0; s <= SEGUNDOS; s++) {
      aguja.write(map(s, 0, SEGUNDOS, 0, 180));   // la aguja avanza
      delay(1000);
    }
    for (int i = 0; i < 5; i++) { tone(8, 2000, 150); delay(300); }
    aguja.write(0);
  }
}
```

💡 ¿Cómo funciona?

map() reparte los 180 grados del servo entre los 30 segundos: la aguja avanza 6 grados por segundo y llega al final justo cuando termina el tiempo.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

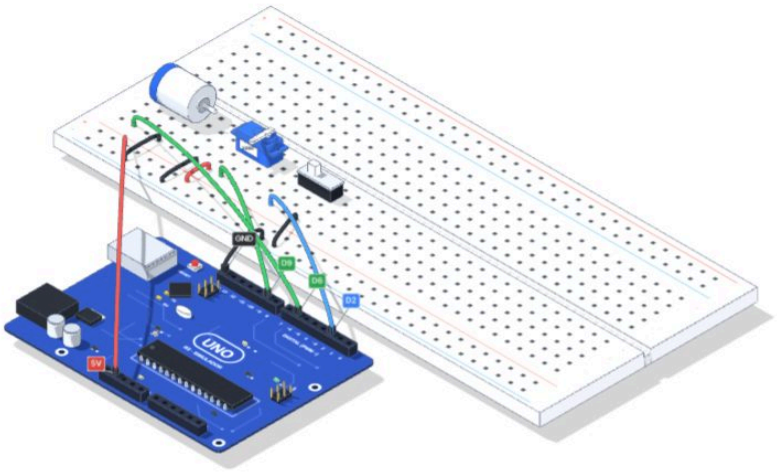
🏆 **Reto:** Usa una perilla para elegir la duración entre 10 y 60 segundos.

PROYECTO 109 · SERVOS Y MOTORES

Ventilador oscilante

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Un motor hace de ventilador y un servo lo gira de lado a lado para repartir el aire.

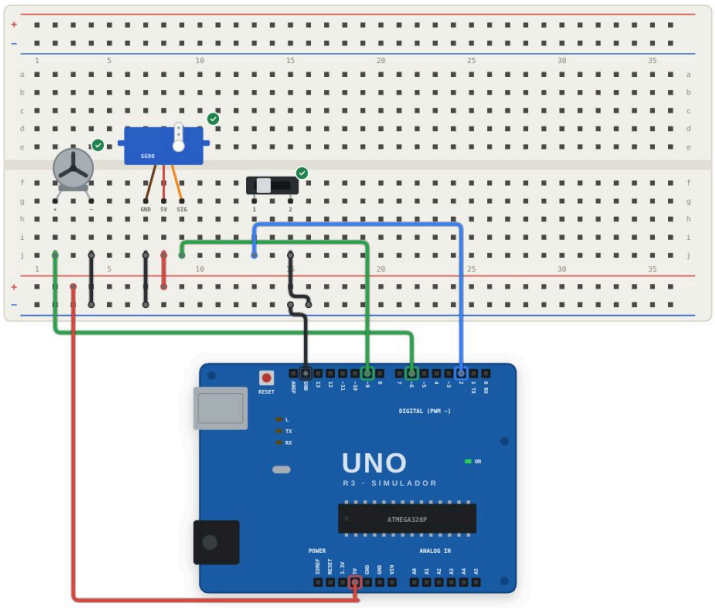


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Aspas (Motor DC)
- 1 × Oscilación (Servo SG90)
- 1 × Encendido (Interruptor)
- Transistor TIP120 (o 2N2222)
- Diodo 1N4007
- Resistencia 1 kΩ
- Fuente 6–12 V para los motores
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Aspas (Motor DC)

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

3 Oscilación (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Encendido (Interruptor)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 200 mA.
- **Motor → necesita transistor:** un pin da solo 20–40 mA y el motor pide 150–250 mA (hasta 1 A al arrancar). Conexión: pin → resistencia de 1 kΩ → base del **TIP120**; motor entre el + de la fuente y el colector; emisor a GND. Pon un **diodo 1N4007** en paralelo con el motor (la raya hacia el +) para frenar los picos de voltaje que dañan el transistor.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al + del motor (el – del motor va al colector del transistor). Nunca alimentes motores desde el pin 5V del Arduino.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~400 mA** funcionando, con picos de **~1700 mA**.

2 El programa

```
#include <Servo.h>
Servo giro;
int angulo = 45, paso = 1;

void setup() {
  giro.attach(9);
  pinMode(2, INPUT_PULLUP);
  pinMode(6, OUTPUT);
}

void loop() {
  if (digitalRead(2) == LOW) {          // interruptor activado
    analogWrite(6, 220);
    angulo += paso;
    if (angulo >= 135 || angulo <= 45) paso = -paso;   // cambiar de sentido
    giro.write(angulo);
    delay(30);
  } else {
    analogWrite(6, 0);
  }
}
```

💡 **¿Cómo funciona?**

La variable paso vale 1 o −1. Al llegar a un extremo se le cambia el signo y el servo empieza a moverse hacia el otro lado.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Cambia la posición del interruptor en su tarjeta del panel «Componentes».

Con piezas reales: sube el programa.

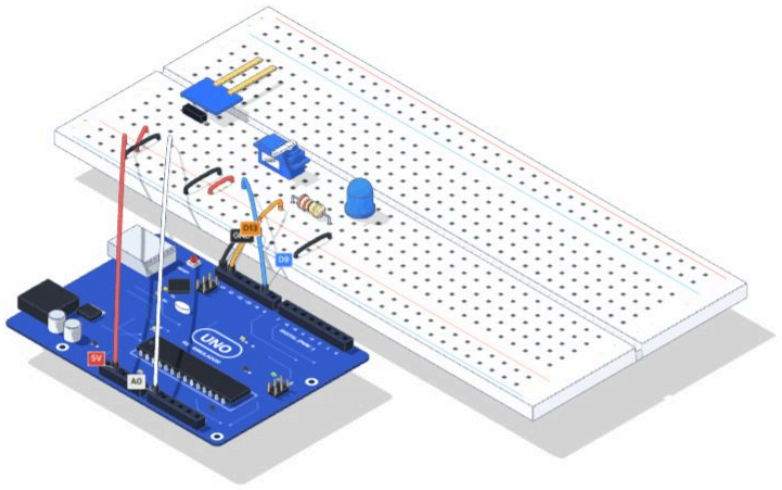
🏆 **Reto:** Agrega una perilla para regular la velocidad de las aspas.

PROYECTO 110 · SERVOS Y MOTORES

Compuerta de riego con servo

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Si la tierra está seca, un servo abre la compuerta del agua; si está húmeda, la cierra.

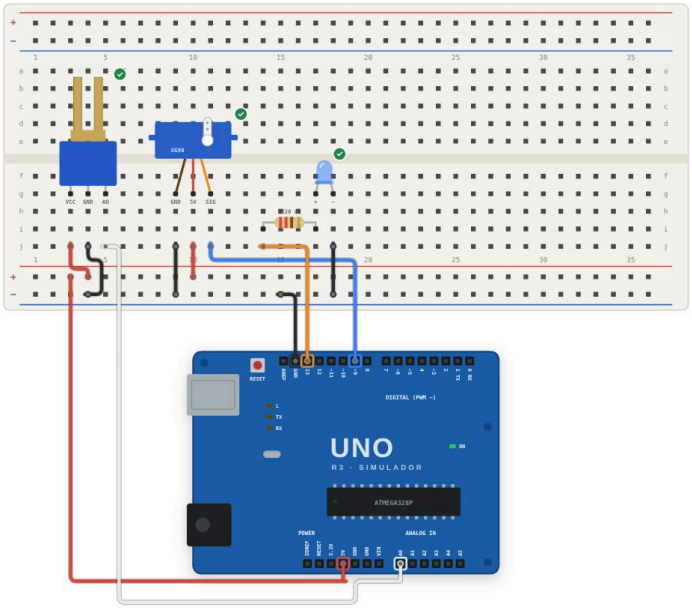


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de humedad de suelo
- 1 × Compuerta (Servo SG90)
- 1 × Regando (LED)
- 1 × Resistencia 220 Ω
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de humedad de suelo (Sensor de humedad de suelo)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- AO (señal) → pin **A0**

3 Compuerta (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Regando (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 220 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~220 mA** funcionando, con picos de **~720 mA**.

2 El programa

```
#include <Servo.h>
Servo compuerta;

void setup() {
  Serial.begin(9600);
  compuerta.attach(9);
  pinMode(13, OUTPUT);
}

void loop() {
  int humedad = analogRead(A0);       // seco ≈ 1000, mojado ≈ 350
  Serial.println(humedad);
  if (humedad > 700) {                // tierra seca
    compuerta.write(90);
    digitalWrite(13, HIGH);
  } else if (humedad < 500) {         // tierra húmeda
    compuerta.write(0);
    digitalWrite(13, LOW);
  }
  delay(500);
}
```

💡 **¿Cómo funciona?**

Entre 500 y 700 el programa no cambia nada: deja la compuerta como estaba. Ese "espacio muerto" evita que la compuerta se abra y cierre sin parar.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de humedad de la tierra.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

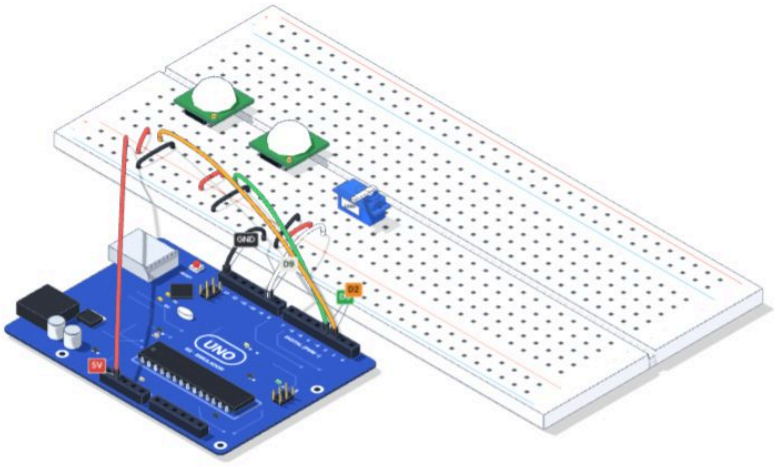
🏆 **Reto:** Muestra el porcentaje de humedad en una LCD.

PROYECTO 111 · SERVOS Y MOTORES

Seguidor de movimiento con servo

★★★ Avanzado · Tiempo: 45 a 90 minutos

Un servo gira hacia la izquierda o la derecha según cuál de dos sensores PIR detecte movimiento.

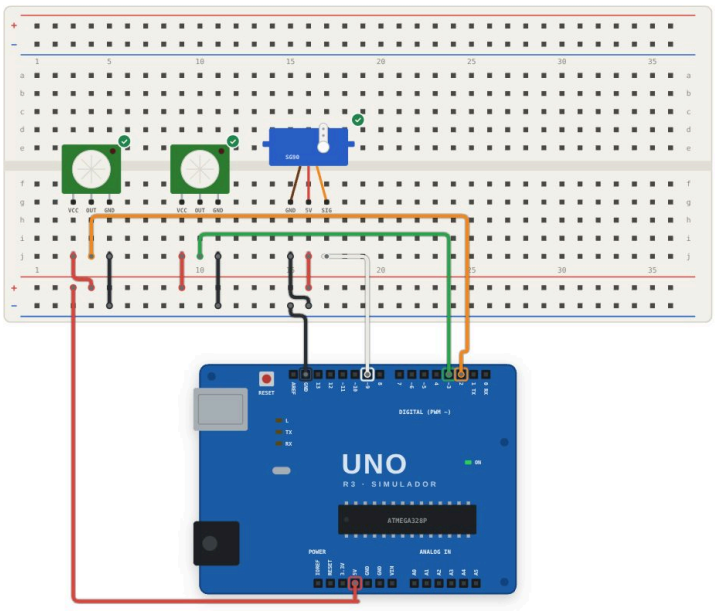


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × PIR izquierdo (Sensor PIR)
- 1 × PIR derecho (Sensor PIR)
- 1 × Cabeza (Servo SG90)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 PIR izquierdo (Sensor PIR)

- VCC → riel **+** (5V)
- OUT (señal) → pin **D2**
- GND → riel **–** (GND)

3 **PIR derecho (Sensor PIR)**

- VCC → riel **+** (5V)
- OUT (señal) → pin **~D3**
- GND → riel **–** (GND)

4 **Cabeza (Servo SG90)**

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 202 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~202 mA** funcionando, con picos de **~702 mA**.

2 El programa

```
#include <Servo.h>
Servo cabeza;

void setup() {
  cabeza.attach(9);
  pinMode(2, INPUT); pinMode(3, INPUT);
  cabeza.write(90);
}

void loop() {
  bool izquierda = digitalRead(2) == HIGH;
  bool derecha = digitalRead(3) == HIGH;
  if (izquierda && !derecha) cabeza.write(150);
  else if (derecha && !izquierda) cabeza.write(30);
  else if (izquierda && derecha) cabeza.write(90);
  delay(200);
}
```

💡 ¿Cómo funciona?

Con dos sensores apuntando a lados distintos, el programa compara cuál detecta movimiento y gira el servo hacia ese lado. Si los dos detectan, mira al centro.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Activa la casilla «hay movimiento» del sensor PIR.

Con piezas reales: sube el programa.

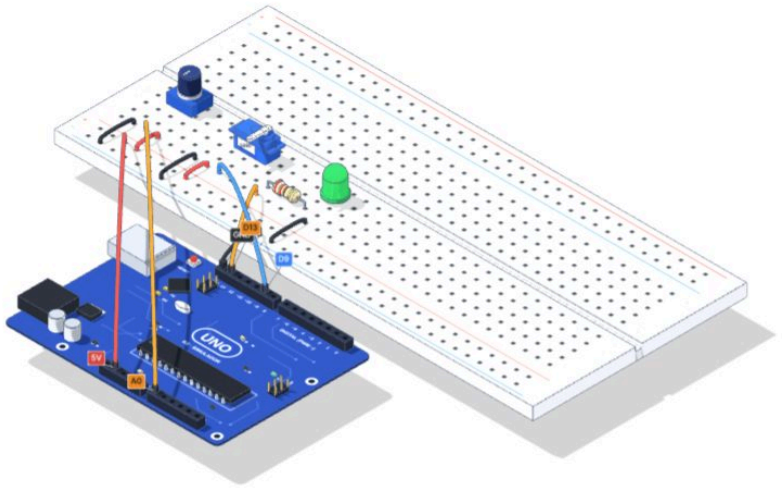
🏆 **Reto:** Haz que vuelva al centro si no hay movimiento durante 5 segundos.

PROYECTO 112 · SERVOS Y MOTORES

Mano robótica de un dedo

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Una perilla dobla y estira un dedo robótico movido por un servo, como una prótesis sencilla.

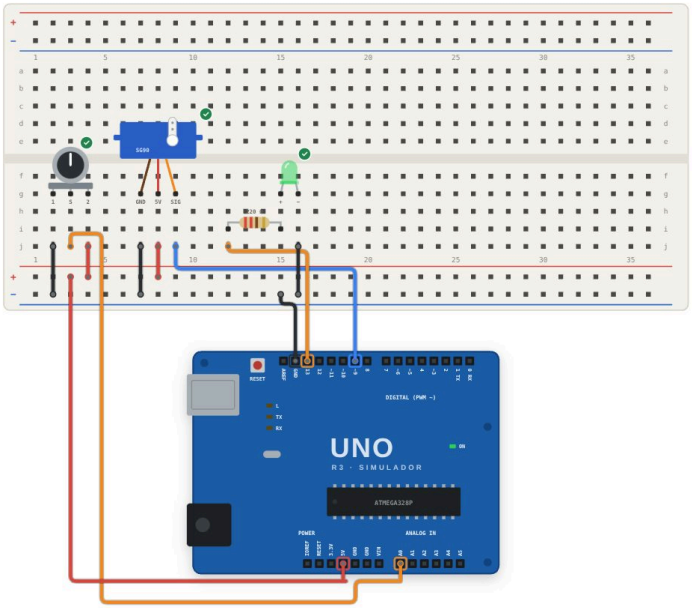


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Control del dedo (Potenciómetro)
- 1 × Dedo (Servo SG90)
- 1 × Agarre (LED)
- 1 × Resistencia 220 Ω
- Condensador 470 µF (recomendado)
- Cartón, pitillos e hilo para el dedo
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Control del dedo (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Dedo (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Agarre (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 216 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~216 mA** funcionando, con picos de **~716 mA**.

2 El programa

```ino
#include <Servo.h>
Servo dedo;

void setup() {
  dedo.attach(9);
  pinMode(13, OUTPUT);
}

void loop() {
  int angulo = map(analogRead(A0), 0, 1023, 0, 160);
  dedo.write(angulo);
  digitalWrite(13, angulo > 120);     // el LED avisa cuando el dedo está cerrado
  delay(15);
}
```

💡 ¿Cómo funciona?

Es la misma idea del servo con perilla, aplicada a un dedo hecho de cartón y un hilo atado al servo. El LED indica cuándo la mano está cerrada.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa.

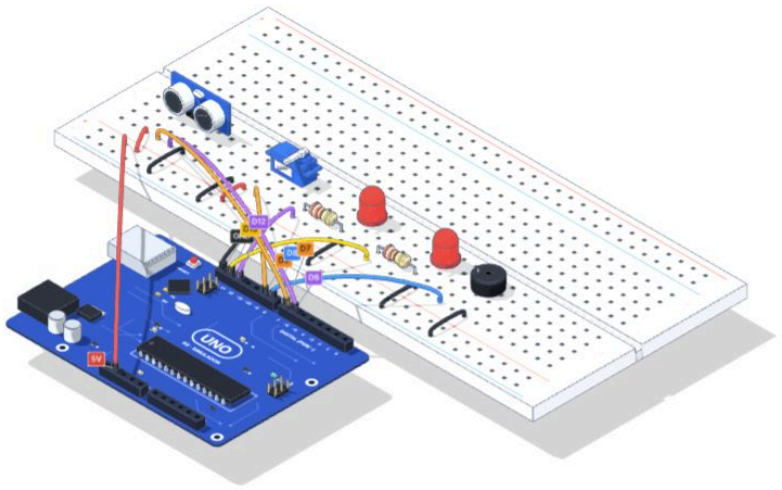
🏆 **Reto:** Usa dos servos y dos perillas para mover dos dedos.

PROYECTO 113 · SERVOS Y MOTORES

Semáforo con barrera de tren

★★★ Avanzado · Tiempo: 45 a 90 minutos

Al detectar el tren, se encienden luces rojas intermitentes, suena una campana y baja la barrera.

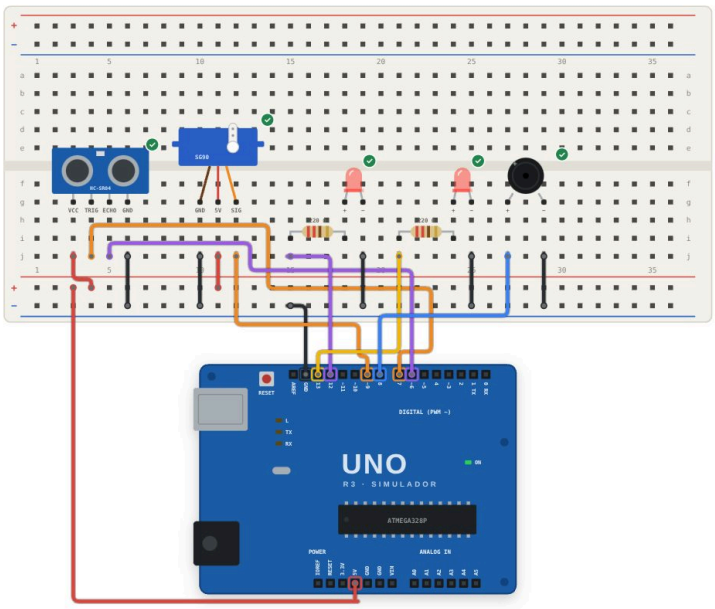


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Detector del tren (Ultrasónico HC-SR04)
- 1 × Barrera (Servo SG90)
- 1 × Luz izquierda (LED)
- 1 × Luz derecha (LED)
- 1 × Campana (Zumbador)
- 2 × Resistencia 220 Ω
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Detector del tren (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 Barrera (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Luz izquierda (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Luz derecha (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Campana (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 275 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~275 mA** funcionando, con picos de **~775 mA**.

2 El programa

```
#include <Servo.h>
Servo barrera;
const int TRIG = 7, ECHO = 6;

long medir() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long cm = pulseIn(ECHO, HIGH) / 58;
  return cm;
}

void setup() {
  barrera.attach(9);
  barrera.write(90);                // levantada
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
}

void loop() {
  if (medir() < 30) {               // viene el tren
    barrera.write(0);               // bajar
    while (medir() < 30) {
```

```
      digitalWrite(12, HIGH); digitalWrite(13, LOW); tone(8, 900, 150); delay(400);
      digitalWrite(12, LOW); digitalWrite(13, HIGH); delay(400);
    }
    digitalWrite(13, LOW);
    delay(1500);
    barrera.write(90);              // subir cuando pasó
  }
}
```

💡 ¿Cómo funciona?

Mientras el tren sigue frente al sensor, las dos luces rojas se alternan y la campana suena. Cuando ya pasó, la barrera vuelve a subir.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa.

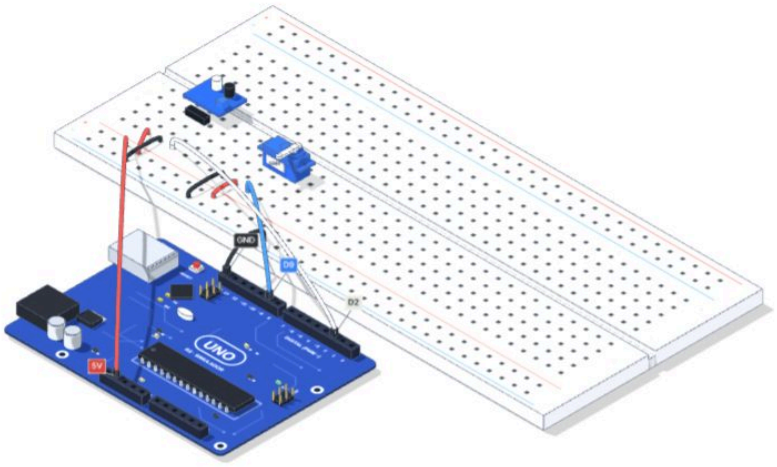
🏆 **Reto:** Agrega una luz verde para los carros cuando la vía esté libre.

PROYECTO 114 · SERVOS Y MOTORES

Mano que saluda al acercarte

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

Cuando alguien se acerca, una mano de cartón en el servo saluda de lado a lado.

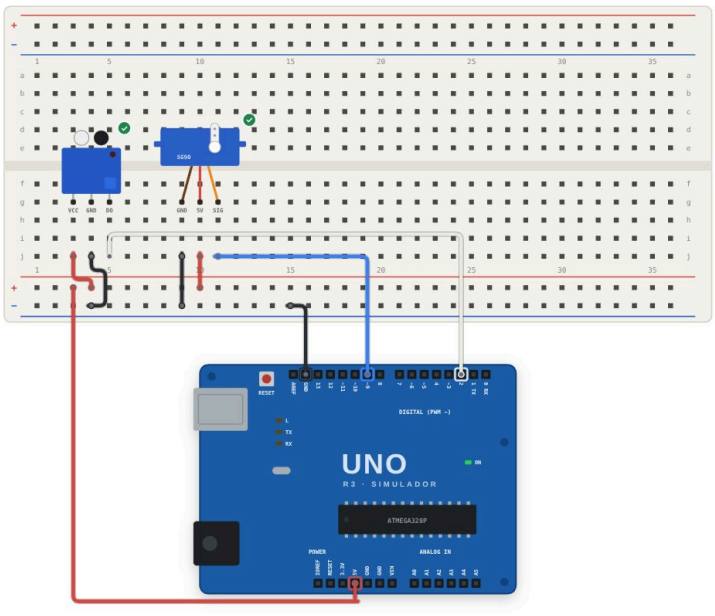


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor IR (Sensor de obstáculos IR)
- 1 × Mano (Servo SG90)
- Condensador 470 µF (recomendado)
- Mano recortada en cartón
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor IR (Sensor de obstáculos IR)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 **Mano (Servo SG90)**

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 205 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~205 mA** funcionando, con picos de **~705 mA**.

2 El programa

```
#include <Servo.h>
Servo mano;

void setup() {
  pinMode(2, INPUT);
  mano.attach(9); mano.write(90);
}

void loop() {
  if (digitalRead(2) == LOW) {
    for (int i = 0; i < 3; i++) { mano.write(50); delay(250); mano.write(130); delay(250); }
    mano.write(90);
    delay(1000);
  }
}
```

💡 ¿Cómo funciona?

El servo va de 50 a 130 grados tres veces, como una mano que saluda, y vuelve al centro.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay un objeto delante».

Con piezas reales: sube el programa.

🏆 **Reto:** Haz que diga "hola" con una melodía en el buzzer.

CAPÍTULO 9

Pantallas

Mostrar mensajes y números en pantallas LCD y OLED, sin depender del computador.

EN ESTE CAPÍTULO · 11 PROYECTOS

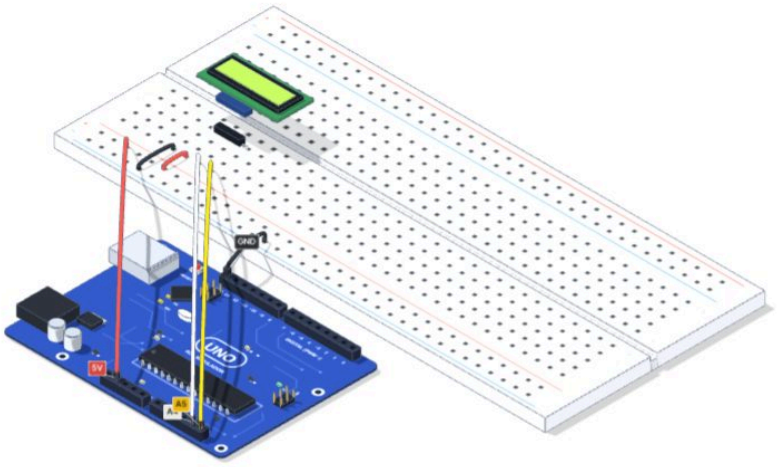
115 Hola mundo en la LCD ★☆☆
116 Cronómetro con LCD y botones ★★☆
117 Mensaje que se desplaza ★★☆
118 Contador de personas en el aula ★★☆
119 Dado con pantalla ★☆☆
120 Nombre en la pantalla OLED ★★☆
121 Reloj de juego de mesa ★★☆
122 Termómetro e higrómetro en OLED ★★★
123 Votación electrónica ★★☆
124 Calculadora de suma con teclado ★★★
125 Mi tarjeta de presentación en LCD ★☆☆

PROYECTO 115 · PANTALLAS

Hola mundo en la LCD

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

Escribe un saludo en la pantalla LCD y un contador de segundos.

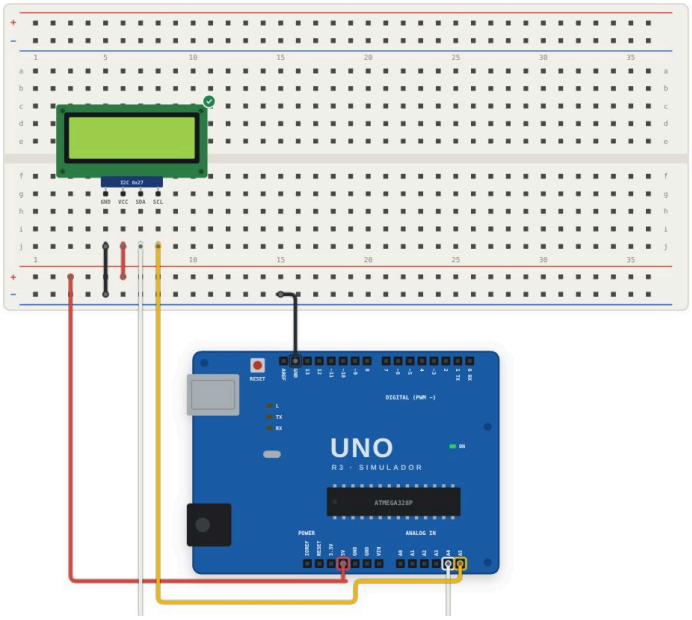


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  lcd.init();          // arranca la pantalla
  lcd.backlight();     // enciende la luz de fondo
  lcd.setCursor(0, 0); // columna 0, fila 0
  lcd.print("Hola, mundo!");
}

void loop() {
  lcd.setCursor(0, 1);
  lcd.print("Segundos: ");
  lcd.print(millis() / 1000);
  delay(200);
}
```

💡 **¿Cómo funciona?**

La LCD tiene 2 filas de 16 caracteres. setCursor() elige dónde escribir y print() escribe texto o números. millis() / 1000 da los segundos desde el encendido.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

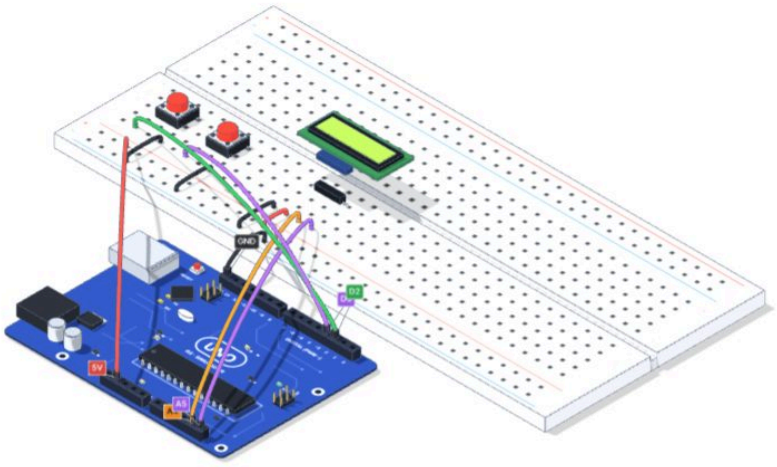
🏆 **Reto:** Escribe tu nombre en la primera fila.

PROYECTO 116 · PANTALLAS

Cronómetro con LCD y botones

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Un cronómetro con dos botones: iniciar/pausar y reiniciar. Muestra minutos, segundos y décimas.

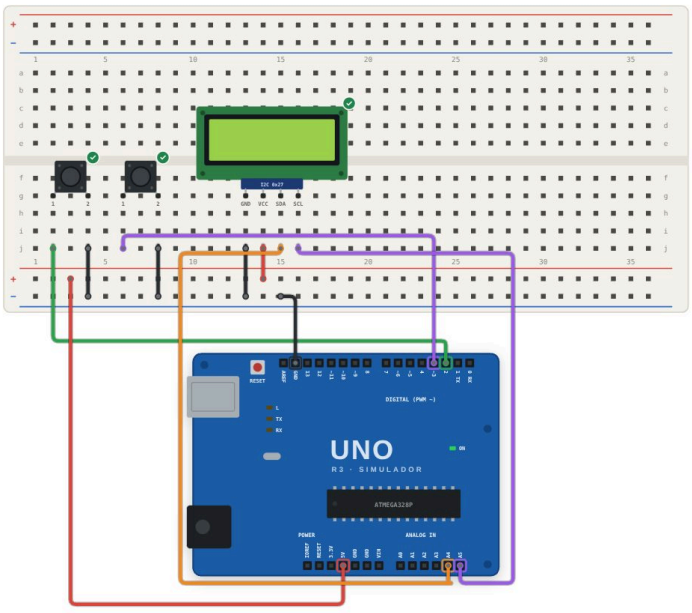


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Iniciar / pausar (Botón)
- 1 × Reiniciar (Botón)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Iniciar / pausar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Reiniciar (Botón)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Pantalla LCD (Pantalla LCD 16×2 I2C)**

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
bool corriendo = false;
unsigned long acumulado = 0, inicio = 0;
int anterior = HIGH;

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP);
  lcd.print("Cronometro");
}

void loop() {
  int ahora = digitalRead(2);
  if (anterior == HIGH && ahora == LOW) {
    if (corriendo) acumulado += millis() - inicio;
    else inicio = millis();
    corriendo = !corriendo;
    delay(50);
  }
  anterior = ahora;
  if (digitalRead(3) == LOW) { corriendo = false; acumulado = 0; }
  unsigned long t = acumulado;
  if (corriendo) t += millis() - inicio;
  int minutos = t / 60000;
  int segundos = (t / 1000) % 60;
  int decimas = (t / 100) % 10;
  lcd.setCursor(0, 1);
  if (minutos < 10) lcd.print("0");
  lcd.print(minutos); lcd.print(":");
  if (segundos < 10) lcd.print("0");
  lcd.print(segundos); lcd.print("."); lcd.print(decimas); lcd.print("   ");
  delay(50);
```

```
35 }
```

💡 **¿Cómo funciona?**

El tiempo se guarda en milisegundos. Para mostrarlo se separa en minutos, segundos y décimas con divisiones y el operador %, que da el residuo de una división.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

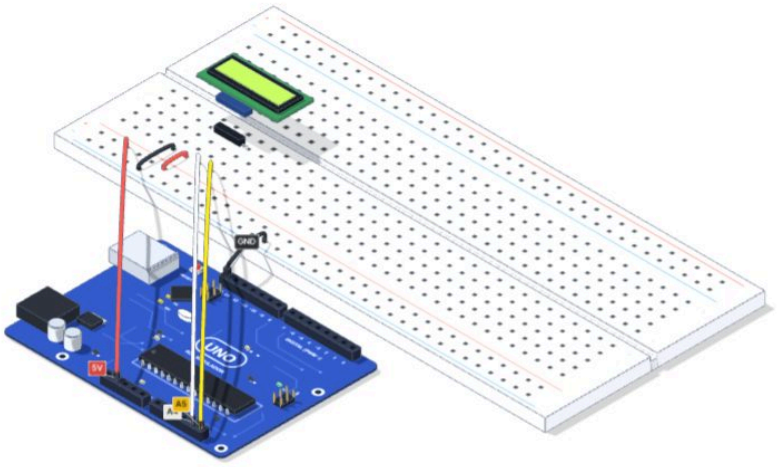
🏆 **Reto:** Agrega un tercer botón para guardar tiempos de vuelta.

PROYECTO 117 · PANTALLAS

Mensaje que se desplaza

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Un letrero luminoso: el mensaje recorre la pantalla de derecha a izquierda.

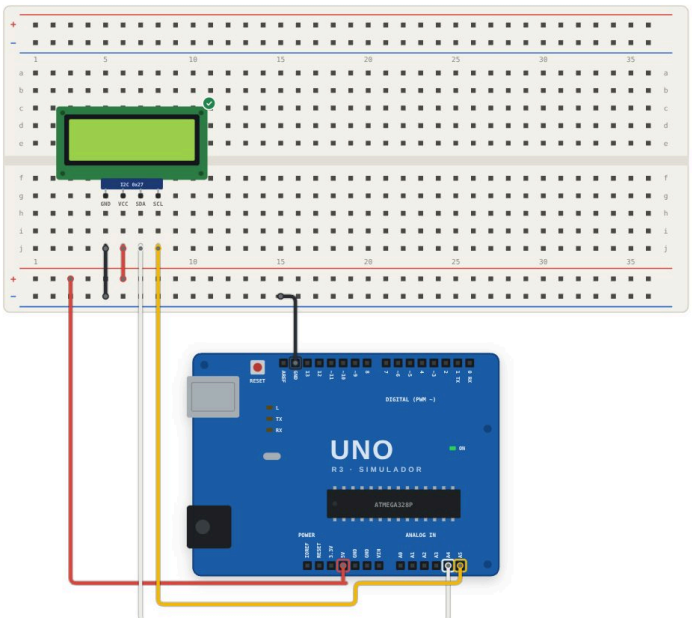


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

3 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
String mensaje = "                Bienvenidos al laboratorio de Arduino!                ";
int pos = 0;

void setup() {
  lcd.init(); lcd.backlight();
  lcd.setCursor(0, 1);
  lcd.print("  ING. ARDUINO");
}

void loop() {
  lcd.setCursor(0, 0);
  lcd.print(mensaje.substring(pos, pos + 16));   // 16 letras desde pos
  pos++;
  if (pos > mensaje.length() - 16) pos = 0;
  delay(250);
}
```

💡 ¿Cómo funciona?

substring() toma un pedazo del texto: 16 letras a partir de la posición pos. Como pos aumenta en cada vuelta, el pedazo visible se corre y parece que el texto se mueve.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

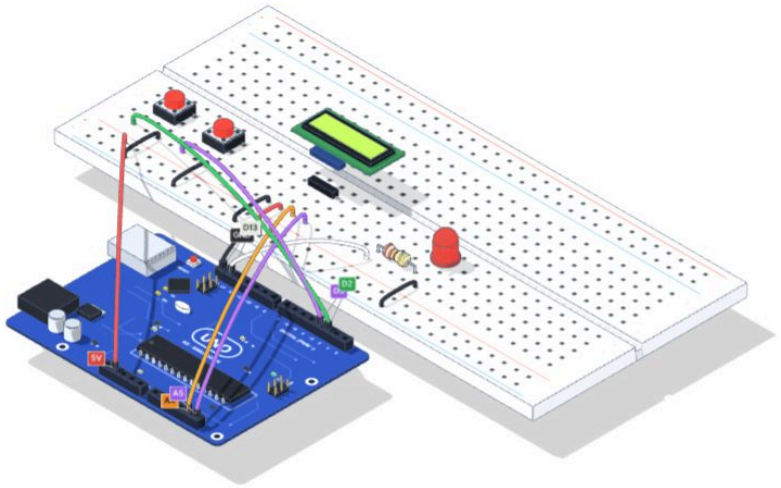
🏆 **Reto:** Escribe el mensaje del día de tu colegio.

PROYECTO 118 · PANTALLAS

Contador de personas en el aula

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Dos botones (entra / sale) llevan la cuenta de personas en el aula y avisan si se supera el cupo.

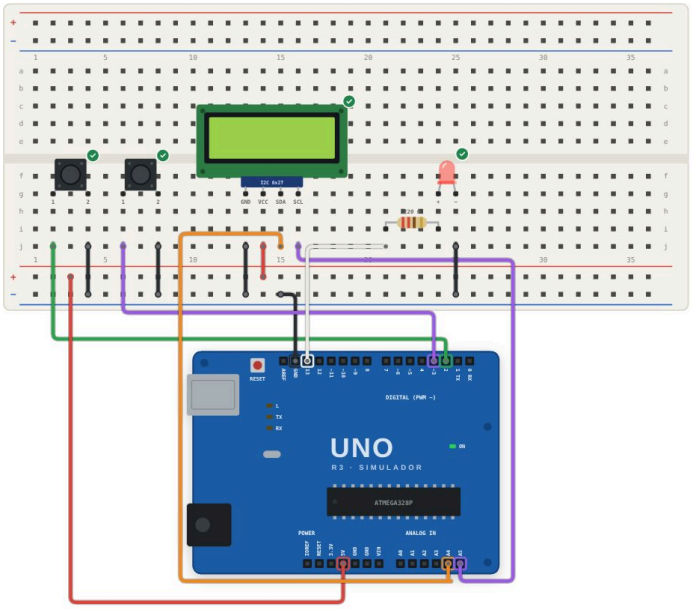


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Entra (Botón)
- 1 × Sale (Botón)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Cupo lleno (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Entra (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 Sale (Botón)

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

5 Cupo lleno (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
const int CUPO = 30;
int personas = 0;

void mostrar() {
  lcd.clear();
  lcd.print("Personas: "); lcd.print(personas);
  lcd.setCursor(0, 1);
  if (personas >= CUPO) lcd.print("CUPO LLENO");
  else { lcd.print("Libres: "); lcd.print(CUPO - personas); }
  digitalWrite(13, personas >= CUPO);
}

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP); pinMode(13, OUTPUT);
  mostrar();
}

void loop() {
  if (digitalRead(2) == LOW) { personas++; mostrar(); delay(300); }
  if (digitalRead(3) == LOW && personas > 0) { personas--; mostrar(); delay(300); }
}
```

💡 ¿Cómo funciona?

La función mostrar() solo se llama cuando cambia la cuenta, así la pantalla no parpadea. Se revisa que personas sea mayor que 0 antes de restar.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

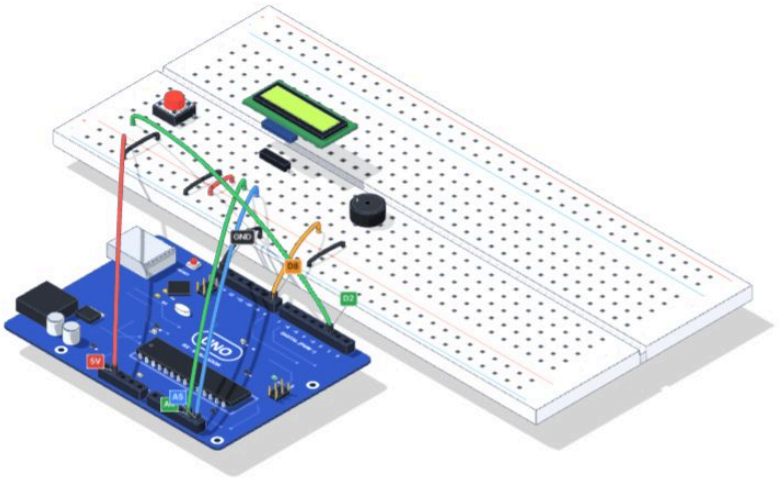
🏆 **Reto:** Cambia los botones por dos sensores infrarrojos en la puerta.

PROYECTO 119 · PANTALLAS

Dado con pantalla

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

Presiona el botón y la LCD muestra el resultado de un dado de 6 caras.

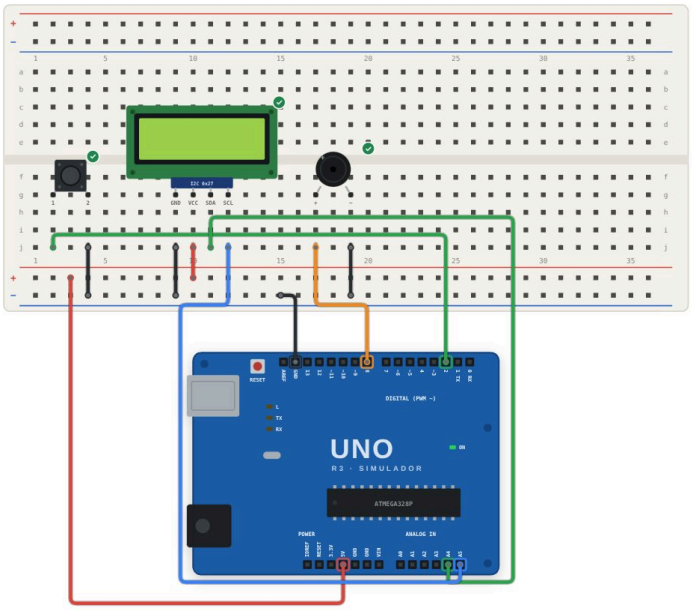


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lanzar (Botón)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Sonido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lanzar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Sonido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT_PULLUP);
  lcd.print("Presiona el boton");
}

void loop() {
  if (digitalRead(2) == LOW) {
    for (int i = 0; i < 8; i++) {          // animación
      lcd.setCursor(0, 1); lcd.print("Rodando... "); lcd.print(random(1, 7));
      tone(8, 600 + i * 80, 40);
      delay(80);
    }
    int dado = random(1, 7);
    lcd.clear();
    lcd.print("Salio el:");
    lcd.setCursor(7, 1);
    lcd.print(dado);
    delay(500);
  }
}
```

💡 ¿Cómo funciona?

La animación muestra números al azar con sonidos que suben, y al final se escribe el resultado definitivo centrado en la segunda fila.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

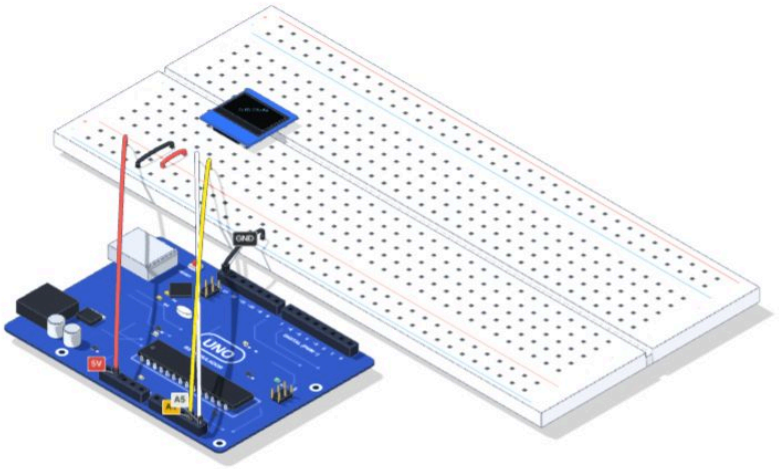
🏆 **Reto:** Haz un dado de 20 caras para juegos de rol.

PROYECTO 120 · PANTALLAS

Nombre en la pantalla OLED

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Escribe tu nombre en una pantalla OLED y dibuja un marco alrededor.

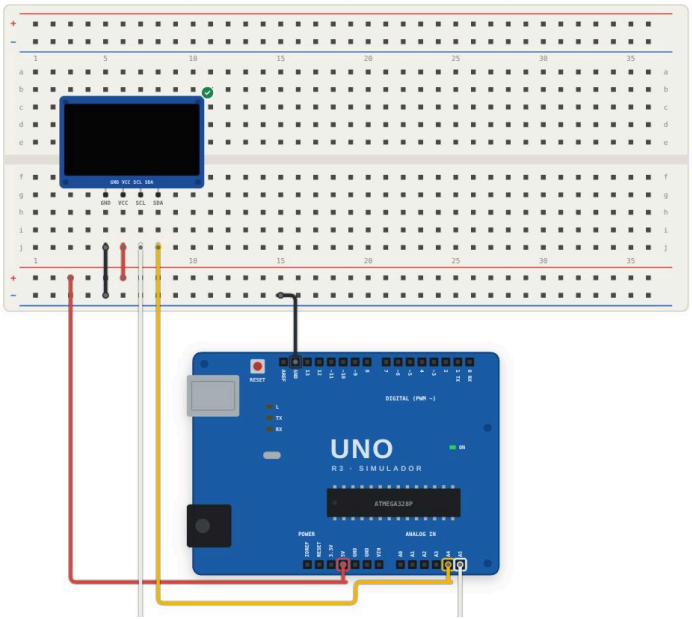


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Pantalla OLED (Pantalla OLED 0.96″)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Adafruit SSD1306 y Adafruit GFX



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Pantalla OLED (Pantalla OLED 0.96'')

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SCL → pin **A5**
- SDA → pin **A4**

3 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 70 mA.
- Consumo estimado: **~70 mA** funcionando, con picos de **~70 mA**.

2 El programa

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 display(128, 64, &Wire, -1);

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.clearDisplay();
  display.drawRect(0, 0, 128, 64, SSD1306_WHITE);   // marco
  display.setTextSize(2);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(16, 16);
  display.print("Hola,");
  display.setCursor(16, 36);
  display.print("Ana!");
  display.display();                                // mostrar lo dibujado
}

void loop() {
}
```

💡 ¿Cómo funciona?

La OLED tiene 128 × 64 puntos. Primero se dibuja todo en la memoria (marco y texto) y display() lo envía a la pantalla de una sola vez.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

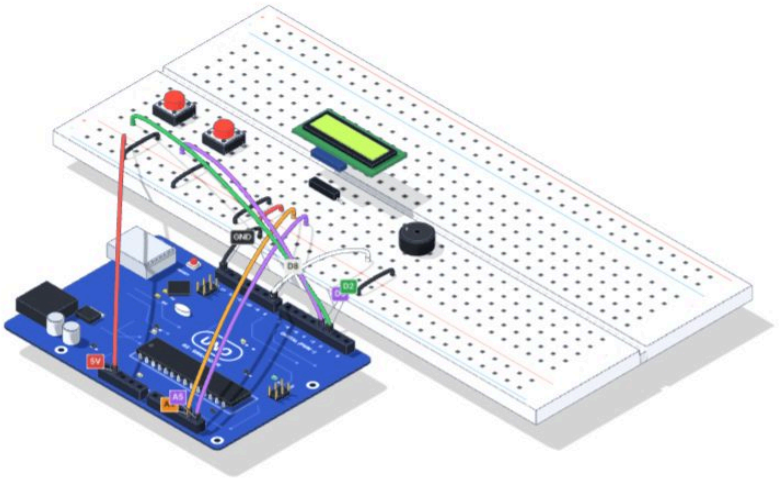
🏆 **Reto:** Dibuja una carita feliz con círculos.

PROYECTO 121 · PANTALLAS

Reloj de juego de mesa

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Un reloj de ajedrez: cada jugador tiene 2 minutos y presiona su botón al terminar su turno.

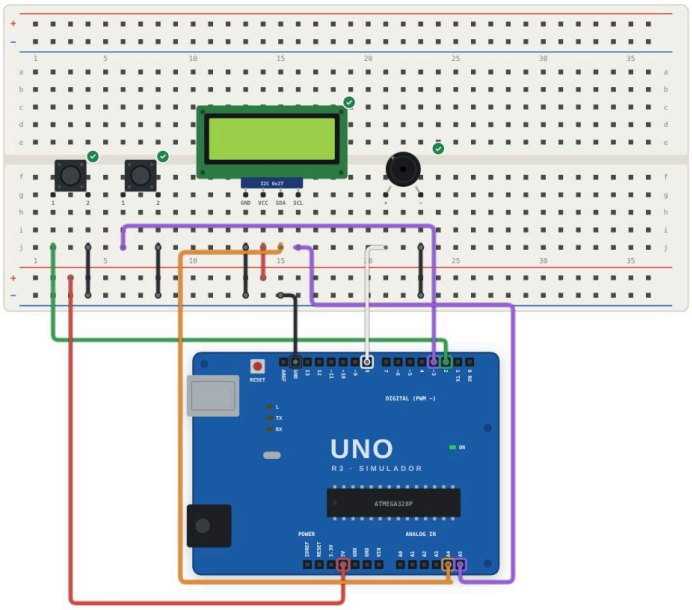


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Jugador 1 (Botón)
- 1 × Jugador 2 (Botón)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Tiempo agotado (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Jugador 1 (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 Jugador 2 (Botón)

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

5 Tiempo agotado (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
long tiempo1 = 120000, tiempo2 = 120000;   // 2 minutos cada uno
int turno = 1;
unsigned long ultimo = 0;

void escribir(int fila, String nombre, long t) {
  lcd.setCursor(0, fila);
  int seg = t / 1000;
  int minutos = seg / 60;
  int resto = seg % 60;
  lcd.print(nombre); lcd.print(minutos); lcd.print(":");
  if (resto < 10) lcd.print("0");
  lcd.print(resto); lcd.print("  ");
}

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP);
  ultimo = millis();
}

void loop() {
  unsigned long ahora = millis();
  if (turno == 1) tiempo1 -= ahora - ultimo;
  else tiempo2 -= ahora - ultimo;
  ultimo = ahora;
  if (turno == 1 && digitalRead(2) == LOW) turno = 2;
  if (turno == 2 && digitalRead(3) == LOW) turno = 1;
  escribir(0, "Jugador 1: ", max(tiempo1, 0));
  escribir(1, "Jugador 2: ", max(tiempo2, 0));
```

```
  if (tiempo1 <= 0 || tiempo2 <= 0) {
    tone(8, 400, 1000);
    while (true) delay(1000);          // fin del juego
  }
  delay(100);
}
```

💡 **¿Cómo funciona?**

Solo el reloj del jugador que tiene el turno se descuenta. Cada vuelta del loop() resta el tiempo que pasó desde la vuelta anterior.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

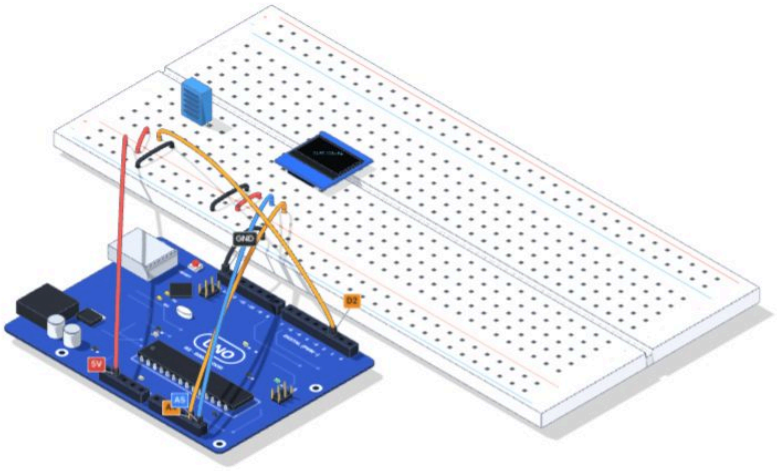
🏆 **Reto:** Agrega 2 segundos de bonificación por cada jugada.

PROYECTO 122 · PANTALLAS

Termómetro e higrómetro en OLED

★★★ Avanzado · Tiempo: 45 a 90 minutos

Muestra la temperatura y la humedad con números grandes en la pantalla OLED.

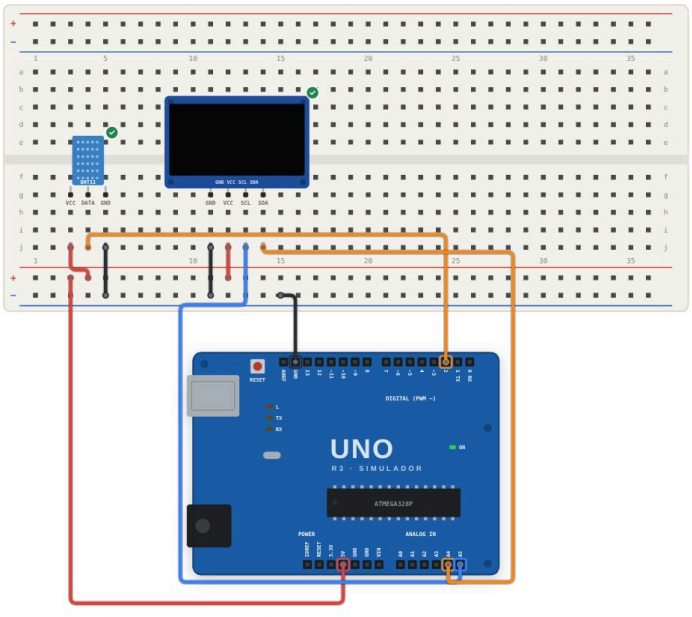


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor DHT11
- 1 × Pantalla OLED (Pantalla OLED 0.96")
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: DHT sensor library (de Adafruit) y Adafruit Unified Sensor; Adafruit SSD1306 y Adafruit GFX



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor DHT11 (Sensor DHT11)

- VCC → riel **+** (5V)
- DATA (señal) → pin **D2**
- GND → riel **–** (GND)

3 Pantalla OLED (Pantalla OLED 0.96'')

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SCL → pin **A5**
- SDA → pin **A4**

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 72 mA.
- Consumo estimado: **~72 mA** funcionando, con picos de **~72 mA**.

2 El programa

```
#include <DHT.h>
DHT dht(2, DHT11);   // DATA del sensor en D2
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 display(128, 64, &Wire, -1);

void setup() {
  dht.begin();
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);  display.print("Temperatura");
  display.setCursor(0, 34); display.print("Humedad");
  display.setTextSize(2);
  display.setCursor(0, 12); display.print(t, 0); display.print(" C");
  display.setCursor(0, 46); display.print(h, 0); display.print(" %");
  display.display();
  delay(2000);
}
```

💡 ¿Cómo funciona?

setTextSize() cambia el tamaño de la letra: los títulos van pequeños (1) y los números grandes (2) para leerlos de lejos.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura y la humedad con los deslizadores del DHT11.

Con piezas reales: sube el programa.

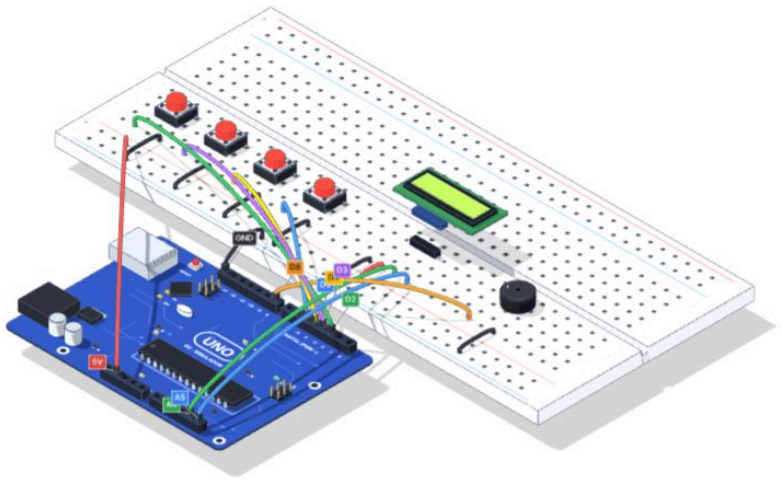
🏆 **Reto:** Dibuja una barra que crezca con la humedad.

PROYECTO 123 · PANTALLAS

Votación electrónica

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Tres botones para votar por tres candidatos; la LCD muestra los resultados al presionar el botón de cierre.

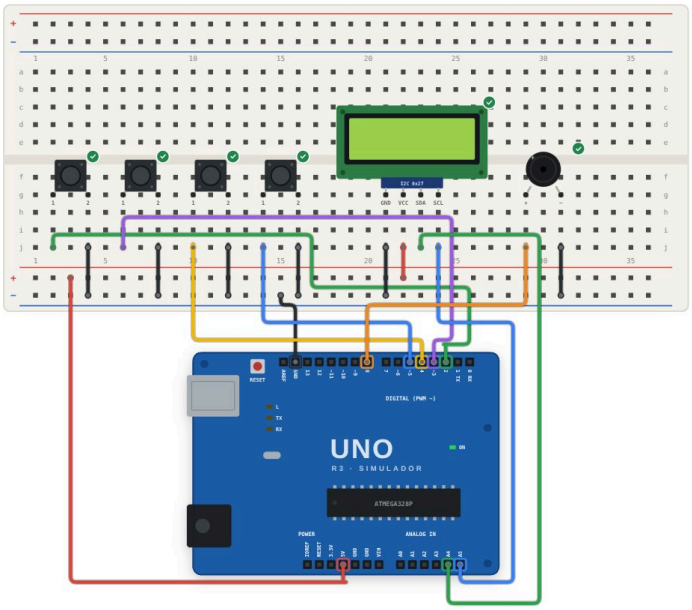


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Candidato A (Botón)
- 1 × Candidato B (Botón)
- 1 × Candidato C (Botón)
- 1 × Ver resultados (Botón)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Confirmación (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Candidato A (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 Candidato B (Botón)

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Candidato C (Botón)

- Pata 1 → pin **D4**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

5 Ver resultados (Botón)

- Pata 1 → pin **~D5**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

6 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

7 Confirmación (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

8 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
int votos[] = {0, 0, 0};
String nombres[] = {"A", "B", "C"};

void setup() {
  lcd.init(); lcd.backlight();
  for (int p = 2; p <= 5; p++) pinMode(p, INPUT_PULLUP);
  lcd.print("Vote por A, B o C");
}

void loop() {
  for (int i = 0; i < 3; i++) {
    if (digitalRead(2 + i) == LOW) {
      votos[i]++;
      tone(8, 1200, 150);
      lcd.setCursor(0, 1); lcd.print("Voto registrado ");
      delay(1500);
      lcd.setCursor(0, 1); lcd.print("                ");
    }
  }
  if (digitalRead(5) == LOW) {
    lcd.clear();
```

```
    for (int i = 0; i < 3; i++) { lcd.print(nombres[i]); lcd.print(":"); lcd.print(votos[i]);
lcd.print(" "); }
    delay(4000);
    lcd.clear(); lcd.print("Vote por A, B o C");
  }
}
```

💡 **¿Cómo funciona?**

Los votos se guardan en un arreglo: votos[0] es el candidato A, votos[1] el B y votos[2] el C. Los botones están en los pines 2, 3 y 4, por eso se usa 2 + i.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

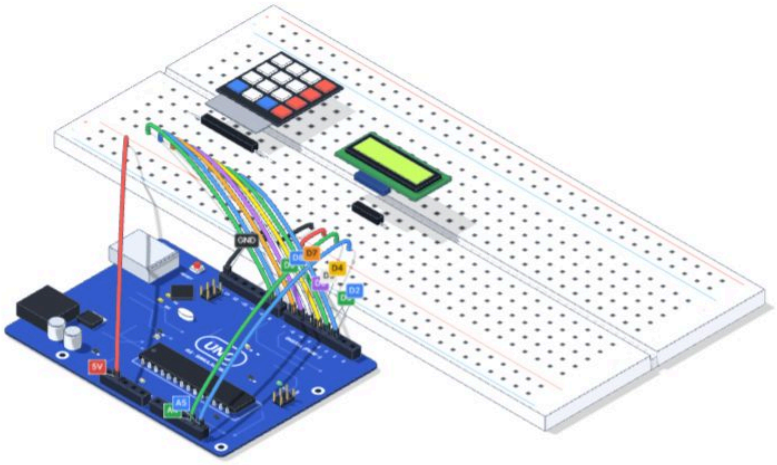
🏆 **Reto:** Muestra quién va ganando.

PROYECTO 124 · PANTALLAS

Calculadora de suma con teclado

★★★ Avanzado · Tiempo: 45 a 90 minutos

Escribe dos números con el teclado, presiona A para sumar y la LCD muestra el resultado.

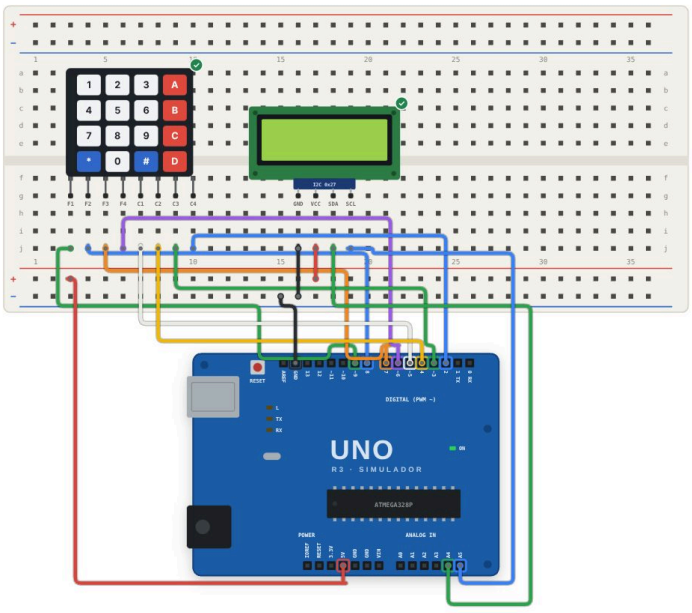


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Teclado (Teclado matricial 4×4)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Keypad (de Mark Stanley y Alexander Brevig); LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Teclado** (Teclado matricial 4×4)

- Fila 1 → pin **~D9**
- Fila 2 → pin **D8**
- Fila 3 → pin **D7**
- Fila 4 → pin **~D6**
- Columna 1 → pin **~D5**
- Columna 2 → pin **D4**
- Columna 3 → pin **~D3**
- Columna 4 → pin **D2**

3 **Pantalla LCD** (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 81 mA.
- Consumo estimado: **~81 mA** funcionando, con picos de **~81 mA**.

2 El programa

```
#include <Keypad.h>
const byte FILAS = 4;
const byte COLUMNAS = 4;
char teclas[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte pinesFilas[FILAS] = {9, 8, 7, 6};
byte pinesColumnas[COLUMNAS] = {5, 4, 3, 2};
Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas, pinesColumnas, FILAS, COLUMNAS);
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
String numero1 = "", numero2 = "";
bool segundo = false;

void setup() {
  lcd.init(); lcd.backlight();
  lcd.print("Calculadora +");
}

void loop() {
  char tecla = teclado.getKey();
  if (!tecla) return;
  if (tecla >= '0' && tecla <= '9') {
    if (segundo) numero2 += tecla;
    else numero1 += tecla;
  } else if (tecla == 'A') {              // el signo +
    segundo = true;
  } else if (tecla == '#') {              // igual
    long r = numero1.toInt() + numero2.toInt();
    lcd.clear(); lcd.print(numero1 + "+" + numero2 + "=");
    lcd.setCursor(0, 1); lcd.print(r);
    numero1 = ""; numero2 = ""; segundo = false;
    return;
```

```
  } else if (tecla == '*') {              // borrar
    numero1 = ""; numero2 = ""; segundo = false;
  }
  lcd.clear();
  lcd.print(numero1);
  if (segundo) { lcd.print("+"); lcd.print(numero2); }
}
```

💡 ¿Cómo funciona?

Los números se van armando como texto, tecla por tecla. Al presionar # se convierten a número con toInt() y se suman. La tecla A hace de signo más y * borra.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca las teclas del teclado dibujado en su tarjeta, como si fuera el teclado real.

Con piezas reales: sube el programa.

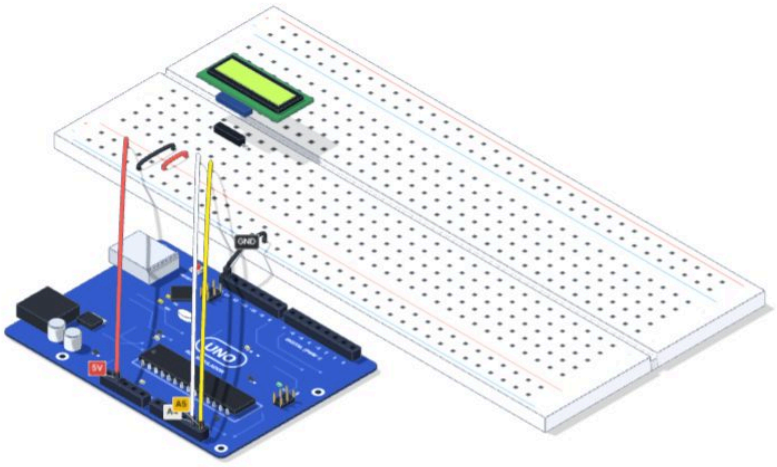
🏆 **Reto:** Agrega resta con la tecla B y multiplicación con la C.

PROYECTO 125 · PANTALLAS

Mi tarjeta de presentación en LCD

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

La pantalla muestra, una tras otra, varias líneas con tus datos: nombre, curso y gusto favorito.

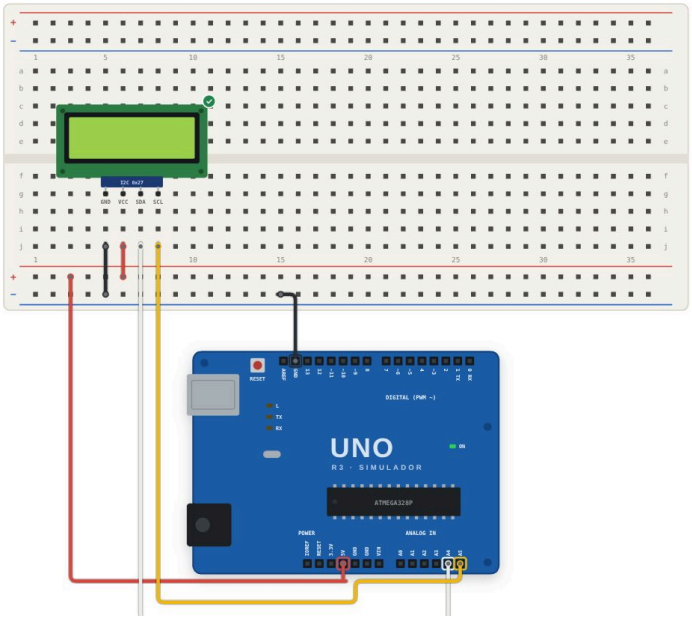


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

3 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
String lineas[] = {"Me llamo:", "Ana Martinez", "Curso:", "Noveno B", "Me gusta:", "la robotica!"};

void setup() {
  lcd.init(); lcd.backlight();
}

void loop() {
  for (int i = 0; i < 6; i += 2) {      // de dos en dos
    lcd.clear();
    lcd.print(lineas[i]);
    lcd.setCursor(0, 1);
    lcd.print(lineas[i + 1]);
    delay(2000);
  }
}
```

💡 **¿Cómo funciona?**

Los textos están en un arreglo de String. El ciclo los toma de dos en dos: el primero va en la fila de arriba y el segundo en la de abajo.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar. **Con piezas reales:** sube el programa.

🏆 **Reto:** Agrega dos pantallas más con tu comida y tu deporte favoritos.

CAPÍTULO 10

Seguridad y control de acceso

Lectores de huellas, tarjetas RFID, teclados, chapas eléctricas de 12 V y alarmas: el control de acceso de verdad.

EN ESTE CAPÍTULO · 25 PROYECTOS

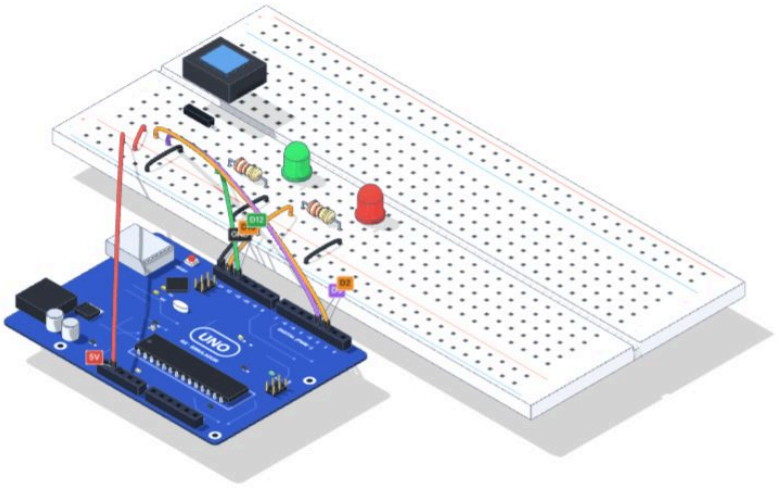
126 Lector de huellas con luz verde y roja ★★☆
127 Cerradura con huella y servo ★★☆
128 Chapa eléctrica con huella ★★★
129 Saludo personalizado con huella ★★★
130 Registro de asistencia con huella ★★★
131 Caja fuerte con huella y alarma ★★★
132 Cerradura con clave y teclado ★★☆
133 Chapa eléctrica con clave ★★★
134 Cerradura con clave, LCD y servo ★★★
135 Control de acceso con tarjeta RFID ★★☆
136 Puerta con tarjeta RFID y servo ★★☆
137 Chapa eléctrica con tarjeta RFID ★★★
138 Bienvenida con tarjeta RFID y LCD ★★★
139 Registro de entradas con RFID ★★☆
140 Alarma con PIR que se desactiva con clave ★★★
141 Alarma de ventana con imán ★★☆
142 Timbre que solo suena con tarjeta ★★☆
143 Doble seguridad: huella y botón de salida ★★★
144 Alarma de vibración para caja fuerte ★★☆
145 Botón de emergencia con sirena y luces ★★☆
146 Contador de intentos de acceso en LCD ★★★
147 Cofre del tesoro con clave ★★☆
148 Lector de huellas con contador de usos ★★☆
149 Doble factor: tarjeta y huella ★★★
150 Alarma antirrobo para bicicleta ★★☆

PROYECTO 126 · SEGURIDAD Y CONTROL DE ACCESO

Lector de huellas con luz verde y roja

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Pon tu dedo en el lector: si la huella está registrada se enciende la luz verde, si no, la roja.

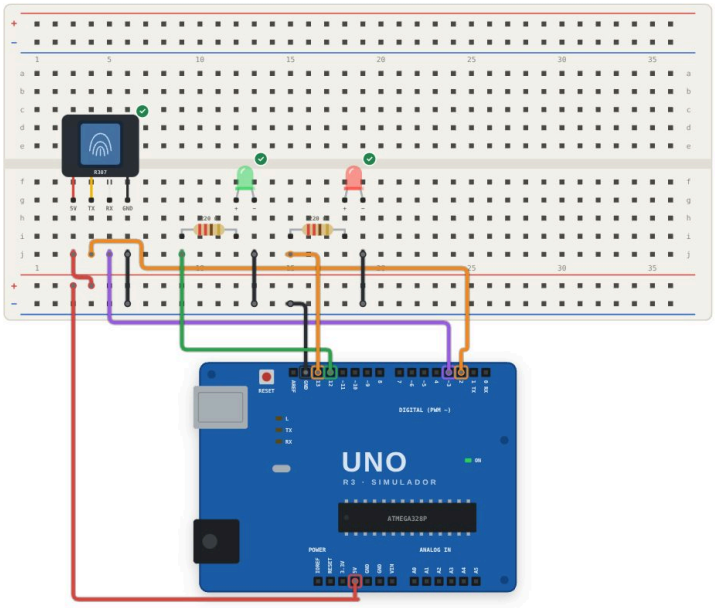


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector de huellas (Lector de huellas R307)
- 1 × Permitido (LED)
- 1 × Denegado (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Adafruit Fingerprint Sensor Library



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector de huellas (Lector de huellas R307)

- Cable rojo (VCC) → riel **+** (5V)
- Cable amarillo (TX del lector) → pin **D2**
- Cable blanco (RX del lector) → pin **~D3**
- Cable negro (GND) → riel **–** (GND)

3 Permitido (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Denegado (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Para el proyecto real

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 140 mA.
- Consumo estimado: **~140 mA** funcionando, con picos de **~200 mA**.

2 El programa

```
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);   // TX del lector en D2, RX en D3
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
// Devuelve el número de la huella reconocida, 0 si no la conoce o -1 si no hay dedo
int leerHuella() {
  if (finger.getImage() != FINGERPRINT_OK) return -1;
  if (finger.image2Tz() != FINGERPRINT_OK) return -1;
  if (finger.fingerFastSearch() != FINGERPRINT_OK) return 0;
  return finger.fingerID;
}

void setup() {
  Serial.begin(9600);
  finger.begin(57600);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
  if (finger.verifyPassword()) Serial.println("Lector de huellas listo");
  else Serial.println("No encuentro el lector");
}

void loop() {
  int id = leerHuella();
  if (id == -1) return;                // no hay dedo
  if (id > 0) {
    Serial.print("Huella reconocida: #"); Serial.println(id);
    digitalWrite(12, HIGH); delay(2000); digitalWrite(12, LOW);
  } else {
    Serial.println("Huella desconocida");
    digitalWrite(13, HIGH); delay(2000); digitalWrite(13, LOW);
```

```
  }
}
```

💡 ¿Cómo funciona?

El lector toma una foto del dedo (getImage), la convierte en un patrón (image2Tz) y lo busca entre las huellas guardadas (fingerFastSearch). Si la encuentra, fingerID dice cuál es.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Huella 1», «Huella 2», «Huella 3» (registradas) o «Desconocida» en la tarjeta del lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

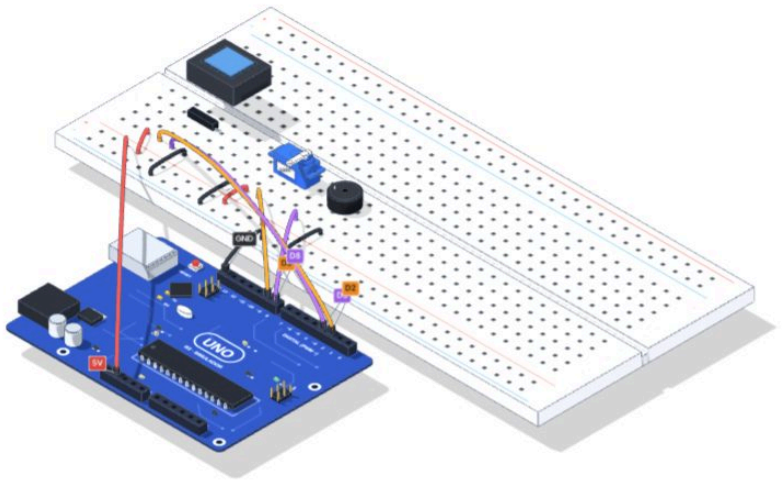
🏆 **Reto:** Haz que la huella #1 sea del profesor y encienda los dos LED.

PROYECTO 127 · SEGURIDAD Y CONTROL DE ACCESO

Cerradura con huella y servo

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Con una huella registrada, un servo abre el pestillo de la puerta durante 3 segundos.

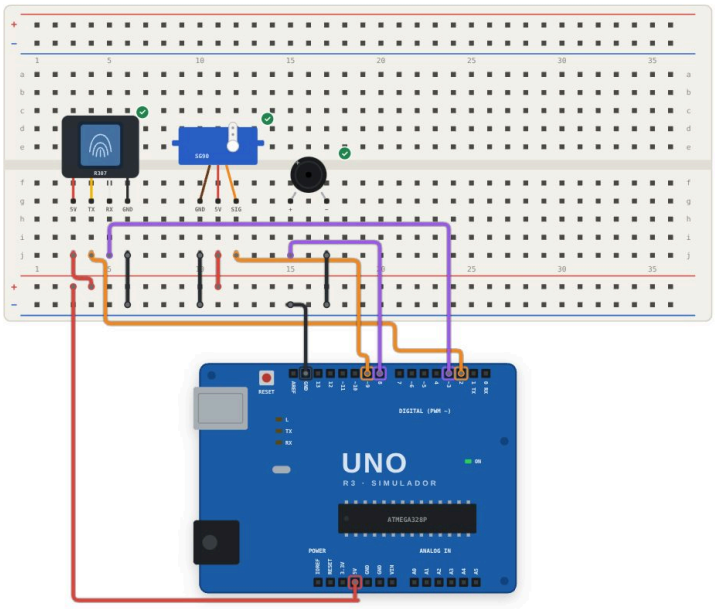


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector de huellas (Lector de huellas R307)
- 1 × Pestillo (Servo SG90)
- 1 × Pitido (Zumbador)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Adafruit Fingerprint Sensor Library; Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector de huellas (Lector de huellas R307)

- Cable rojo (VCC) → riel **+** (5V)
- Cable amarillo (TX del lector) → pin **D2**
- Cable blanco (RX del lector) → pin **~D3**
- Cable negro (GND) → riel **–** (GND)

3 Pestillo (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Para el proyecto real

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 290 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~290 mA** funcionando, con picos de **~850 mA**.

2 El programa

```
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);   // TX del lector en D2, RX en D3
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
#include <Servo.h>
Servo pestillo;
// Devuelve el número de la huella reconocida, 0 si no la conoce o -1 si no hay dedo
int leerHuella() {
  if (finger.getImage() != FINGERPRINT_OK) return -1;
  if (finger.image2Tz() != FINGERPRINT_OK) return -1;
  if (finger.fingerFastSearch() != FINGERPRINT_OK) return 0;
  return finger.fingerID;
}

void setup() {
  Serial.begin(9600);
  finger.begin(57600);
  pestillo.attach(9);
  pestillo.write(0);                  // cerrado
}

void loop() {
  int id = leerHuella();
  if (id > 0) {
    Serial.print("Bienvenido, huella #"); Serial.println(id);
    tone(8, 1500, 150);
    pestillo.write(90);               // abrir
    delay(3000);
    pestillo.write(0);                // cerrar
  } else if (id == 0) {
```

```
    Serial.println("Acceso denegado");
    tone(8, 300, 600);
  }
}
```

💡 ¿Cómo funciona?

Cuando la huella es válida el servo gira 90 grados y corre el pestillo. Si la huella no está registrada, suena un tono grave de error y la puerta sigue cerrada.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Huella 1», «Huella 2», «Huella 3» (registradas) o «Desconocida» en la tarjeta del lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

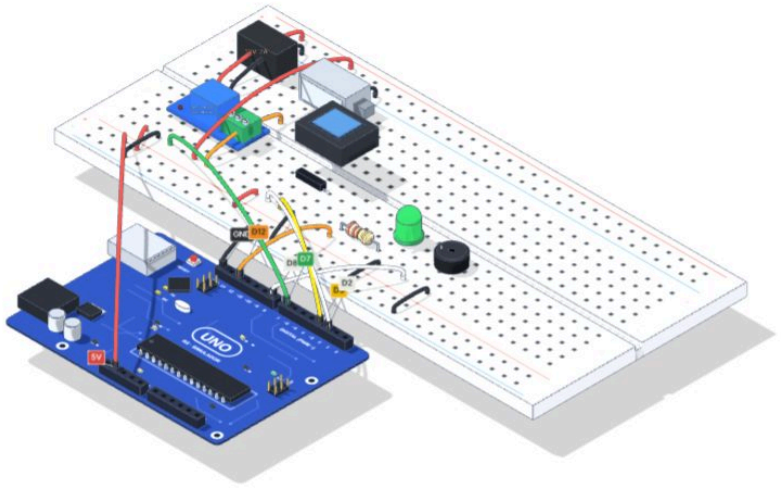
🏆 **Reto:** Agrega una LCD que diga "Bienvenido" o "Denegado".

PROYECTO 128 · SEGURIDAD Y CONTROL DE ACCESO

Chapa eléctrica con huella

★★★ Avanzado · Tiempo: 45 a 90 minutos

El control de acceso de una oficina: una huella válida activa el relé y la chapa eléctrica abre la puerta.

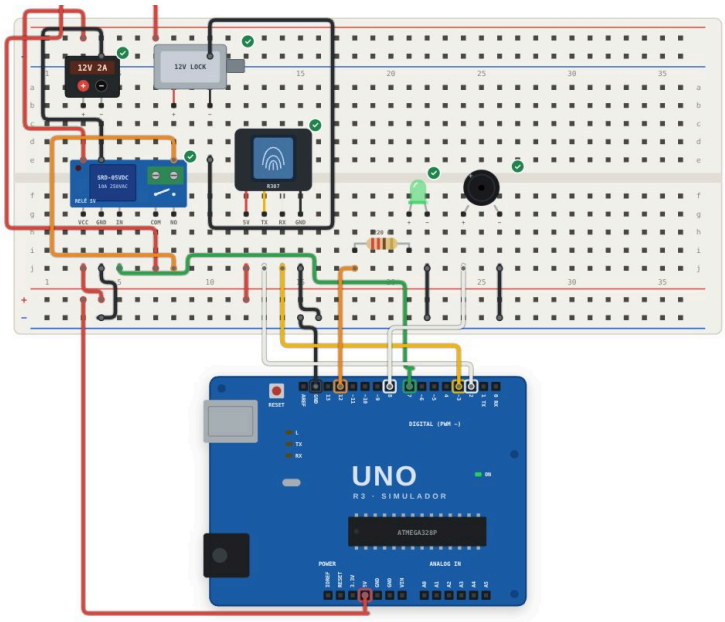


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector de huellas (Lector de huellas R307)
- 1 × Relé de la chapa (Módulo relé 5 V)
- 1 × Chapa eléctrica 12 V
- 1 × Fuente 12 V
- 1 × Abierto (LED)
- 1 × Pitido (Zumbador)
- 1 × Resistencia 220 Ω
- Diodo 1N4007 en paralelo con la chapa
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Adafruit Fingerprint Sensor Library



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector de huellas (Lector de huellas R307)

- Cable rojo (VCC) → riel **+** (5V)
- Cable amarillo (TX del lector) → pin **D2**
- Cable blanco (RX del lector) → pin **~D3**
- Cable negro (GND) → riel **–** (GND)

3 Relé de la chapa (Módulo relé 5 V)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- IN (señal) → pin **D7**
- COM (común) → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- NO (normalmente abierto) → **+ de la carga** (bomba, chapa o motor de 12 V)

4 Chapa eléctrica 12 V (Chapa eléctrica 12 V)

- Cable + (rojo) de la chapa → borne **NO** del relé
- Cable – (negro) de la chapa → **– de 12 V** (riel – de arriba)

5 Fuente 12 V (Fuente 12 V)

- Borne +12V → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- Borne – → **– de 12 V** (riel – de arriba)

6 Abierto (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

7 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

8 Para el proyecto real

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.
- La chapa de 12 V nunca va al Arduino: el relé la conecta a su propia fuente de 12 V (COM al + de la fuente, NO al + de la chapa, – de la chapa al – de la fuente).

9 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 225 mA.
- **Relé:** el lado de control (VCC, GND, IN) va al Arduino con 5 V; la carga va por COM y NO con su propia fuente. Algunos módulos se activan con LOW: si el tuyo funciona al revés, invierte HIGH y LOW en el código.
- Consumo estimado: **~225 mA** funcionando, con picos de **~285 mA**.

2 El programa

```
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);   // TX del lector en D2, RX en D3
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
// Devuelve el número de la huella reconocida, 0 si no la conoce o -1 si no hay dedo
int leerHuella() {
  if (finger.getImage() != FINGERPRINT_OK) return -1;
  if (finger.image2Tz() != FINGERPRINT_OK) return -1;
  if (finger.fingerFastSearch() != FINGERPRINT_OK) return 0;
  return finger.fingerID;
}
const int RELE = 7, VERDE = 12, PITO = 8;

void setup() {
  Serial.begin(9600);
```

```
  finger.begin(57600);
  pinMode(RELE, OUTPUT); pinMode(VERDE, OUTPUT);
  digitalWrite(RELE, LOW);              // chapa cerrada
}

void loop() {
  int id = leerHuella();
  if (id > 0) {
    Serial.print("Acceso permitido a la huella #"); Serial.println(id);
    digitalWrite(VERDE, HIGH);
    tone(PITO, 1800, 120);
    digitalWrite(RELE, HIGH);           // la chapa recibe 12 V y abre
    delay(3000);
    digitalWrite(RELE, LOW);            // vuelve a cerrar
    digitalWrite(VERDE, LOW);
  } else if (id == 0) {
    Serial.println("Huella no autorizada");
    for (int i = 0; i < 3; i++) { tone(PITO, 300, 150); delay(250); }
  }
}
```

💡 ¿Cómo funciona?

El Arduino no puede dar los 12 V que necesita la chapa: activa un relé, que es un interruptor eléctrico. Mientras el pin está en HIGH, el relé une la fuente de 12 V con la chapa y el pestillo se recoge.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Huella 1», «Huella 2», «Huella 3» (registradas) o «Desconocida» en la tarjeta del lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.
- La chapa de 12 V nunca va al Arduino: el relé la conecta a su propia fuente de 12 V (COM al + de la fuente, NO al + de la chapa, − de la chapa al − de la fuente).

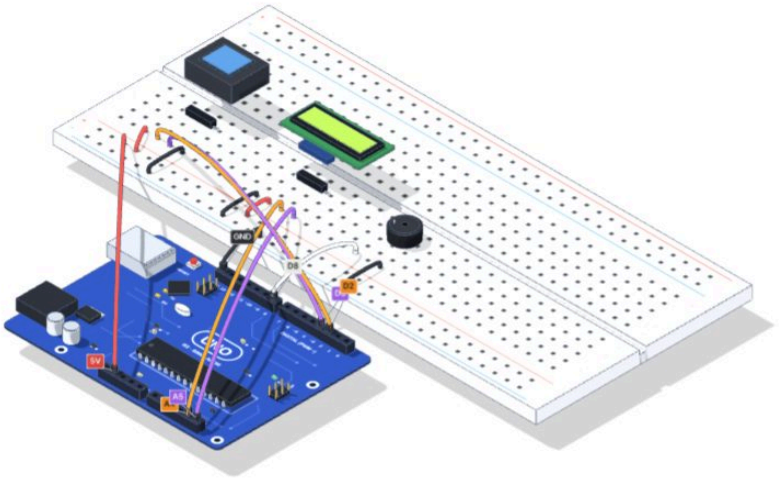
🏆 **Reto:** Registra en el monitor la hora de cada entrada con millis().

PROYECTO 129 · SEGURIDAD Y CONTROL DE ACCESO

Saludo personalizado con huella

★★★ Avanzado · Tiempo: 45 a 90 minutos

La LCD saluda por su nombre a cada persona según su huella.

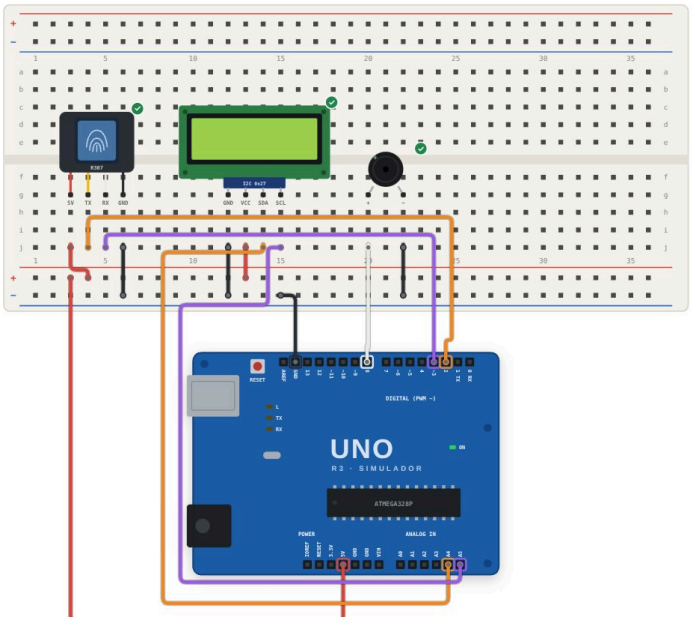


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector de huellas (Lector de huellas R307)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Pitido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Adafruit Fingerprint Sensor Library; LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector de huellas (Lector de huellas R307)

- Cable rojo (VCC) → riel **+** (5V)
- Cable amarillo (TX del lector) → pin **D2**
- Cable blanco (RX del lector) → pin **~D3**
- Cable negro (GND) → riel **–** (GND)

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Para el proyecto real

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 170 mA.
- Consumo estimado: **~170 mA** funcionando, con picos de **~230 mA**.

2 El programa

```
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);   // TX del lector en D2, RX en D3
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
// Devuelve el número de la huella reconocida, 0 si no la conoce o -1 si no hay dedo
int leerHuella() {
  if (finger.getImage() != FINGERPRINT_OK) return -1;
  if (finger.image2Tz() != FINGERPRINT_OK) return -1;
  if (finger.fingerFastSearch() != FINGERPRINT_OK) return 0;
  return finger.fingerID;
}
String nombres[] = {"Desconocido", "Profe Jair", "Ana", "Luis"};   // huellas 1, 2 y 3

void setup() {
  finger.begin(57600);
  lcd.init(); lcd.backlight();
  lcd.print("Ponga su dedo");
}

void loop() {
  int id = leerHuella();
  if (id == -1) return;
  lcd.clear();
  if (id > 0 && id <= 3) {
    lcd.print("Hola,");
    lcd.setCursor(0, 1); lcd.print(nombres[id]);
    tone(8, 1500, 150);
  } else {
    lcd.print("No te conozco");
```

```
    tone(8, 300, 500);
  }
  delay(2500);
  lcd.clear(); lcd.print("Ponga su dedo");
}
```

💡 ¿Cómo funciona?

El número de la huella sirve como posición en el arreglo nombres: la huella 1 muestra nombres[1]. La posición 0 queda para "desconocido".

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Huella 1», «Huella 2», «Huella 3» (registradas) o «Desconocida» en la tarjeta del lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

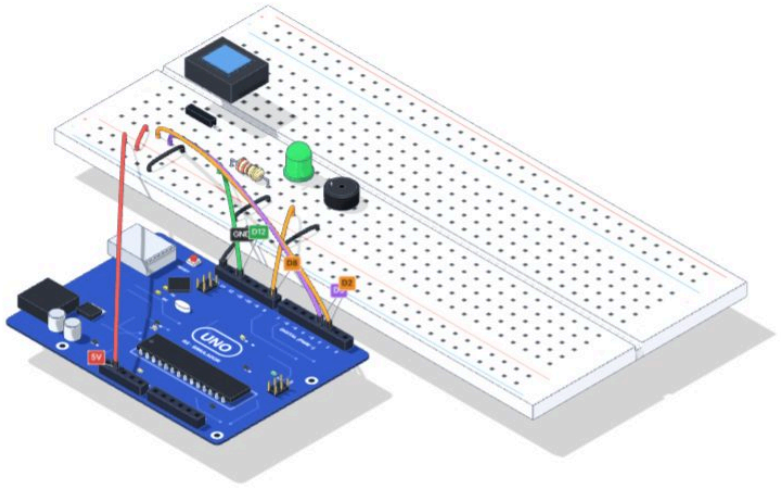
🏆 **Reto:** Cambia los nombres por los de tu grupo.

PROYECTO 130 · SEGURIDAD Y CONTROL DE ACCESO

Registro de asistencia con huella

★★★ Avanzado · Tiempo: 45 a 90 minutos

Cada estudiante pone su huella al llegar y el monitor serie va armando la lista de asistencia.

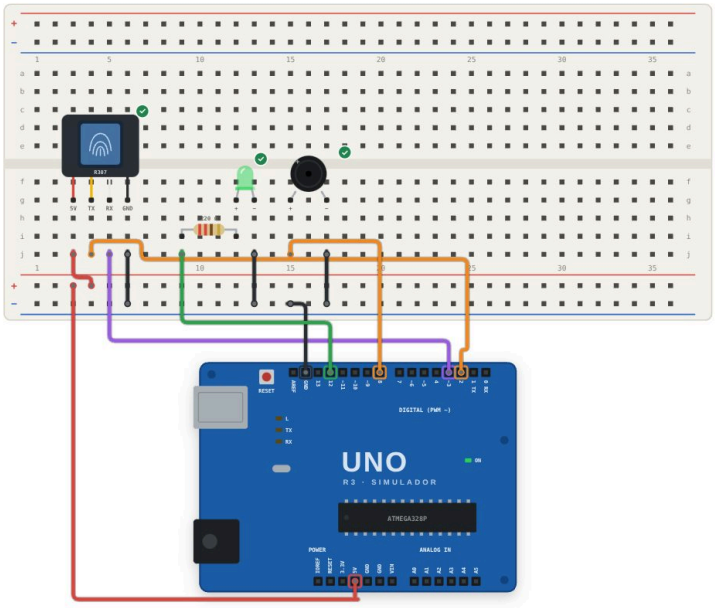


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector de huellas (Lector de huellas R307)
- 1 × Registrado (LED)
- 1 × Pitido (Zumbador)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Adafruit Fingerprint Sensor Library



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector de huellas (Lector de huellas R307)

- Cable rojo (VCC) → riel **+** (5V)
- Cable amarillo (TX del lector) → pin **D2**
- Cable blanco (RX del lector) → pin **~D3**
- Cable negro (GND) → riel **–** (GND)

3 Registrado (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Para el proyecto real

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 155 mA.
- Consumo estimado: **~155 mA** funcionando, con picos de **~215 mA**.

2 El programa

```
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);   // TX del lector en D2, RX en D3
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
// Devuelve el número de la huella reconocida, 0 si no la conoce o -1 si no hay dedo
int leerHuella() {
  if (finger.getImage() != FINGERPRINT_OK) return -1;
  if (finger.image2Tz() != FINGERPRINT_OK) return -1;
  if (finger.fingerFastSearch() != FINGERPRINT_OK) return 0;
  return finger.fingerID;
}
bool presente[] = {false, false, false, false};
String nombres[] = {"", "Ana", "Luis", "Sofia"};

void setup() {
  Serial.begin(9600);
  finger.begin(57600);
  pinMode(12, OUTPUT);
  Serial.println("Asistencia: pongan su dedo al llegar");
}

void loop() {
  int id = leerHuella();
  if (id > 0 && id <= 3) {
    if (!presente[id]) {
      presente[id] = true;
      Serial.print(nombres[id]); Serial.print(" llegó a los ");
      Serial.print(millis() / 1000); Serial.println(" segundos");
    } else {
```

```
30        Serial.print(nombres[id]); Serial.println(" ya estaba registrado");
31      }
32      digitalWrite(12, HIGH); tone(8, 1500, 100); delay(1500); digitalWrite(12, LOW);
33    }
34  }
```

💡 ¿Cómo funciona?

El arreglo presente recuerda quién ya llegó, para no registrar a nadie dos veces. La hora de llegada se toma con millis().

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Huella 1», «Huella 2», «Huella 3» (registradas) o «Desconocida» en la tarjeta del lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

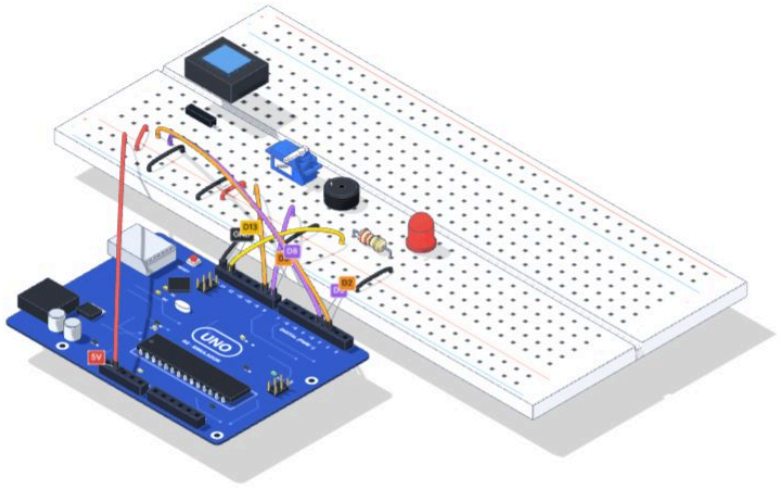
🏆 **Reto:** Agrega un botón que imprima la lista de los que faltan.

PROYECTO 131 · SEGURIDAD Y CONTROL DE ACCESO

Caja fuerte con huella y alarma

★★★ Avanzado · Tiempo: 45 a 90 minutos

Una caja fuerte que abre con huella; si alguien falla tres veces seguidas, suena la alarma.

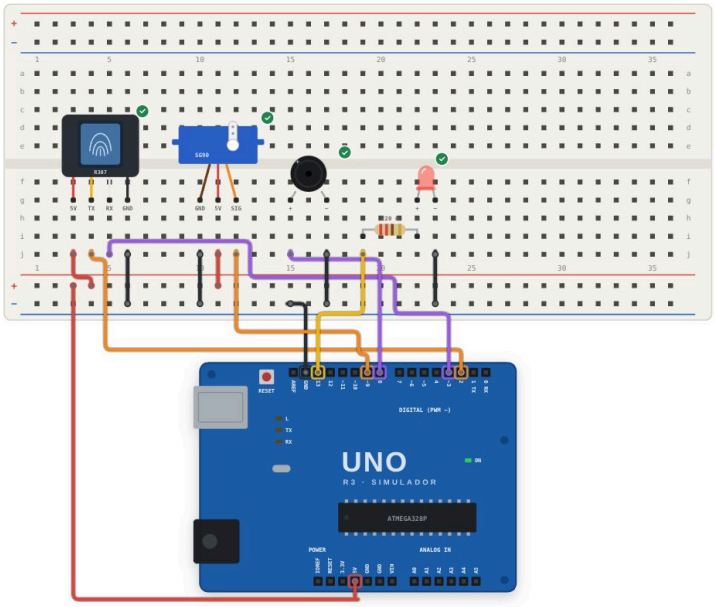


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector de huellas (Lector de huellas R307)
- 1 × Pestillo (Servo SG90)
- 1 × Alarma (Zumbador)
- 1 × Bloqueada (LED)
- 1 × Resistencia 220 Ω
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Adafruit Fingerprint Sensor Library; Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector de huellas (Lector de huellas R307)

- Cable rojo (VCC) → riel **+** (5V)
- Cable amarillo (TX del lector) → pin **D2**
- Cable blanco (RX del lector) → pin **~D3**
- Cable negro (GND) → riel **–** (GND)

3 Pestillo (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Bloqueada (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 Para el proyecto real

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 305 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~305 mA** funcionando, con picos de **~865 mA**.

2 El programa

```
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);   // TX del lector en D2, RX en D3
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
#include <Servo.h>
Servo pestillo;
// Devuelve el número de la huella reconocida, 0 si no la conoce o -1 si no hay dedo
int leerHuella() {
  if (finger.getImage() != FINGERPRINT_OK) return -1;
  if (finger.image2Tz() != FINGERPRINT_OK) return -1;
  if (finger.fingerFastSearch() != FINGERPRINT_OK) return 0;
  return finger.fingerID;
}
int fallos = 0;

void setup() {
  Serial.begin(9600);
  finger.begin(57600);
  pestillo.attach(9); pestillo.write(0);
  pinMode(13, OUTPUT);
}

void loop() {
  int id = leerHuella();
  if (id > 0) {
```

```
    fallos = 0;
    Serial.println("Caja abierta");
    pestillo.write(90); delay(4000); pestillo.write(0);
  } else if (id == 0) {
    fallos++;
    Serial.print("Huella incorrecta. Intentos fallidos: "); Serial.println(fallos);
    tone(8, 300, 300);
    if (fallos >= 3) {
      Serial.println("¡ALARMA! Demasiados intentos");
      for (int i = 0; i < 20; i++) { digitalWrite(13, HIGH); tone(8, 2000, 100); delay(150);
digitalWrite(13, LOW); delay(100); }
      fallos = 0;
    }
    delay(800);
  }
}
```

💡 ¿Cómo funciona?

La variable fallos cuenta los intentos incorrectos seguidos y vuelve a 0 con una huella válida. Al llegar a 3, la alarma suena unos 5 segundos.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Huella 1», «Huella 2», «Huella 3» (registradas) o «Desconocida» en la tarjeta del lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

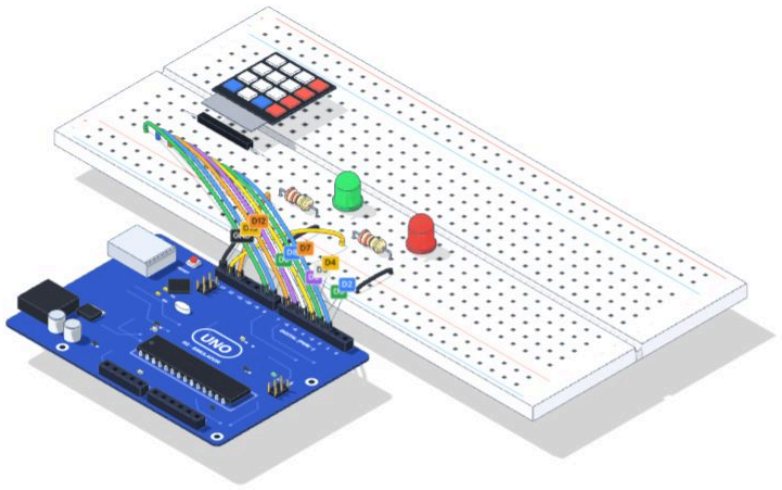
🏆 **Reto:** Bloquea la caja durante 30 segundos después de la alarma.

PROYECTO 132 · SEGURIDAD Y CONTROL DE ACCESO

Cerradura con clave y teclado

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Escribe la clave de 4 dígitos en el teclado y termina con #: si es correcta, la luz verde se enciende.

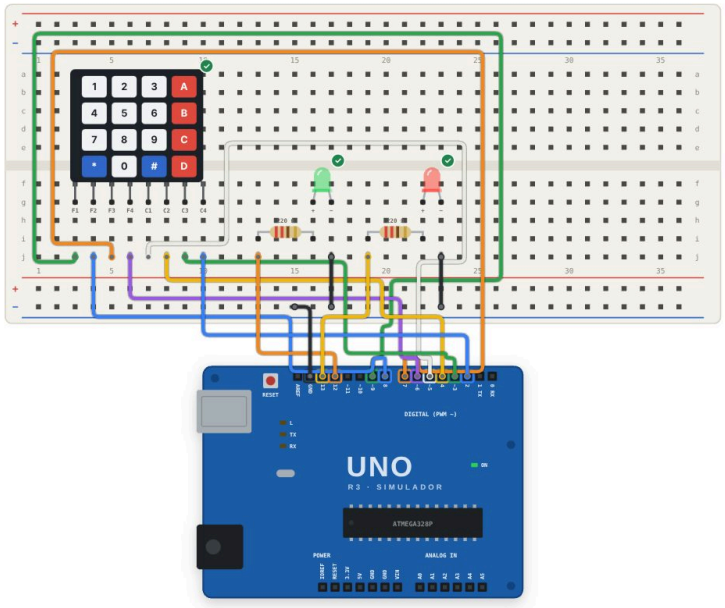


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Teclado (Teclado matricial 4×4)
- 1 × Correcta (LED)
- 1 × Incorrecta (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Keypad (de Mark Stanley y Alexander Brevig)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Teclado** (Teclado matricial 4×4)

- Fila 1 → pin **~D9**
- Fila 2 → pin **D8**
- Fila 3 → pin **D7**
- Fila 4 → pin **~D6**
- Columna 1 → pin **~D5**
- Columna 2 → pin **D4**
- Columna 3 → pin **~D3**
- Columna 4 → pin **D2**

3 **Correcta** (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Incorrecta** (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 81 mA.
- Consumo estimado: **~81 mA** funcionando, con picos de **~81 mA**.

2 El programa

```
#include <Keypad.h>
const byte FILAS = 4;
const byte COLUMNAS = 4;
char teclas[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte pinesFilas[FILAS] = {9, 8, 7, 6};
byte pinesColumnas[COLUMNAS] = {5, 4, 3, 2};
Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas, pinesColumnas, FILAS, COLUMNAS);
const String CLAVE = "1234";
String escrito = "";

void setup() {
  Serial.begin(9600);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
  Serial.println("Escribe la clave y presiona #");
}

void loop() {
  char tecla = teclado.getKey();
  if (!tecla) return;
  if (tecla == '#') {
    if (escrito == CLAVE) { Serial.println("Clave correcta"); digitalWrite(12, HIGH); delay(2000);
digitalWrite(12, LOW); }
    else { Serial.println("Clave incorrecta"); digitalWrite(13, HIGH); delay(2000);
digitalWrite(13, LOW); }
```

```
    escrito = "";
  } else if (tecla == '*') {
    escrito = "";
    Serial.println("Borrado");
  } else {
    escrito += tecla;
    Serial.print("*");                // no mostrar la clave
  }
}
```

💡 **¿Cómo funciona?**

Cada tecla se agrega al texto escrito. Al presionar # se compara con la clave guardada. En el monitor solo se muestran asteriscos para que nadie vea la clave.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Toca las teclas del teclado dibujado en su tarjeta, como si fuera el teclado real.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

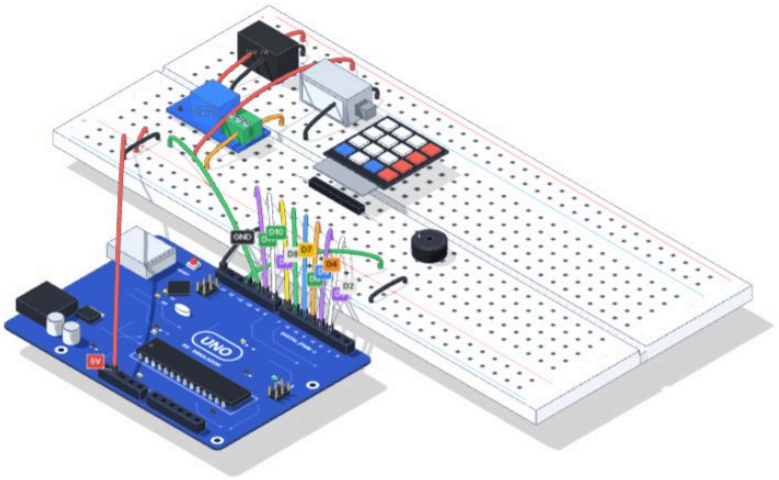
🏆 **Reto:** Cambia la clave y agrega un servo que abra la puerta.

PROYECTO 133 · SEGURIDAD Y CONTROL DE ACCESO

Chapa eléctrica con clave

★★★ Avanzado · Tiempo: 45 a 90 minutos

La puerta de un laboratorio: con la clave correcta el relé abre la chapa eléctrica de 12 V.

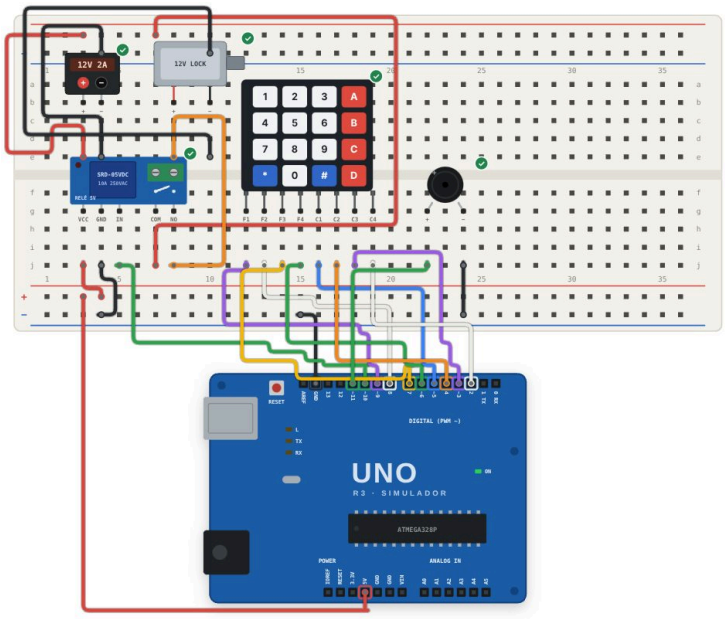


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Teclado (Teclado matricial 4×4)
- 1 × Relé de la chapa (Módulo relé 5 V)
- 1 × Chapa eléctrica 12 V
- 1 × Fuente 12 V
- 1 × Pitido (Zumbador)
- Diodo 1N4007 en paralelo con la chapa
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Keypad (de Mark Stanley y Alexander Brevig)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Teclado (Teclado matricial 4×4)

- Fila 1 → pin **~D9**
- Fila 2 → pin **D8**
- Fila 3 → pin **D7**
- Fila 4 → pin **~D6**
- Columna 1 → pin **~D5**
- Columna 2 → pin **D4**
- Columna 3 → pin **~D3**
- Columna 4 → pin **D2**

3 Relé de la chapa (Módulo relé 5 V)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- IN (señal) → pin **~D10**
- COM (común) → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- NO (normalmente abierto) → **+ de la carga** (bomba, chapa o motor de 12 V)

4 Chapa eléctrica 12 V (Chapa eléctrica 12 V)

- Cable + (rojo) de la chapa → borne **NO** del relé
- Cable – (negro) de la chapa → **– de 12 V** (riel – de arriba)

5 Fuente 12 V (Fuente 12 V)

- Borne +12V → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- Borne – → **– de 12 V** (riel – de arriba)

6 Pitido (Zumbador)

- Pata + → pin **~D11**
- Pata – → riel **–** (GND)

7 Para el proyecto real

- La chapa de 12 V nunca va al Arduino: el relé la conecta a su propia fuente de 12 V (COM al + de la fuente, NO al + de la chapa, – de la chapa al – de la fuente).

8 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 151 mA.
- **Relé:** el lado de control (VCC, GND, IN) va al Arduino con 5 V; la carga va por COM y NO con su propia fuente. Algunos módulos se activan con LOW: si el tuyo funciona al revés, invierte HIGH y LOW en el código.
- Consumo estimado: **~151 mA** funcionando, con picos de **~151 mA**.

2 El programa

```
#include <Keypad.h>
const byte FILAS = 4;
const byte COLUMNAS = 4;
char teclas[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte pinesFilas[FILAS] = {9, 8, 7, 6};
byte pinesColumnas[COLUMNAS] = {5, 4, 3, 2};
Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas, pinesColumnas, FILAS, COLUMNAS);
const String CLAVE = "2580";
const int RELE = 10, PITO = 11;
String escrito = "";

void setup() {
  Serial.begin(9600);
```

```
  pinMode(RELE, OUTPUT);
  digitalWrite(RELE, LOW);
}

void loop() {
  char tecla = teclado.getKey();
  if (!tecla) return;
  tone(PITO, 2000, 40);                 // clic de cada tecla
  if (tecla == '#') {
    if (escrito == CLAVE) {
      Serial.println("Puerta abierta");
      digitalWrite(RELE, HIGH); delay(3000); digitalWrite(RELE, LOW);
    } else {
      Serial.println("Clave incorrecta");
      tone(PITO, 300, 700);
    }
    escrito = "";
  } else {
    escrito += tecla;
  }
}
```

💡 ¿Cómo funciona?

Es la cerradura con teclado, pero ahora el resultado mueve un relé. El relé conecta la fuente de 12 V a la chapa solo durante 3 segundos, suficientes para empujar la puerta.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca las teclas del teclado dibujado en su tarjeta, como si fuera el teclado real.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- La chapa de 12 V nunca va al Arduino: el relé la conecta a su propia fuente de 12 V (COM al + de la fuente, NO al + de la chapa, − de la chapa al − de la fuente).

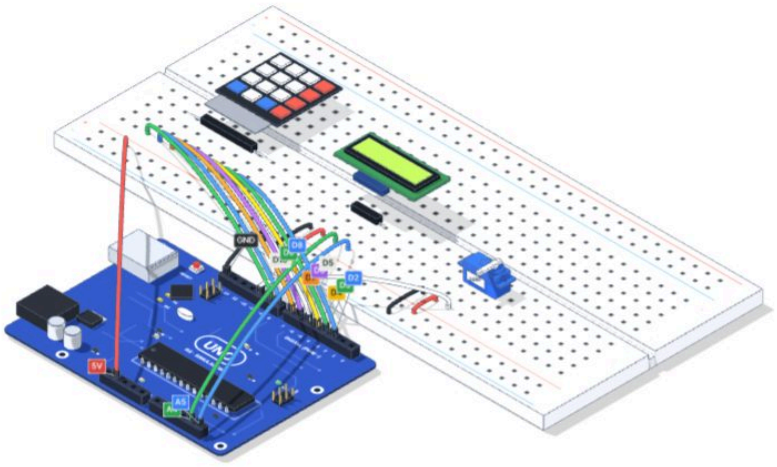
🏆 **Reto:** Muestra en una LCD los asteriscos de la clave mientras se escribe.

PROYECTO 134 · SEGURIDAD Y CONTROL DE ACCESO

Cerradura con clave, LCD y servo

★★★ Avanzado · Tiempo: 45 a 90 minutos

Un sistema completo: la LCD pide la clave, muestra asteriscos y un servo abre la puerta si es correcta.

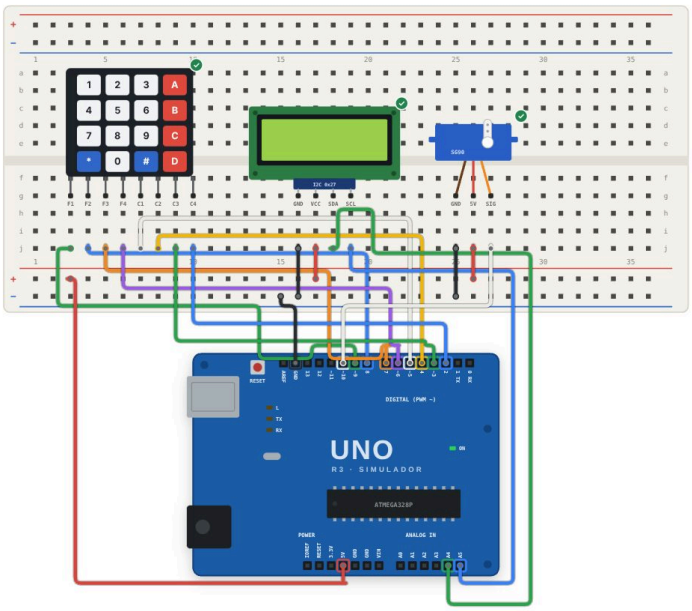


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Teclado (Teclado matricial 4×4)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Puerta (Servo SG90)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Keypad (de Mark Stanley y Alexander Brevig); LiquidCrystal I2C (de Frank de Brabander); Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Teclado (Teclado matricial 4×4)

- Fila 1 → pin **~D9**
- Fila 2 → pin **D8**
- Fila 3 → pin **D7**
- Fila 4 → pin **~D6**
- Columna 1 → pin **~D5**
- Columna 2 → pin **D4**
- Columna 3 → pin **~D3**
- Columna 4 → pin **D2**

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Puerta (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D10**

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 231 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~231 mA** funcionando, con picos de **~731 mA**.

2 El programa

```ino
#include <Keypad.h>
const byte FILAS = 4;
const byte COLUMNAS = 4;
char teclas[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte pinesFilas[FILAS] = {9, 8, 7, 6};
byte pinesColumnas[COLUMNAS] = {5, 4, 3, 2};
Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas, pinesColumnas, FILAS, COLUMNAS);
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
#include <Servo.h>
Servo puerta;
const String CLAVE = "1379";
String escrito = "";

void pedir() {
  lcd.clear(); lcd.print("Ingrese la clave"); lcd.setCursor(0, 1);
}

void setup() {
  lcd.init(); lcd.backlight();
  puerta.attach(10); puerta.write(0);
  pedir();
}
```

```
30
31 void loop() {
32   char tecla = teclado.getKey();
33   if (!tecla) return;
34   if (tecla == '#') {
35     lcd.clear();
36     if (escrito == CLAVE) {
37       lcd.print("Bienvenido!"); puerta.write(90); delay(3000); puerta.write(0);
38     } else {
39       lcd.print("Clave incorrecta"); delay(2000);
40     }
41     escrito = ""; pedir();
42   } else if (tecla == '*') {
43     escrito = ""; pedir();
44   } else if (escrito.length() < 8) {
45     escrito += tecla;
46     lcd.print("*");
47   }
48 }
```

💡 ¿Cómo funciona?

Mientras escribes, la LCD muestra un asterisco por cada tecla. La función pedir() deja la pantalla lista para un nuevo intento y se reutiliza en varios lugares del programa.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca las teclas del teclado dibujado en su tarjeta, como si fuera el teclado real.

Con piezas reales: sube el programa.

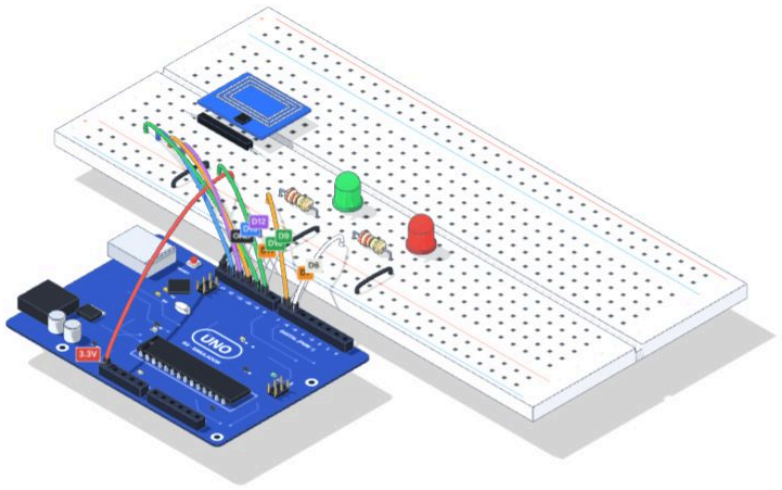
🏆 **Reto:** Después de 3 errores, bloquea el teclado 30 segundos.

PROYECTO 135 · SEGURIDAD Y CONTROL DE ACCESO

Control de acceso con tarjeta RFID

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Acerca una tarjeta al lector: la autorizada enciende la luz verde y las demás la roja.

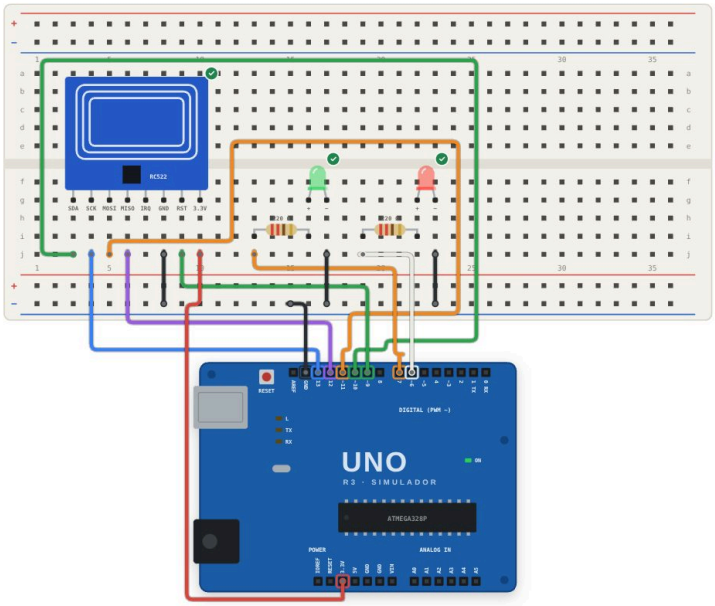


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector RFID (Lector RFID RC522)
- 1 × Autorizada (LED)
- 1 × Rechazada (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: MFRC522 (de GithubCommunity / miguelbalboa)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Lector RFID (Lector RFID RC522)**

- SDA (SS) → pin **~D10**
- SCK → pin **D13**
- MOSI → pin **~D11**
- MISO → pin **D12**
- GND → riel **–** (GND)
- RST → pin **~D9**
- 3.3V → pin **3.3V** del Arduino (no al 5V)

3 **Autorizada (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D7**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Rechazada (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D6**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **Para el proyecto real**

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

6 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 106 mA.
- Consumo estimado: **~106 mA** funcionando, con picos de **~106 mA**.

2 El programa

```
#include <SPI.h>
#include <MFRC522.h>
MFRC522 rfid(10, 9);   // SDA en D10 y RST en D9
// Devuelve 1 si la tarjeta leída es la autorizada (blanca: 4A 3F 21 9C), 0 si es otra y -1 si no hay tarjeta
int leerTarjeta() {
  if (!rfid.PICC_IsNewCardPresent()) return -1;
  if (!rfid.PICC_ReadCardSerial()) return -1;
  Serial.print("Tarjeta:");
  for (byte i = 0; i < rfid.uid.size; i++) {
    Serial.print(" ");
    Serial.print(rfid.uid.uidByte[i], HEX);
  }
  Serial.println();
  bool ok = rfid.uid.uidByte[0] == 0x4A && rfid.uid.uidByte[1] == 0x3F && rfid.uid.uidByte[2] == 0x21 && rfid.uid.uidByte[3] == 0x9C;
  rfid.PICC_HaltA();
  if (ok) return 1;
  return 0;
}

void setup() {
  Serial.begin(9600);
  SPI.begin();
  rfid.PCD_Init();
  pinMode(7, OUTPUT); pinMode(6, OUTPUT);
  Serial.println("Acerque su tarjeta");
}

```

```
28 void loop() {
29   int r = leerTarjeta();
30   if (r == 1) { Serial.println("Acceso permitido"); digitalWrite(7, HIGH); delay(2000);
   digitalWrite(7, LOW); }
31   if (r == 0) { Serial.println("Acceso denegado"); digitalWrite(6, HIGH); delay(2000);
   digitalWrite(6, LOW); }
32 }
```

💡 ¿Cómo funciona?

Cada tarjeta RFID tiene un número único (UID) de 4 bytes. El programa lo lee, lo muestra en hexadecimal y lo compara con el de la tarjeta autorizada.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Tarjeta blanca» (4A 3F 21 9C), «Llavero azul» o «Desconocida» para acercarla al lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

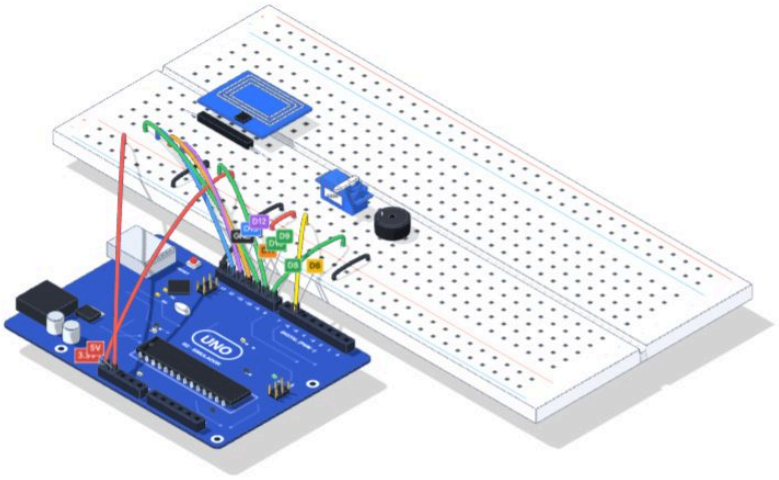
🏆 **Reto:** Autoriza también el llavero azul (93 12 AB 07).

PROYECTO 136 · SEGURIDAD Y CONTROL DE ACCESO

Puerta con tarjeta RFID y servo

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Con la tarjeta autorizada, un servo abre la puerta 3 segundos.

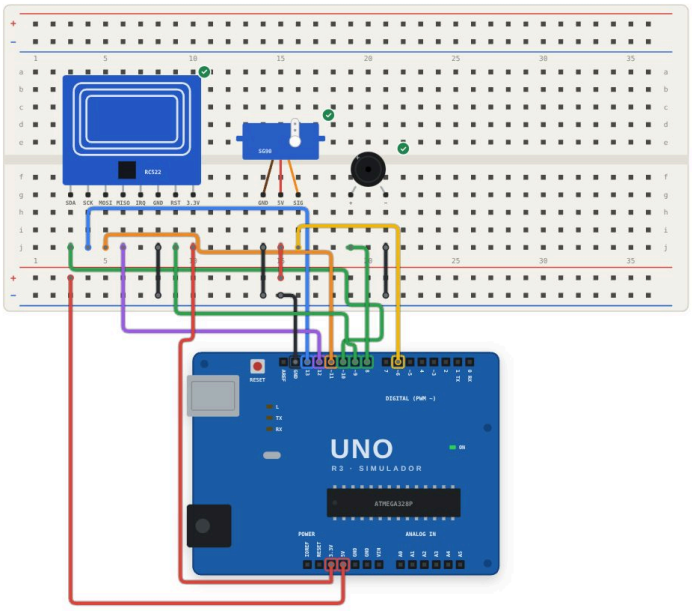


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector RFID (Lector RFID RC522)
- 1 × Puerta (Servo SG90)
- 1 × Pitido (Zumbador)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: MFRC522 (de GithubCommunity / miguelbalboa); Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector RFID (Lector RFID RC522)

- SDA (SS) → pin **~D10**
- SCK → pin **D13**
- MOSI → pin **~D11**
- MISO → pin **D12**
- GND → riel **–** (GND)
- RST → pin **~D9**
- 3.3V → pin **3.3V** del Arduino (no al 5V)

3 Puerta (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D6**

4 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Para el proyecto real

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 256 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~256 mA** funcionando, con picos de **~756 mA**.

2 El programa

```
#include <SPI.h>
#include <MFRC522.h>
MFRC522 rfid(10, 9);   // SDA en D10 y RST en D9
#include <Servo.h>
Servo puerta;
// Devuelve 1 si la tarjeta leída es la autorizada (blanca: 4A 3F 21 9C), 0 si es otra y -1 si no hay tarjeta
int leerTarjeta() {
  if (!rfid.PICC_IsNewCardPresent()) return -1;
  if (!rfid.PICC_ReadCardSerial()) return -1;
  Serial.print("Tarjeta:");
  for (byte i = 0; i < rfid.uid.size; i++) {
    Serial.print(" ");
    Serial.print(rfid.uid.uidByte[i], HEX);
  }
  Serial.println();
  bool ok = rfid.uid.uidByte[0] == 0x4A && rfid.uid.uidByte[1] == 0x3F && rfid.uid.uidByte[2] == 0x21 && rfid.uid.uidByte[3] == 0x9C;
  rfid.PICC_HaltA();
  if (ok) return 1;
  return 0;
}

void setup() {
  Serial.begin(9600);
  SPI.begin(); rfid.PCD_Init();
  puerta.attach(6); puerta.write(0);
}

```

```
28 void loop() {
29   int r = leerTarjeta();
30   if (r == 1) {
31     tone(8, 1600, 150);
32     puerta.write(90); delay(3000); puerta.write(0);
33   } else if (r == 0) {
34     tone(8, 300, 500);
35   }
36 }
```

💡 ¿Cómo funciona?

Se reutiliza la función leerTarjeta(): devuelve 1, 0 o −1. Así el loop() solo decide qué hacer con la respuesta.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Tarjeta blanca» (4A 3F 21 9C), «Llavero azul» o «Desconocida» para acercarla al lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

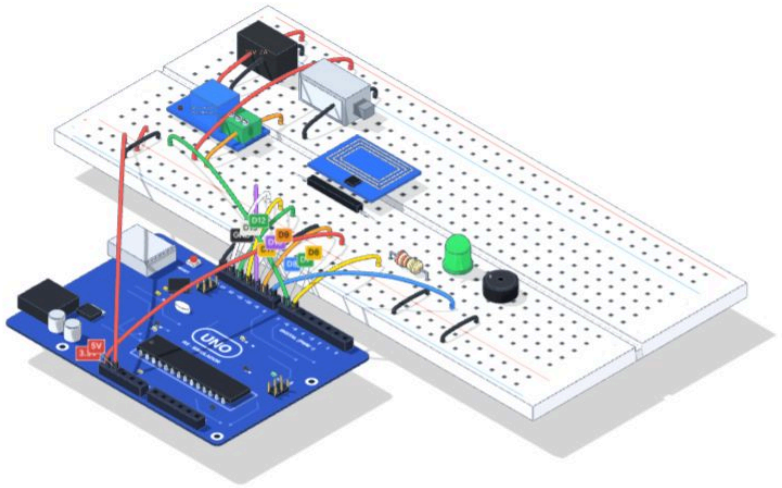
🏆 **Reto:** Cuenta cuántas veces se abrió la puerta.

PROYECTO 137 · SEGURIDAD Y CONTROL DE ACCESO

Chapa eléctrica con tarjeta RFID

★★★ Avanzado · Tiempo: 45 a 90 minutos

Como en las oficinas: la tarjeta autorizada activa el relé y la chapa eléctrica abre la puerta.

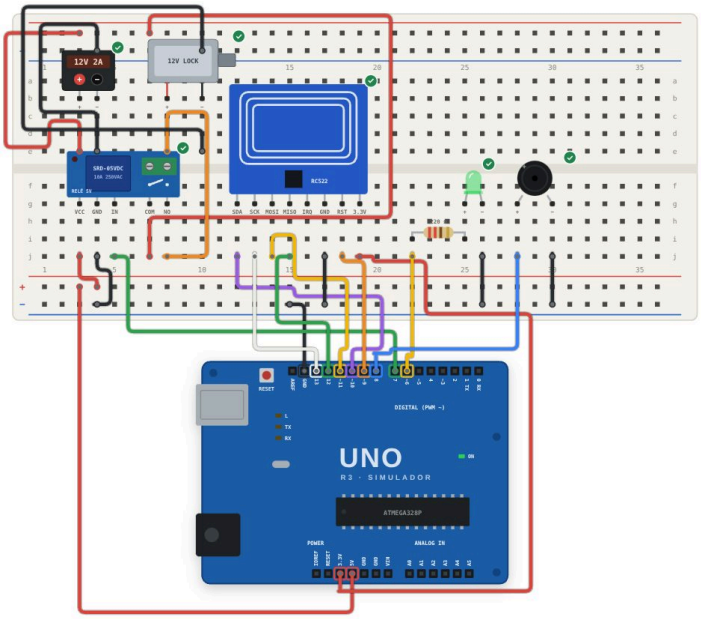


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector RFID (Lector RFID RC522)
- 1 × Relé de la chapa (Módulo relé 5 V)
- 1 × Chapa eléctrica 12 V
- 1 × Fuente 12 V
- 1 × Abierta (LED)
- 1 × Pitido (Zumbador)
- 1 × Resistencia 220 Ω
- Diodo 1N4007 en paralelo con la chapa
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: MFRC522 (de GithubCommunity / miguelbalboa)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector RFID (Lector RFID RC522)

- SDA (SS) → pin **~D10**
- SCK → pin **D13**
- MOSI → pin **~D11**
- MISO → pin **D12**
- GND → riel **–** (GND)
- RST → pin **~D9**
- 3.3V → pin **3.3V** del Arduino (no al 5V)

3 Relé de la chapa (Módulo relé 5 V)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- IN (señal) → pin **D7**
- COM (común) → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- NO (normalmente abierto) → **+ de la carga** (bomba, chapa o motor de 12 V)

4 Chapa eléctrica 12 V (Chapa eléctrica 12 V)

- Cable + (rojo) de la chapa → borne **NO** del relé
- Cable – (negro) de la chapa → **– de 12 V** (riel – de arriba)

5 Fuente 12 V (Fuente 12 V)

- Borne +12V → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- Borne – → **– de 12 V** (riel – de arriba)

6 Abierta (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D6**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

7 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

8 Para el proyecto real

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).
- La chapa de 12 V nunca va al Arduino: el relé la conecta a su propia fuente de 12 V (COM al + de la fuente, NO al + de la chapa, – de la chapa al – de la fuente).

9 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 191 mA.
- **Relé:** el lado de control (VCC, GND, IN) va al Arduino con 5 V; la carga va por COM y NO con su propia fuente. Algunos módulos se activan con LOW: si el tuyo funciona al revés, invierte HIGH y LOW en el código.
- Consumo estimado: **~191 mA** funcionando, con picos de **~191 mA**.

2 El programa

```
#include <SPI.h>
#include <MFRC522.h>
MFRC522 rfid(10, 9);   // SDA en D10 y RST en D9
// Devuelve 1 si la tarjeta leída es la autorizada (blanca: 4A 3F 21 9C), 0 si es otra y -1 si no hay tarjeta
int leerTarjeta() {
  if (!rfid.PICC_IsNewCardPresent()) return -1;
  if (!rfid.PICC_ReadCardSerial()) return -1;
  Serial.print("Tarjeta:");
  for (byte i = 0; i < rfid.uid.size; i++) {
    Serial.print(" ");
    Serial.print(rfid.uid.uidByte[i], HEX);
  }
```

```
  Serial.println();
  bool ok = rfid.uid.uidByte[0] == 0x4A && rfid.uid.uidByte[1] == 0x3F && rfid.uid.uidByte[2] ==
0x21 && rfid.uid.uidByte[3] == 0x9C;
  rfid.PICC_HaltA();
  if (ok) return 1;
  return 0;
}
const int RELE = 7, VERDE = 6, PITO = 8;

void setup() {
  Serial.begin(9600);
  SPI.begin(); rfid.PCD_Init();
  pinMode(RELE, OUTPUT); pinMode(VERDE, OUTPUT);
}

void loop() {
  int r = leerTarjeta();
  if (r == 1) {
    digitalWrite(VERDE, HIGH); tone(PITO, 1800, 120);
    digitalWrite(RELE, HIGH);            // abre la chapa
    delay(3000);
    digitalWrite(RELE, LOW);
    digitalWrite(VERDE, LOW);
  } else if (r == 0) {
    for (int i = 0; i < 2; i++) { tone(PITO, 300, 200); delay(300); }
  }
}
```

💡 ¿Cómo funciona?

La lógica es la misma de la puerta con servo, pero la salida va a un relé. El relé maneja los 12 V de la chapa, que el Arduino no puede dar.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Tarjeta blanca» (4A 3F 21 9C), «Llavero azul» o «Desconocida» para acercarla al lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).
- La chapa de 12 V nunca va al Arduino: el relé la conecta a su propia fuente de 12 V (COM al + de la fuente, NO al + de la chapa, − de la chapa al − de la fuente).

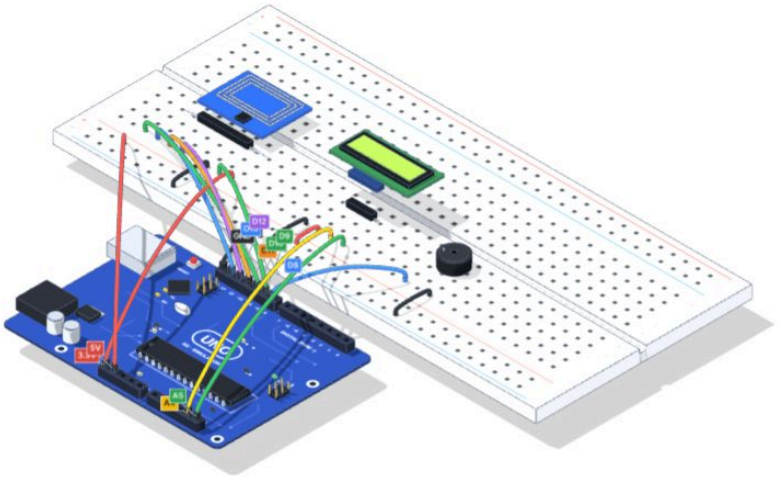
🏆 **Reto:** Usa el llavero azul como "llave maestra" que deja la puerta abierta hasta pasarlo de nuevo.

PROYECTO 138 · SEGURIDAD Y CONTROL DE ACCESO

Bienvenida con tarjeta RFID y LCD

★★★ Avanzado · Tiempo: 45 a 90 minutos

Cada tarjeta tiene un dueño: la LCD saluda por su nombre a quien la acerca.

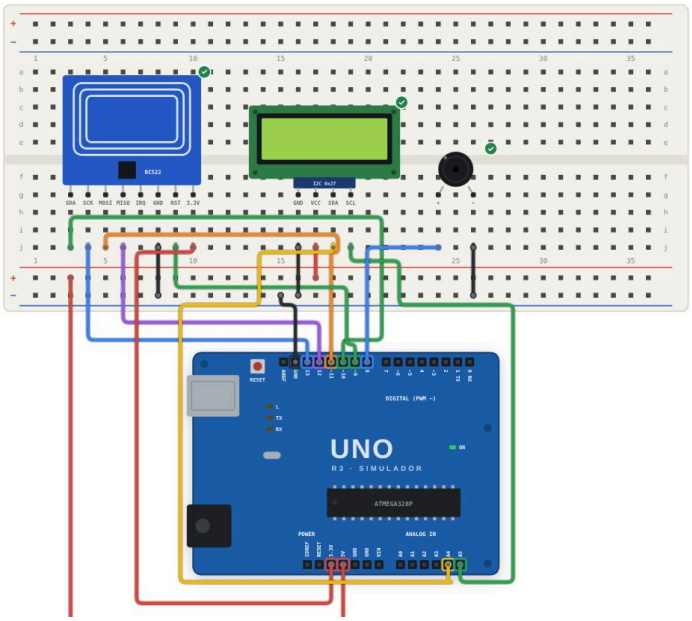


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector RFID (Lector RFID RC522)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Pitido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: MFRC522 (de GithubCommunity / miguelbalboa); LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector RFID (Lector RFID RC522)

- SDA (SS) → pin **~D10**
- SCK → pin **D13**
- MOSI → pin **~D11**
- MISO → pin **D12**
- GND → riel **–** (GND)
- RST → pin **~D9**
- 3.3V → pin **3.3V** del Arduino (no al 5V)

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Para el proyecto real

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 136 mA.
- Consumo estimado: **~136 mA** funcionando, con picos de **~136 mA**.

2 El programa

```
#include <SPI.h>
#include <MFRC522.h>
MFRC522 rfid(10, 9);   // SDA en D10 y RST en D9
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  SPI.begin(); rfid.PCD_Init();
  lcd.init(); lcd.backlight();
  lcd.print("Acerque tarjeta");
}

void loop() {
  if (!rfid.PICC_IsNewCardPresent()) return;
  if (!rfid.PICC_ReadCardSerial()) return;
  byte primero = rfid.uid.uidByte[0];
  lcd.clear();
  if (primero == 0x4A) { lcd.print("Hola, Ana!"); tone(8, 1500, 150); }
  else if (primero == 0x93) { lcd.print("Hola, Profe!"); tone(8, 1800, 150); }
  else { lcd.print("Tarjeta nueva"); tone(8, 400, 400); }
  lcd.setCursor(0, 1);
  lcd.print("UID: "); lcd.print(rfid.uid.uidByte[0], HEX); lcd.print(" ");
lcd.print(rfid.uid.uidByte[1], HEX);
  rfid.PICC_HaltA();
  delay(2500);
  lcd.clear(); lcd.print("Acerque tarjeta");
}
```

💡 ¿Cómo funciona?

Para simplificar, aquí se distingue cada tarjeta por su primer byte. En un sistema real se comparan los 4 bytes, como en la función leerTarjeta().

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Tarjeta blanca» (4A 3F 21 9C), «Llavero azul» o «Desconocida» para acercarla al lector.

Con piezas reales: sube el programa.

⚠️ Ten en cuenta

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

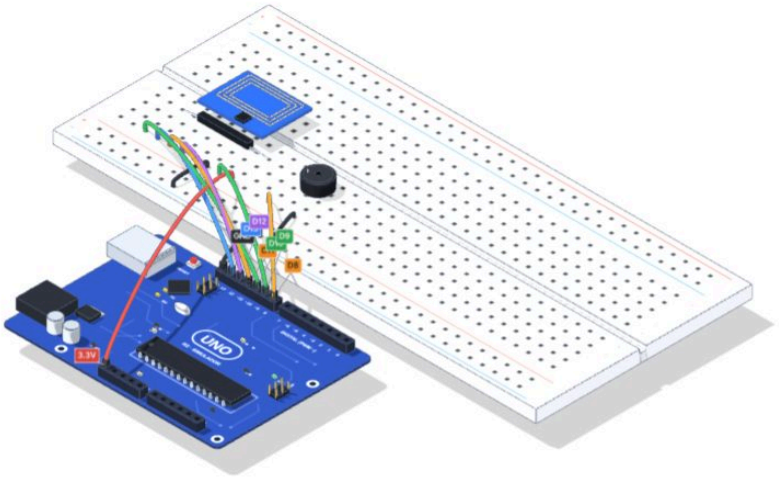
🏆 **Reto:** Agrega una tercera persona con su propia tarjeta.

PROYECTO 139 · SEGURIDAD Y CONTROL DE ACCESO

Registro de entradas con RFID

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Cuenta cuántas veces ha entrado cada tarjeta y lo muestra en el monitor serie.

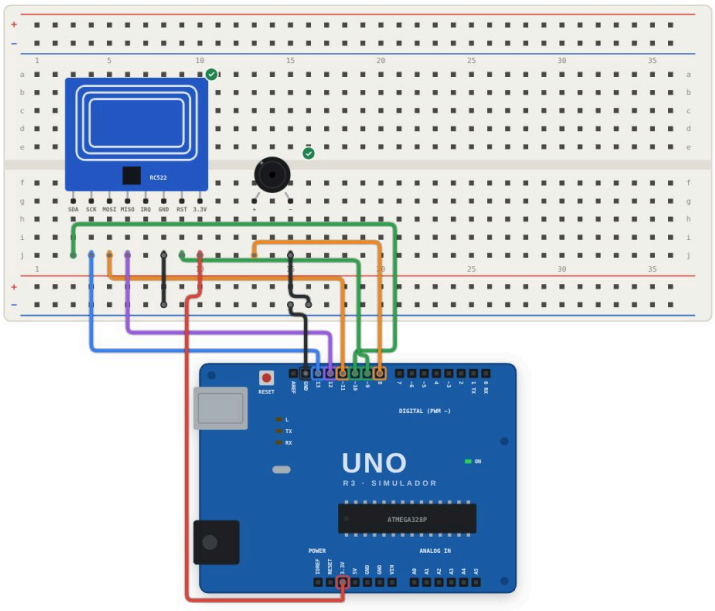


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector RFID (Lector RFID RC522)
- 1 × Pitido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: MFRC522 (de GithubCommunity / miguelbalboa)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Lector RFID (Lector RFID RC522)**

- SDA (SS) → pin **~D10**
- SCK → pin **D13**
- MOSI → pin **~D11**
- MISO → pin **D12**
- GND → riel **–** (GND)
- RST → pin **~D9**
- 3.3V → pin **3.3V** del Arduino (no al 5V)

3 **Pitido (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 **Para el proyecto real**

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 106 mA.
- Consumo estimado: **~106 mA** funcionando, con picos de **~106 mA**.

2 El programa

```
#include <SPI.h>
#include <MFRC522.h>
MFRC522 rfid(10, 9);   // SDA en D10 y RST en D9
int entradasBlanca = 0, entradasAzul = 0, otras = 0;

void setup() {
  Serial.begin(9600);
  SPI.begin(); rfid.PCD_Init();
}

void loop() {
  if (!rfid.PICC_IsNewCardPresent()) return;
  if (!rfid.PICC_ReadCardSerial()) return;
  if (rfid.uid.uidByte[0] == 0x4A) entradasBlanca++;
  else if (rfid.uid.uidByte[0] == 0x93) entradasAzul++;
  else otras++;
  tone(8, 1500, 80);
  Serial.print("Blanca: "); Serial.print(entradasBlanca);
  Serial.print("  Azul: "); Serial.print(entradasAzul);
  Serial.print("  Otras: "); Serial.println(otras);
  rfid.PICC_HaltA();
}
```

💡 ¿Cómo funciona?

Cada tarjeta tiene su propio contador. Después de cada lectura se imprime el resumen completo, como una bitácora de la portería.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Tarjeta blanca» (4A 3F 21 9C), «Llavero azul» o «Desconocida» para acercarla al lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ **Ten en cuenta**

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

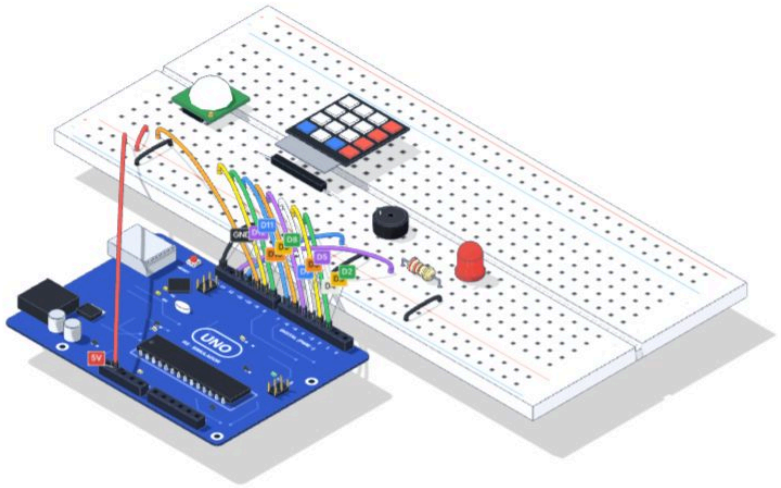
🏆 **Reto:** Muestra la hora (en segundos) de la última entrada de cada tarjeta.

PROYECTO 140 · SEGURIDAD Y CONTROL DE ACCESO

Alarma con PIR que se desactiva con clave

★★★ Avanzado · Tiempo: 45 a 90 minutos

Si el PIR detecta a alguien, hay 10 segundos para escribir la clave; si no, suena la sirena.

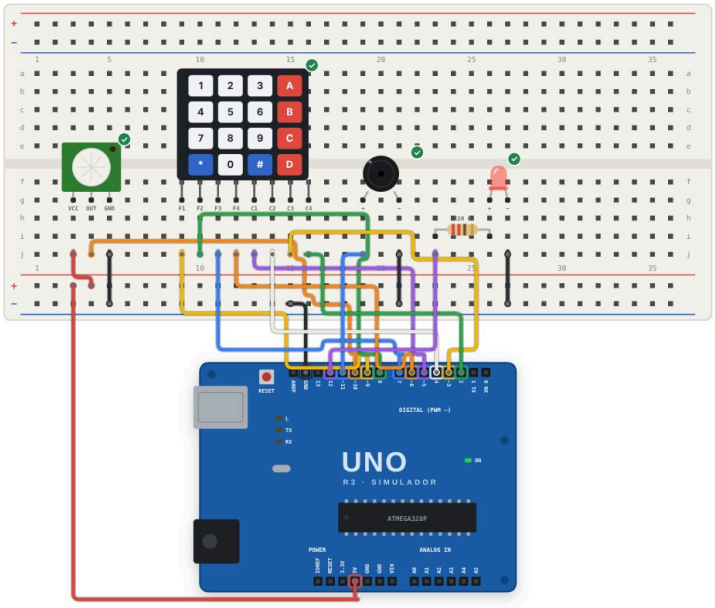


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor PIR
- 1 × Teclado (Teclado matricial 4×4)
- 1 × Sirena (Zumbador)
- 1 × Armada (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Keypad (de Mark Stanley y Alexander Brevig)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor PIR (Sensor PIR)

- VCC → riel **+** (5V)
- OUT (señal) → pin **~D10**
- GND → riel **–** (GND)

3 Teclado (Teclado matricial 4×4)

- Fila 1 → pin **~D9**
- Fila 2 → pin **D8**
- Fila 3 → pin **D7**
- Fila 4 → pin **~D6**
- Columna 1 → pin **~D5**
- Columna 2 → pin **D4**
- Columna 3 → pin **~D3**
- Columna 4 → pin **D2**

4 Sirena (Zumbador)

- Pata + → pin **~D11**
- Pata – → riel **–** (GND)

5 Armada (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 97 mA.
- Consumo estimado: **~97 mA** funcionando, con picos de **~97 mA**.

2 El programa

```
#include <Keypad.h>
const byte FILAS = 4;
const byte COLUMNAS = 4;
char teclas[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte pinesFilas[FILAS] = {9, 8, 7, 6};
byte pinesColumnas[COLUMNAS] = {5, 4, 3, 2};
Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas, pinesColumnas, FILAS, COLUMNAS);
const String CLAVE = "0000";
String escrito = "";
bool armada = true;

void setup() {
  Serial.begin(9600);
  pinMode(10, INPUT); pinMode(12, OUTPUT);
}

void loop() {
  digitalWrite(12, armada);
  char tecla = teclado.getKey();
  if (tecla) {
    if (tecla == '#') { if (escrito == CLAVE) { armada = !armada; Serial.println(armada ? "Alarma armada" : "Alarma desarmada"); } escrito = ""; }
    else escrito += tecla;
```

```
  }
  if (armada && digitalRead(10) == HIGH) {
    Serial.println("¡Movimiento! Tienes 10 segundos para la clave");
    unsigned long inicio = millis();
    escrito = "";
    while (millis() - inicio < 10000 && armada) {
      tone(11, 2000, 30); delay(300);
      char t = teclado.getKey();
      if (t == '#') { if (escrito == CLAVE) armada = false; escrito = ""; }
      else if (t) escrito += t;
    }
    if (armada) {
      Serial.println("¡INTRUSO!");
      for (int i = 0; i < 30; i++) { tone(11, 1500); delay(150); tone(11, 900); delay(150); }
      noTone(11);
    } else Serial.println("Alarma desarmada a tiempo");
  }
}
```

💡 ¿Cómo funciona?

Las alarmas de casa dan unos segundos para desarmarlas al entrar. Mientras corre ese tiempo, el programa sigue leyendo el teclado; si la clave llega a tiempo, la sirena no suena.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Activa la casilla «hay movimiento» del sensor PIR.
- Toca las teclas del teclado dibujado en su tarjeta, como si fuera el teclado real.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

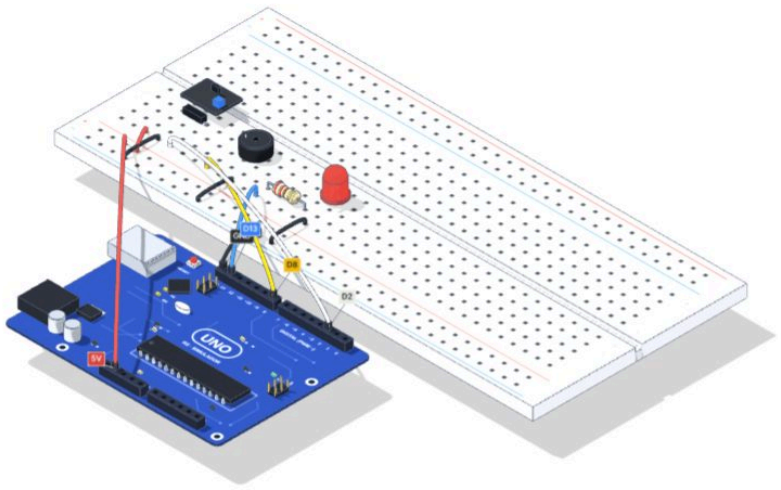
🏆 **Reto:** Agrega una LCD que muestre la cuenta regresiva.

PROYECTO 141 · SEGURIDAD Y CONTROL DE ACCESO

Alarma de ventana con imán

★★☆ Intermedio · Tiempo: 30 a 45 minutos

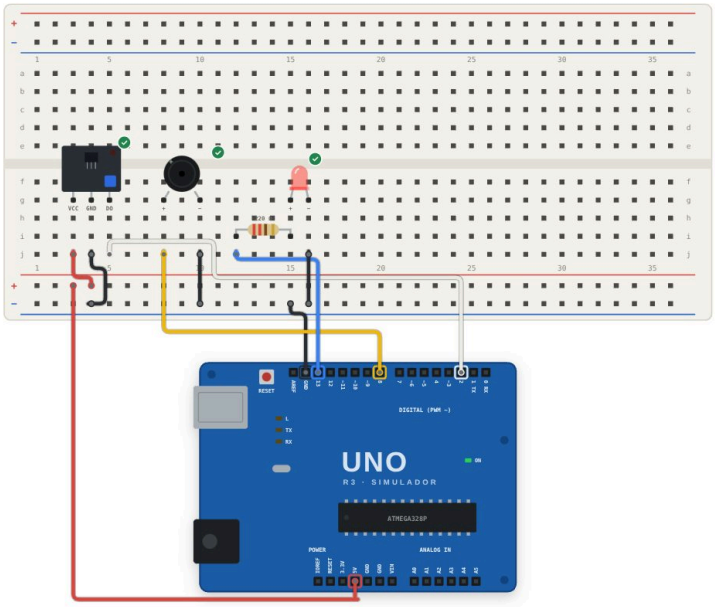
Un imán en la ventana y un sensor Hall en el marco: si la ventana se abre, el imán se aleja y suena la alarma.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor Hall (Sensor magnético Hall)
- 1 × Alarma (Zumbador)
- 1 × Ventana abierta (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor Hall (Sensor magnético Hall)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Ventana abierta (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Para el proyecto real

- En el simulador, marca «Hay un imán cerca» para simular la ventana cerrada.

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 100 mA.
- Consumo estimado: **~100 mA** funcionando, con picos de **~100 mA**.

2 El programa

```
// Alarma de ventana con sensor Hall
const int HALL = 2;          // LOW cuando el imán está cerca (ventana cerrada)

void setup() {
  Serial.begin(9600);
  pinMode(HALL, INPUT);
  pinMode(13, OUTPUT);
}

void loop() {
  if (digitalRead(HALL) == HIGH) {          // el imán se alejó
    Serial.println("¡Ventana abierta!");
    digitalWrite(13, HIGH);
    tone(8, 2200, 200);
    delay(400);
  } else {
    digitalWrite(13, LOW);
  }
}
```

💡 ¿Cómo funciona?

El sensor Hall detecta campos magnéticos. Con la ventana cerrada el imán está pegado al sensor (LOW); al abrirla, el imán se aleja y la salida pasa a HIGH.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca o desmarca la casilla «Hay un imán cerca».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ **Ten en cuenta**

- En el simulador, marca «Hay un imán cerca» para simular la ventana cerrada.

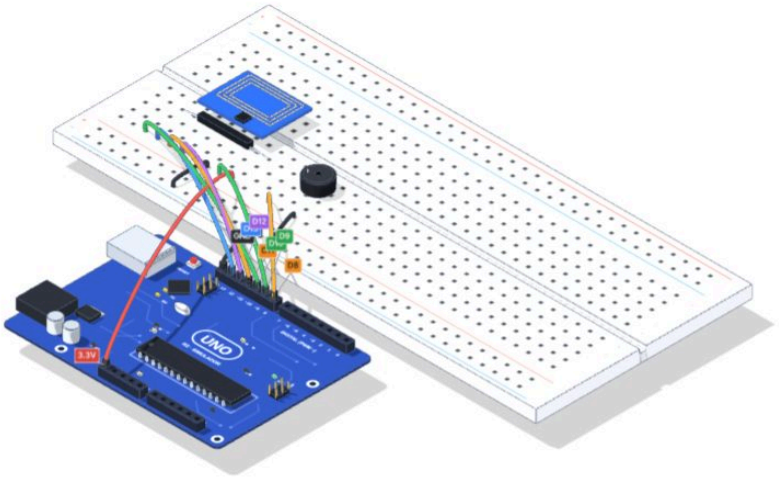
🏆 **Reto:** Agrega un segundo sensor para la puerta.

PROYECTO 142 · SEGURIDAD Y CONTROL DE ACCESO

Timbre que solo suena con tarjeta

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Un timbre inteligente: si la tarjeta es de la familia toca una melodía alegre; si no, un timbre normal.

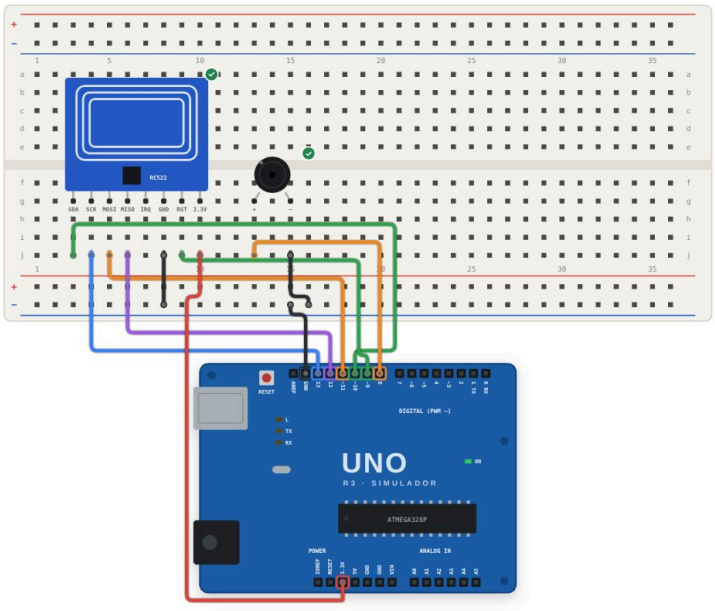


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector RFID (Lector RFID RC522)
- 1 × Timbre (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: MFRC522 (de GithubCommunity / miguelbalboa)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Lector RFID (Lector RFID RC522)**

- SDA (SS) → pin **~D10**
- SCK → pin **D13**
- MOSI → pin **~D11**
- MISO → pin **D12**
- GND → riel **–** (GND)
- RST → pin **~D9**
- 3.3V → pin **3.3V** del Arduino (no al 5V)

3 **Timbre (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 **Para el proyecto real**

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 106 mA.
- Consumo estimado: **~106 mA** funcionando, con picos de **~106 mA**.

2 El programa

```
#include <SPI.h>
#include <MFRC522.h>
MFRC522 rfid(10, 9);   // SDA en D10 y RST en D9
// Devuelve 1 si la tarjeta leída es la autorizada (blanca: 4A 3F 21 9C), 0 si es otra y -1 si no hay tarjeta
int leerTarjeta() {
  if (!rfid.PICC_IsNewCardPresent()) return -1;
  if (!rfid.PICC_ReadCardSerial()) return -1;
  Serial.print("Tarjeta:");
  for (byte i = 0; i < rfid.uid.size; i++) {
    Serial.print(" ");
    Serial.print(rfid.uid.uidByte[i], HEX);
  }
  Serial.println();
  bool ok = rfid.uid.uidByte[0] == 0x4A && rfid.uid.uidByte[1] == 0x3F && rfid.uid.uidByte[2] == 0x21 && rfid.uid.uidByte[3] == 0x9C;
  rfid.PICC_HaltA();
  if (ok) return 1;
  return 0;
}

void melodiaFamilia() {
  int notas[] = {523, 659, 784, 1047};
  for (int i = 0; i < 4; i++) { tone(8, notas[i], 150); delay(180); }
}

void setup() {
  Serial.begin(9600);
  SPI.begin(); rfid.PCD_Init();
}

void loop() {
  int r = leerTarjeta();
  if (r == 1) { Serial.println("Llegó alguien de la familia"); melodiaFamilia(); }
  if (r == 0) { Serial.println("Visitante"); tone(8, 880, 300); delay(350); tone(8, 660, 500); }
```

```
34 }
```

💡 ¿Cómo funciona?

El mismo lector sirve para distinguir quién llega. La función melodiaFamilia() guarda las notas en un arreglo local, que solo existe dentro de ella.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Tarjeta blanca» (4A 3F 21 9C), «Llavero azul» o «Desconocida» para acercarla al lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

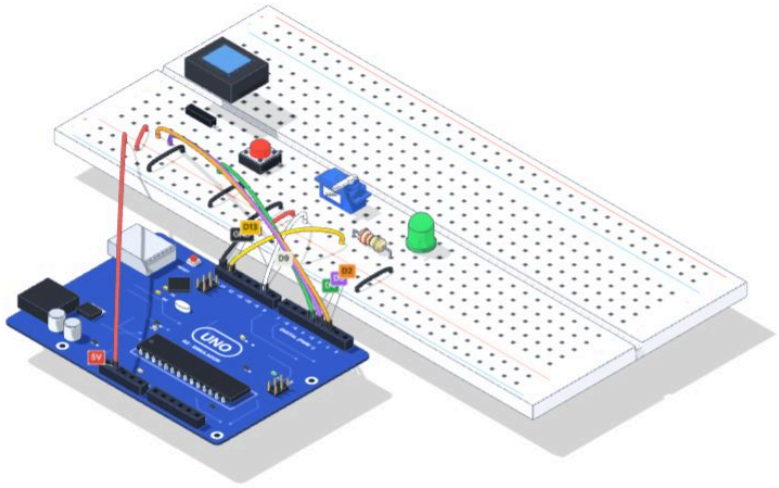
🏆 **Reto:** Agrega un LED que se encienda cuando llega un visitante.

PROYECTO 143 · SEGURIDAD Y CONTROL DE ACCESO

Doble seguridad: huella y botón de salida

★★★ Avanzado · Tiempo: 45 a 90 minutos

Para entrar se necesita la huella; para salir basta con presionar el botón de salida, como en las oficinas.

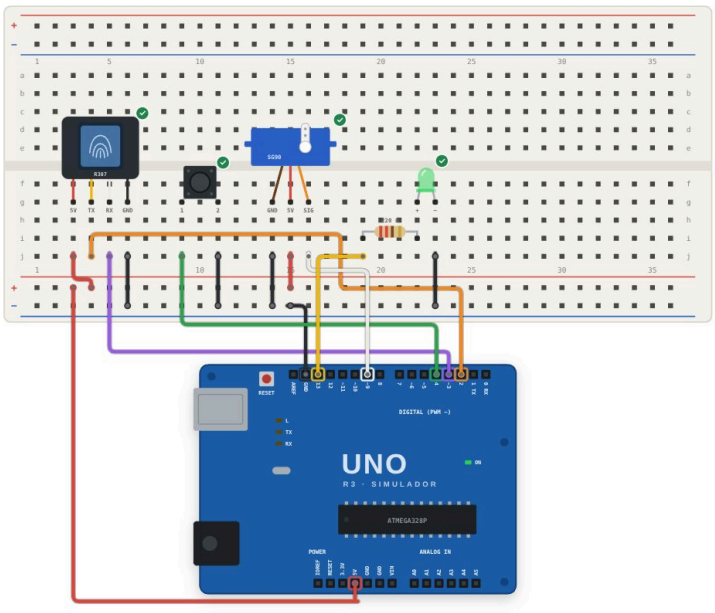


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector de huellas (Lector de huellas R307)
- 1 × Botón de salida (Botón)
- 1 × Pestillo (Servo SG90)
- 1 × Abierta (LED)
- 1 × Resistencia 220 Ω
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Adafruit Fingerprint Sensor Library; Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector de huellas (Lector de huellas R307)

- Cable rojo (VCC) → riel **+** (5V)
- Cable amarillo (TX del lector) → pin **D2**
- Cable blanco (RX del lector) → pin **~D3**
- Cable negro (GND) → riel **–** (GND)

3 Botón de salida (Botón)

- Pata 1 → pin **D4**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Pestillo (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

5 Abierta (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 Para el proyecto real

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 275 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~275 mA** funcionando, con picos de **~835 mA**.

2 El programa

```
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);   // TX del lector en D2, RX en D3
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
#include <Servo.h>
Servo pestillo;
// Devuelve el número de la huella reconocida, 0 si no la conoce o -1 si no hay dedo
int leerHuella() {
  if (finger.getImage() != FINGERPRINT_OK) return -1;
  if (finger.image2Tz() != FINGERPRINT_OK) return -1;
  if (finger.fingerFastSearch() != FINGERPRINT_OK) return 0;
  return finger.fingerID;
}

void abrir(String motivo) {
  Serial.println(motivo);
  digitalWrite(13, HIGH); pestillo.write(90);
  delay(3000);
  pestillo.write(0); digitalWrite(13, LOW);
}

void setup() {
  Serial.begin(9600);
  finger.begin(57600);
  pestillo.attach(9); pestillo.write(0);
```

```
  pinMode(4, INPUT_PULLUP); pinMode(13, OUTPUT);
}

void loop() {
  if (digitalRead(4) == LOW) abrir("Salida con botón");
  int id = leerHuella();
  if (id > 0) abrir("Entrada con huella");
}
```

💡 ¿Cómo funciona?

La función abrir() recibe un texto que explica por qué se abrió y hace todo el movimiento. Así no se repite el mismo código para la huella y para el botón.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Huella 1», «Huella 2», «Huella 3» (registradas) o «Desconocida» en la tarjeta del lector.
- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

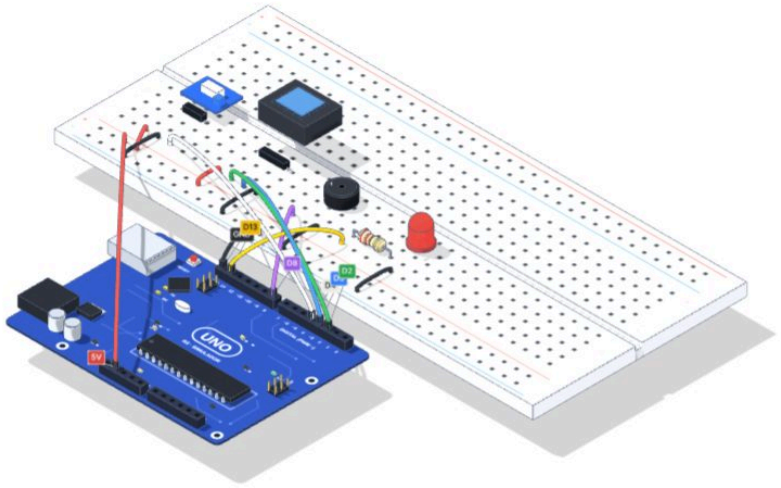
🏆 **Reto:** Registra en el monitor cuántas entradas y cuántas salidas hubo.

PROYECTO 144 · SEGURIDAD Y CONTROL DE ACCESO

Alarma de vibración para caja fuerte

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Si alguien golpea o intenta mover la caja fuerte, se enciende la sirena y no se apaga hasta poner la huella.

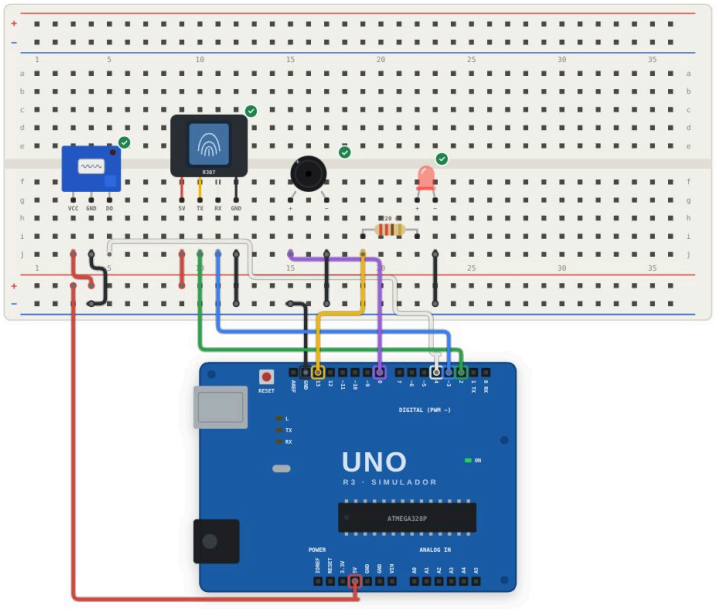


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de vibración
- 1 × Lector de huellas (Lector de huellas R307)
- 1 × Sirena (Zumbador)
- 1 × Alarma (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Adafruit Fingerprint Sensor Library



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de vibración (Sensor de vibración)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D4**

3 Lector de huellas (Lector de huellas R307)

- Cable rojo (VCC) → riel **+** (5V)
- Cable amarillo (TX del lector) → pin **D2**
- Cable blanco (RX del lector) → pin **~D3**
- Cable negro (GND) → riel **–** (GND)

4 Sirena (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Alarma (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 Para el proyecto real

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 160 mA.
- Consumo estimado: **~160 mA** funcionando, con picos de **~220 mA**.

2 El programa

```
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);   // TX del lector en D2, RX en D3
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
// Devuelve el número de la huella reconocida, 0 si no la conoce o -1 si no hay dedo
int leerHuella() {
  if (finger.getImage() != FINGERPRINT_OK) return -1;
  if (finger.image2Tz() != FINGERPRINT_OK) return -1;
  if (finger.fingerFastSearch() != FINGERPRINT_OK) return 0;
  return finger.fingerID;
}
bool alarma = false;

void setup() {
  Serial.begin(9600);
  finger.begin(57600);
  pinMode(4, INPUT); pinMode(13, OUTPUT);
}

void loop() {
  if (digitalRead(4) == HIGH && !alarma) { alarma = true; Serial.println("¡Golpe detectado!"); }
  if (alarma) {
    digitalWrite(13, HIGH); tone(8, 1700); delay(150);
    digitalWrite(13, LOW); tone(8, 1100); delay(150);
    if (leerHuella() > 0) { alarma = false; noTone(8); Serial.println("Alarma apagada por el dueño"); }
  }
```

```
27 }
```

💡 ¿Cómo funciona?

Mientras la alarma está activa, en cada vuelta se intenta leer una huella. Solo una huella registrada la apaga; desconectar el sensor no sirve.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay vibración o un golpe».
- Toca «Huella 1», «Huella 2», «Huella 3» (registradas) o «Desconocida» en la tarjeta del lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

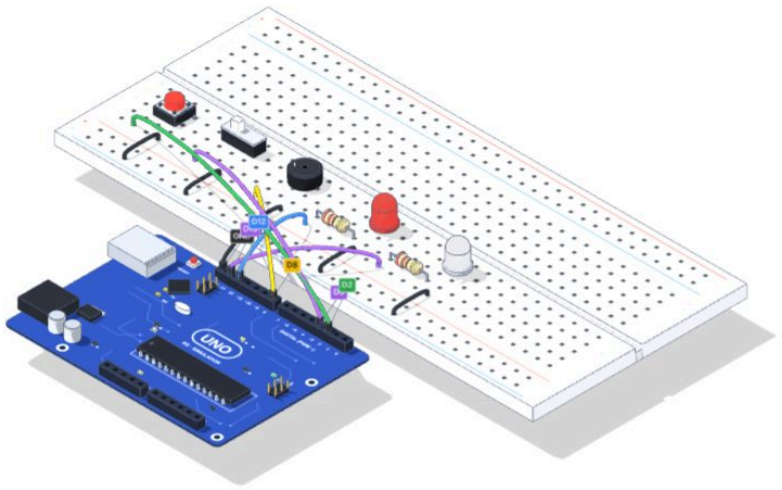
🏆 **Reto:** Haz que la alarma también envíe un mensaje por el monitor cada 5 segundos.

PROYECTO 145 · SEGURIDAD Y CONTROL DE ACCESO

Botón de emergencia con sirena y luces

★★☆ Intermedio · Tiempo: 30 a 45 minutos

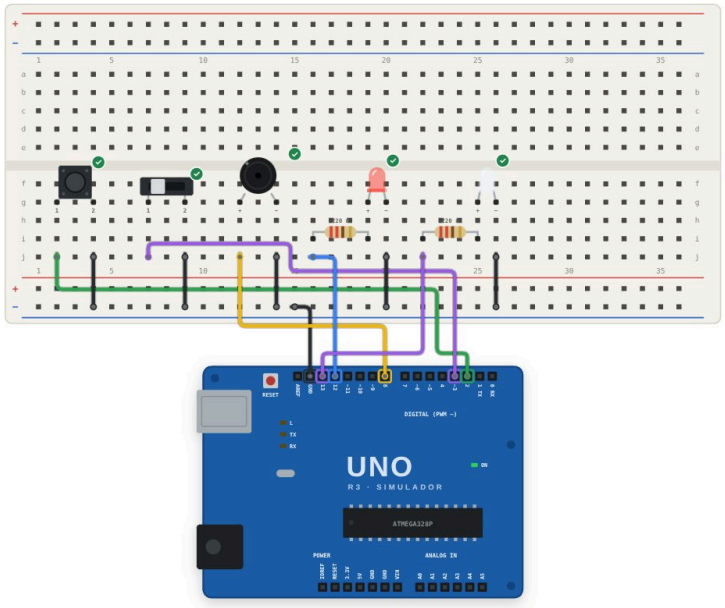
Un botón grande activa la alarma de evacuación con luces y sirena; un interruptor la apaga.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Emergencia (Botón)
- 1 × Restablecer (Interruptor)
- 1 × Sirena (Zumbador)
- 1 × Luz roja (LED)
- 1 × Luz blanca (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Emergencia (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Restablecer (Interruptor)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**.

4 **Sirena (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 **Luz roja (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 **Luz blanca (LED blanco)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
// Alarma de evacuación
bool emergencia = false;

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
}

void loop() {
  if (digitalRead(2) == LOW && !emergencia) { emergencia = true; Serial.println("¡EVACUACIÓN!"); }
  if (digitalRead(3) == LOW) emergencia = false;
  if (emergencia) {
    for (int f = 700; f < 1600; f += 50) { tone(8, f); delay(10); }
    digitalWrite(12, HIGH); digitalWrite(13, LOW); delay(150);
    digitalWrite(12, LOW); digitalWrite(13, HIGH); delay(150);
  } else {
    noTone(8); digitalWrite(12, LOW); digitalWrite(13, LOW);
  }
}
```

💡 **¿Cómo funciona?**

La sirena sube de tono en cada ciclo y las dos luces se alternan. El interruptor de restablecer está pensado para que solo el encargado pueda apagar la alarma.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).
- Cambia la posición del interruptor en su tarjeta del panel «Componentes».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

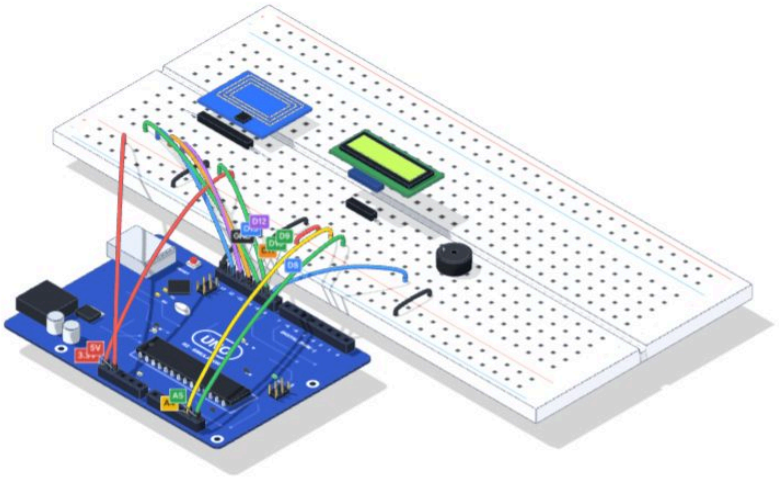
🏆 **Reto:** Agrega un sensor de llama que active la misma alarma.

PROYECTO 146 · SEGURIDAD Y CONTROL DE ACCESO

Contador de intentos de acceso en LCD

★★★ Avanzado · Tiempo: 45 a 90 minutos

La LCD lleva la cuenta de accesos permitidos y denegados con tarjeta RFID.

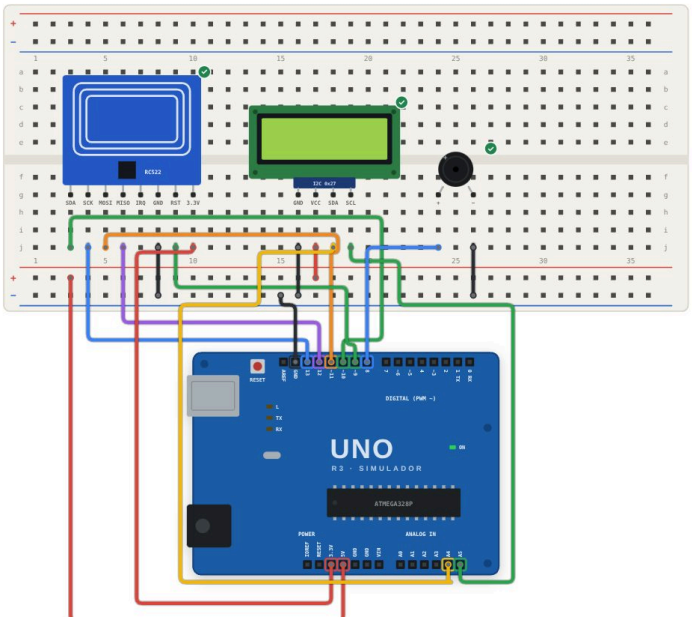


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector RFID (Lector RFID RC522)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Pitido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: MFRC522 (de GithubCommunity / miguelbalboa); LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector RFID (Lector RFID RC522)

- SDA (SS) → pin **~D10**
- SCK → pin **D13**
- MOSI → pin **~D11**
- MISO → pin **D12**
- GND → riel **–** (GND)
- RST → pin **~D9**
- 3.3V → pin **3.3V** del Arduino (no al 5V)

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Para el proyecto real

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 136 mA.
- Consumo estimado: **~136 mA** funcionando, con picos de **~136 mA**.

2 El programa

```
#include <SPI.h>
#include <MFRC522.h>
MFRC522 rfid(10, 9);   // SDA en D10 y RST en D9
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
// Devuelve 1 si la tarjeta leída es la autorizada (blanca: 4A 3F 21 9C), 0 si es otra y -1 si no hay tarjeta
int leerTarjeta() {
  if (!rfid.PICC_IsNewCardPresent()) return -1;
  if (!rfid.PICC_ReadCardSerial()) return -1;
  Serial.print("Tarjeta:");
  for (byte i = 0; i < rfid.uid.size; i++) {
    Serial.print(" ");
    Serial.print(rfid.uid.uidByte[i], HEX);
  }
  Serial.println();
  bool ok = rfid.uid.uidByte[0] == 0x4A && rfid.uid.uidByte[1] == 0x3F && rfid.uid.uidByte[2] == 0x21 && rfid.uid.uidByte[3] == 0x9C;
  rfid.PICC_HaltA();
  if (ok) return 1;
  return 0;
}
int permitidos = 0, denegados = 0;

void mostrar() {
  lcd.clear();
  lcd.print("Permitidos: "); lcd.print(permitidos);
  lcd.setCursor(0, 1);
```

```
  lcd.print("Denegados:  "); lcd.print(denegados);
}

void setup() {
  Serial.begin(9600);
  SPI.begin(); rfid.PCD_Init();
  lcd.init(); lcd.backlight();
  mostrar();
}

void loop() {
  int r = leerTarjeta();
  if (r == 1) { permitidos++; tone(8, 1600, 120); mostrar(); }
  if (r == 0) { denegados++; tone(8, 300, 400); mostrar(); }
}
```

💡 **¿Cómo funciona?**

Las dos variables cuentan los resultados de leerTarjeta(). La pantalla solo se actualiza cuando cambia algún número.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Toca «Tarjeta blanca» (4A 3F 21 9C), «Llavero azul» o «Desconocida» para acercarla al lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ **Ten en cuenta**

- Para usar tu propia tarjeta, mira su código en el monitor serie y cámbialo en el programa (4A 3F 21 9C es la tarjeta blanca del simulador).

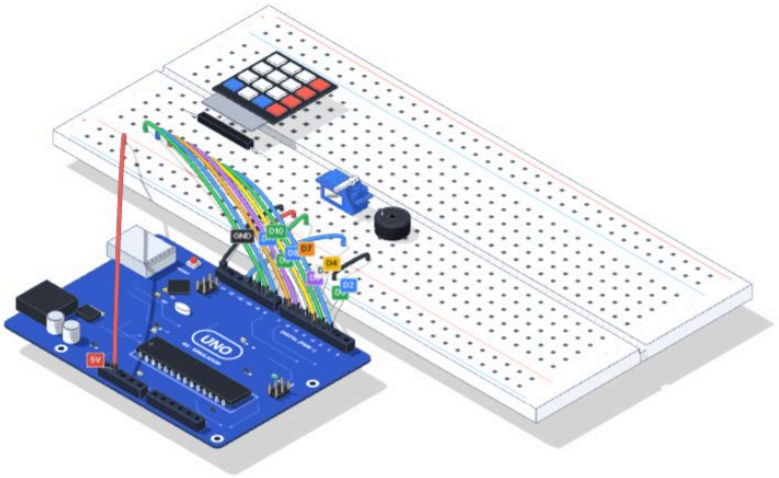
🏆 **Reto:** Muestra el porcentaje de accesos denegados.

PROYECTO 147 · SEGURIDAD Y CONTROL DE ACCESO

Cofre del tesoro con clave

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Para un juego de escape: el cofre (servo) solo se abre con la clave 7 3 9 y tocando una melodía de victoria.

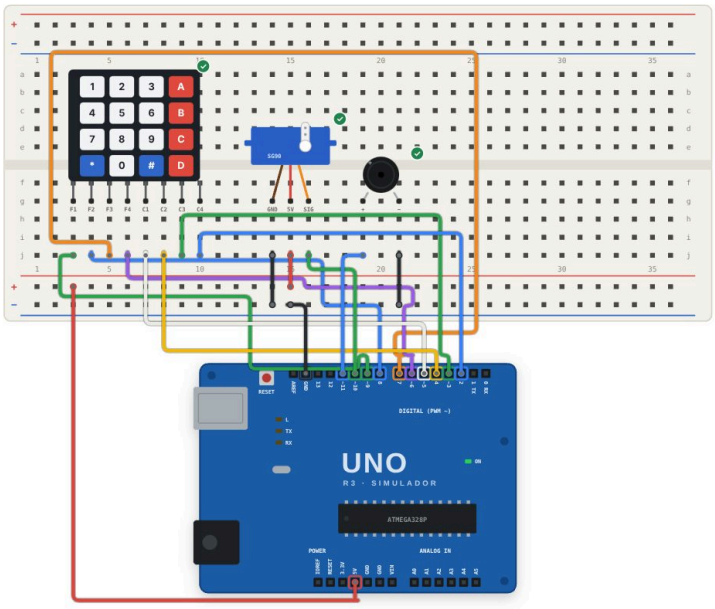


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Teclado (Teclado matricial 4×4)
- 1 × Tapa del cofre (Servo SG90)
- 1 × Música (Zumbador)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Keypad (de Mark Stanley y Alexander Brevig); Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Teclado (Teclado matricial 4×4)

- Fila 1 → pin **~D9**
- Fila 2 → pin **D8**
- Fila 3 → pin **D7**
- Fila 4 → pin **~D6**
- Columna 1 → pin **~D5**
- Columna 2 → pin **D4**
- Columna 3 → pin **~D3**
- Columna 4 → pin **D2**

3 Tapa del cofre (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D10**

4 Música (Zumbador)

- Pata + → pin **~D11**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 231 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~231 mA** funcionando, con picos de **~731 mA**.

2 El programa

```
#include <Keypad.h>
const byte FILAS = 4;
const byte COLUMNAS = 4;
char teclas[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte pinesFilas[FILAS] = {9, 8, 7, 6};
byte pinesColumnas[COLUMNAS] = {5, 4, 3, 2};
Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas, pinesColumnas, FILAS, COLUMNAS);
#include <Servo.h>
Servo tapa;
String escrito = "";

void victoria() {
  int notas[] = {523, 659, 784, 659, 784, 1047};
  for (int i = 0; i < 6; i++) { tone(11, notas[i], 150); delay(170); }
}

void setup() {
  tapa.attach(10); tapa.write(0);
}

void loop() {
  char tecla = teclado.getKey();
  if (!tecla) return;
  tone(11, 1800, 30);
  escrito += tecla;
  if (escrito.length() == 3) {
```

```
    if (escrito == "739") { victoria(); tapa.write(100); delay(5000); tapa.write(0); }
    else tone(11, 250, 600);
    escrito = "";
  }
}
```

💡 ¿Cómo funciona?

Aquí no hace falta la tecla #: al completar 3 dígitos se revisa la clave automáticamente. Los juegos de escape usan cofres así con pistas escondidas en el salón.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca las teclas del teclado dibujado en su tarjeta, como si fuera el teclado real.

Con piezas reales: sube el programa.

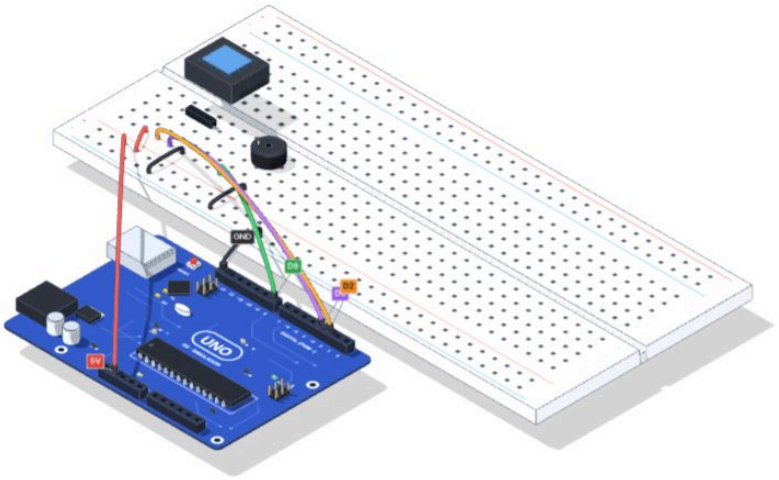
🏆 **Reto:** Esconde la clave en un acertijo para tus compañeros.

PROYECTO 148 · SEGURIDAD Y CONTROL DE ACCESO

Lector de huellas con contador de usos

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Lleva la cuenta de cuántas veces ha usado el lector cada huella registrada.

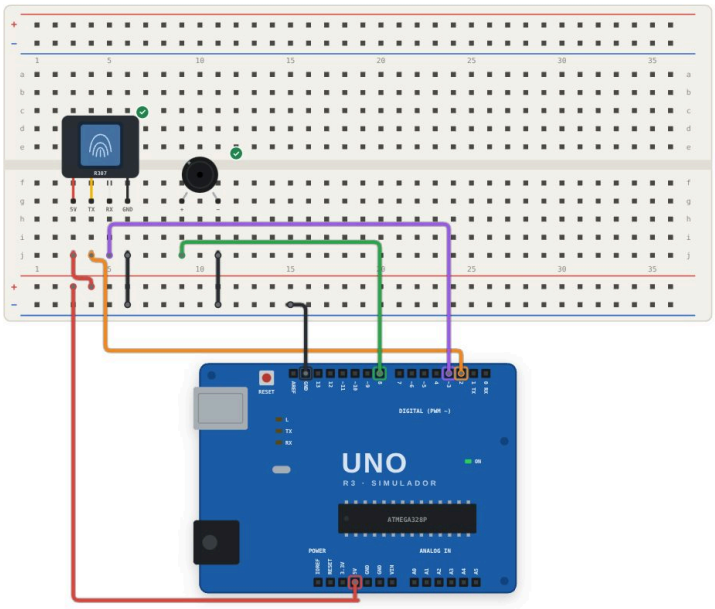


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector de huellas (Lector de huellas R307)
- 1 × Pitido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Adafruit Fingerprint Sensor Library



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector de huellas (Lector de huellas R307)

- Cable rojo (VCC) → riel **+** (5V)
- Cable amarillo (TX del lector) → pin **D2**
- Cable blanco (RX del lector) → pin **~D3**
- Cable negro (GND) → riel **–** (GND)

3 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Para el proyecto real

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 140 mA.
- Consumo estimado: **~140 mA** funcionando, con picos de **~200 mA**.

2 El programa

```
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);   // TX del lector en D2, RX en D3
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
// Devuelve el número de la huella reconocida, 0 si no la conoce o -1 si no hay dedo
int leerHuella() {
  if (finger.getImage() != FINGERPRINT_OK) return -1;
  if (finger.image2Tz() != FINGERPRINT_OK) return -1;
  if (finger.fingerFastSearch() != FINGERPRINT_OK) return 0;
  return finger.fingerID;
}
int usos[] = {0, 0, 0, 0};

void setup() {
  Serial.begin(9600);
  finger.begin(57600);
}

void loop() {
  int id = leerHuella();
  if (id > 0 && id <= 3) {
    usos[id]++;
    tone(8, 1500, 100);
    Serial.print("Huella #"); Serial.print(id);
    Serial.print(" lleva "); Serial.print(usos[id]); Serial.println(" usos");
    delay(1500);
  } else if (id == 0) {
    Serial.println("Huella no registrada");
    delay(1500);
  }
}
```

💡 ¿Cómo funciona?

El arreglo usos tiene una casilla por huella. El número de la huella sirve para saber qué casilla aumentar.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Huella 1», «Huella 2», «Huella 3» (registradas) o «Desconocida» en la tarjeta del lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Las huellas se guardan antes en el lector con el ejemplo «enroll» de la librería Adafruit Fingerprint (Archivo → Ejemplos). En el simulador ya están guardadas las huellas 1, 2 y 3.

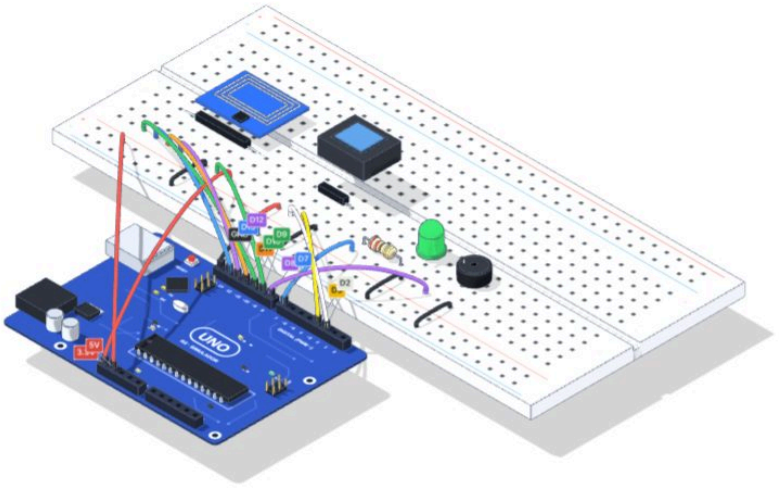
🏆 **Reto:** Muestra al final del día quién usó más el lector.

PROYECTO 149 · SEGURIDAD Y CONTROL DE ACCESO

Doble factor: tarjeta y huella

★★★ Avanzado · Tiempo: 45 a 90 minutos

Para abrir se necesitan dos cosas: primero la tarjeta autorizada y luego una huella registrada.

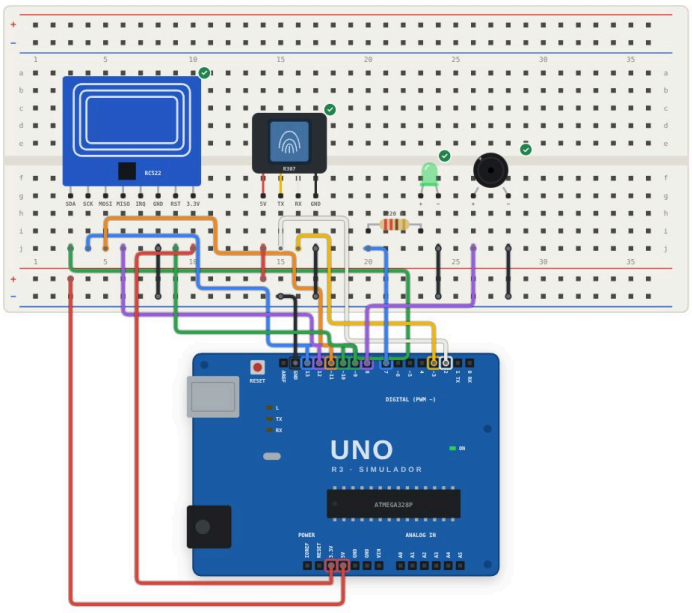


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lector RFID (Lector RFID RC522)
- 1 × Lector de huellas (Lector de huellas R307)
- 1 × Abierto (LED)
- 1 × Pitido (Zumbador)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: MFRC522 (de GithubCommunity / miguelbalboa); Adafruit Fingerprint Sensor Library



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lector RFID (Lector RFID RC522)

- SDA (SS) → pin **~D10**
- SCK → pin **D13**
- MOSI → pin **~D11**
- MISO → pin **D12**
- GND → riel **–** (GND)
- RST → pin **~D9**
- 3.3V → pin **3.3V** del Arduino (no al 5V)

3 Lector de huellas (Lector de huellas R307)

- Cable rojo (VCC) → riel **+** (5V)
- Cable amarillo (TX del lector) → pin **D2**
- Cable blanco (RX del lector) → pin **~D3**
- Cable negro (GND) → riel **–** (GND)

4 Abierto (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D7**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

6 Para el proyecto real

- Las huellas se registran antes con el ejemplo «enroll» de Adafruit Fingerprint. En el simulador ya están guardadas las huellas 1, 2 y 3.
- Para usar tu tarjeta, mira su código en el monitor serie y cámbialo en el programa.

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 181 mA.
- Consumo estimado: **~181 mA** funcionando, con picos de **~241 mA**.

2 El programa

```
#include <SPI.h>
#include <MFRC522.h>
MFRC522 rfid(10, 9);   // SDA en D10 y RST en D9
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h>
SoftwareSerial mySerial(2, 3);   // TX del lector en D2, RX en D3
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);
// Devuelve 1 si la tarjeta leída es la autorizada (blanca: 4A 3F 21 9C), 0 si es otra y -1 si no hay tarjeta
int leerTarjeta() {
  if (!rfid.PICC_IsNewCardPresent()) return -1;
  if (!rfid.PICC_ReadCardSerial()) return -1;
  Serial.print("Tarjeta:");
  for (byte i = 0; i < rfid.uid.size; i++) {
    Serial.print(" ");
    Serial.print(rfid.uid.uidByte[i], HEX);
  }
  Serial.println();
  bool ok = rfid.uid.uidByte[0] == 0x4A && rfid.uid.uidByte[1] == 0x3F && rfid.uid.uidByte[2] == 0x21 && rfid.uid.uidByte[3] == 0x9C;
  rfid.PICC_HaltA();
  if (ok) return 1;
  return 0;
}
```

```ino
// Devuelve el número de la huella reconocida, 0 si no la conoce o -1 si no hay dedo
int leerHuella() {
  if (finger.getImage() != FINGERPRINT_OK) return -1;
  if (finger.image2Tz() != FINGERPRINT_OK) return -1;
  if (finger.fingerFastSearch() != FINGERPRINT_OK) return 0;
  return finger.fingerID;
}
bool tarjetaOk = false;
unsigned long cuando = 0;

void setup() {
  Serial.begin(9600);
  SPI.begin(); rfid.PCD_Init();
  finger.begin(57600);
  pinMode(7, OUTPUT);
  Serial.println("Paso 1: acerque la tarjeta");
}

void loop() {
  if (!tarjetaOk) {
    int r = leerTarjeta();
    if (r == 1) { tarjetaOk = true; cuando = millis(); tone(8, 1500, 100); Serial.println("Paso 2: ponga su dedo (10 s)"); }
    if (r == 0) tone(8, 300, 400);
    return;
  }
  int id = leerHuella();
  if (id > 0) {
    Serial.println("Acceso concedido");
    digitalWrite(7, HIGH); delay(3000); digitalWrite(7, LOW);
    tarjetaOk = false;
  } else if (id == 0 || millis() - cuando > 10000) {
    Serial.println("Acceso negado: empiece de nuevo");
    tone(8, 300, 600);
    tarjetaOk = false;
  }
}
```

💡 ¿Cómo funciona?

Los bancos usan «doble factor»: algo que tienes (la tarjeta) y algo que eres (tu huella). Después de la tarjeta hay 10 segundos para poner el dedo; si no, todo vuelve a empezar.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca «Tarjeta blanca» (4A 3F 21 9C), «Llavero azul» o «Desconocida» para acercarla al lector.
- Toca «Huella 1», «Huella 2», «Huella 3» (registradas) o «Desconocida» en la tarjeta del lector.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ **Ten en cuenta**

- Las huellas se registran antes con el ejemplo «enroll» de Adafruit Fingerprint. En el simulador ya están guardadas las huellas 1, 2 y 3.
- Para usar tu tarjeta, mira su código en el monitor serie y cámbialo en el programa.

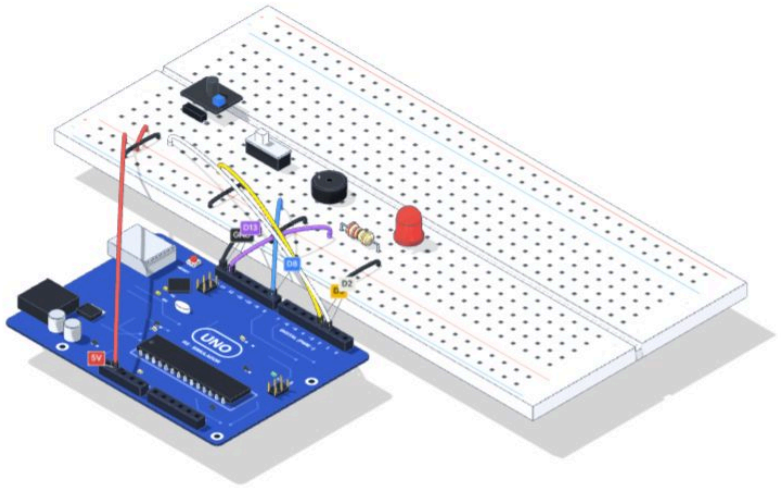
🏆 **Reto:** Agrega un teclado para una clave como tercer factor.

PROYECTO 150 · SEGURIDAD Y CONTROL DE ACCESO

Alarma antirrobo para bicicleta

★★☆ Intermedio · Tiempo: 30 a 45 minutos

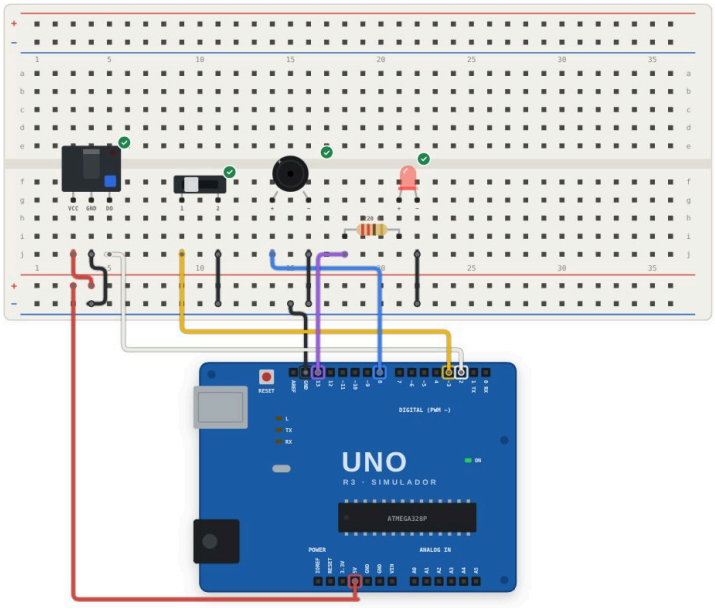
Si alguien mueve o inclina la bicicleta estando la alarma armada, suena la sirena.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de inclinación
- 1 × Armar alarma (Interruptor)
- 1 × Sirena (Zumbador)
- 1 × Armada (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de inclinación (Sensor de inclinación)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Armar alarma (Interruptor)

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**.

4 Sirena (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Armada (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 100 mA.
- Consumo estimado: **~100 mA** funcionando, con picos de **~100 mA**.

2 El programa

```
1  void setup() {
2    pinMode(2, INPUT); pinMode(3, INPUT_PULLUP); pinMode(13, OUTPUT);
3  }
4
5  void loop() {
6    bool armada = digitalRead(3) == LOW;
7    digitalWrite(13, armada);
8    if (armada && digitalRead(2) == HIGH) {
9      for (int i = 0; i < 15; i++) { tone(8, 2000); delay(120); tone(8, 1300); delay(120); }
10     noTone(8);
11   }
12 }
```

💡 ¿Cómo funciona?

El interruptor arma o desarma la alarma. Solo cuando está armada, un cambio en el sensor de inclinación dispara la sirena.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Está inclinado».
- Cambia la posición del interruptor en su tarjeta del panel «Componentes».

Con piezas reales: sube el programa.

🏆 **Reto:** Usa una tarjeta RFID para armar y desarmar.

CAPÍTULO 11

Hogar y ambiente

Domótica: gas, fuego, agua, plantas, ventiladores y luces automáticas para una casa más segura.

EN ESTE CAPÍTULO · 22 PROYECTOS

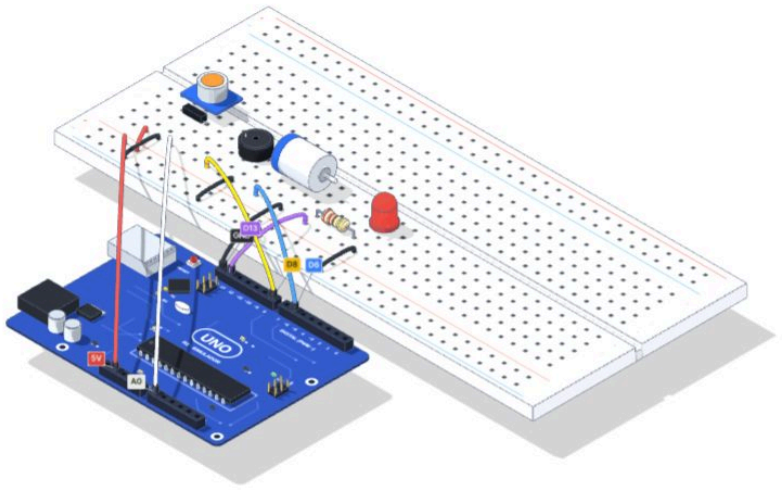
151 Detector de fuga de gas en la cocina ★★☆
152 Detector de incendio con llama ★☆☆
153 Bomba contra incendios automática ★★★
154 Luz de pasillo con sensor de movimiento ★☆☆
155 Luz nocturna inteligente ★★☆
156 Aplausos que encienden la luz ★★☆
157 Lámpara táctil ★☆☆
158 Alarma de inundación ★☆☆
159 Llenado automático de tanque con bomba ★★★
160 Tendedero que avisa la lluvia ★★☆
161 Ventilador de dos velocidades ★★☆
162 Calentador con relé y termostato ★★★
163 Monitor de humedad para el baño ★★☆
164 Recordatorio para regar la planta ★☆☆
165 Cortina que abre con el sol ★★☆
166 Timbre con luz para personas sordas ★☆☆
167 Pastillero con recordatorio ★★☆
168 Medidor de calidad del aire con semáforo ★★☆
169 Medidor de consumo de un aparato ★★★
170 Casa inteligente con menú en LCD ★★★
171 Indicador de gas en LCD ★★☆
172 Nivel de tanque en LCD ★★☆

PROYECTO 151 · HOGAR Y AMBIENTE

Detector de fuga de gas en la cocina

★★☆ Intermedio · Tiempo: 30 a 45 minutos

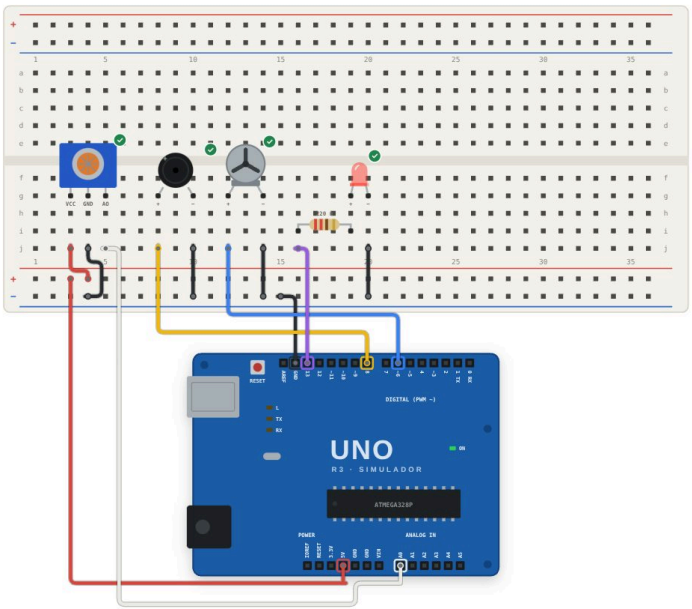
El sensor MQ-2 vigila el aire de la cocina: si sube el gas, suena la alarma y se enciende el extractor.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de gas MQ-2
- 1 × Alarma (Zumbador)
- 1 × Extractor (Motor DC)
- 1 × Peligro (LED)
- 1 × Resistencia 220 Ω
- Transistor TIP120 (o 2N2222)
- Diodo 1N4007
- Resistencia 1 kΩ
- Fuente 6–12 V para los motores
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de gas MQ-2 (Sensor de gas MQ-2)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- AO (señal) → pin **A0**

3 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Extractor (Motor DC)

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

5 Peligro (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 Para el proyecto real

- El MQ-2 necesita calentarse 1 o 2 minutos antes de dar lecturas confiables.

7 Energía y controladores

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motor → necesita transistor:** un pin da solo 20–40 mA y el motor pide 150–250 mA (hasta 1 A al arrancar). Conexión: pin → resistencia de 1 kΩ → base del **TIP120**; motor entre el + de la fuente y el colector; emisor a GND. Pon un **diodo 1N4007** en paralelo con el motor (la raya hacia el +) para frenar los picos de voltaje que dañan el transistor.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al + del motor (el – del motor va al colector del transistor). Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~445 mA** funcionando, con picos de **~1245 mA**.

2 El programa

```
const int GAS = A0, LIMITE = 400;

void setup() {
  Serial.begin(9600);
  pinMode(8, OUTPUT); pinMode(6, OUTPUT); pinMode(13, OUTPUT);
}

void loop() {
  int nivel = analogRead(GAS);
  Serial.print("Gas: "); Serial.println(nivel);
  if (nivel > LIMITE) {
    digitalWrite(13, HIGH); digitalWrite(6, HIGH);
    tone(8, 2000, 200);
  } else {
    digitalWrite(13, LOW); digitalWrite(6, LOW);
  }
  delay(300);
```

```
18 }
```

💡 ¿Cómo funciona?

El MQ-2 cambia su resistencia cuando hay gas o humo, y el Arduino lo lee como un número de 0 a 1023. Por encima del límite se enciende la alarma y el extractor saca el aire.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Sube el deslizador de gas o humo.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- El MQ-2 necesita calentarse 1 o 2 minutos antes de dar lecturas confiables.

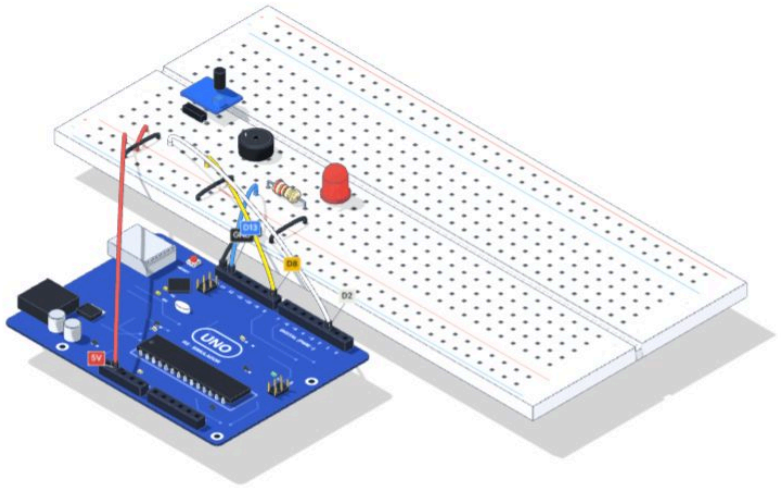
🏆 **Reto:** Haz que la alarma siga sonando 10 segundos aunque el gas ya haya bajado.

PROYECTO 152 · HOGAR Y AMBIENTE

Detector de incendio con llama

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

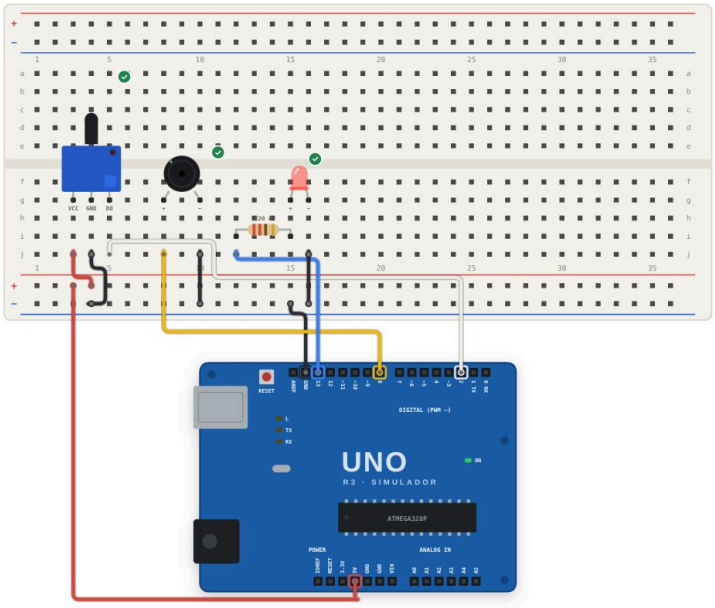
Si el sensor ve una llama, suena la alarma de incendio.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de llama
- 1 × Alarma (Zumbador)
- 1 × Fuego (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de llama (Sensor de llama)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Fuego (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 100 mA.
- Consumo estimado: **~100 mA** funcionando, con picos de **~100 mA**.

2 El programa

```
1  void setup() {
2    Serial.begin(9600);
3    pinMode(2, INPUT); pinMode(13, OUTPUT);
4  }
5
6  void loop() {
7    if (digitalRead(2) == LOW) {          // LOW = hay llama
8      Serial.println("¡FUEGO!");
9      digitalWrite(13, HIGH);
10     tone(8, 2500, 150); delay(200);
11     tone(8, 1500, 150); delay(200);
12   } else {
13     digitalWrite(13, LOW);
14   }
15 }
```

💡 ¿Cómo funciona?

El sensor de llama ve la luz infrarroja del fuego. Su salida DO baja a LOW cuando detecta una llama, por eso se pregunta por LOW.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay fuego cerca».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

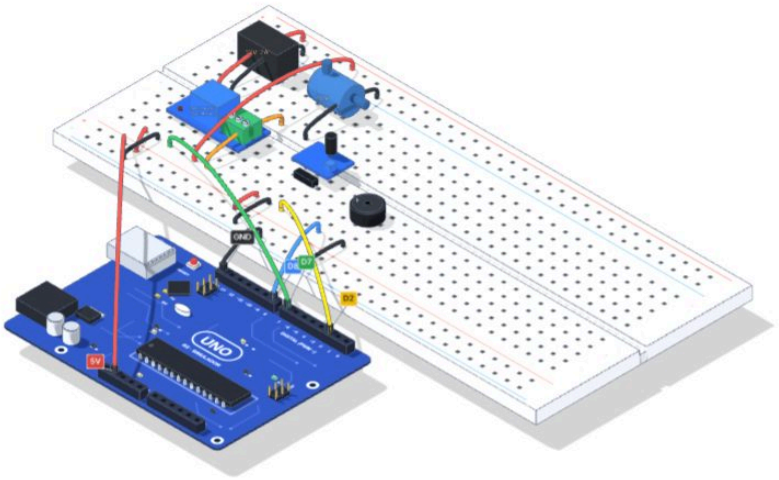
🏆 **Reto:** Agrega una bomba de agua que se encienda con el fuego.

PROYECTO 153 · HOGAR Y AMBIENTE

Bomba contra incendios automática

★★★ Avanzado · Tiempo: 45 a 90 minutos

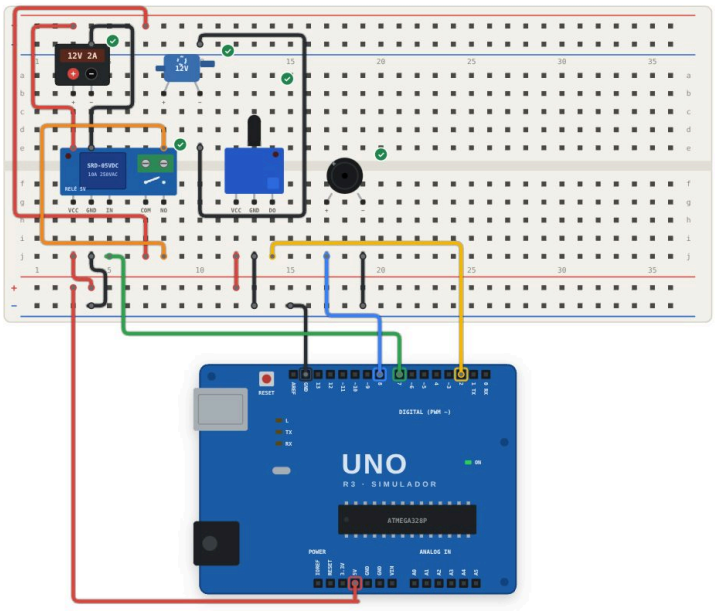
Un rociador automático: si hay fuego, el relé enciende la bomba de 12 V hasta que la llama se apaga.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de llama
- 1 × Relé de la bomba (Módulo relé 5 V)
- 1 × Bomba de agua 12 V (Bomba / motor 12 V)
- 1 × Fuente 12 V
- 1 × Alarma (Zumbador)
- Diodo 1N4007 en paralelo con la bomba (recomendado)
- Manguera delgada y recipiente con agua
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de llama (Sensor de llama)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Relé de la bomba (Módulo relé 5 V)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- IN (señal) → pin **D7**
- COM (común) → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- NO (normalmente abierto) → **+ de la carga** (bomba, chapa o motor de 12 V)

4 Bomba de agua 12 V (Bomba / motor 12 V)

- Cable + (rojo) de la bomba → borne **NO** del relé
- Cable – (negro) de la bomba → **– de 12 V** (riel – de arriba)

5 Fuente 12 V (Fuente 12 V)

- Borne +12V → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- Borne – → **– de 12 V** (riel – de arriba)

6 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

7 Para el proyecto real

- La bomba va con su propia fuente de 12 V a través del relé; nunca la conectes al Arduino.

8 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 155 mA.
- **Bomba o motor de 12 V → relé + fuente de 12 V:** el Arduino no puede darle 12 V ni la corriente que pide (≈1 A). El pin solo activa el **módulo relé de 5 V** (IN), y el relé hace de interruptor: **+12 V → COM**, **NO → + de la bomba**, **– de la bomba → – de la fuente**.
- **Fuente de 12 V:** un adaptador de **12 V y 2 A** (o lo que diga la etiqueta de la bomba, con algo de margen). Los 12 V nunca tocan los pines ni el 5V del Arduino. Si quieres una sola fuente, puedes alimentar el Arduino por **VIN** con esos 12 V y unir los –.
- **Diodo de protección:** pon un 1N4007 en paralelo con la bomba (la raya hacia el +) para absorber el pico de voltaje al apagarla.
- **¿Unir los –?** Con relé no hace falta: el relé aísla los 12 V del Arduino. Solo únelos si alimentas el Arduino por VIN con la misma fuente.
- Consumo estimado: **~155 mA** funcionando, con picos de **~155 mA**.

2 El programa

```
void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT); pinMode(7, OUTPUT);
}

void loop() {
  if (digitalRead(2) == LOW) {
    Serial.println("Fuego: rociando agua");
    digitalWrite(7, HIGH);
    tone(8, 2000, 100);
  } else {
    digitalWrite(7, LOW);
  }
  delay(200);
}
```

💡 ¿Cómo funciona?

La bomba trabaja a 12 V, así que el Arduino la controla con un relé. Mientras el sensor siga viendo la llama, la bomba sigue encendida.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay fuego cerca».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- La bomba va con su propia fuente de 12 V a través del relé; nunca la conectes al Arduino.

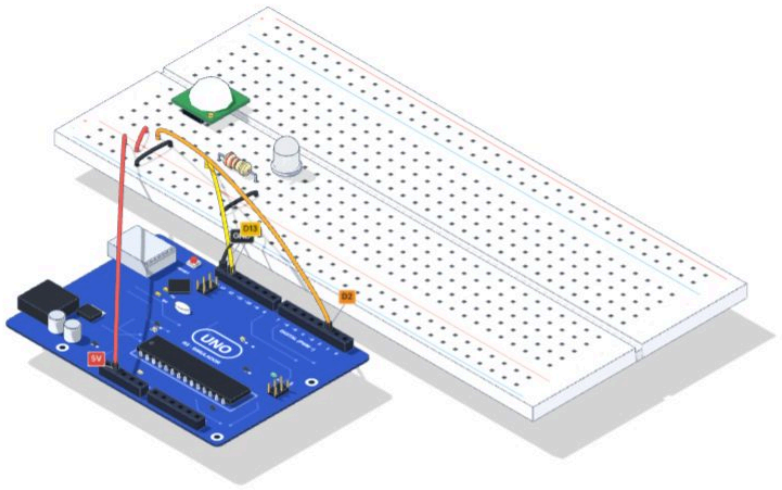
🏆 **Reto:** Deja la bomba encendida 5 segundos más después de que se apague el fuego.

PROYECTO 154 · HOGAR Y AMBIENTE

Luz de pasillo con sensor de movimiento

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

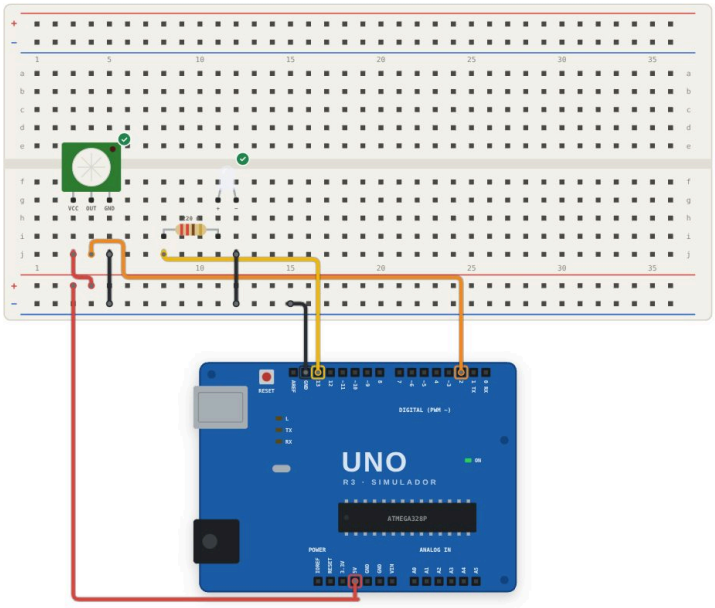
Al pasar alguien, la luz del pasillo se enciende 10 segundos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor PIR
- 1 × Luz del pasillo (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor PIR (Sensor PIR)

- VCC → riel **+** (5V)
- OUT (señal) → pin **D2**
- GND → riel **–** (GND)

3 Luz del pasillo (LED blanco)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 66 mA.
- Consumo estimado: **~66 mA** funcionando, con picos de **~66 mA**.

2 El programa

```
unsigned long ultimo = 0;

void setup() {
  pinMode(2, INPUT); pinMode(13, OUTPUT);
}

void loop() {
  if (digitalRead(2) == HIGH) ultimo = millis();
  if (millis() - ultimo < 10000 && ultimo > 0) digitalWrite(13, HIGH);
  else digitalWrite(13, LOW);
}
```

💡 ¿Cómo funciona?

Cada vez que el PIR detecta movimiento se guarda la hora en ultimo. La luz queda encendida mientras no hayan pasado 10 segundos desde el último movimiento.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Activa la casilla «hay movimiento» del sensor PIR.

Con piezas reales: sube el programa.

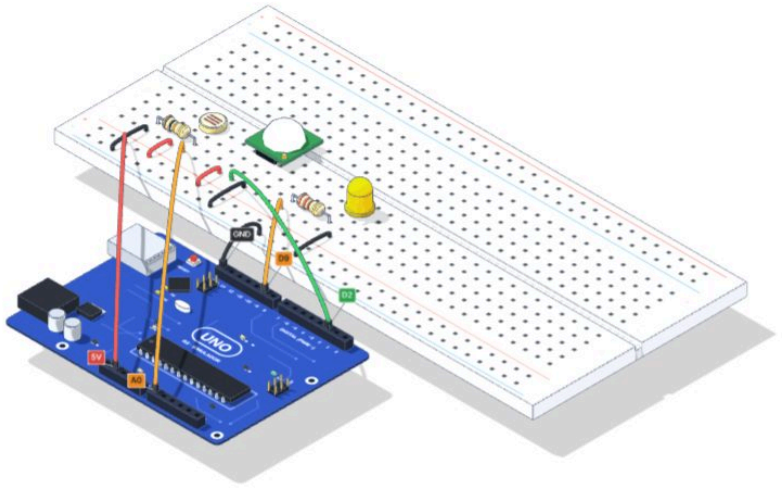
🏆 **Reto:** Haz que solo funcione de noche con una fotorresistencia.

PROYECTO 155 · HOGAR Y AMBIENTE

Luz nocturna inteligente

★★☆ Intermedio · Tiempo: 30 a 45 minutos

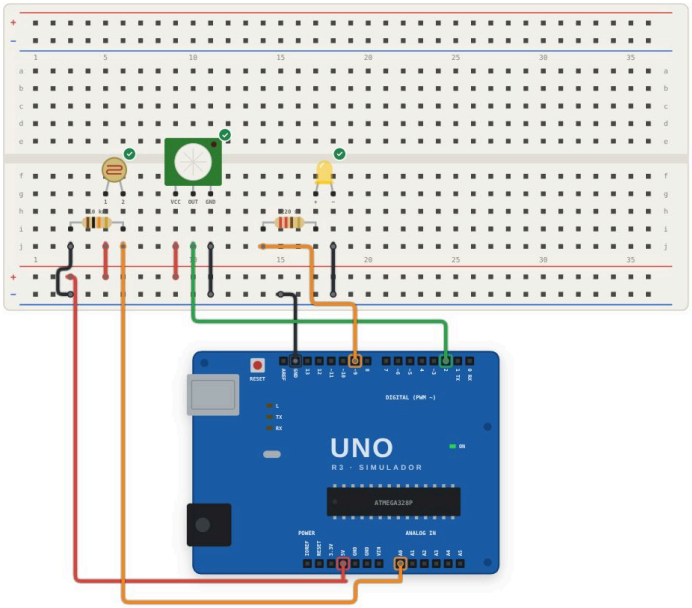
La luz se enciende solo si es de noche y hay alguien pasando: así se ahorra energía.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Fotorresistencia
- 1 × Sensor PIR
- 1 × Luz (LED)
- 1 × Resistencia 220 Ω
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Fotorresistencia (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Sensor PIR (Sensor PIR)

- VCC → riel **+** (5V)
- OUT (señal) → pin **D2**
- GND → riel **–** (GND)

4 Luz (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 67 mA.
- Consumo estimado: **~67 mA** funcionando, con picos de **~67 mA**.

2 El programa

```
1  void setup() {
2    Serial.begin(9600);
3    pinMode(2, INPUT); pinMode(9, OUTPUT);
4  }
5
6  void loop() {
7    int luz = analogRead(A0);
8    bool oscuro = luz < 300;
9    bool alguien = digitalRead(2) == HIGH;
10   if (oscuro && alguien) { digitalWrite(9, HIGH); delay(5000); }
11   else digitalWrite(9, LOW);
12 }
```

💡 ¿Cómo funciona?

El operador && exige que las dos condiciones se cumplan a la vez: oscuridad y movimiento. Si falta cualquiera, la luz queda apagada.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).
- Activa la casilla «hay movimiento» del sensor PIR.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

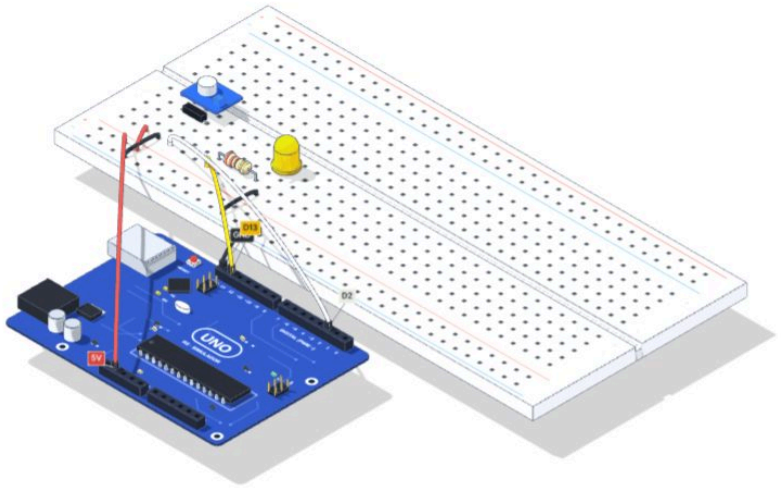
🏆 **Reto:** En vez de encender de golpe, usa analogWrite para que la luz suba despacio.

PROYECTO 156 · HOGAR Y AMBIENTE

Aplausos que encienden la luz

★★☆ Intermedio · Tiempo: 30 a 45 minutos

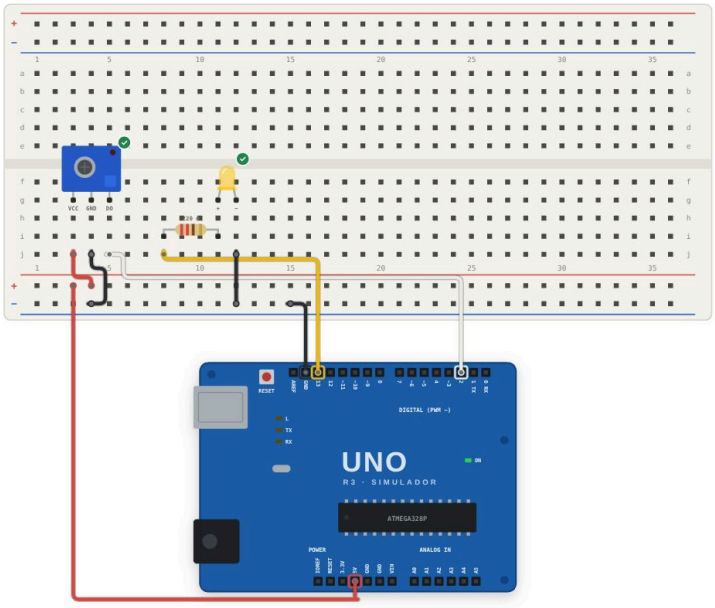
Un aplauso enciende la luz y otro la apaga.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de sonido
- 1 × Lámpara (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de sonido (Sensor de sonido)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Lámpara (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 70 mA.
- Consumo estimado: **~70 mA** funcionando, con picos de **~70 mA**.

2 El programa

```
1  bool encendida = false;
2
3  void setup() {
4    pinMode(2, INPUT); pinMode(13, OUTPUT);
5  }
6
7  void loop() {
8    if (digitalRead(2) == HIGH) {
9      encendida = !encendida;
10     digitalWrite(13, encendida);
11     delay(500);                          // ignora el eco del aplauso
12   }
13 }
```

💡 ¿Cómo funciona?

El signo ! invierte el valor: si estaba encendida pasa a apagada y al revés. El delay de medio segundo evita que el mismo aplauso cuente dos veces.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay ruido o un aplauso».

Con piezas reales: sube el programa.

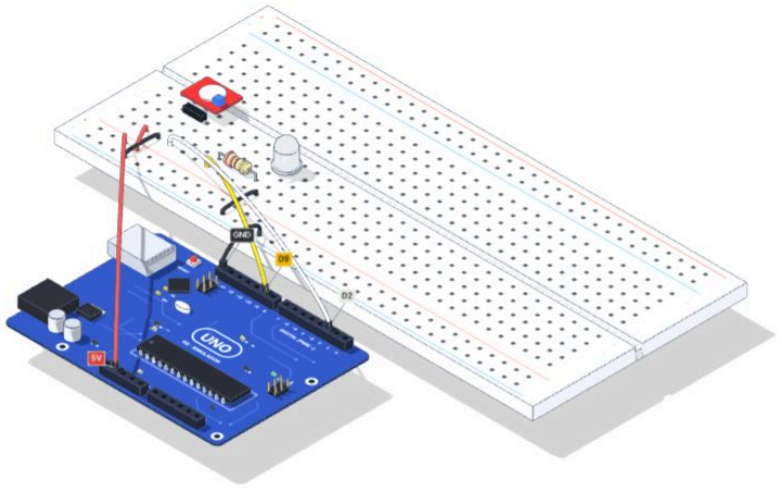
🏆 **Reto:** Haz que se necesiten dos aplausos seguidos para cambiar.

PROYECTO 157 · HOGAR Y AMBIENTE

Lámpara táctil

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

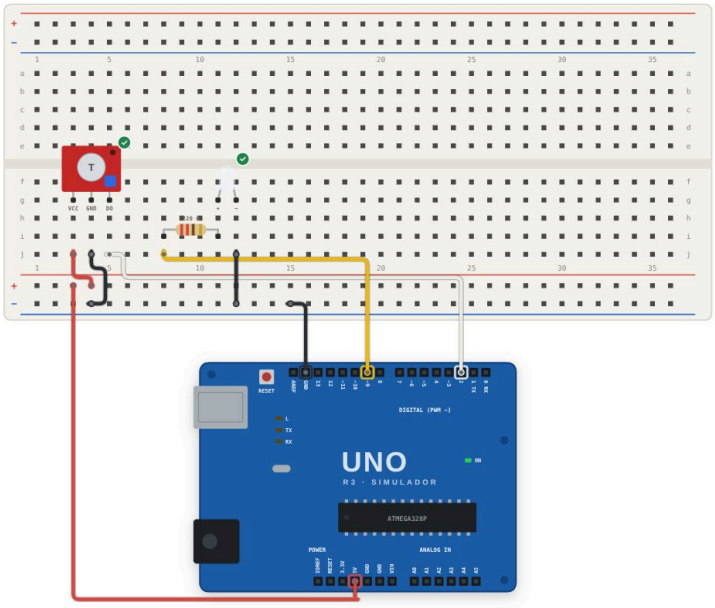
Toca el sensor para cambiar la lámpara entre apagada, baja, media y alta.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor táctil (Sensor táctil TTP223)
- 1 × Lámpara (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor táctil (Sensor táctil TTP223)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Lámpara (LED blanco)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 70 mA.
- Consumo estimado: **~70 mA** funcionando, con picos de **~70 mA**.

2 El programa

```
1  int nivel = 0;
2  int brillo[] = {0, 60, 150, 255};
3
4  void setup() {
5    pinMode(2, INPUT); pinMode(9, OUTPUT);
6  }
7
8  void loop() {
9    if (digitalRead(2) == HIGH) {
10     nivel = (nivel + 1) % 4;
11     analogWrite(9, brillo[nivel]);
12     delay(400);
13   }
14 }
```

💡 ¿Cómo funciona?

El operador % (residuo) hace que el nivel vuelva a 0 después del 3. Con analogWrite en un pin PWM (~9) se logran brillos intermedios.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Lo están tocando».

Con piezas reales: sube el programa.

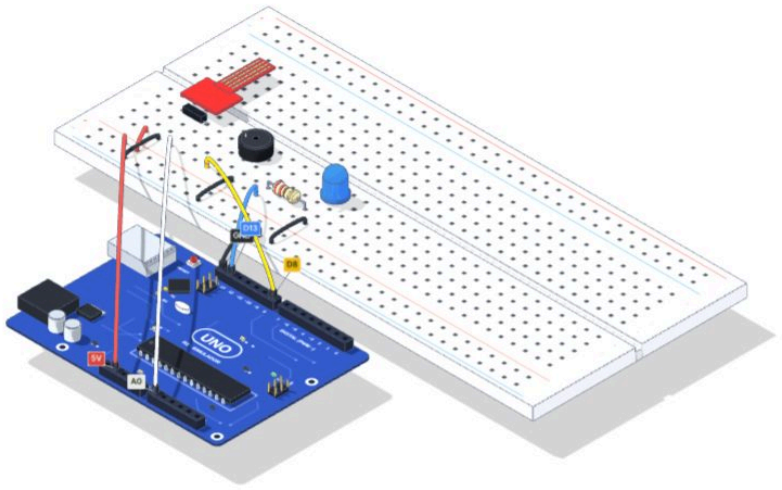
🏆 **Reto:** Agrega un quinto nivel que haga parpadear la lámpara.

PROYECTO 158 · HOGAR Y AMBIENTE

Alarma de inundación

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

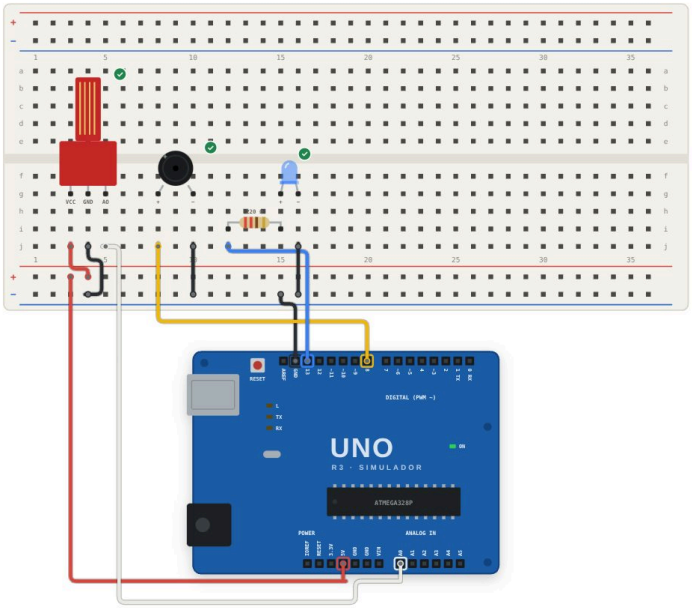
Si el agua sube en el piso o el sótano, suena una alarma.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de nivel de agua
- 1 × Alarma (Zumbador)
- 1 × Agua (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de nivel de agua (Sensor de nivel de agua)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- AO (señal) → pin **A0**

3 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Agua (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 100 mA.
- Consumo estimado: **~100 mA** funcionando, con picos de **~100 mA**.

2 El programa

```
1  void setup() {
2    Serial.begin(9600);
3    pinMode(13, OUTPUT);
4  }
5
6  void loop() {
7    int agua = analogRead(A0);
8    Serial.print("Agua: "); Serial.println(agua);
9    if (agua > 200) { digitalWrite(13, HIGH); tone(8, 1800, 300); }
10   else digitalWrite(13, LOW);
11   delay(500);
12 }
```

💡 ¿Cómo funciona?

El sensor de nivel tiene pistas metálicas: cuanto más mojadas, más corriente pasa y mayor es el número leído.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del nivel de agua.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

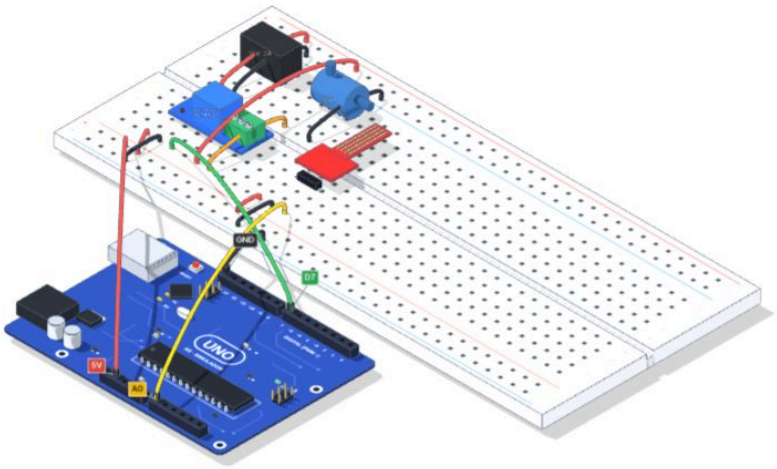
🏆 **Reto:** Haz que el pitido sea más rápido cuanto más agua haya.

PROYECTO 159 · HOGAR Y AMBIENTE

Llenado automático de tanque con bomba

★★★ Avanzado · Tiempo: 45 a 90 minutos

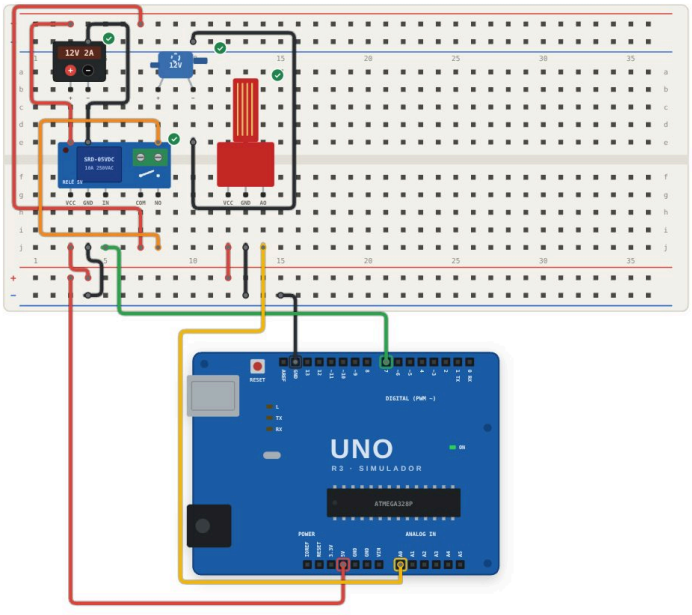
La bomba llena el tanque cuando el agua baja del 20 % y se apaga al llegar al 80 %.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de nivel (Sensor de nivel de agua)
- 1 × Relé de la bomba (Módulo relé 5 V)
- 1 × Bomba de agua 12 V (Bomba / motor 12 V)
- 1 × Fuente 12 V
- Diodo 1N4007 en paralelo con la bomba (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de nivel (Sensor de nivel de agua)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- AO (señal) → pin **A0**

3 Relé de la bomba (Módulo relé 5 V)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- IN (señal) → pin **D7**
- COM (común) → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- NO (normalmente abierto) → **+ de la carga** (bomba, chapa o motor de 12 V)

4 Bomba de agua 12 V (Bomba / motor 12 V)

- Cable + (rojo) de la bomba → borne **NO** del relé
- Cable – (negro) de la bomba → **– de 12 V** (riel – de arriba)

5 Fuente 12 V (Fuente 12 V)

- Borne +12V → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- Borne – → **– de 12 V** (riel – de arriba)

6 Para el proyecto real

- La bomba usa la fuente de 12 V a través del relé.

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 125 mA.
- **Bomba o motor de 12 V → relé + fuente de 12 V:** el Arduino no puede darle 12 V ni la corriente que pide (≈1 A). El pin solo activa el **módulo relé de 5 V** (IN), y el relé hace de interruptor: **+12 V → COM**, **NO → + de la bomba**, **– de la bomba → – de la fuente**.
- **Fuente de 12 V:** un adaptador de **12 V y 2 A** (o lo que diga la etiqueta de la bomba, con algo de margen). Los 12 V nunca tocan los pines ni el 5V del Arduino. Si quieres una sola fuente, puedes alimentar el Arduino por **VIN** con esos 12 V y unir los –.
- **Diodo de protección:** pon un 1N4007 en paralelo con la bomba (la raya hacia el +) para absorber el pico de voltaje al apagarla.
- **¿Unir los –?** Con relé no hace falta: el relé aísla los 12 V del Arduino. Solo únelos si alimentas el Arduino por VIN con la misma fuente.
- Consumo estimado: **~125 mA** funcionando, con picos de **~125 mA**.

2 El programa

```
bool llenando = false;

void setup() {
  Serial.begin(9600);
  pinMode(7, OUTPUT);
}

void loop() {
  int porc = map(analogRead(A0), 0, 650, 0, 100);
  if (porc < 20) llenando = true;
  if (porc > 80) llenando = false;
  digitalWrite(7, llenando);
  Serial.print("Tanque: "); Serial.print(porc); Serial.println(llenando ? "%  llenando" : "%");
  delay(500);
}
```

💡 ¿Cómo funciona?

Hay dos límites distintos (20 % y 80 %). Así la bomba no se prende y apaga a cada rato, sino que llena el tanque completo de una vez. Esto se llama histéresis.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del nivel de agua.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- La bomba usa la fuente de 12 V a través del relé.

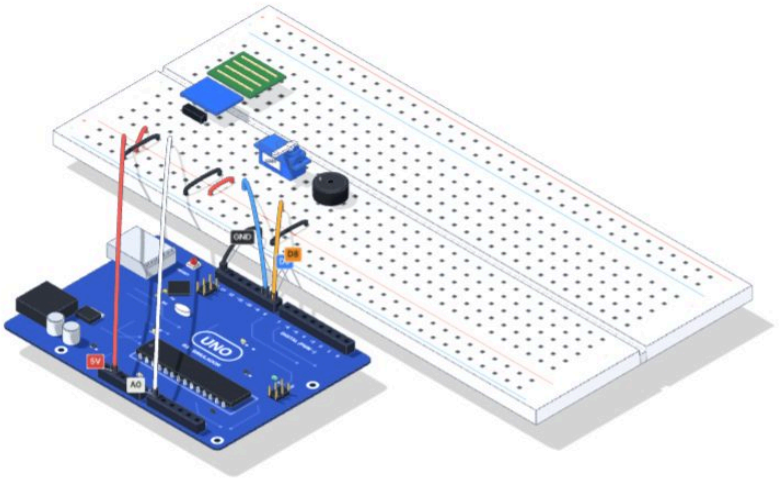
🏆 **Reto:** Muestra el nivel del tanque en una LCD.

PROYECTO 160 · HOGAR Y AMBIENTE

Tendedero que avisa la lluvia

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Si empieza a llover, suena un aviso para recoger la ropa y un servo cubre el tendedero.

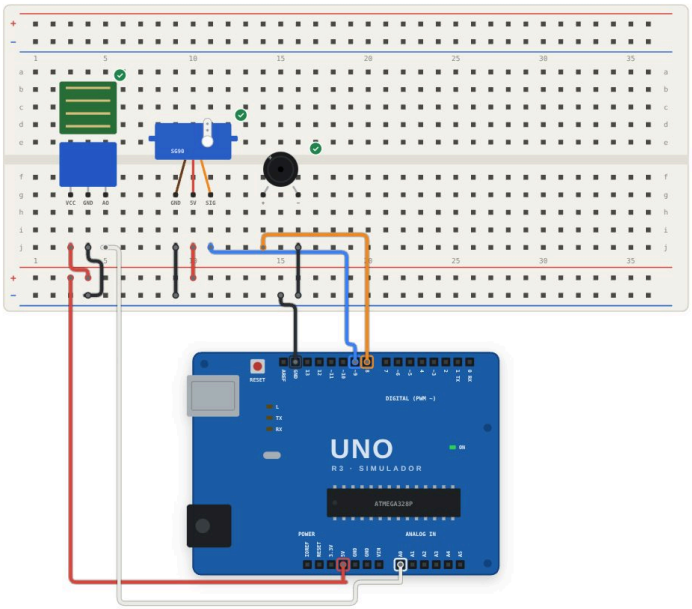


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de lluvia
- 1 × Techo (Servo SG90)
- 1 × Aviso (Zumbador)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de lluvia (Sensor de lluvia)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- AO (señal) → pin **A0**

3 Techo (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Aviso (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 235 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~235 mA** funcionando, con picos de **~735 mA**.

2 El programa

```ino
#include <Servo.h>
Servo techo;

void setup() {
  Serial.begin(9600);
  techo.attach(9); techo.write(0);
}

void loop() {
  int seco = analogRead(A0);              // 1023 seco, baja con agua
  if (seco < 600) {
    techo.write(120);
    tone(8, 1000, 200);
    Serial.println("¡Está lloviendo! Techo cerrado");
  } else {
    techo.write(0);
  }
  delay(1000);
}
```

💡 ¿Cómo funciona?

El sensor de lluvia da un número alto cuando está seco. Cuando le caen gotas el número baja; por debajo de 600 el servo cierra el techo.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de agua sobre la placa.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

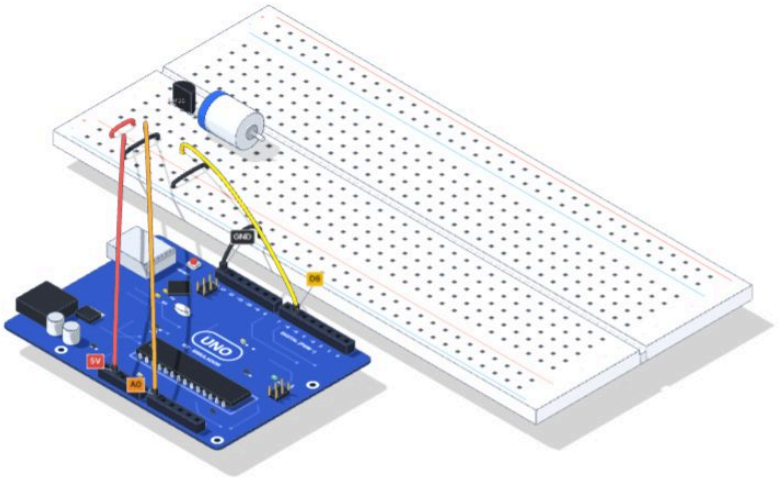
🏆 **Reto:** Avisa solo una vez, no cada segundo.

PROYECTO 161 · HOGAR Y AMBIENTE

Ventilador de dos velocidades

★★☆ Intermedio · Tiempo: 30 a 45 minutos

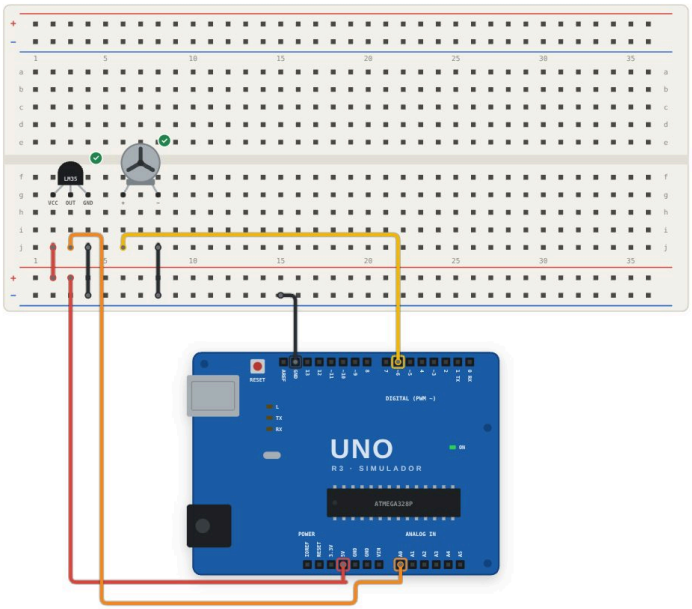
Si hace calor el ventilador gira despacio; si hace mucho calor, a toda velocidad.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor LM35
- 1 × Ventilador (Motor DC)
- Transistor TIP120 (o 2N2222)
- Diodo 1N4007
- Resistencia 1 kΩ
- Fuente 6–12 V para los motores
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Ventilador (Motor DC)

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

4 Energía y controladores

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motor → necesita transistor:** un pin da solo 20–40 mA y el motor pide 150–250 mA (hasta 1 A al arrancar). Conexión: pin → resistencia de 1 kΩ → base del **TIP120**; motor entre el + de la fuente y el colector; emisor a GND. Pon un **diodo 1N4007** en paralelo con el motor (la raya hacia el +) para frenar los picos de voltaje que dañan el transistor.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al + del motor (el – del motor va al colector del transistor). Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~251 mA** funcionando, con picos de **~1051 mA**.

2 El programa

```
1  void setup() {
2    Serial.begin(9600);
3    pinMode(6, OUTPUT);
4  }
5
6  void loop() {
7    float temp = analogRead(A0) * 500.0 / 1023.0;
8    Serial.print("Temperatura: "); Serial.println(temp, 1);
9    if (temp > 32) analogWrite(6, 255);
10   else if (temp > 27) analogWrite(6, 130);
11   else analogWrite(6, 0);
12   delay(1000);
13 }
```

💡 ¿Cómo funciona?

El LM35 entrega 10 mV por cada grado. La fórmula convierte el número leído en °C, y según la temperatura se elige la velocidad con analogWrite.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

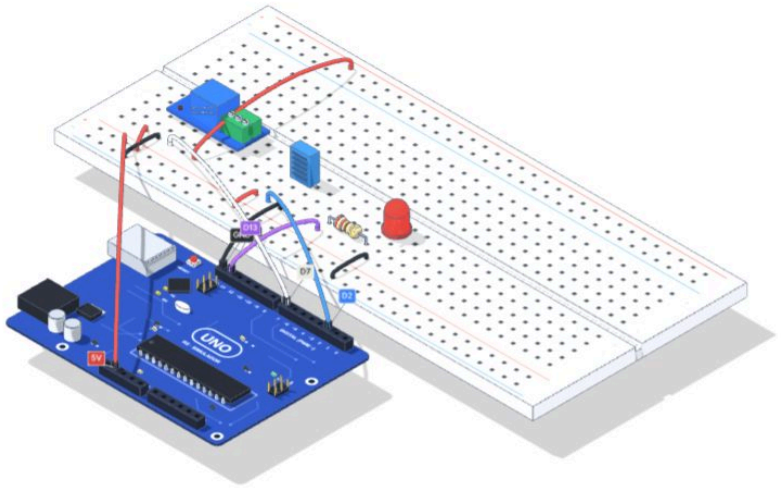
🏆 **Reto:** Agrega una tercera velocidad intermedia.

PROYECTO 162 · HOGAR Y AMBIENTE

Calentador con relé y termostato

★★★ Avanzado · Tiempo: 45 a 90 minutos

Un termostato: el relé enciende el calentador por debajo de 22 °C y lo apaga por encima de 25 °C.

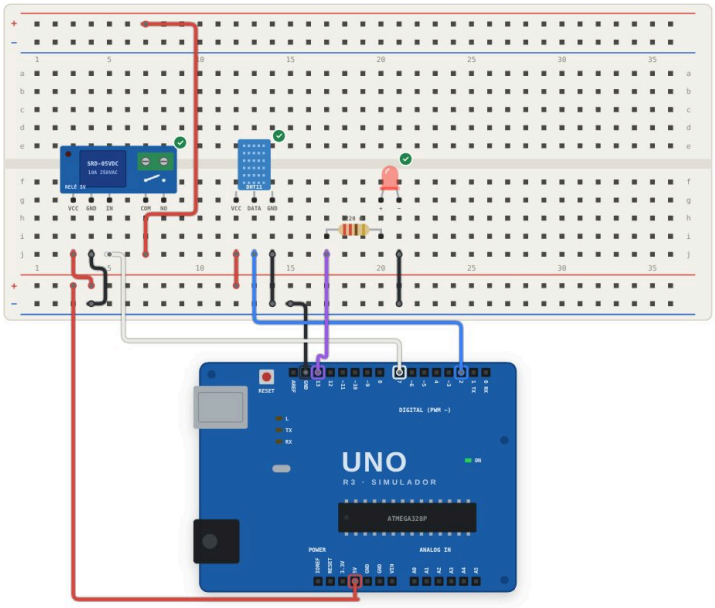


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor DHT11
- 1 × Relé del calentador (Módulo relé 5 V)
- 1 × Calentando (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: DHT sensor library (de Adafruit) y Adafruit Unified Sensor



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor DHT11 (Sensor DHT11)

- VCC → riel **+** (5V)
- DATA (señal) → pin **D2**
- GND → riel **–** (GND)

3 Relé del calentador (Módulo relé 5 V)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- IN (señal) → pin **D7**
- COM (común) → **+12 V** (riel + de arriba, separado del 5V del Arduino)
- NO (normalmente abierto) → **+ de la carga** (bomba, chapa o motor de 12 V)

4 Calentando (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Para el proyecto real

- Si vas a usar un calentador de 110 V, pide ayuda a un adulto o electricista: es corriente peligrosa.

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 137 mA.
- **Relé:** el lado de control (VCC, GND, IN) va al Arduino con 5 V; la carga va por COM y NO con su propia fuente. Algunos módulos se activan con LOW: si el tuyo funciona al revés, invierte HIGH y LOW en el código.
- Consumo estimado: **~137 mA** funcionando, con picos de **~137 mA**.

2 El programa

```ino
#include <DHT.h>
DHT dht(2, DHT11);   // DATA del sensor en D2
bool calentando = false;

void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(7, OUTPUT); pinMode(13, OUTPUT);
}

void loop() {
  float t = dht.readTemperature();
  if (isnan(t)) return;
  if (t < 22) calentando = true;
  if (t > 25) calentando = false;
  digitalWrite(7, calentando); digitalWrite(13, calentando);
  Serial.print(t, 1); Serial.println(calentando ? " °C - calentando" : " °C");
  delay(2000);
}
```

💡 ¿Cómo funciona?

Como en el tanque, se usan dos límites para que el relé no se active y desactive a cada rato. Así funcionan los termostatos de las neveras y los hornos.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura y la humedad con los deslizadores del DHT11.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Si vas a usar un calentador de 110 V, pide ayuda a un adulto o electricista: es corriente peligrosa.

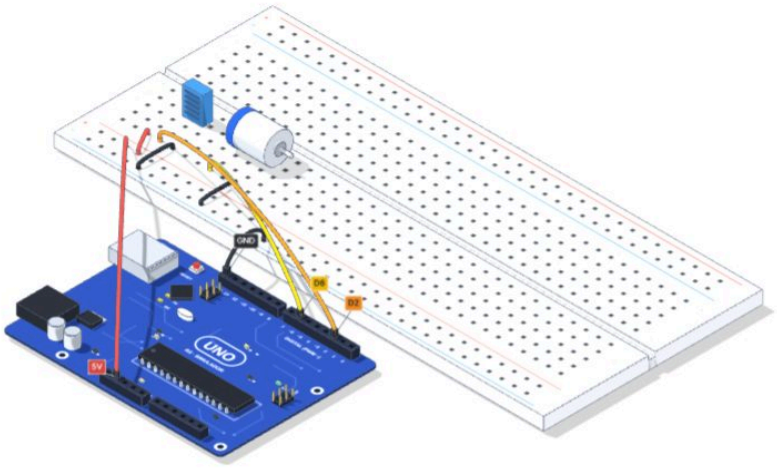
🏆 **Reto:** Muestra la temperatura deseada en una LCD y cámbiala con botones.

PROYECTO 163 · HOGAR Y AMBIENTE

Monitor de humedad para el baño

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Si la humedad del baño pasa del 70 %, se enciende el extractor para evitar el moho.

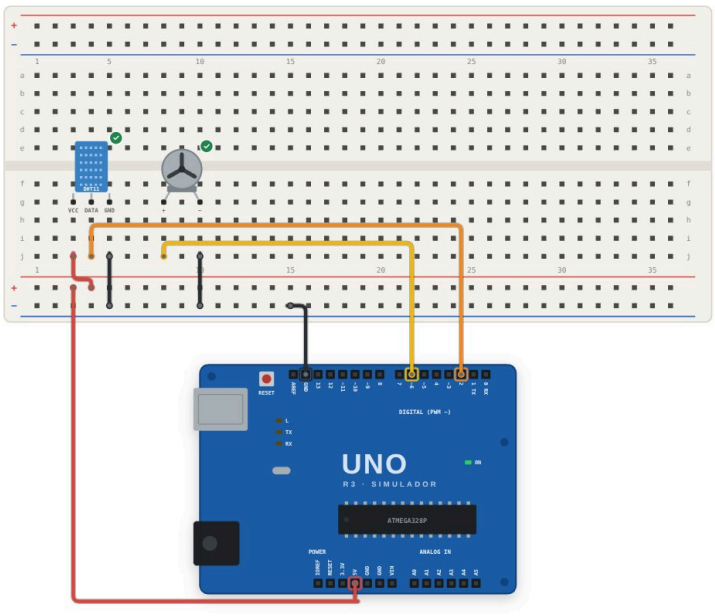


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor DHT11
- 1 × Extractor (Motor DC)
- Transistor TIP120 (o 2N2222)
- Diodo 1N4007
- Resistencia 1 kΩ
- Fuente 6–12 V para los motores
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: DHT sensor library (de Adafruit) y Adafruit Unified Sensor



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor DHT11 (Sensor DHT11)

- VCC → riel **+** (5V)
- DATA (señal) → pin **D2**
- GND → riel **–** (GND)

3 **Extractor (Motor DC)**

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

4 **Energía y controladores**

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motor → necesita transistor:** un pin da solo 20–40 mA y el motor pide 150–250 mA (hasta 1 A al arrancar). Conexión: pin → resistencia de 1 kΩ → base del **TIP120**; motor entre el + de la fuente y el colector; emisor a GND. Pon un **diodo 1N4007** en paralelo con el motor (la raya hacia el +) para frenar los picos de voltaje que dañan el transistor.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al + del motor (el – del motor va al colector del transistor). Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~252 mA** funcionando, con picos de **~1052 mA**.

2 El programa

```
#include <DHT.h>
DHT dht(2, DHT11);   // DATA del sensor en D2

void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(6, OUTPUT);
}

void loop() {
  float h = dht.readHumidity();
  if (isnan(h)) return;
  Serial.print("Humedad: "); Serial.print(h, 0); Serial.println(" %");
  digitalWrite(6, h > 70 ? HIGH : LOW);
  delay(2000);
}
```

💡 ¿Cómo funciona?

El DHT11 mide la humedad relativa del aire. La expresión «condición ? A : B» elige A si la condición es verdadera y B si no.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura y la humedad con los deslizadores del DHT11.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

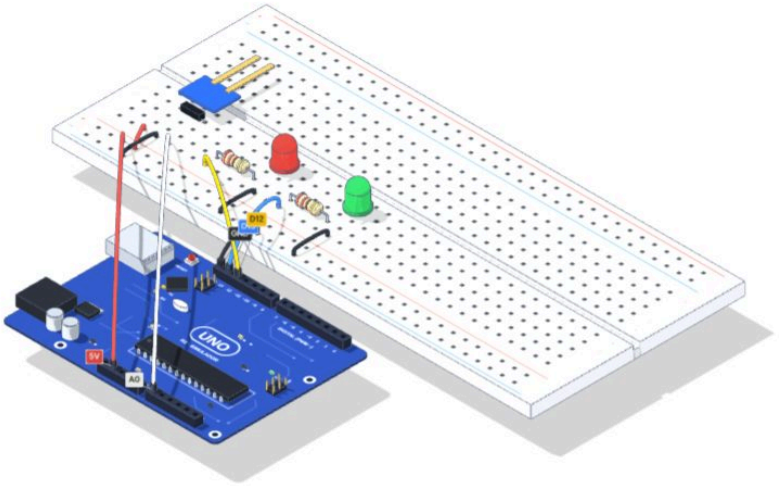
🏆 **Reto:** Agrega un LED que avise cuando el aire está muy seco (menos de 30 %).

PROYECTO 164 · HOGAR Y AMBIENTE

Recordatorio para regar la planta

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

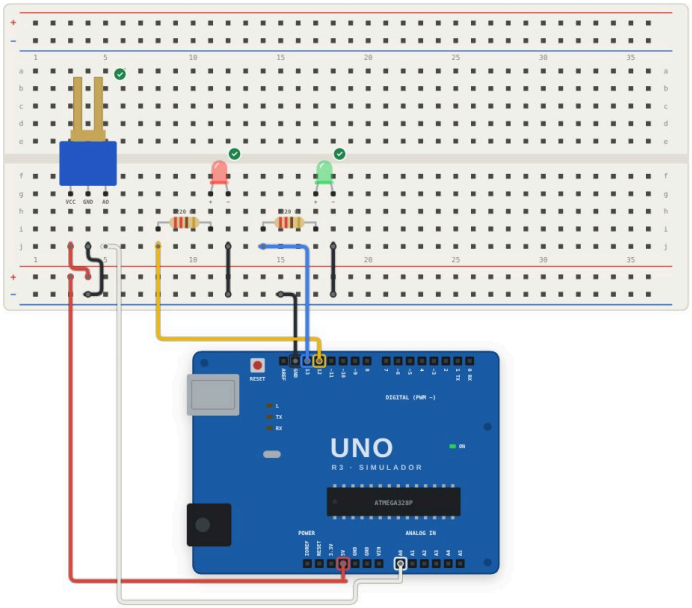
La maceta avisa con luz roja cuando la tierra está seca y con verde cuando está bien.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de humedad de suelo
- 1 × Seca (LED)
- 1 × Bien (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de humedad de suelo (Sensor de humedad de suelo)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- AO (señal) → pin **A0**

3 Seca (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Bien (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 85 mA.
- Consumo estimado: **~85 mA** funcionando, con picos de **~85 mA**.

2 El programa

```
void setup() {
  Serial.begin(9600);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
}

void loop() {
  int lectura = analogRead(A0);          // alto = seco
  int humedad = map(lectura, 1000, 350, 0, 100);
  Serial.print("Humedad de la tierra: "); Serial.print(humedad); Serial.println(" %");
  digitalWrite(12, humedad < 30);
  digitalWrite(13, humedad >= 30);
  delay(1000);
}
```

💡 ¿Cómo funciona?

El sensor de suelo da números altos cuando la tierra está seca. Con map() se convierte a un porcentaje fácil de entender.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de humedad de la tierra.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

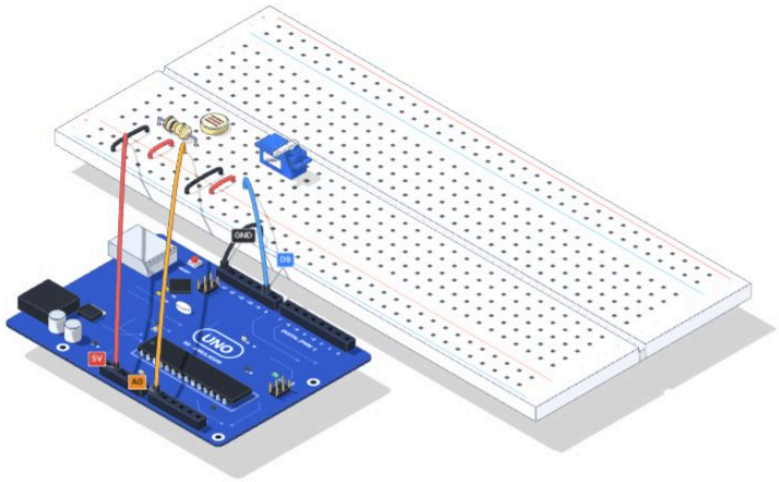
🏆 **Reto:** Haz que el LED rojo parpadee cuando la tierra esté muy seca (menos de 10 %).

PROYECTO 165 · HOGAR Y AMBIENTE

Cortina que abre con el sol

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Al amanecer el servo abre la cortina y al anochecer la cierra.

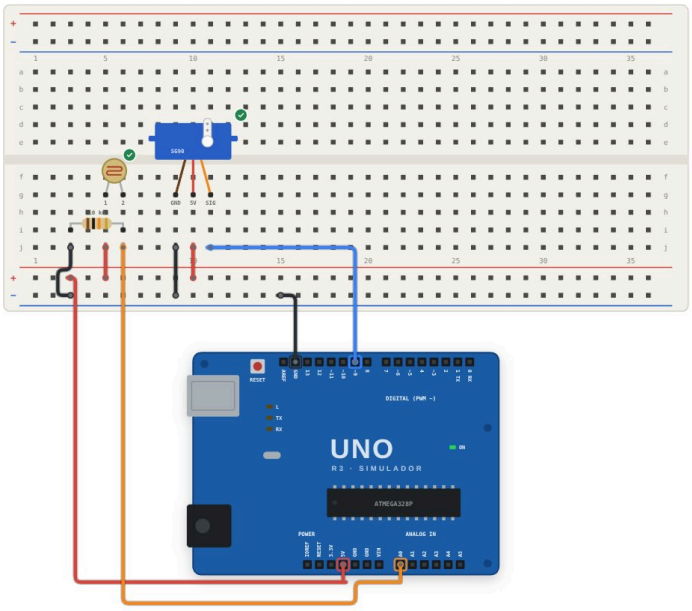


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Fotorresistencia
- 1 × Cortina (Servo SG90)
- 1 × Resistencia 10 kΩ
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Fotorresistencia (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Cortina (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 201 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~201 mA** funcionando, con picos de **~701 mA**.

2 El programa

```
#include <Servo.h>
Servo cortina;

void setup() {
  Serial.begin(9600);
  cortina.attach(9);
}

void loop() {
  int luz = analogRead(A0);
  if (luz > 600) cortina.write(180);     // de día: abierta
  if (luz < 300) cortina.write(0);       // de noche: cerrada
  Serial.println(luz);
  delay(1000);
}
```

💡 ¿Cómo funciona?

Entre 300 y 600 la cortina no se mueve: así no se abre y cierra con cada nube que pasa.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

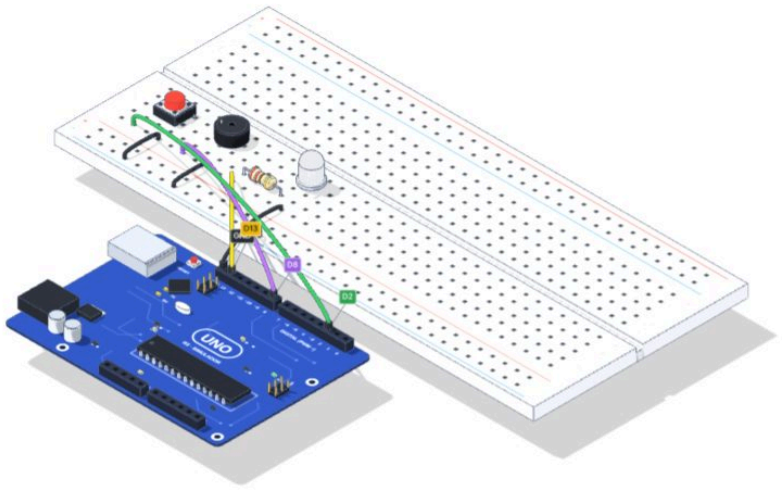
🏆 **Reto:** Agrega un botón para abrirla a mano.

PROYECTO 166 · HOGAR Y AMBIENTE

Timbre con luz para personas sordas

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

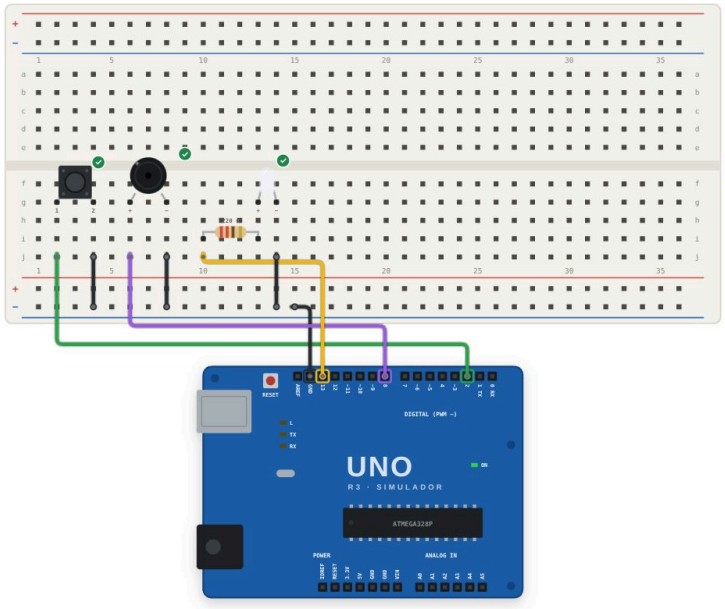
Al tocar el timbre, además del sonido parpadea una luz brillante, útil para personas con discapacidad auditiva.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Timbre (Botón)
- 1 × Sonido (Zumbador)
- 1 × Luz (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Timbre (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 Sonido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Luz (LED blanco)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
void setup() {
  pinMode(2, INPUT_PULLUP); pinMode(13, OUTPUT);
}

void loop() {
  if (digitalRead(2) == LOW) {
    for (int i = 0; i < 6; i++) {
      digitalWrite(13, HIGH); tone(8, 1200, 120); delay(150);
      digitalWrite(13, LOW); delay(150);
    }
  }
}
```

💡 ¿Cómo funciona?

La tecnología también sirve para incluir: el mismo aviso llega por dos sentidos, el oído y la vista.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

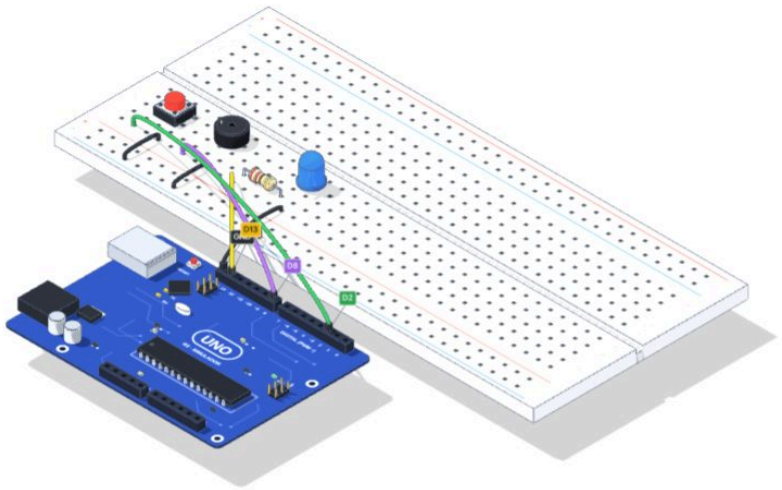
🏆 **Reto:** Agrega una vibración (motor) para quien está dormido.

PROYECTO 167 · HOGAR Y AMBIENTE

Pastillero con recordatorio

★★☆ Intermedio · Tiempo: 30 a 45 minutos

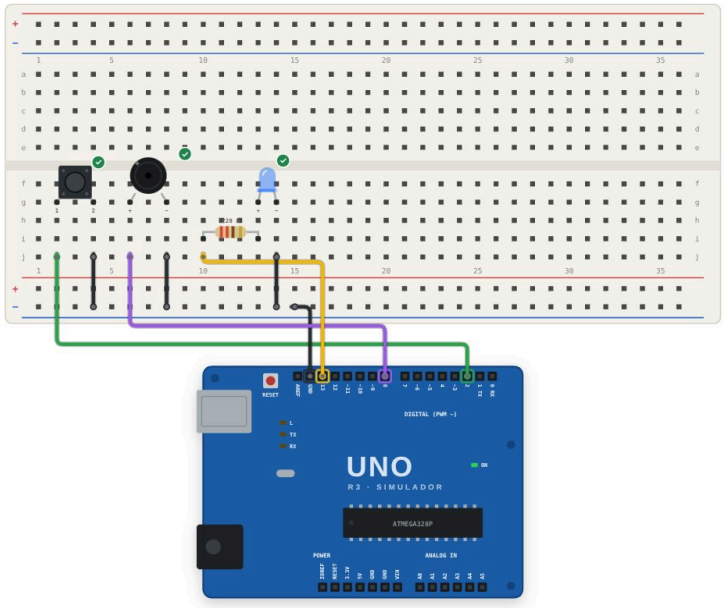
Cada cierto tiempo suena un aviso para tomar la medicina; se apaga al presionar el botón.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Ya la tomé (Botón)
- 1 × Aviso (Zumbador)
- 1 × Medicina (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Ya la tomé (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Aviso (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 **Medicina (LED azul)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
const unsigned long CADA = 20000;        // 20 s para probar (8 horas = 28800000)
unsigned long ultima = 0;
bool avisando = false;

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP); pinMode(13, OUTPUT);
}

void loop() {
  if (millis() - ultima > CADA) avisando = true;
  if (avisando) {
    digitalWrite(13, HIGH); tone(8, 1500, 100); delay(500);
    digitalWrite(13, LOW); delay(500);
    if (digitalRead(2) == LOW) { avisando = false; ultima = millis(); Serial.println("Medicina tomada"); }
  }
}
```

💡 ¿Cómo funciona?

Con millis() se mide el tiempo sin detener el programa. Cuando pasan los milisegundos de CADA, empieza el aviso hasta que alguien presiona el botón.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

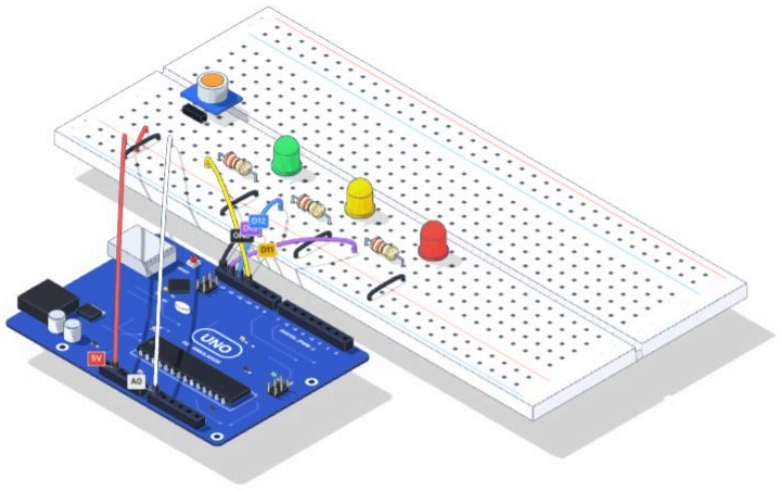
🏆 **Reto:** Cuenta cuántas dosis se han tomado en el día.

PROYECTO 168 · HOGAR Y AMBIENTE

Medidor de calidad del aire con semáforo

★★☆ Intermedio · Tiempo: 30 a 45 minutos

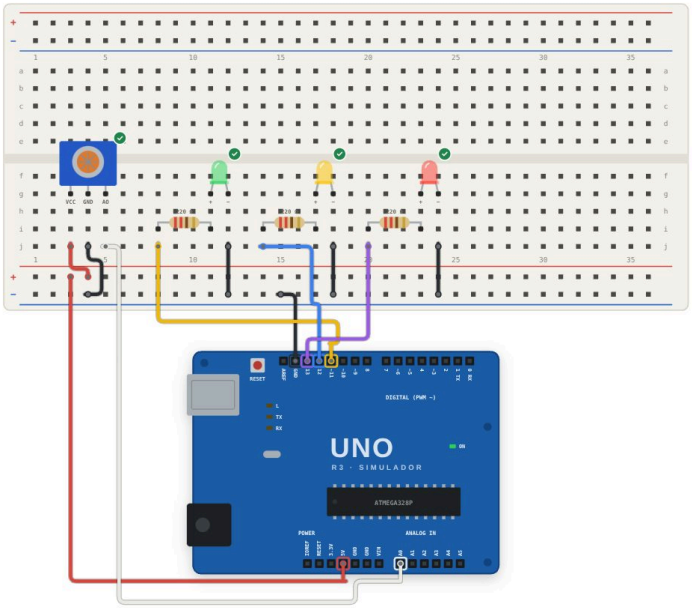
Un semáforo indica si el aire está limpio (verde), regular (amarillo) o contaminado (rojo).



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor MQ-2 (Sensor de gas MQ-2)
- 1 × Limpio (LED)
- 1 × Regular (LED)
- 1 × Contaminado (LED)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor MQ-2 (Sensor de gas MQ-2)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- AO (señal) → pin **A0**

3 Limpio (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Regular (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Contaminado (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 245 mA.
- Consumo estimado: **~245 mA** funcionando, con picos de **~245 mA**.

2 El programa

```
1  void setup() {
2    Serial.begin(9600);
3    pinMode(11, OUTPUT); pinMode(12, OUTPUT); pinMode(13, OUTPUT);
4  }
5
6  void loop() {
7    int aire = analogRead(A0);
8    Serial.print("Aire: "); Serial.println(aire);
9    digitalWrite(11, aire < 250);
10   digitalWrite(12, aire >= 250 && aire < 450);
11   digitalWrite(13, aire >= 450);
12   delay(500);
13 }
```

💡 ¿Cómo funciona?

Cada LED recibe el resultado de una comparación: verdadero enciende, falso apaga. Solo una de las tres comparaciones es verdadera a la vez.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Sube el deslizador de gas o humo.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

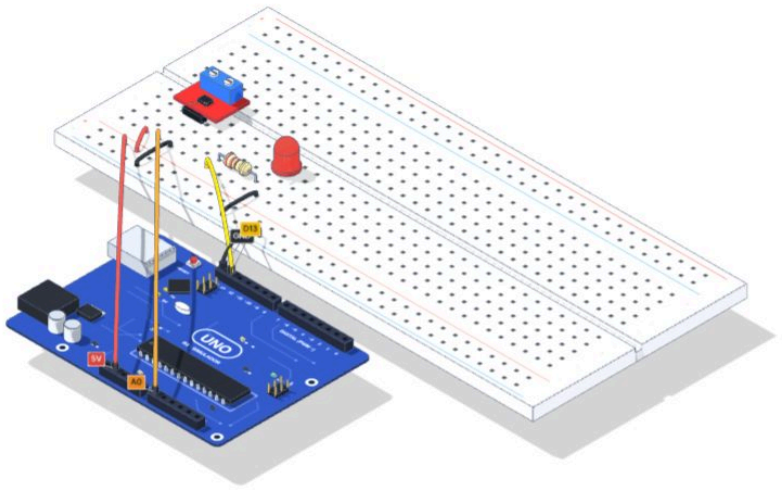
🏆 **Reto:** Abre una ventana (servo) cuando el aire esté contaminado.

PROYECTO 169 · HOGAR Y AMBIENTE

Medidor de consumo de un aparato

★★★ Avanzado · Tiempo: 45 a 90 minutos

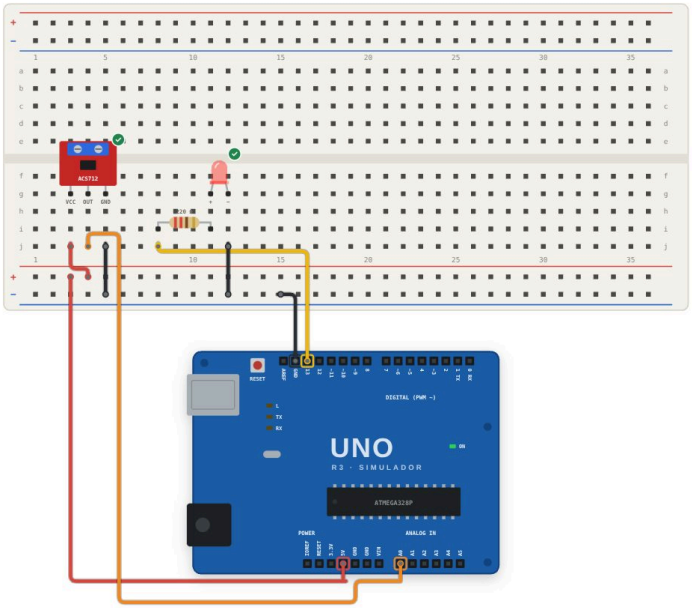
Con el sensor ACS712 se mide cuánta corriente consume un aparato y se calcula su potencia.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de corriente ACS712
- 1 × Consumo alto (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de corriente ACS712 (Sensor de corriente ACS712)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Consumo alto (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Para el proyecto real

- La carga que se mide pasa por los dos tornillos azules del sensor. No midas aparatos de 110 V sin supervisión.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 75 mA.
- Consumo estimado: **~75 mA** funcionando, con picos de **~75 mA**.

2 El programa

```
1  void setup() {
2    Serial.begin(9600);
3    pinMode(13, OUTPUT);
4  }
5
6  void loop() {
7    float voltios = analogRead(A0) * 5.0 / 1023.0;
8    float amperios = (voltios - 2.5) / 0.185;
9    if (amperios < 0) amperios = -amperios;
10   float vatios = amperios * 12.0;        // aparato de 12 V
11   Serial.print("Corriente: "); Serial.print(amperios, 2);
12   Serial.print(" A   Potencia: "); Serial.print(vatios, 1); Serial.println(" W");
13   digitalWrite(13, amperios > 2.0);
14   delay(1000);
15 }
```

💡 ¿Cómo funciona?

Sin corriente, el ACS712 entrega 2,5 V. Por cada amperio la salida cambia 0,185 V. La potencia es voltaje por corriente (P = V × I).

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la corriente que pasa por el sensor con su deslizador.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ **Ten en cuenta**

- La carga que se mide pasa por los dos tornillos azules del sensor. No midas aparatos de 110 V sin supervisión.

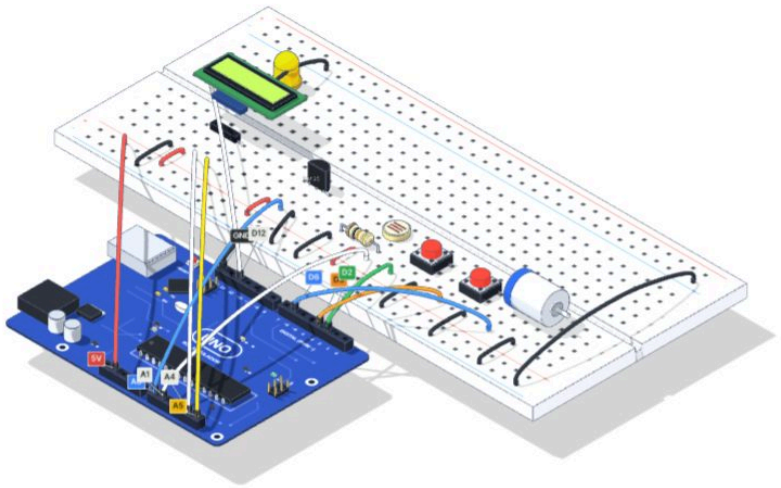
🏆 **Reto:** Calcula cuánta energía (W·h) se ha gastado desde que se encendió.

PROYECTO 170 · HOGAR Y AMBIENTE

Casa inteligente con menú en LCD

★★★ Avanzado · Tiempo: 45 a 90 minutos

Un panel de la casa: la LCD muestra la temperatura y la luz, y con dos botones se encienden la sala y el ventilador.

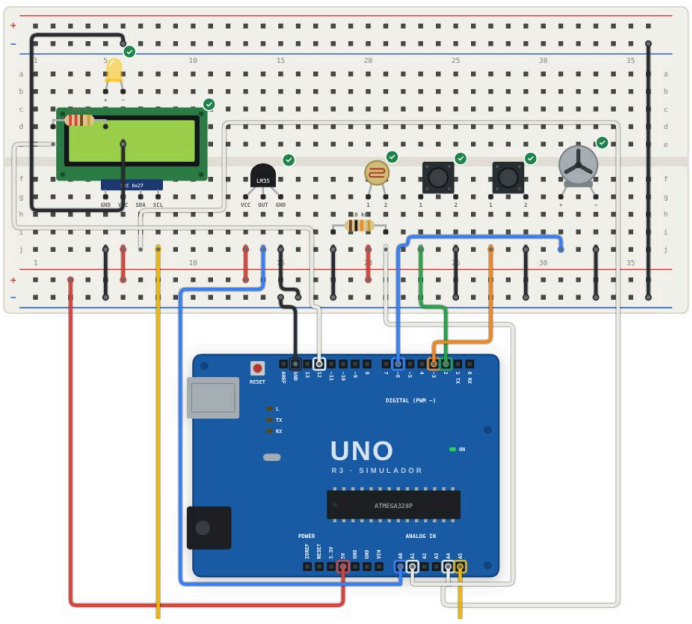


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Sensor LM35
- 1 × Fotorresistencia
- 1 × Sala (Botón)
- 1 × Ventilador (Botón)
- 1 × Luz de la sala (LED)
- 1 × Ventilador (Motor DC)
- 1 × Resistencia 220 Ω
- 1 × Resistencia 10 kΩ
- Transistor TIP120 (o 2N2222)
- Diodo 1N4007
- Resistencia 1 kΩ
- Fuente 6–12 V para los motores
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

3 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

4 Fotorresistencia (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A1** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

5 Sala (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

6 Ventilador (Botón)

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

7 Luz de la sala (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

8 Ventilador (Motor DC)

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

9 Energía y controladores

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motor → necesita transistor:** un pin da solo 20–40 mA y el motor pide 150–250 mA (hasta 1 A al arrancar). Conexión: pin → resistencia de 1 kΩ → base del **TIP120**; motor entre el + de la fuente y el colector; emisor a GND. Pon un **diodo 1N4007** en paralelo con el motor (la raya hacia el +) para frenar los picos de voltaje que dañan el transistor.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al + del motor (el – del motor va al colector del transistor). Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~297 mA** funcionando, con picos de **~1097 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
bool sala = false, venti = false;

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP);
  pinMode(12, OUTPUT); pinMode(6, OUTPUT);
}

void loop() {
```

```
  if (digitalRead(2) == LOW) { sala = !sala; delay(300); }
  if (digitalRead(3) == LOW) { venti = !venti; delay(300); }
  digitalWrite(12, sala); digitalWrite(6, venti);
  int temp = analogRead(A0) * 500.0 / 1023;
  int luz = map(analogRead(A1), 0, 1023, 0, 100);
  lcd.setCursor(0, 0); lcd.print("T:"); lcd.print(temp); lcd.print("C Luz:"); lcd.print(luz); lcd.print("%  ");
  lcd.setCursor(0, 1); lcd.print(sala ? "Sala ON " : "Sala OFF"); lcd.print(venti ? " Ven ON " : " Ven OFF");
  delay(200);
}
```

💡 ¿Cómo funciona?

Es una pequeña domótica: sensores para leer la casa, botones para dar órdenes y una pantalla para ver todo. Los espacios al final de cada texto borran restos de números anteriores.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.
- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).
- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

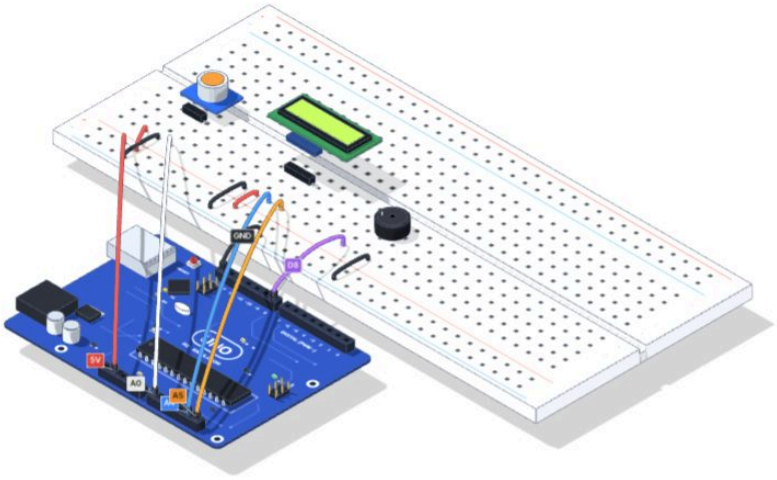
🏆 **Reto:** Haz que el ventilador se encienda solo si la temperatura pasa de 30 °C.

PROYECTO 171 · HOGAR Y AMBIENTE

Indicador de gas en LCD

★★☆ Intermedio · Tiempo: 30 a 45 minutos

La LCD muestra el nivel de gas con una barra y la palabra SEGURO o PELIGRO.

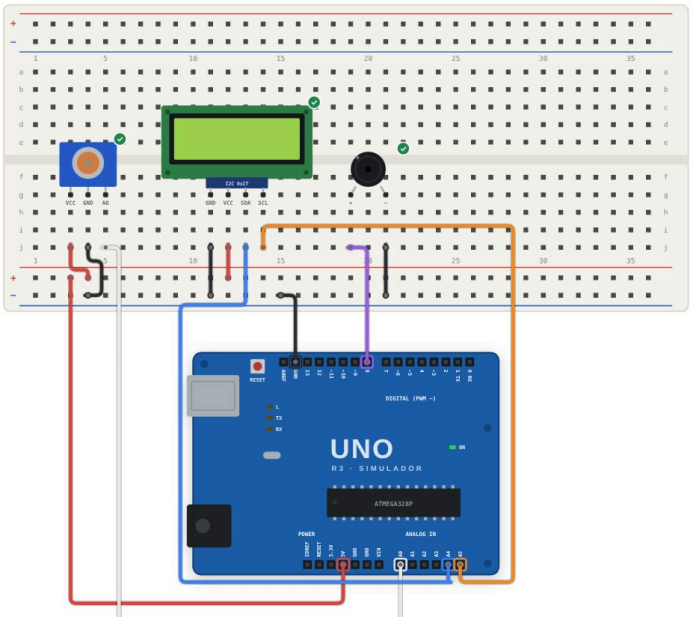


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor MQ-2 (Sensor de gas MQ-2)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Alarma (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor MQ-2 (Sensor de gas MQ-2)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- AO (señal) → pin **A0**

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Alarma (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 260 mA.
- Consumo estimado: **~260 mA** funcionando, con picos de **~260 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  lcd.init(); lcd.backlight();
}

void loop() {
  int g = analogRead(A0);
  int barras = map(g, 0, 1023, 0, 16);
  lcd.clear();
  lcd.print(g > 400 ? "PELIGRO " : "SEGURO  "); lcd.print(g);
  lcd.setCursor(0, 1);
  for (int i = 0; i < barras; i++) lcd.print("#");
  if (g > 400) tone(8, 2000, 200);
  delay(500);
}
```

💡 ¿Cómo funciona?

map() convierte la lectura (0 a 1023) en un número de casillas (0 a 16) para dibujar la barra.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Sube el deslizador de gas o humo.

Con piezas reales: sube el programa.

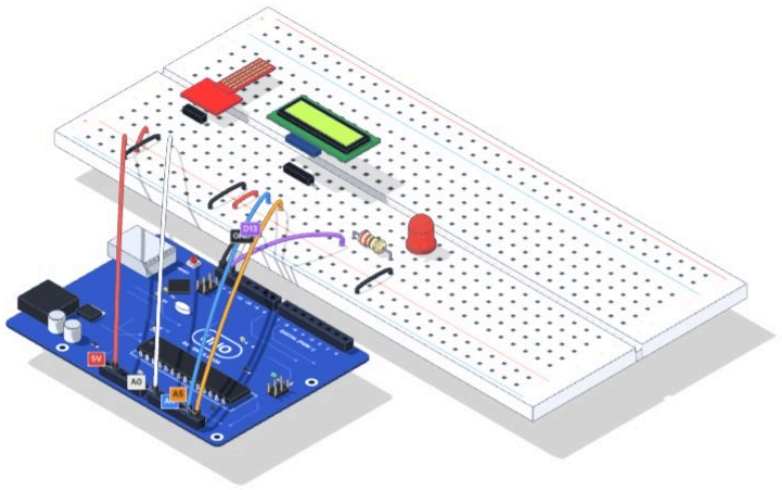
🏆 **Reto:** Guarda el valor más alto registrado.

PROYECTO 172 · HOGAR Y AMBIENTE

Nivel de tanque en LCD

★★☆ Intermedio · Tiempo: 30 a 45 minutos

La LCD muestra el porcentaje de agua del tanque y avisa cuando está casi vacío.

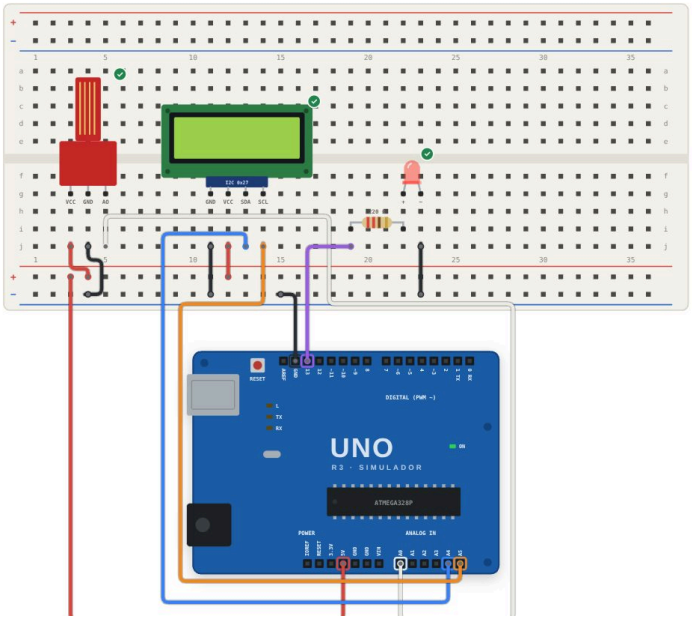


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de nivel (Sensor de nivel de agua)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Casi vacío (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de nivel (Sensor de nivel de agua)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- AO (señal) → pin **A0**

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Casi vacío (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 100 mA.
- Consumo estimado: **~100 mA** funcionando, con picos de **~100 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(13, OUTPUT);
}

void loop() {
  int porc = constrain(map(analogRead(A0), 0, 650, 0, 100), 0, 100);
  lcd.clear();
  lcd.print("Tanque: "); lcd.print(porc); lcd.print(" %");
  lcd.setCursor(0, 1);
  if (porc < 15) { lcd.print("Casi vacio!"); digitalWrite(13, HIGH); }
  else { lcd.print("Nivel normal"); digitalWrite(13, LOW); }
  delay(1000);
}
```

💡 ¿Cómo funciona?

constrain() asegura que el porcentaje nunca quede por debajo de 0 ni por encima de 100, aunque el sensor dé un valor raro.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del nivel de agua.

Con piezas reales: sube el programa.

🏆 **Reto:** Agrega la bomba con relé para llenar el tanque.

CAPÍTULO 12

Juegos

Programar también es divertido: juegos de memoria, reflejos, azar y concursos.

EN ESTE CAPÍTULO · 14 PROYECTOS

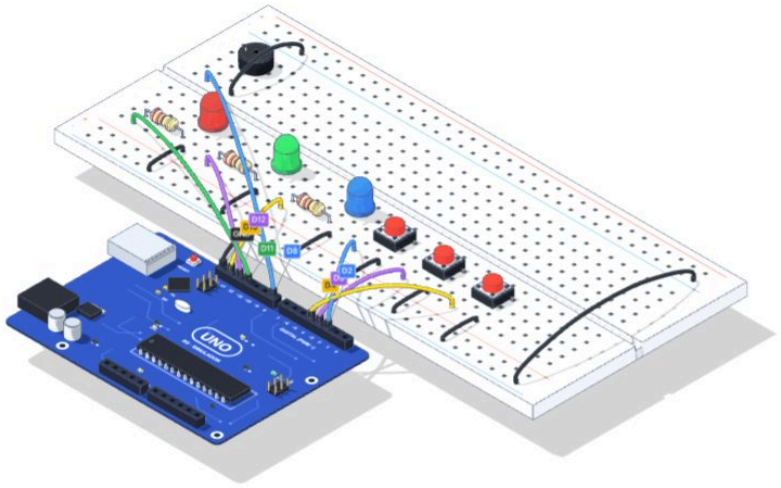
173 Simón dice de 3 colores ★★★
174 Adivina el número ★★☆
175 Ruleta de la suerte ★☆☆
176 Piedra, papel o tijera ★★☆
177 Contador de puntos para partido ★★☆
178 Juego de puntería con luz ★★☆
179 Carrera de botones para dos ★★☆
180 Dado doble con LCD ★☆☆
181 Esquiva obstáculos en la LCD ★★★
182 Tirar la moneda ★☆☆
183 Laberinto con joystick ★★★
184 Juego de equilibrio con inclinación ★★☆
185 Pregunta y respuesta con botones ★★☆
186 Reloj de ajedrez ★★☆

PROYECTO 173 · JUEGOS

Simón dice de 3 colores

★★★ Avanzado · Tiempo: 45 a 90 minutos

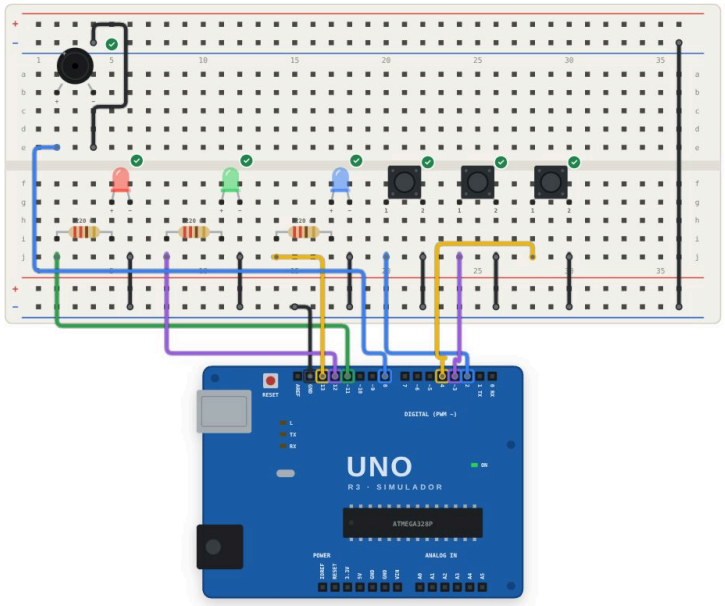
El Arduino muestra una secuencia de luces que crece cada ronda; repítela con los botones sin equivocarte.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Rojo (LED)
- 1 × Verde (LED)
- 1 × Azul (LED)
- 1 × Botón rojo (Botón)
- 1 × Botón verde (Botón)
- 1 × Botón azul (Botón)
- 1 × Sonido (Zumbador)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Rojo (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

3 Verde (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

4 Azul (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 Botón rojo (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

6 Botón verde (Botón)

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

7 Botón azul (Botón)

- Pata 1 → pin **D4**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

8 Sonido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

9 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 125 mA.
- Consumo estimado: **~125 mA** funcionando, con picos de **~125 mA**.

2 El programa

```
int secuencia[20];
int largo = 0;
int leds[] = {11, 12, 13};
int botones[] = {2, 3, 4};
int tonos[] = {440, 660, 880};

void mostrar(int c) {
  digitalWrite(leds[c], HIGH); tone(8, tonos[c], 300); delay(400);
  digitalWrite(leds[c], LOW); delay(150);
}

int esperarBoton() {
  while (true) {
    for (int i = 0; i < 3; i++) {
      if (digitalRead(botones[i]) == LOW) { mostrar(i); return i; }
    }
  }
}

void setup() {
  Serial.begin(9600);
```

```
  for (int i = 0; i < 3; i++) { pinMode(leds[i], OUTPUT); pinMode(botones[i], INPUT_PULLUP); }
  randomSeed(analogRead(A5));
}

void loop() {
  secuencia[largo] = random(3);
  largo++;
  Serial.print("Ronda "); Serial.println(largo);
  for (int i = 0; i < largo; i++) mostrar(secuencia[i]);
  for (int i = 0; i < largo; i++) {
    if (esperarBoton() != secuencia[i]) {
      Serial.print("Perdiste. Llegaste a la ronda "); Serial.println(largo);
      tone(8, 150, 1000); delay(1500);
      largo = 0;
      return;
    }
  }
  delay(600);
}
```

💡 ¿Cómo funciona?

La secuencia se guarda en un arreglo y crece un color en cada ronda. Luego el programa espera tus botones y los compara uno por uno con lo guardado.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

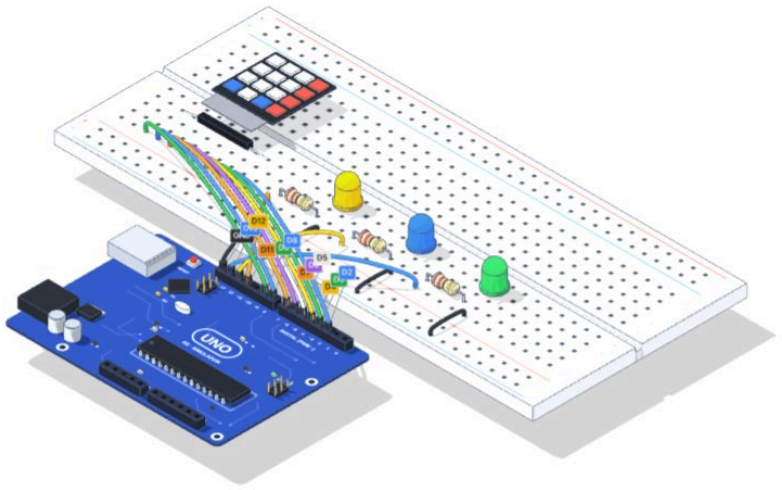
🏆 **Reto:** Agrega un cuarto color y un récord.

PROYECTO 174 · JUEGOS

Adivina el número

★★☆ Intermedio · Tiempo: 30 a 45 minutos

El Arduino piensa un número del 1 al 9; escríbelo en el teclado y te dice si es mayor o menor.

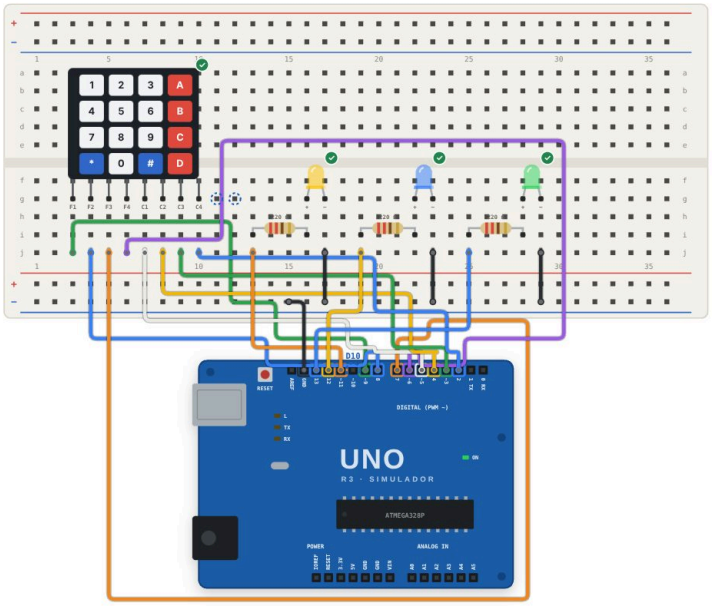


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Teclado (Teclado matricial 4×4)
- 1 × Más alto (LED)
- 1 × Más bajo (LED)
- 1 × Correcto (LED)
- 3 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Keypad (de Mark Stanley y Alexander Brevig)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 **Teclado** (Teclado matricial 4×4)

- Fila 1 → pin **~D9**
- Fila 2 → pin **D8**
- Fila 3 → pin **D7**
- Fila 4 → pin **~D6**
- Columna 1 → pin **~D5**
- Columna 2 → pin **D4**
- Columna 3 → pin **~D3**
- Columna 4 → pin **D2**

3 **Más alto** (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Más bajo** (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

5 **Correcto** (LED verde)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 96 mA.
- Consumo estimado: **~96 mA** funcionando, con picos de **~96 mA**.

2 El programa

```
#include <Keypad.h>
const byte FILAS = 4;
const byte COLUMNAS = 4;
char teclas[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte pinesFilas[FILAS] = {9, 8, 7, 6};
byte pinesColumnas[COLUMNAS] = {5, 4, 3, 2};
Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas, pinesColumnas, FILAS, COLUMNAS);
int secreto, intentos = 0;

void setup() {
  Serial.begin(9600);
  pinMode(11, OUTPUT); pinMode(12, OUTPUT); pinMode(13, OUTPUT);
  randomSeed(analogRead(A5));
  secreto = random(1, 10);
  Serial.println("Pensé un número del 1 al 9");
}

void loop() {
  char t = teclado.getKey();
  if (t < '1' || t > '9') return;
  int n = t - '0';
  intentos++;
  digitalWrite(11, LOW); digitalWrite(12, LOW);
```

```
  if (n < secreto) { Serial.println("Es más alto"); digitalWrite(11, HIGH); }
  else if (n > secreto) { Serial.println("Es más bajo"); digitalWrite(12, HIGH); }
  else {
    Serial.print("¡Correcto! en "); Serial.print(intentos); Serial.println(" intentos");
    digitalWrite(13, HIGH); delay(2000); digitalWrite(13, LOW);
    secreto = random(1, 10); intentos = 0;
  }
}
```

💡 ¿Cómo funciona?

Restar el carácter '0' convierte la tecla en número: '7' − '0' = 7. Cada intento enciende la luz que da la pista.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Toca las teclas del teclado dibujado en su tarjeta, como si fuera el teclado real.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

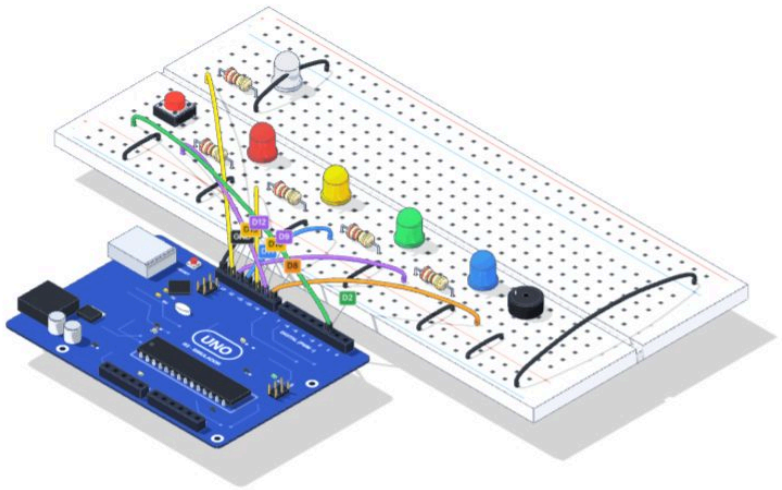
🏆 **Reto:** Amplía el juego del 1 al 99 usando dos teclas.

PROYECTO 175 · JUEGOS

Ruleta de la suerte

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

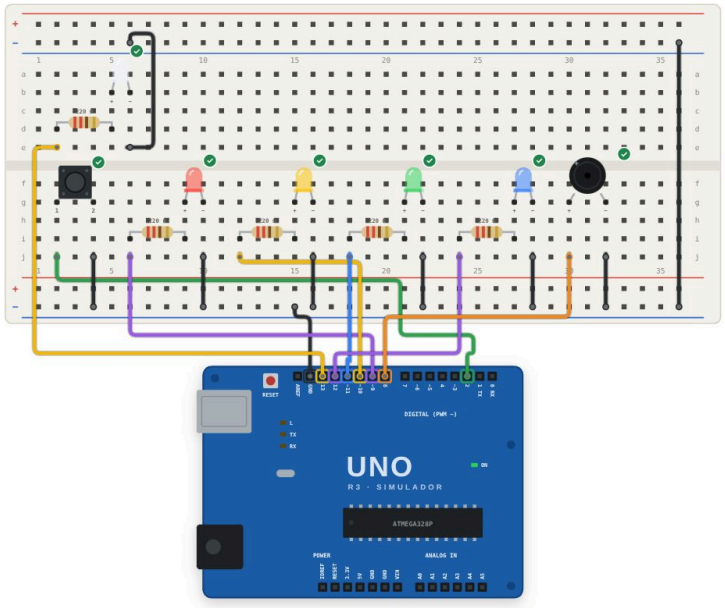
Presiona el botón y la luz gira por 5 LED cada vez más lento hasta detenerse en uno al azar.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Girar (Botón)
- 1 × L1 (LED)
- 1 × L2 (LED)
- 1 × L3 (LED)
- 1 × L4 (LED)
- 1 × L5 (LED)
- 1 × Clic (Zumbador)
- 5 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Girar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **L1 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D9**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **L2 (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

5 **L3 (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

6 **L4 (LED azul)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

7 **L5 (LED blanco)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

8 **Clic (Zumbador)**

- Pata + → pin **D8**
- Pata − → riel **−** (GND)

9 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 155 mA.
- Consumo estimado: **~155 mA** funcionando, con picos de **~155 mA**.

2 El programa

```
void setup() {
  pinMode(2, INPUT_PULLUP);
  for (int p = 9; p <= 13; p++) pinMode(p, OUTPUT);
  randomSeed(analogRead(A5));
}

void loop() {
  if (digitalRead(2) == HIGH) return;
  int vueltas = random(15, 30);
  int pausa = 30;
  for (int i = 0; i < vueltas; i++) {
    int p = 9 + i % 5;
    digitalWrite(p, HIGH); tone(8, 1500, 10); delay(pausa);
    digitalWrite(p, LOW);
    pausa += 10;
  }
  digitalWrite(9 + (vueltas - 1) % 5, HIGH);
  delay(2000);
  digitalWrite(9 + (vueltas - 1) % 5, LOW);
}
```

💡 ¿Cómo funciona?

La pausa crece en cada paso, por eso la ruleta frena poco a poco. El número de vueltas es aleatorio, así nadie sabe dónde parará.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

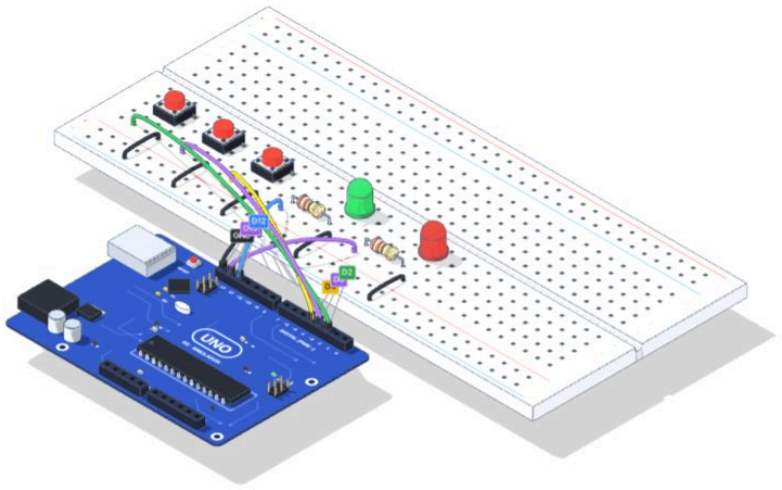
🏆 **Reto:** Asigna un premio a cada color y muéstralo en el monitor.

PROYECTO 176 · JUEGOS

Piedra, papel o tijera

★★☆ Intermedio · Tiempo: 30 a 45 minutos

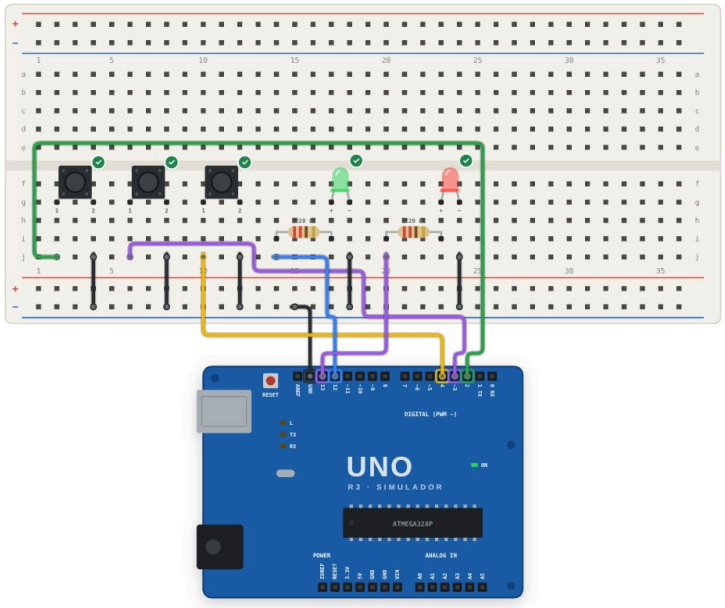
Elige con un botón y el Arduino elige al azar; el monitor dice quién ganó.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Piedra (Botón)
- 1 × Papel (Botón)
- 1 × Tijera (Botón)
- 1 × Ganaste (LED)
- 1 × Perdiste (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Piedra (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Papel (Botón)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Tijera (Botón)**

- Pata 1 → pin **D4**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

5 **Ganaste (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

6 **Perdiste (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

7 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
String nombre[] = {"Piedra", "Papel", "Tijera"};

void setup() {
  Serial.begin(9600);
  for (int b = 2; b <= 4; b++) pinMode(b, INPUT_PULLUP);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
  randomSeed(analogRead(A5));
}

void loop() {
  int yo = -1;
  for (int b = 0; b < 3; b++) if (digitalRead(2 + b) == LOW) yo = b;
  if (yo < 0) return;
  int ard = random(3);
  Serial.print("Tú: "); Serial.print(nombre[yo]); Serial.print("  Arduino: ");
Serial.println(nombre[ard]);
  if (yo == ard) Serial.println("Empate");
  else if ((yo + 1) % 3 == ard) { Serial.println("Ganó el Arduino"); digitalWrite(13, HIGH); }
  else { Serial.println("¡Ganaste!"); digitalWrite(12, HIGH); }
  delay(1500);
  digitalWrite(12, LOW); digitalWrite(13, LOW);
}
```

💡 ¿Cómo funciona?

Truco matemático: cada opción pierde contra la siguiente (piedra < papel < tijera < piedra). Por eso (yo + 1) % 3 es la opción que te gana.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

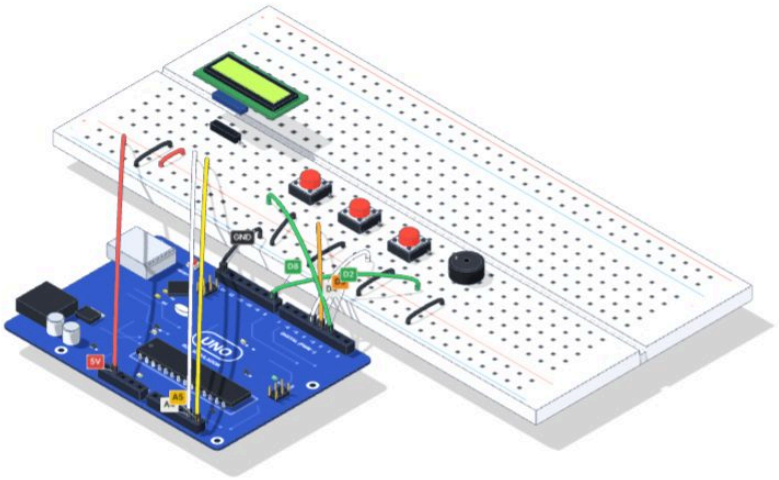
🏆 **Reto:** Lleva el marcador del juego al mejor de 5.

PROYECTO 177 · JUEGOS

Contador de puntos para partido

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Un marcador para dos equipos: cada botón suma un punto y la LCD muestra el resultado.

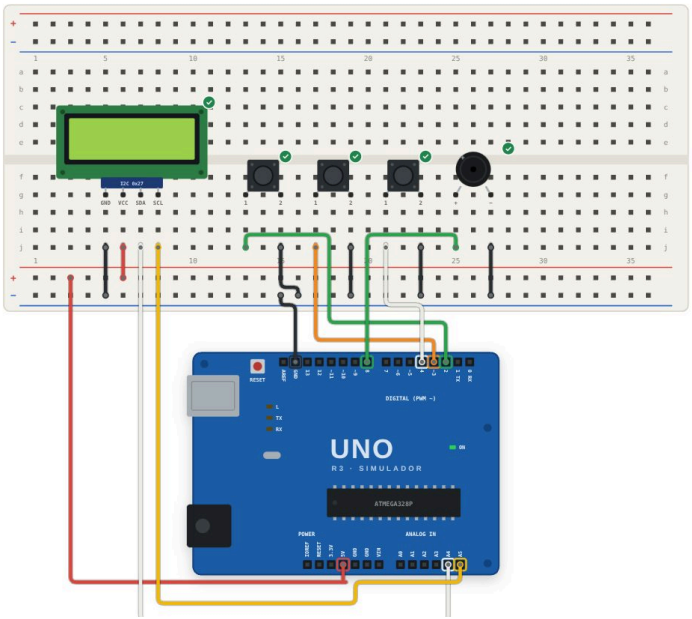


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Equipo A (Botón)
- 1 × Equipo B (Botón)
- 1 × Reiniciar (Botón)
- 1 × Pito (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

3 Equipo A (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Equipo B (Botón)

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

5 Reiniciar (Botón)

- Pata 1 → pin **D4**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

6 Pito (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
int a = 0, b = 0;

void mostrar() {
  lcd.clear();
  lcd.print("Equipo A: "); lcd.print(a);
  lcd.setCursor(0, 1); lcd.print("Equipo B: "); lcd.print(b);
}

void setup() {
  lcd.init(); lcd.backlight();
  for (int p = 2; p <= 4; p++) pinMode(p, INPUT_PULLUP);
  mostrar();
}

void loop() {
  if (digitalRead(2) == LOW) { a++; tone(8, 1200, 100); mostrar(); delay(300); }
  if (digitalRead(3) == LOW) { b++; tone(8, 900, 100); mostrar(); delay(300); }
  if (digitalRead(4) == LOW) { a = 0; b = 0; tone(8, 400, 400); mostrar(); delay(300); }
}
```

💡 **¿Cómo funciona?**

Cada botón cambia su contador y vuelve a dibujar la pantalla. El delay evita que una sola pulsación sume varios puntos.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

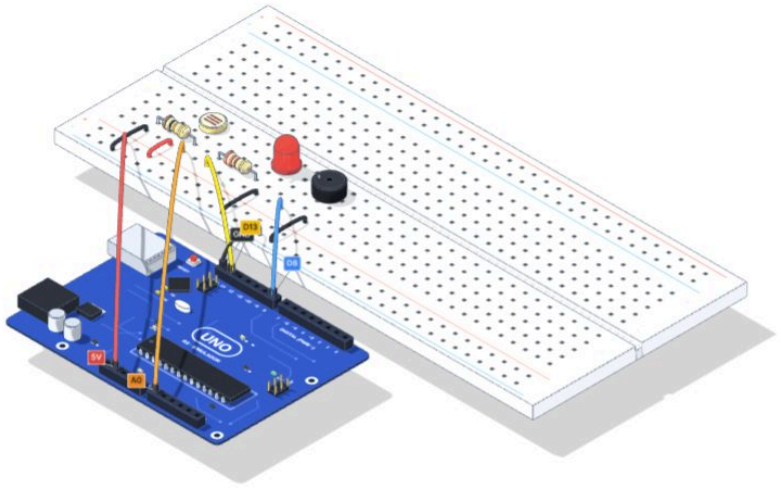
🏆 **Reto:** Haz que suene una fanfarria cuando un equipo llegue a 10.

PROYECTO 178 · JUEGOS

Juego de puntería con luz

★★☆ Intermedio · Tiempo: 30 a 45 minutos

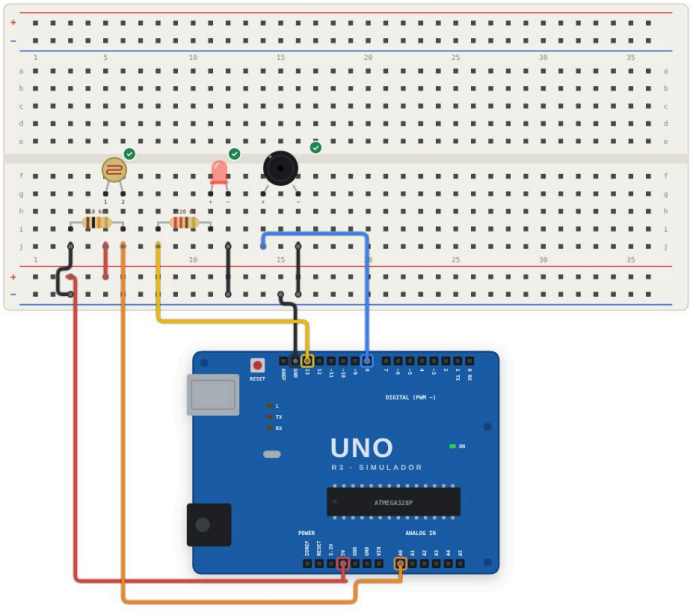
Ilumina la fotorresistencia con una linterna en menos de 3 segundos cada vez que se encienda el LED.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Blanco (LDR) (Fotorresistencia)
- 1 × Señal (LED)
- 1 × Sonido (Zumbador)
- 1 × Resistencia 220 Ω
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Blanco (LDR) (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Señal (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Sonido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 96 mA.
- Consumo estimado: **~96 mA** funcionando, con picos de **~96 mA**.

2 El programa

```
int puntos = 0;

void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
  randomSeed(analogRead(A5));
}

void loop() {
  delay(random(1000, 4000));
  digitalWrite(13, HIGH);
  unsigned long inicio = millis();
  bool acerto = false;
  while (millis() - inicio < 3000) {
    if (analogRead(A0) > 800) { acerto = true; break; }
  }
  digitalWrite(13, LOW);
  if (acerto) { puntos++; tone(8, 1800, 200); Serial.print("¡Blanco! Puntos: ");
Serial.println(puntos); }
  else { tone(8, 200, 400); Serial.println("Fallaste"); }
}
```

💡 ¿Cómo funciona?

El juego espera un tiempo al azar, enciende la señal y mide si la luz de la linterna llega a tiempo. La instrucción break sale del while antes de terminar los 3 segundos.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

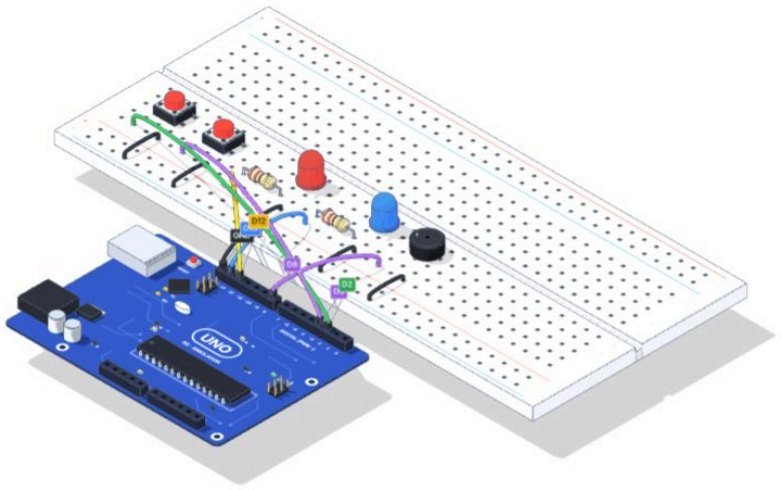
🏆 **Reto:** Reduce el tiempo disponible a medida que suman puntos.

PROYECTO 179 · JUEGOS

Carrera de botones para dos

★★☆ Intermedio · Tiempo: 30 a 45 minutos

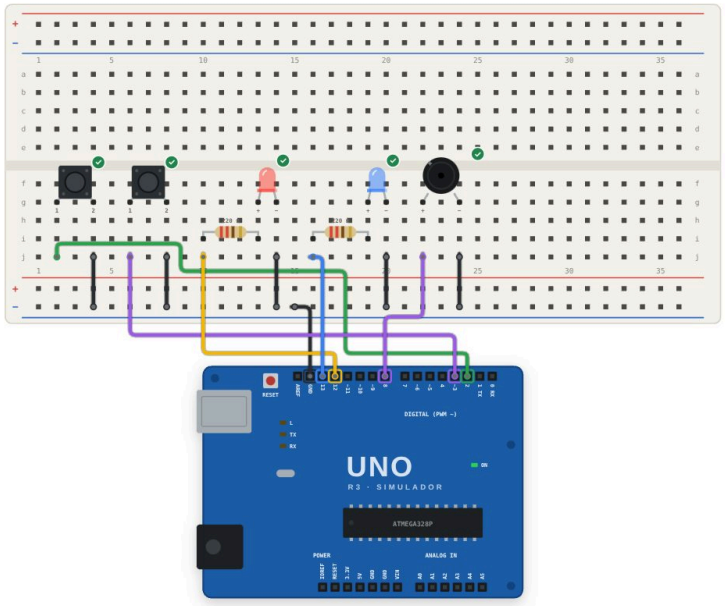
Dos jugadores presionan su botón lo más rápido posible durante 10 segundos; gana quien más veces lo presione.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Jugador 1 (Botón)
- 1 × Jugador 2 (Botón)
- 1 × Gana 1 (LED)
- 1 × Gana 2 (LED)
- 1 × Salida (Zumbador)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Jugador 1 (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 Jugador 2 (Botón)

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Gana 1 (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Gana 2 (LED azul)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 Salida (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
}

void loop() {
  Serial.println("¡Ya!");
  tone(8, 1000, 500);
  int c1 = 0, c2 = 0;
  bool antes1 = HIGH, antes2 = HIGH;
  unsigned long inicio = millis();
  while (millis() - inicio < 10000) {
    bool ahora1 = digitalRead(2), ahora2 = digitalRead(3);
    if (antes1 == HIGH && ahora1 == LOW) c1++;
    if (antes2 == HIGH && ahora2 == LOW) c2++;
    antes1 = ahora1; antes2 = ahora2;
    delay(5);
  }
  Serial.print("Jugador 1: "); Serial.print(c1); Serial.print("  Jugador 2: ");
Serial.println(c2);
  digitalWrite(12, c1 > c2); digitalWrite(13, c2 > c1);
  delay(5000);
  digitalWrite(12, LOW); digitalWrite(13, LOW);
```

```
24 }
```

💡 **¿Cómo funciona?**

Para contar pulsaciones (no el tiempo que se mantiene presionado) se compara el estado de antes con el de ahora: solo cuenta el cambio de HIGH a LOW.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

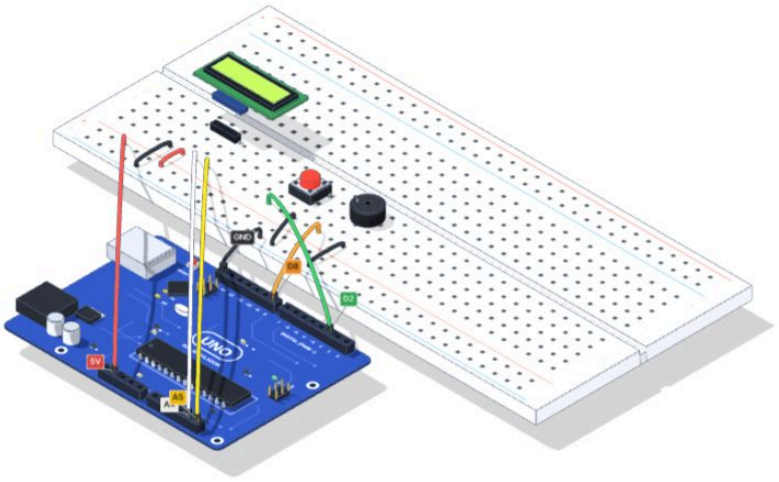
🏆 **Reto:** Muestra la cuenta en una LCD mientras juegan.

PROYECTO 180 · JUEGOS

Dado doble con LCD

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

Presiona el botón y la pantalla muestra dos dados y su suma, ideal para el parqués.

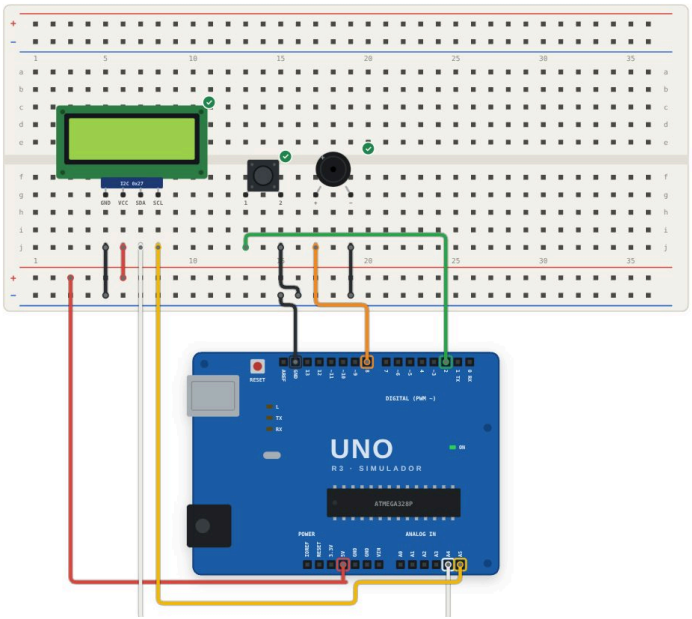


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Lanzar (Botón)
- 1 × Sonido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

3 Lanzar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Sonido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```ino
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT_PULLUP);
  randomSeed(analogRead(A5));
  lcd.print("Presiona lanzar");
}

void loop() {
  if (digitalRead(2) == HIGH) return;
  for (int i = 0; i < 8; i++) { tone(8, 600 + i * 100, 30); delay(60); }
  int d1 = random(1, 7), d2 = random(1, 7);
  lcd.clear();
  lcd.print("Dados: "); lcd.print(d1); lcd.print(" y "); lcd.print(d2);
  lcd.setCursor(0, 1); lcd.print("Suma: "); lcd.print(d1 + d2);
  if (d1 == d2) lcd.print(" PAR!");
  delay(500);
}
```

💡 ¿Cómo funciona?

random(1, 7) da un número del 1 al 6 (el 7 no se incluye). Si los dos dados son iguales, en el parqués se llama "par" y se vuelve a tirar.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

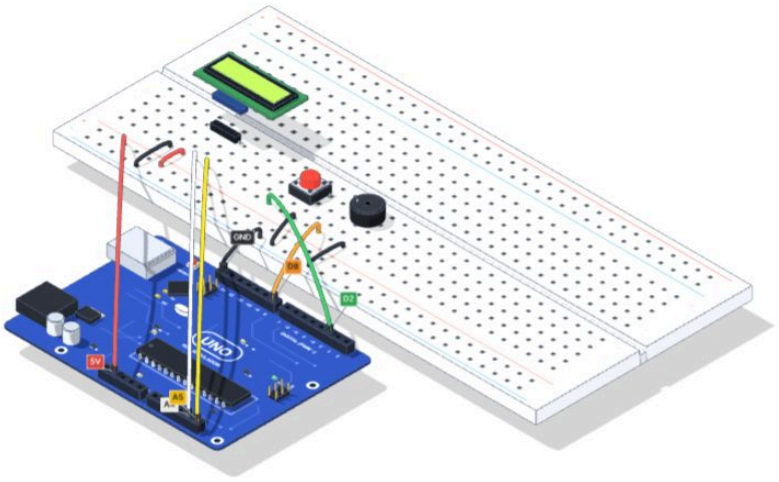
🏆 **Reto:** Cuenta cuántos pares han salido.

PROYECTO 181 · JUEGOS

Esquiva obstáculos en la LCD

★★★ Avanzado · Tiempo: 45 a 90 minutos

Un muñeco corre en la LCD: presiona el botón para saltar los obstáculos que vienen.

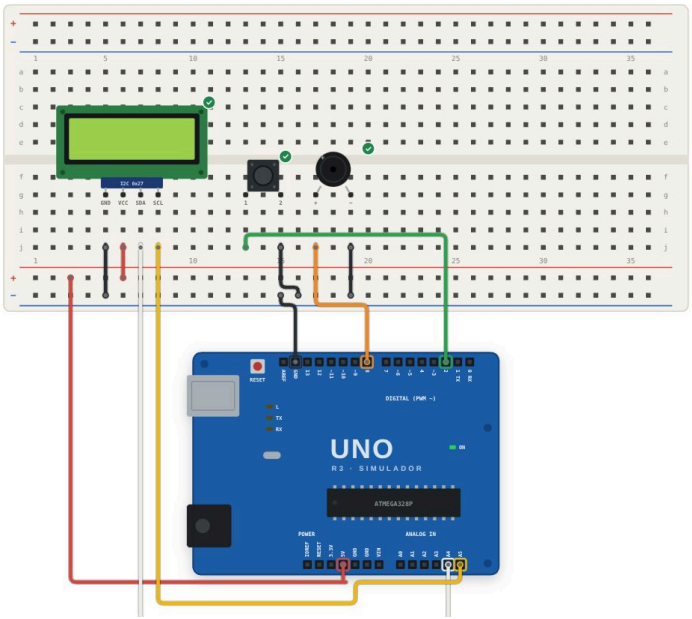


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Saltar (Botón)
- 1 × Sonido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

3 Saltar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Sonido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```ino
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
int obst = 15, puntos = 0;

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT_PULLUP);
}

void loop() {
  bool arriba = digitalRead(2) == LOW;
  lcd.clear();
  lcd.setCursor(1, arriba ? 0 : 1); lcd.print("o");
  lcd.setCursor(obst, 1); lcd.print("#");
  lcd.setCursor(10, 0); lcd.print(puntos);
  if (obst == 1 && !arriba) {
    tone(8, 150, 800);
    lcd.clear(); lcd.print("Perdiste! "); lcd.print(puntos);
    delay(2000);
    puntos = 0; obst = 15;
    return;
  }
  obst--;
  if (obst < 0) { obst = 15; puntos++; tone(8, 1500, 30); }
  delay(250);
}
```

💡 **¿Cómo funciona?**

El obstáculo (#) avanza una columna hacia la izquierda en cada vuelta. Si llega a la columna del muñeco y este no está saltando, el juego termina.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

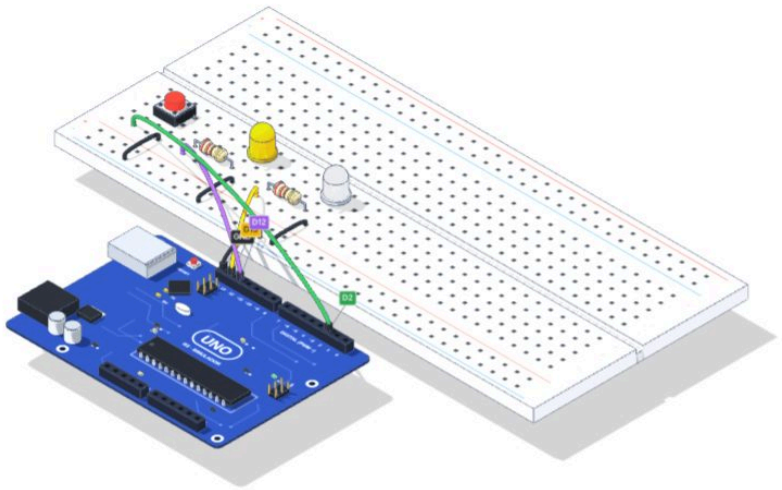
🏆 **Reto:** Aumenta la velocidad cada 5 puntos.

PROYECTO 182 · JUEGOS

Tirar la moneda

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

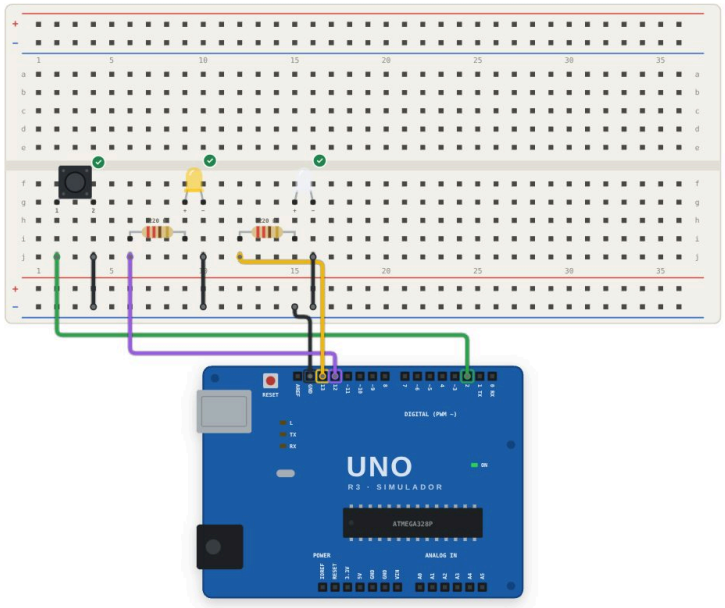
Para decidir algo: presiona el botón y se enciende "cara" o "sello" al azar.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Lanzar (Botón)
- 1 × Cara (LED)
- 1 × Sello (LED)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Lanzar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Cara (LED amarillo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Sello (LED blanco)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 80 mA.
- Consumo estimado: **~80 mA** funcionando, con picos de **~80 mA**.

2 El programa

```
1  void setup() {
2    Serial.begin(9600);
3    pinMode(2, INPUT_PULLUP); pinMode(12, OUTPUT); pinMode(13, OUTPUT);
4    randomSeed(analogRead(A5));
5  }
6
7  void loop() {
8    if (digitalRead(2) == LOW) {
9      for (int i = 0; i < 10; i++) { digitalWrite(12, i % 2); digitalWrite(13, (i + 1) % 2);
   delay(80); }
10     int r = random(2);
11     digitalWrite(12, r == 0); digitalWrite(13, r == 1);
12     Serial.println(r == 0 ? "Cara" : "Sello");
13     delay(1000);
14   }
15 }
```

💡 ¿Cómo funciona?

random(2) da 0 o 1 con la misma probabilidad. El parpadeo alternado antes del resultado simula la moneda girando en el aire.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

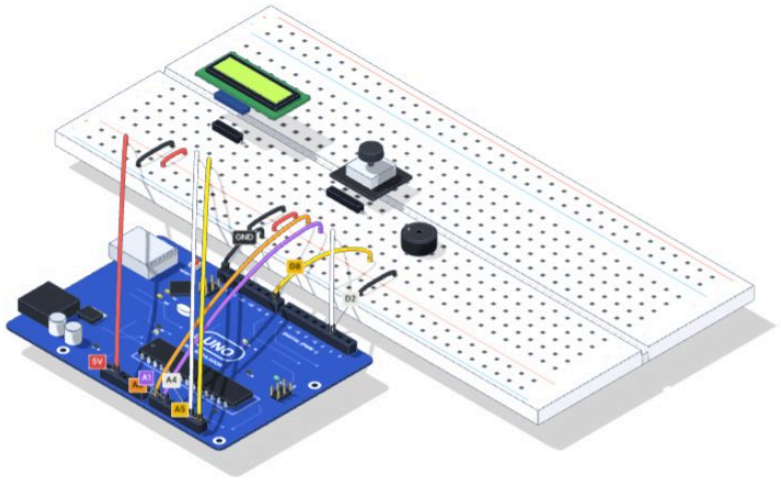
🏆 **Reto:** Lanza 100 monedas y cuenta cuántas caras salen. ¿Se acerca a 50?

PROYECTO 183 · JUEGOS

Laberinto con joystick

★★★ Avanzado · Tiempo: 45 a 90 minutos

Mueve una estrella por la LCD con el joystick hasta llegar a la meta.

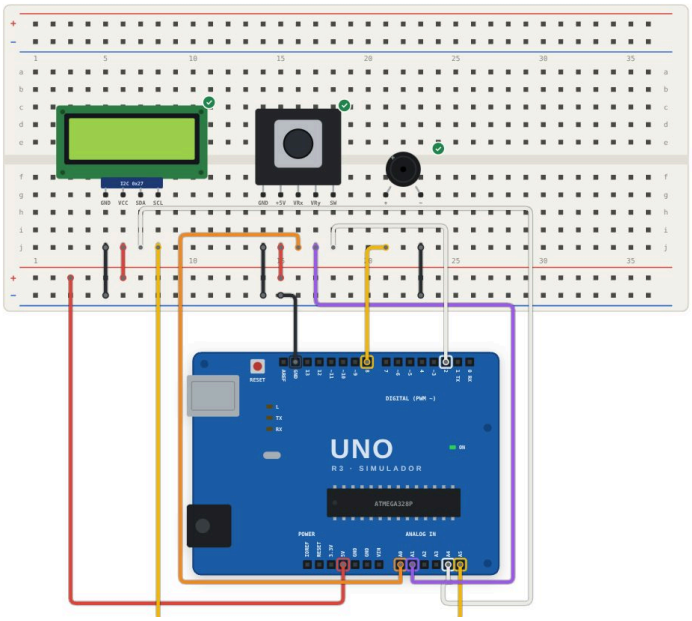


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Joystick
- 1 × Sonido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

3 Joystick (Joystick)

- GND → riel **–** (GND)
- +5V → riel **+** (5V)
- VRx (eje X) → pin **A0**
- VRy (eje Y) → pin **A1**
- SW (botón) → pin **D2**

4 Sonido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 111 mA.
- Consumo estimado: **~111 mA** funcionando, con picos de **~111 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
int x = 0, y = 0;
int metaX = 15, metaY = 1;

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT_PULLUP);
  randomSeed(analogRead(A5));
}

void loop() {
  int jx = analogRead(A0), jy = analogRead(A1);
  if (jx > 800 && x < 15) x++;
  if (jx < 200 && x > 0) x--;
  if (jy > 800) y = 1;
  if (jy < 200) y = 0;
  lcd.clear();
  lcd.setCursor(metaX, metaY); lcd.print("M");
  lcd.setCursor(x, y); lcd.print("*");
  if (x == metaX && y == metaY) {
    tone(8, 1800, 300);
    lcd.clear(); lcd.print("Llegaste!"); delay(1500);
    metaX = random(16); metaY = random(2); x = 0; y = 0;
  }
  delay(150);
}
```

💡 **¿Cómo funciona?**

El joystick son dos perillas: cerca de 0 o de 1023 significa que se movió a un lado. La estrella solo avanza si no se sale de la pantalla.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Mueve los deslizadores X y Y del joystick y presiona su botón.

Con piezas reales: sube el programa.

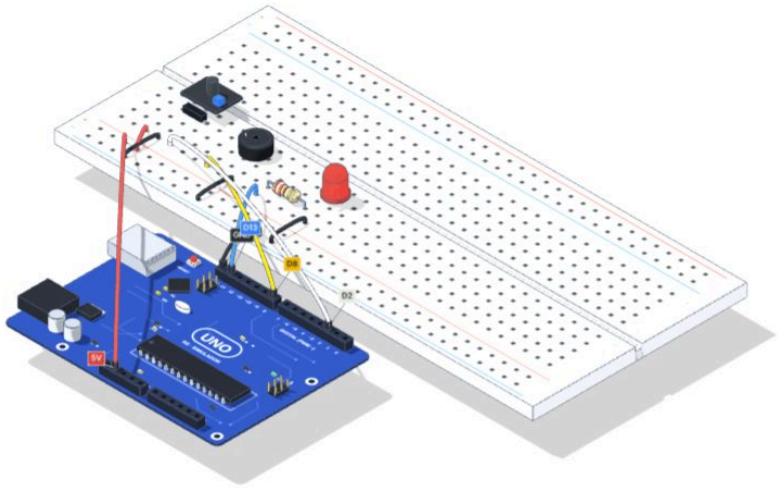
🏆 **Reto:** Agrega un cronómetro para ver cuánto tardas.

PROYECTO 184 · JUEGOS

Juego de equilibrio con inclinación

★★☆ Intermedio · Tiempo: 30 a 45 minutos

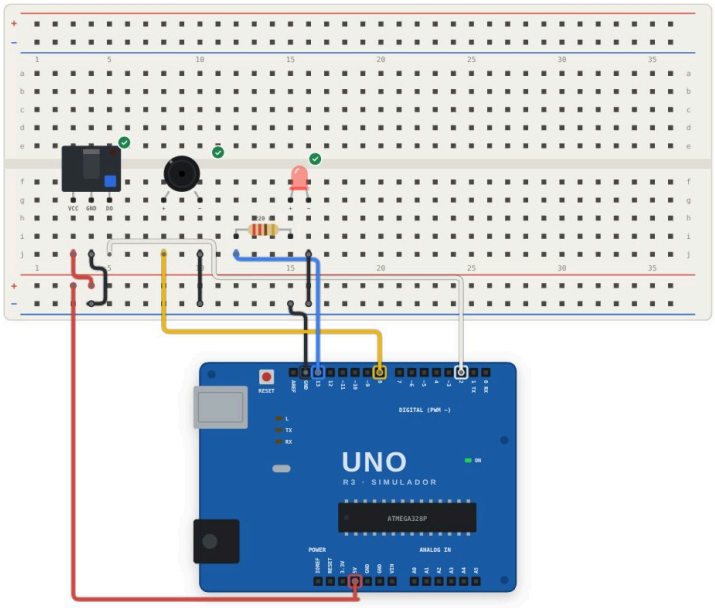
Mantén el sensor derecho el mayor tiempo posible; si se inclina, pierdes.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de inclinación
- 1 × Sonido (Zumbador)
- 1 × Perdiste (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de inclinación (Sensor de inclinación)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Sonido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Perdiste (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 100 mA.
- Consumo estimado: **~100 mA** funcionando, con picos de **~100 mA**.

2 El programa

```
unsigned long inicio;

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT); pinMode(13, OUTPUT);
  inicio = millis();
}

void loop() {
  if (digitalRead(2) == HIGH) {
    unsigned long segundos = (millis() - inicio) / 1000;
    Serial.print("Aguantaste "); Serial.print(segundos); Serial.println(" segundos");
    digitalWrite(13, HIGH); tone(8, 200, 800); delay(2000);
    digitalWrite(13, LOW);
    inicio = millis();
  }
}
```

💡 ¿Cómo funciona?

El sensor de inclinación tiene una bolita metálica: al inclinarse cambia la salida. El tiempo aguantado se calcula con millis().

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Está inclinado».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

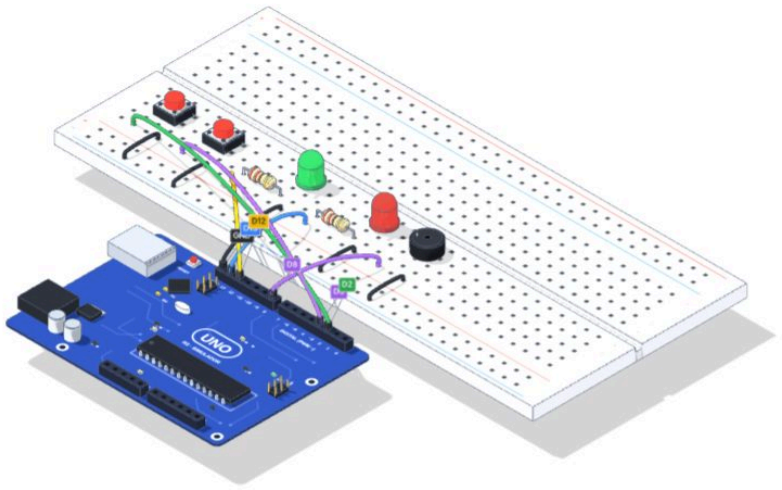
🏆 **Reto:** Pon el sensor sobre una cuchara y haz carreras con él.

PROYECTO 185 · JUEGOS

Pregunta y respuesta con botones

★★☆ Intermedio · Tiempo: 30 a 45 minutos

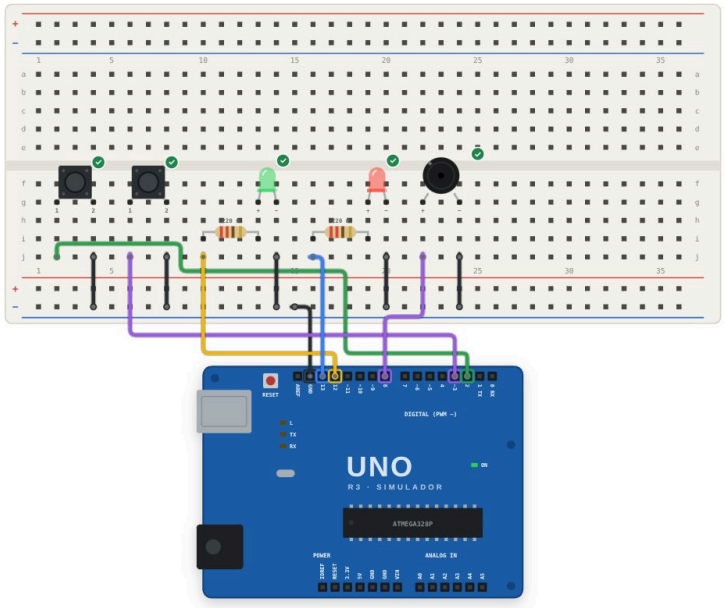
Un concurso de preguntas en el monitor serie: responde con los botones A o B.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Opción A (Botón)
- 1 × Opción B (Botón)
- 1 × Correcto (LED)
- 1 × Incorrecto (LED)
- 1 × Sonido (Zumbador)
- 2 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Opción A (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Opción B (Botón)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Correcto (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 **Incorrecto (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)

6 **Sonido (Zumbador)**

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

7 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```
String preguntas[] = {"¿Cuántos pines digitales tiene el Arduino UNO? A) 14  B) 20",
                      "¿Qué mide un LM35? A) Luz  B) Temperatura",
                      "¿Ley de Ohm? A) V = I x R  B) V = I + R"};
int correctas[] = {0, 1, 0};
int n = 0, puntos = 0;

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP);
  pinMode(12, OUTPUT); pinMode(13, OUTPUT);
  Serial.println(preguntas[0]);
}

void loop() {
  int r = -1;
  if (digitalRead(2) == LOW) r = 0;
  if (digitalRead(3) == LOW) r = 1;
  if (r < 0) return;
  if (r == correctas[n]) { puntos++; digitalWrite(12, HIGH); tone(8, 1500, 200); }
  else { digitalWrite(13, HIGH); tone(8, 200, 400); }
  delay(1000); digitalWrite(12, LOW); digitalWrite(13, LOW);
  n++;
  if (n == 3) { Serial.print("Puntaje final: "); Serial.print(puntos); Serial.println(" de 3"); n = 0; puntos = 0; }
  Serial.println(preguntas[n]);
}
```

💡 **¿Cómo funciona?**

Las preguntas y sus respuestas correctas están en dos arreglos con el mismo orden. La variable n indica en qué pregunta vamos.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

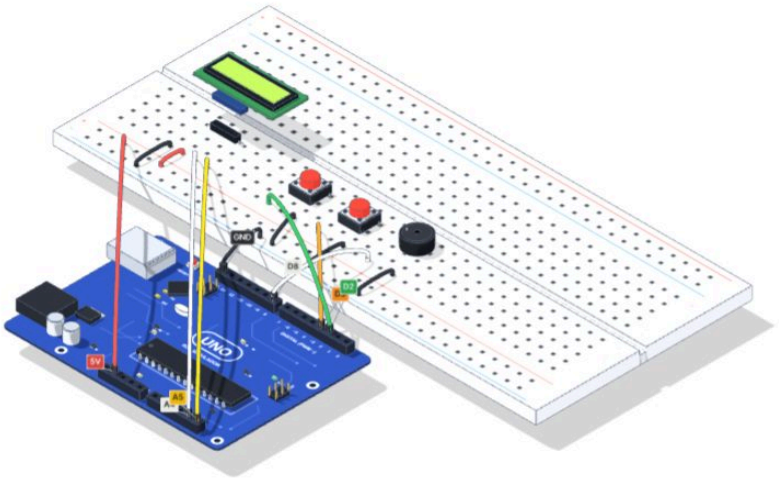
🏆 **Reto:** Escribe 5 preguntas de otra materia.

PROYECTO 186 · JUEGOS

Reloj de ajedrez

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Cada jugador tiene 60 segundos; al terminar su jugada presiona su botón y corre el tiempo del otro.

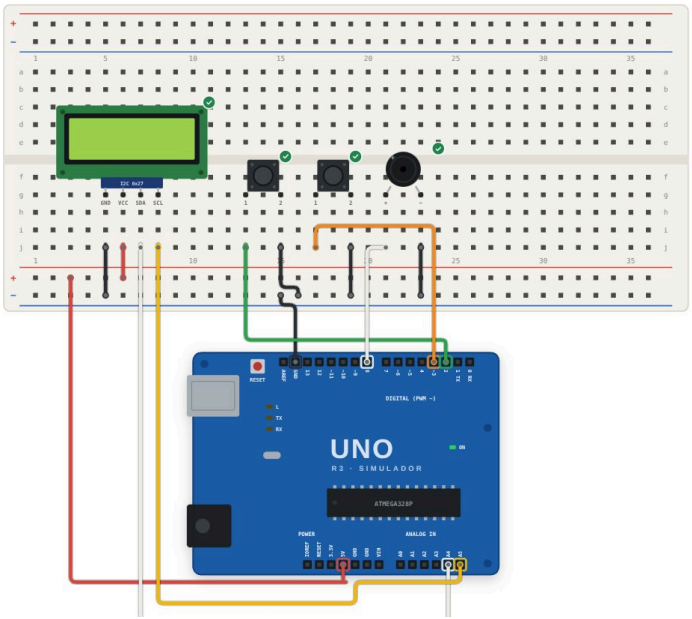


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Jugador 1 (Botón)
- 1 × Jugador 2 (Botón)
- 1 × Fin (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

3 Jugador 1 (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Jugador 2 (Botón)

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

5 Fin (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

6 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 110 mA.
- Consumo estimado: **~110 mA** funcionando, con picos de **~110 mA**.

2 El programa

```ino
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
long t1 = 60000, t2 = 60000;
int turno = 1;
unsigned long antes;

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP);
  antes = millis();
}

void loop() {
  unsigned long ahora = millis();
  if (turno == 1) t1 -= ahora - antes; else t2 -= ahora - antes;
  antes = ahora;
  if (digitalRead(2) == LOW && turno == 1) turno = 2;
  if (digitalRead(3) == LOW && turno == 2) turno = 1;
  int s1 = t1 / 1000;
  int s2 = t2 / 1000;
  lcd.setCursor(0, 0); lcd.print("J1: "); lcd.print(s1); lcd.print(" s  ");
  lcd.setCursor(0, 1); lcd.print("J2: "); lcd.print(s2); lcd.print(" s  ");
  if (t1 <= 0 || t2 <= 0) {
    lcd.clear(); lcd.print(t1 <= 0 ? "Gana jugador 2" : "Gana jugador 1");
    tone(8, 600, 1500); delay(4000);
    t1 = 60000; t2 = 60000; turno = 1; antes = millis();
  }
  delay(100);
}
```

💡 ¿Cómo funciona?

En cada vuelta se resta el tiempo transcurrido solo al jugador que tiene el turno. Así funcionan los relojes de los torneos de ajedrez.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

🏆 **Reto:** Agrega 2 segundos de bonificación por jugada.

CAPÍTULO 13

Medición y ciencia

El Arduino como instrumento de laboratorio: voltímetros, amperímetros, cronómetros y experimentos.

EN ESTE CAPÍTULO · 12 PROYECTOS

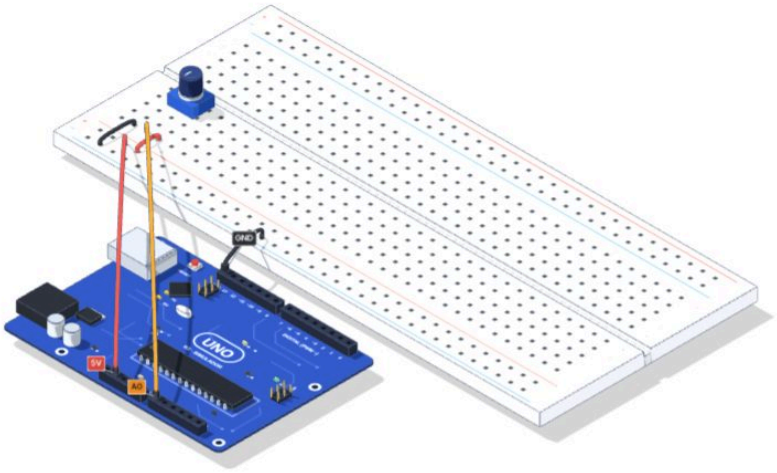
187 Voltímetro de 0 a 5 V ★☆☆
188 Voltímetro con pantalla LCD ★★☆
189 Amperímetro con ACS712 y LCD ★★★
190 Termómetro con máximo y mínimo ★★☆
191 Cronómetro para experimentos ★★☆
192 Medidor de velocidad con dos sensores ★★★
193 Medidor de estatura ★★☆
194 Contador de vueltas con imán ★★☆
195 Experimento: luz y distancia ★★☆
196 Detector de terremotos (sismógrafo) ★★☆
197 Contador de productos en banda ★★☆
198 Alcancía que cuenta monedas ★★☆

PROYECTO 187 · MEDICIÓN Y CIENCIA

Voltímetro de 0 a 5 V

★ ☆ ☆ Básico · Tiempo: 20 a 30 minutos

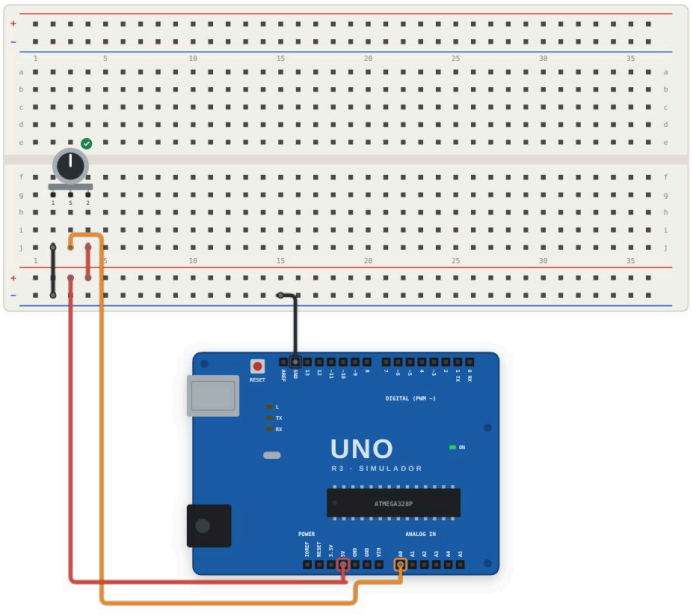
El Arduino mide el voltaje de la perilla y lo muestra en voltios con dos decimales.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Perilla (voltaje a medir) (Potenciómetro)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Perilla (voltaje a medir) (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Para el proyecto real

- Nunca midas más de 5 V directamente en una entrada analógica.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 51 mA.
- Consumo estimado: **~51 mA** funcionando, con picos de **~51 mA**.

2 El programa

```arduino
void setup() {
  Serial.begin(9600);
}

void loop() {
  int lectura = analogRead(A0);
  float voltios = lectura * 5.0 / 1023.0;
  Serial.print("Lectura: "); Serial.print(lectura);
  Serial.print("   Voltaje: "); Serial.print(voltios, 2); Serial.println(" V");
  delay(500);
}
```

💡 ¿Cómo funciona?

El convertidor del Arduino divide 5 V en 1024 partes. Multiplicando por 5 y dividiendo por 1023 se pasa del número al voltaje real.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Nunca midas más de 5 V directamente en una entrada analógica.

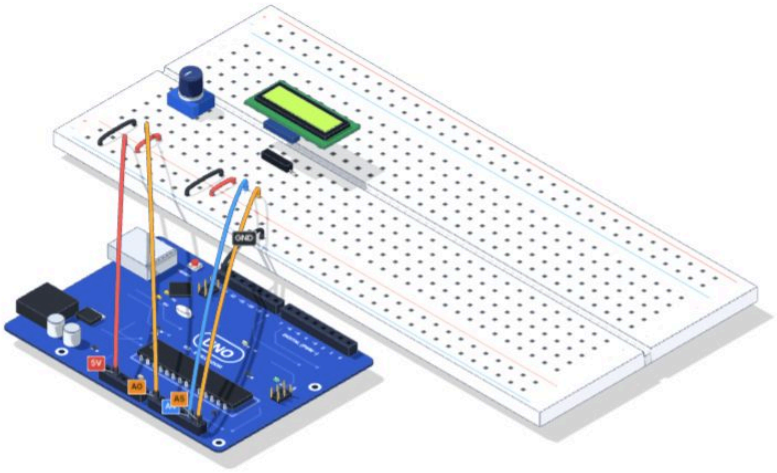
🏆 **Reto:** Muestra una alerta si el voltaje pasa de 4 V.

PROYECTO 188 · MEDICIÓN Y CIENCIA

Voltímetro con pantalla LCD

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Muestra el voltaje en la LCD con una barra que crece.

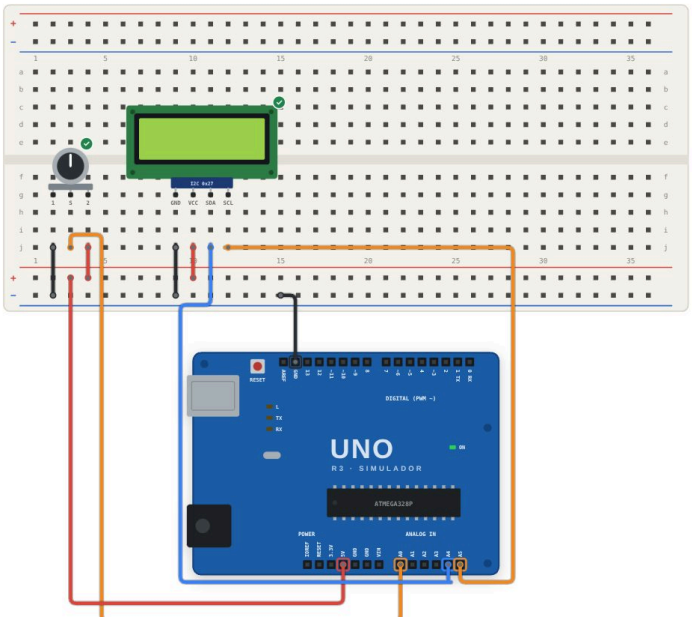


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Perilla (Potenciómetro)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Perilla (Potenciómetro)

- Extremo 1 (izquierda) → riel **–** (GND)
- Centro S (cursor) → pin **A0**
- Extremo 2 (derecha) → riel **+** (5V)

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 81 mA.
- Consumo estimado: **~81 mA** funcionando, con picos de **~81 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  lcd.init(); lcd.backlight();
}

void loop() {
  float v = analogRead(A0) * 5.0 / 1023.0;
  int barras = v * 16 / 5;
  lcd.setCursor(0, 0); lcd.print("Voltaje: "); lcd.print(v, 2); lcd.print(" V ");
  lcd.setCursor(0, 1);
  for (int i = 0; i < 16; i++) lcd.print(i < barras ? "#" : " ");
  delay(200);
}
```

💡 ¿Cómo funciona?

La barra tiene 16 casillas: 5 V las llena todas. El número de casillas encendidas es proporcional al voltaje.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador del potenciómetro en su tarjeta.

Con piezas reales: sube el programa.

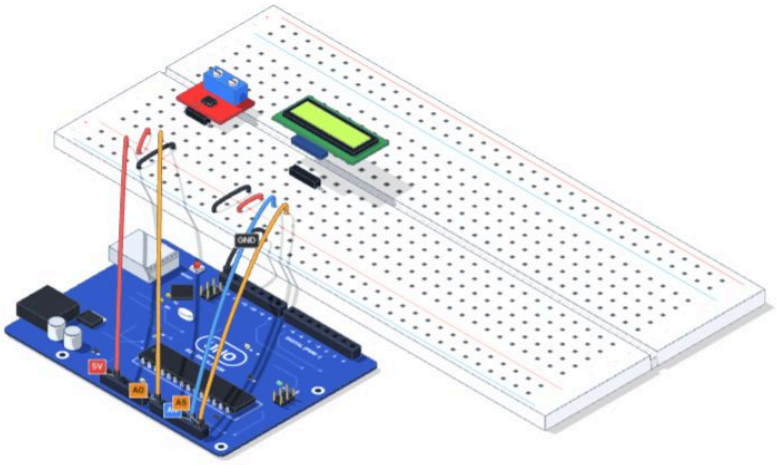
🏆 **Reto:** Haz que suene un pitido al llegar a 5 V.

PROYECTO 189 · MEDICIÓN Y CIENCIA

Amperímetro con ACS712 y LCD

★★★ Avanzado · Tiempo: 45 a 90 minutos

Mide la corriente que pasa por el sensor ACS712 y la muestra en la LCD.

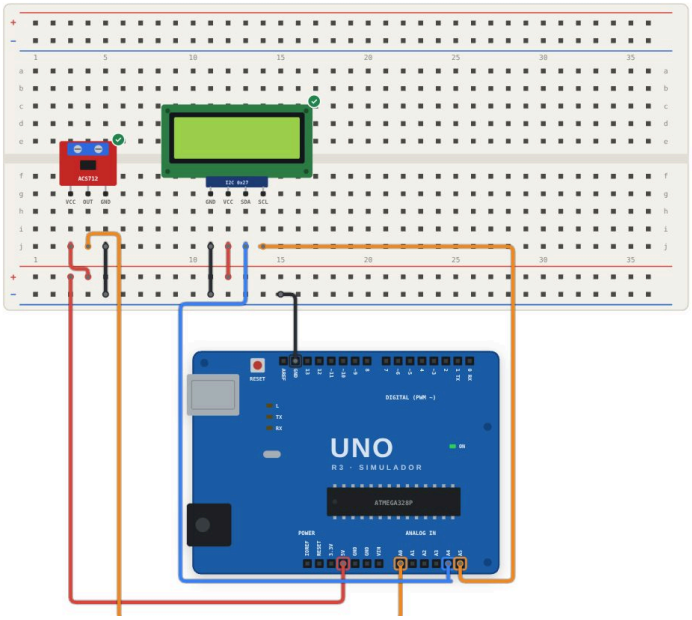


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de corriente (Sensor de corriente ACS712)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de corriente (Sensor de corriente ACS712)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 **Pantalla LCD (Pantalla LCD 16×2 I2C)**

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 90 mA.
- Consumo estimado: **~90 mA** funcionando, con picos de **~90 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)

void setup() {
  lcd.init(); lcd.backlight();
  lcd.print("Amperimetro");
  delay(1000);
}

void loop() {
  long suma = 0;
  for (int i = 0; i < 50; i++) suma += analogRead(A0);
  float v = suma / 50.0 * 5.0 / 1023.0;
  float a = (v - 2.5) / 0.185;
  lcd.clear();
  lcd.print("I = "); lcd.print(a, 2); lcd.print(" A");
  lcd.setCursor(0, 1); lcd.print("Vout = "); lcd.print(v, 2);
  delay(500);
}
```

💡 ¿Cómo funciona?

Se toman 50 lecturas y se promedian para que el número no salte. El ACS712 de 5 A cambia 0,185 V por cada amperio.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la corriente que pasa por el sensor con su deslizador.

Con piezas reales: sube el programa.

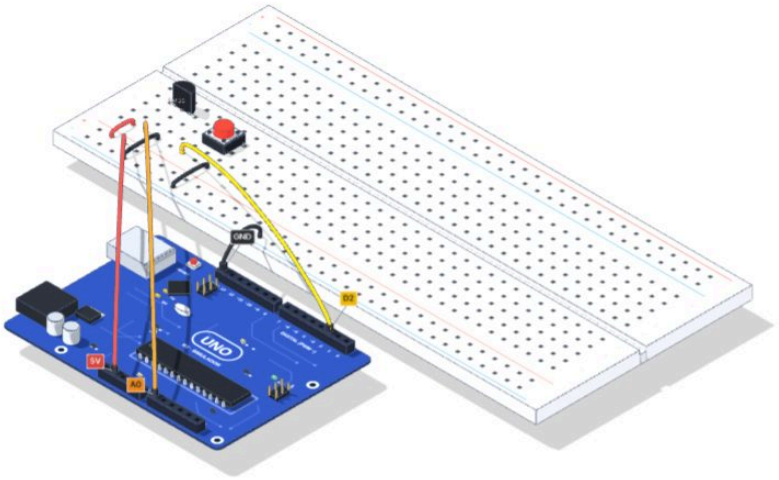
🏆 **Reto:** Muestra también la corriente máxima medida.

PROYECTO 190 · MEDICIÓN Y CIENCIA

Termómetro con máximo y mínimo

★★☆ Intermedio · Tiempo: 30 a 45 minutos

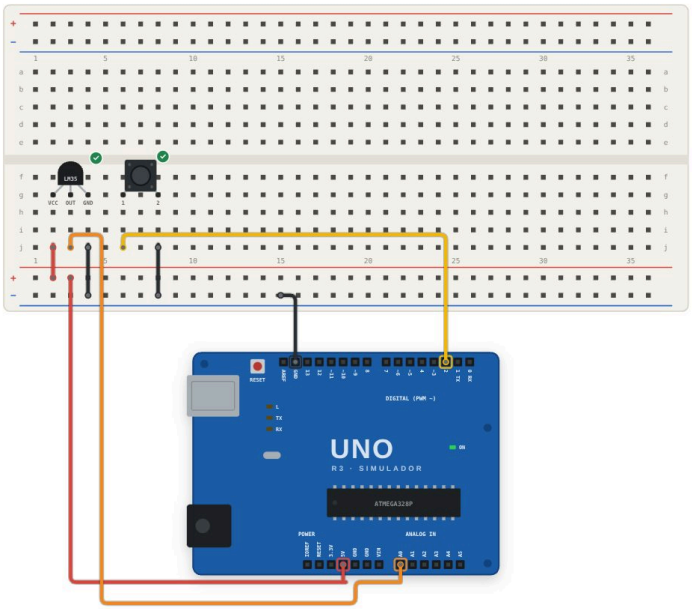
Guarda la temperatura más alta y la más baja desde que se encendió.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor LM35
- 1 × Borrar (Botón)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor LM35 (Sensor LM35)

- VCC → riel **+** (5V)
- OUT (señal) → pin **A0**
- GND → riel **–** (GND)

3 Borrar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 51 mA.
- Consumo estimado: **~51 mA** funcionando, con picos de **~51 mA**.

2 El programa

```ino
float maxima = -100, minima = 200;

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP);
}

void loop() {
  float t = analogRead(A0) * 500.0 / 1023.0;
  if (t > maxima) maxima = t;
  if (t < minima) minima = t;
  if (digitalRead(2) == LOW) { maxima = t; minima = t; }
  Serial.print("Ahora: "); Serial.print(t, 1);
  Serial.print("  Máx: "); Serial.print(maxima, 1);
  Serial.print("  Mín: "); Serial.println(minima, 1);
  delay(1000);
}
```

💡 ¿Cómo funciona?

Las variables empiezan con valores imposibles para que la primera lectura las reemplace. Luego solo cambian si aparece un valor mayor o menor.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Cambia la temperatura con el deslizador del LM35.
- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

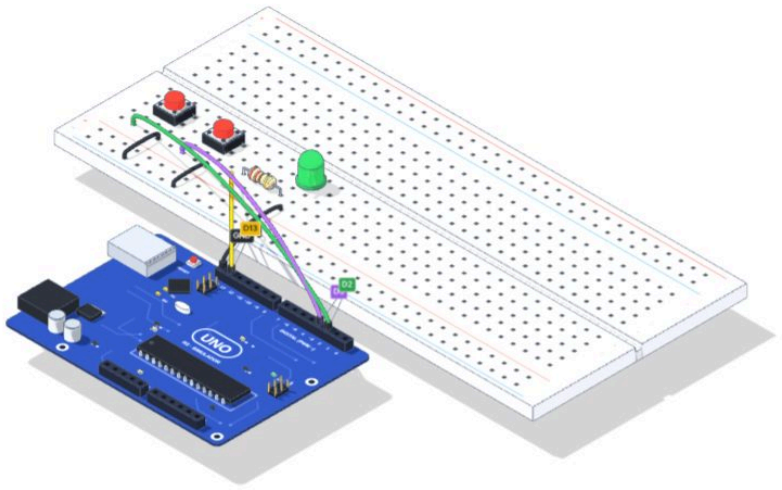
🏆 **Reto:** Guarda también a qué segundo ocurrió el máximo.

PROYECTO 191 · MEDICIÓN Y CIENCIA

Cronómetro para experimentos

★★☆ Intermedio · Tiempo: 30 a 45 minutos

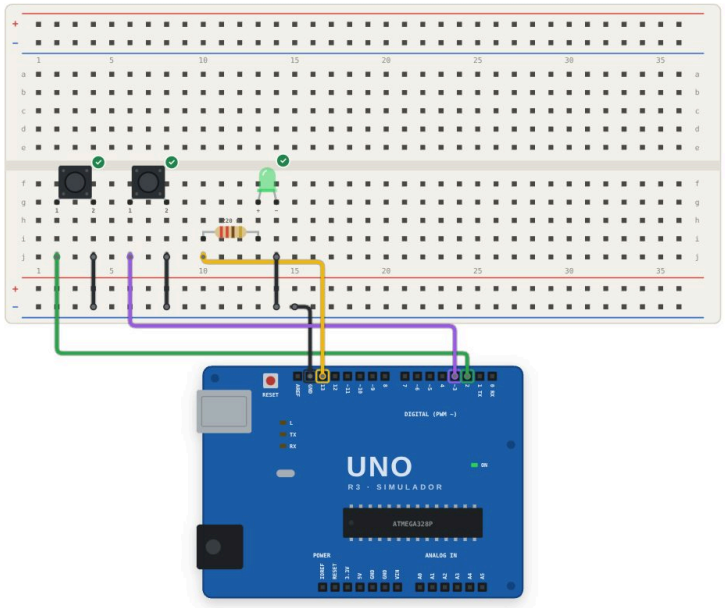
Un botón inicia y otro detiene el cronómetro, con precisión de milésimas de segundo.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Iniciar (Botón)
- 1 × Detener (Botón)
- 1 × Corriendo (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Iniciar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Detener (Botón)**

- Pata 1 → pin **~D3**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 **Corriendo (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 65 mA.
- Consumo estimado: **~65 mA** funcionando, con picos de **~65 mA**.

2 El programa

```
unsigned long inicio = 0;
bool corriendo = false;

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP); pinMode(13, OUTPUT);
}

void loop() {
  if (digitalRead(2) == LOW && !corriendo) { inicio = millis(); corriendo = true; digitalWrite(13, HIGH); }
  if (digitalRead(3) == LOW && corriendo) {
    corriendo = false; digitalWrite(13, LOW);
    float seg = (millis() - inicio) / 1000.0;
    Serial.print("Tiempo: "); Serial.print(seg, 3); Serial.println(" s");
  }
}
```

💡 ¿Cómo funciona?

millis() cuenta los milisegundos desde que el Arduino se encendió. La resta entre el final y el inicio da la duración exacta.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

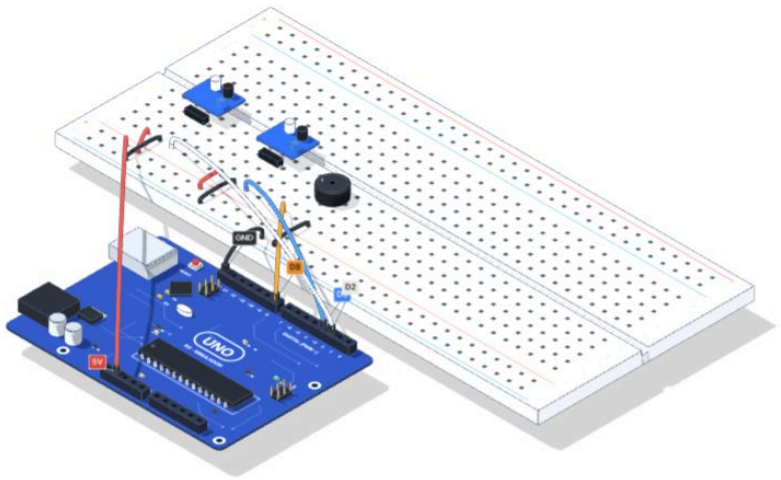
🏆 **Reto:** Usa dos sensores de obstáculos como barreras de inicio y llegada.

PROYECTO 192 · MEDICIÓN Y CIENCIA

Medidor de velocidad con dos sensores

★★★ Avanzado · Tiempo: 45 a 90 minutos

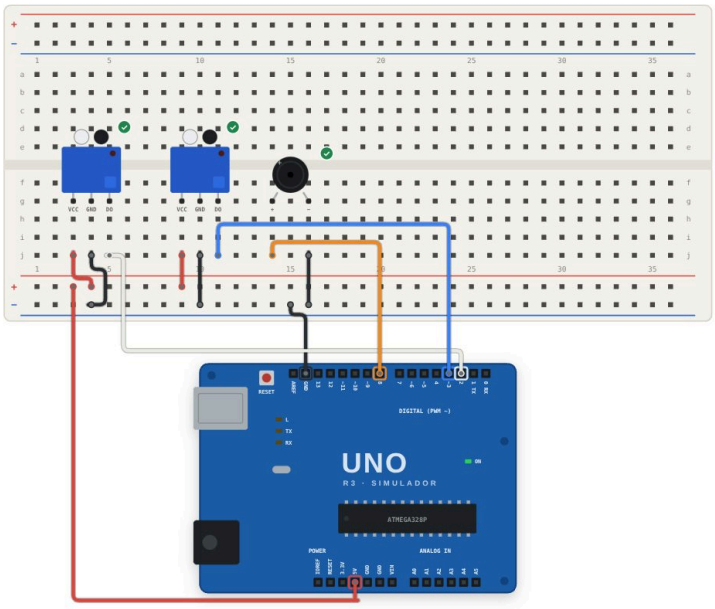
Dos sensores infrarrojos separados 20 cm miden cuánto tarda un carrito en pasar y calculan su velocidad.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de entrada (Sensor de obstáculos IR)
- 1 × Sensor de salida (Sensor de obstáculos IR)
- 1 × Pitido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de entrada (Sensor de obstáculos IR)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Sensor de salida (Sensor de obstáculos IR)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **~D3**

4 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 90 mA.
- Consumo estimado: **~90 mA** funcionando, con picos de **~90 mA**.

2 El programa

```
1  const float DISTANCIA = 0.20;           // metros entre sensores
2
3  void setup() {
4    Serial.begin(9600);
5    pinMode(2, INPUT); pinMode(3, INPUT);
6    Serial.println("Pasa el carrito por los sensores");
7  }
8
9  void loop() {
10   if (digitalRead(2) == LOW) {
11     unsigned long t0 = millis();
12     while (digitalRead(3) == HIGH && millis() - t0 < 5000) {}
13     float seg = (millis() - t0) / 1000.0;
14     if (seg < 5) {
15       float vel = DISTANCIA / seg;
16       Serial.print("Velocidad: "); Serial.print(vel, 2); Serial.println(" m/s");
17       tone(8, 1500, 150);
18     }
19     delay(1000);
20   }
21 }
```

💡 ¿Cómo funciona?

Velocidad = distancia ÷ tiempo. El primer sensor arranca el conteo y el segundo lo detiene; si en 5 segundos no pasa nada, se descarta.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay un objeto delante».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

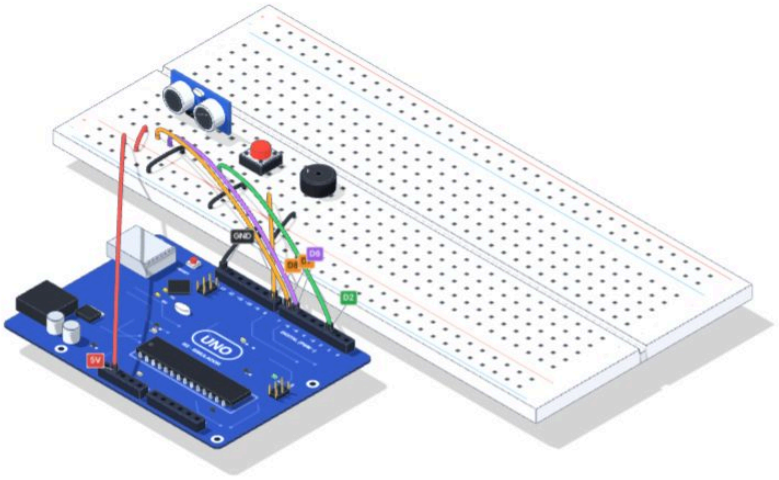
🏆 **Reto:** Convierte el resultado a km/h (multiplica por 3,6).

PROYECTO 193 · MEDICIÓN Y CIENCIA

Medidor de estatura

★★☆ Intermedio · Tiempo: 30 a 45 minutos

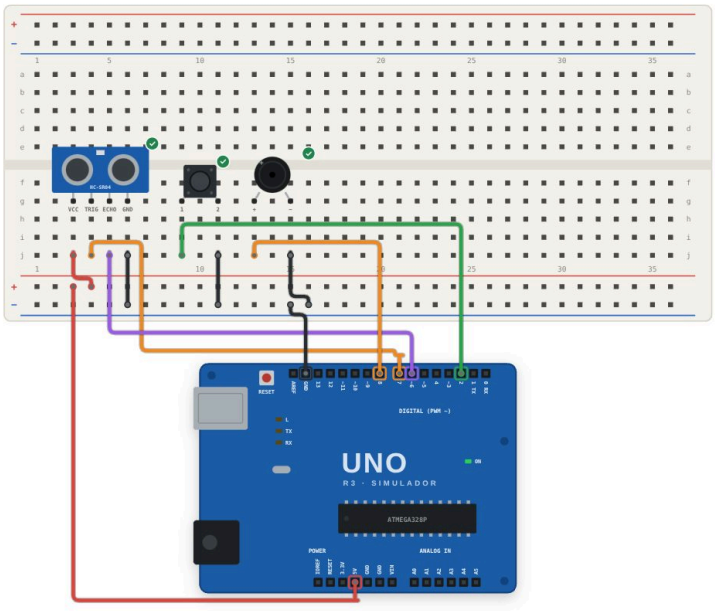
El sensor ultrasónico en el techo mide la distancia a la cabeza y calcula la estatura.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor ultrasónico (Ultrasónico HC-SR04)
- 1 × Medir (Botón)
- 1 × Pitido (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor ultrasónico (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 Medir (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

4 Pitido (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 95 mA.
- Consumo estimado: **~95 mA** funcionando, con picos de **~95 mA**.

2 El programa

```
const int ALTO_SENSOR = 220;            // cm del piso al sensor

long medir() {
  digitalWrite(7, LOW); delayMicroseconds(2);
  digitalWrite(7, HIGH); delayMicroseconds(10); digitalWrite(7, LOW);
  long cm = pulseIn(6, HIGH) / 58;
  return cm;
}

void setup() {
  Serial.begin(9600);
  pinMode(7, OUTPUT); pinMode(6, INPUT); pinMode(2, INPUT_PULLUP);
}

void loop() {
  if (digitalRead(2) == LOW) {
    long estatura = ALTO_SENSOR - medir();
    tone(8, 1500, 150);
    Serial.print("Tu estatura: "); Serial.print(estatura); Serial.println(" cm");
    delay(1000);
  }
}
```

💡 ¿Cómo funciona?

El sensor mide la distancia de él hasta la cabeza. Restando eso de la altura a la que está el sensor se obtiene la estatura.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.
- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

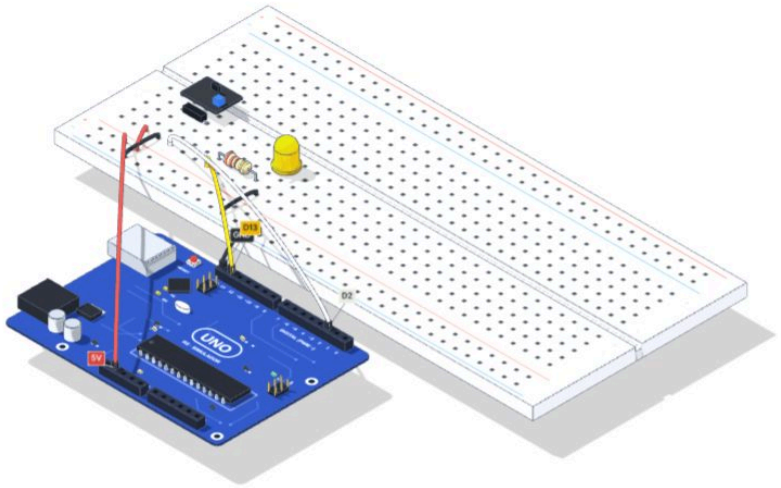
🏆 **Reto:** Calcula el promedio de estatura del salón.

PROYECTO 194 · MEDICIÓN Y CIENCIA

Contador de vueltas con imán

★★☆ Intermedio · Tiempo: 30 a 45 minutos

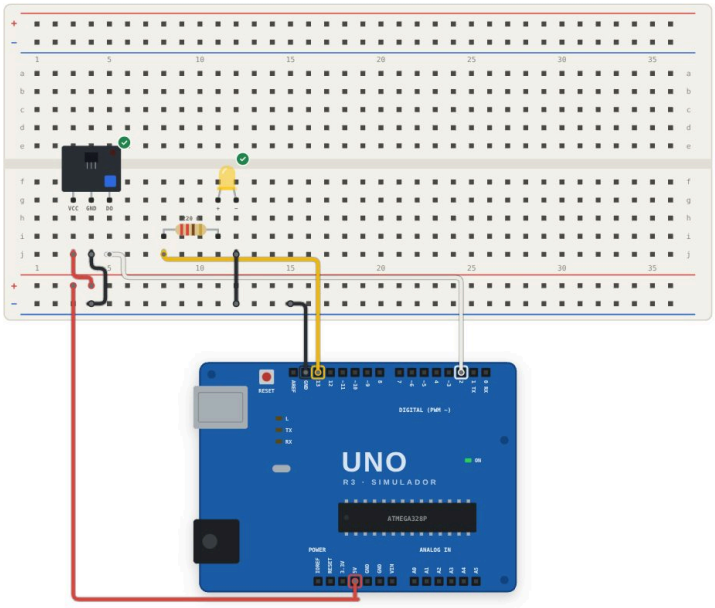
Un imán en la rueda pasa frente al sensor Hall; el Arduino cuenta vueltas y calcula las RPM.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor Hall (Sensor magnético Hall)
- 1 × Pulso (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor Hall (Sensor magnético Hall)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Pulso (LED amarillo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 70 mA.
- Consumo estimado: **~70 mA** funcionando, con picos de **~70 mA**.

2 El programa

```
int vueltas = 0;
bool antes = HIGH;
unsigned long ultimoReporte = 0;
int vueltasAntes = 0;

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT); pinMode(13, OUTPUT);
}

void loop() {
  bool ahora = digitalRead(2);
  if (antes == HIGH && ahora == LOW) { vueltas++; digitalWrite(13, HIGH); }
  else digitalWrite(13, LOW);
  antes = ahora;
  if (millis() - ultimoReporte >= 5000) {
    int rpm = (vueltas - vueltasAntes) * 12;   // 5 s × 12 = 1 minuto
    Serial.print("Vueltas: "); Serial.print(vueltas); Serial.print("   RPM: ");
Serial.println(rpm);
    vueltasAntes = vueltas; ultimoReporte = millis();
  }
}
```

💡 ¿Cómo funciona?

Cada vez que el imán pasa, la salida baja a LOW: eso es una vuelta. Las vueltas de 5 segundos multiplicadas por 12 dan las vueltas por minuto.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca o desmarca la casilla «Hay un imán cerca».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

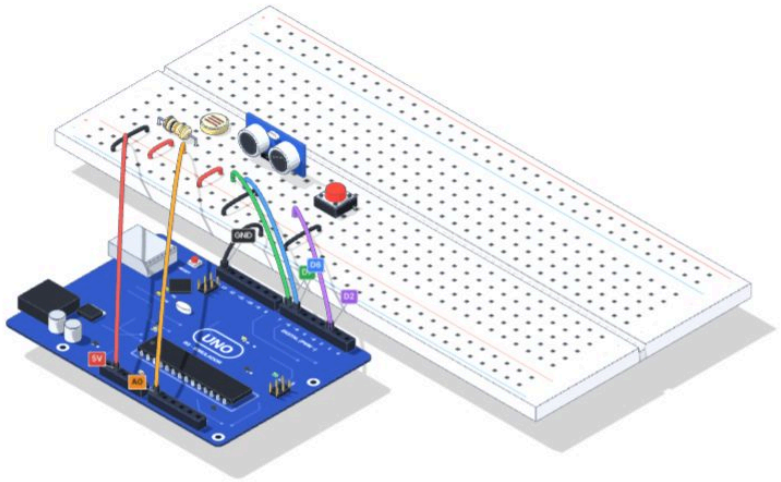
🏆 **Reto:** Con el diámetro de la rueda, calcula los kilómetros recorridos.

PROYECTO 195 · MEDICIÓN Y CIENCIA

Experimento: luz y distancia

★★☆ Intermedio · Tiempo: 30 a 45 minutos

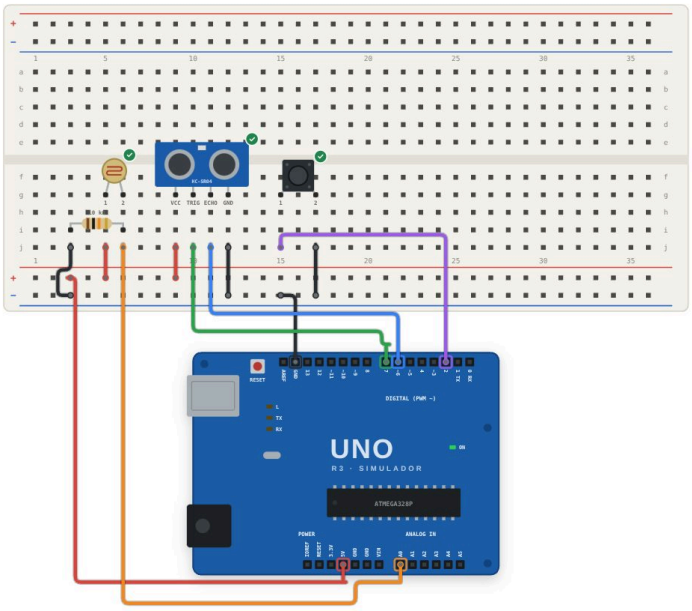
Mide cómo cambia la luz que recibe la fotorresistencia al alejar una linterna, y guarda los datos.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Fotorresistencia
- 1 × Sensor de distancia (Ultrasónico HC-SR04)
- 1 × Tomar dato (Botón)
- 1 × Resistencia 10 kΩ
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Fotorresistencia (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Sensor de distancia (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

4 Tomar dato (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 66 mA.
- Consumo estimado: **~66 mA** funcionando, con picos de **~66 mA**.

2 El programa

```
1  int dato = 0;
2
3  long medir() {
4    digitalWrite(7, LOW); delayMicroseconds(2);
5    digitalWrite(7, HIGH); delayMicroseconds(10); digitalWrite(7, LOW);
6    long cm = pulseIn(6, HIGH) / 58;
7    return cm;
8  }
9
10 void setup() {
11   Serial.begin(9600);
12   pinMode(7, OUTPUT); pinMode(6, INPUT); pinMode(2, INPUT_PULLUP);
13   Serial.println("dato, distancia_cm, luz");
14 }
15
16 void loop() {
17   if (digitalRead(2) == LOW) {
18     dato++;
19     Serial.print(dato); Serial.print(", "); Serial.print(medir()); Serial.print(", ");
   Serial.println(analogRead(A0));
20     delay(500);
21   }
22 }
```

💡 ¿Cómo funciona?

Los datos salen separados por comas: se pueden copiar a una hoja de cálculo y hacer una gráfica. Así se trabaja en un laboratorio de ciencias.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).
- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.
- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

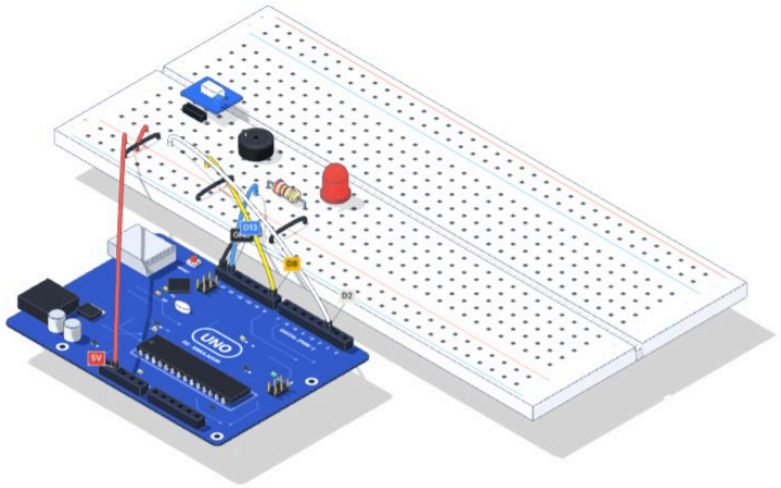
🏆 **Reto:** Grafica tus datos y describe qué pasa con la luz al duplicar la distancia.

PROYECTO 196 · MEDICIÓN Y CIENCIA

Detector de terremotos (sismógrafo)

★★☆ Intermedio · Tiempo: 30 a 45 minutos

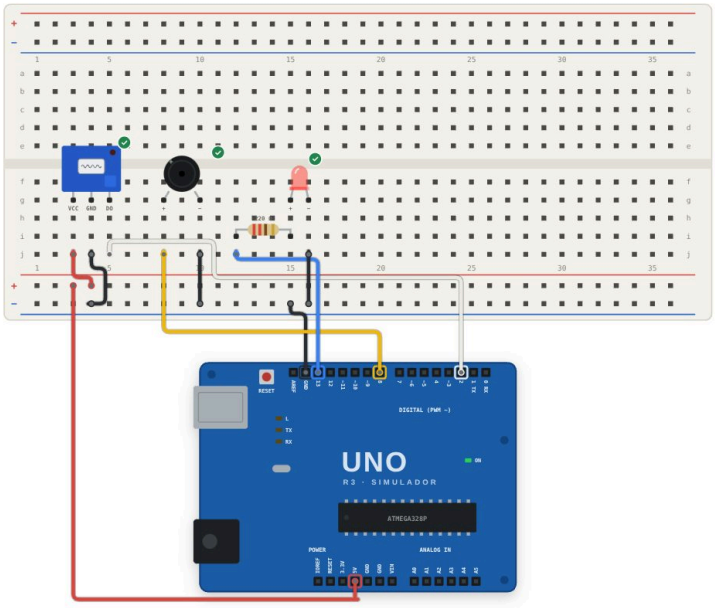
Cuenta las vibraciones por segundo; si son muchas, activa la alerta sísmica.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor de vibración
- 1 × Alerta (Zumbador)
- 1 × Sismo (LED)
- 1 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor de vibración (Sensor de vibración)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Alerta (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

4 Sismo (LED rojo)

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata – (cátodo, la corta, lado plano) → riel **–** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 100 mA.
- Consumo estimado: **~100 mA** funcionando, con picos de **~100 mA**.

2 El programa

```
1  void setup() {
2    Serial.begin(9600);
3    pinMode(2, INPUT); pinMode(13, OUTPUT);
4  }
5
6  void loop() {
7    int golpes = 0;
8    unsigned long inicio = millis();
9    while (millis() - inicio < 1000) {
10     if (digitalRead(2) == HIGH) golpes++;
11     delay(10);
12   }
13   Serial.print("Intensidad: "); Serial.println(golpes);
14   if (golpes > 30) {
15     Serial.println("¡ALERTA SÍSMICA!");
16     for (int i = 0; i < 10; i++) { digitalWrite(13, HIGH); tone(8, 2200, 100); delay(150);
   digitalWrite(13, LOW); delay(100); }
17   }
18 }
```

💡 ¿Cómo funciona?

Un golpe aislado no es un sismo. Por eso se cuentan las vibraciones durante un segundo: solo una vibración sostenida activa la alerta.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay vibración o un golpe».

Con piezas reales: sube el programa y abre el monitor serie a 9600.

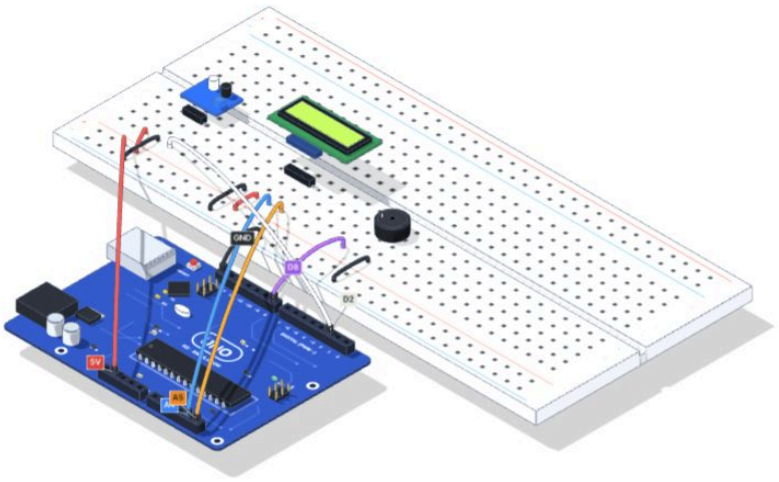
🏆 **Reto:** Muestra la intensidad con 3 LED (débil, medio, fuerte).

PROYECTO 197 · MEDICIÓN Y CIENCIA

Contador de productos en banda

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Como en una fábrica: el sensor infrarrojo cuenta cada producto que pasa y cada 10 suena un aviso de caja llena.

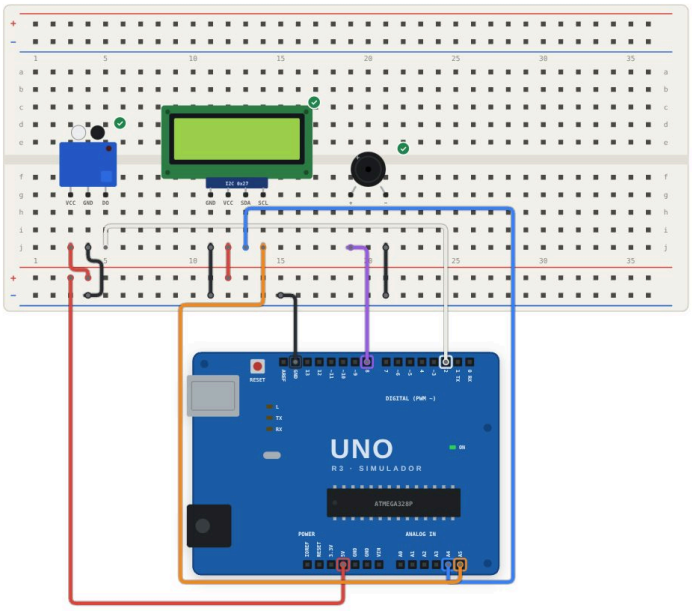


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor IR (Sensor de obstáculos IR)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Caja llena (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor IR (Sensor de obstáculos IR)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Caja llena (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 115 mA.
- Consumo estimado: **~115 mA** funcionando, con picos de **~115 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
int productos = 0;
bool antes = HIGH;

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT);
  lcd.print("Productos: 0");
}

void loop() {
  bool ahora = digitalRead(2);
  if (antes == HIGH && ahora == LOW) {
    productos++;
    int cajas = productos / 10;
    lcd.clear();
    lcd.print("Productos: "); lcd.print(productos);
    lcd.setCursor(0, 1); lcd.print("Cajas: "); lcd.print(cajas);
    if (productos % 10 == 0) tone(8, 1500, 500);
  }
  antes = ahora;
}
```

💡 ¿Cómo funciona?

Se cuenta solo el instante en que aparece el objeto (cambio de HIGH a LOW). La división entera entre 10 da las cajas completas y el residuo % dice si se acaba de llenar una.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay un objeto delante».

Con piezas reales: sube el programa.

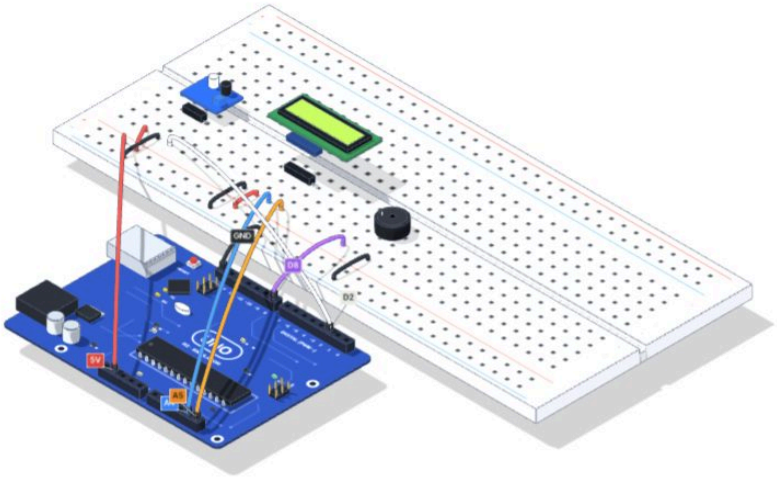
🏆 **Reto:** Agrega un servo que desvíe un producto de cada diez para control de calidad.

PROYECTO 198 · MEDICIÓN Y CIENCIA

Alcancía que cuenta monedas

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Cada moneda que cae interrumpe el sensor infrarrojo; la alcancía cuenta el ahorro en pesos.

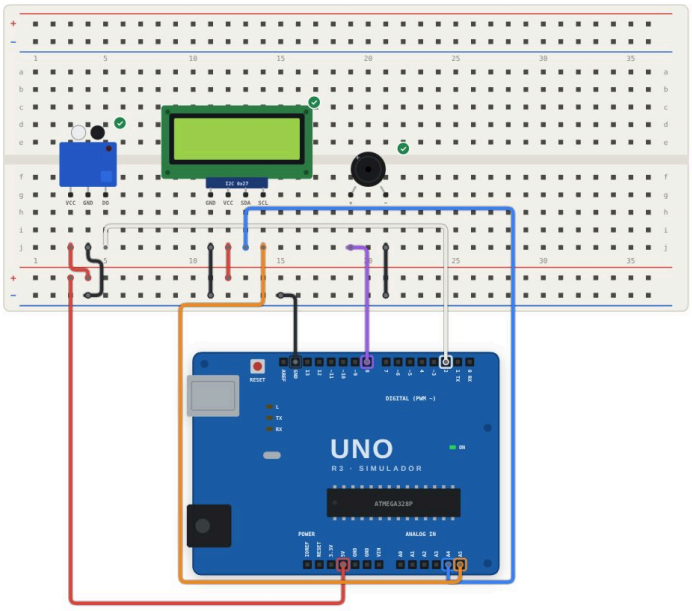


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor IR (Sensor de obstáculos IR)
- 1 × Pantalla LCD (Pantalla LCD 16×2 I2C)
- 1 × Clinc (Zumbador)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: LiquidCrystal I2C (de Frank de Brabander)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor IR (Sensor de obstáculos IR)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Pantalla LCD (Pantalla LCD 16×2 I2C)

- GND → riel **–** (GND)
- VCC → riel **+** (5V)
- SDA → pin **A4**
- SCL → pin **A5**

4 Clinc (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 115 mA.
- Consumo estimado: **~115 mA** funcionando, con picos de **~115 mA**.

2 El programa

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);   // pantalla en A4 (SDA) y A5 (SCL)
long ahorro = 0;
const int VALOR = 500;                    // todas las monedas de 500

void setup() {
  lcd.init(); lcd.backlight();
  pinMode(2, INPUT);
  lcd.print("Ahorro: $0");
}

void loop() {
  if (digitalRead(2) == LOW) {
    ahorro += VALOR;
    tone(8, 2500, 60);
    lcd.clear(); lcd.print("Ahorro: $"); lcd.print(ahorro);
    lcd.setCursor(0, 1); lcd.print(ahorro >= 10000 ? "Meta cumplida!" : "Sigue ahorrando");
    delay(300);
  }
}
```

💡 ¿Cómo funciona?

Se usa long porque el ahorro puede pasar de 32 767, el número más grande que cabe en un int del Arduino UNO.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay un objeto delante».

Con piezas reales: sube el programa.

🏆 **Reto:** Pon una meta con el teclado.

CAPÍTULO 14

Robótica y vehículos

Robots que esquivan obstáculos, siguen líneas o la luz, grúas y radares.

EN ESTE CAPÍTULO · 7 PROYECTOS

199 Carro que esquiva obstáculos ★★★

200 Robot seguidor de luz ★★★

201 Robot seguidor de línea ★★★

202 Grúa con joystick ★★★

203 Semáforo para pista de carritos ★★☆

204 Radar de barrido con aviso de cercanía ★★★

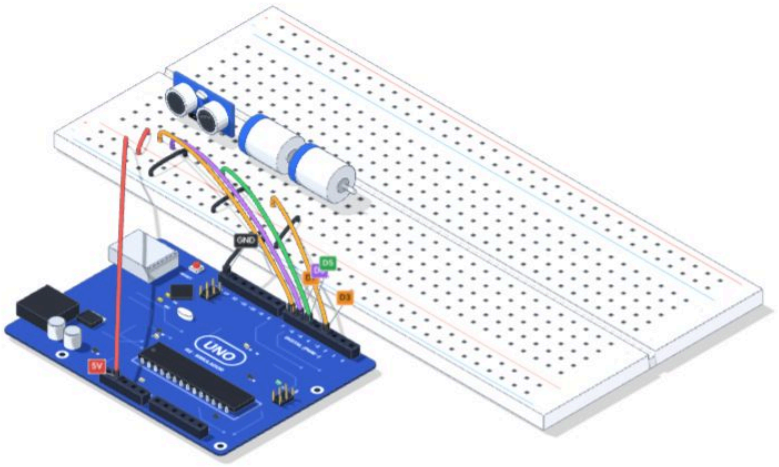
205 Robot mascota que reacciona ★★☆

PROYECTO 199 · ROBÓTICA Y VEHÍCULOS

Carro que esquiva obstáculos

★★★ Avanzado · Tiempo: 45 a 90 minutos

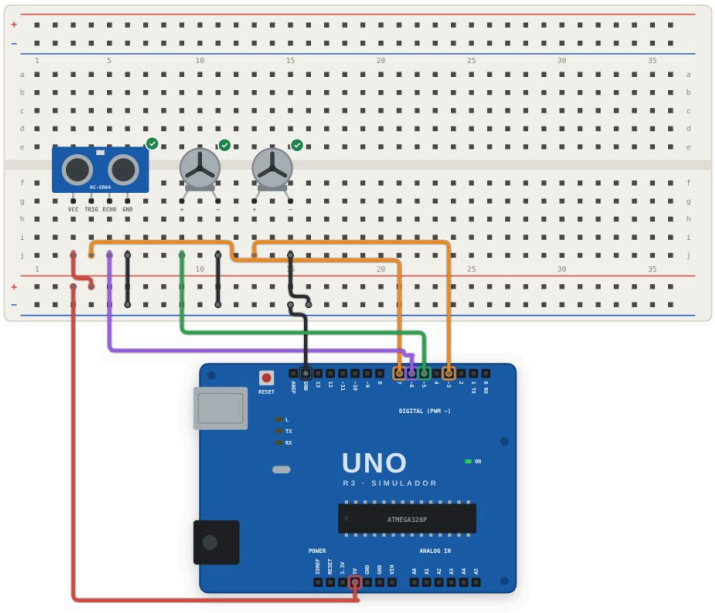
Un robot de dos motores avanza y, si encuentra algo a menos de 20 cm, gira para esquivarlo.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor ultrasónico (Ultrasónico HC-SR04)
- 1 × Motor izquierdo (Motor DC)
- 1 × Motor derecho (Motor DC)
- Driver de motores L298N (o L9110S)
- Fuente 6–12 V para los motores
- Chasis de robot con 2 ruedas
- Portapilas de 4 AA
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor ultrasónico (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

3 **Motor izquierdo (Motor DC)**

- Pata + → pin **~D5**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

4 **Motor derecho (Motor DC)**

- Pata + → pin **~D3**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

5 **Para el proyecto real**

- Un motor real no se conecta directo al pin: se usa un transistor o un módulo L298N.

6 **Energía y controladores**

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motores → necesitan driver:** usa un **driver L298N**, que maneja 2 motores en los dos sentidos. Del Arduino salen los pines de velocidad (PWM) a **ENA/ENB** y los de sentido a **IN1–IN4**; los motores van a **OUT1–OUT4**.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al borne **+12V** del driver. Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~465 mA** funcionando, con picos de **~2065 mA**.

2 El programa

```
long medir() {
  digitalWrite(7, LOW); delayMicroseconds(2);
  digitalWrite(7, HIGH); delayMicroseconds(10); digitalWrite(7, LOW);
  long cm = pulseIn(6, HIGH) / 58;
  return cm;
}

void setup() {
  Serial.begin(9600);
  pinMode(7, OUTPUT); pinMode(6, INPUT);
  pinMode(5, OUTPUT); pinMode(3, OUTPUT);
}

void loop() {
  long d = medir();
  if (d > 20) {
    analogWrite(5, 200); analogWrite(3, 200);       // adelante
  } else {
    Serial.println("Obstáculo: girando");
    analogWrite(5, 200); analogWrite(3, 0);         // gira a la derecha
    delay(600);
  }
  delay(50);
}
```

💡 ¿Cómo funciona?

Para girar, un motor sigue andando y el otro se detiene. Así funcionan los robots con dos ruedas motrices (tracción diferencial).

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Un motor real no se conecta directo al pin: se usa un transistor o un módulo L298N.

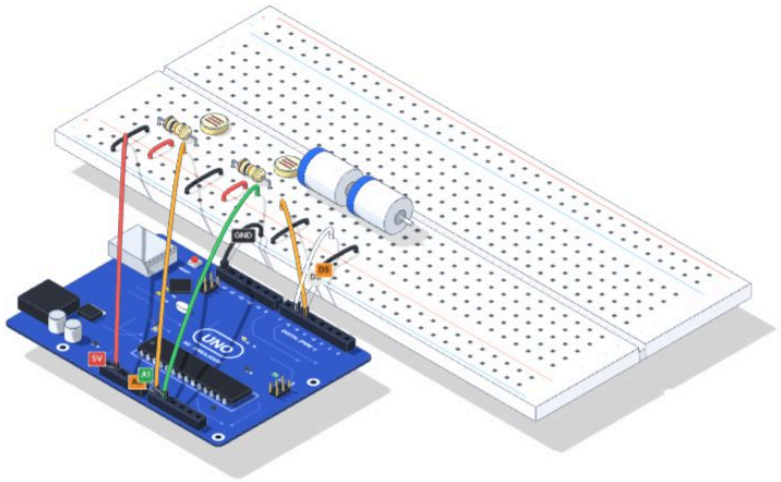
🏆 **Reto:** Haz que el robot retroceda antes de girar (necesitas un puente H como el L298N).

PROYECTO 200 · ROBÓTICA Y VEHÍCULOS

Robot seguidor de luz

★★★ Avanzado · Tiempo: 45 a 90 minutos

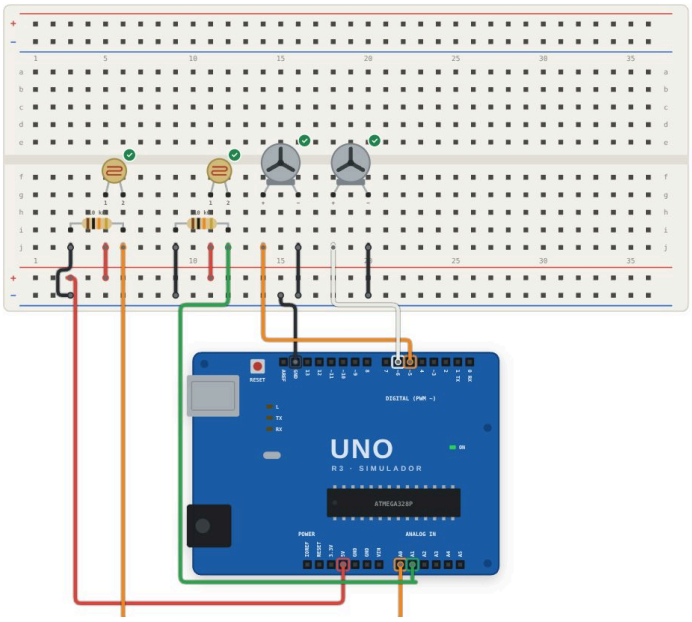
Dos fotorresistencias funcionan como ojos: el robot gira hacia el lado con más luz.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Ojo izquierdo (Fotorresistencia)
- 1 × Ojo derecho (Fotorresistencia)
- 1 × Motor izquierdo (Motor DC)
- 1 × Motor derecho (Motor DC)
- 2 × Resistencia 10 kΩ
- Driver de motores L298N (o L9110S)
- Fuente 6–12 V para los motores
- Chasis de robot con 2 ruedas
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Ojo izquierdo (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A0** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

3 Ojo derecho (Fotorresistencia)

- Pata 1 → riel **+** (5V)
- Pata 2 → pin **A1** y, desde esa misma columna, una resistencia de **10 kΩ** al riel – (GND)

4 Motor izquierdo (Motor DC)

- Pata + → pin **~D5**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

5 Motor derecho (Motor DC)

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

6 Para el proyecto real

- Los motores se manejan con transistores o con un L298N.

7 Energía y controladores

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motores → necesitan driver:** usa un **driver L298N**, que maneja 2 motores en los dos sentidos. Del Arduino salen los pines de velocidad (PWM) a **ENA/ENB** y los de sentido a **IN1–IN4**; los motores van a **OUT1–OUT4**.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al borne **+12V** del driver. Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~452 mA** funcionando, con picos de **~2052 mA**.

2 El programa

```
1  void setup() {
2    Serial.begin(9600);
3    pinMode(5, OUTPUT); pinMode(6, OUTPUT);
4  }
5
6  void loop() {
7    int izq = analogRead(A0), der = analogRead(A1);
8    if (izq > der + 50) { analogWrite(5, 80); analogWrite(6, 200); }        // más luz a la
   izquierda
9    else if (der > izq + 50) { analogWrite(5, 200); analogWrite(6, 80); }   // más luz a la derecha
10   else { analogWrite(5, 180); analogWrite(6, 180); }
11   delay(50);
12 }
```

💡 ¿Cómo funciona?

Para girar a la izquierda, la rueda izquierda va más despacio que la derecha. El margen de 50 evita que el robot tiemble cuando las dos luces son casi iguales.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador «luz» para simular día (alto) o noche (bajo).

Con piezas reales: sube el programa y abre el monitor serie a 9600.

⚠️ Ten en cuenta

- Los motores se manejan con transistores o con un L298N.

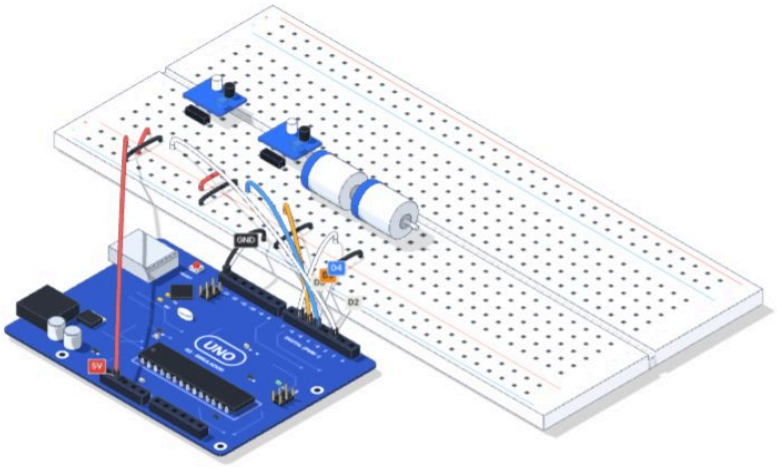
🏆 **Reto:** Haz que se detenga si hay demasiada luz (llegó a la meta).

PROYECTO 201 · ROBÓTICA Y VEHÍCULOS

Robot seguidor de línea

★★★ Avanzado · Tiempo: 45 a 90 minutos

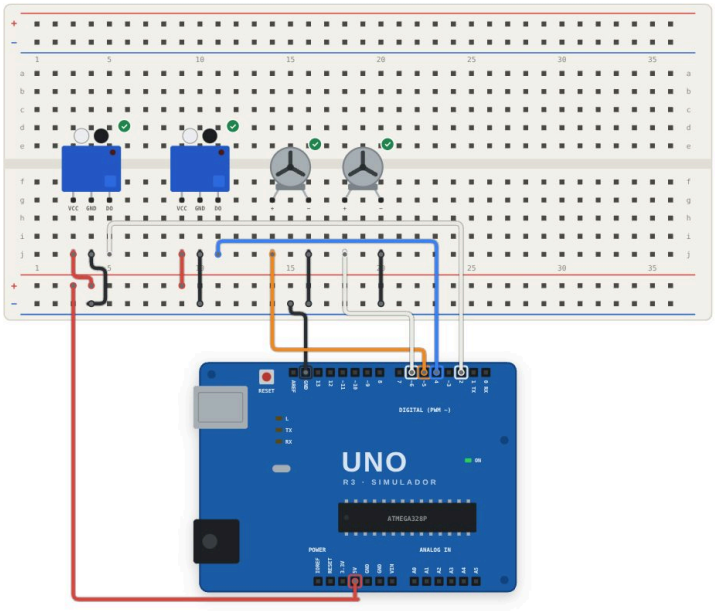
Dos sensores infrarrojos miran el piso y el robot corrige su rumbo para seguir una línea negra.



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Sensor IR izquierdo (Sensor de obstáculos IR)
- 1 × Sensor IR derecho (Sensor de obstáculos IR)
- 1 × Motor izquierdo (Motor DC)
- 1 × Motor derecho (Motor DC)
- Driver de motores L298N (o L9110S)
- Fuente 6–12 V para los motores
- Cinta aislante negra para la pista
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Sensor IR izquierdo (Sensor de obstáculos IR)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Sensor IR derecho (Sensor de obstáculos IR)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D4**

4 Motor izquierdo (Motor DC)

- Pata + → pin **~D5**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

5 Motor derecho (Motor DC)

- Pata + → pin **~D6**
- Pata – → riel **–** (GND)
- En el simulador va directo al pin para practicar; en la vida real ese pin va al transistor o al driver (ver «Energía y controladores»).

6 Para el proyecto real

- Los motores se manejan con transistores o con un L298N.

7 Energía y controladores

- **Arduino:** puede ir por USB (5 V); los motores van con su propia fuente.
- **Motores → necesitan driver:** usa un **driver L298N**, que maneja 2 motores en los dos sentidos. Del Arduino salen los pines de velocidad (PWM) a **ENA/ENB** y los de sentido a **IN1–IN4**; los motores van a **OUT1–OUT4**.
- **Energía de los motores:** fuente o pilas de 6–12 V conectada al borne **+12V** del driver. Nunca alimentes motores desde el pin 5V del Arduino.
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~460 mA** funcionando, con picos de **~2060 mA**.

2 El programa

```
void setup() {
  pinMode(2, INPUT); pinMode(4, INPUT);
  pinMode(5, OUTPUT); pinMode(6, OUTPUT);
}

void loop() {
  bool izqBlanco = digitalRead(2) == LOW;   // el blanco refleja la luz
  bool derBlanco = digitalRead(4) == LOW;
  if (izqBlanco && derBlanco) { analogWrite(5, 180); analogWrite(6, 180); }   // línea al centro
  else if (!izqBlanco) { analogWrite(5, 0); analogWrite(6, 180); }           // la línea está a la izquierda
  else if (!derBlanco) { analogWrite(5, 180); analogWrite(6, 0); }           // la línea está a la derecha
}
```

💡 ¿Cómo funciona?

El blanco refleja la luz infrarroja y el negro la absorbe. Si un sensor ve negro, el robot gira hacia ese lado para volver a centrar la línea.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Hay un objeto delante».

Con piezas reales: sube el programa.

⚠️ Ten en cuenta

- Los motores se manejan con transistores o con un L298N.

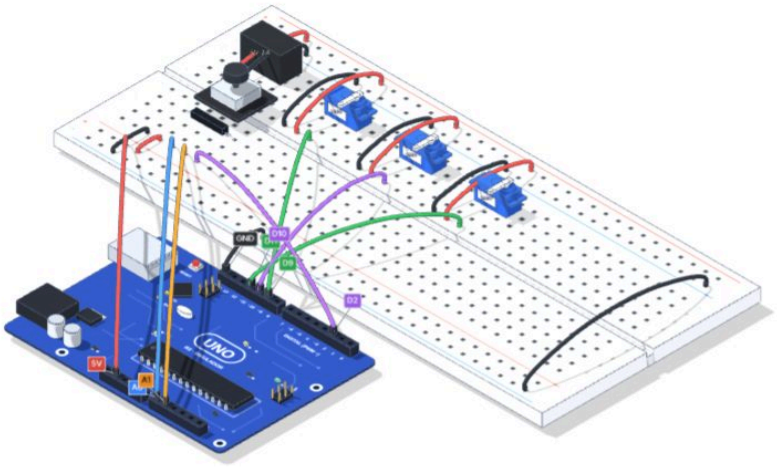
🏆 **Reto:** Agrega un sensor ultrasónico para detenerse si hay algo en el camino.

PROYECTO 202 · ROBÓTICA Y VEHÍCULOS

Grúa con joystick

★★★ Avanzado · Tiempo: 45 a 90 minutos

Una grúa: el joystick gira la base (servo 1) y sube o baja el brazo (servo 2); el botón abre y cierra la pinza.

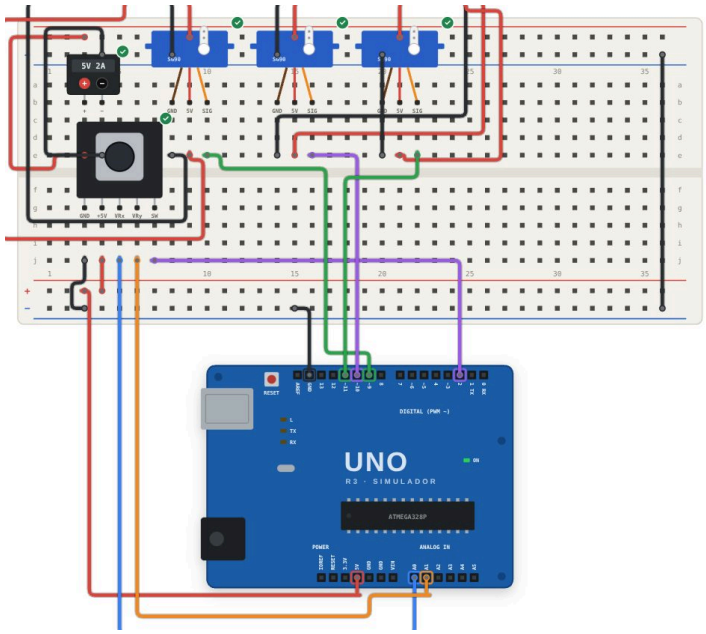


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Joystick
- 1 × Base (Servo SG90)
- 1 × Brazo (Servo SG90)
- 1 × Pinza (Servo SG90)
- 1 × Fuente 5 V para servos (Fuente 5 V externa)
- Controlador de servos PCA9685 (16 canales)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Joystick (Joystick)

- GND → riel **–** (GND)
- +5V → riel **+** (5V)
- VRx (eje X) → pin **A0**
- VRy (eje Y) → pin **A1**
- SW (botón) → pin **D2**

3 Base (Servo SG90)

- Cable café (GND) → riel **– de arriba** (unido al GND del Arduino)
- Cable rojo (5V) → riel **+ de arriba**, que alimenta la fuente externa
- Cable naranja (señal) → pin **~D9**
- En la vida real va a una salida del PCA9685 (ver «Energía y controladores»).

4 Brazo (Servo SG90)

- Cable café (GND) → riel **– de arriba** (unido al GND del Arduino)
- Cable rojo (5V) → riel **+ de arriba**, que alimenta la fuente externa
- Cable naranja (señal) → pin **~D10**
- En la vida real va a una salida del PCA9685 (ver «Energía y controladores»).

5 Pinza (Servo SG90)

- Cable café (GND) → riel **– de arriba** (unido al GND del Arduino)
- Cable rojo (5V) → riel **+ de arriba**, que alimenta la fuente externa
- Cable naranja (señal) → pin **~D11**
- En la vida real va a una salida del PCA9685 (ver «Energía y controladores»).

6 Fuente 5 V para servos (Fuente 5 V externa)

- Borne + (rojo) → riel **+ de arriba**, solo para los servos (no lo unas al 5V del Arduino)
- Borne – (negro) → riel **– de arriba**, y ese riel unido con un cable al riel – de abajo, que va al GND del Arduino (GND común)

7 Para el proyecto real

- Tres servos consumen mucho: usa una fuente externa de 5 V con GND común.

8 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 51 mA.
- **Servos → controlador PCA9685:** con 3 servos usa un **PCA9685**, que maneja hasta 16 servos con solo 2 pines del Arduino: SDA → A4, SCL → A5, VCC → 5V, GND → GND. Los servos se enchufan a sus salidas y se alimentan por el borne **V+** con una fuente de 5 V de al menos 3 A. En el código se usa la librería **Adafruit_PWMServoDriver** en lugar de Servo.h (en este simulador practicas con pines directos).
- **GND común:** une el – de la fuente o batería externa con un GND del Arduino. Sin esa unión las señales no tienen referencia y nada responde bien.
- Consumo estimado: **~501 mA** funcionando, con picos de **~2001 mA**.

2 El programa

```
#include <Servo.h>
Servo base, brazo, pinza;
bool cerrada = false;

void setup() {
  base.attach(9); brazo.attach(10); pinza.attach(11);
  pinMode(2, INPUT_PULLUP);
}

void loop() {
```

```
  base.write(map(analogRead(A0), 0, 1023, 0, 180));
  brazo.write(map(analogRead(A1), 0, 1023, 30, 150));
  if (digitalRead(2) == LOW) {
    cerrada = !cerrada;
    pinza.write(cerrada ? 90 : 10);
    delay(300);
  }
  delay(20);
}
```

💡 ¿Cómo funciona?

Con map() cada eje del joystick se convierte en un ángulo. El botón del joystick (al presionarlo hacia abajo) alterna la pinza entre abierta y cerrada.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Mueve los deslizadores X y Y del joystick y presiona su botón.

Con piezas reales: sube el programa.

⚠️ Ten en cuenta

- Tres servos consumen mucho: usa una fuente externa de 5 V con GND común.

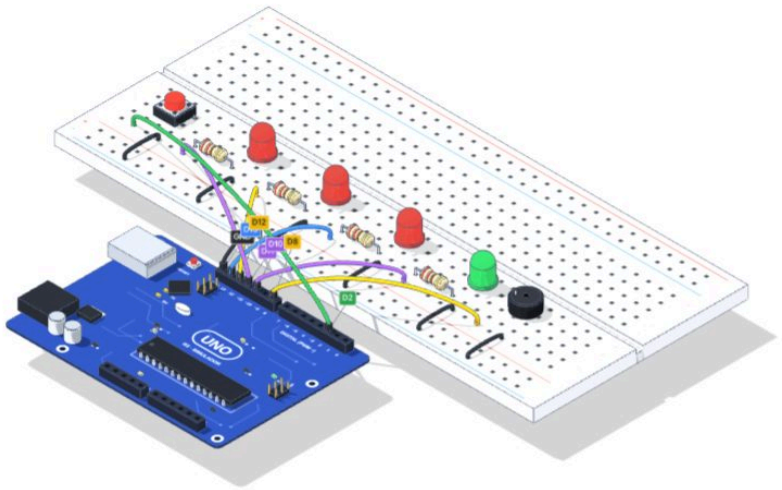
🏆 **Reto:** Graba una secuencia de movimientos y repítela sola.

PROYECTO 203 · ROBÓTICA Y VEHÍCULOS

Semáforo para pista de carritos

★★☆ Intermedio · Tiempo: 30 a 45 minutos

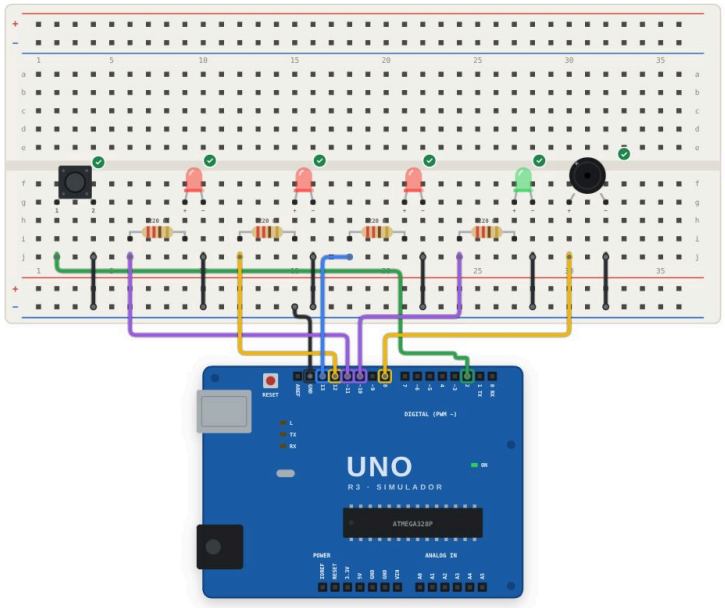
Un semáforo de largada como en la Fórmula 1: 3 luces rojas se encienden y al apagarse... ¡salida!



Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Iniciar (Botón)
- 1 × Rojo 1 (LED)
- 1 × Rojo 2 (LED)
- 1 × Rojo 3 (LED)
- 1 × Salida (LED)
- 1 × Pito (Zumbador)
- 4 × Resistencia 220 Ω
- Cables
- Protoboard
- Arduino Uno
- Cable USB



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Iniciar (Botón)

- Pata 1 → pin **D2**
- Pata 2 → riel **–** (GND)
- En el código usa **INPUT_PULLUP**: sin pulsar lee HIGH y al pulsar LOW.

3 **Rojo 1 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D11**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)
- La resistencia puede ir antes del ánodo o después del cátodo; lo importante es que esté en serie.

4 **Rojo 2 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D12**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

5 **Rojo 3 (LED rojo)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **D13**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

6 **Salida (LED verde)**

- Pata + (ánodo, la más larga) → resistencia de **220 Ω** → pin **~D10**
- Pata − (cátodo, la corta, lado plano) → riel **−** (GND)

7 **Pito (Zumbador)**

- Pata + → pin **D8**
- Pata − → riel **−** (GND)

8 **Energía y controladores**

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 140 mA.
- Consumo estimado: **~140 mA** funcionando, con picos de **~140 mA**.

2 El programa

```
void setup() {
  pinMode(2, INPUT_PULLUP);
  for (int p = 10; p <= 13; p++) pinMode(p, OUTPUT);
  randomSeed(analogRead(A5));
}

void loop() {
  if (digitalRead(2) == HIGH) return;
  for (int p = 11; p <= 13; p++) { digitalWrite(p, HIGH); tone(8, 600, 150); delay(1000); }
  delay(random(500, 3000));            // espera al azar
  for (int p = 11; p <= 13; p++) digitalWrite(p, LOW);
  digitalWrite(10, HIGH); tone(8, 1500, 600);
  delay(3000);
  digitalWrite(10, LOW);
}
```

💡 ¿Cómo funciona?

La espera al azar antes de la salida evita que los corredores adivinen el momento: hay que reaccionar a la luz.

✅ Pruébalo

En el simulador: pulsa ▶ Ejecutar.

- Presiona el botón en su tarjeta del panel «Componentes» (o haz clic sobre él en la protoboard).

Con piezas reales: sube el programa.

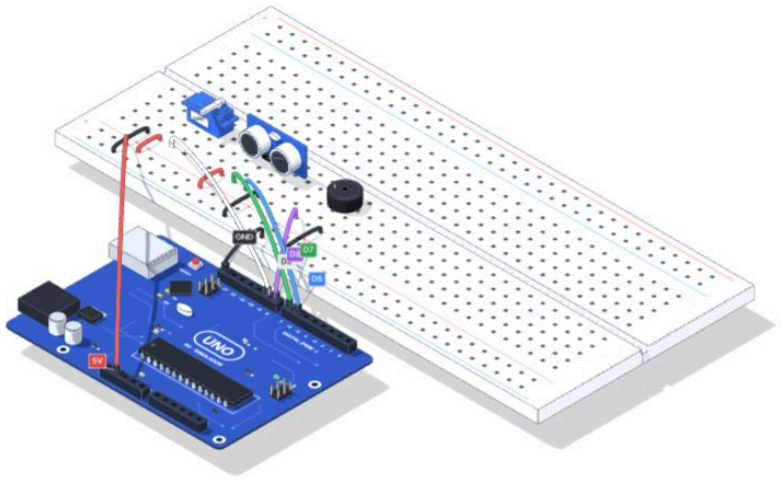
🏆 **Reto:** Agrega dos botones para detectar quién salió antes de tiempo.

PROYECTO 204 · ROBÓTICA Y VEHÍCULOS

Radar de barrido con aviso de cercanía

★★★ Avanzado · Tiempo: 45 a 90 minutos

El servo gira el sensor ultrasónico de lado a lado y el monitor muestra la distancia en cada ángulo, como un radar.

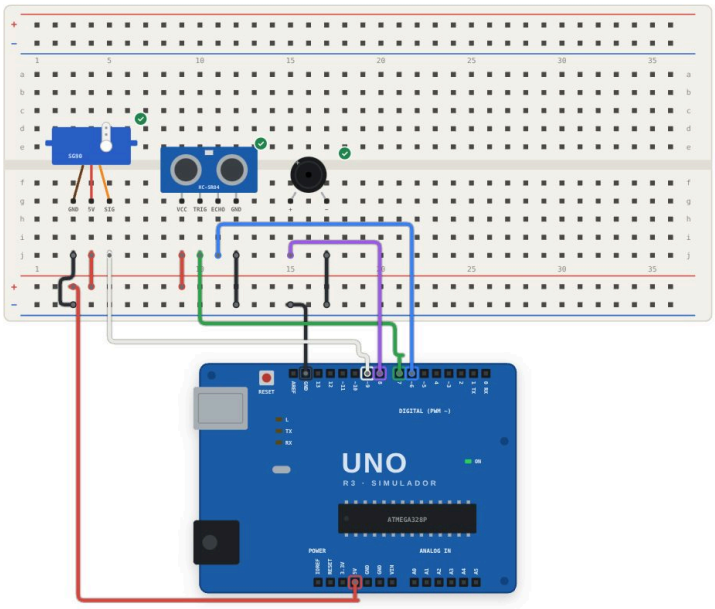


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Base del radar (Servo SG90)
- 1 × Sensor ultrasónico (Ultrasónico HC-SR04)
- 1 × Alerta (Zumbador)
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Base del radar (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

3 Sensor ultrasónico (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

4 Alerta (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

5 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 245 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~245 mA** funcionando, con picos de **~745 mA**.

2 El programa

```
#include <Servo.h>
Servo radar;

long medir() {
  digitalWrite(7, LOW); delayMicroseconds(2);
  digitalWrite(7, HIGH); delayMicroseconds(10); digitalWrite(7, LOW);
  long cm = pulseIn(6, HIGH) / 58;
  return cm;
}

void barrer(int desde, int hasta, int paso) {
  for (int a = desde; a != hasta; a += paso) {
    radar.write(a);
    delay(30);
    long d = medir();
    Serial.print(a); Serial.print(" grados: "); Serial.print(d); Serial.println(" cm");
    if (d < 15) tone(8, 2000, 40);
  }
}

void setup() {
  Serial.begin(9600);
  radar.attach(9);
  pinMode(7, OUTPUT); pinMode(6, INPUT);
}

void loop() {
  barrer(0, 180, 10);
  barrer(180, 0, -10);
}
```

💡 **¿Cómo funciona?**

La función barrer() recibe desde dónde, hasta dónde y de a cuánto girar. Se usa dos veces: de ida y de vuelta.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa y abre el monitor serie a 9600.

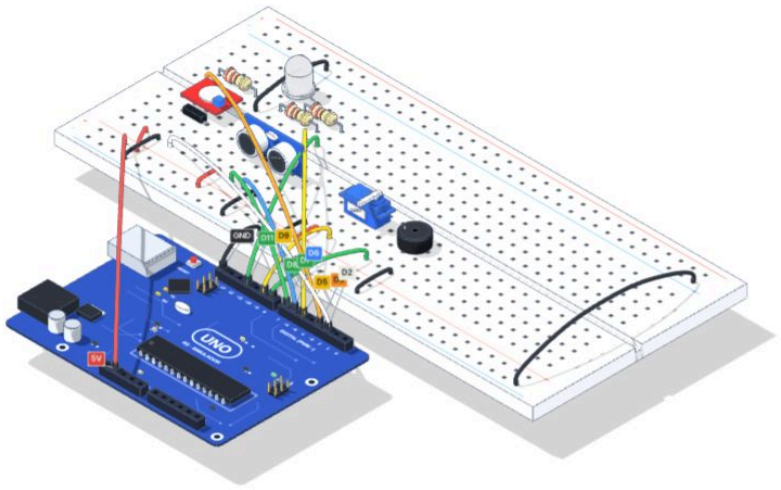
🏆 **Reto:** Muestra en la LCD el ángulo del objeto más cercano.

PROYECTO 205 · ROBÓTICA Y VEHÍCULOS

Robot mascota que reacciona

★★☆ Intermedio · Tiempo: 30 a 45 minutos

Una mascota robot: si la acaricias (sensor táctil) mueve la cola y canta; si alguien se acerca mucho, se asusta.

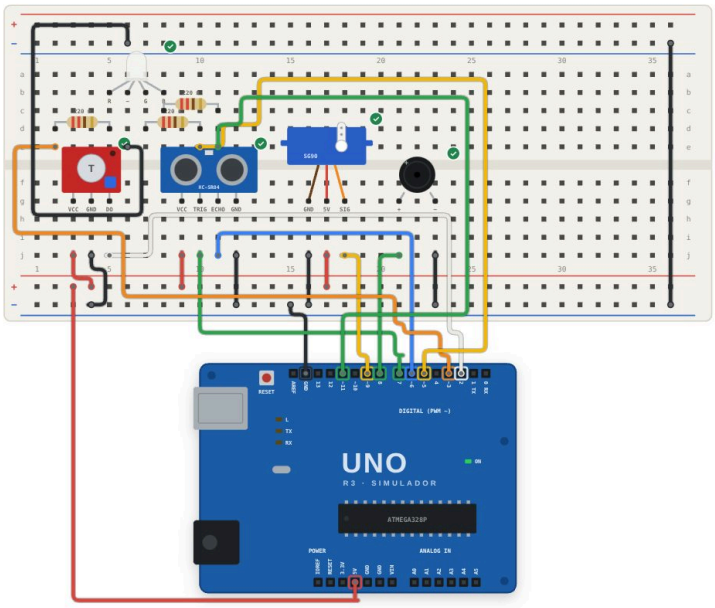


Así se ve armado (vista ilustrada del simulador)

🧰 Materiales

- 1 × Cabeza (táctil) (Sensor táctil TTP223)
- 1 × Ojos (ultrasonido) (Ultrasónico HC-SR04)
- 1 × Cola (Servo SG90)
- 1 × Voz (Zumbador)
- 1 × Corazón (LED RGB)
- 3 × Resistencia 220 Ω
- Condensador 470 µF (recomendado)
- Cables
- Protoboard
- Arduino Uno
- Cable USB

Librerías: Servo (viene instalada con el Arduino IDE)



Plano de conexiones: sigue los colores de los cables.

1 Conexiones paso a paso

1 Alimentación de la protoboard

- Cable del pin **GND** del Arduino al riel **–** (línea azul).
- Cable del pin **5V** del Arduino al riel **+** (línea roja).
- Coloca cada componente con sus patas en columnas distintas: los 5 agujeros de una columna están unidos.

2 Cabeza (táctil) (Sensor táctil TTP223)

- VCC → riel **+** (5V)
- GND → riel **–** (GND)
- DO (señal) → pin **D2**

3 Ojos (ultrasonido) (Ultrasónico HC-SR04)

- VCC → riel **+** (5V)
- TRIG → pin **D7**
- ECHO → pin **~D6**
- GND → riel **–** (GND)

4 Cola (Servo SG90)

- Cable café (GND) → riel **–** (GND)
- Cable rojo (5V) → riel **+** (5V)
- Cable naranja (señal) → pin **~D9**

5 Voz (Zumbador)

- Pata + → pin **D8**
- Pata – → riel **–** (GND)

6 Corazón (LED RGB)

- Pata R (rojo) → resistencia de **220 Ω** → pin **~D3**
- Pata – (común, la más larga) → riel **–** (GND)
- Pata G (verde) → resistencia de **220 Ω** → pin **~D5**
- Pata B (azul) → resistencia de **220 Ω** → pin **~D11**

7 Energía y controladores

- **Arduino:** puede ir por el **USB del computador** (5 V, máximo 500 mA). Lo que cuelga del Arduino consume unos 295 mA.
- **Servo:** un SG90 puede ir al 5V del Arduino, pero al moverse pide picos de hasta 650 mA. Si el Arduino se reinicia o el servo tiembla, pon un **condensador de 470 µF** entre 5V y GND (la pata – a GND) o usa una fuente externa de 5 V.
- Consumo estimado: **~295 mA** funcionando, con picos de **~795 mA**.

2 El programa

```
#include <Servo.h>
Servo cola;

long medir() {
  digitalWrite(7, LOW); delayMicroseconds(2);
  digitalWrite(7, HIGH); delayMicroseconds(10); digitalWrite(7, LOW);
  long cm = pulseIn(6, HIGH) / 58;
  return cm;
}

void color(int r, int g, int b) { analogWrite(3, r); analogWrite(5, g); analogWrite(11, b); }

void setup() {
  cola.attach(9);
  pinMode(2, INPUT); pinMode(7, OUTPUT); pinMode(6, INPUT);
}

void loop() {
  if (digitalRead(2) == HIGH) {                  // feliz
    color(0, 255, 0);
    for (int i = 0; i < 3; i++) { cola.write(60); tone(8, 1200, 80); delay(200); cola.write(120);
tone(8, 1600, 80); delay(200); }
  } else if (medir() < 10) {                     // asustado
    color(255, 0, 0); tone(8, 300, 300); cola.write(90); delay(500);
  } else {                                       // tranquilo
    color(0, 0, 120); cola.write(90);
  }
```

```
  delay(100);
}
```

💡 **¿Cómo funciona?**

El robot tiene tres «estados de ánimo», cada uno con su color, sonido y movimiento. Elegir entre estados con if / else if es la base de muchos robots y videojuegos.

✅ **Pruébalo**

En el simulador: pulsa ▶ Ejecutar.

- Marca la casilla «Lo están tocando».
- Mueve el deslizador de distancia (en centímetros) del sensor ultrasónico.

Con piezas reales: sube el programa.

🏆 **Reto:** Agrega un estado "aburrido" si nadie lo toca en 30 segundos.

ANEXO

Solución de problemas

Cuando algo no funciona casi siempre es un cable, un pin o una librería. Revisa esta lista antes de cambiar piezas.

El IDE dice «Puerto no encontrado» o no deja subir el programa.

Revisa el cable USB (algunos solo cargan). En Herramientas → Puerto elige el COM del Arduino. Si es un clon con chip CH340, instala su controlador.

Error de compilación «No such file or directory» con un .h

Falta instalar la librería. Ve a Herramientas → Administrar bibliotecas, búscala por su nombre e instálala.

El LED no enciende.

Revisa que la pata larga vaya hacia el pin y la corta hacia GND, que la resistencia esté en serie y que el pin del código sea el mismo de la conexión.

El botón funciona al revés o se activa solo.

Con INPUT_PULLUP el botón presionado se lee LOW. Si se activa solo, falta INPUT_PULLUP o una resistencia de 10 kΩ.

El monitor serie muestra símbolos raros.

La velocidad del monitor (abajo a la derecha) debe ser la misma de Serial.begin(), normalmente 9600.

El servo tiembla o el Arduino se reinicia.

El servo pide más corriente de la que da el USB. Usa una fuente externa de 5 V y une su GND con el del Arduino.

La LCD enciende pero no muestra letras.

Gira el potenciómetro azul del módulo I2C. Si sigue igual, prueba la dirección 0x3F en vez de 0x27.

El lector RFID no lee tarjetas.

Debe ir a 3.3 V. Revisa SDA en D10 y RST en D9, y que SCK, MOSI y MISO vayan a D13, D11 y D12.

El lector de huellas dice que no lo encuentra.

Revisa que TX del lector vaya a D2 y RX a D3 (cruzados respecto al Arduino) y que la velocidad sea 57600.

La chapa o la bomba no funciona aunque el relé suena.

Falta el lado de 12 V: + de la fuente al COM del relé, NO del relé al + de la carga y − de la carga al − de la fuente.

El sensor ultrasónico marca 0 o valores enormes.

Revisa TRIG y ECHO (no los cambies de lugar) y que haya algo delante a menos de 4 metros.

El DHT11 devuelve «nan».

El sensor no respondió: revisa el cable de datos y espera al menos 2 segundos entre lecturas.

ANEXO

Glosario

Analógica	Señal que puede tomar muchos valores. El Arduino la lee de 0 a 1023.
Arduino UNO	Placa con un microcontrolador ATmega328P que se programa desde el computador.
Arreglo (array)	Lista de valores del mismo tipo guardados con un solo nombre: int notas[] = {262, 294};
Corriente (I)	Cantidad de electricidad que pasa por un cable. Se mide en amperios (A) o miliamperios (mA).
Cátodo	Pata negativa de un LED o diodo (la más corta, lado plano).
delay()	Detiene el programa la cantidad de milisegundos indicada. 1000 ms = 1 segundo.
Digital	Señal con solo dos valores: HIGH (5 V) o LOW (0 V).
Fuente de alimentación	Aparato que entrega la energía: USB (5 V), batería o adaptador de 12 V.
Función	Bloque de código con nombre que se puede usar muchas veces, como leerHuella().
GND	Tierra o negativo: el punto de 0 V del circuito. Todos los GND deben estar unidos.
HIGH / LOW	Encendido (5 V) y apagado (0 V) en un pin digital.
I2C	Comunicación de dos cables (SDA y SCL) que usan la LCD y la OLED.
IDE	Programa donde se escribe y se sube el código al Arduino (Arduino IDE).
INPUT_PULLUP	Modo de entrada que activa una resistencia interna: el pin lee HIGH hasta que un botón lo une a GND.
Librería	Código ya hecho por otras personas que se agrega con #include para usar un sensor fácilmente.
loop()	Función que el Arduino repite para siempre.
map()	Convierte un número de un rango a otro: map(valor, 0, 1023, 0, 180).
millis()	Milisegundos que han pasado desde que se encendió el Arduino. Sirve para medir tiempo sin detener el programa.
Monitor serie	Ventana del IDE donde se ven los mensajes que envía el Arduino con Serial.print().
Pin	Cada conector de la placa. D0–D13 son digitales y A0–A5 analógicos.
Protoboard	Tablero con agujeros conectados por dentro para armar circuitos sin soldar.
PWM	Encender y apagar muy rápido un pin (~) para simular un voltaje intermedio: brillo o velocidad.
Relé	Interruptor controlado eléctricamente que permite manejar cargas de 12 V o 110 V.
Resistencia (R)	Componente que limita la corriente. Se mide en ohmios (Ω).
RFID	Identificación por radiofrecuencia: tarjetas que se leen sin contacto.
Riel	Filas largas de la protoboard (roja +, azul –) que reparten la energía.
Sensor	Componente que mide algo del mundo: luz, temperatura, distancia, movimiento.

Serial.begin(9600)	Inicia la comunicación con el computador a 9600 baudios.
setup()	Función que se ejecuta una sola vez al encender el Arduino.
UID	Número único de cada tarjeta RFID.
Variable	Caja con nombre donde el programa guarda un dato que puede cambiar.
Voltaje (V)	La «fuerza» que empuja la corriente. Se mide en voltios (V).
Ánodo	Pata positiva de un LED o diodo (la más larga).

ANEXO

Materiales para el kit

Con estas piezas se pueden armar todos los proyectos del libro. La columna «cantidad» es lo máximo que usa un solo proyecto.

Componente	Cantidad	Proyectos que lo usan
LED	6	113
Zumbador (buzzer)	1	81
Pulsador	4	51
Pantalla LCD 16×2 con I2C	1	36
Servomotor SG90	3	28
Fotorresistencia (LDR)	2	16
Sensor ultrasónico HC-SR04	1	16
Potenciómetro	3	15
Motor DC	2	12
Sensor de temperatura LM35	1	12
Lector de huellas R307 / AS608	1	10
Sensor DHT11	1	9
Sensor de movimiento PIR	2	8
Lector RFID RC522	1	8
Módulo relé 5 V	1	7
Teclado matricial 4×4	1	7
LED RGB	1	6
Interruptor	1	6
Sensor de obstáculos IR	2	6
Fuente de 12 V	1	5
Joystick	1	4
Sensor de sonido	1	3
Sensor de vibración SW-420	1	3
Chapa eléctrica 12 V	1	3
Sensor de gas MQ-2	1	3
Sensor táctil TTP223	3	3
Sensor de nivel de agua	1	3
Sensor de lluvia	1	2
Sensor de humedad de suelo	1	2
Pantalla OLED 0,96"	1	2

Sensor magnético Hall	1	2
Sensor de llama	1	2
Bomba de agua 12 V	1	2
Sensor de corriente ACS712	1	2
Sensor de inclinación	1	2
Arduino UNO con cable USB	1	205
Protoboard de 830 puntos	1	205
Resistencias de 220 Ω y 10 kΩ	10 c/u	—
Cables macho-macho y macho-hembra	40 c/u	—

ANEXO

Mis notas

Aprende electrónica construyendo

205 proyectos para armar con Arduino: luces, sonidos, sensores, pantallas, motores, lectores de huellas, chapas eléctricas, tarjetas RFID, domótica, juegos y robótica.

Cada proyecto incluye la lista de materiales, la vista ilustrada del circuito, el plano de conexiones en la protoboard, las conexiones explicadas cable por cable, el programa completo con comentarios, cómo funciona, cómo probarlo y un reto para seguir aprendiendo.

Todos los proyectos están dentro del simulador **Banco Arduino**, para practicarlos en el computador, la tableta o el celular antes de armarlos con piezas reales.

- Luces y LED
- Botones e interruptores
- Sonido y música
- Perillas y señales analógicas
- Luz y oscuridad
- Temperatura y clima
- Distancia y movimiento
- Servos y motores
- Pantallas
- Seguridad y control de acceso
- Hogar y ambiente
- Juegos
- Medición y ciencia
- Robótica y vehículos

ING. JAIR MURILLO

Libro 2026 – 2027 · https://andresyairrenteriamurillo-maker.github.io/banco-arduino/